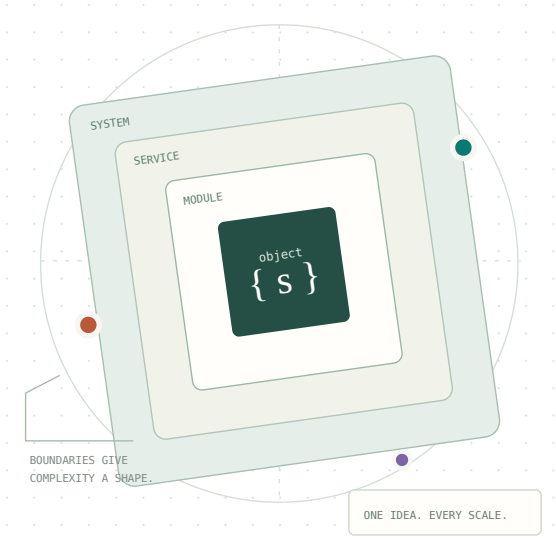
THE SYSTEM DESIGN ATLAS

From objects to *planet-scale systems.*

A system is more than its boxes and arrows. Learn the promises, mechanisms, and trade-offs that connect a single function to a distributed world.

Build the simple version. Find its limit. Understand the next design.



A CONCEPTUAL COVER STUDY · AUTHOR-CREATED

22	99	575	12
UNITS MAPPED	CHAPTERS TO READ	ORIGINAL FIGURES	COMPLETE CASE STUDIES

- **The complete first edition.** All 22 units, 99 chapters, and twelve complete case studies are ready, with worked mechanisms, original figures, exercises, and separate explained solutions. Inspect the coverage → · Download the complete PDF ↓

THE TERRITORY

Six parts. One connected story.

See the concept map ↗

PART 01

System design foundations

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 02

Low-level design & software architecture

19 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 03

Data, storage & scalability

20 chapters · 5 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 04

Distributed systems & reliability

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 05

Security & production engineering

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 06

Complete system design case studies

12 chapters · 1 units · twelve complete design reviews

Ready to read ↗

A PLACE TO BEGIN / UNIT 01

Before you scale a system, learn to *see one*.

Build a to-do application step by step. Save a task, organise its code, choose a design, and follow a request through the system.

Open the foundations unit ↗

1.1 From a program to a system ↗

1.2 Organising code into modules ↗

1.3 From requirements to design choices ↗

1.4 Drawing the system at different levels ↗

01 / THE PROBLEM

Begin where it breaks.

A concept earns its place by solving an observable problem.

02 / THE MECHANISM

Make execution visible.

Inspect the state, ownership, messages, and ordering.

03 / THE JUDGMENT

Keep the trade-off.

Understand what improves, what it costs, and when to choose differently.

PART 1 / UNIT 01

Introduction to System Design

How do we turn a small program into an application people can rely on? We will build a to-do application, one decision at a time.

Begin with a task kept in memory. Save it between runs, organise the code into modules, and work out what the application must do. Then draw the design and trace a request through it.

The running example

Design a basic to-do application. Begin with a single program and progressively identify modules, persistent storage, a web API, and deployment boundaries.

Unit 1 assessment → Explained solutions →

Concept coverage

From a program to a system

Prerequisites: Basic programming 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

Imagine that you have written a small to-do program. You enter “Read the design notes,” and the program adds it to your list. You close the program. When you open it the next morning, the list is empty.

The program could add a task, but it could not keep one. To make it useful, we must decide where tasks are stored and what should happen when the program stops. These are our first system design questions.

In this chapter: We will identify the parts of this application, follow a change in its data, and decide when it is safe to display “Saved.”

1.1.1 · What makes a program part of a system?

A **software system** consists of parts that work together to perform a task. In our example, the program accepts a title, checks it, stores a task, and displays the list. Once we add permanent storage, the program and the stored data must work together.

It helps to state what we are designing. Our first application is for one person using one machine. The person can add tasks, list them, and mark them complete. A task reported as saved must still be present after the program restarts. For now, we assume that the machine’s disk remains intact.

We can draw a line around the application and its storage. This is the **system boundary**: it tells us which parts we are considering together. The person uses the system from outside this line.

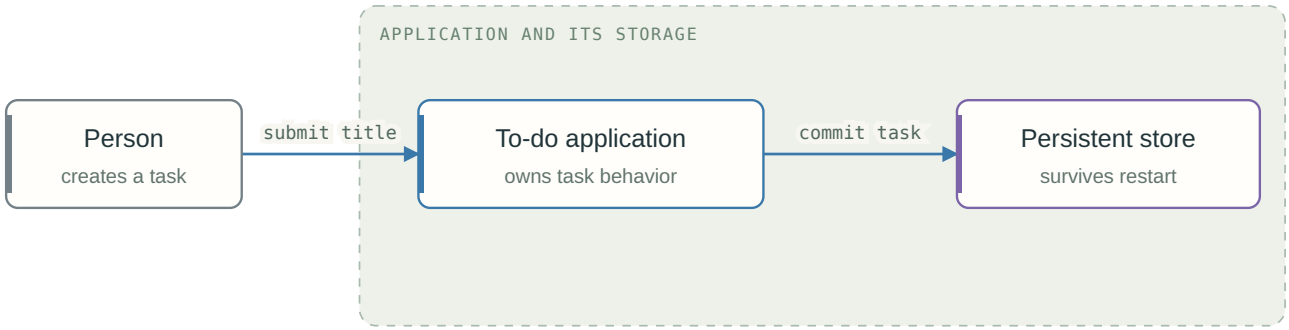


FIG. 1.1.1 **The application and storage work together**

The application needs storage to keep a task after restart. We include both inside the boundary when discussing that requirement.

Why this view: Include the parts needed to explain the behaviour you are studying. Author-created illustration.

The boundary depends on the question. To investigate a slow function, we may study only the running program. To investigate a missing task, we also need to examine storage. When drawing a boundary, state which question it helps answer.

System design is the work of deciding how these parts should be organised and how they should interact to meet the requirements. For our application, this includes choosing where to store tasks, which code checks their titles, and when to tell the user that a task has been saved.

> START WITH THE USER'S EXPECTATION
>
> If the screen says "Saved," what should the user be able to rely on?

1.1.2 · Follow one task through the program

Let us first keep tasks in memory. Memory holds the data used by a running program. In this example, the program creates a new, empty dictionary each time it starts.

The following Python code assumes that calls run one at a time, that the title is text, and that the caller supplies a positive integer identifier. It checks the title length and prevents two tasks from using the same identifier.

```
tasks = {}

def create(task_id, title):
    title = title.strip()
    if not 1 <= len(title) <= 200:
        raise ValueError("title must contain 1–200 characters")
    if task_id in tasks:
        raise ValueError("task id already exists")

    tasks[task_id] = {"title": title, "done": False}
    return {"id": task_id, **tasks[task_id]}

print(create(7, "  Read  "))
# {'id': 7, 'title': 'Read', 'done': False}
```

The call `create(7, " Read ")` first removes spaces from the ends of the title. It checks that the resulting title is between 1 and 200 characters long and that identifier 7 is unused. It then adds one task and returns its details.

Before the call, `tasks` is empty. After the call, it contains task 7. This is a **state change**. The state of a system is the information it currently holds that affects what it will do next. If we now ask for the task list, task 7 will appear because the state has changed.

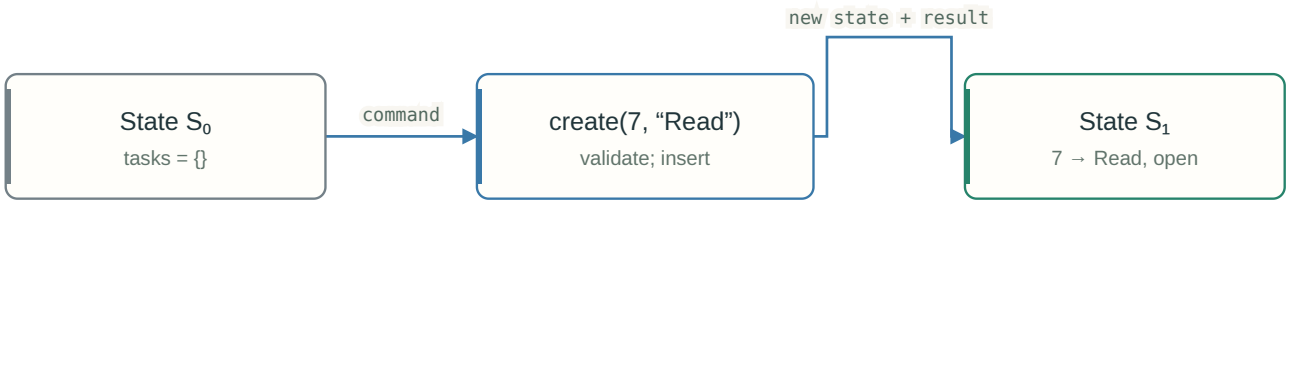


FIG. 1.1.2 **Creating a task changes the stored state**

Before creation, the collection is empty. After create(7, “Read”), it contains task 7, marked incomplete. The function returns the task details.

Why this view: Read the state before and after the operation to see exactly what changed. Author-created illustration.

We can use this one operation to distinguish four related ideas. The title “Read,” the number 7, and the value `False` are **data**. The collection of tasks currently stored is **state**. Trimming and checking the title are **computation**. Passing the title into the function and returning the result are **communication**. Later, communication may also involve files or a network.

Some rules must hold whenever the task collection is valid: each task must have a unique identifier, and each title must contain 1–200 characters after trimming. A rule of this kind is called an **invariant**. If validation fails, the function raises an error before changing the collection.

To complete task 7, we would check that it exists and set `tasks[7]["done"] = True`. Setting that flag to `True` again leaves the task completed. Creating another task is different: it adds another entry. We will return to this distinction when we discuss repeated requests.

<details>
<summary>▼ A closer look · Describing a state change with notation</summary>

We can write an operation as `T(S, c) → (S′, result)`. Here, `S` is the state before the operation, `c` is the command, and `S′` is the state afterward.

For our example, `S` is an empty task collection, `c` is “create task 7 with title Read,” and `S′` contains that task. If the title is empty, the result is an error and `S′ = S`. This notation describes the intended change. By itself, it says nothing about crashes or two callers running at the same time.
</details>

1.1.3 · Decide what “saved” means

The dictionary works while the program is running. When the program restarts, it creates a new dictionary and loses the old list. To meet our restart requirement, we need **persistent storage**: storage that keeps data after the program stops.

For this application, we will use SQLite, a database that can store its data in a local file. The application groups a write into a **transaction**. Completing the transaction successfully is called a **commit**. We will report success only after the database has committed the task.

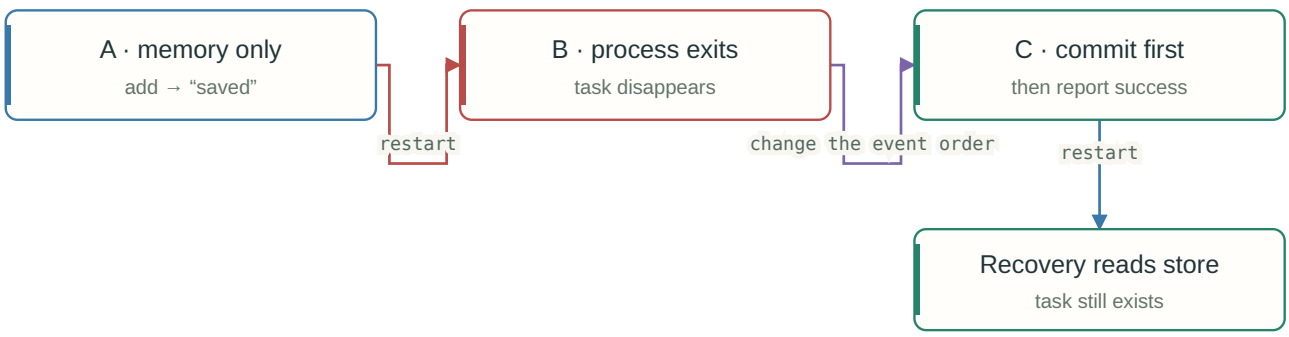


FIG. 1.1.3 **Save before reporting success**

Memory-only tasks disappear after restart. In the revised version, the application commits the task before reporting success. The task remains after a process restart if committed storage is intact.

Why this view: Compare the two versions by restarting the process. Author-created illustration.

The order matters. Suppose the application displays “Saved” and then tries to write the task. If it crashes between these events, the user has been told that a task was saved even though it is missing after restart. Waiting for commit avoids this particular failure.

This is a promise about restarting the program while its committed storage remains intact. If the only disk is destroyed, a local database cannot recover the data from that disk. The SQLite atomic-commit documentation explains how its commit mechanism works and which storage assumptions it relies on.

TRY IT · INTERACTIVE MODEL

What survives a restart?

Choose when the application reports success. Use “Next event” to follow the request, then restart the process and check whether the task remains.

Choice: C · commit before reporting success · 0 events completed

The caller has not received success.

APPLICATION MEMORY	SAVED IN STORAGE	WHAT THE CALLER SEES
Empty	Empty	No success received

Choose “Next event” to continue, or restart to see what remains.

This model follows one request. A commit either finishes completely or does not happen. Restart removes memory but preserves committed storage. No real files are written.

Try the policy that reports success before commit. Advance until the user receives “Saved,” then restart the process. Now repeat with commit before success. The different results come from the order of two events.

The model removes memory when the process restarts and keeps committed storage. It does not simulate a damaged disk or a power failure. The runnable Python example lets you create a task, close the real database connection, and read the task through a new connection.

1.1.4 · Give each part a clear contract

A **component** is a part of the system with a defined responsibility. We will soon separate the code that accepts input, the code that applies task rules, and the code that stores tasks. These components can all run inside the same program.

Components need a way to use one another. An **interface** describes the operations they offer, such as `create(title)` or `list_tasks()`. A **contract** explains how those operations behave.

For example, the name `create(title)` does not tell us whether blank titles are accepted or whether the result has already been saved. We must state those rules. The following contract describes the persistent version of our application, which will assign identifiers itself.

Table 1.1.1 · The contract for creating a task

Situation	Required behaviour	How to check it

Situation	Required behaviour	How to check it
The title has leading or trailing spaces.	Remove those spaces before checking and saving it.	Submit “ Read ” and check that the title is “Read.”
The trimmed title is empty or longer than 200 characters.	Reject it without adding a task.	Compare the task list before and after the attempt.
Creation succeeds.	Return the new identifier after commit.	Restart the program and find the task.
The database write fails.	Report the error instead of reporting success.	Make storage fail and inspect the result.
The task is committed, but its reply is lost.	The task remains saved; the caller may be unsure what happened.	Interrupt the reply after commit.

The last row introduces a problem we will examine in Chapter 1.4. Saving a task and delivering its reply are separate events. The application must account for both.

1.1.5 · Look at the same design at two levels

We can now ask two sets of questions. Where does the application store tasks, and how does it communicate with that storage? Within the application, which functions validate a title and perform the write?

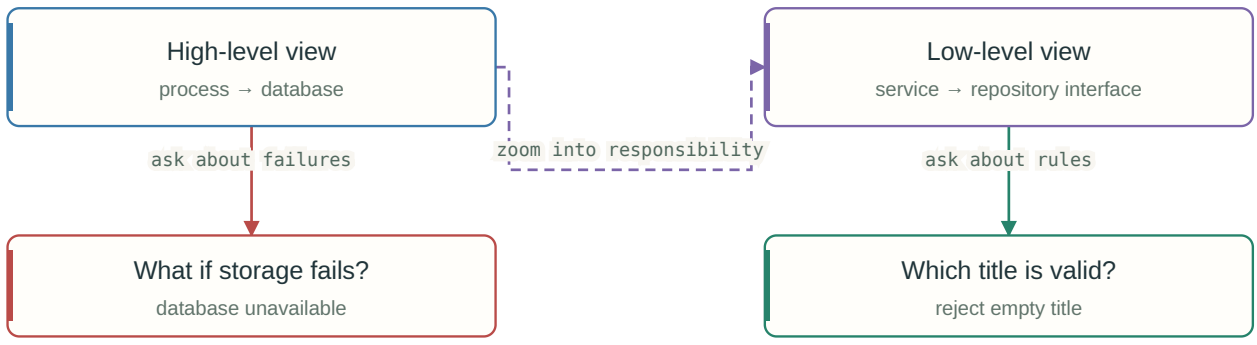


FIG. 1.1.4 **Two levels of detail in one design**

The high-level view shows the application and database. The low-level view shows the service, repository interface, and rules inside the application.

Why this view: Check that both views agree about saving a task before reporting success. Author-created illustration.

High-Level Design (HLD) usually describes the main parts of a system and how they work together. For our application, it shows the user, application, and persistent store. It also states that the application waits for commit before reporting success.

Low-Level Design (LLD) describes how a part is implemented: its functions, classes, interfaces, data structures, and algorithms. Here, it explains how the task service checks a title and calls the storage code.

Software architecture concerns the major choices that shape the system and the reasons for them. Choosing to keep tasks in a local database is one such choice. Different teams use HLD, LLD, and architecture somewhat differently; the useful distinction is the level of detail being discussed.

Both levels must agree. The high-level design may say “save before replying,” but the code must actually wait for the save to finish.

> CHECK YOUR UNDERSTANDING
>
> Putting a dictionary in a class does not make it persistent. Adding a database does not make a reply arrive. A successful restart test does not show what happens when the disk is destroyed. Each result depends on a different part of the design.

Remember: Start with what the application must do. Identify the parts and the data they hold. Then trace the events that must happen before the user receives success.

Before continuing: The application reports “Saved,” then crashes before writing to storage. Which event must move earlier to prevent the task from being lost?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. Atomic Commit in SQLite ↗

   Official documentation · accessed 21 Oct 2026. Commit and operating-system assumptions.

   `Access reviewed · 21 October 2026`

2. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

Organising code into modules

Prerequisites: 1.1 · From a program to a system 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

Our to-do application can now save tasks between runs. We add a web form and a command-line interface. Both contain code that checks the title. Later, we change the title limit from 80 characters to 200, but update only the web form.

The same application now accepts a title through one entry point and rejects it through another. To fix the cause, we need to decide where the rule belongs.

In this chapter: We will divide the application into modules, examine the dependencies between them, and follow a change through the revised design.

1.2.1 · Why does one change require several edits?

A first version of the web handler may do everything in one place: read the request, check the title, write SQL, and build the response. For a small program, this is easy to follow. The difficulty appears when another feature needs some of the same behaviour.

Suppose the command-line interface copies the title check. A CSV exporter also reads the database table directly. We now have two copies of a rule and several places that know how tasks are stored. Changing either the rule or the table can affect several features.



FIG. 1.2.1 **Shared storage details make changes spread**

Several parts read or change the same task data directly. A change to a field or rule may therefore require edits in several places.

Why this view: Follow the arrows to find which parts know about the shared data. Author-created illustration.

Coupling describes the dependencies between parts of a system. In this example, the exporter is coupled to the database layout: it must know the column names. Rename a column, and its query may stop working.

A dependency is often necessary. The task service needs some way to store a task. The design question is how much it needs to know. Calling `repository.insert(title)` requires less knowledge of storage than constructing SQL and opening database files throughout the application.

Cohesion describes how closely the work inside a module belongs together. Checking task titles and completing tasks serve a shared purpose. Password handling, CSV formatting, and database connection setup have different purposes; placing them in a single “helpers” module gives that module little focus.

Coupling asks how one module depends on another. Cohesion asks whether the work within one module belongs together. These ideas help us decide where to put code.

1.2.2 · Separate input, task rules, and storage

Let us give the application three main responsibilities.

- The **input adapter** translates an incoming request into a call the application understands. A web adapter reads an HTTP request; a command-line adapter reads command-line input.
- The **task service** applies task rules. It checks the title and asks storage to create a task.
- The **repository** provides operations for storing and retrieving tasks. Our SQLite implementation contains the database queries and commits each change before returning.

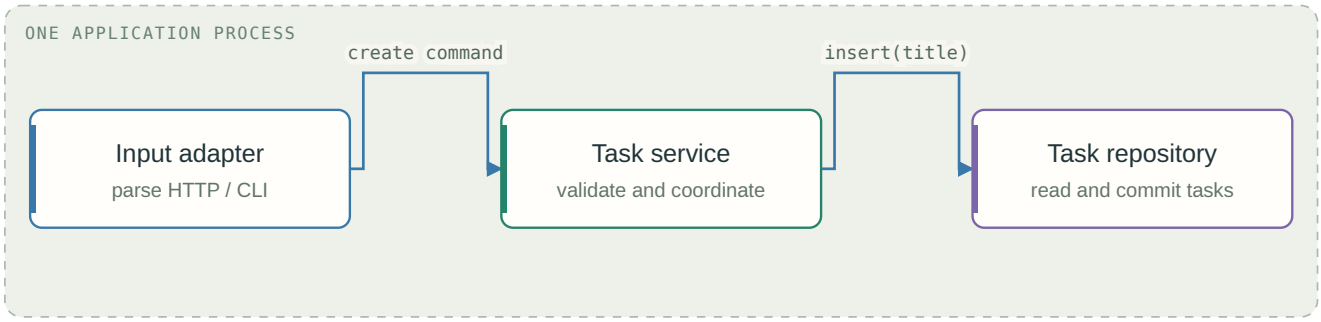


FIG. 1.2.2 **Separate responsibilities inside one process**

The adapter reads input, the service checks task rules, and the repository saves and reads tasks. These parts all run inside one application process.

Why this view: Each kind of work has a clear place in the code. Author-created illustration.

This division is an example of **separation of concerns**: giving different kinds of work their own place. The web adapter handles web details, the service handles task rules, and the repository handles storage details.

Here is the essential structure. This is pseudocode, so it shows the responsibilities without implementing a complete web server.

```
interface TaskRepository:
    insert(title) -> saved Task, or StorageError
    list() -> list of Task records
    set_done(id) -> saved Task, or NotFound, or StorageError

class TaskService:
    constructor(repository):
        self.repository = repository

    create(raw_title):
        title = trim(raw_title)
        require 1 <= length(title) <= 200
        return self.repository.insert(title)

handle_http(request, service):
    title = parse_json(request.body)["title"]
    task = service.create(title)
    return response(201, json(task))
```

Follow a request through this code. The web adapter reads the title from JSON. The service checks it. The repository inserts and commits the task. The saved task returns through the service to the adapter, which builds the response.

The command-line adapter can call the same `service.create(title)`. When the title limit changes, we update the shared rule. Each adapter still checks its own input format: for example, whether a web request contains valid JSON. Task rules and input-format rules have different jobs.

The real Python example implements the service and repository. It also checks that a title is text. A complete HTTP adapter would additionally handle missing fields, invalid JSON, and storage errors.

1.2.3 · Hide details that callers do not need

The service needs to request a saved task. It does not need to know the table name, SQL statement, or file path used to store it. The repository presents a simpler operation, `insert(title)`, while hiding those details. This is **abstraction**.

The contract still matters. In our design, `insert` must return only after commit. The service relies on that behaviour when it returns success. An abstraction should hide implementation details while explaining the behaviour callers depend on.

> ONE OPERATION, SEVERAL HIDDEN STEPS
>
> The service asks “store this task.” The repository handles the query and commit needed to carry out that request.

Encapsulation controls access to a component’s internal data. If every part of the application can change the repository’s records directly, it can bypass the rules we have just defined. Returning a separate task record, rather than a reference that callers can use to edit internal state, helps preserve that control.

Modularity is the organisation of a system into parts with clear responsibilities and interfaces. Separate files can help, but the division is useful only if it limits what each part must know about the others. Ten files that all edit the same internal data can still be difficult to change.

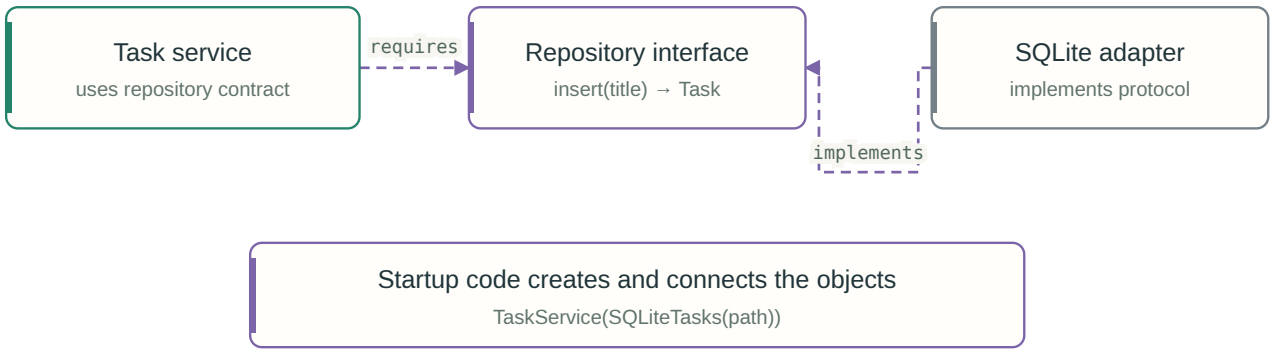


FIG. 1.2.3 **The service uses a repository contract**

The service uses the repository interface. The SQLite adapter implements that interface. Startup code creates the adapter and passes it to the service; during execution, the service calls it.

Why this view: The dashed arrows show code dependencies. These objects can run in the same process. Author-created illustration.

At application startup, we create the SQLite repository and pass it to the service: `TaskService(SQLiteTasks(path))`. Supplying a dependency this way is called **dependency injection**. The startup code that creates and connects these objects is sometimes called the **composition root**. Unit 6 examines these ideas in detail.

Read the dashed arrows in the figure as code dependencies: the service uses the repository contract, and the SQLite adapter implements it. During execution, the service calls that adapter. All of this can happen in one process, without a network call.

1.2.4 · Check the design by changing it

We can judge the division by asking where an edit would be needed. Consider four changes to the application.

Table 1.2.1 · Where should a change be made?

Change	Main place to edit	What else needs checking?
Accept a web form instead of JSON.	Web input adapter.	Does it pass the same title to the service and display errors correctly?

Change	Main place to edit	What else needs checking?
Increase the title limit.	Task service.	Can the database store the longer title? Update any matching database constraint.
Rename a database column.	SQLite repository and the procedure that updates existing data.	Can old records still be read after the change?
Recognise a repeated create request.	Input contract, service, and repository.	Can the application identify the request and recover its earlier result?

The last change affects several parts because it adds new behaviour. A caller must identify a repeated request, and storage must keep enough information to recognise it. Good modules help us see where agreement is needed; they cannot make every change fit inside one file.

TRY IT · INTERACTIVE MODEL

Where would you make the change?

Choose a change and compare the parts that need editing in the two designs.

Scenario: Replace JSON input with a form

Tangled design: Handler · Task logic · SQL inside the handler

Separated design: Input adapter

Update the adapter that reads input. It can pass the same title to the service, which saves it in the same way.

The results follow our example's division of responsibilities. They show where edits are needed, without estimating how long those edits take.

1.2.5 · Understand what a layer hides

We can also arrange the application as a series of **abstraction layers**. Each layer uses the operations offered by the one below it. A user asks to remember a task. The application turns that into a storage operation. The storage implementation uses the operating system to read and write files.

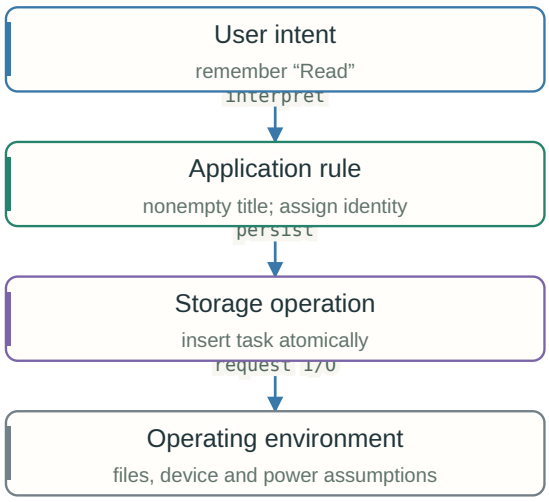


FIG. 1.2.4 **Each layer works at a different level of detail**

The user asks to remember a task. The application applies task rules and requests a storage write. Storage uses files and the underlying device to carry out the write.

Why this view: Follow one request through the layers to connect the user’s action to the work below it. Author-created illustration.

Each step lets us work at a different level of detail. When writing title validation, we do not need to reason about disk blocks. When investigating a damaged database file, those lower-level details may become important.

Some details must be visible in the contract. For example, suppose a repository returns results one at a time and keeps a database connection open until the caller finishes reading. A slow caller can then hold that connection for a long time. The caller needs to know when and how to close the result. Alternatively, the repository could return a bounded page of records and release the connection before returning.

▼ A closer look · Why a test substitute cannot check persistence

We can test title validation using a simple repository that stores tasks in memory. This makes the test quick and easy to control. But the substitute loses its tasks when it is discarded. It cannot demonstrate that SQLite saves a task across a restart.

For that check, use the real repository: create a task, close the connection, open a new connection to the same database, and read the task. The two tests answer different questions. Matching method names does not mean that two implementations have the same behaviour in every situation.

1.2.6 · Use the amount of structure the application needs

In this example, separating the title rule helps because both the web and command-line interfaces use it. Keeping SQL in a repository helps because several features need tasks without needing to know the table layout. These are concrete reasons for the boundaries.

Every added interface also takes work to understand and maintain. A short, temporary script may need only a few functions. As the program grows, introduce a boundary where it protects a rule, hides a storage detail, or allows a real change to be made safely.

The modules still run in one application. If that process crashes, all of them stop. Code organisation improves how we understand, change, and test the application; keeping it running through a machine failure requires further design work.

> COMMON MISTAKES
>
> Separate files do not necessarily have separate responsibilities. An interface does not automatically hide useful details. And fewer dependencies are not always better: the application needs well-defined dependencies to get its work done.

> FURTHER READING
>
> Parnas's 1972 paper examines how the same system can be divided into modules in different ways. It asks which design decisions each module should hide. Our storage example gives a small application of that question. Paper record and abstract ↗

Remember: Give related work a clear home. Let other modules use it through a contract. To check the division, follow a likely change and see which parts need editing.

Before continuing: Both the web form and command-line interface check the title limit. Where should the shared rule move, and which checks should remain in each adapter?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   `Access reviewed · 21 October 2026`

From requirements to design choices

Prerequisites: 1.2 · Organising code into modules 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

The team asks us to make the to-do application “fast, safe, and easy to maintain.” These are reasonable goals, but they leave important questions unanswered. How fast should a task list appear? Which failures should saved tasks survive? What changes should be easy to make?

Before choosing a design, we need to turn these goals into requirements that people can understand and check.

In this chapter: We will write a small set of requirements, compare storage choices, and record why one choice fits our application.

1.3.1 · State what the application must do

A **requirement** states something the system must do or a condition it must satisfy. “A user can mark a task complete” describes behaviour. “A saved task remains after the application restarts” describes a condition that behaviour must meet.

Requirements about what the system does are called **functional requirements**. Requirements about qualities such as speed, security, and ease of maintenance are commonly called **nonfunctional requirements**. Both influence the design.

Consider a shared version of our application. For this example, assume a team of 20 people, one application machine, and a small budget. During normal use, the application receives at most 5 requests per second. These are teaching assumptions, rather than measurements of a real service.

The team needs to create, list, and complete tasks. Each person’s tasks are private. A successful save must survive an application restart, provided the committed storage remains intact. At least 95% of list requests should finish within 300 milliseconds under an agreed test workload.

The timing requirement gives us something to measure. If a test sends 1,000 list requests, at least 950 must finish within 300 milliseconds. To make results comparable, we must also specify where the test runs, how many tasks it lists, and how much traffic it sends. Unit 2 develops this measurement process.

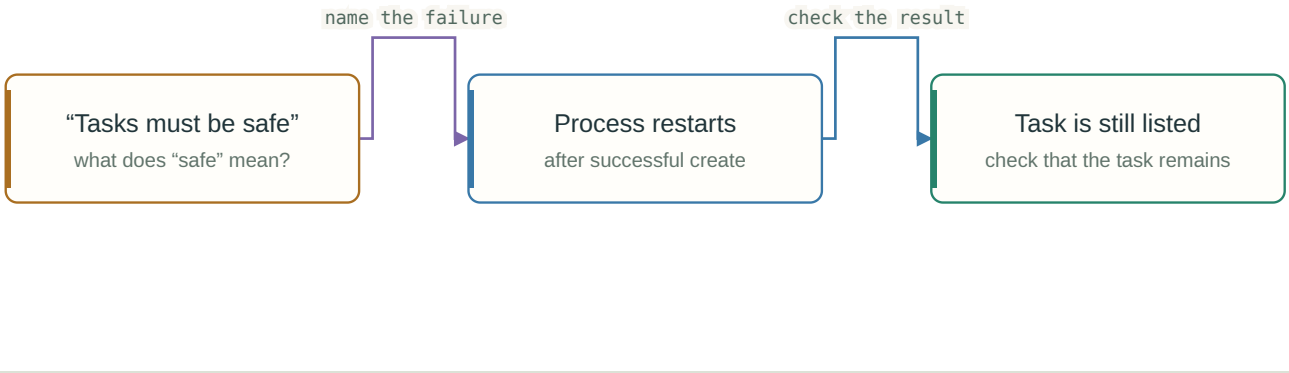


FIG. 1.3.1 **Turn “safe” into a checkable requirement**

Create a task successfully, restart the process, and check that the task remains. This requirement concerns a process restart with committed storage intact.

Why this view: A specific requirement tells us which test to run. Author-created illustration.

Table 1.3.1 · Making a requirement precise

Starting statement	Question to ask	A requirement we can check
“The app is fast.”	Which operation, under how much load?	At least 95% of list requests finish within 300 ms under the agreed workload.
“Tasks are safe.”	Which failure should they survive?	A task reported as saved can be read after the process restarts, with committed storage intact.
“Tasks are private.”	Who is allowed to do what?	Users cannot read or change another person’s tasks through any task endpoint.
“The code is easy to change.”	Which change should be easy?	Add a command-line interface that reuses the task service and repository.

Qualities such as performance, security, reliability, and maintainability are also called **quality attributes** or **architectural characteristics**. Their names help organise a review. A concrete scenario tells us what each one means for this application.

Some requirements fit more than one category. Checking whether a user may complete a task is application behaviour and part of its security. The classification is useful for organising work; the important step is to implement and check the requirement.

1.3.2 · Separate constraints, assumptions, and risks

Suppose the team says that the first version must run on an existing machine. That limits the choices available to us. Such a limit is a **constraint**. A fixed budget or a delivery deadline can also be a constraint.

We have assumed that the disk remains intact when the process restarts. An **assumption** is something we treat as true while making a decision. Writing it down makes clear where the reasoning applies.

Now suppose the only disk fails. We may lose the saved tasks. This is a **risk**: a possible event that could prevent the system from meeting its needs. The team must decide whether to accept it or change the design, for example by keeping recoverable copies elsewhere.

These terms answer different questions. A constraint asks what limits our choices. An assumption asks what our reasoning relies on. A risk asks what could go wrong. They often connect: relying on one intact disk exposes us to the risk of losing that disk.

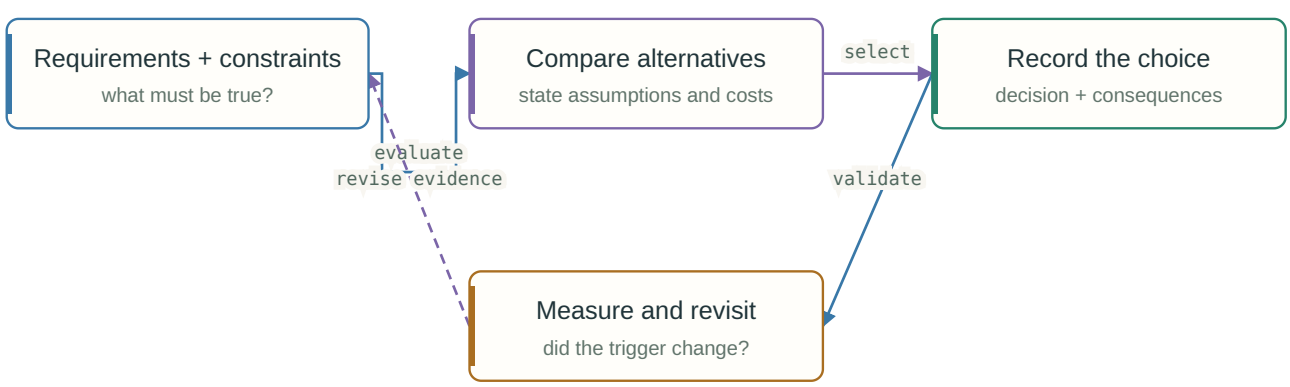


FIG. 1.3.2 **Choose, record, and review a design**

Write down the requirements, compare choices, and record the decision. Later, use measurements and changed requirements to decide whether the design still fits.

Why this view: Keep the reasons for a decision alongside the decision itself. Author-created illustration.

1.3.3 · Compare choices against the requirements

We have three storage candidates. We could keep tasks only in memory, use an embedded database such as SQLite, or run a separate database service. An **embedded database** runs as part of the application rather than as a separate server; here, it stores data in a local file.

First check the mandatory restart requirement. Memory alone fails it. However easy the code is to write, that option cannot meet the agreed need.

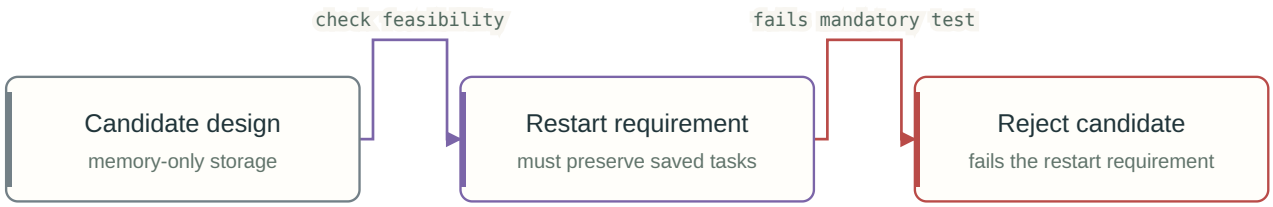


FIG. 1.3.3 **Check requirements before scoring preferences**

Memory-only storage loses tasks after restart, so we reject it for this application. We compare preference scores only for choices that meet the required behaviour.

Why this view: An easy-to-run choice is useful only if it meets the requirements. Author-created illustration.

The other two candidates can meet this requirement with suitable commit behaviour and storage settings. We can then compare their costs and benefits. Embedded storage keeps deployment small. A separate database can be accessed by multiple application machines, but also needs a network connection, access controls, and operating work.

A choice between competing benefits and costs is a **trade-off**. Which cost is worth paying depends on the requirements and on the people who will operate the system.

Table 1.3.2 · Example preference scores: 1 = poor fit, 5 = strong fit

Candidate	Survives process restart?	Simple to run here	Supports multiple app machines	Storage separate from app machine
Memory only	No; reject.	5	1	1

Candidate	Survives process restart?	Simple to run here	Supports multiple app machines	Storage separate from app machine
Embedded store	Yes, under our storage assumptions.	5	2	1
Separate database	Yes, with suitable commit settings.	2	4	4

The scores express judgments about fit for this example. They are not performance measurements. The low score for separating an embedded store from its application machine reflects our local-file setup, rather than a claim about every possible use of SQLite.

We can assign a **weight** to each preference. A higher weight means that preference matters more to the decision. A weighted score multiplies each preference score by its weight, adds the results, and divides by the sum of the weights.

If simplicity has weight 5 and the other two preferences have weight 1 each, the embedded store receives `(5×5 + 1×2 + 1×1) / 7 = 4.00`. The separate database receives `(5×2 + 1×4 + 1×4) / 7 ≈ 2.57`. With these priorities, the embedded store ranks higher.

TRY IT · INTERACTIVE MODEL

Which preferences matter most?

Weights (simplicity, multiple app machines, separate storage): 5, 1, 1.



Embedded store has the higher score with these weights. These are preference scores; they do not measure speed or availability.

Memory only: rejected because tasks disappear after restart.

The scores come from Table 1.3.2. A higher weight gives that preference more influence. Every acceptable choice must still preserve tasks across a process restart.

Try increasing the importance of multiple application machines and separate storage. The ranking may change. This is useful: it reveals how the decision depends on priorities. The memory-only option still fails

the restart requirement, regardless of its score.

1.3.4 · Record the decision and its reasons

A few months later, someone may ask why the team chose a local database. A short **Architecture Decision Record (ADR)** can preserve the answer. It records the situation, the decision, and the consequences. It should also say when the team needs to reconsider.

ADR 001 · ACCEPTED FOR THIS EXAMPLE

Use a local database and commit before reporting success

Situation

A small team uses one application machine. Tasks must survive a process restart. The current requirement does not include keeping the service available after that machine fails.

Decision

Use SQLite on the application machine. Keep input handling, task rules, and storage code in separate modules. Wait for commit before returning success.

Alternatives

Memory alone fails the restart requirement. A separate database is possible, but its extra operating work is not yet justified by the requirements.

Consequences

Deployment is small, and the application has a clear point at which a task is committed. Losing the machine stops the service. Losing its only disk can lose data. We still need backups, restore checks, access controls, and monitoring.

Reconsider when

The application must run on several machines, measured storage contention causes unacceptable delays, or tasks must survive loss of the machine or disk.

Checks to run

Create a task and restart the process. Simulate a storage error. Restore a backup. Measure request times with representative data and traffic.

An ADR need not be long. It needs enough detail for a later reader to understand why the choice made sense. Michael Nygard's article on documenting architecture decisions describes this practice.

If the requirements change, keep the old record and write a new one that replaces it. The earlier decision helps explain which assumption or need has changed.

1.3.5 · Distinguish the qualities we are checking

Some quality terms sound interchangeable. We need to separate them because they require different checks.

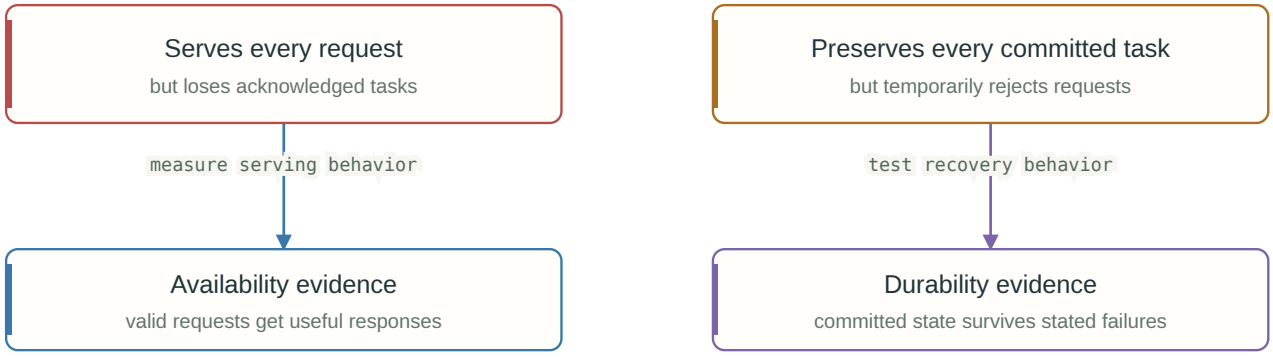


FIG. 1.3.4 **Availability and durability answer different questions**

A service may reply to requests yet lose tasks after restart. Another may keep every committed task but stop accepting requests while it recovers.

Why this view: Check both whether requests succeed and whether saved data survives the stated failure. Author-created illustration.

Availability concerns whether the system can provide the required service when needed. If users cannot list their tasks while the application restarts, it is unavailable for that operation during that interval. When measuring availability, define which requests count and what a useful response is.

Durability concerns whether committed data survives specified failures. Our restart test checks whether a saved task remains after the process stops and starts. It says nothing about losing the only disk.

Reliability usually refers more broadly to providing correct service over time. Teams sometimes use it differently, so a requirement should explain the failures and outcomes it covers.

Consistency concerns which observations of state the system allows. For example, after completing a task, should the user's next list request show it as complete? If the application later has several copies of the data, we must specify when each copy may show the change. Unit 14 examines precise consistency models.

These distinctions explain two possible outcomes. An application may keep replying to requests yet lose tasks it held only in memory. A database may preserve every committed task while refusing requests during recovery. The first has a data-loss problem; the second has a period of unavailability.

▼ A closer look · What makes a design simple enough?

Count the work needed to understand, change, operate, and recover the system. One large function may use few components yet be hard to maintain. A separate managed database may add a network connection while reducing backup work for a team.

The simplest sufficient design meets the agreed requirements with manageable work. Low traffic does not remove the need for access controls or correct saves. At the same time, an imagined future need does not automatically justify extra infrastructure today. Record likely changes and the conditions that would make them necessary.

COMMON MISTAKES

A high preference score cannot compensate for a failed requirement. More users do not, by themselves, tell us which part needs scaling. And a good uptime figure does not show whether saved data survives a failure.

Remember: Write requirements that can be checked. Reject choices that fail them. Compare the remaining choices, then record the reasons and the conditions for reconsidering.

Before continuing: The team still has only 20 users, but it now requires tasks to survive destruction of the only disk. Which part of ADR 001 must be reconsidered?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. Documenting Architecture Decisions — Michael Nygard, 2011 ↗

   Original practice article. ADR structure and decision history.

   Access reviewed · 21 October 2026

2. AWS Well-Architected: the six pillars ↗

   Official guidance · accessed 21 Oct 2026. Review lenses, not correctness guarantees.

   Access reviewed · 21 October 2026

Drawing the system at different levels

Prerequisites: 1.3 · From requirements to design choices 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

A diagram shows three boxes: input adapter, task service, and repository. A reviewer asks whether the application needs three servers. The author intended all three to be modules inside one program.

Both readers saw the same boxes but gave them different meanings. A useful system diagram must make its level of detail clear.

In this chapter: We will draw the to-do application from several viewpoints, trace a request, and use the drawings to find possible failures.

1.4.1 · Choose a view that answers a question

Suppose we want to explain the application to a new team member. We might begin with the person who uses it and the task system as a whole. A developer changing validation needs a closer view of the modules. Someone investigating a machine failure needs to see where the application and data run.

These are different **views** of the same system. Each selects the details needed for a particular question.

The C4 model names four levels for describing system structure: context, containers, components, and code. Moving through them is like zooming in on the application.

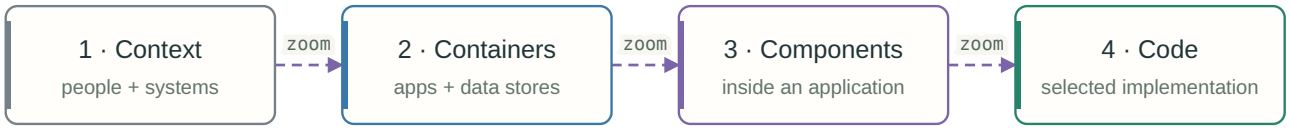


FIG. 1.4.1 **C4: zoom from the system into its code**

Each step shows more internal detail. The dashed arrows mean "zoom in." In C4, a container is an application or data store; it need not be a Docker container.

Why this view: Select the level that helps your reader answer a particular question. Author-created illustration.

Table 1.4.1 · Choosing a diagram

View	Question it answers	What it shows here
System context	Who uses the system, and which other systems interact with it?	A team member uses the task system.
Container	Which applications and data stores make up the system?	Browser client, server application, and task store.
Component	How is an application divided internally?	Input adapter, task service, and storage adapter inside the server application.
Code	How is a selected component implemented?	Task records, service methods, and the repository interface.
Sequence	In what order do events occur in one scenario?	The application validates a request, saves the task, commits, and replies.
Deployment	Where do the application and data run?	One machine runs the application and holds the database file.

A **container** has a particular meaning in C4: an application or data store. It can be a browser application, a server application, or a database. It does not have to be a Docker container. The C4 container definition explains this terminology.

Sequence and deployment views add information about order and location. They support the four structural levels rather than adding more levels of zoom. We can use whichever views help explain the design.

TRY IT · INTERACTIVE MODEL

Zoom in on the same application

Who uses the system?

Team member → uses → Task system

This view shows who uses the system. It leaves the internal parts for a closer view.

Select a level to see which parts it shows. Figure 1.4.1 and Table 1.4.1 explain all four levels without using these controls.

1.4.2 · Show where the program and data live

Our first version was an in-memory script. The second used modules and a local database. Now the team needs to access tasks from several computers, so we add a web interface. A browser sends requests to the server application, which uses the task service and repository.

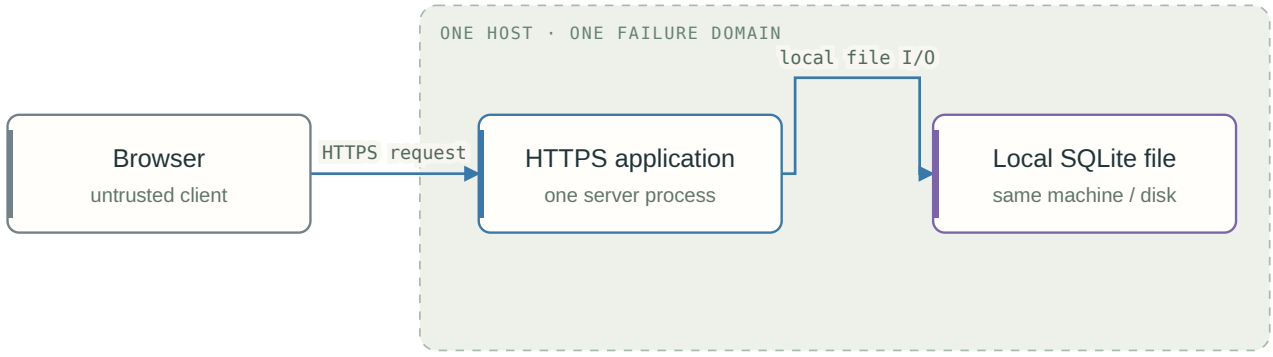


FIG. 1.4.2 **The application and database share one machine**

The browser sends requests to an application on one machine. The database file is on that same machine. If the machine fails, the application and access to the file are both affected.

Why this view: Show physical placement when asking what can fail together. Author-created illustration.

The deployment view shows an important detail: the server application and SQLite file are on the same machine. The browser is elsewhere. If the server machine fails, both the application and access to its database are affected.

Parts that can fail together because of one cause belong to a **failure domain**. Here, the machine is a shared failure domain. Drawing the service and repository as separate boxes does not give them independent machines.

The browser also introduces a **trust boundary**. The server cannot assume that a request is valid merely because it came from the application's usual web page. A user can construct requests directly.

The server must establish who the caller is; this is **authentication**. It must then check whether that caller may access the requested task; this is **authorization**. Every task operation needs the relevant check. Hiding another person's task from the web page does not prevent a direct request for it. Unit 18 develops these checks in detail.

1.4.3 · Trace a request from input to reply

Let us create a task through the web interface. The browser sends `POST /tasks` with a JSON body containing `{"title": "Read"}`. The following sequence describes the intended API; the local Python example is not an HTTP server.

```
1. Identify the caller from a validated session.
2. Read the title and check the input.
3. Begin a database transaction.
4. Insert the task with the caller as its owner.
5. Commit the transaction.
6. Return:
   201 Created
   Location: /tasks/7
   {"id": 7, "title": "Read", "done": false}
```

The `201 Created` response tells the client that creation succeeded. The `Location` header identifies the new resource. HTTP's specification defines this response. Waiting for the database commit is an additional rule of our application.

Now interrupt the sequence. If title validation fails, the application writes no task. If the transaction fails before commit, the application reports an error. If the process crashes after commit but before the reply, task 7 exists even though the browser has not received success.



FIG. 1.4.3 **The task is saved, but the reply is lost**

Task 7 is committed before its reply is lost. The client cannot tell what happened. A fresh create request may add task 8 unless the application recognises it as a retry.

Why this view: Trace the events in order to see why the client can be unsure even when a task exists. Author-created illustration.

What should the browser conclude after waiting too long for a reply? It cannot tell whether the task was saved. The request might never have reached the application, or the task might have been committed and the reply lost.

If the browser sends a fresh create request, the application may add task 8. The user intended one task but now has two. To recognise retries, the client and server need a stable identifier for the original operation. The server must store that identifier with its result so that a repeated attempt can retrieve the earlier result. Chapter 8.2 develops this mechanism, called request deduplication.

This is why event order belongs in the diagram. A drawing with only “request” and “response” can hide the commit that determines whether a task exists.

1.4.4 · Label arrows and copies of data

An arrow can mean a function call, a network request, a code dependency, or a step in a sequence. Label it with the actual relationship. For example, “HTTPS request,” “calls insert,” and “implements interface” describe different things.

Use a legend when colour or line style carries meaning. In this atlas, dashed arrows often show dependencies or zoom relationships, while solid arrows show the labelled action. Read the labels and caption to find the meaning in that particular view.

Also show where copies of data are held. The browser may display a task list it fetched a minute ago. The application may hold a task record in memory. The database contains the committed record. If another user

completes a task, the browser's old list does not update itself. We must decide when it fetches or receives new data.

A diagram that distinguishes these copies makes the freshness question visible. A single label saying "task data" does not tell us which copy a reader is seeing.

▼ A closer look · Four ways to review a diagram

1. **Stop a component.** What stops with it? If several modules share one process, a process crash affects all of them.
2. **Delay a call.** Who is waiting? Decide how long that caller can wait and what happens when the time runs out.
3. **Repeat a request.** Does it add another task? Point to the code and stored information that recognise a retry, if that behaviour is required.
4. **Change a data format.** Which readers and writers must change together? Check how old and new versions will work during the update.

These questions help uncover omissions. Tests are then needed to check the actual behaviour.

1.4.5 · Change the design when there is a reason

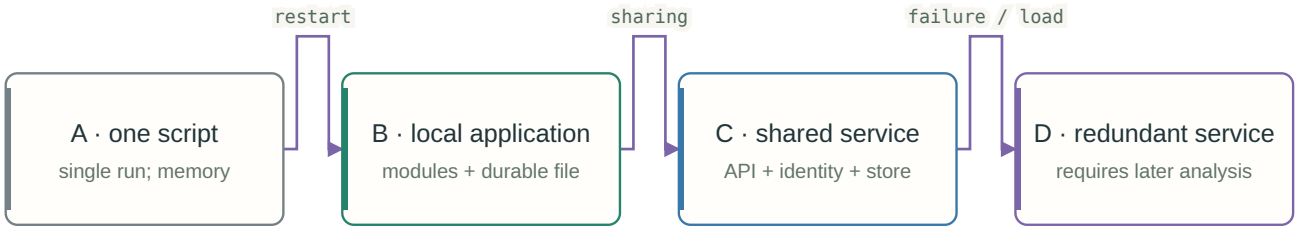


FIG. 1.4.4 **Each design change answers a new need**

Add storage when tasks must survive restart. Add a remote interface when people need shared access. Extra machines require further decisions about data, failures, and load.

Why this view: State the requirement that makes each change necessary. Author-created illustration.

Each change in our example answered a particular need. Persistent storage let tasks survive a process restart. Modules gave shared rules a clear home. A web interface let people use the application remotely. None of these changes made it survive failure of its only machine.

Suppose the application becomes slow. Before adding another server, measure where requests spend their time. A slow database query may need a better query or index. Adding application servers could send even more work to that same query. If computation in the application is the bottleneck and storage has spare capacity, additional workers may help.

Adding a second machine also changes our storage assumptions. If each machine has its own unrelated task database, a user may see different lists depending on which machine handles the request. We need a design for shared or coordinated data; a load balancer alone cannot supply it.

The drawings help us review these consequences together. A structure view assigns responsibilities. A sequence view shows event order. A deployment view shows where execution and data live. They should describe a consistent design.

> COMMON MISTAKES
>
> Three module boxes need not mean three servers. A timeout does not establish that a write failed. And another application machine does not automatically provide another usable copy of the data.

Remember: Choose a diagram for a specific question. Label what the boxes and arrows mean. Check that the code structure, request sequence, and deployment tell the same story.

Before continuing: Which view would help you find a reply sent before commit? Which view would show that the application and its only backup are on the same disk?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. C4: diagrams ↗

   Original model documentation. Formal diagram levels and supporting views.

   Access reviewed · 21 October 2026

2. C4: container abstraction ↗

   Original model documentation. Meaning of container.

   Access reviewed · 21 October 2026

3. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

PART 1 / UNIT 02

Requirements, Estimation, and Engineering Trade-offs

The running example

A social platform has 10 million daily active users. Estimate requests per second, traffic at peak, storage growth, image bandwidth, and approximate infrastructure requirements.

Unit 2 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

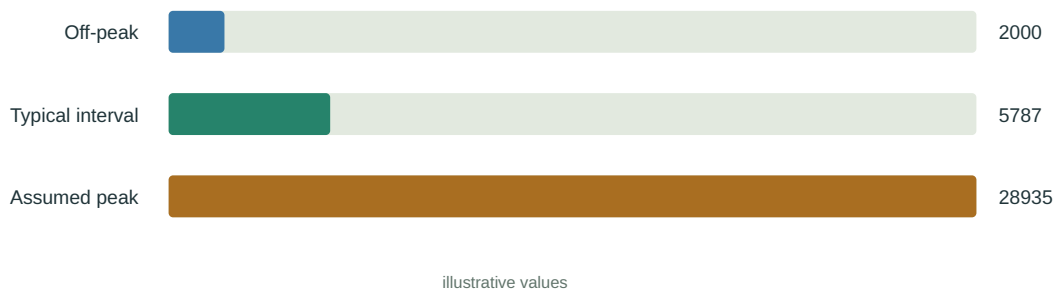


FIG. 2.0.1 **Request-volume timeline**

Request volume across illustrative intervals; these are assumed rates, not a measured daily traffic trace.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

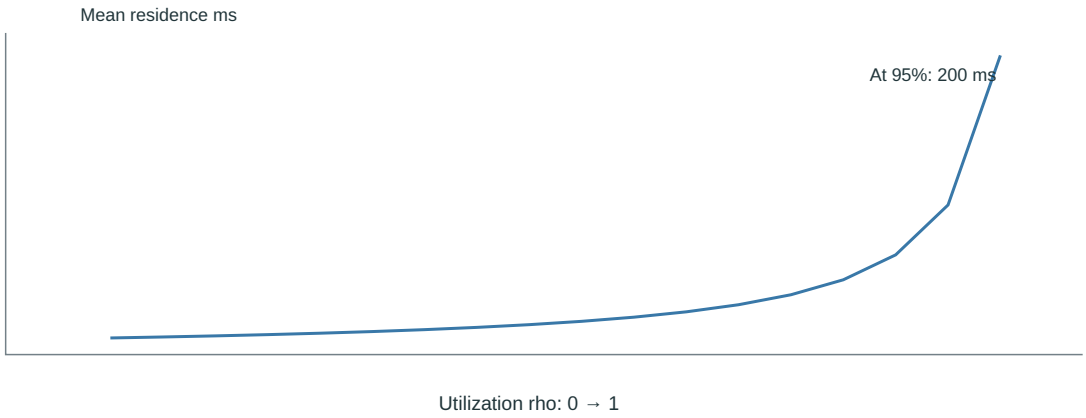


FIG. 2.0.2 **throughput-latency curve**

Stable M/M/1 teaching model with 10 ms service time: mean residence rises as utilization approaches one. It does not predict production p99.

Why this view: Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

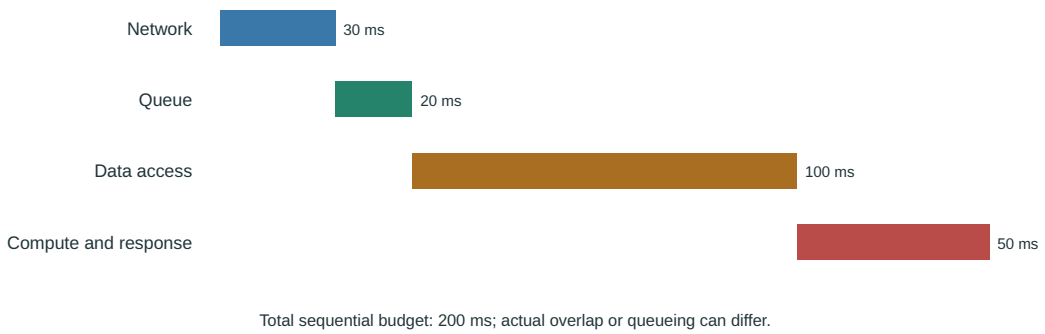


FIG. 2.0.3 **latency waterfall**

A 200 ms sequential budget allocates every stage; real overlap and tails require measurement.

Why this view: Separate stage waits and execution rather than attach one unexplained number to the entire request. Author-created illustration. Numbers are illustrative unless stated otherwise.

Input	Calculation	Result
Daily API requests	10 million × 50	500 million
Average RPS	500 million / 86400	5787.04
Assumed peak	average × 5	28935.19
Post bytes/day	10 million × 2000	20 GB

FIG. 2.0.4 **capacity planning worksheet**

Explicit workload assumptions create a worksheet, not a benchmark.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Turn user journeys into workloads

Prerequisites: Introduction to System Design 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social platform has 10 million daily active users. Estimate requests per second, traffic at peak, storage growth, image bandwidth, and approximate infrastructure requirements.

This chapter examines turn user journeys into workloads through concrete mechanisms, state snapshots, failure cases, and alternatives.

2.1.1 · Gathering and prioritizing requirements

A request to build a feed hides several products: reading posts, publishing, moderating, and following. Observe a user journey and distinguish mandatory outcomes from negotiable targets. “Publish once, show followers within five seconds” is actionable; “real-time social app” is not. Prioritize by business harm and uncertainty. Prototype the riskiest assumption first, and record what is excluded: direct messages do not belong in the estimate until explicitly included.

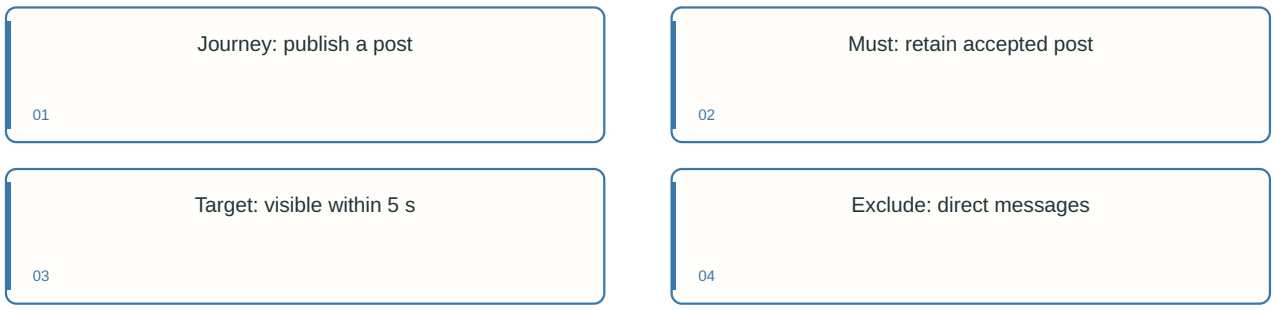


FIG. 2.1.1 **Gathering and prioritizing requirements · worked snapshots**

Journey: publish a post · Must: retain accepted post · Target: visible within 5 s · Exclude: direct messages. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.1.2 · User journeys and workload characteristics

Users do not generate a constant stream of identical requests. Break a session into operations and attach payload size, arrival rate, fan-out, and dependency cost to each. In an assumed session, opening a feed causes one feed request, twenty image reads, and one analytics event. Counting only the first request underestimates network work. A journey model helps predict which component saturates; production telemetry should replace its assumptions as soon as evidence exists.

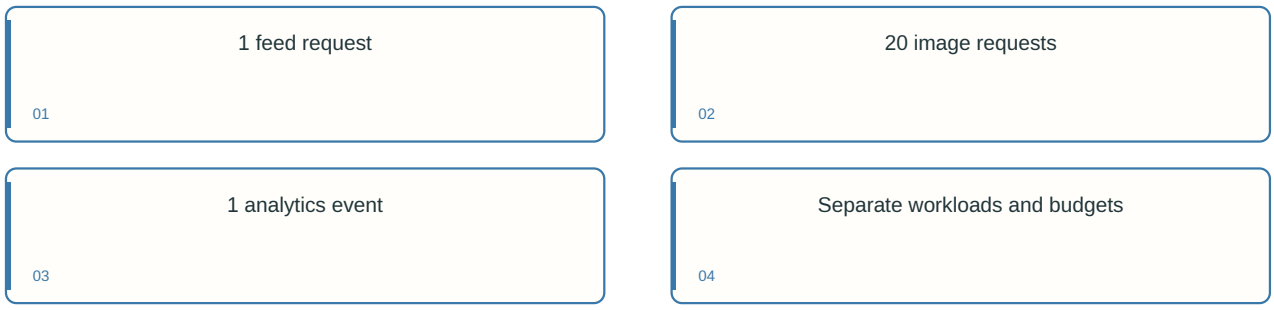


FIG. 2.1.2 **User journeys and workload characteristics · worked snapshots**

1 feed request · 20 image requests · 1 analytics event · Separate workloads and budgets. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.1.3 · Read-heavy vs write-heavy systems

A read/write ratio is meaningful only at a named boundary. Ten feed reads per post can cause thousands of writes if publishing materializes follower feeds. Conversely, fan-out-on-read shifts work into read-time joins. Count logical business actions separately from physical operations and amplification. Read replicas help appropriate reads, but cannot eliminate a serialized writer or guarantee read-your-writes when replicas lag. Choose the implementation after identifying the actual pressure.

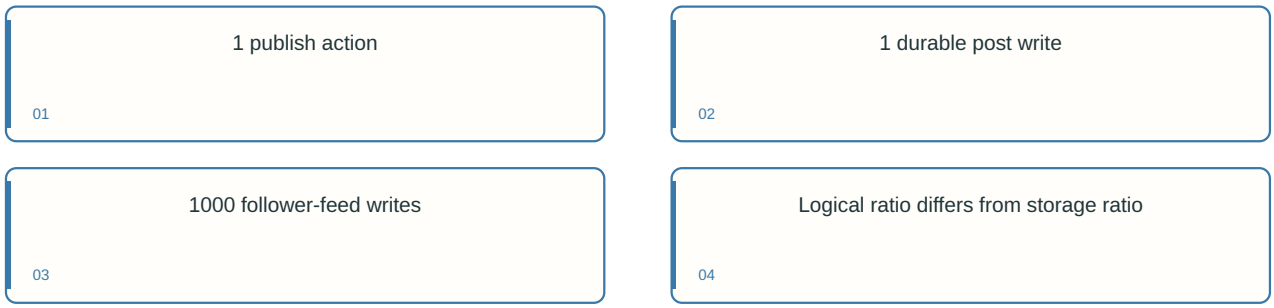


FIG. 2.1.3 **Read-heavy vs write-heavy systems · worked snapshots**

1 publish action · 1 durable post write · 1000 follower-feed writes · Logical ratio differs from storage ratio. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

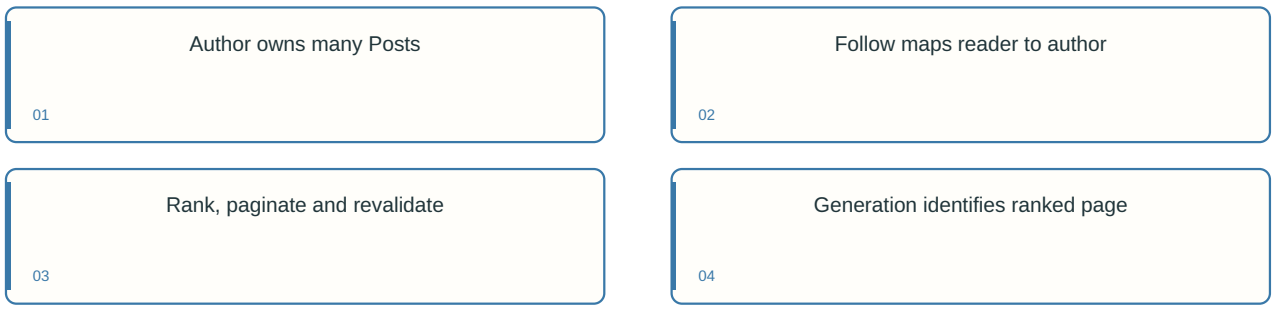


FIG. 2.1.4 **The running system · invariant and recovery review**

Review the assumptions about inactive readers, celebrity skew, and access revocation. Test a deleted or private post already present in several cached candidate pages, duplicate fan-out, a paused worker, and an unavailable feature service. Verify the fallback feed remains authorized. A queue drained to zero does not prove that every intended follower received exactly one usable candidate.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would a coordinated retry burst change an estimate based only on independent average user sessions?

Use capacity() and compare 5787.04 average RPS, 28935.19 peak RPS, 81 active instances at the stated hypothetical capacity/utilization, and 82 for one-instance failure headroom. This arithmetic does not prove the database or complete architecture meets the workload.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Read replicas help appropriate reads, but cannot eliminate a serialized writer or guarantee read-your-writes when replicas lag.

Carry forward: 1 publish action · 1 durable post write · 1000 follower-feed writes · Logical ratio differs from storage ratio.

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   Access reviewed · 21 October 2026

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Measure the experience

Prerequisites: 2.1 · Turn user journeys into workloads 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social platform has 10 million daily active users. Estimate requests per second, traffic at peak, storage growth, image bandwidth, and approximate infrastructure requirements.

This chapter examines measure the experience through concrete mechanisms, state snapshots, failure cases, and alternatives.

2.2.1 · Latency, throughput, QPS, RPS, TPS

Latency measures elapsed time; throughput measures completed work per time interval. Offered requests per second can exceed successful completions per second while a queue grows. QPS usually counts queries, RPS requests, and TPS transactions, but teams must define these denominators. At 200 offered requests/s and 180 completions/s, backlog grows by twenty requests each second unless requests are rejected. Reporting only accepted traffic conceals overload and abandoned work.

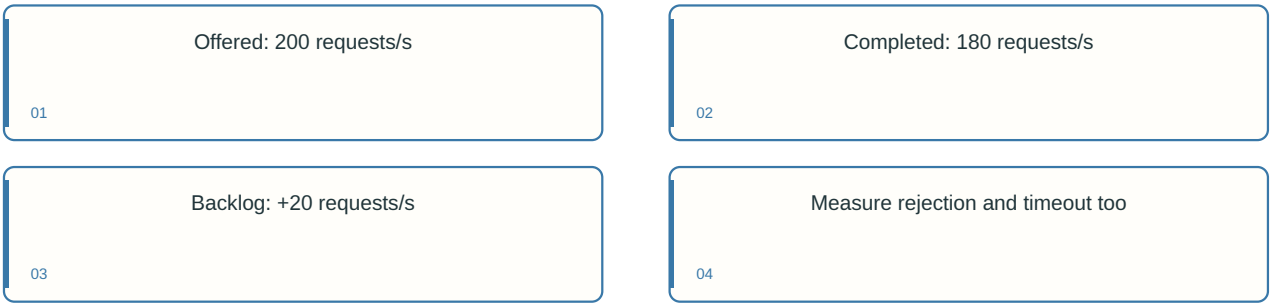


FIG. 2.2.1 **Latency, throughput, QPS, RPS, TPS · worked snapshots**

Offered: 200 requests/s · Completed: 180 requests/s · Backlog: +20 requests/s · Measure rejection and timeout too. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.2 · Average vs median vs p95/p99 latency

A mean can hide users who wait much longer. For 100 illustrative requests, 99 at 10 ms and one at 1000 ms, the mean is 19.9 ms; the slow request is still a second. Percentiles describe the distribution under a specified estimation convention. Do not average machine p99 values to obtain fleet p99; merge distributions or appropriate histograms. Include errors and timed-out operations in the experience model rather than quietly dropping them.

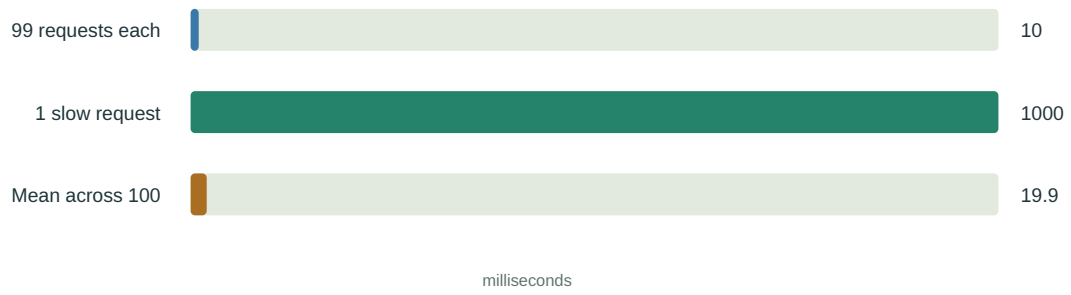


FIG. 2.2.2 **A small mean can hide one slow request**

The mean is (99×10+1000)/100 = 19.9 ms. This is a synthetic distribution, not a percentile convention.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.3 · Availability, reliability, durability, consistency

A reachable service can serve stale or lost data. Availability is useful response delivery, durability is retention under a failure model, reliability is correct service over time under a stated definition, and consistency constrains allowed observations. A payment status page may remain available while returning an outdated state. Decide which property each operation needs; an inventory decrement and a popularity counter often justify different consistency and latency costs.

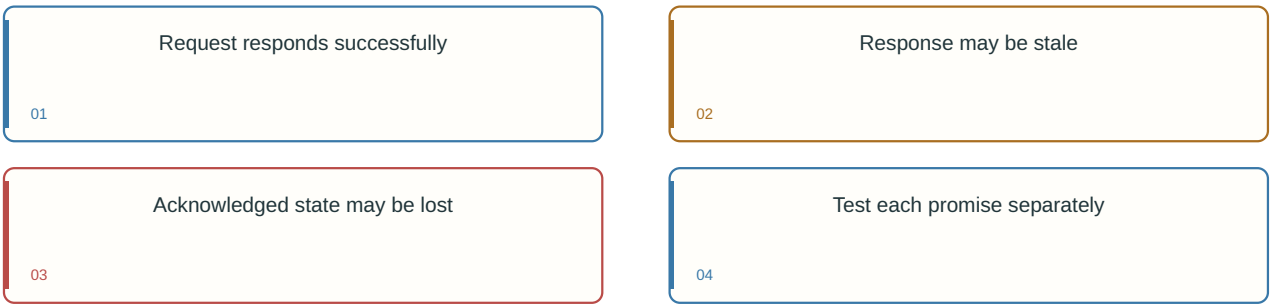


FIG. 2.2.3 **Availability, reliability, durability, consistency · worked snapshots**

Request responds successfully · Response may be stale · Acknowledged state may be lost · Test each promise separately. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.4 · SLI, SLO, SLA, and error budgets

Choose a user-visible SLI, an objective and window, then describe any contractual SLA separately. A 99.9% event-based success objective across one million eligible requests allows 1000 bad events. Excluding failures by definition invalidates the budget. Error budgets inform release and reliability decisions; they do not authorize spending all failures in one outage. A time-based SLO uses time rather than request count and gives a different allowance.

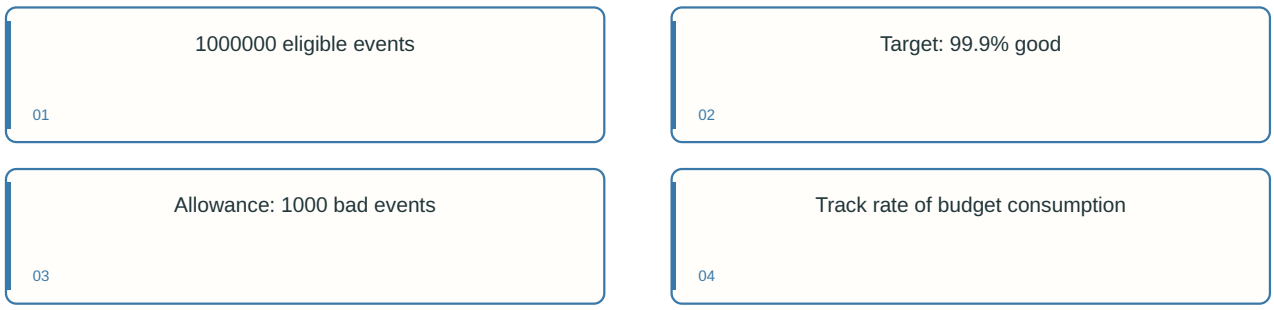


FIG. 2.2.4 **SLI, SLO, SLA, and error budgets · worked snapshots**

1000000 eligible events · Target: 99.9% good · Allowance: 1000 bad events · Track rate of budget consumption. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would a coordinated retry burst change an estimate based only on independent average user sessions?

Use capacity() and compare 5787.04 average RPS, 28935.19 peak RPS, 81 active instances at the stated hypothetical capacity/utilization, and 82 for one-instance failure headroom. This arithmetic does not prove the database or complete architecture meets the workload.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Error budgets inform release and reliability decisions; they do not authorize spending all failures in one outage.

Carry forward: 1000000 eligible events · Target: 99.9% good · Allowance: 1000 bad events · Track rate of budget consumption.

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   Access reviewed · 21 October 2026

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Estimate the resources

Prerequisites: 2.2 · Measure the experience 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social platform has 10 million daily active users. Estimate requests per second, traffic at peak, storage growth, image bandwidth, and approximate infrastructure requirements.

This chapter examines estimate the resources through concrete mechanisms, state snapshots, failure cases, and alternatives.

2.3.1 · Capacity and growth planning

Plan for growth with scenarios rather than a single extrapolation. Separate stored data, working set, background jobs, failure headroom, and migration overlap. An assumed 10% monthly growth gives 1.1^12, about 3.14 times the starting volume after a year. It does not prove traffic or CPU grows at the same rate. A capacity plan should name its limiting resource, procurement or scaling delay, and the measurement that triggers an update.

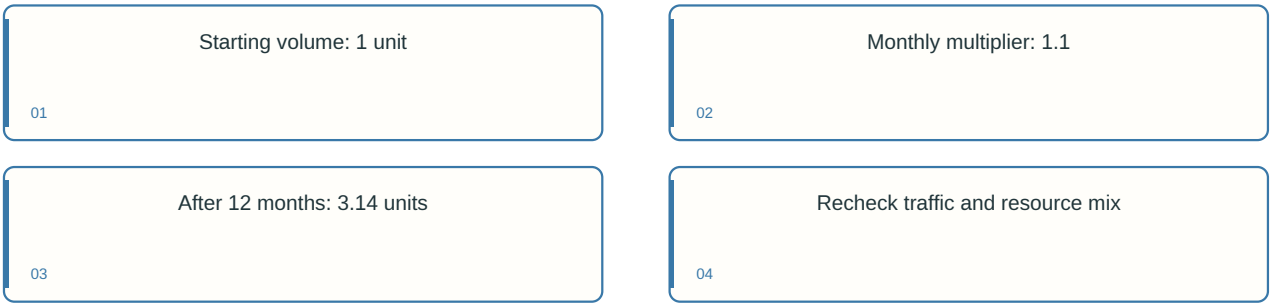


FIG. 2.3.1 **Capacity and growth planning · worked snapshots**

Starting volume: 1 unit · Monthly multiplier: 1.1 · After 12 months: 3.14 units · Recheck traffic and resource mix. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.3.2 · Storage, memory, bandwidth, and CPU estimates

Assume ten million daily users, fifty API requests each, one 2 kB post per user daily, and ten 200 kB images viewed each. Average API load is 500 million/86400, about 5787 requests/s. Raw post growth is 20 GB/day using decimal units; image transfer is 20 TB/day before caching. Replication, indexes, metadata, compression, and retention are separate factors. These are workload calculations, not claims about any company's infrastructure.

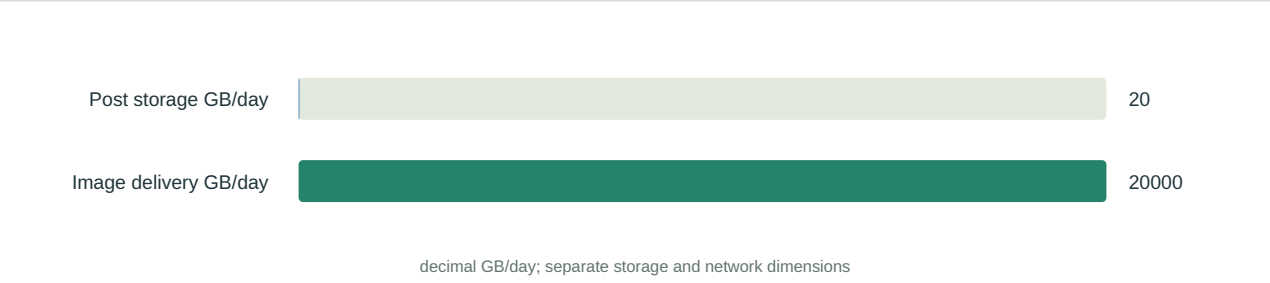


FIG. 2.3.2 **Different workload dimensions have different magnitudes**

Ten million assumed users each write one 2 kB post and view ten 200 kB images. Storage and delivered traffic are separate workloads.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.3.3 · Peak vs average load

Average traffic hides synchronized arrivals. With an assumed peak factor of five, the preceding 5787 requests/s becomes about 28935 at peak. A uniform fivefold multiplier is only a planning hypothesis: image and write peaks may differ, and retries can amplify outages. Evaluate a spike's duration as well as height because an autoscaler that starts new instances after the spike may provide little protection. Admission limits must work before scaling catches up.

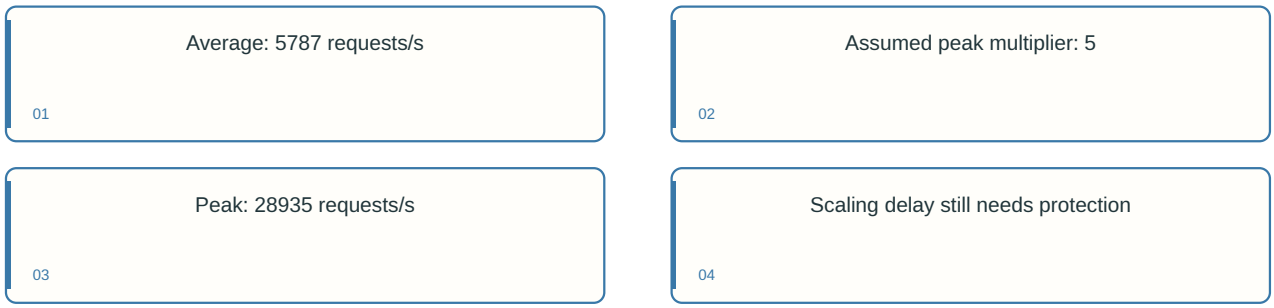


FIG. 2.3.3 **Peak vs average load · worked snapshots**

Average: 5787 requests/s · Assumed peak multiplier: 5 · Peak: 28935 requests/s · Scaling delay still needs protection. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

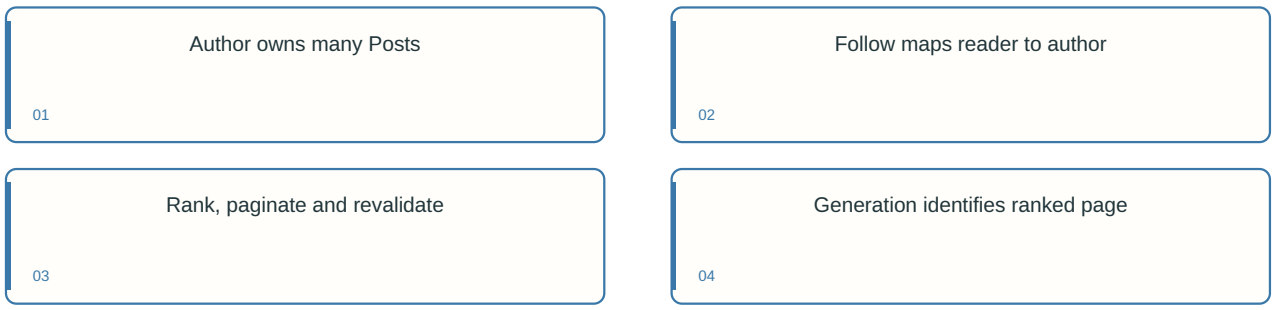


FIG. 2.3.4 **The running system · invariant and recovery review**

Review the assumptions about inactive readers, celebrity skew, and access revocation. Test a deleted or private post already present in several cached candidate pages, duplicate fan-out, a paused worker, and an unavailable feature service. Verify the fallback feed remains authorized. A queue drained to zero does not prove that every intended follower received exactly one usable candidate.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

INFRASTRUCTURE WORKSHEET · ASSUMED CAPACITY

At 28,935.19 peak requests/s, a hypothetical tested instance capacity of 600 requests/s and a 60% utilization target imply ceil(28,935.19 / 360) = 81 active instances. Reserving capacity for one lost instance gives 82. This does not size the database, image delivery path, or failure domains. Validate each separately; the 600 figure is an assumed input, not a reported benchmark.

▼ Architecture review · How would a coordinated retry burst change an estimate based only on independent average user sessions?

Use capacity() and compare 5787.04 average RPS, 28935.19 peak RPS, 81 active instances at the stated hypothetical capacity/utilization, and 82 for one-instance failure headroom. This arithmetic does not prove the database or complete architecture meets the workload.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Evaluate a spike's duration as well as height because an autoscaler that starts new instances after the spike may provide little protection.

Carry forward: Average: 5787 requests/s · Assumed peak multiplier: 5 · Peak: 28935 requests/s · Scaling delay still needs protection.

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   `Access reviewed · 21 October 2026`

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   `Access reviewed · 21 October 2026`

Queues, bottlenecks & defensible decisions

Prerequisites: 2.3 · Estimate the resources 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social platform has 10 million daily active users. Estimate requests per second, traffic at peak, storage growth, image bandwidth, and approximate infrastructure requirements.

This chapter examines queues, bottlenecks & defensible decisions through concrete mechanisms, state snapshots, failure cases, and alternatives.

2.4.1 · Little's Law and queueing intuition

Little's Law relates long-run averages across the same stable boundary: L = lambda W. At 500 successful operations/s and 40 ms average residence time, average in-flight work is twenty. If residence time becomes 400 ms at the same stable throughput, it becomes two hundred. The equation is an accounting relationship, not a p99 model or proof that an overloaded queue will stabilize. Rejections, retries, and work outside the boundary need explicit accounting.

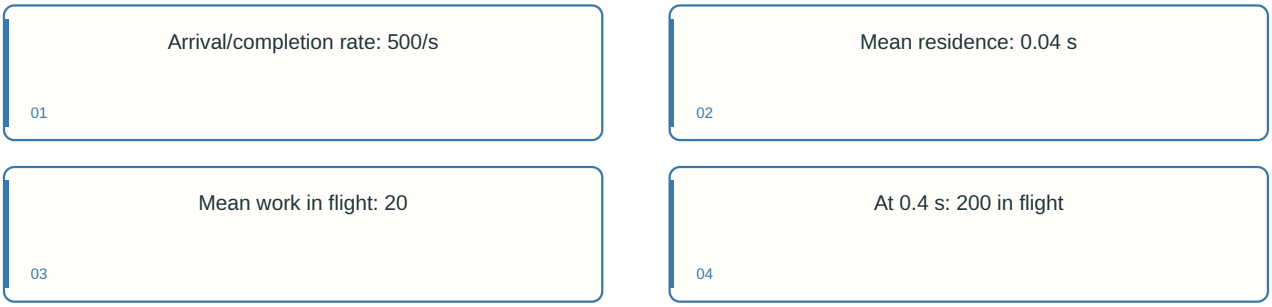


FIG. 2.4.1 **Little's Law and queueing intuition · worked snapshots**

Arrival/completion rate: 500/s · Mean residence: 0.04 s · Mean work in flight: 20 · At 0.4 s: 200 in flight. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.2 · Amdahl's Law and bottlenecks

Amdahl's Law bounds ideal fixed-work speedup when only part of the work can be parallelized. If 80% parallelizes perfectly, four workers give 1/(0.2+0.8/4)=2.5 times speedup, while infinitely many approach five. Coordination and changed cache behavior can make real speedup worse. Optimize the actual limiting stage: doubling web workers is ineffective when every request waits for a serialized database operation. Re-measure after each improvement because the bottleneck moves.

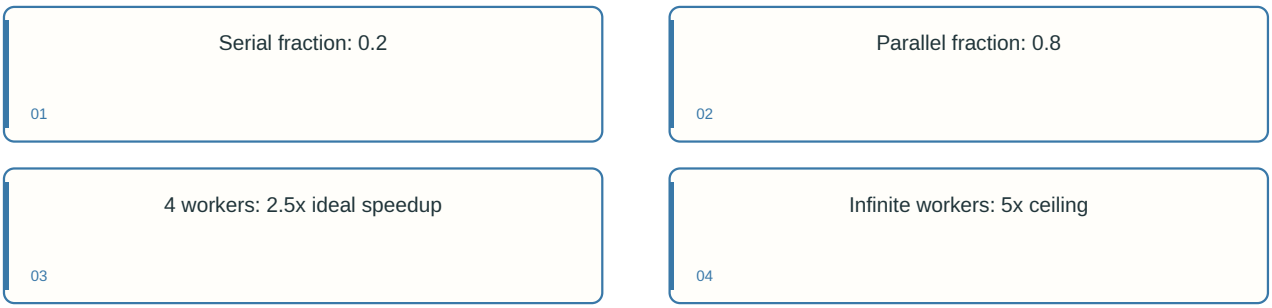


FIG. 2.4.2 **Amdahl's Law and bottlenecks · worked snapshots**

Serial fraction: 0.2 · Parallel fraction: 0.8 · 4 workers: 2.5x ideal speedup · Infinite workers: 5x ceiling. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.3 · Tail latency and latency budgets

A request's tail can be dominated by the slowest dependency. If it waits for all fifty independent calls and each has a 1% chance of exceeding a threshold, the chance that at least one exceeds it is 1−0.99^50, about 39.5%. Independence is an assumption; shared failures can correlate tails. Allocate an end-to-end deadline across network, queues, execution, and response. Hedging may reduce some tails but spends extra capacity and requires safe duplicate handling.

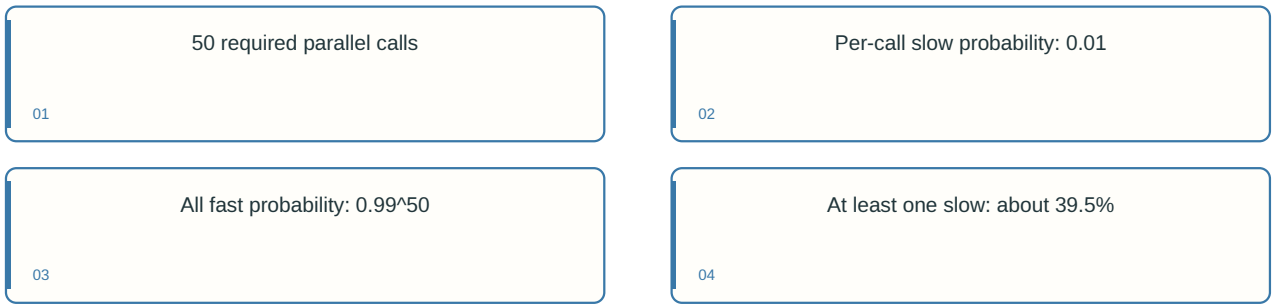


FIG. 2.4.3 **Tail latency and latency budgets · worked snapshots**

50 required parallel calls · Per-call slow probability: 0.01 · All fast probability: 0.99^50 · At least one slow: about 39.5%. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.4 · Cost, security, operational complexity, and maintainability as constraints

Capacity is not the only constraint. A cheap design that violates tenant isolation is infeasible; a high-throughput design requiring unsupported operational expertise may be a poor fit. Classify constraints as mandatory and preferences as negotiable. Include staffing, incident recovery, migration, security review, and maintenance in total cost. A vendor-neutral comparison should model the same workload and service promises for each candidate; comparing unlike failure guarantees makes the apparent price advantage misleading.

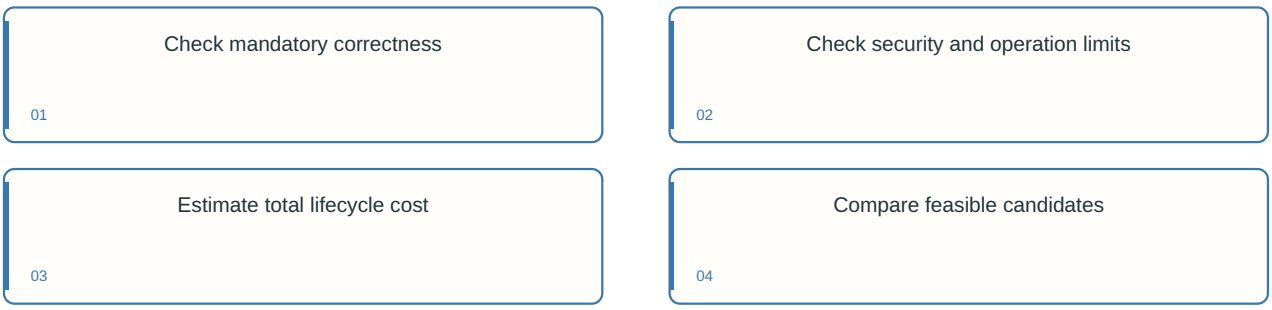


FIG. 2.4.4 **Cost, security, operational complexity, and maintainability as constraints · worked snapshots**

Check mandatory correctness · Check security and operation limits · Estimate total lifecycle cost · Compare feasible candidates. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.5 · Decision matrices and trade-off analysis

A decision matrix makes judgments visible but does not turn them into measurements. Apply hard constraints first, then score candidates against defined criteria with explicit weights and uncertainty. If a small weight change reverses the winner, gather more evidence or prefer a reversible choice. Record alternatives and revisit triggers in an ADR. One weighted total cannot establish an invariant, eliminate an unknown dependency, or substitute for a representative load test.

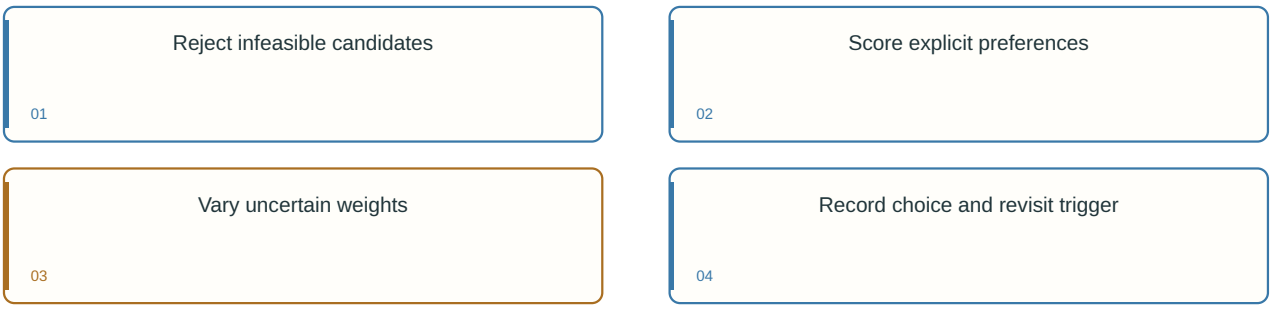


FIG. 2.4.5 **Decision matrices and trade-off analysis · worked snapshots**

Reject infeasible candidates · Score explicit preferences · Vary uncertain weights · Record choice and revisit trigger. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · EXAMPLES/MECHANISMS.PY

```
from mechanisms import capacity, amdahl
print(capacity())
print(amdahl(.8, 4))
```

The default assumptions yield 5787.04 average RPS, 28935.19 peak RPS, 81 nominal active instances at hypothetical tested capacity and utilization, and 82 for one-instance loss. This is a worksheet result, not a validated infrastructure deployment.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

Size the assumed social workload

Average API rate: **5,787.04 requests/s**; peak: **28,935.19 requests/s**.

Nominal active instances: **81**; arithmetic headroom for one-instance loss: **82**.

Raw posts: 20 GB/day; image delivery: 20 TB/day under the fixed image assumptions.

Decimal units. Assumed image views: 10 × 200 kB/user/day; posts: 1 × 2 kB/user/day. Per-instance capacity is an input, not an atlas benchmark. Database and failure-domain feasibility need independent evidence.

▼ Architecture review · How would a coordinated retry burst change an estimate based only on independent average user sessions?

Use capacity() and compare 5787.04 average RPS, 28935.19 peak RPS, 81 active instances at the stated hypothetical capacity/utilization, and 82 for one-instance failure headroom. This arithmetic does not prove the database or complete architecture meets the workload.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Record alternatives and revisit triggers in an ADR.

Carry forward: Reject infeasible candidates · Score explicit preferences · Vary uncertain weights · Record choice and revisit trigger.

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   Access reviewed · 21 October 2026

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

PART 1 / UNIT 03

Networking Foundations for System Design

The running example

Trace what happens when someone enters a website URL, from DNS lookup through server execution to browser rendering.

Unit 3 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

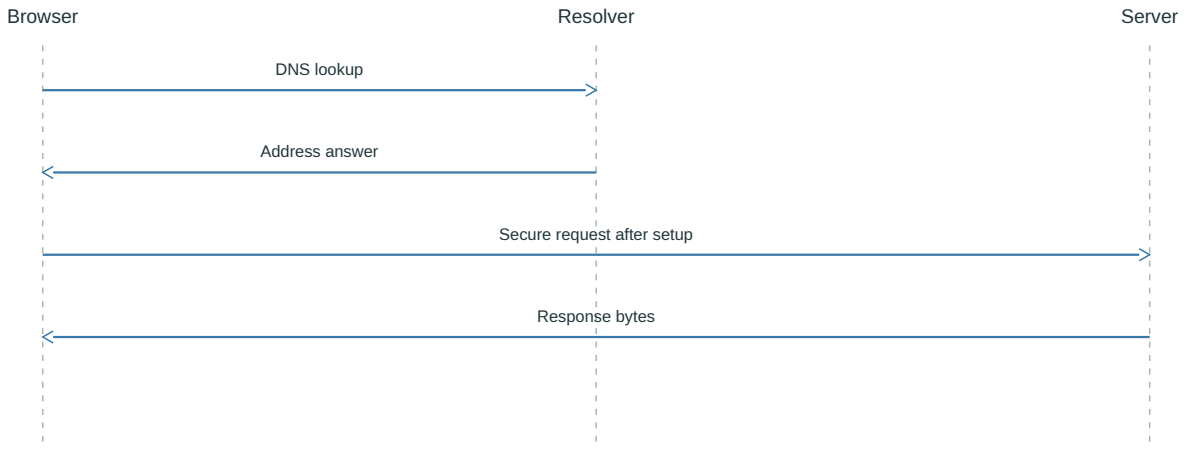


FIG. 3.0.1 **Complete HTTP request timeline**

The full request includes name resolution, connection/security setup, execution and rendering. This is a simplified ordered scenario.

Why this view: Follow explicit message ordering and locate the acknowledgment or effect boundary. Author-created illustration. Numbers are illustrative unless stated otherwise.

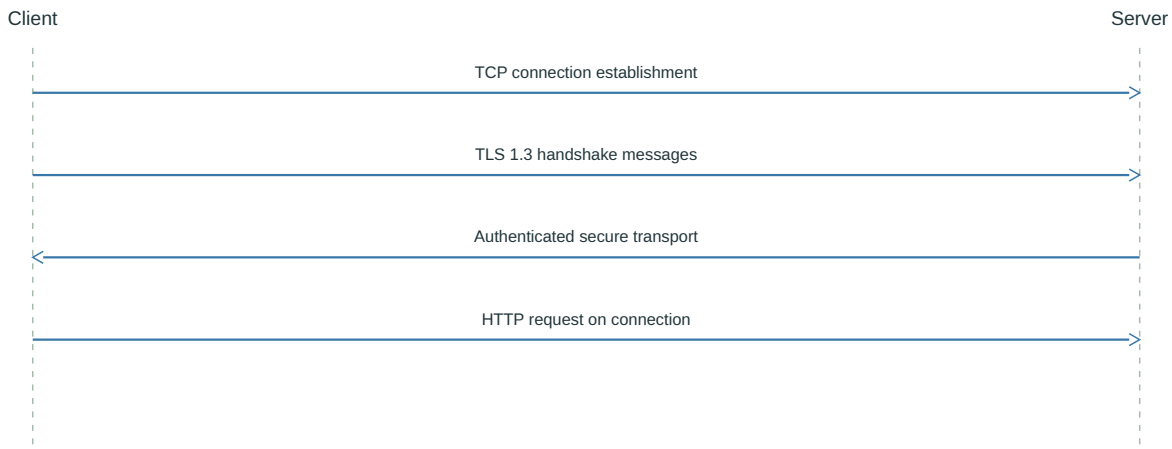


FIG. 3.0.2 **TCP/TLS interaction**

TCP/TLS interaction is shown at protocol-stage granularity; it is not a packet-exact capture or a universal RTT count.

Why this view: Follow explicit message ordering and locate the acknowledgment or effect boundary. Author-created illustration. Numbers are illustrative unless stated otherwise.

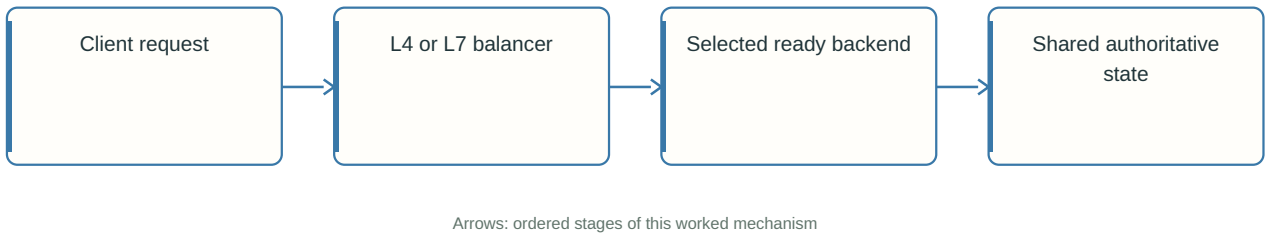


FIG. 3.0.3 **load-balancing topology**

Balancing distributes suitable request paths, while state and failure ownership remain explicit.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Mode	Normal update path	Recovery obligation
Polling	Repeated client queries	Choose polling interval
SSE	Server streams HTTP events	Reconnect and replay policy
WebSocket	Bidirectional framed channel	Heartbeat, reconnect, history

FIG. 3.0.4 **WebSocket vs polling communication**

Transport patterns differ in connection and polling work; none automatically supplies durable business delivery.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Addresses, sockets & transports

Prerequisites: Introduction to System Design 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Trace what happens when someone enters a website URL, from DNS lookup through server execution to browser rendering.

This chapter examines addresses, sockets & transports through concrete mechanisms, state snapshots, failure cases, and alternatives.

3.1.1 · How the internet works from the application perspective

Entering a URL begins a chain involving name resolution, a route, transport, a secure application exchange, server execution, and rendering. Each stage may reuse cached state or an existing connection. Trace the actual path before assigning a “network latency” number. A fast server can still deliver a slow page if DNS, connection setup, large assets, or rendering dominate. The application perspective is a set of dependent waits and transfers rather than one instantaneous remote call.

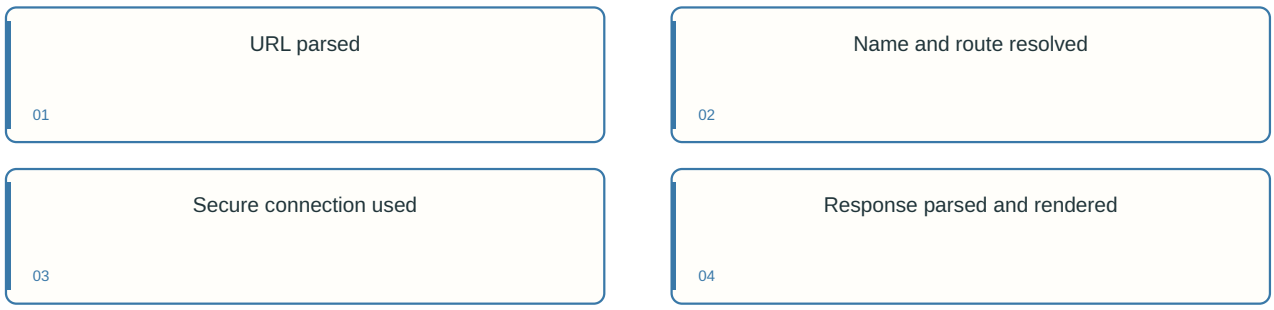


FIG. 3.1.1 **How the internet works from the application perspective · worked snapshots**

URL parsed · Name and route resolved · Secure connection used · Response parsed and rendered. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.1.2 · IP addresses, ports, sockets, routing

An IP address identifies a network endpoint within a routing context; a port identifies a transport endpoint at that host. A socket represents a communication endpoint managed by the operating system. Routing forwards packets toward destinations, potentially through several networks and translations. A service listening on port 443 is not necessarily reachable: binding, routing, firewall policy, and return paths all matter. Ports are not authentication credentials, and private address space is not an authorization boundary.

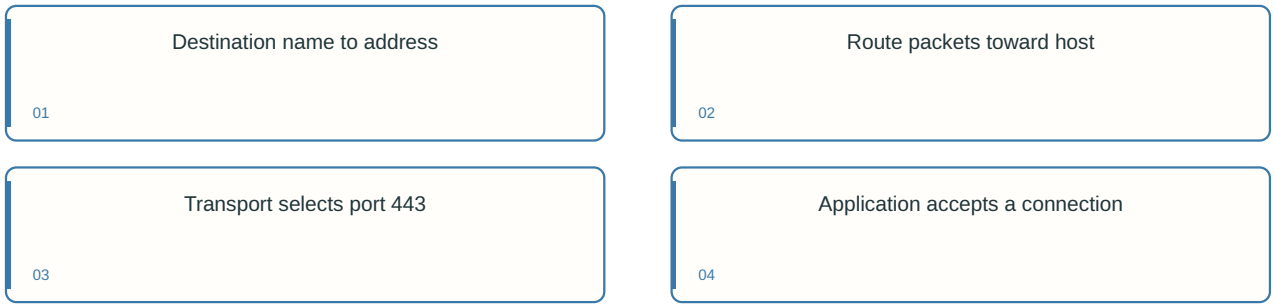


FIG. 3.1.2 **IP addresses, ports, sockets, routing · worked snapshots**

Destination name to address · Route packets toward host · Transport selects port 443 · Application accepts a connection. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.1.3 · TCP vs UDP vs QUIC

TCP supplies an ordered reliable byte stream; it preserves bytes, not application-message boundaries. UDP sends datagrams without those stream guarantees. QUIC builds secure streams and transport behavior over UDP, including stream-level loss isolation and connection identifiers. Choose by application requirements and deployed infrastructure, not by claiming UDP is always faster. A lost TCP packet can delay later bytes on the connection; QUIC separates streams but still shares congestion and application dependencies.

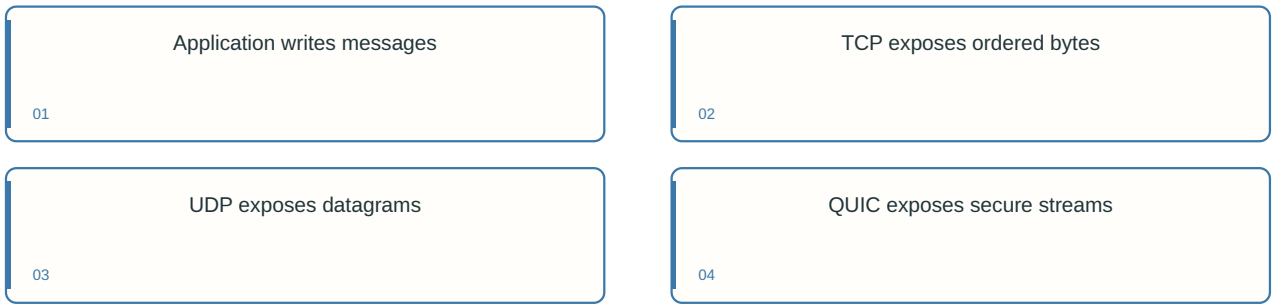


FIG. 3.1.3 **TCP vs UDP vs QUIC · worked snapshots**

Application writes messages · TCP exposes ordered bytes · UDP exposes datagrams · QUIC exposes secure streams. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

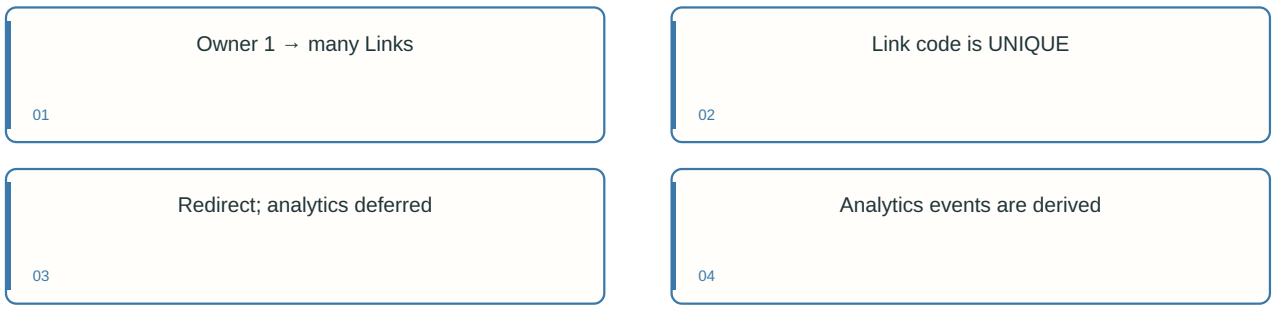


FIG. 3.1.4 **The running system · invariant and recovery review**

The weakest assumptions are cache freshness during revocation, cold-cache origin capacity, and global uniqueness after partitioning. Review them with a revoke-under-load test, a full cache-loss exercise, and a forced code-collision test spanning the actual ownership boundary. Prefer an explicit bounded-degradation policy over a claim that cache replication automatically solves all three concerns.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What changes when a supposedly local service call becomes a cross-region network dependency?

Use the stated 20+40+40+55 = 155 ms cold example, then remove only stages actually reused. Do not subtract server execution because the connection is warm. Reject a negative stage time and keep the result labeled an assumed model, not a measured protocol guarantee.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Choose by application requirements and deployed infrastructure, not by claiming UDP is always faster.

Carry forward: Application writes messages · TCP exposes ordered bytes · UDP exposes datagrams · QUIC exposes secure streams.

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   `Access reviewed · 21 October 2026`

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   `Access reviewed · 21 October 2026`

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   `Access reviewed · 21 October 2026`

A secure request, one round trip at a time

Prerequisites: 3.1 · Addresses, sockets & transports 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Trace what happens when someone enters a website URL, from DNS lookup through server execution to browser rendering.

This chapter examines a secure request, one round trip at a time through concrete mechanisms, state snapshots, failure cases, and alternatives.

3.2.1 · DNS resolution and DNS caching

A resolver obtains records for a name, often through caches and authoritative servers. TTL limits intended cache lifetime, not an instantaneous global invalidation deadline. Negative answers can also be cached. During a failover, clients with cached addresses may continue contacting an old endpoint, and long-lived connections may bypass DNS entirely. Test resolver behavior and connection lifetimes together; changing a DNS record alone cannot guarantee immediate traffic movement.

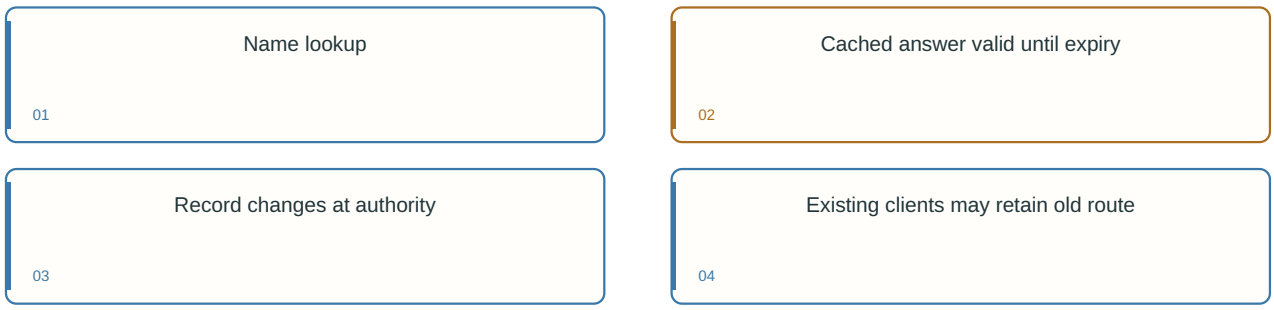


FIG. 3.2.1 **DNS resolution and DNS caching · worked snapshots**

Name lookup · Cached answer valid until expiry · Record changes at authority · Existing clients may retain old route. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.2 · TLS and HTTPS

TLS authenticates peers according to the configured trust model and protects traffic integrity and confidentiality. HTTPS is HTTP over a secure transport arrangement. Certificate validation, hostname checks, key protection, and termination placement are architectural decisions. TLS does not authorize a user to read another tenant's data. If an edge proxy terminates encryption, the next hop needs its own trust and transport policy; a lock icon does not describe every internal connection.

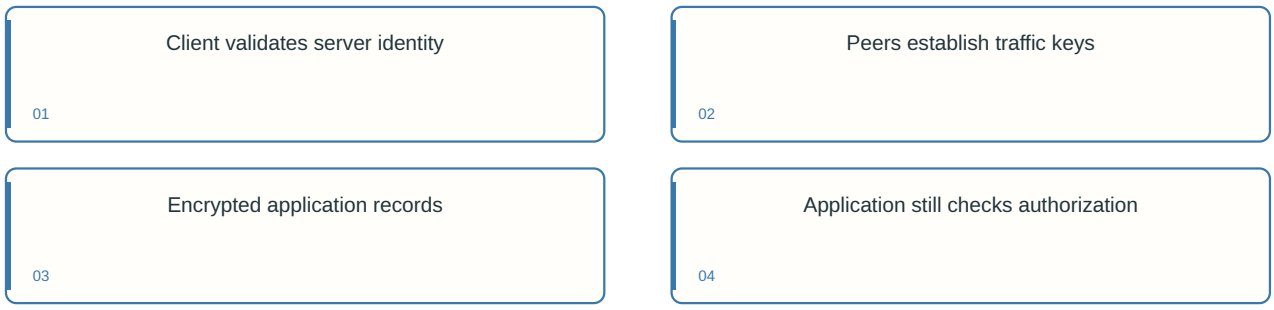


FIG. 3.2.2 **TLS and HTTPS · worked snapshots**

Client validates server identity · Peers establish traffic keys · Encrypted application records · Application still checks authorization. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.3 · HTTP/1.1, HTTP/2, HTTP/3

HTTP/1.1 often needs multiple connections to avoid response blocking among requests. HTTP/2 multiplexes streams on a TCP connection, but transport loss can stall all dependent bytes. HTTP/3 uses QUIC streams and has different transport behavior. All share HTTP semantics at the application level. Protocol negotiation, proxies, payload sizes, and client implementations influence results. Do not infer that a protocol upgrade removes database time, server queues, or every form of head-of-line blocking.

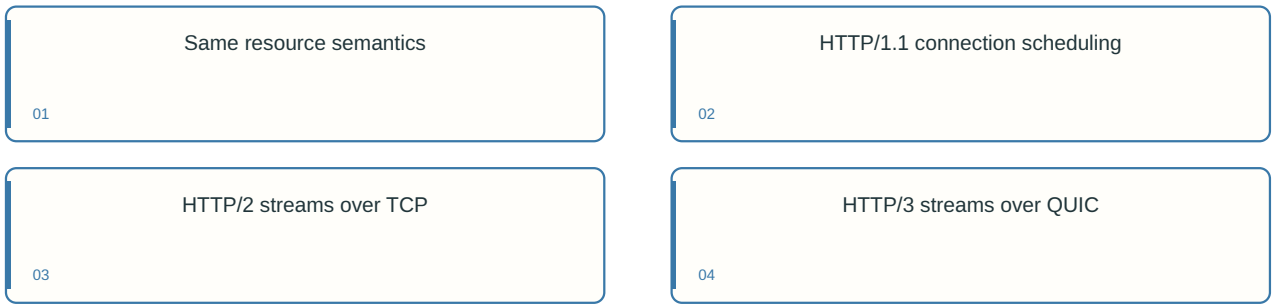


FIG. 3.2.3 **HTTP/1.1, HTTP/2, HTTP/3 · worked snapshots**

Same resource semantics · HTTP/1.1 connection scheduling · HTTP/2 streams over TCP · HTTP/3 streams over QUIC. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.4 · Connection establishment and connection reuse

An existing connection avoids some setup work. A new TCP connection usually requires a transport handshake before normal traffic; secure connection establishment adds protocol-specific exchanges. Reuse connections with appropriate idle limits and connection-pool bounds. Excessively long lifetimes can retain unhealthy endpoints or stale DNS choices; excessively short ones waste CPU and round trips. The right lifetime depends on infrastructure timeouts, workload burstiness, and cancellation behavior rather than a universal keepalive number.

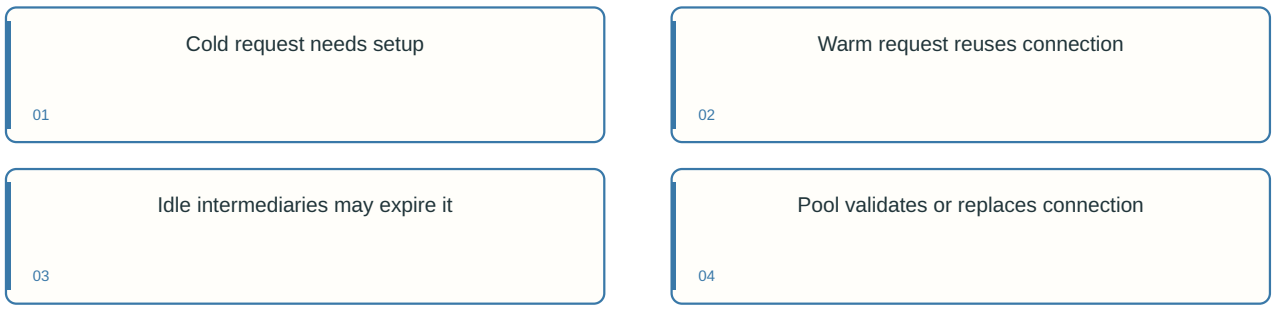


FIG. 3.2.4 **Connection establishment and connection reuse · worked snapshots**

Cold request needs setup · Warm request reuses connection · Idle intermediaries may expire it · Pool validates or replaces connection. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.5 · Latency caused by network round trips

Round trips create a latency floor even before useful computation. In an illustrative cold path, DNS costs 20 ms, TCP 40 ms, TLS 40 ms, and request/response plus execution 55 ms: total 155 ms. This is a simplified sequential example, not a benchmark; resumption, QUIC, cached DNS, and overlap change it. Moving a service closer can reduce propagation delay, while batching or reuse reduces the number of required exchanges.

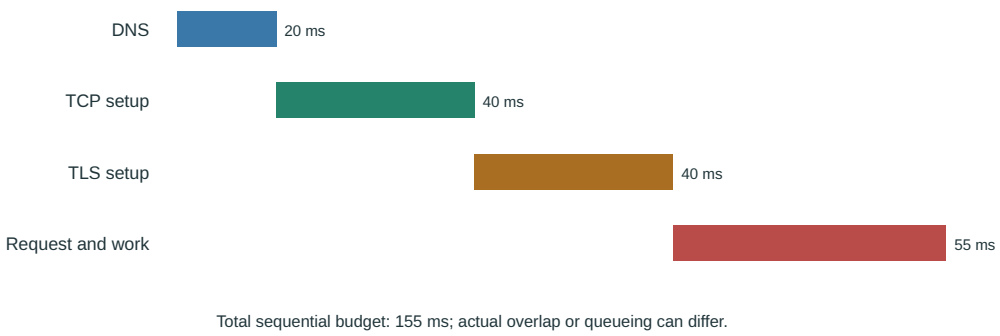


FIG. 3.2.5 **A cold request spends time before server work**

A sequential illustrative cold path totals 155 ms. Reuse, protocol, overlap, and actual distance change the result.

Why this view: Separate stage waits and execution rather than attach one unexplained number to the entire request. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What changes when a supposedly local service call becomes a cross-region network dependency?

Use the stated 20+40+40+55 = 155 ms cold example, then remove only stages actually reused. Do not subtract server execution because the connection is warm. Reject a negative stage time and keep the result labeled an assumed model, not a measured protocol guarantee.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

This is a simplified sequential example, not a benchmark; resumption, QUIC, cached DNS, and overlap change it.

Carry forward: DNS: 20 ms · TCP setup: 40 ms · TLS setup: 40 ms · Request plus work: 55 ms.

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

Intermediaries & persistent connections

Prerequisites: 3.2 · A secure request, one round trip at a time 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Trace what happens when someone enters a website URL, from DNS lookup through server execution to browser rendering.

This chapter examines intermediaries & persistent connections through concrete mechanisms, state snapshots, failure cases, and alternatives.

3.3.1 · Reverse proxies vs forward proxies

A forward proxy acts on behalf of clients; a reverse proxy fronts servers. A corporate forward proxy may control outbound access, while a reverse proxy terminates TLS, routes traffic, and enforces limits before an application. Both add another failure and timeout boundary. Configure trusted forwarding headers deliberately: accepting arbitrary client-supplied identity or source-address headers can bypass controls. Always identify which party selected the proxy and which upstream identity it can legitimately assert.

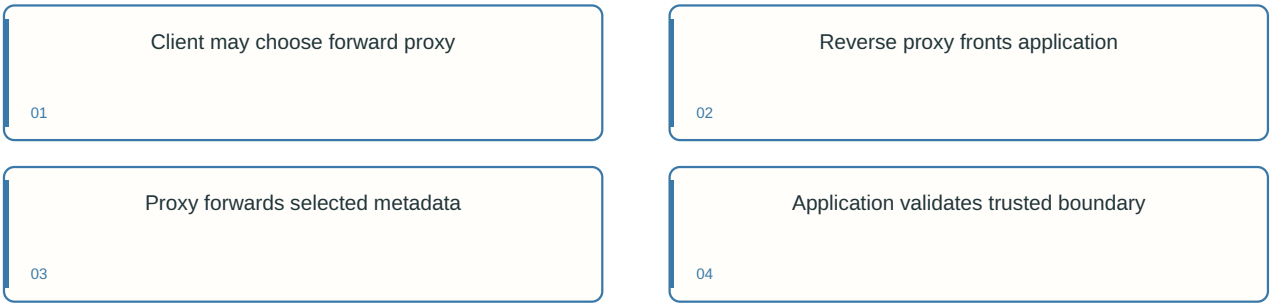


FIG. 3.3.1 **Reverse proxies vs forward proxies · worked snapshots**

Client may choose forward proxy · Reverse proxy fronts application · Proxy forwards selected metadata · Application validates trusted boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.2 · Load balancers vs API gateways

A load balancer distributes traffic among backends; an API gateway commonly adds API-specific policy such as authentication integration, quotas, and request transformation. Products overlap, so compare responsibilities instead of names. A gateway that centralizes every business rule becomes a coupled deployment bottleneck. Keep domain authorization where resource ownership is known, and let an edge gateway perform appropriate coarse admission and routing. Neither component fixes an overloaded database by itself.

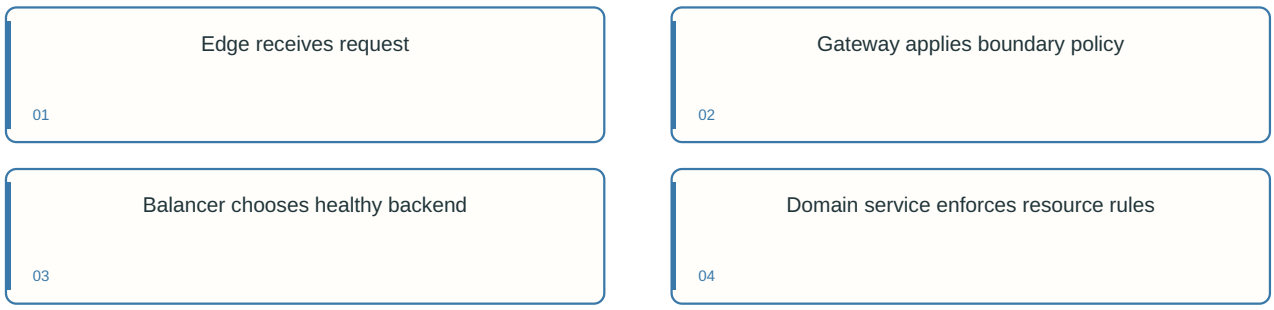


FIG. 3.3.2 **Load balancers vs API gateways · worked snapshots**

Edge receives request · Gateway applies boundary policy · Balancer chooses healthy backend · Domain service enforces resource rules. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.3 · L4 vs L7 load balancing

Layer-4 routing selects using transport information and may pass encrypted traffic through. Layer-7 routing understands application semantics such as HTTP host or path and generally needs access to those messages. L7 policy can be richer but consumes processing and creates parsing and trust obligations. A long-lived stream may remain attached to one backend even if later connections use another. Choose the level based on the routing decision, encryption placement, and connection behavior required.

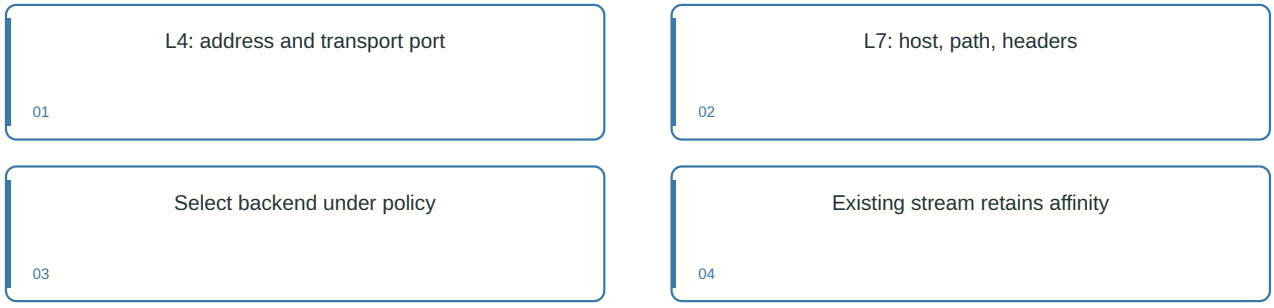


FIG. 3.3.3 **L4 vs L7 load balancing · worked snapshots**

L4: address and transport port · L7: host, path, headers · Select backend under policy · Existing stream retains affinity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.4 · WebSockets, Server-Sent Events, and polling

Polling repeatedly asks for updates; server-sent events stream server-to-client events over HTTP; WebSockets support bidirectional messaging after protocol setup. With 1000 idle clients polling every five seconds, the baseline is 200 polls/s before useful updates. Streaming reduces redundant polls but holds connections and requires reconnect, heartbeat, buffering, and recovery design. Neither WebSockets nor SSE automatically supplies durable message delivery or application-level replay after a disconnected client returns.

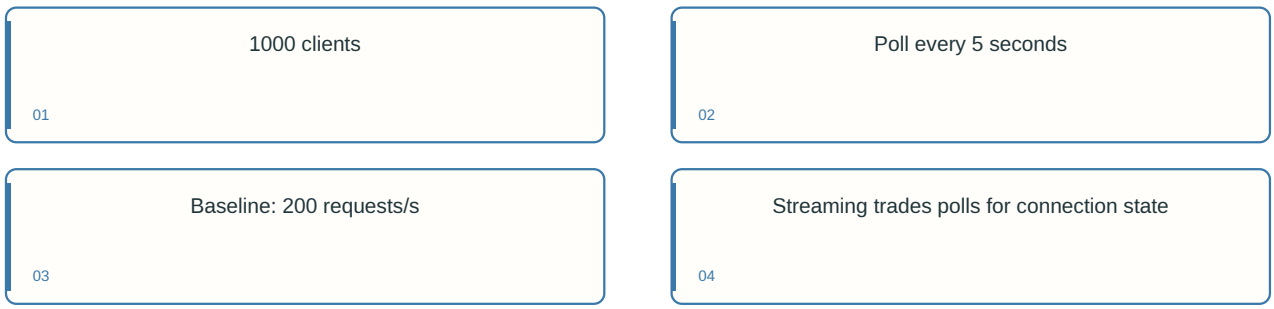


FIG. 3.3.4 **WebSockets, Server-Sent Events, and polling · worked snapshots**

1000 clients · Poll every 5 seconds · Baseline: 200 requests/s · Streaming trades polls for connection state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What changes when a supposedly local service call becomes a cross-region network dependency?

Use the stated 20+40+40+55 = 155 ms cold example, then remove only stages actually reused. Do not subtract server execution because the connection is warm. Reject a negative stage time and keep the result labeled an assumed model, not a measured protocol guarantee.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Streaming reduces redundant polls but holds connections and requires reconnect, heartbeat, buffering, and recovery design.

Carry forward: 1000 clients · Poll every 5 seconds · Baseline: 200 requests/s · Streaming trades polls for connection state.

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

Discover, route & survive the network

Prerequisites: 3.3 · Intermediaries & persistent connections 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Trace what happens when someone enters a website URL, from DNS lookup through server execution to browser rendering.

This chapter examines discover, route & survive the network through concrete mechanisms, state snapshots, failure cases, and alternatives.

3.4.1 · NAT, firewalls, ingress, egress

NAT rewrites addressing, firewalls apply traffic policy, ingress enters a boundary, and egress leaves it. These functions can be implemented at different layers. An application timeout can result from an exhausted NAT table or a blocked return path, not only a slow server. Design outbound allowlists and connection lifetimes together. Network segmentation limits reachability but cannot replace per-request authentication and authorization, particularly when several tenants share a service network.

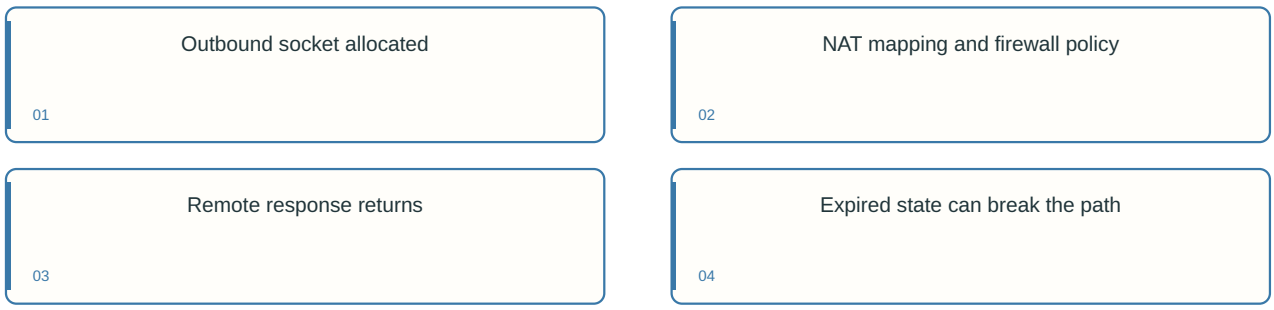


FIG. 3.4.1 **NAT, firewalls, ingress, egress · worked snapshots**

Outbound socket allocated · NAT mapping and firewall policy · Remote response returns · Expired state can break the path. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.2 · Service discovery

Service discovery maps a logical service to reachable instances. It may use DNS, a registry, a platform control plane, or client configuration. Registrations and health information can lag reality. A client must handle stale endpoints and retry safely within a deadline. Cache discovery results to reduce control-plane dependence, but refresh them deliberately; a successful lookup is not proof the selected instance can serve the request or has capacity.

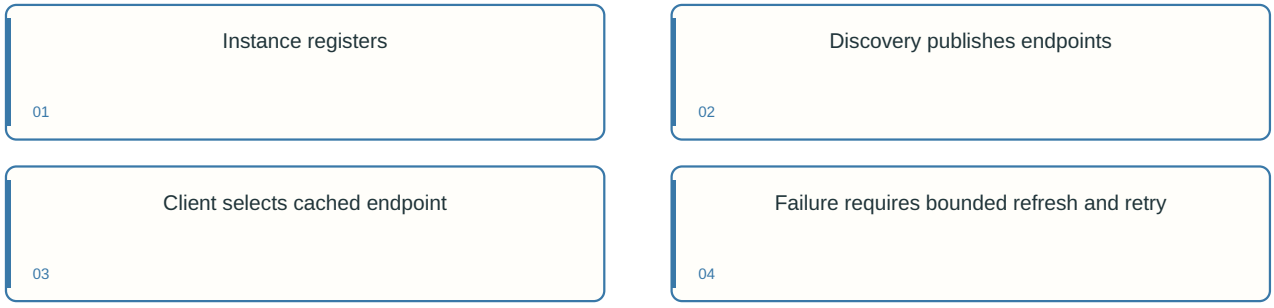


FIG. 3.4.2 **Service discovery · worked snapshots**

Instance registers · Discovery publishes endpoints · Client selects cached endpoint · Failure requires bounded refresh and retry. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.3 · CDN and edge networking

A CDN places cacheable representations near users; the origin remains responsible for authoritative content. Cache keys, freshness, authorization, and purge behavior determine correctness. Public thumbnails may tolerate stale copies, while personalized account pages require careful cache policy and partitioning. Edge delivery reduces some network and origin work but introduces another state copy. A cacheable error or omitted tenant dimension in a cache key can distribute the wrong result very efficiently.

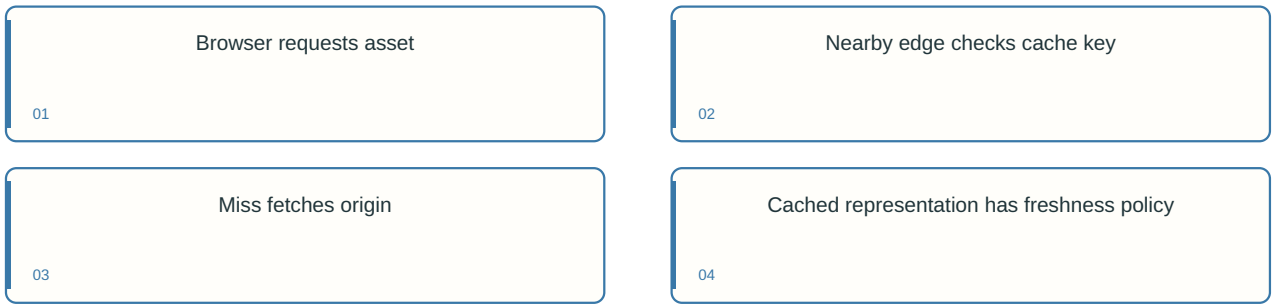


FIG. 3.4.3 **CDN and edge networking · worked snapshots**

Browser requests asset · Nearby edge checks cache key · Miss fetches origin · Cached representation has freshness policy. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.4 · Network partitions, timeouts, packet loss

A timeout means the caller stopped waiting; it does not reveal whether the remote side executed. Packets may be delayed, dropped, reordered, or blocked while other paths remain healthy. Distinguish connection establishment, idle, and overall request deadlines. Retrying a non-idempotent action can duplicate effects; waiting indefinitely can exhaust resources. Treat partial failure as a normal execution history and reconcile ambiguous outcomes using stable operation identity or status queries.

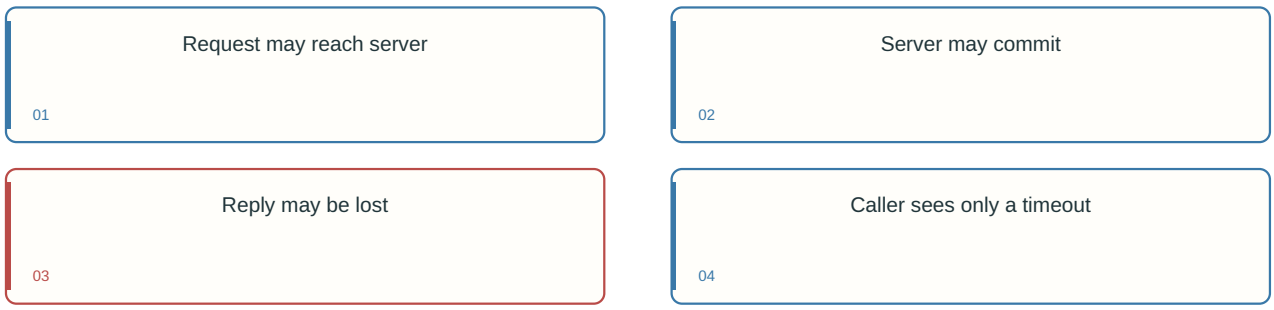


FIG. 3.4.4 **Network partitions, timeouts, packet loss · worked snapshots**

Request may reach server · Server may commit · Reply may be lost · Caller sees only a timeout. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.5 · Connection pooling and keepalive

A connection pool reuses a bounded set of connections. Borrowing can itself queue, so include acquisition in the deadline. A leak or oversized pool can exhaust both application workers and database capacity. Keepalive tests a transport path under particular conditions, not business correctness. In an assumed pool of twenty connections with 50 ms mean hold time, the ideal busy-capacity bound is 400 operations/s before overhead and variance; real safe load requires measurement.

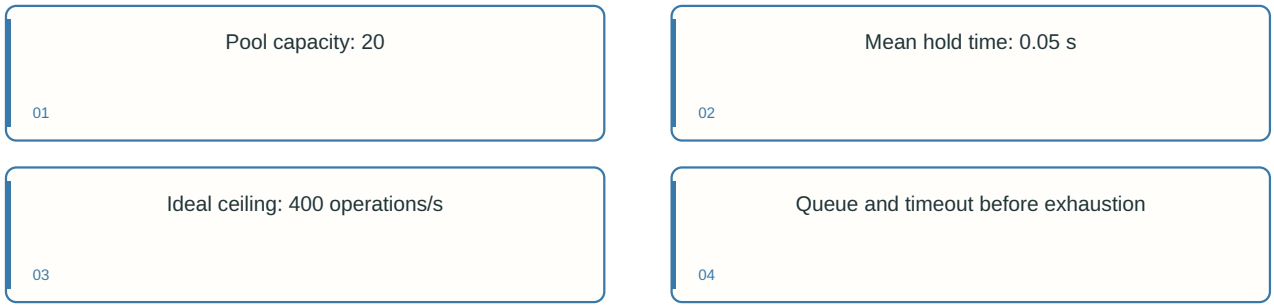


FIG. 3.4.5 **Connection pooling and keepalive · worked snapshots**

Pool capacity: 20 · Mean hold time: 0.05 s · Ideal ceiling: 400 operations/s · Queue and timeout before exhaustion. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · ARITHMETIC MODEL

```
cold = dict(dns=20, tcp=40, tls=40, request_and_work=55)
assert sum(cold.values()) == 155
warm = dict(request_and_work=55)
assert sum(warm.values()) == 55
```

The warm example removes only assumed reusable setup stages. Real protocol negotiation, caches, overlap, connection expiry, and distance change those inputs. This code does not open a socket or benchmark TLS.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · What changes when a supposedly local service call becomes a cross-region network dependency?

Use the stated 20+40+40+55 = 155 ms cold example, then remove only stages actually reused. Do not subtract server execution because the connection is warm. Reject a negative stage time and keep the result labeled an assumed model, not a measured protocol guarantee.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Keepalive tests a transport path under particular conditions, not business correctness.

Carry forward: Pool capacity: 20 · Mean hold time: 0.05 s · Ideal ceiling: 400 operations/s · Queue and timeout before exhaustion.

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   `Access reviewed · 21 October 2026`

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   `Access reviewed · 21 October 2026`

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   `Access reviewed · 21 October 2026`

PART 1 / UNIT 04

Runtime, Processes, Threads, and Concurrency

The running example

Build a service that processes 1,000 file uploads. Compare sequential execution, threads, asynchronous I/O, and worker queues. Identify which architecture helps under which workload.

Unit 4 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

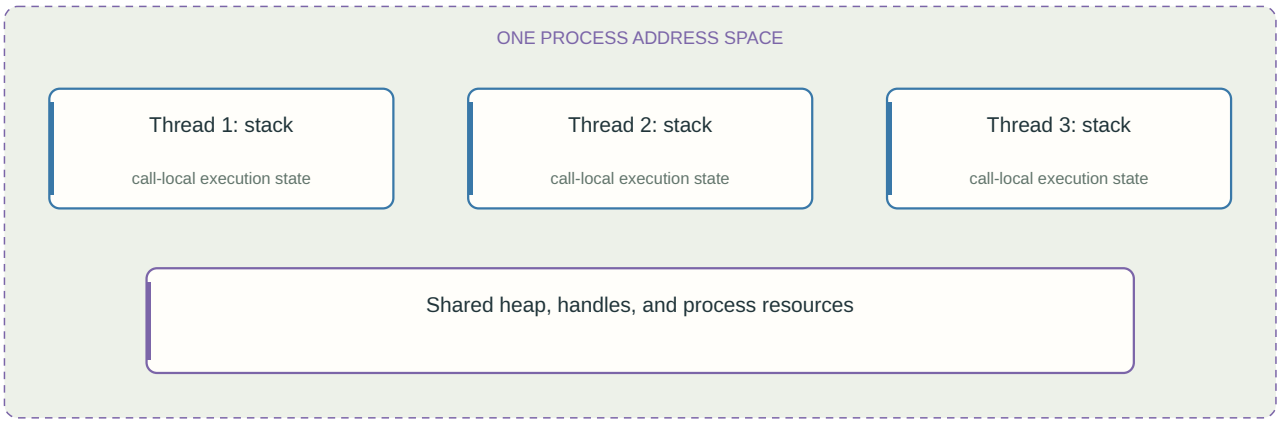
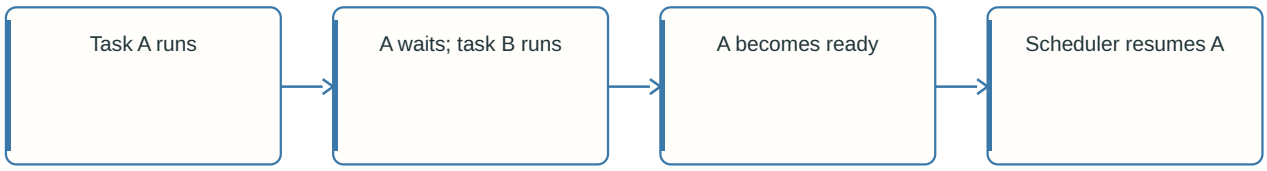


FIG. 4.0.1 **Threads share a heap but have execution-local stacks**

One process contains three thread contexts and a shared heap/resource region. Source-module boundaries do not isolate their failures.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

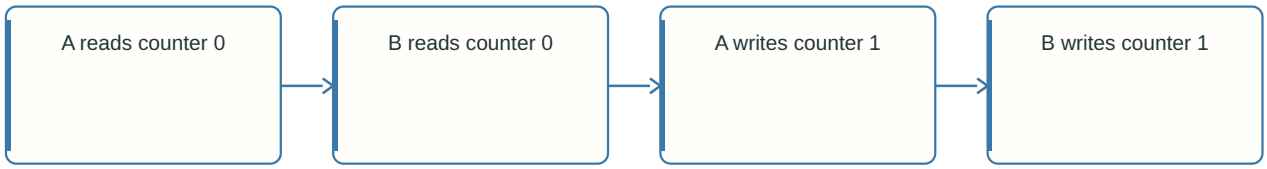


Arrows: ordered stages of this worked mechanism

FIG. 4.0.2 **scheduling timeline**

Concurrency interleaves activities; parallel execution requires actual simultaneous resources.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

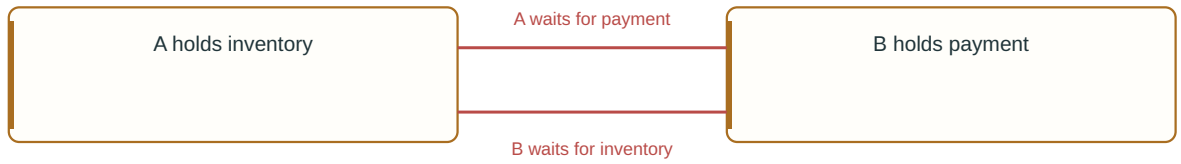


Arrows: ordered stages of this worked mechanism

FIG. 4.0.3 **race-condition interleaving**

Two increments become one without an atomic read-modify-write boundary.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

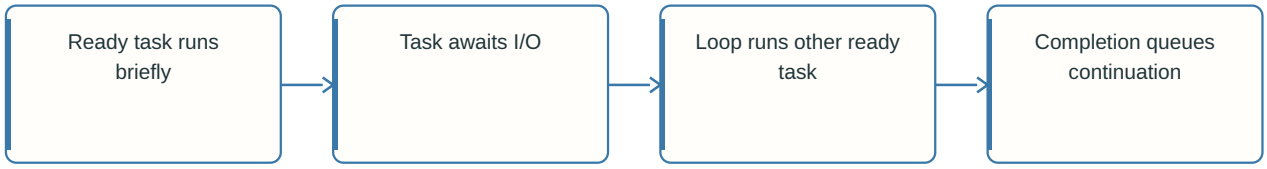


Two opposing wait dependencies form a cycle; neither holder can proceed.

FIG. 4.0.4 **A circular wait prevents both participants progressing**

A holds inventory and waits for payment; B holds payment and waits for inventory. Consistent resource ordering or redesigned ownership can remove this cycle.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 4.0.5 **event-loop flow**

The event loop cannot serve other work during a long blocking callback.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

The runtime owns your resources

Prerequisites: Introduction to System Design 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Build a service that processes 1,000 file uploads. Compare sequential execution, threads, asynchronous I/O, and worker queues. Identify which architecture helps under which workload.

This chapter examines the runtime owns your resources through concrete mechanisms, state snapshots, failure cases, and alternatives.

4.1.1 · Program vs process vs thread

A program is code; a process is an executing instance with an address space and resources; threads are execution contexts within a process. Threads commonly share heap memory while maintaining separate stacks and scheduling state. A process crash can terminate every thread regardless of source-module boundaries. Use processes when isolation or separate resource ownership matters; use threads when shared-address-space execution is appropriate. Neither choice alone makes shared application state race-free.

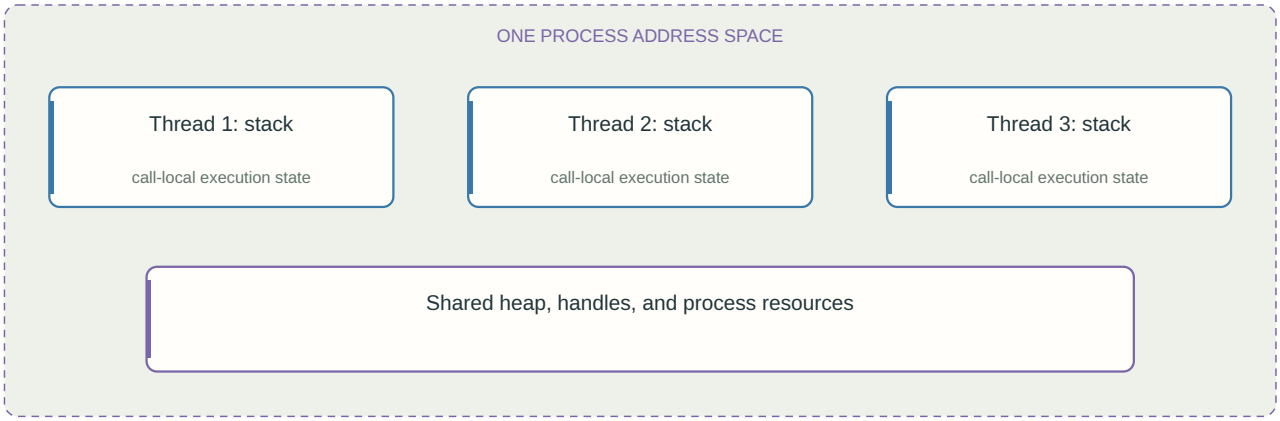


FIG. 4.1.1 **Threads share a heap but have execution-local stacks**

One process contains three thread contexts and a shared heap/resource region. Source-module boundaries do not isolate their failures.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.2 · Memory, stack, heap, and object lifetimes

A stack records call-local execution state; a heap stores dynamically managed objects under the runtime's allocation model. Lifetimes need not match lexical scope, particularly with closures, async tasks, and shared references. An upload buffer retained by an unfinished task can keep hundreds of megabytes live after its initiating request is gone. Explicitly bound ownership and release scarce external resources; garbage collection of memory does not guarantee timely closure of files or connections.

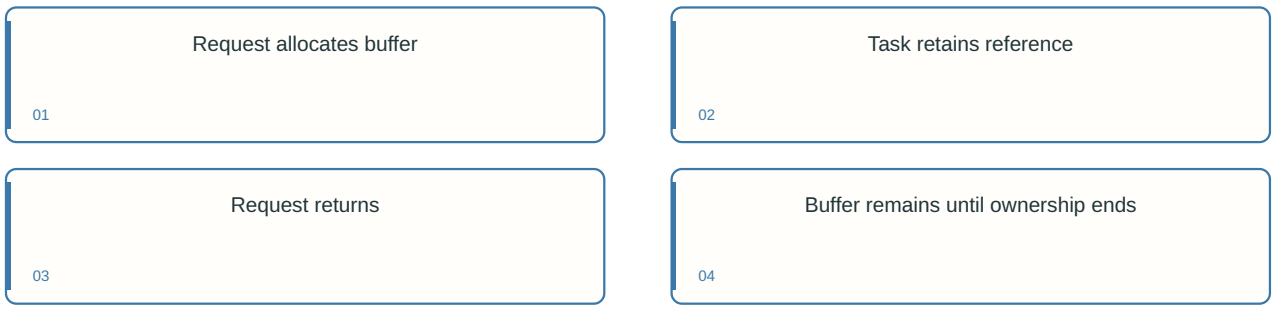


FIG. 4.1.2 **Memory, stack, heap, and object lifetimes · worked snapshots**

Request allocates buffer · Task retains reference · Request returns · Buffer remains until ownership ends. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.3 · Scheduling and context switching

A scheduler decides which runnable execution context receives CPU time. Context switching saves and restores state and can disturb caches. More runnable threads may increase latency without increasing completed work when a fixed CPU is saturated. Blocking I/O can free CPU for another thread, whereas busy waiting consumes it. Investigate run-queue length, CPU utilization, blocking, and contention before selecting a thread count; one worker per request is an unbounded admission policy.

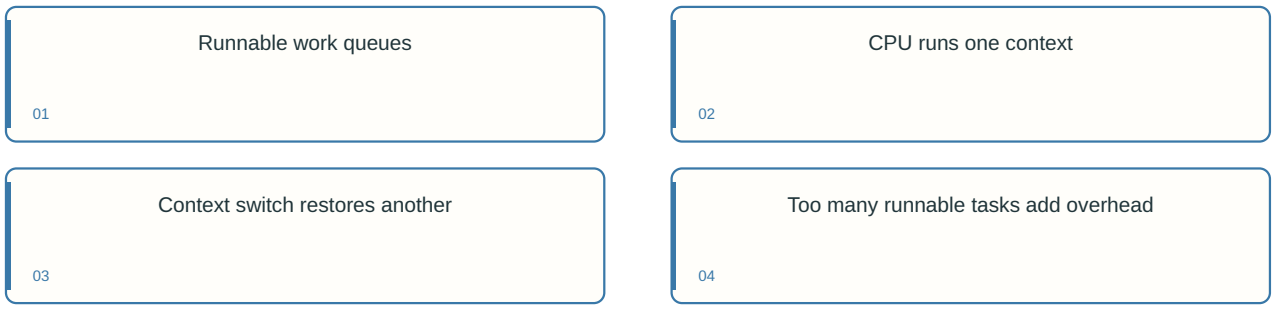


FIG. 4.1.3 **Scheduling and context switching · worked snapshots**

Runnable work queues · CPU runs one context · Context switch restores another · Too many runnable tasks add overhead. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.4 · CPU-bound vs I/O-bound workloads

CPU-bound work spends its limiting time computing; I/O-bound work spends much of it waiting on external operations. For 1000 assumed uploads that each wait 100 ms for storage, sequential waiting takes about 100 seconds before other costs. Fifty concurrent waits have an ideal waiting-time total around two seconds, subject to bandwidth and provider limits. CPU-heavy compression instead needs actual compute parallelism or faster algorithms; asynchronous syntax alone does not create additional CPU capacity.

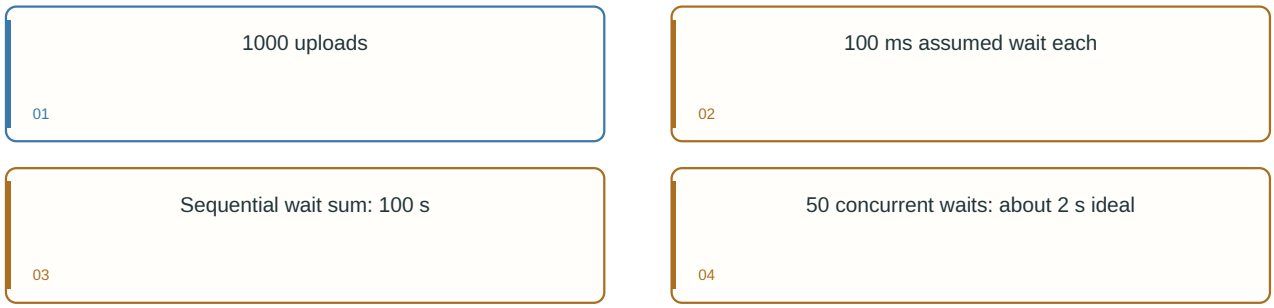


FIG. 4.1.4 **CPU-bound vs I/O-bound workloads · worked snapshots**

1000 uploads · 100 ms assumed wait each · Sequential wait sum: 100 s · 50 concurrent waits: about 2 s ideal. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.5 · Synchronous vs asynchronous execution

Synchronous execution ties the caller's progress to completing an operation; asynchronous execution separates initiation from later completion. Blocking describes whether the executing thread waits; the two distinctions are related but not identical. An async function that calls a blocking filesystem operation can still stall an event-loop thread. Specify the entire path, including library and driver behavior. Prefer the simplest model that keeps deadlines and resource use bounded under the expected workload.

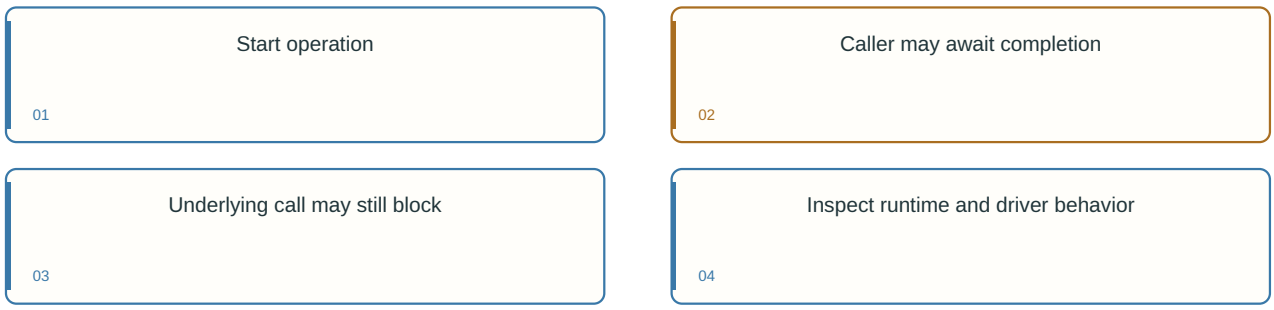


FIG. 4.1.5 **Synchronous vs asynchronous execution · worked snapshots**

Start operation · Caller may await completion · Underlying call may still block · Inspect runtime and driver behavior. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What should happen to accepted uploads when the request process is cancelled but the business still requires processing?

Show A-read-zero, B-read-zero, A-write-one, B-write-one before repairing the critical section. Bound queued payload references and active workers separately. For I/O waiting, compare concurrency; for CPU work, account for runtime parallelism. An async keyword alone does not repair the race or add CPU capacity.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Specify the entire path, including library and driver behavior.

Carry forward: Start operation · Caller may await completion · Underlying call may still block · Inspect runtime and driver behavior.

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Tasks, threads & event loops

Prerequisites: 4.1 · The runtime owns your resources 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Build a service that processes 1,000 file uploads. Compare sequential execution, threads, asynchronous I/O, and worker queues. Identify which architecture helps under which workload.

This chapter examines tasks, threads & event loops through concrete mechanisms, state snapshots, failure cases, and alternatives.

4.2.1 · Blocking vs nonblocking I/O

Nonblocking I/O reports readiness or completion without holding the thread throughout the wait. The event loop resumes a task when progress is possible. This helps large numbers of waiting connections but does not make every operation immediately complete. A loop that repeatedly polls without yielding is still wasteful. Use the platform's readiness or completion mechanism and isolate unavoidable blocking work in a bounded executor, with explicit cancellation and queue limits.

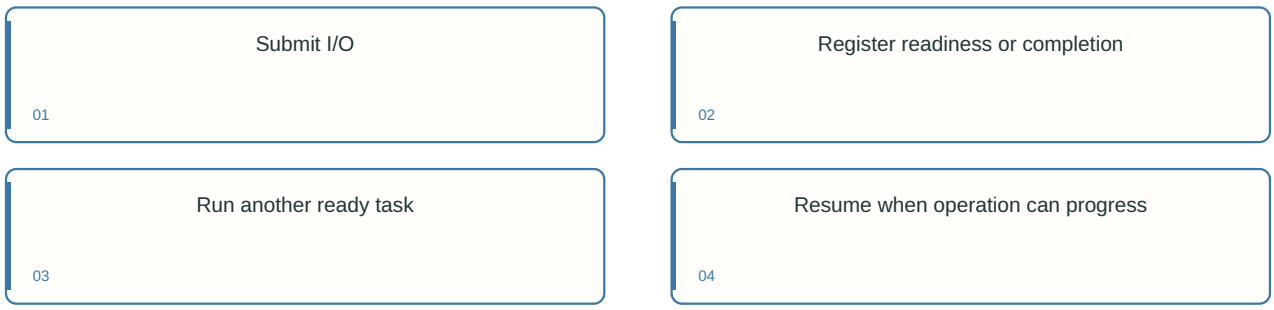


FIG. 4.2.1 **Blocking vs nonblocking I/O · worked snapshots**

Submit I/O · Register readiness or completion · Run another ready task · Resume when operation can progress. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.2 · Concurrency vs parallelism

Concurrency means activities can be in progress over overlapping periods; parallelism means execution actually occurs simultaneously. One core can interleave several tasks without running their instructions at the same instant. Concurrent code still has races because interleaving can occur between read and write operations. Parallelism adds hardware execution opportunities but not automatic correctness. Decide independently how much work may be in progress and how many resources may execute it at once.

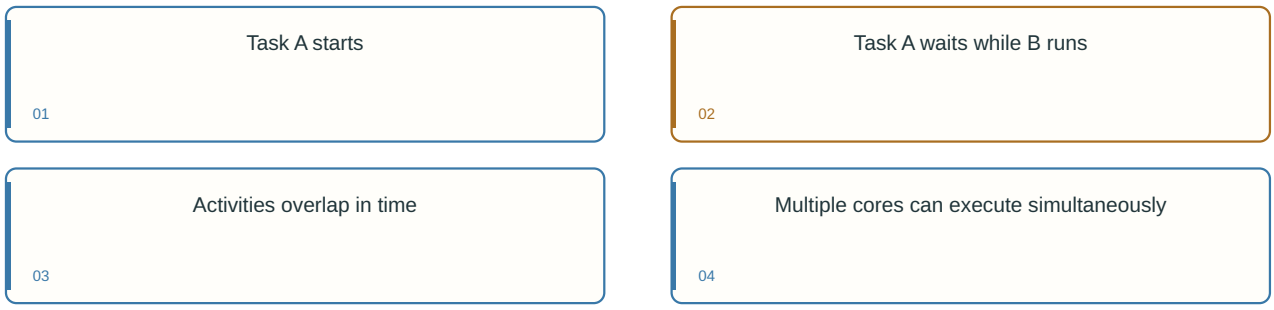


FIG. 4.2.2 **Concurrency vs parallelism · worked snapshots**

Task A starts · Task A waits while B runs · Activities overlap in time · Multiple cores can execute simultaneously. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.3 · Event loops, tasks, futures, promises

An event loop runs ready callbacks or task continuations. A future or promise represents a result that may become available later; awaiting it registers a continuation or suspends a task according to the runtime. A callback that computes for two seconds can delay every other task on a single loop. Keep each turn bounded and make cancellation visible. Futures do not guarantee that work is durable, isolated, or successfully executed merely because it was scheduled.

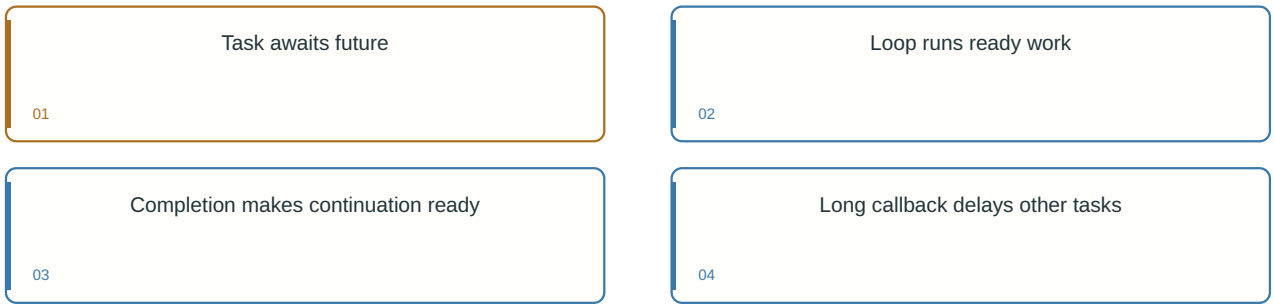


FIG. 4.2.3 **Event loops, tasks, futures, promises · worked snapshots**

Task awaits future · Loop runs ready work · Completion makes continuation ready · Long callback delays other tasks. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.4 · Thread pools and worker pools

A worker pool caps simultaneously executing jobs, while a bounded queue caps waiting jobs. With eight CPU workers and 1000 tasks, accepting every task into memory can still exhaust resources through queued payloads. Keep large upload bytes in object storage and queue compact references. Decide what happens when capacity is full: wait with a deadline, reject, or redirect work. A pool limits execution, not necessarily admission or total memory footprint.

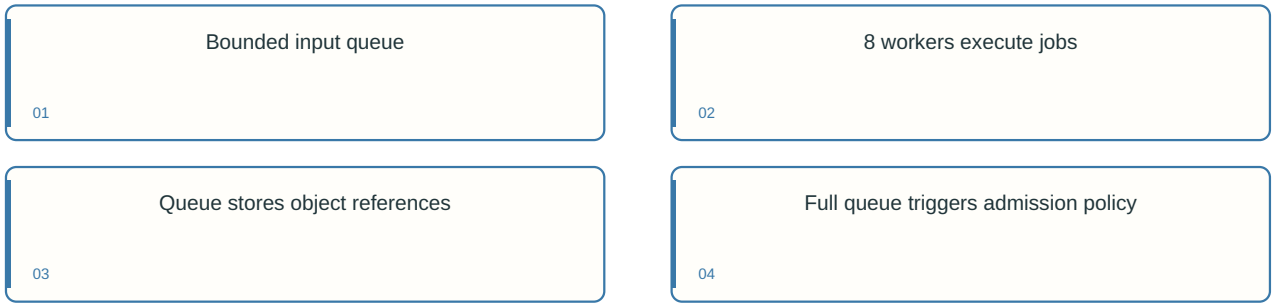


FIG. 4.2.4 **Thread pools and worker pools · worked snapshots**

Bounded input queue · 8 workers execute jobs · Queue stores object references · Full queue triggers admission policy. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What should happen to accepted uploads when the request process is cancelled but the business still requires processing?

Show A-read-zero, B-read-zero, A-write-one, B-write-one before repairing the critical section. Bound queued payload references and active workers separately. For I/O waiting, compare concurrency; for CPU work, account for runtime parallelism. An async keyword alone does not repair the race or add CPU capacity.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Decide what happens when capacity is full: wait with a deadline, reject, or redirect work.

Carry forward: Bounded input queue · 8 workers execute jobs · Queue stores object references · Full queue triggers admission policy.

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Shared memory & coordination

Prerequisites: 4.2 · Tasks, threads & event loops 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Build a service that processes 1,000 file uploads. Compare sequential execution, threads, asynchronous I/O, and worker queues. Identify which architecture helps under which workload.

This chapter examines shared memory & coordination through concrete mechanisms, state snapshots, failure cases, and alternatives.

4.3.1 · Shared memory vs message passing

Shared memory allows direct access but needs synchronization; message passing transfers information between owners and requires protocol decisions. A single owner processing inventory commands sequentially can simplify local invariants. It can still fail, become a bottleneck, or replay a duplicate message after recovery. Passing a pointer to mutable shared state is not the same as transferring ownership. Pick the model based on state ownership and communication cost, not an assertion that one eliminates all concurrency problems.

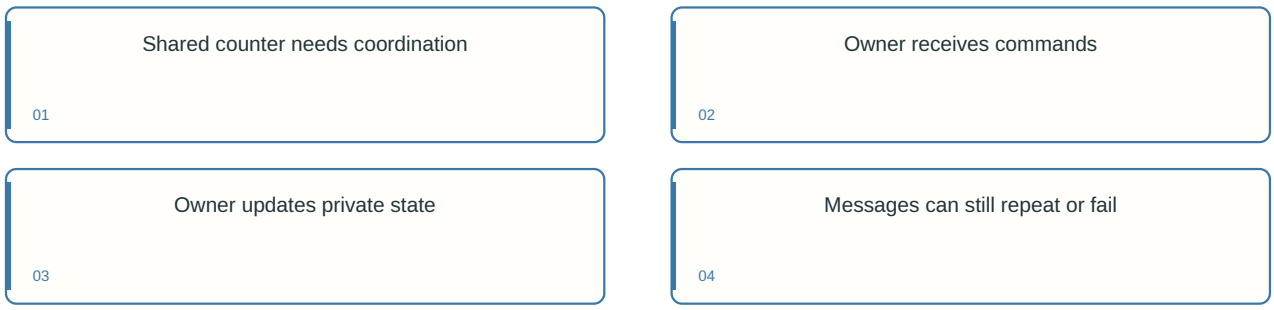


FIG. 4.3.1 **Shared memory vs message passing · worked snapshots**

Shared counter needs coordination · Owner receives commands · Owner updates private state · Messages can still repeat or fail. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.2 · Race conditions and critical sections

A read-modify-write sequence can lose updates when its steps interleave. Starting at zero, A reads zero, B reads zero, A writes one, and B writes one: two increments produced one. Protect the entire invariant, not only the final assignment. A critical section may use a lock or an appropriate atomic operation; separate process or database writers require coordination at their own shared boundary. A local mutex cannot protect another process's memory.

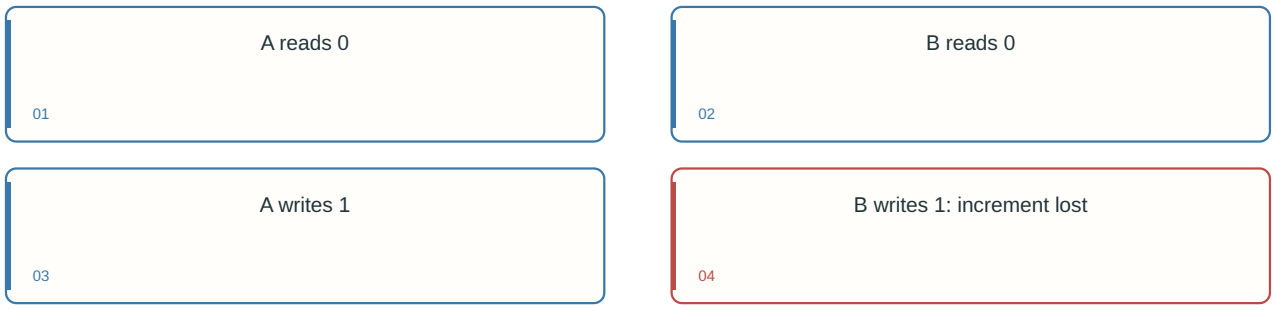


FIG. 4.3.2 **Race conditions and critical sections · worked snapshots**

A reads 0 · B reads 0 · A writes 1 · B writes 1: increment lost. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.3 · Mutexes, semaphores, atomics, condition variables

A mutex grants exclusive access; a semaphore limits concurrent permits; an atomic operation provides defined indivisibility and ordering for specified operations; a condition variable coordinates waiting for a predicate while releasing a lock. Recheck the predicate in a loop because wakeups do not guarantee it holds. Protect related values together if the invariant spans them. Replacing every field with an atomic does not automatically make a multi-field account transfer atomic.

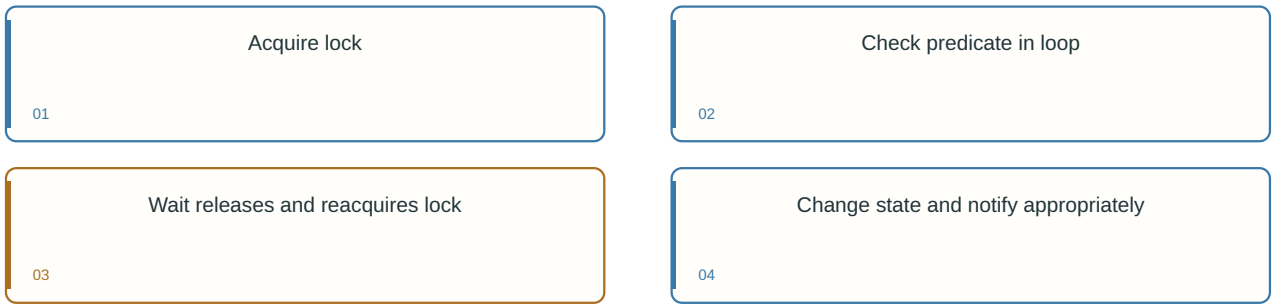


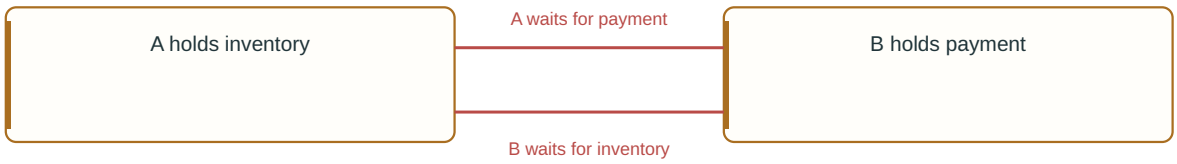
FIG. 4.3.3 **Mutexes, semaphores, atomics, condition variables · worked snapshots**

Acquire lock · Check predicate in loop · Wait releases and reacquires lock · Change state and notify appropriately. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.4 · Deadlocks, starvation, livelocks

Deadlock is a cycle of dependencies that prevents progress; starvation repeatedly denies a participant needed service; livelock keeps participants active without useful progress. A holds inventory then requests payment while B holds payment then requests inventory: neither can proceed. Use consistent lock ordering, narrower critical sections, or redesign ownership; timeouts need rollback and cleanup. A retry loop can convert deadlock into livelock unless it changes coordination or uses bounded randomized delay.



Two opposing wait dependencies form a cycle; neither holder can proceed.

FIG. 4.3.4 **A circular wait prevents both participants progressing**

A holds inventory and waits for payment; B holds payment and waits for inventory. Consistent resource ordering or redesigned ownership can remove this cycle.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What should happen to accepted uploads when the request process is cancelled but the business still requires processing?

Show A-read-zero, B-read-zero, A-write-one, B-write-one before repairing the critical section. Bound queued payload references and active workers separately. For I/O waiting, compare concurrency; for CPU work, account for runtime parallelism. An async keyword alone does not repair the race or add CPU capacity.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Use consistent lock ordering, narrower critical sections, or redesign ownership; timeouts need rollback and cleanup.

Carry forward: A holds inventory lock · B holds payment lock · A waits for payment · B waits for inventory: cycle.

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Visibility, cancellation & overload

Prerequisites: 4.3 · Shared memory & coordination 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Build a service that processes 1,000 file uploads. Compare sequential execution, threads, asynchronous I/O, and worker queues. Identify which architecture helps under which workload.

This chapter examines visibility, cancellation & overload through concrete mechanisms, state snapshots, failure cases, and alternatives.

4.4.1 · Memory visibility and happens-before relationships

Visibility is not established by intuition about elapsed time. A writer's update and another thread's read need the ordering guarantees defined by the language and synchronization primitives. Happens-before is a formal relation used to reason about such guarantees, not a statement that every earlier wall-clock event is visible. Publish an immutable object through a synchronized channel or appropriate atomic reference. The Go memory model is one concrete specification; other languages must be checked separately.

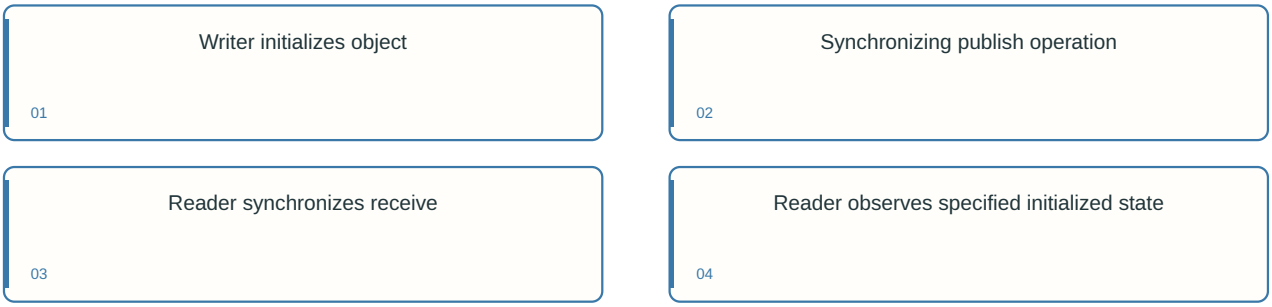


FIG. 4.4.1 **Memory visibility and happens-before relationships · worked snapshots**

Writer initializes object · Synchronizing publish operation · Reader synchronizes receive · Reader observes specified initialized state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.2 · Thread safety and immutability

Thread safety is a behavioral property under a stated concurrent-use contract. Immutable values avoid mutation races after safe publication, but references, initialization, and surrounding resources still need ownership rules. A frozen task record can be shared; a mutable cache mapping task IDs to records may still need synchronization. Avoid returning writable internal collections. Confinement, immutability, locks, and message ownership are complementary tools, with copying and coordination costs that depend on the workload.

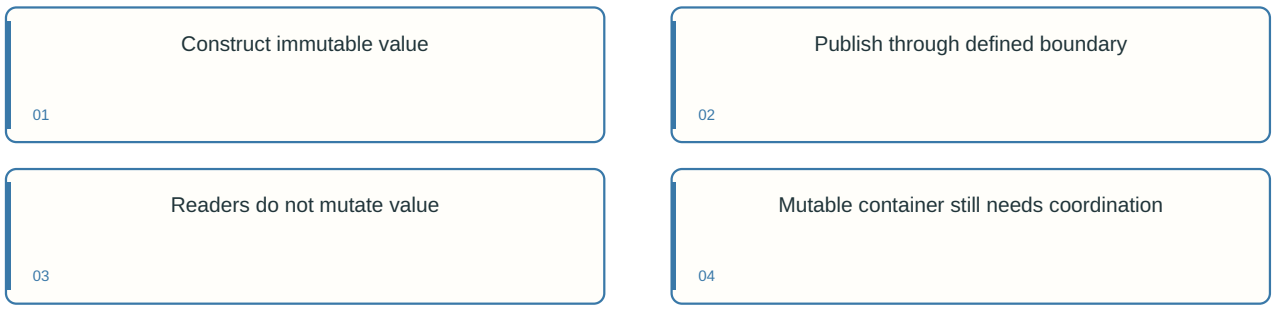


FIG. 4.4.2 **Thread safety and immutability · worked snapshots**

Construct immutable value · Publish through defined boundary · Readers do not mutate value · Mutable container still needs coordination. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.3 · Resource exhaustion and backpressure

Unlimited concurrency converts downstream slowness into memory and connection exhaustion. Backpressure bounds or slows producers according to available capacity. For an assumed worker rate of 100 jobs/s and arrivals of 150 jobs/s, a queue grows by fifty jobs/s until a limit is reached. A larger queue delays rejection but does not fix the rate mismatch. Prefer bounded queues, admission deadlines, compact payloads, and explicit overload responses; monitor queue age as well as length.

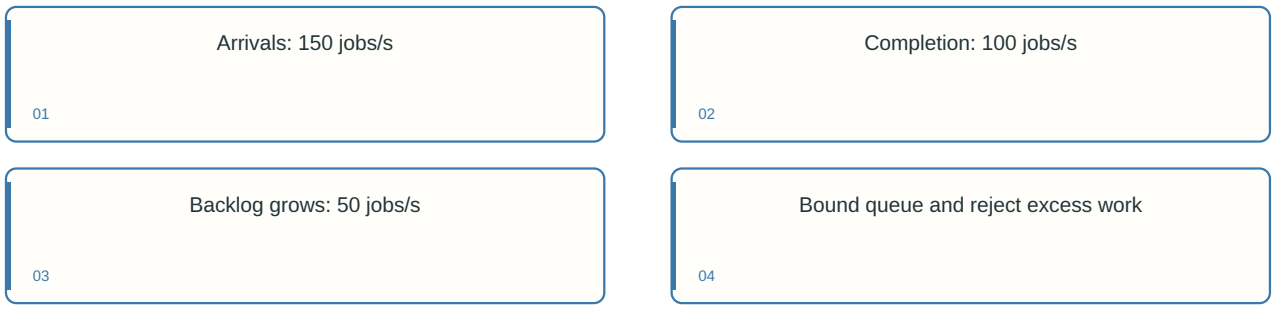


FIG. 4.4.3 **Resource exhaustion and backpressure · worked snapshots**

Arrivals: 150 jobs/s · Completion: 100 jobs/s · Backlog grows: 50 jobs/s · Bound queue and reject excess work. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.4 · Cancellation and structured concurrency

Cancellation must propagate to owned child work and release resources. Structured concurrency ties child-task lifetimes to a parent scope, so failures and completion have an explicit owner. Cancelling a caller does not prove an external side effect was undone: a payment may already have committed. Use finally-style cleanup for local resources and reconciliation for uncertain remote effects. Detached tasks need a durable owner when their business outcome must survive the request process.

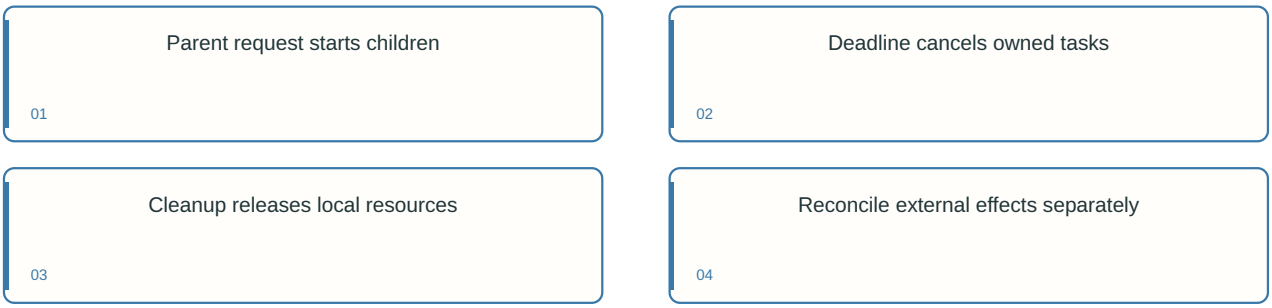


FIG. 4.4.4 **Cancellation and structured concurrency · worked snapshots**

Parent request starts children · Deadline cancels owned tasks · Cleanup releases local resources · Reconcile external effects separately. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PSEUDOCODE · BOUNDED UPLOAD OWNERSHIP

```
queue = bounded_queue(capacity=100)
start_workers(count=50)
for upload in arrivals:
    reference = persist_upload_or_reject(upload)
    queue.put_with_deadline(reference)
worker():
    while reference = queue.take():
        try: process(reference)
        finally: release_owned_resources()
```

The queue bounds waiting references and the worker count bounds active execution. Large bytes remain in storage. The durable job and rejection policy must account for a queue handoff failure; a process-local queue alone is not durable work acceptance.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

A bigger queue cannot repair a permanent rate mismatch

Arrived: **9,000** · completed: **6,000** · waiting: **100** · rejected: **2,900**.

Completed + waiting + rejected equals offered work in this constant-rate model. Increasing the buffer changes waiting versus rejection, not service capacity.

Fluid approximation with constant rates and initially empty waiting queue. The model accounts for completed, waiting, and rejected work; real scheduling, startup, and per-job variance are omitted.

1,000 uploads · compare the resource that dominates

Assume each independent upload needs exactly 100 ms of waiting for storage and no meaningful CPU work. Ignore startup, connection limits, bandwidth, queue overhead, and variance. Sequential execution then needs 100 seconds; allowing 50 independent waits at once has an ideal lower bound of 2 seconds. These are constructed lower bounds, not measurements.

`Same I/O workload, different execution and recovery boundaries`

Execution model	Ideal elapsed time	What must be bounded	Crash recovery
Sequential	100 s	One active upload; backlog elsewhere	Requires a durable record
50 threads with blocking I/O	2 s	Threads, stacks, connections	Memory alone loses accepted work
Async I/O with 50 in flight	2 s	Tasks, buffers, sockets	Memory alone loses accepted work
Durable queue, 50 workers	At least 2 s, plus queue overhead	Admission, queue age, worker count	Durable intent plus retry and idempotent completion

Change the workload: each upload instead requires 20 ms of CPU and negligible I/O. The batch needs 20 CPU-seconds. One executing core therefore needs at least 20 seconds; eight truly parallel cores have an ideal lower bound of 2.5 seconds. An event loop creates no extra cores. Runtime restrictions can also prevent CPU-heavy threads from running in parallel. Measure overhead and contention before choosing threads or processes. A durable queue changes acknowledgment and recovery behavior; it does not erase service demand.

▼ Architecture review · What should happen to accepted uploads when the request process is cancelled but the business still requires processing?

Show A-read-zero, B-read-zero, A-write-one, B-write-one before repairing the critical section. Bound queued payload references and active workers separately. For I/O waiting, compare concurrency; for CPU work, account for runtime parallelism. An async keyword alone does not repair the race or add CPU capacity.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Use finally-style cleanup for local resources and reconciliation for uncertain remote effects.

Carry forward: Parent request starts children · Deadline cancels owned tasks · Cleanup releases local resources · Reconcile external effects separately.

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

PART 2 / UNIT 05

Object-Oriented and Modular Software Design

The running example

Start with a poorly designed payment-processing class. Refactor it into appropriate components, showing which problems each design improvement solves.

Unit 5 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Responsibility	Before	After
Validation	Payment god class	Task/domain policy
Provider calls	Hardcoded SDK creation	Injected gateway adapter
Persistence	Mixed with rendering	Owned repository contract

FIG. 5.0.1 **Before/after class diagrams**

Refactoring moves knowledge while preserving approved, declined, and uncertain outcomes.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

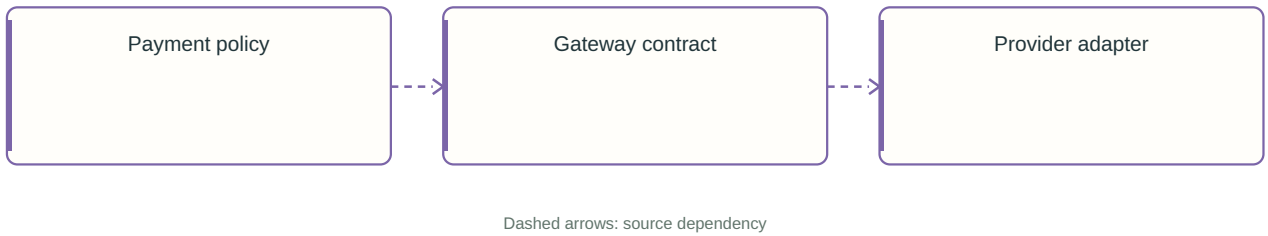


FIG. 5.0.2 **dependency graph**

Dashed arrows denote source dependencies around an application-owned contract.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Variation	Inheritance approach	Composition approach
Channel × content	Subclass combinations	Content receives transport
Behavioral contract	Subclass must substitute	Collaborator contract explicit
Cost	Hierarchy coupling	Wiring and interfaces

FIG. 5.0.3 **inheritance vs composition**

Composition can separate independent variation; inheritance is appropriate only with valid behavioral substitution.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

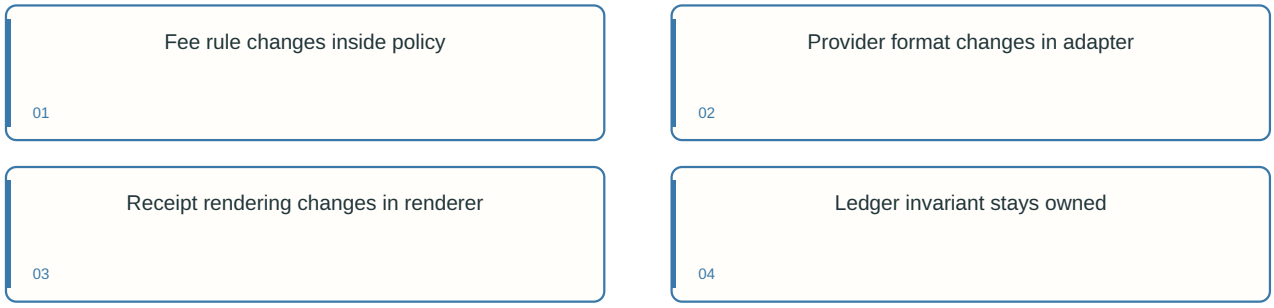


FIG. 5.0.4 **cohesion/coupling illustrations**

Cohesion and coupling are evaluated through concrete change propagation, not class-count metrics.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Model behavior with objects and values

Prerequisites: Introduction to System Design 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Start with a poorly designed payment-processing class. Refactor it into appropriate components, showing which problems each design improvement solves.

This chapter examines model behavior with objects and values through concrete mechanisms, state snapshots, failure cases, and alternatives.

5.1.1 · Abstraction, encapsulation, polymorphism, inheritance

A payment class that validates an order, creates a provider SDK, calculates fees, and writes audit records hides several reasons to change. Abstraction exposes charge(command); encapsulation protects payment state; polymorphism permits different implementations of a behavior. Inheritance is one mechanism for sharing and specializing behavior, not a requirement for these benefits. Extract provider communication behind a contract while keeping the amount and allowed state transitions under application control.

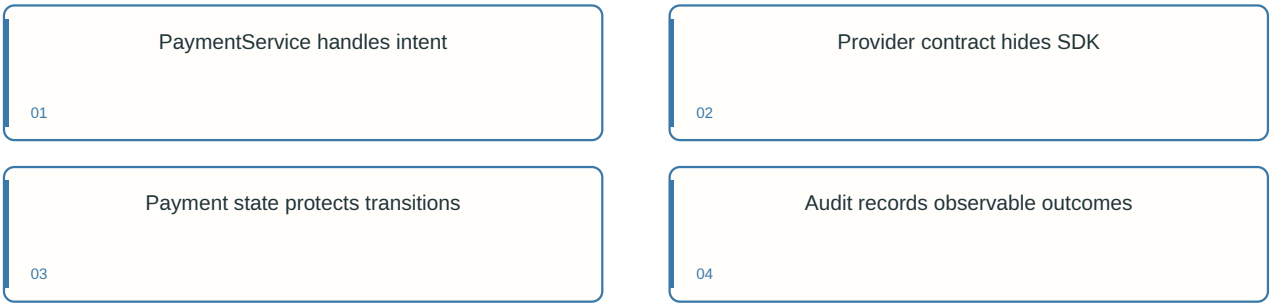


FIG. 5.1.1 **Abstraction, encapsulation, polymorphism, inheritance · worked snapshots**

PaymentService handles intent · Provider contract hides SDK · Payment state protects transitions · Audit records observable outcomes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.2 · Composition vs inheritance

A subclass can violate assumptions inherited by callers: a ReadOnlyAccount subclass that throws from withdraw is not substitutable for a contract promising withdrawal. Composition combines collaborators through explicit contracts instead. A payment service receiving a fraud policy and a gateway can vary them independently without a subclass for every combination. Composition introduces wiring and interface design; inheritance remains useful when the behavioral relationship is genuinely stable, rather than merely when two classes share fields.

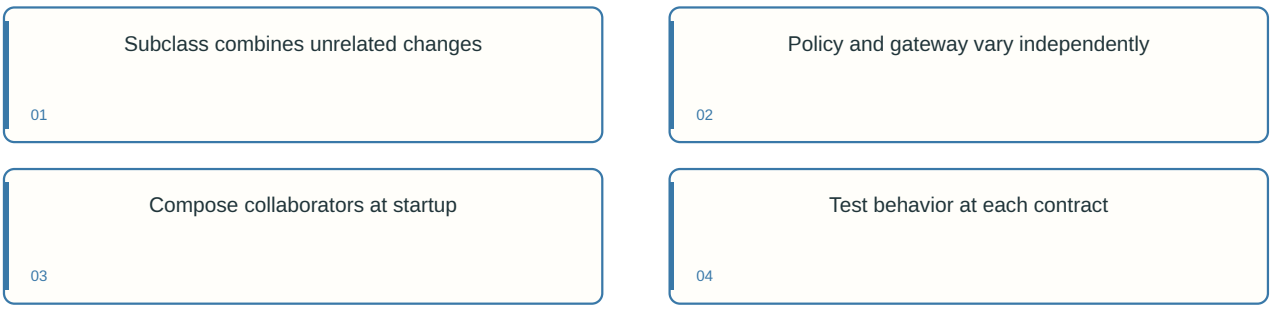


FIG. 5.1.2 **Composition vs inheritance · worked snapshots**

Subclass combines unrelated changes · Policy and gateway vary independently · Compose collaborators at startup · Test behavior at each contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.3 · Interfaces and contracts

A gateway interface must specify currency, amount units, timeouts, duplicate identity, and error meaning. charge(100) is ambiguous: one currency unit, one hundred, or one hundred minor units? Return a structured outcome such as approved, declined, or uncertain with a provider reference. A transport exception after submission is not proof of decline. Behavioral contracts let adapters be replaced without silently changing business semantics, but require integration tests against real provider behavior.

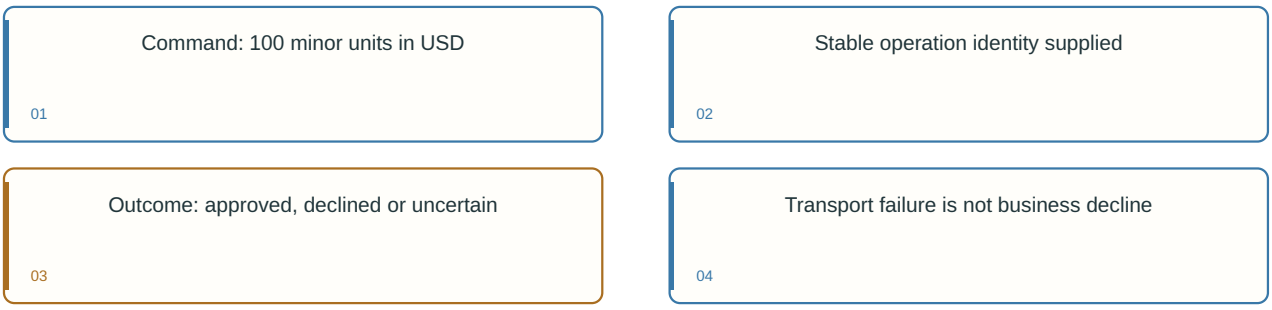


FIG. 5.1.3 **Interfaces and contracts · worked snapshots**

Command: 100 minor units in USD · Stable operation identity supplied · Outcome: approved, declined or uncertain · Transport failure is not business decline. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.4 · High cohesion and low coupling

Cohesion improves when related payment rules live together; coupling falls when business code does not depend on provider-specific field names. Measure the effect using a change: adding an SMS receipt should not alter ledger arithmetic. Several focused modules may still share one deployment. Excessive fragmentation can make a simple path harder to follow, so extract a responsibility when it owns an invariant, hides unstable knowledge, or supports a concrete change independently.

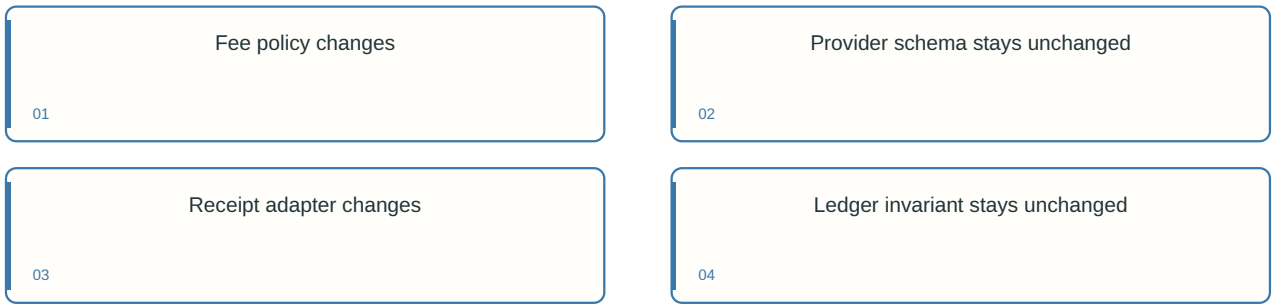


FIG. 5.1.4 **High cohesion and low coupling · worked snapshots**

Fee policy changes · Provider schema stays unchanged · Receipt adapter changes · Ledger invariant stays unchanged. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.5 · SOLID principles, including their limitations

SOLID is a set of design heuristics: single responsibility, open/closed extension, behavioral substitution, focused interfaces, and dependency inversion. A payment service can depend on a narrow charge contract while its adapter depends on an SDK. These ideas do not require one interface per class or forbid editing existing code. An extension mechanism with twenty configuration switches may be harder to understand than a deliberate edit. Judge each heuristic against the actual change and testing cost.

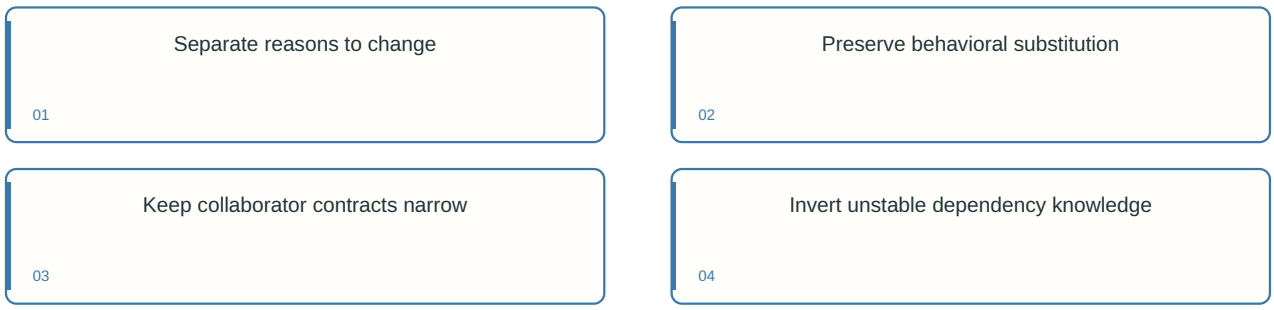


FIG. 5.1.5 **SOLID principles, including their limitations · worked snapshots**

Separate reasons to change · Preserve behavioral substitution · Keep collaborator contracts narrow · Invert unstable dependency knowledge. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which payment abstractions remain useful if the implementation moves from objects to pure functions?

Test approved, declined, and uncertain outcomes without real charges. Keep integer units and currency explicit. A test fake validates orchestration only; real serialization, timeouts, and persistence need separate integration evidence. Do not map a timeout to decline simply to make a test pass.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

An extension mechanism with twenty configuration switches may be harder to understand than a deliberate edit.

Carry forward: Separate reasons to change · Preserve behavioral substitution · Keep collaborator contracts narrow · Invert unstable dependency knowledge.

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

Principles under pressure

Prerequisites: 5.1 · Model behavior with objects and values 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Start with a poorly designed payment-processing class. Refactor it into appropriate components, showing which problems each design improvement solves.

This chapter examines principles under pressure through concrete mechanisms, state snapshots, failure cases, and alternatives.

5.2.1 · DRY, KISS, YAGNI

DRY avoids duplicated knowledge, not every repeated line. KISS prefers a design understandable for the required behavior; YAGNI defers speculative capability. Two similar validations may represent different policies and should not be merged solely for visual similarity. Conversely, copying the same currency conversion rule into every gateway creates a real consistency risk. Share a stable business concept after identifying its owner, and keep a reversible simple implementation until evidence justifies a larger abstraction.

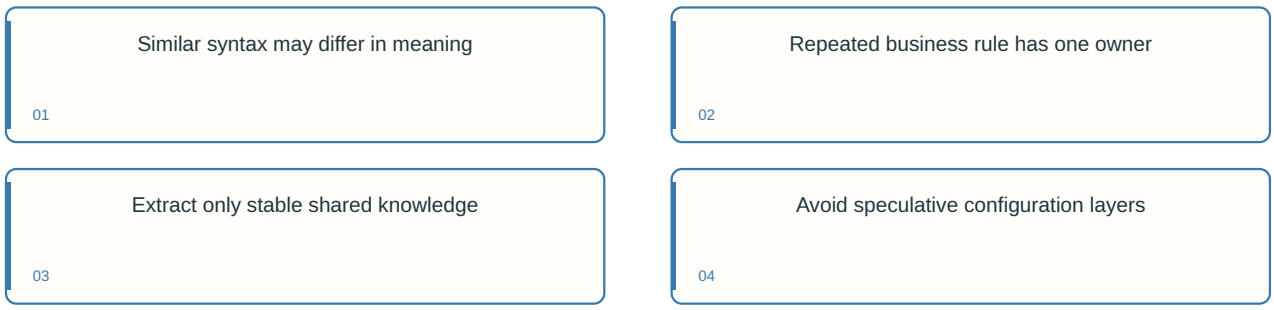


FIG. 5.2.1 **DRY, KISS, YAGNI · worked snapshots**

Similar syntax may differ in meaning · Repeated business rule has one owner · Extract only stable shared knowledge · Avoid speculative configuration layers. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.2 · Law of Demeter

The Law of Demeter discourages reaching deeply through another object's internal graph. order.customer.account.settings.currency exposes representation and creates change propagation. Ask an appropriate owner for the information or behavior needed, such as order.paymentCurrency(). It is a heuristic, not a ban on every fluent API or data traversal. Adding dozens of meaningless forwarding methods can obscure the model; the objective is to reduce knowledge of unstable structure, not count dots in source code.

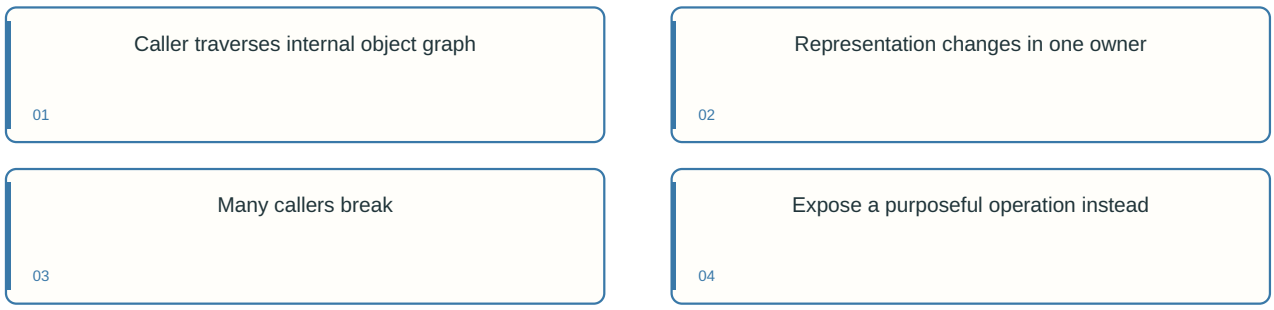


FIG. 5.2.2 **Law of Demeter · worked snapshots**

Caller traverses internal object graph · Representation changes in one owner · Many callers break · Expose a purposeful operation instead. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.3 · Tell, Don't Ask

Tell, Don't Ask encourages sending an intention to the state owner. Rather than reading account.balance, comparing it outside, and writing it back, request reserve(amount) so the account can enforce its invariant. Reading a value for a report remains appropriate. The principle does not authorize hiding surprising side effects behind getters. The owner must still coordinate concurrent changes at the true shared-state boundary, such as an atomic database transaction.

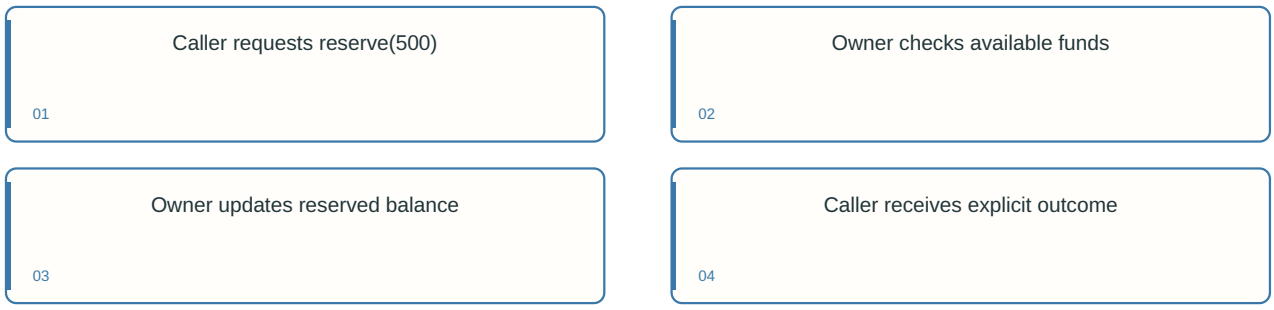


FIG. 5.2.3 **Tell, Don't Ask · worked snapshots**

Caller requests reserve(500) · Owner checks available funds · Owner updates reserved balance · Caller receives explicit outcome. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.4 · Command-query separation

Command-query separation distinguishes operations that change state from queries that report it. A payment status getter should not unexpectedly charge a card. Some practical commands return useful results, such as the created payment identifier, without violating the underlying intent of clear side effects. Document these conventions. Separating read and write models into CQRS is a larger architectural choice; ordinary command-query separation does not require separate services, databases, or event sourcing.

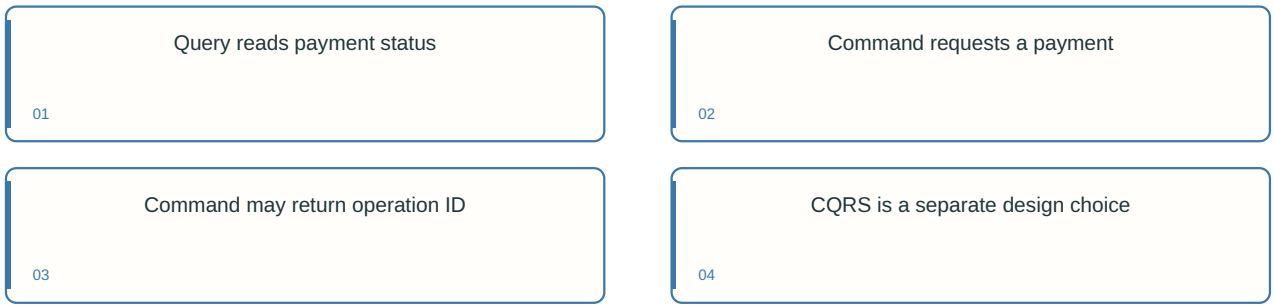


FIG. 5.2.4 **Command-query separation · worked snapshots**

Query reads payment status · Command requests a payment · Command may return operation ID · CQRS is a separate design choice. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which payment abstractions remain useful if the implementation moves from objects to pure functions?

Test approved, declined, and uncertain outcomes without real charges. Keep integer units and currency explicit. A test fake validates orchestration only; real serialization, timeouts, and persistence need separate integration evidence. Do not map a timeout to decline simply to make a test pass.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Document these conventions.

Carry forward: Query reads payment status · Command requests a payment · Command may return operation ID · CQRS is a separate design choice.

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

Side effects, domain models & collaboration

Prerequisites: 5.2 · Principles under pressure 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Start with a poorly designed payment-processing class. Refactor it into appropriate components, showing which problems each design improvement solves.

This chapter examines side effects, domain models & collaboration through concrete mechanisms, state snapshots, failure cases, and alternatives.

5.3.1 · Immutability and side-effect management

Immutable values make state transitions easier to inspect: compute a new PaymentSummary from an old value rather than mutating a shared object in several helpers. Keep effectful operations at explicit boundaries and pass their results into pure calculations. Immutability can cost allocation and copying, and cannot undo a provider charge. An immutable command still needs idempotency and authorization when executed; local referential transparency is not an end-to-end transaction guarantee.

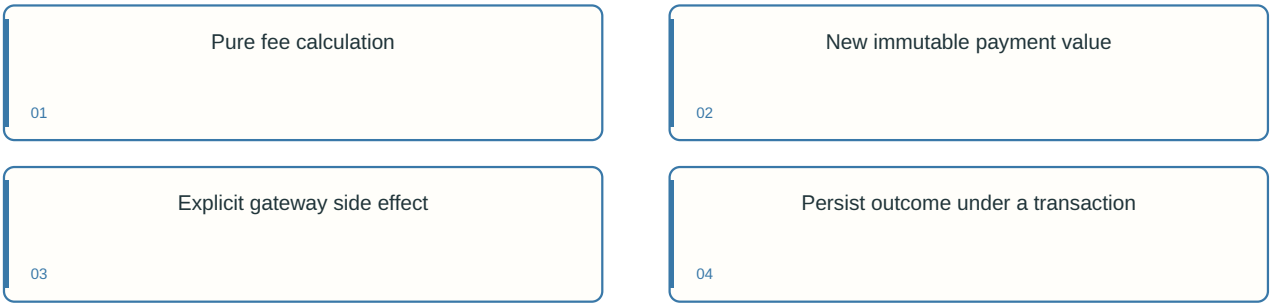


FIG. 5.3.1 **Immutability and side-effect management · worked snapshots**

Pure fee calculation · New immutable payment value · Explicit gateway side effect · Persist outcome under a transaction. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.2 · Domain models, entities, value objects

An entity has identity across changes; a value object is defined by its value; a domain model expresses meaningful rules. Payment 42 remains the same entity after status changes. Money(USD,100) is a value that should reject incompatible arithmetic or apply an explicit conversion policy. A domain service can coordinate a rule that belongs to no single entity. Avoid an anemic model that scatters every invariant across controllers, while keeping trivial data transport simple.

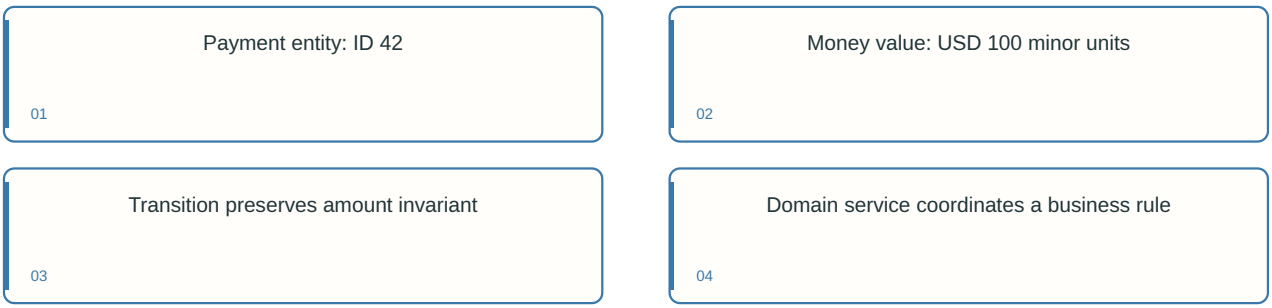


FIG. 5.3.2 **Domain models, entities, value objects · worked snapshots**

Payment entity: ID 42 · Money value: USD 100 minor units · Transition preserves amount invariant · Domain service coordinates a business rule. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.3 · Object collaboration

An object collaboration describes messages and ownership, not just fields. The checkout orchestrator asks inventory to reserve, asks payments to authorize, and records order state. Neither inventory nor payments should silently mutate the other's private records. Make results explicit so partial failure can be handled. Too much conversational object traffic can become a distributed performance problem if moved across network boundaries; a good in-process collaboration is not automatically a good remote API.

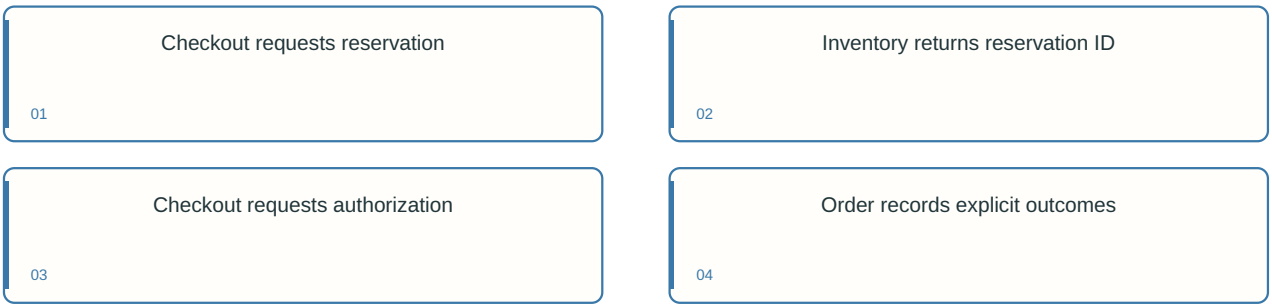


FIG. 5.3.3 **Object collaboration · worked snapshots**

Checkout requests reservation · Inventory returns reservation ID · Checkout requests authorization · Order records explicit outcomes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.4 · Testability and design for change

Testability follows controllable inputs, visible outcomes, and explicit effects. Inject a fake gateway to test declined-payment behavior without network calls, then test the real adapter against a contract environment. A mock expectation that asserts every internal method call can make refactoring expensive while proving little business behavior. Prefer invariant and outcome tests, and retain integration coverage for transactions, serialization, timeouts, and resource cleanup that a fake cannot faithfully establish.

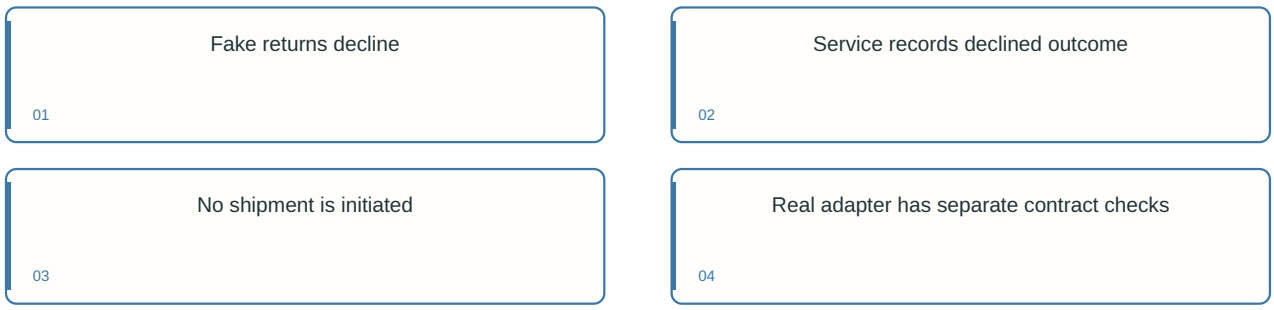


FIG. 5.3.4 **Testability and design for change · worked snapshots**

Fake returns decline · Service records declined outcome · No shipment is initiated · Real adapter has separate contract checks. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which payment abstractions remain useful if the implementation moves from objects to pure functions?

Test approved, declined, and uncertain outcomes without real charges. Keep integer units and currency explicit. A test fake validates orchestration only; real serialization, timeouts, and persistence need separate integration evidence. Do not map a timeout to decline simply to make a test pass.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

> CHECK THE BOUNDARY
>
> A mock expectation that asserts every internal method call can make refactoring expensive while proving little business behavior.

Carry forward: Fake returns decline · Service records declined outcome · No shipment is initiated · Real adapter has separate contract checks.

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

Design for change, testing & functions

Prerequisites: 5.3 · Side effects, domain models & collaboration 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Start with a poorly designed payment-processing class. Refactor it into appropriate components, showing which problems each design improvement solves.

This chapter examines design for change, testing & functions through concrete mechanisms, state snapshots, failure cases, and alternatives.

5.4.1 · Refactoring and recognizing code smells

Refactoring preserves intended behavior while improving structure. Start by characterizing the tangled payment class: valid charge, decline, provider timeout, and persistence failure. Then extract parsing, fee calculation, gateway calls, and storage incrementally. A long method, duplicated conditional, or feature envy is evidence to investigate, not proof that a design pattern is required. Keep tests focused on observable obligations so moving code does not change payment semantics accidentally.

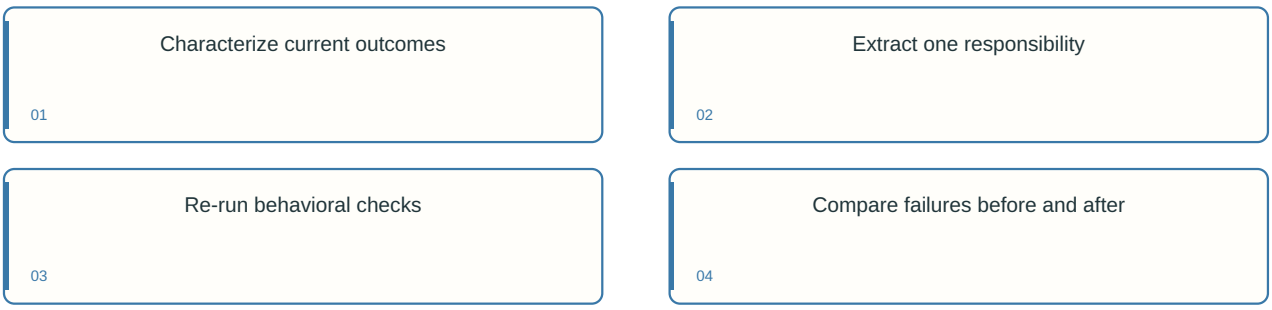


FIG. 5.4.1 **Refactoring and recognizing code smells · worked snapshots**

Characterize current outcomes · Extract one responsibility · Re-run behavioral checks · Compare failures before and after. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

5.4.2 · Functional programming alternatives to object-centric design

A functional design can represent payment decisions as pure transformations of values, with effects interpreted at the boundary. decide(state, command) may yield events and requested external actions; an interpreter commits events or calls a gateway. This reduces hidden mutation and makes replaying logic easier. It still needs ownership, durable execution, and effect deduplication. Objects and functions are complementary tools; choose the representation that makes invariants and collaboration clearest in the language and team context.

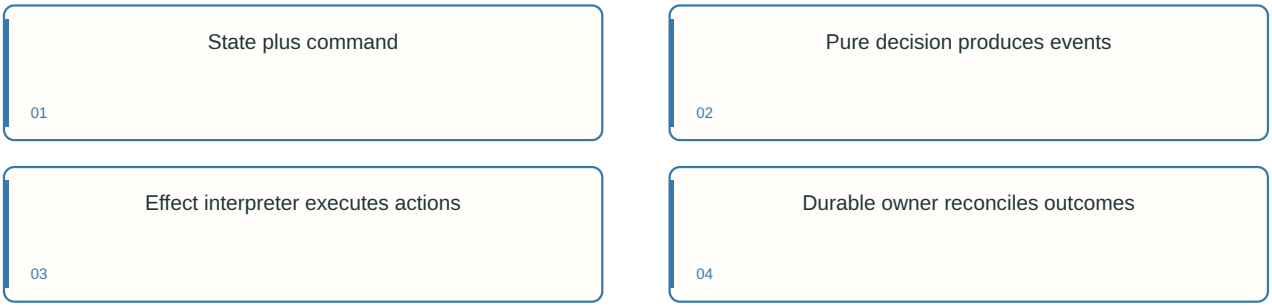
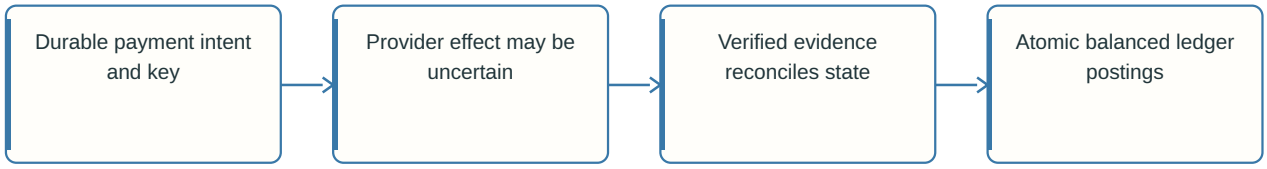


FIG. 5.4.2 **Functional programming alternatives to object-centric design · worked snapshots**

State plus command · Pure decision produces events · Effect interpreter executes actions · Durable owner reconciles outcomes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 5.4.3 **The running system · ordered responsibility**

Commit intent and operation identity, dispatch a stable provider key, then record authoritative response or uncertainty. Apply settlement or capture evidence through an idempotent transition that creates balanced ledger postings once. Verified callbacks can arrive late or before a synchronous response and must not regress state incorrectly. Reconciliation queries stable provider references and repairs discrepancies without blindly resubmitting a charge.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

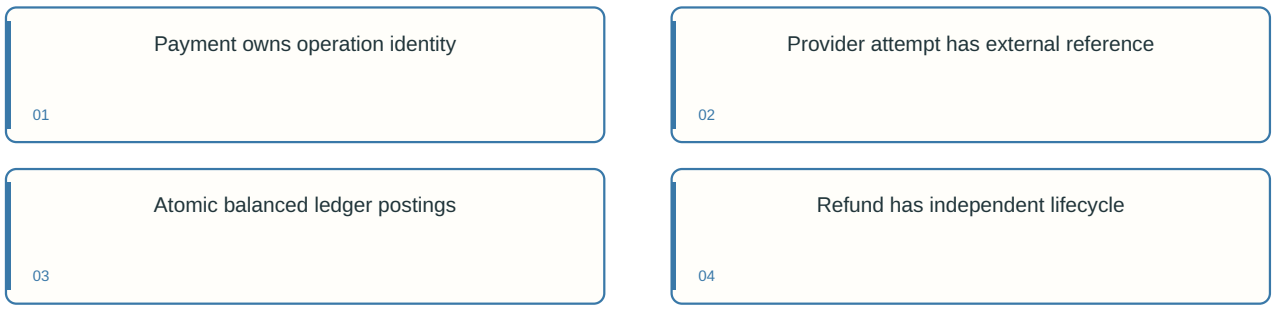


FIG. 5.4.4 **The running system · invariant and recovery review**

Review which event means financial success, who owns the ledger invariant, and what recovery does after deduplication history is restored behind provider effects. Test charge-then-crash, callback-before-response, duplicate settlement, partial refund, and restore from an older snapshot. Verify no duplicate local postings and no unsupported claim that an unknown provider outcome is safely declined.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · INJECTED COLLABORATOR

```
class RecordingGateway:
    def __init__(self): self.calls = []
    def charge(self, command):
        self.calls.append(command)
        return "uncertain"

def pay(command, gateway):
    return gateway.charge(command)

gateway = RecordingGateway()
assert pay({"amount_minor":100,"currency":"USD"}, gateway) == "uncertain"
```

The collaborator is injected and its uncertain result remains explicit. A production service also validates the money and owns persistence and reconciliation; this small function isolates the dependency, not the entire payment system.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · Which payment abstractions remain useful if the implementation moves from objects to pure functions?

Test approved, declined, and uncertain outcomes without real charges. Keep integer units and currency explicit. A test fake validates orchestration only; real serialization, timeouts, and persistence need separate integration evidence. Do not map a timeout to decline simply to make a test pass.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

> CHECK THE BOUNDARY
>
> It still needs ownership, durable execution, and effect deduplication.

Carry forward: State plus command · Pure decision produces events · Effect interpreter executes actions · Durable owner reconciles outcomes.

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

PART 2 / UNIT 06

Design Patterns, Dependency Injection, and Object Lifecycles

The running example

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

Unit 6 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

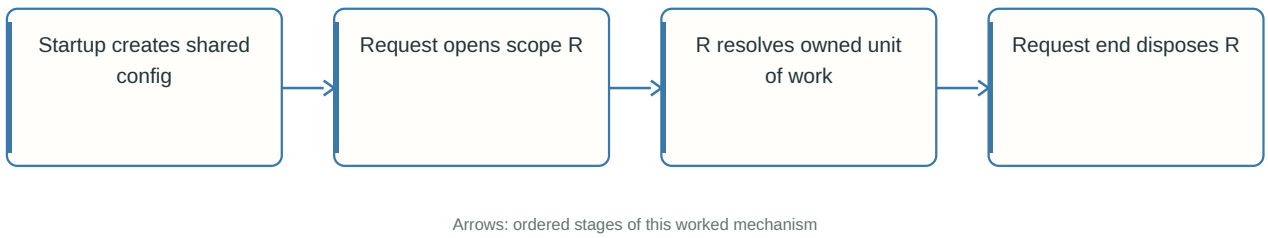


FIG. 6.0.1 **Object creation timelines**

Construction, borrowing, and disposal form a timeline; background jobs need independent scopes.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Resolution	Request scope 1	Request scope 2
Scoped: first / second	A / A	B / B
Transient: first / second	C / D	E / F
Singleton in this process	S / S	S / S

FIG. 6.0.2 **Scope identities make lifetime semantics visible**

These are identity observations in one application. Framework registration and disposal rules must still be checked.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

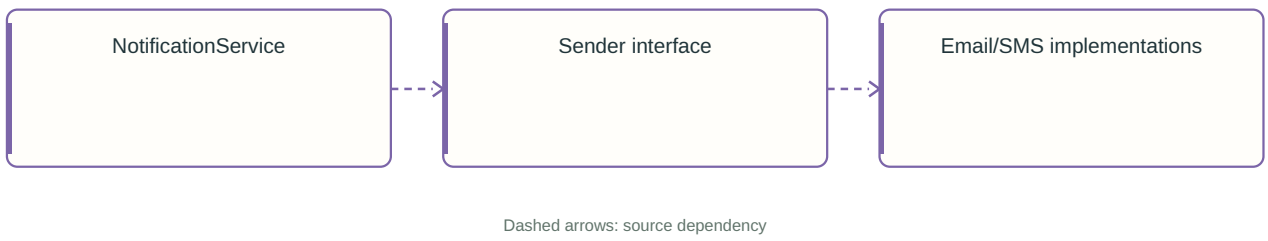


FIG. 6.0.3 **UML class relationships**

Roles are source-level class/interface relationships, not separate deployed services.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Question	Before inversion	After inversion
High-level dependency	Concrete provider SDK	Application-owned contract
Creation	Inside service	Composition root
Testing	Real effect coupled	Controlled collaborator

FIG. 6.0.4 **dependency inversion before/after**

Injection and inversion are different: supplying a concrete SDK alone may retain its semantic coupling.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Compare	Principal intent	Extra responsibility
Adapter / Decorator	Translate / add behavior	Preserve outcome meaning
Factory / DI	Create / supply	Define returned lifetime
Strategy / State	Algorithm / lifecycle	Validate allowed state change

FIG. 6.0.5 **pattern comparisons**

Pattern intent and contract matter more than all three being implemented as wrappers or interfaces.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Dependencies, inversion & composition

Prerequisites: Object-Oriented and Modular Software Design 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

This chapter examines dependencies, inversion & composition through concrete mechanisms, state snapshots, failure cases, and alternatives.

6.1.1 · Dependency, dependency direction, and inversion

A notification service that imports a concrete email SDK knows both business behavior and infrastructure details. Dependency direction describes that source-level knowledge; inversion makes high-level policy depend on an abstract sending contract that adapters implement. Runtime calls still reach a concrete implementation. Merely moving the SDK constructor into a helper does not invert the dependency if the service still consumes provider-specific types and exceptions. Protect the semantic contract as well as the import graph.

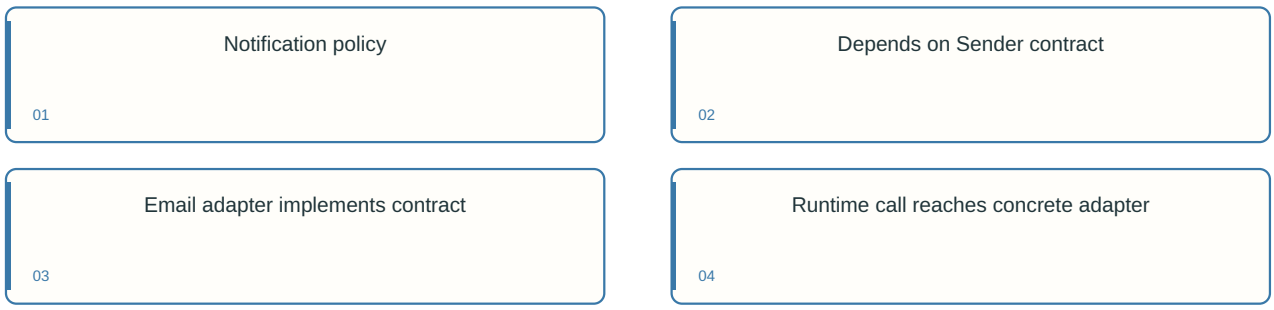


FIG. 6.1.1 **Dependency, dependency direction, and inversion · worked snapshots**

Notification policy · Depends on Sender contract · Email adapter implements contract · Runtime call reaches concrete adapter. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.2 · Inversion of Control (IoC)

Inversion of Control means some control is delegated to a framework or external coordinator: an event loop invokes your callback, or a host constructs your request handler. Dependency Injection is one way to supply collaborators under that arrangement, not a synonym for every form of IoC. Registration order, callback lifetime, and failure propagation still need documentation. Giving control to a framework relocates decisions; it does not remove responsibility for them.

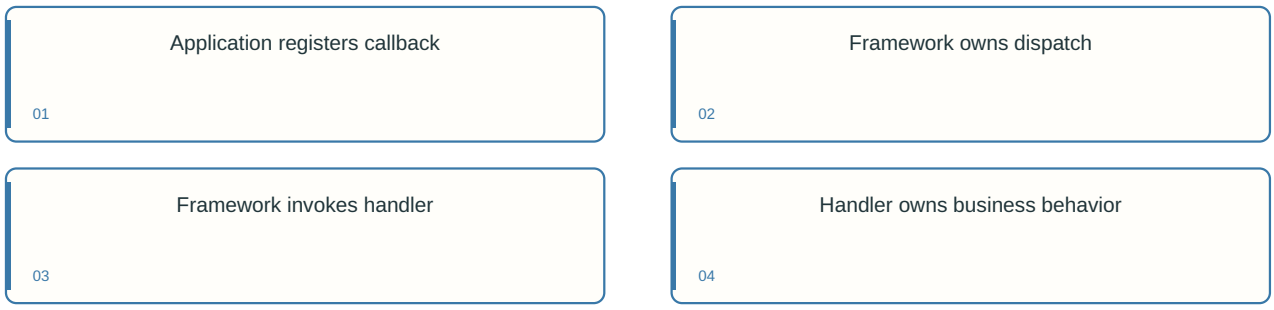


FIG. 6.1.2 **Inversion of Control (IoC) · worked snapshots**

Application registers callback · Framework owns dispatch · Framework invokes handler · Handler owns business behavior. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.3 · Dependency Injection (DI)

Dependency Injection supplies a collaborator rather than constructing or locating it internally. NotificationService(sender) can run with email, SMS, or a recording test sender. The composition root chooses the concrete implementation. DI can be ordinary argument passing; a container is optional. If the caller still supplies one giant object exposing every infrastructure dependency, the design retains broad coupling despite using injection syntax. Keep the contract narrow and its behavior explicit.

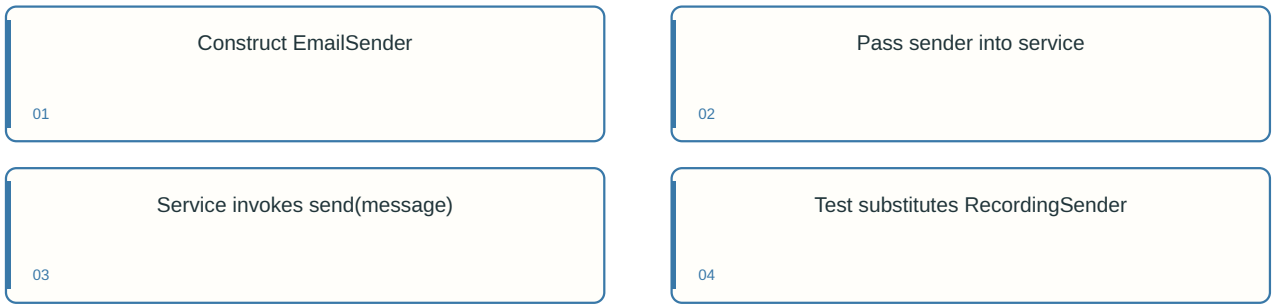


FIG. 6.1.3 **Dependency Injection (DI) · worked snapshots**

Construct EmailSender · Pass sender into service · Service invokes send(message) · Test substitutes RecordingSender. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.4 · Constructor, property/setter, and method injection

Constructor injection makes mandatory dependencies available throughout an object's valid lifetime. Setter injection can model optional or later configuration but risks partially initialized objects. Method injection supplies a collaborator only for one operation and makes per-call variation explicit. Do not store a request-specific dependency passed to a long-lived object unless ownership and concurrency permit it. Select the injection form from required lifetime and validity, rather than assuming framework conventions are universal.

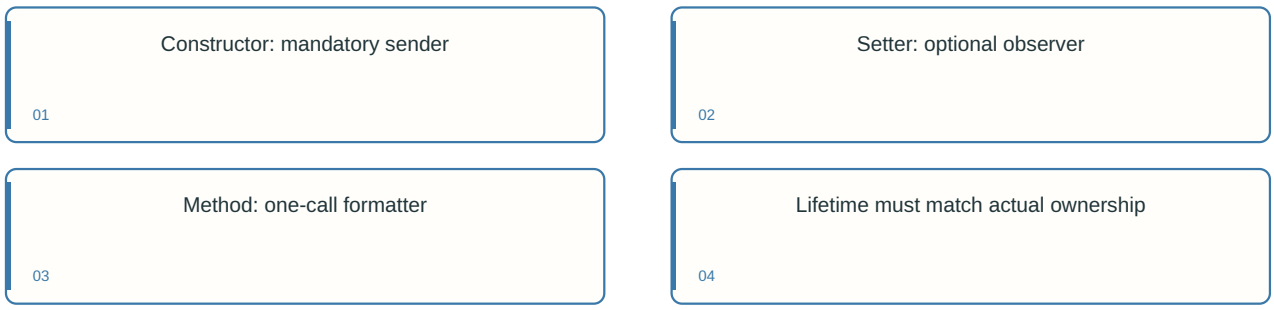


FIG. 6.1.4 **Constructor, property/setter, and method injection · worked snapshots**

Constructor: mandatory sender · Setter: optional observer · Method: one-call formatter · Lifetime must match actual ownership. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.5 · Dependency inversion principle vs DI

Dependency inversion is a design principle about which knowledge depends on which abstraction. DI is a construction technique. A service can receive a concrete SDK through its constructor and still violate the intended abstraction boundary; conversely, a manually wired function can depend on a stable protocol without using a container. Review provider-specific errors, schemas, and resource handles crossing the boundary. Syntax alone cannot prove that high-level policy has been isolated from volatile details.

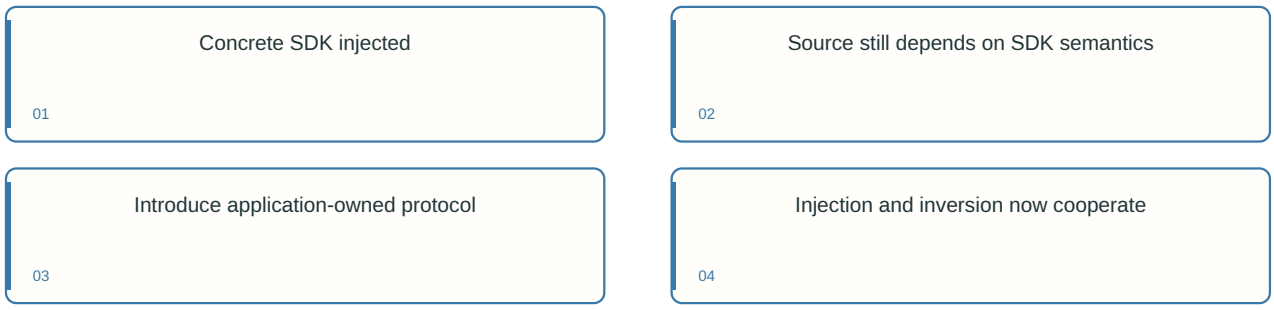


FIG. 6.1.5 **Dependency inversion principle vs DI · worked snapshots**

Concrete SDK injected · Source still depends on SDK semantics · Introduce application-owned protocol · Injection and inversion now cooperate. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.6 · Composition roots and DI containers

The composition root builds the application graph in one comprehensible place. A DI container automates construction and lifetime resolution; it can detect some invalid graphs but can also hide dependencies and defer errors until runtime. Validate registrations at startup and prefer explicit ownership for disposable resources. For three collaborators, manual construction may be easiest. Use container features when they solve real wiring or integration complexity, not to turn every value into a globally resolvable service.

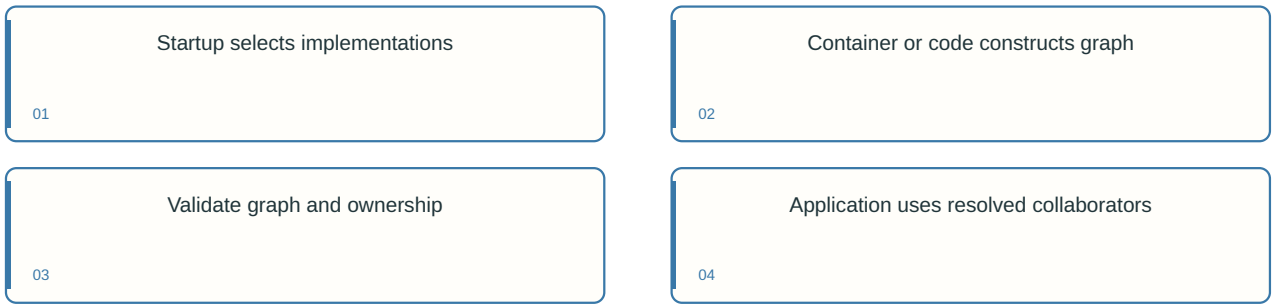


FIG. 6.1.6 **Composition roots and DI containers · worked snapshots**

Startup selects implementations · Container or code constructs graph · Validate graph and ownership · Application uses resolved collaborators. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would the ownership rules change if a job outlives both a request scope and a user session?

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

For three collaborators, manual construction may be easiest.

Carry forward: Startup selects implementations · Container or code constructs graph · Validate graph and ownership · Application uses resolved collaborators.

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Lifetimes, ownership & cleanup

Prerequisites: 6.1 · Dependencies, inversion & composition 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

This chapter examines lifetimes, ownership & cleanup through concrete mechanisms, state snapshots, failure cases, and alternatives.

6.2.1 · Transient, scoped, singleton, and framework-specific lifetimes

Transient means a resolution creates a new instance; scoped means instances are reused within a defined scope; singleton means reuse within the container or application boundary specified by the framework. None implies global uniqueness across processes. If two request scopes each resolve a scoped unit of work twice, they see identities A,A and B,B. A singleton shared by many requests must meet its concurrency contract. Framework names and disposal rules differ and must be checked.

Resolution	Request scope 1	Request scope 2
Scoped: first / second	A / A	B / B
Transient: first / second	C / D	E / F
Singleton in this process	S / S	S / S

FIG. 6.2.1 **Scope identities make lifetime semantics visible**

These are identity observations in one application. Framework registration and disposal rules must still be checked.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.2 · Request scopes, session scopes, and custom scopes

A request scope lasts for one request, while a session scope can span many requests and hold different privacy or memory risks. Custom job scopes are useful for background processing independent of HTTP. Do not let a job borrow a request-scoped connection after the response has disposed it. Create a job scope with a defined end and propagate only necessary immutable identifiers. A scope is an ownership convention, not automatically a database transaction or authenticated-user boundary.

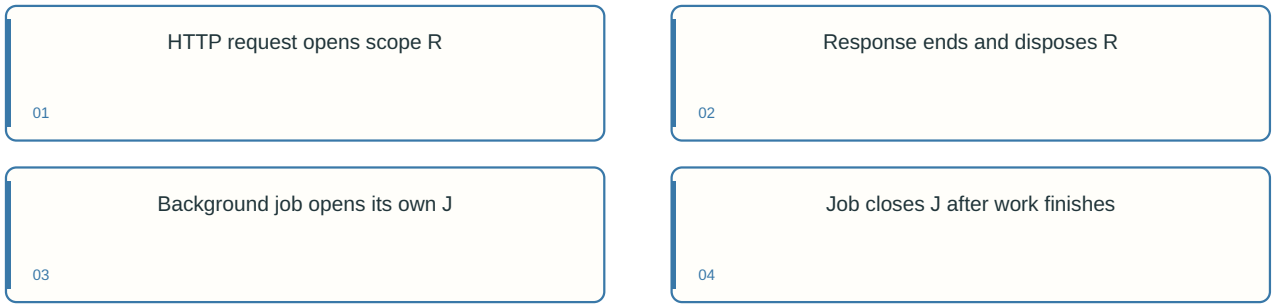


FIG. 6.2.2 **Request scopes, session scopes, and custom scopes · worked snapshots**

HTTP request opens scope R · Response ends and disposes R · Background job opens its own J · Job closes J after work finishes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.3 · Object creation vs object ownership

Creation, reference, and ownership are different relationships. A factory may create a connection while the caller owns disposal; a container may create and retain it until scope end. Two components holding a reference must not both assume exclusive closing rights. Write ownership into the contract. Returning a lazily evaluated query from a disposed repository is a common lifetime bug even when the return type matches; materialize bounded data or transfer an explicit streaming owner.

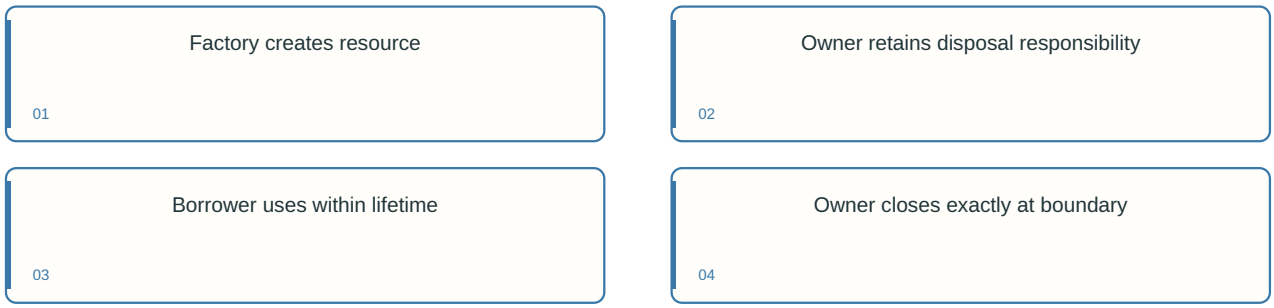


FIG. 6.2.3 **Object creation vs object ownership · worked snapshots**

Factory creates resource · Owner retains disposal responsibility · Borrower uses within lifetime · Owner closes exactly at boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.4 · Disposal, cleanup, and resource lifecycles

Cleanup must run on success, failure, and cancellation. Use structured scopes or finally blocks to return pool permits, close streams, and roll back unfinished transactions. Garbage collection is not a timely release guarantee for external resources. Cleanup can itself fail, so preserve the primary failure while recording cleanup problems. Resource lifetimes affect capacity: leaking one database connection per failed request can exhaust a pool even if the healthy request path is fast.

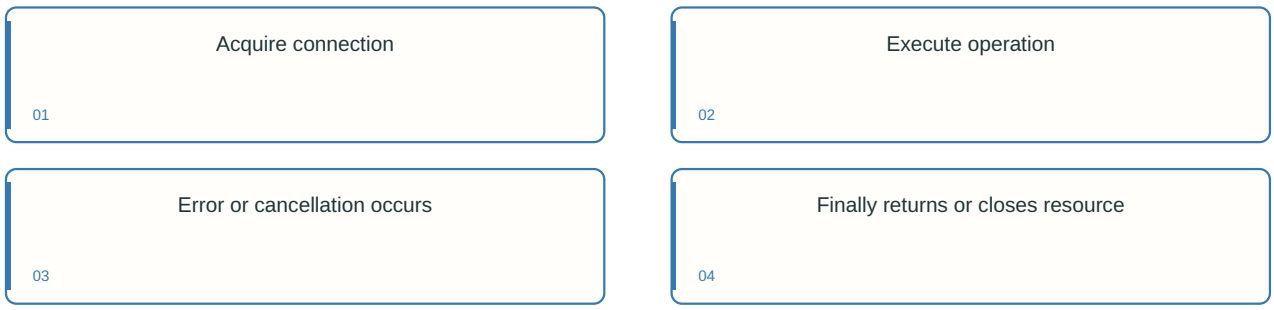


FIG. 6.2.4 **Disposal, cleanup, and resource lifecycles · worked snapshots**

Acquire connection · Execute operation · Error or cancellation occurs · Finally returns or closes resource. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.5 · Lazy initialization

Lazy initialization defers creation until the resource is needed. This can avoid unused work, but moves latency and failure to the first caller and introduces a concurrent initialization race. Define whether failed initialization is retried, cached, or surfaced permanently. A lock or runtime lazy primitive must protect publication as required. Warming essential dependencies at startup may provide clearer readiness behavior when accepting traffic without them would immediately fail.

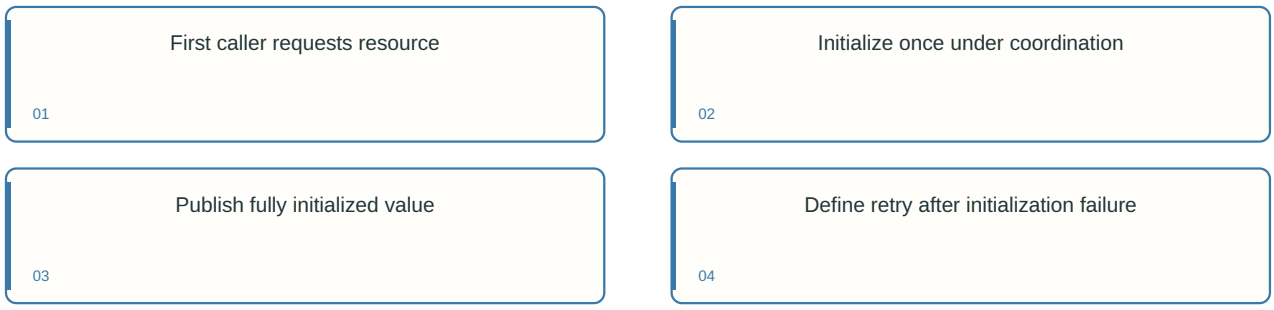


FIG. 6.2.5 **Lazy initialization · worked snapshots**

First caller requests resource · Initialize once under coordination · Publish fully initialized value · Define retry after initialization failure. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.6 · Dependency cycles and captive dependencies

A dependency cycle A→B→A often signals mixed responsibilities or missing coordination; breaking it with a service locator can hide rather than solve the model. A captive dependency occurs when a long-lived owner retains a shorter-lived collaborator, such as a singleton keeping a request's unit of work. Refactor the operation to receive a valid scope or narrow factory. The fix must preserve ownership and authorization context rather than merely satisfy the container's graph validator.

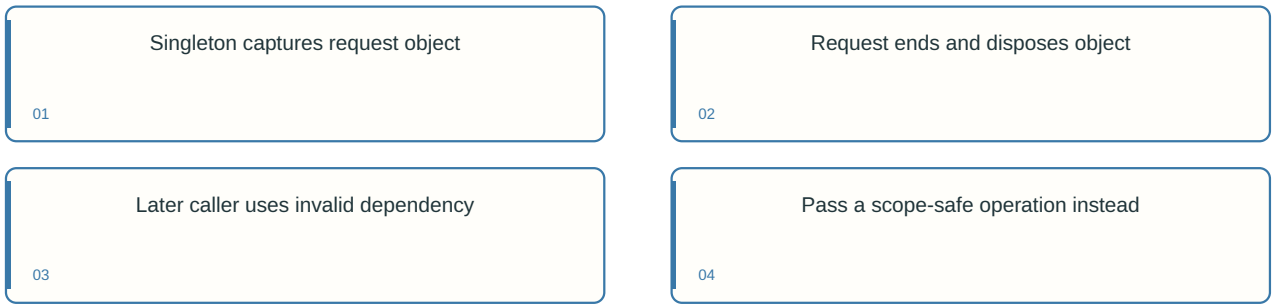


FIG. 6.2.6 **Dependency cycles and captive dependencies · worked snapshots**

Singleton captures request object · Request ends and disposes object · Later caller uses invalid dependency · Pass a scope-safe operation instead. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would the ownership rules change if a job outlives both a request scope and a user session?

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Refactor the operation to receive a valid scope or narrow factory.

Carry forward: Singleton captures request object · Request ends and disposes object · Later caller uses invalid dependency · Pass a scope-safe operation instead.

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Injection, locators & factories

Prerequisites: 6.2 · Lifetimes, ownership & cleanup 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

This chapter examines injection, locators & factories through concrete mechanisms, state snapshots, failure cases, and alternatives.

6.3.1 · DI vs service locator

DI makes collaborators visible in construction or function parameters. A service locator lets code request them from a registry, hiding dependencies behind runtime lookups. Locators can be appropriate at a deliberate integration boundary, but application code that reaches into a global locator is harder to reason about and test. Avoid replacing constructor arguments with one injected container; that often preserves the locator pattern under a different name. Resolve dependencies near the composition boundary.

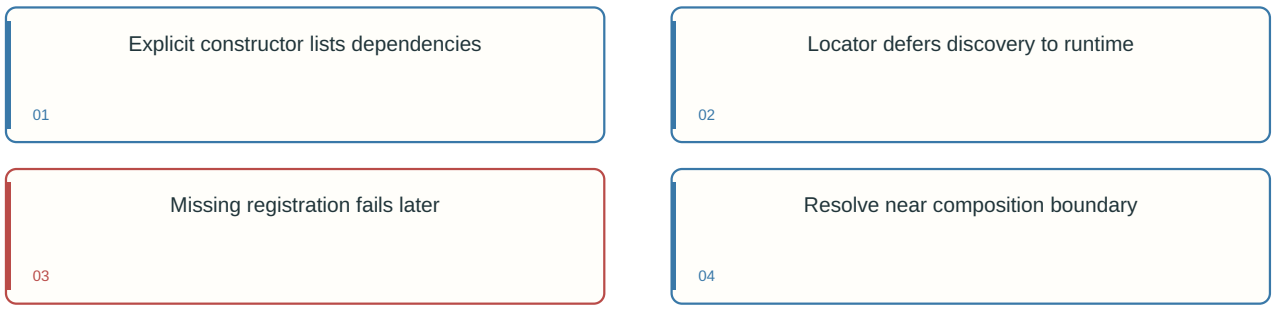


FIG. 6.3.1 **DI vs service locator · worked snapshots**

Explicit constructor lists dependencies · Locator defers discovery to runtime · Missing registration fails later · Resolve near composition boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

6.3.2 · DI vs factory pattern

A factory encapsulates creation or selection; DI supplies an already selected collaborator or a factory itself. A notification service receiving sender_factory can choose a channel at runtime without importing concrete SDKs. The factory's returned lifetime must be clear: is each sender owned by the caller, shared, or scoped? Do not dispose a shared singleton as though every factory call created a private object. These patterns answer different questions and frequently work together.

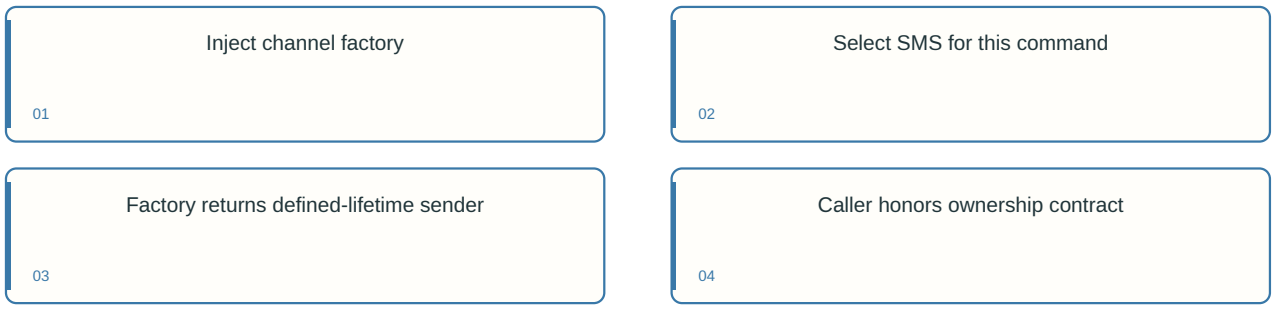
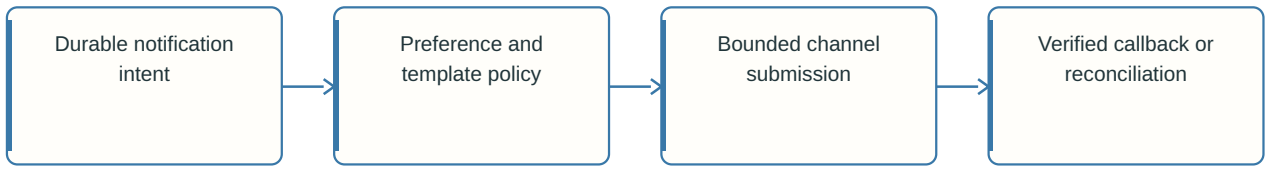


FIG. 6.3.2 **DI vs factory pattern · worked snapshots**

Inject channel factory · Select SMS for this command · Factory returns defined-lifetime sender · Caller honors ownership contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 6.3.3 **The running system · ordered responsibility**

Accept and deduplicate intent, resolve permitted channel and template, persist delivery task, and dispatch under channel capacity. Before sending, verify expiry and relevant current policy. Submit with stable provider identity when supported, persist result or uncertain state, and acknowledge the job. Apply signed callbacks idempotently and prevent an older callback from regressing a terminal delivery state improperly.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

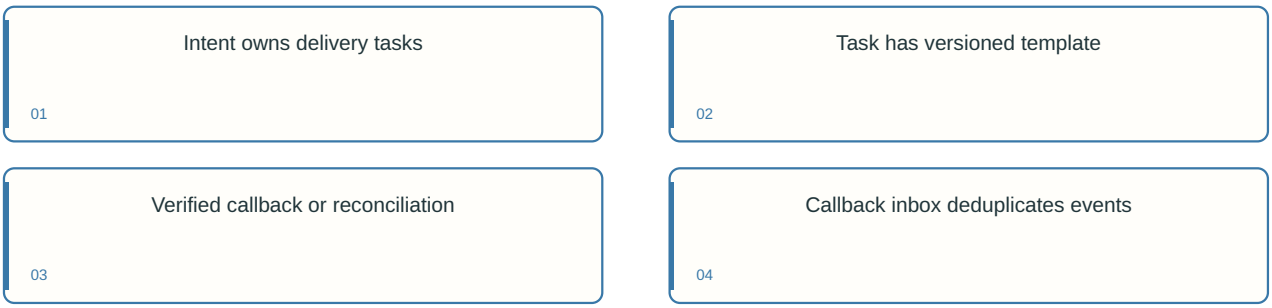


FIG. 6.3.4 **The running system · invariant and recovery review**

Review whether “delivered” is being overstated, whether expired work can still send, and whether provider failover duplicates effects. Test send-then-crash, forged and repeated callbacks, preference revocation after enqueue, template version changes, and channel saturation. A dashboard of successful enqueue operations cannot certify that users received the intended messages under the stated policy.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would the ownership rules change if a job outlives both a request scope and a user session?

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and

prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

The factory's returned lifetime must be clear: is each sender owned by the caller, shared, or scoped? Do not dispose a shared singleton as though every factory call created a private object.

Carry forward: Inject channel factory · Select SMS for this command · Factory returns defined-lifetime sender · Caller honors ownership contract.

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Creational patterns

Prerequisites: 6.3 · Injection, locators & factories 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

This chapter examines creational patterns through concrete mechanisms, state snapshots, failure cases, and alternatives.

6.4.1 · Factory Method, Abstract Factory, Builder, Prototype, Singleton

Creational patterns organize construction. Factory Method varies creation through a creator operation; Abstract Factory supplies a compatible family; Builder assembles a complex value through explicit steps; Prototype copies a configured instance; Singleton constrains instance access within a specified boundary. They are alternatives only when solving the same construction problem. A simple constructor is better when it already expresses all valid states. Pay particular attention to invalid partial builders, shallow-copy aliases, and global mutable singleton state.

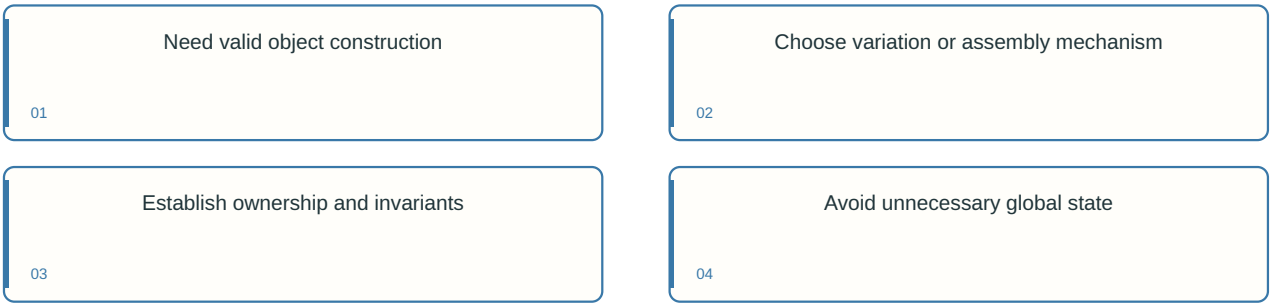


FIG. 6.4.1 **Factory Method, Abstract Factory, Builder, Prototype, Singleton · worked snapshots**

Need valid object construction · Choose variation or assembly mechanism · Establish ownership and invariants · Avoid unnecessary global state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Factory Method

A creator varies the product it constructs through an overridable or supplied creation operation. A report exporter can ask make_writer() without selecting a concrete writer in its processing skeleton. Use it when construction is a meaningful variation point; a plain injected function is often enough. It differs from Abstract Factory, which creates a compatible family. Avoid inheritance merely to rename a constructor.

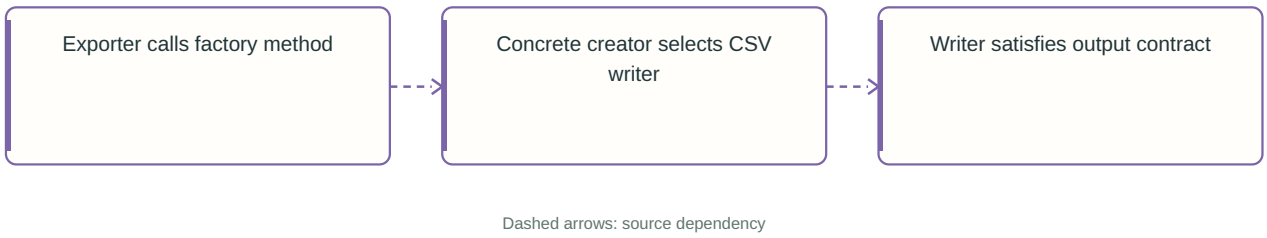


FIG. 6.4.2 **Factory Method · collaborator roles**

Exporter calls factory method · Concrete creator selects CSV writer · Writer satisfies output contract. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
export(rows):
    writer = make_writer()
    writer.write(rows)
```

PATTERN / MECHANISM, USE, AND COST

Abstract Factory

A family factory constructs mutually compatible collaborators, such as a renderer and template engine for one notification format. It prevents accidentally mixing incompatible families. It adds interfaces and can make introducing a new product kind expensive. Prefer direct construction when there is one stable family; choose it when clients genuinely switch whole families rather than only one implementation.



FIG. 6.4.3 **Abstract Factory · collaborator roles**

Select HTML family · Create HTML renderer and template · Both share compatible content model. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
family = HtmlNotificationFactory()
renderer = family.renderer()
template = family.template()
renderer.render(template, data)
```

PATTERN / MECHANISM, USE, AND COST

Builder

A builder accumulates a complex construction request and validates it before creating a usable value. A notification can require recipient and template while allowing optional locale and expiry. Do not expose an invalid partially built notification as if ready. A direct constructor or named parameters is clearer for a small value. Unlike a factory family, the builder emphasizes staged assembly and validation.

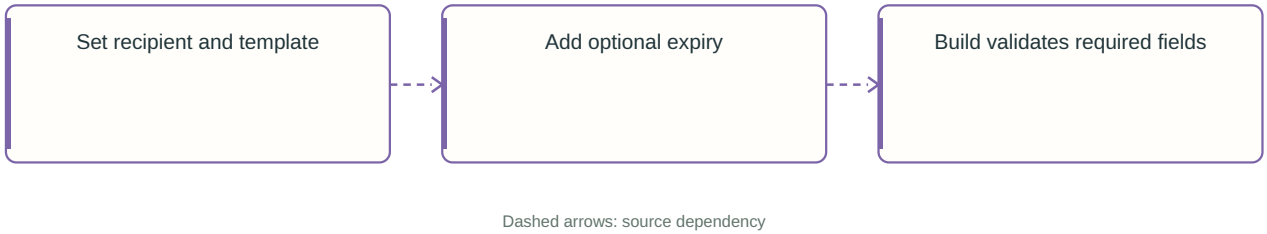


FIG. 6.4.4 **Builder · collaborator roles**

Set recipient and template · Add optional expiry · Build validates required fields. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
builder.to(recipient).template(template_id)
notification = builder.expires_at(deadline).build()
# build rejects missing recipient or incompatible fields
```

PATTERN / MECHANISM, USE, AND COST

Prototype

A prototype creates an object by copying an existing configured instance. Copying a report layout can be convenient when configuration is expensive. Define shallow versus deep copy and ownership of shared resources; copying a connection handle does not create an independent connection. Prefer immutable shared configuration plus new per-request state when that makes aliasing clearer.

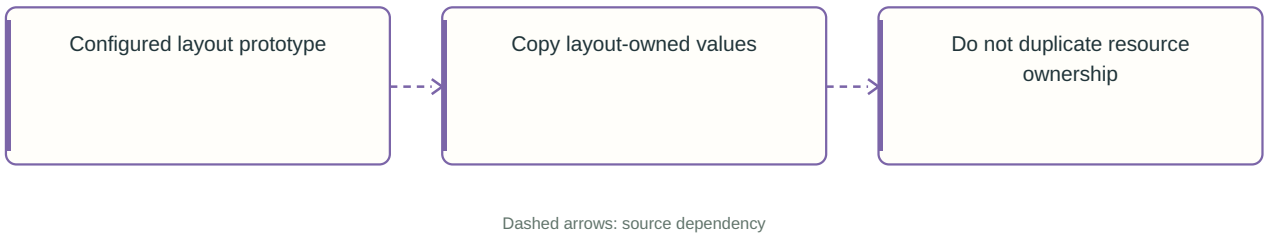


FIG. 6.4.5 **Prototype · collaborator roles**

Configured layout prototype · Copy layout-owned values · Do not duplicate resource ownership. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
layout = prototype.clone_owned_values()
layout.title = "Monthly report"
# Shared immutable font data remains shared
```

PATTERN / MECHANISM, USE, AND COST

Singleton

A singleton restricts instance access within a specified program or container boundary. It is not globally unique across processes. Shared immutable configuration can be appropriate; shared mutable request state creates races and testing interference. Container-managed singleton lifetime and the classic global-access pattern are related but distinct. Prefer explicit injection and documented ownership so global access does not hide dependencies or prevent isolation.

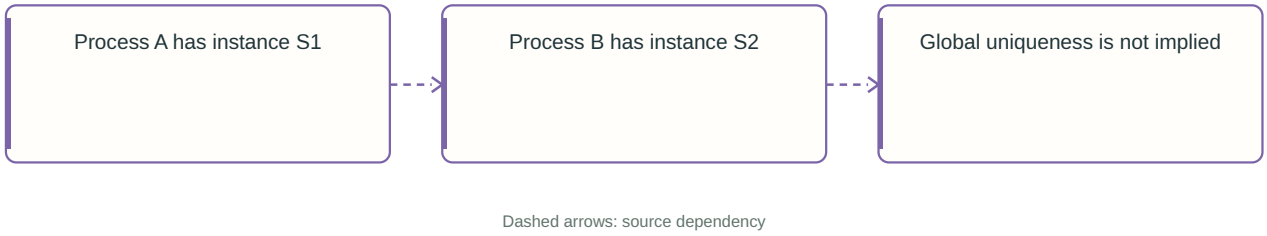


FIG. 6.4.6 **Singleton · collaborator roles**

Process A has instance S1 · Process B has instance S2 · Global uniqueness is not implied. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
configuration = load_immutable_configuration()
service = Service(configuration)
# One application-owned instance; no hidden global lookup
```

▼ Architecture review · How would the ownership rules change if a job outlives both a request scope and a user session?

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A simple constructor is better when it already expresses all valid states.

Carry forward: Need valid object construction · Choose variation or assembly mechanism · Establish ownership and invariants · Avoid unnecessary global state.

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Structural patterns

Prerequisites: 6.4 · Creational patterns 8 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

This chapter examines structural patterns through concrete mechanisms, state snapshots, failure cases, and alternatives.

6.5.1 · Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

Structural patterns organize collaboration: Adapter translates an interface; Bridge separates two independently varying dimensions; Composite treats leaves and groups uniformly; Decorator adds behavior around a collaborator; Facade offers a focused entry point; Flyweight shares immutable intrinsic state; Proxy controls access to a target. They do not all mean “wrapper.” Compare the intent and preserved contract. A decorator that retries a non-idempotent charge changes observable semantics and needs more justification than a transparent logging wrapper.

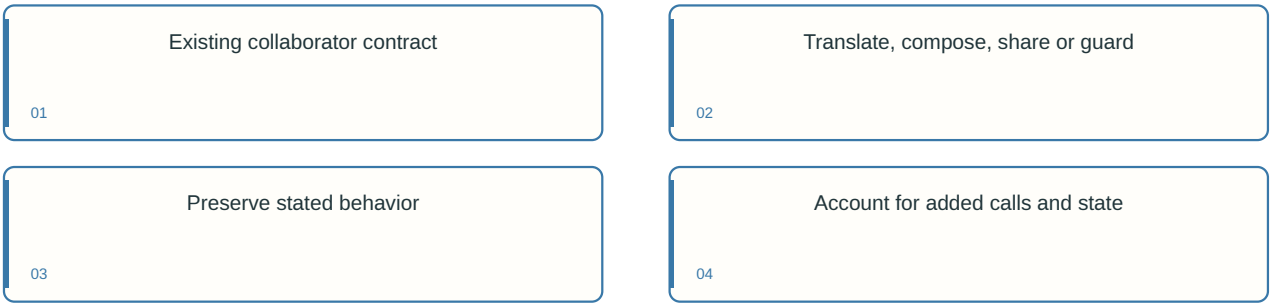


FIG. 6.5.1 **Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy · worked snapshots**

Existing collaborator contract · Translate, compose, share or guard · Preserve stated behavior · Account for added calls and state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Adapter

An adapter translates an existing interface or representation into the contract its client needs. A provider-specific SMS response becomes accepted, declined, or uncertain with a stable reference. Translation must preserve meaning, not only field names. Use it at a real interoperability boundary; avoid wrappers that expose the same volatile SDK types. Unlike Decorator, its principal intent is interface translation.

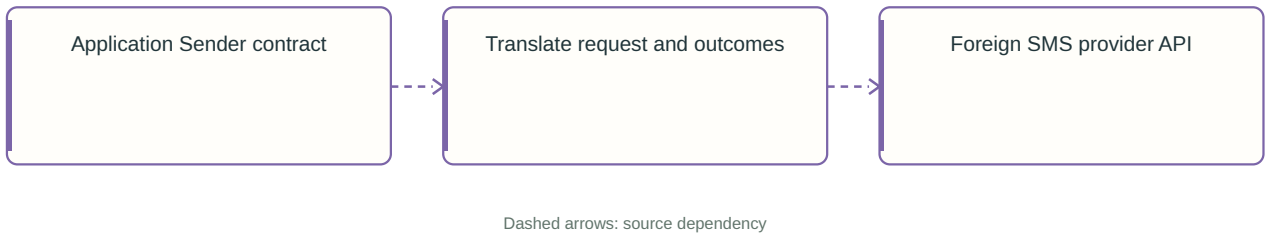


FIG. 6.5.2 **Adapter · collaborator roles**

Application Sender contract · Translate request and outcomes · Foreign SMS provider API. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
send(message):
    raw = sdk.submit(encode(message))
    return map_provider_outcome(raw)
```

PATTERN / MECHANISM, USE, AND COST

Bridge

A bridge separates two independent dimensions of variation, such as notification content types and delivery transports. Each content model uses a transport contract rather than subclassing every content-channel combination. It costs explicit composition and boundaries. It differs from Adapter, which usually reconciles an existing mismatch; Bridge deliberately designs independent variation before combinations multiply.

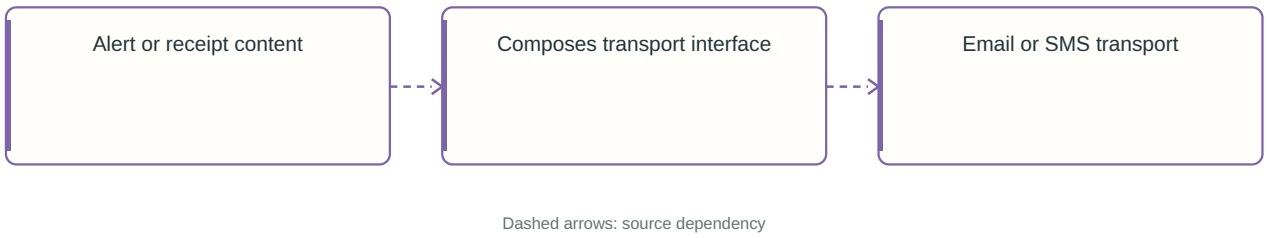


FIG. 6.5.3 **Bridge · collaborator roles**

Alert or receipt content · Composes transport interface · Email or SMS transport. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
class Receipt:
    constructor(transport): self.transport = transport
    deliver(data): return self.transport.send(format_receipt(data))
```

PATTERN / MECHANISM, USE, AND COST

Composite

A composite exposes a compatible interface for leaves and groups. A notification audience may be a user or a group of audiences. Recursive traversal must define cycles, duplicates, failure aggregation, and size limits. Do not treat a group send as atomically successful just because it shares a method with a leaf. Prefer an ordinary collection loop when a tree-shaped structure does not clarify the domain.

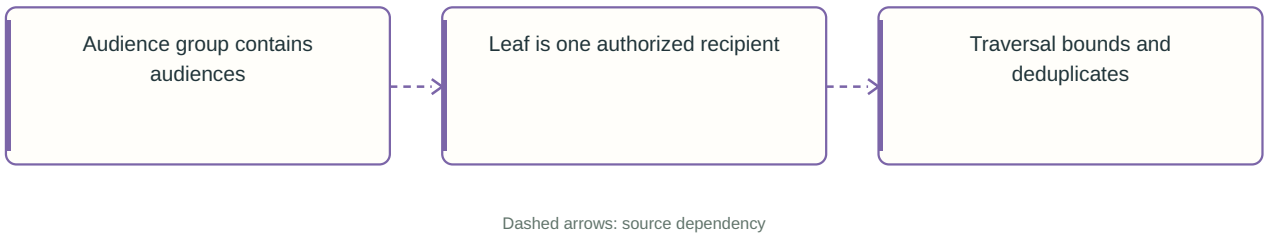


FIG. 6.5.4 **Composite · collaborator roles**

Audience group contains audiences · Leaf is one authorized recipient · Traversal bounds and deduplicates. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
recipients(node, seen):
    if node.id in seen: return []
    seen.add(node.id)
    return node.users_or_recursive_children(seen)
```

PATTERN / MECHANISM, USE, AND COST

Decorator

A decorator preserves a collaborator interface while adding behavior around calls, such as metrics or formatting. Composition order matters: retry(metrics(sender)) counts attempts differently from metrics(retry(sender)). A retry decorator changes effect risk unless operations are safe under a stable identity contract. Use it for orthogonal behavior, not to bury business policy in an opaque stack. Unlike Adapter, it normally retains the same interface.

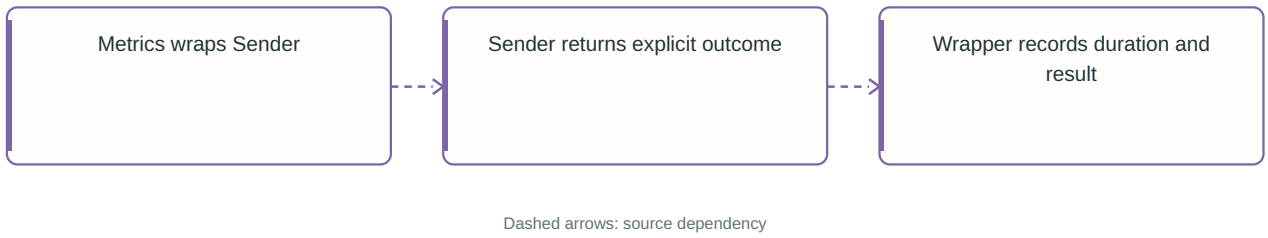


FIG. 6.5.5 **Decorator · collaborator roles**

Metrics wraps Sender · Sender returns explicit outcome · Wrapper records duration and result. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
send(message):
    start = clock()
    try: return inner.send(message)
    finally: metrics.observe(clock() - start)
```

PATTERN / MECHANISM, USE, AND COST

Facade

A facade offers a focused entry point to a subsystem, such as submit_notification() coordinating validation, templating, and durable intent. It reduces the client's need to understand subsystem details. It must not turn into an unbounded god object or conceal important asynchronous outcomes. Unlike a proxy, its main purpose is a simpler subsystem interface, not controlling access to an otherwise identical target.

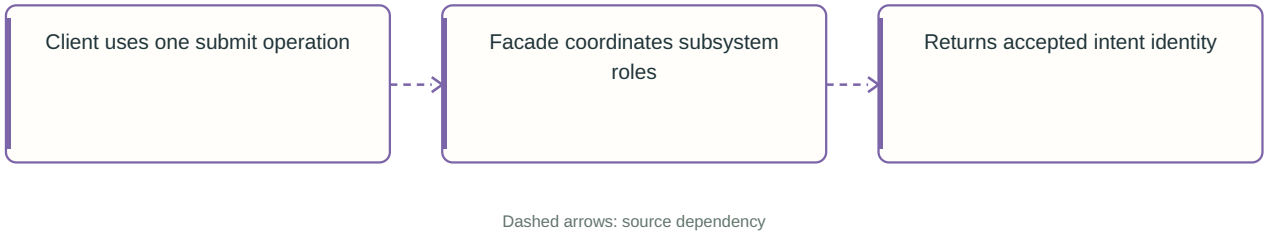


FIG. 6.5.6 **Facade · collaborator roles**

Client uses one submit operation · Facade coordinates subsystem roles · Returns accepted intent identity. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
submit(command):
    validated = validate(command)
    return intents.persist(render_plan(validated))
```

PATTERN / MECHANISM, USE, AND COST

Flyweight

A flyweight shares immutable intrinsic state while keeping per-use context external. Many rendered notifications can reuse one parsed template definition while each supplies a recipient and variables. Sharing mutable recipient state would leak data. Measure memory pressure before adding a pool or cache; ordinary immutable reuse can be sufficient. A flyweight does not automatically provide cache invalidation or lifetime management.

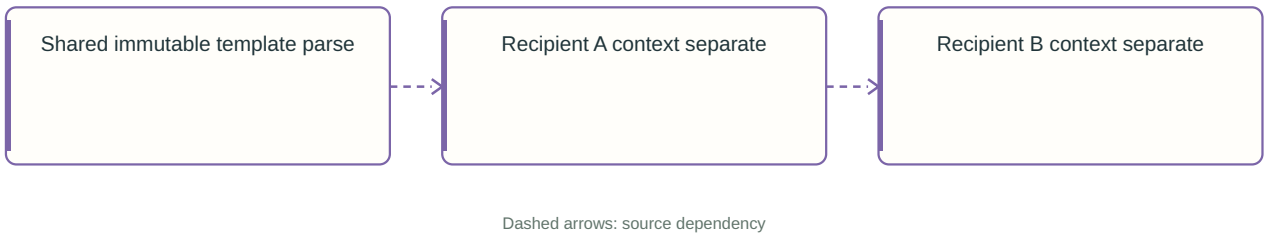


FIG. 6.5.7 **Flyweight · collaborator roles**

Shared immutable template parse · Recipient A context separate · Recipient B context separate. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
template = parsed_templates.get(version)
output_a = render(template, context_a)
output_b = render(template, context_b)
```

PATTERN / MECHANISM, USE, AND COST

Proxy

A proxy implements a target interface while controlling access, placement, or initialization. An authorization proxy can check permission before forwarding a document read. Remote proxies must expose network uncertainty rather than pretending every local call is instant and reliable. Unlike a facade, it commonly stands in for the same conceptual target. Avoid a transparent-looking API that hides expensive retries or unbounded lazy loads.

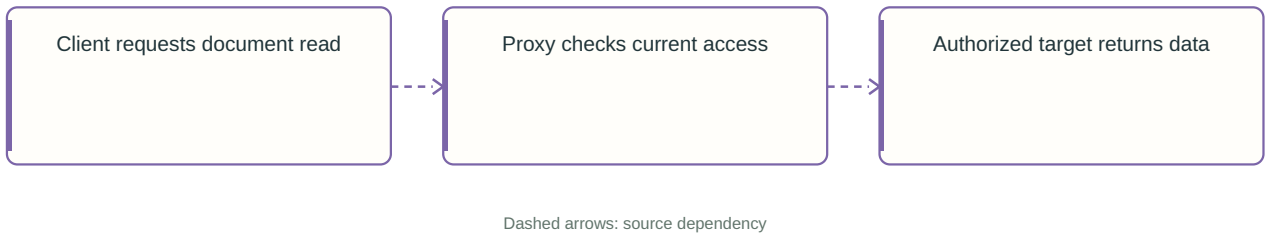


FIG. 6.5.8 **Proxy · collaborator roles**

Client requests document read · Proxy checks current access · Authorized target returns data. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
read(document_id, actor):
    authorize(actor, document_id)
    return target.read(document_id)
```

▼ Architecture review · How would the ownership rules change if a job outlives both a request scope and a user session?

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

They do not all mean "wrapper." Compare the intent and preserved contract.

Carry forward: Existing collaborator contract · Translate, compose, share or guard · Preserve stated behavior · Account for added calls and state.

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Behavioral patterns

Prerequisites: 6.5 · Structural patterns 11 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

This chapter examines behavioral patterns through concrete mechanisms, state snapshots, failure cases, and alternatives.

6.6.1 · Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor

Behavioral patterns organize decisions and coordination. Strategy selects interchangeable algorithms; Observer notifies subscribers; Command represents an intention; State varies behavior by lifecycle; Chain routes a request through handlers; Template Method fixes a skeleton with extension points; Iterator traverses; Mediator coordinates; Memento captures recoverable local state; Visitor adds operations over a known structure. Each introduces a particular extension boundary. Choose it when that boundary matches expected change; do not add a pattern merely to name a few conditionals.

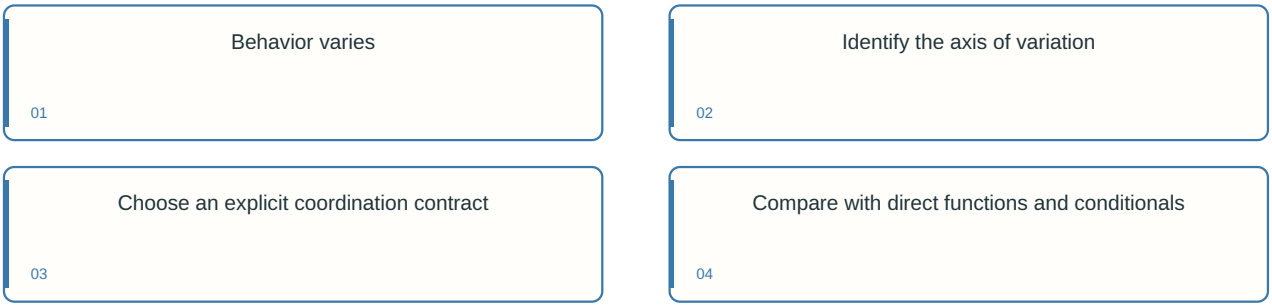


FIG. 6.6.1 **Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor · worked snapshots**

Behavior varies · Identify the axis of variation · Choose an explicit coordination contract · Compare with direct functions and conditionals. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Strategy

A strategy supplies an interchangeable algorithm behind a stable behavioral contract, such as channel selection or fee calculation. It lets one algorithm vary without changing its caller. The strategies must agree on units, errors, and effect semantics. A single stable formula can be a function rather than a class hierarchy. Strategy differs from State because the variation is selected policy rather than lifecycle-dependent behavior.

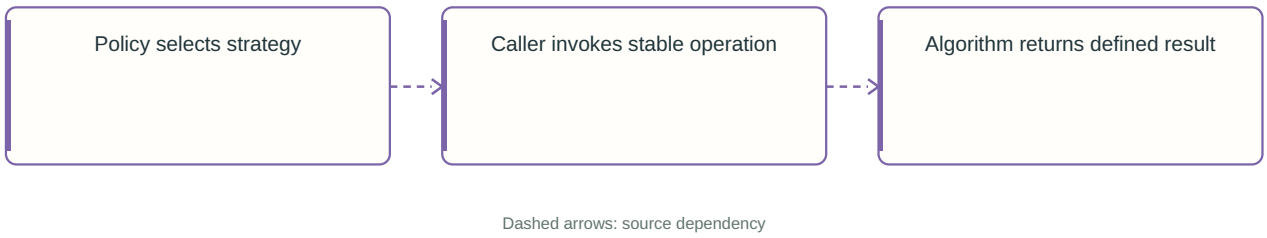


FIG. 6.6.2 **Strategy · collaborator roles**

Policy selects strategy · Caller invokes stable operation · Algorithm returns defined result. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
price = pricing_strategy.calculate(order)
require(price.currency == order.currency)
```

PATTERN / MECHANISM, USE, AND COST

Observer

An observer subscribes to notifications from a subject. Define synchronous versus asynchronous delivery, subscription lifetime, ordering, and exceptions. A local callback list is not a durable broker. A slow or failing observer can delay the subject if invoked inline; an unremoved observer can retain memory. Use for real one-to-many reactions, but use durable event publication when the reaction must survive a process crash.

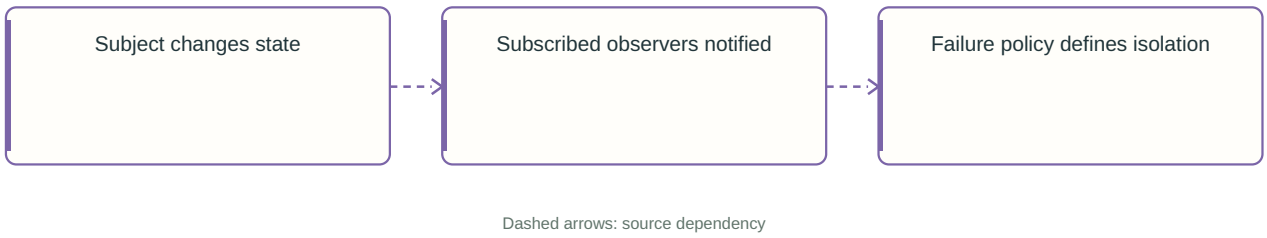


FIG. 6.6.3 **Observer · collaborator roles**

Subject changes state · Subscribed observers notified · Failure policy defines isolation. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
for observer in snapshot(subscribers):
    notify_under_defined_failure_policy(observer, event)
```

PATTERN / MECHANISM, USE, AND COST

Command

A command represents an intention with explicit inputs and identity, enabling scheduling or audit. It differs from an event, which records a fact already accepted. A retryable command must define duplicate handling and validity at execution time. Capturing an arbitrary closure may retain invalid request resources and cannot automatically be persisted. Prefer serializable domain data and an explicit handler for durable work.

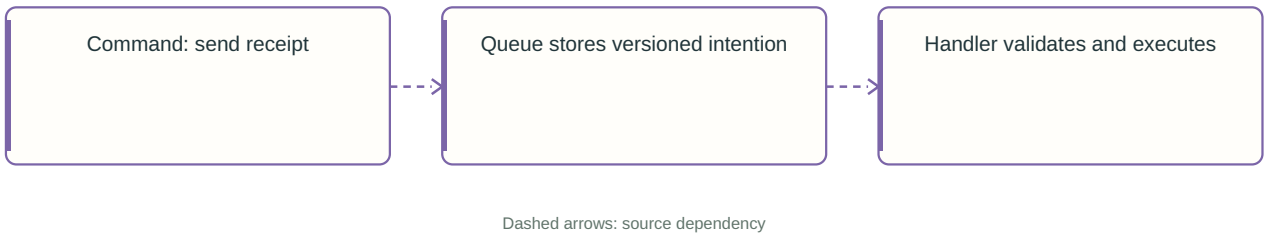


FIG. 6.6.4 **Command · collaborator roles**

Command: send receipt · Queue stores versioned intention · Handler validates and executes. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
command = SendReceipt(order_id, operation_id)
handler.handle(command)
# Handler checks current authorization and state
```

PATTERN / MECHANISM, USE, AND COST

State

State objects or transition tables vary behavior by an entity lifecycle. A pending payment accepts settlement evidence; a refunded payment rejects shipment under the domain policy. Keep transitions and guards explicit and persist them atomically. One class per state is optional; a small transition table may be clearer. Unlike Strategy, current state is constrained by history and cannot be selected arbitrarily by a caller.

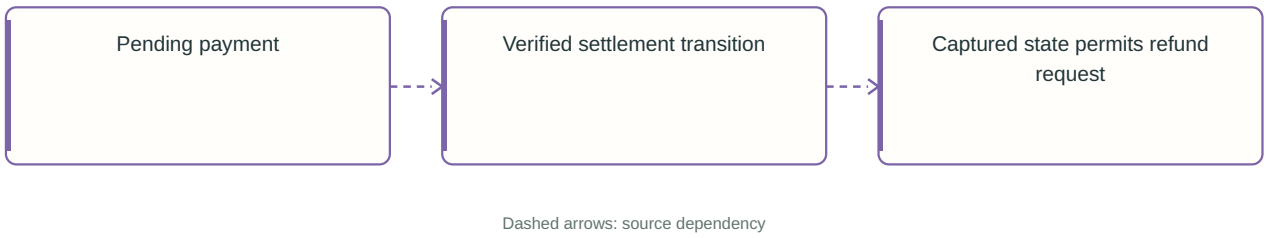


FIG. 6.6.5 **State · collaborator roles**

Pending payment · Verified settlement transition · Captured state permits refund request. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
require((payment.state, event.kind) in allowed_transitions)
payment.state = next_state(payment.state, event)
```

PATTERN / MECHANISM, USE, AND COST

Chain of Responsibility

A chain passes a request through ordered handlers that may handle, transform, reject, or forward it. Authentication, admission, and parsing middleware are common examples. Define termination and propagation rules so an omitted next() does not silently swallow work. Handler order can change security and cost. It differs from a workflow requiring every mandatory stage to commit; a chain alone supplies no durable progress or compensation.

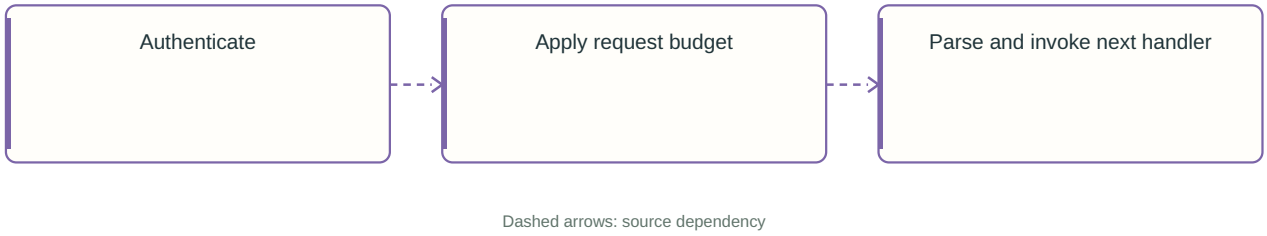


FIG. 6.6.6 **Chain of Responsibility · collaborator roles**

Authenticate · Apply request budget · Parse and invoke next handler. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
auth(rate_limit(parse(application)))(request)
# Each handler has explicit rejection and continuation rules
```

PATTERN / MECHANISM, USE, AND COST

Template Method

A template method fixes an algorithm skeleton and permits selected steps to vary, commonly through inheritance. A base import job can enforce validate, transform, and commit order while formats customize transformation. The base class must preserve valid extension points. Strategy composition is often easier when several steps vary independently. Avoid inheritance that allows subclasses to skip the invariant-protecting skeleton entirely.

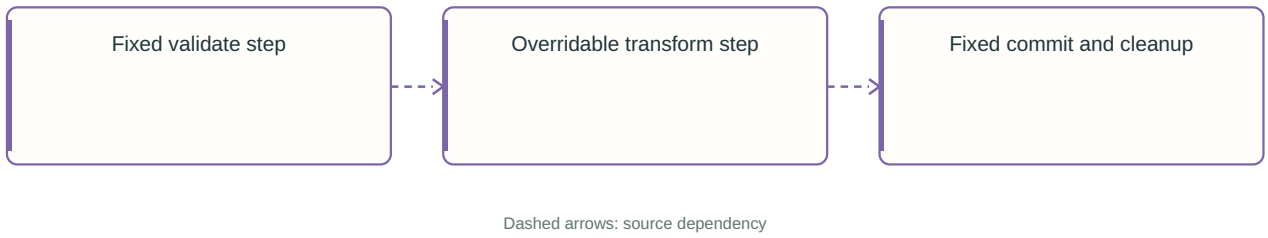


FIG. 6.6.7 **Template Method · collaborator roles**

Fixed validate step · Overridable transform step · Fixed commit and cleanup. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
run(input):
    validate(input)
    result = transform(input)
    return commit(result)
```

PATTERN / MECHANISM, USE, AND COST

Iterator

An iterator exposes traversal without requiring clients to know the collection representation. Database cursors or paginated readers also carry lifetime, ordering, and resource contracts. A lazy iterator can hold a connection while a slow client waits; disposing its owner too early can break traversal. Define snapshot and mutation behavior. For small bounded results, a materialized list may be simpler and safer.

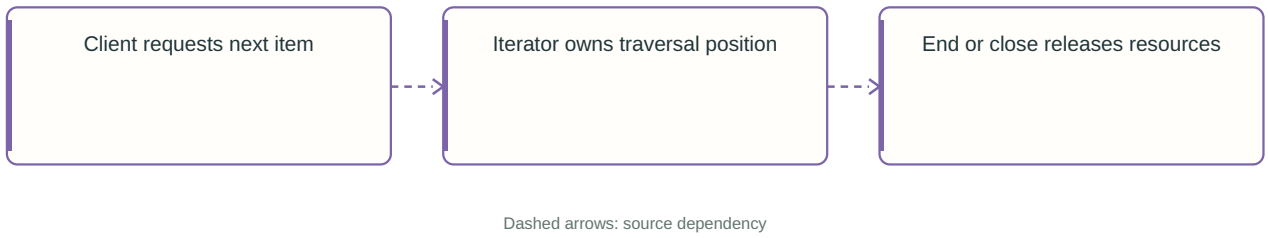


FIG. 6.6.8 **Iterator · collaborator roles**

Client requests next item · Iterator owns traversal position · End or close releases resources. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
with open_bounded_cursor(query) as rows:
    for row in rows:
        consume(row)
```

PATTERN / MECHANISM, USE, AND COST

Mediator

A mediator coordinates collaborators that would otherwise communicate through many direct relationships. A checkout coordinator can sequence inventory and payments while each remains focused. The mediator must have explicit state and failure behavior; otherwise it becomes a god object with hidden dependencies. A durable workflow orchestrator is one possible distributed extension, not a guarantee supplied by the object pattern itself.

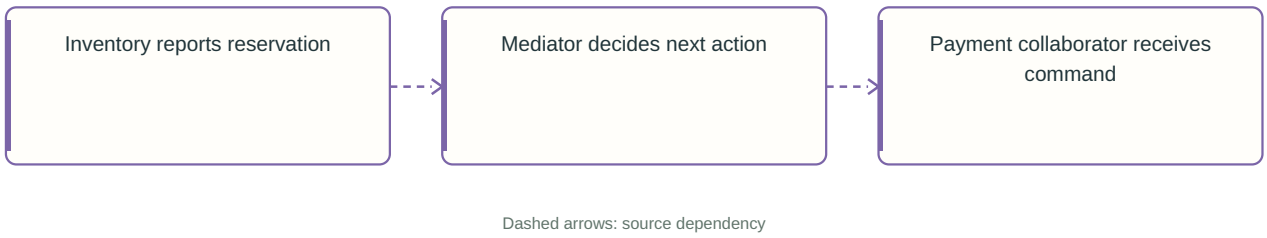


FIG. 6.6.9 **Mediator · collaborator roles**

Inventory reports reservation · Mediator decides next action · Payment collaborator receives command. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
reservation = inventory.reserve(command)
outcome = payments.authorize(command.operation_id)
coordinate_outcomes(reservation, outcome)
```

PATTERN / MECHANISM, USE, AND COST

Memento

A memento captures state for a supported restoration boundary without exposing arbitrary internals. An editor may snapshot local draft state for undo. Define what is included and whether references are copied safely. Restoring local memory cannot reverse a payment or erase a message already sent. Event history, compensations, and backups solve different problems. Use when state capture is manageable and restoration semantics are genuinely well defined.

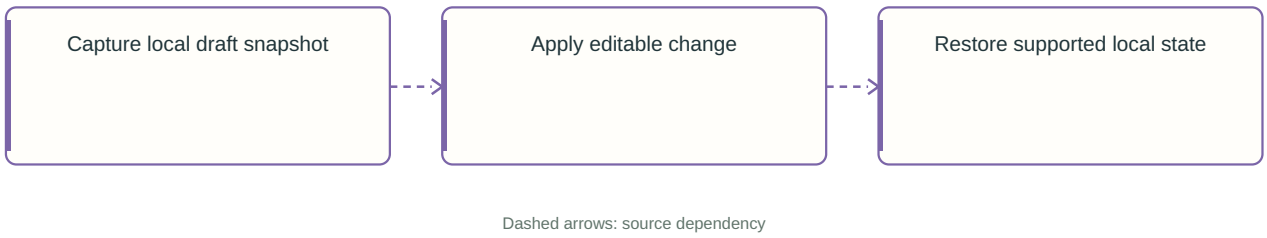


FIG. 6.6.10 **Memento · collaborator roles**

Capture local draft snapshot · Apply editable change · Restore supported local state. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
before = draft.snapshot_owned_values()
draft.apply(edit)
draft.restore(before)
# No claim about undoing external effects
```

PATTERN / MECHANISM, USE, AND COST

Visitor

A visitor adds operations over a known structure by dispatching on element type. A document tree can support export, validation, or counting without placing every operation on every node. It eases adding operations but makes adding new element types more expensive because visitors must handle them. Prefer ordinary pattern matching when it is simpler in the language. Keep exhaustive handling and unknown-element behavior explicit during schema evolution.



FIG. 6.6.11 **Visitor · collaborator roles**

Known document node variants · Visitor supplies export operation · Each node dispatches appropriate behavior. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
visit(node):
    match node.kind:
        case "text": return export_text(node)
        case "image": return export_image(node)
        case _: raise UnsupportedNode
```

▼ Architecture review · How would the ownership rules change if a job outlives both a request scope and a user session?

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Each introduces a particular extension boundary.

Carry forward: Behavior varies · Identify the axis of variation · Choose an explicit coordination contract · Compare with direct functions and conditionals.

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Enterprise patterns & boundaries

Prerequisites: 6.6 · Behavioral patterns 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

This chapter examines enterprise patterns & boundaries through concrete mechanisms, state snapshots, failure cases, and alternatives.

6.7.1 · Repository, Unit of Work, Specification, CQRS, and dependency boundaries

Repository mediates domain-oriented access to persistent data; Unit of Work groups changes under one commit decision; Specification represents a reusable predicate; CQRS separates command and query models where their needs justify it. A repository should not pretend remote calls and local transactions share identical failure semantics. A Unit of Work must own its transaction and cleanup. CQRS can be implemented inside one process and one database; event sourcing and independent services are optional additional choices.

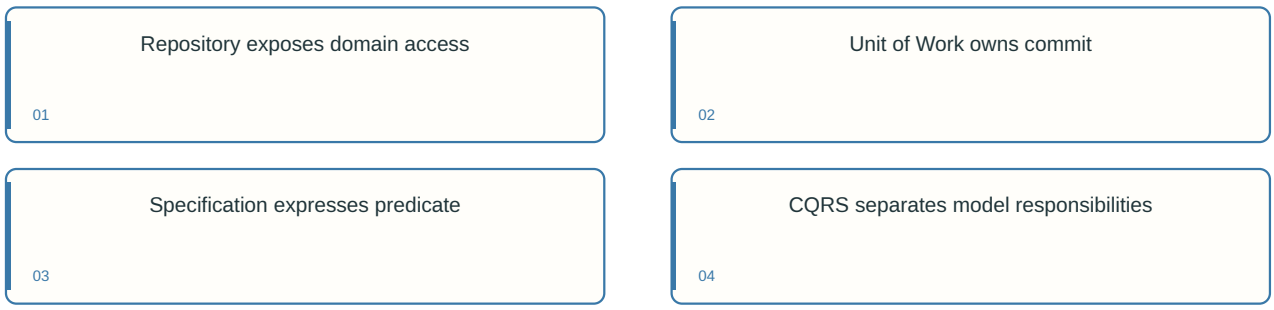


FIG. 6.7.1 **Repository, Unit of Work, Specification, CQRS, and dependency boundaries · worked snapshots**

Repository exposes domain access · Unit of Work owns commit · Specification expresses predicate · CQRS separates model responsibilities. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / MECHANISM, USE, AND COST

Repository

A repository mediates domain-oriented persistence rather than exposing storage syntax everywhere. Its contract must define transactions, errors, query bounds, and returned-value lifetime. A repository per table is not always the right aggregate boundary. Do not hide slow remote calls or lazy resources behind a misleadingly cheap-looking method. Use it when it protects meaningful knowledge, while retaining real storage integration tests.

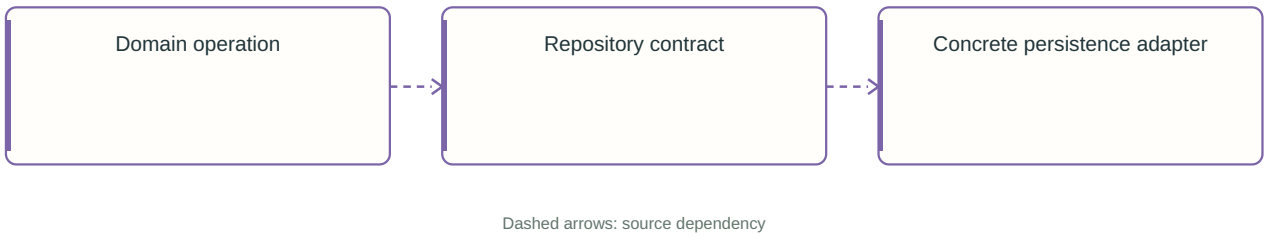


FIG. 6.7.2 **Repository · collaborator roles**

Domain operation · Repository contract · Concrete persistence adapter. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
reservation = repository.get_for_update(reservation_id)
reservation.confirm(payment_evidence)
repository.save(reservation)
```

PATTERN / MECHANISM, USE, AND COST

Unit of Work

A Unit of Work groups local changes under one commit and tracks their ownership. Inventory decrement and reservation insert can belong to one transaction. Commit and cleanup semantics must be explicit, and collaborators must not secretly commit independently. It cannot atomically include an unrelated payment provider simply because both are called inside a language scope. Keep the unit short and free of unnecessary network waits.

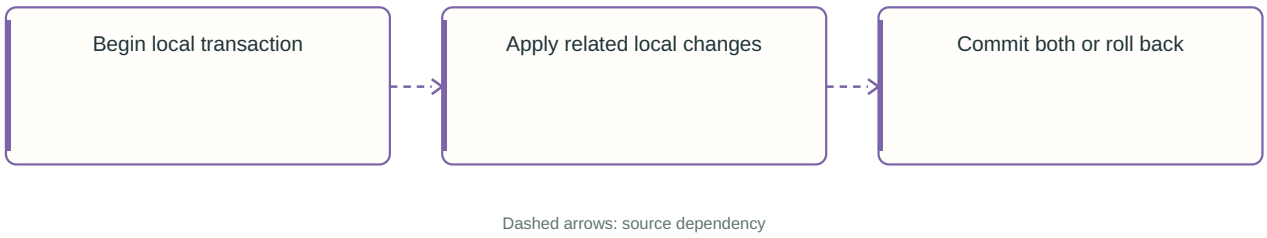


FIG. 6.7.3 **Unit of Work · collaborator roles**

Begin local transaction · Apply related local changes · Commit both or roll back. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
with unit_of_work() as uow:
    uow.inventory.reserve(seat)
    uow.reservations.add(record)
    uow.commit()
```

PATTERN / MECHANISM, USE, AND COST

Specification

A specification names a reusable predicate or selection rule, such as eligible_for_refund(payment,now). Keep its semantics explicit when translating it into a database query, especially around nulls, time, and authorization. Reusing a pure predicate is simple; an elaborate expression framework is justified only by real composition needs. A specification describes eligibility, not the atomic transition that prevents eligibility from changing concurrently.

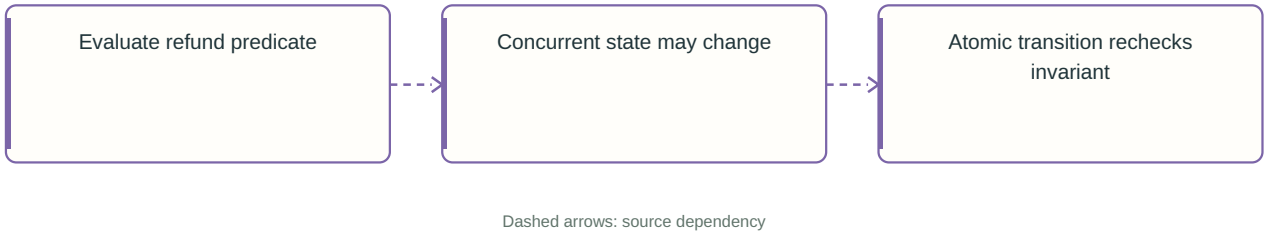


FIG. 6.7.4 **Specification · collaborator roles**

Evaluate refund predicate · Concurrent state may change · Atomic transition rechecks invariant. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
eligible = RefundPolicy.satisfied_by(payment, now)
if eligible:
    repository.refund_if_still_eligible(payment.version)
```

PATTERN / MECHANISM, USE, AND COST

CQRS

Separate command and query models when they need different behavior or shape. A transaction-safe payment aggregate can emit changes to a read-friendly statement view. The read side can lag, so freshness and rebuilding must be specified. It need not use separate processes, storage engines, or event sourcing. Avoid the additional synchronization and model maintenance when one straightforward model already serves the workload.

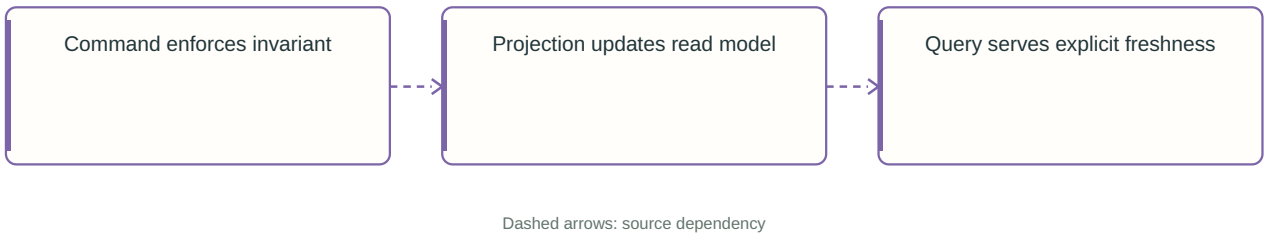


FIG. 6.7.5 **CQRS · collaborator roles**

Command enforces invariant · Projection updates read model · Query serves explicit freshness. This is the pattern's original miniature example, not a claim that every role must be a separate class or deployed service.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

```
commands.capture(payment_id, operation_id)
statement = queries.account_statement(account_id)
# Projection lag is part of the query contract
```

Worked implementation · connect the unit

PYTHON · LIFETIME IDENTITIES

```
singleton = object()
def new_scope(): return {"scoped": object()}
a, b = new_scope(), new_scope()
assert a["scoped"] is a["scoped"]
assert a["scoped"] is not b["scoped"]
assert object() is not object()
assert singleton is singleton
```

The identities model one process. Actual containers differ in registration, disposal, and scope behavior. A real request scope must clean up owned resources; storing its object in a singleton changes the lifetime and can be invalid.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · How would the ownership rules change if a job outlives both a request scope and a user session?

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A Unit of Work must own its transaction and cleanup.

Carry forward: Repository exposes domain access · Unit of Work owns commit · Specification expresses predicate · CQRS separates model responsibilities.

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

PART 2 / UNIT 07

Application Architecture and Domain-Driven Design

The running example

Design an online shopping application first as a modular monolith, then examine the conditions under which selected domains should become independent services.

Unit 7 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

View	Boundary rule	Runtime implication
Layered	Separate technical roles	Can remain one process
Hexagonal	Ports express app needs	Adapters call in or out
Clean/Onion	Policy dependencies inward	Runtime effects still reach adapters

FIG. 7.0.1 **Layered vs Hexagonal**

The styles organize source responsibilities, not mandatory physical deployment layers.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Design	Deployment	Main cost
Modular monolith	One release unit	Must enforce internal ownership
Microservices	Independent processes	Network, versions, operations
Distributed monolith	Many coupled processes	Costs without independence

FIG. 7.0.2 **modular monolith vs microservices**

Choose according to actual independent operation and change needs.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

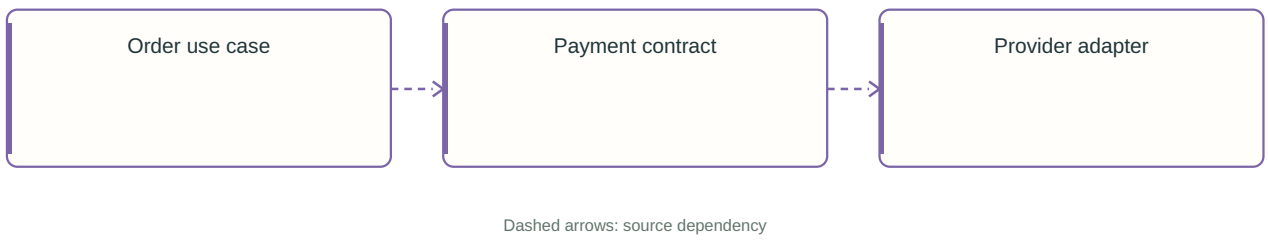


FIG. 7.0.3 **dependency arrows**

Dependency arrows express knowledge; runtime execution reaches the selected adapter.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

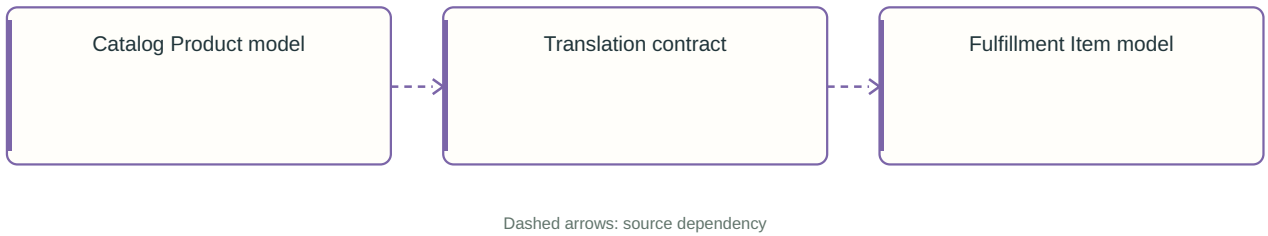


FIG. 7.0.4 **bounded context map**

Bounded contexts retain different valid meanings rather than sharing every field globally.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Layers, monoliths & ports

Prerequisites: Object-Oriented and Modular Software Design, Design Patterns, Dependency Injection, and Object Lifecycles

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an online shopping application first as a modular monolith, then examine the conditions under which selected domains should become independent services.

This chapter examines layers, monoliths & ports through concrete mechanisms, state snapshots, failure cases, and alternatives.

7.1.1 · Layered/N-tier architecture

Layered architecture separates presentation, application behavior, domain policy, and infrastructure at a chosen granularity. Dependencies should be intentional: a controller should not bypass a domain invariant merely because an SQL helper is nearby. Layers can run in one process; physical N-tier deployment is a separate decision. Avoid mandatory pass-through classes that add no policy or abstraction. Trace one change, such as a new pricing rule, to verify that the layer boundary actually localizes knowledge.

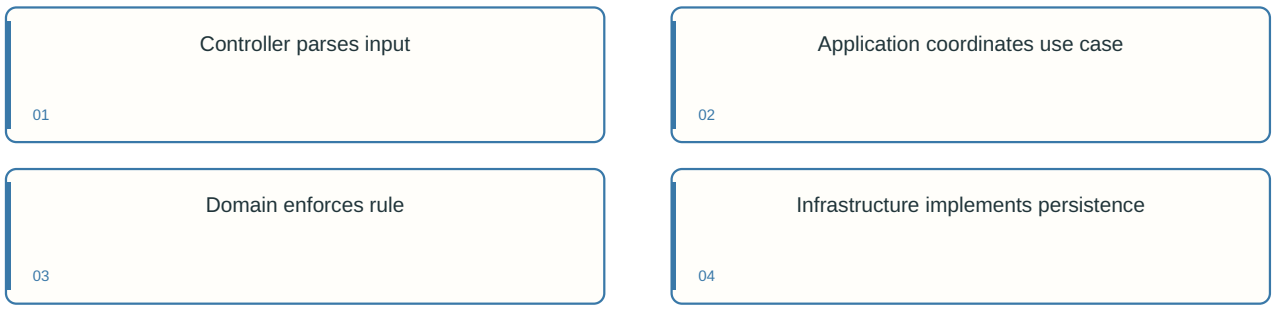


FIG. 7.1.1 **Layered/N-tier architecture · worked snapshots**

Controller parses input · Application coordinates use case · Domain enforces rule · Infrastructure implements persistence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.2 · Modular monoliths

A modular monolith is deployed as a unit while its modules maintain explicit boundaries and ownership. Orders, catalog, and payments can have separate interfaces and schema ownership in one application. This keeps local calls and coordinated deployment simple while retaining future extraction options. Shared tables writable by every module defeat much of the benefit. Enforce module dependencies in code review or automated checks, and make cross-module operations pass through agreed contracts.

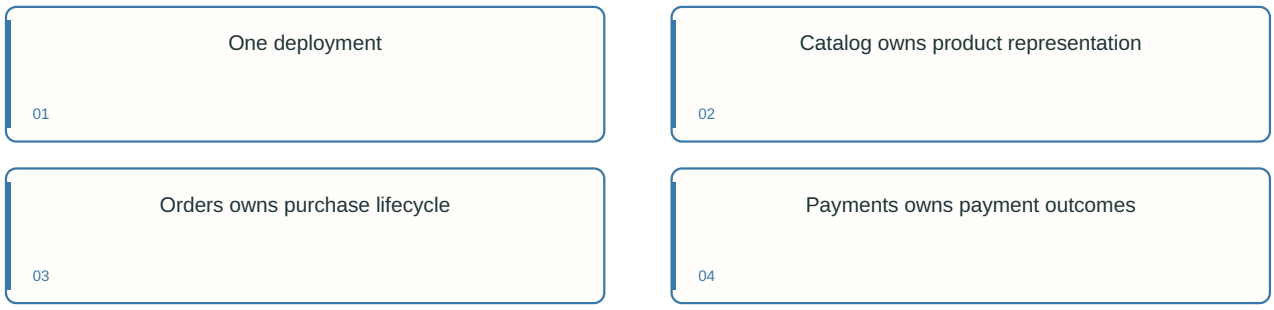


FIG. 7.1.2 **Modular monoliths · worked snapshots**

One deployment · Catalog owns product representation · Orders owns purchase lifecycle · Payments owns payment outcomes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.3 · Monolith vs microservices

Microservices add independently deployable processes and data ownership boundaries, along with network failures, version skew, and distributed operations. A monolith can be modular; a set of services can be tightly coupled. Extract inventory when independent scaling, failure isolation, or team ownership gives a concrete benefit. If every change requires simultaneous deployment of five services, independence is mostly nominal. Compare operational cost and transaction complexity against the change that motivated extraction.

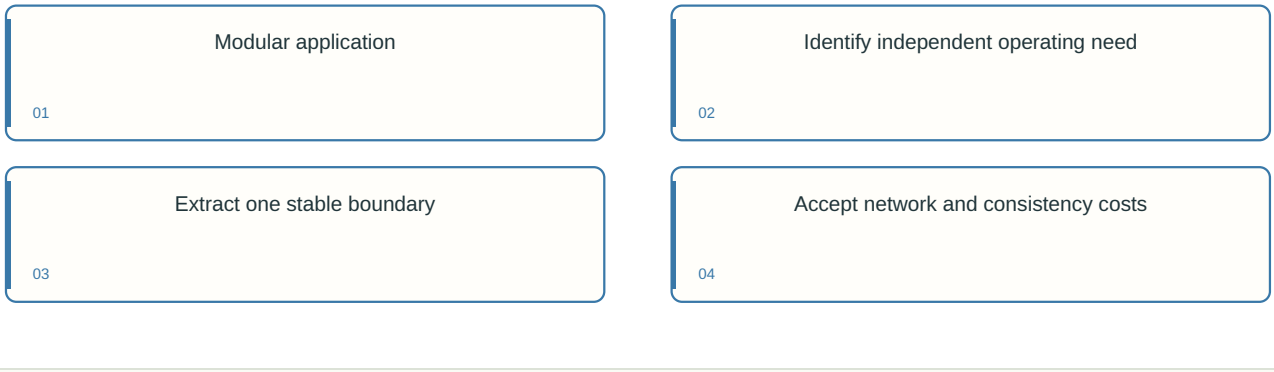


FIG. 7.1.3 **Monolith vs microservices · worked snapshots**

Modular application · Identify independent operating need · Extract one stable boundary · Accept network and consistency costs. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.4 · Clean Architecture

Clean Architecture places business policy inside dependencies on application-owned abstractions; framework and storage details depend inward. The drawing's circles describe source dependency rules, not the physical path of runtime messages. A use case might call an injected repository implemented by an outer adapter. Avoid leaking ORM entities or HTTP response types into core policy. The cost is extra mapping and contracts; a small CRUD tool may need only a modest version of the same separation.

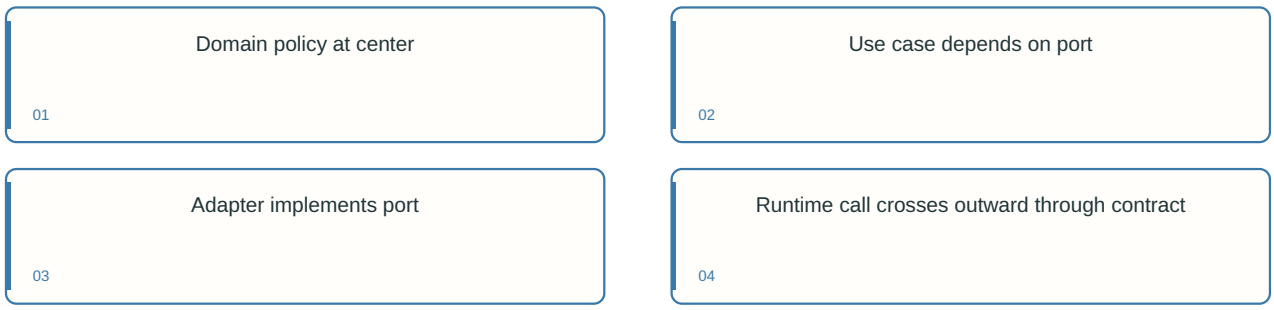


FIG. 7.1.4 **Clean Architecture · worked snapshots**

Domain policy at center · Use case depends on port · Adapter implements port · Runtime call crosses outward through contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.5 · Hexagonal/Ports and Adapters

Hexagonal architecture names inbound and outbound ports with adapters around the application. A CLI and HTTP controller can invoke the same create-order port; an SQL repository and test fake implement a storage port. Ports express the application's needs rather than every operation a vendor SDK offers. Network isolation is not implied by the hexagon. Test the core with controlled adapters and separately verify serialization, transaction behavior, and failures at real boundaries.

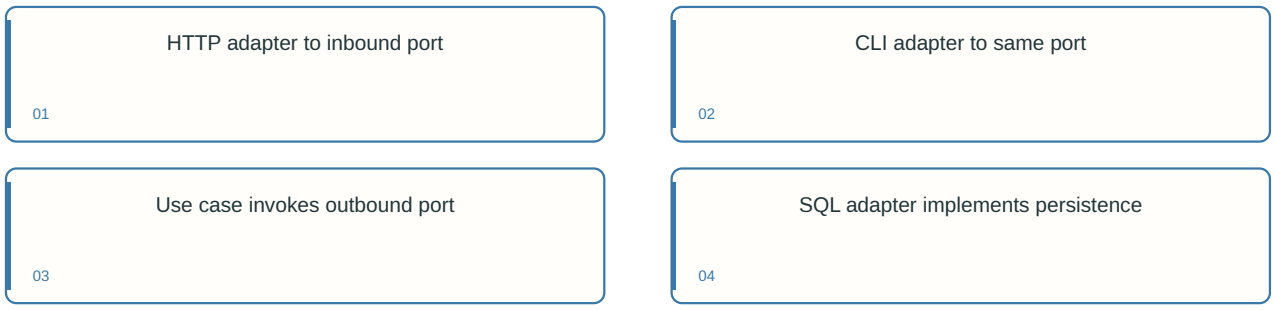


FIG. 7.1.5 **Hexagonal/Ports and Adapters · worked snapshots**

HTTP adapter to inbound port · CLI adapter to same port · Use case invokes outbound port · SQL adapter implements persistence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.6 · Onion Architecture

Onion Architecture also emphasizes dependencies toward domain policy, often organizing domain, application services, and infrastructure as concentric responsibilities. Names and layer placement vary across communities. Do not argue that one diagram guarantees better design than another with equivalent obligations. Ask whether the order rules can run independently of the web framework and whether storage errors remain explicit. Architectural styles are organizing tools; the concrete contracts and change behavior determine their usefulness.

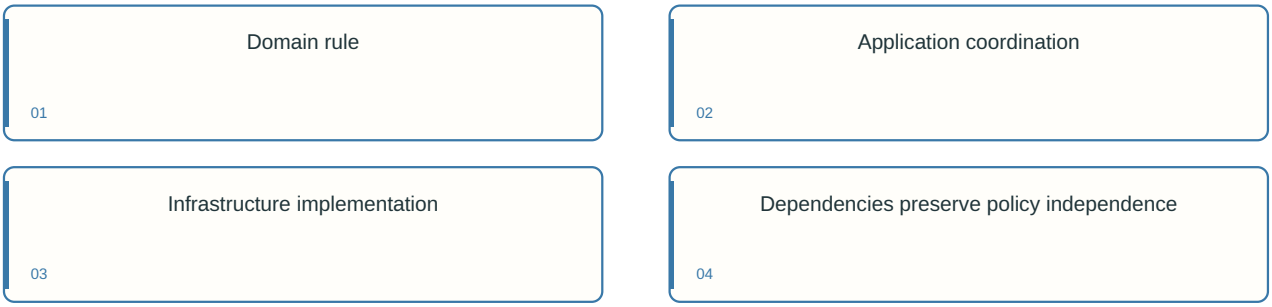


FIG. 7.1.6 **Onion Architecture · worked snapshots**

Domain rule · Application coordination · Infrastructure implementation · Dependencies preserve policy independence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which evidence would justify extracting payments from a modular monolith while retaining the same business invariants?

Let orders request a payment through a contract; disallow direct imports of the payment storage adapter or writes to its private table. A test should enforce a meaningful dependency rule rather than every incidental class name. Show how a provider adapter can change while the order rule remains stable.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Ask whether the order rules can run independently of the web framework and whether storage errors remain explicit.

Carry forward: Domain rule · Application coordination · Infrastructure implementation · Dependencies preserve policy independence.

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Model a bounded context

Prerequisites: 7.1 · Layers, monoliths & ports 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an online shopping application first as a modular monolith, then examine the conditions under which selected domains should become independent services.

This chapter examines model a bounded context through concrete mechanisms, state snapshots, failure cases, and alternatives.

7.2.1 · Domain-Driven Design (DDD)

Domain-Driven Design builds a shared model of a complex business with domain experts. Its value is making ambiguous concepts and invariants explicit, not turning every database row into a class. In shopping, “available inventory” may differ between merchandising and reservation processing. Explore those meanings before choosing service boundaries. For a simple reference table, elaborate aggregates and events may cost more than they clarify; apply modeling effort where business behavior is genuinely complex.

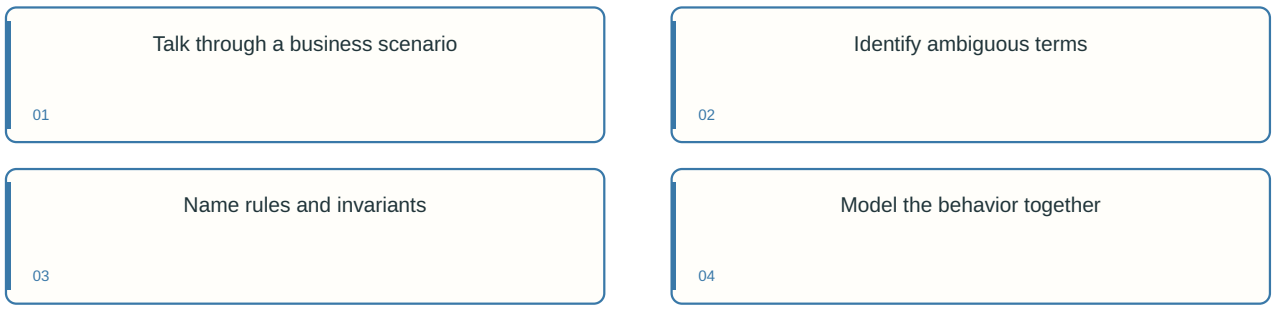


FIG. 7.2.1 **Domain-Driven Design (DDD) · worked snapshots**

Talk through a business scenario · Identify ambiguous terms · Name rules and invariants · Model the behavior together. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.2 · Bounded contexts and ubiquitous language

A bounded context defines where a particular model and vocabulary are valid. The catalog's Product can emphasize description and merchandising; fulfillment's Item can emphasize a physical unit and location. Translating between them is deliberate, not evidence that one universal Product schema is missing. Ubiquitous language means consistent domain language inside a context, not forcing every team to use one model. Map interactions and ownership before deciding whether a context is a separate process.

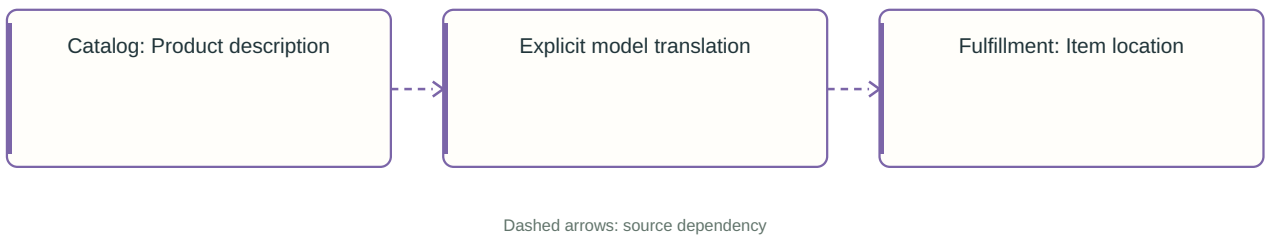


FIG. 7.2.2 **Context boundaries protect different business meanings**

The source-dependency boundary is not automatically a network deployment. Translation prevents one universal schema from erasing meaningful domain differences.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.3 · Aggregates, entities, value objects, domain services

An aggregate is a consistency boundary whose root controls changes to its members. An Order might protect the relationship between line items and total amount within a transaction. Entities carry identity; value objects represent interchangeable values; domain services express rules spanning concepts. A huge aggregate containing the entire shop creates contention and awkward loading. Keep invariants that must change atomically together, and handle cross-aggregate coordination explicitly rather than assuming object references imply distributed transactions.

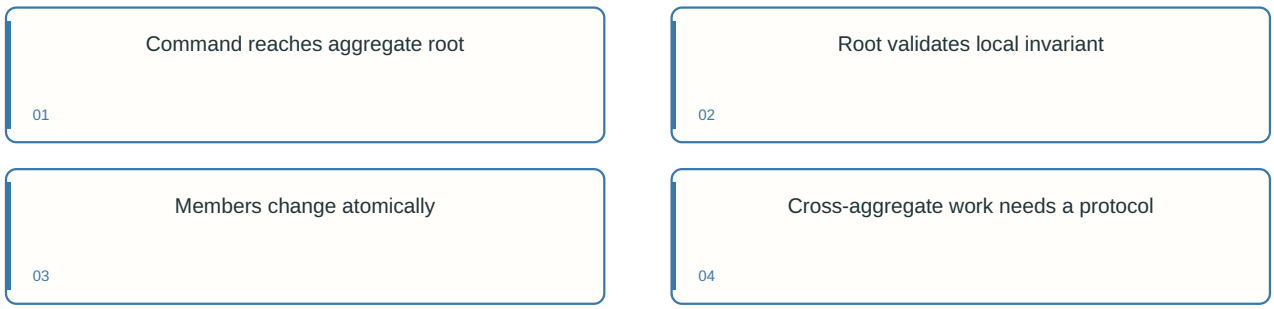


FIG. 7.2.3 **Aggregates, entities, value objects, domain services · worked snapshots**

Command reaches aggregate root · Root validates local invariant · Members change atomically · Cross-aggregate work needs a protocol. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.4 · Domain events

A domain event records a meaningful fact such as OrderPlaced, not a vague instruction to “do something.” Include stable identity, time semantics, and enough versioned information for intended consumers without exposing every internal field. Recording an event in memory is not reliable publication. If state and publication intent must agree, persist them atomically through an outbox or another justified mechanism. Consumers must still handle duplicates, delayed delivery, and compatible schema evolution.

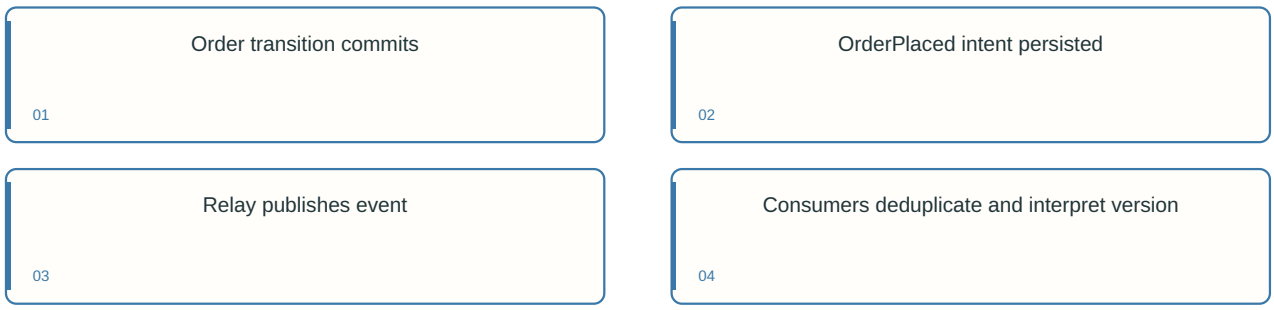


FIG. 7.2.4 **Domain events · worked snapshots**

Order transition commits · OrderPlaced intent persisted · Relay publishes event · Consumers deduplicate and interpret version. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which evidence would justify extracting payments from a modular monolith while retaining the same business invariants?

Let orders request a payment through a contract; disallow direct imports of the payment storage adapter or writes to its private table. A test should enforce a meaningful dependency rule rather than every incidental class name. Show how a provider adapter can change while the order rule remains stable.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

> CHECK THE BOUNDARY
>
> If state and publication intent must agree, persist them atomically through an outbox or another justified mechanism.

Carry forward: Order transition commits · OrderPlaced intent persisted · Relay publishes event · Consumers deduplicate and interpret version.

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Keep business policy inside

Prerequisites: 7.2 · Model a bounded context 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an online shopping application first as a modular monolith, then examine the conditions under which selected domains should become independent services.

This chapter examines keep business policy inside through concrete mechanisms, state snapshots, failure cases, and alternatives.

7.3.1 · Dependency rules and module boundaries

Dependency rules constrain which modules can know about others. A payments module should expose a payment contract rather than allow orders to update its tables directly. Enforce these rules with import boundaries, schema permissions, and architecture tests where useful. Cycles often reveal unclear coordination. A shared utility can become a disguised common domain dependency; prefer explicit data translations when two contexts deliberately have different meanings for apparently similar concepts.

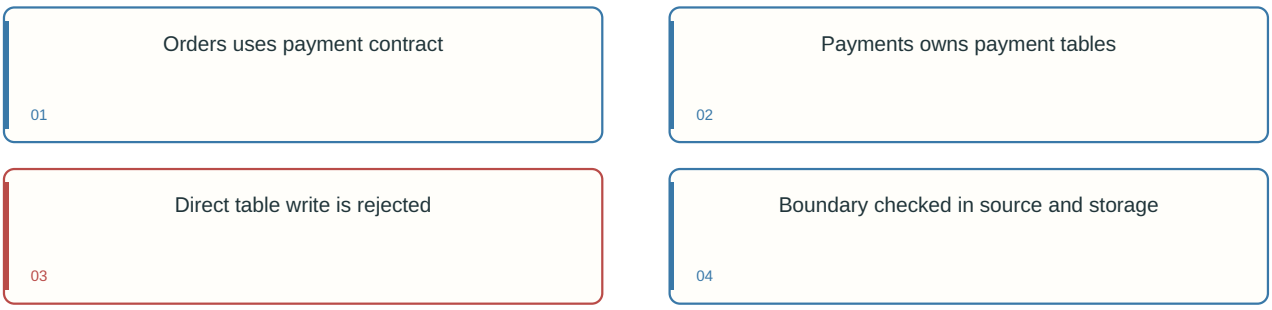


FIG. 7.3.1 **Dependency rules and module boundaries · worked snapshots**

Orders uses payment contract · Payments owns payment tables · Direct table write is rejected · Boundary checked in source and storage. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.2 · Business logic vs orchestration vs infrastructure

Business logic determines allowed outcomes, orchestration sequences collaborators, and infrastructure performs technical effects. “A refunded order cannot ship” is business logic; coordinating a refund request and status update is orchestration; calling the provider is infrastructure. One function may initially perform all three, but naming the distinctions reveals where tests and failure handling belong. Do not move a business rule into a message broker configuration merely because orchestration happens to be asynchronous.

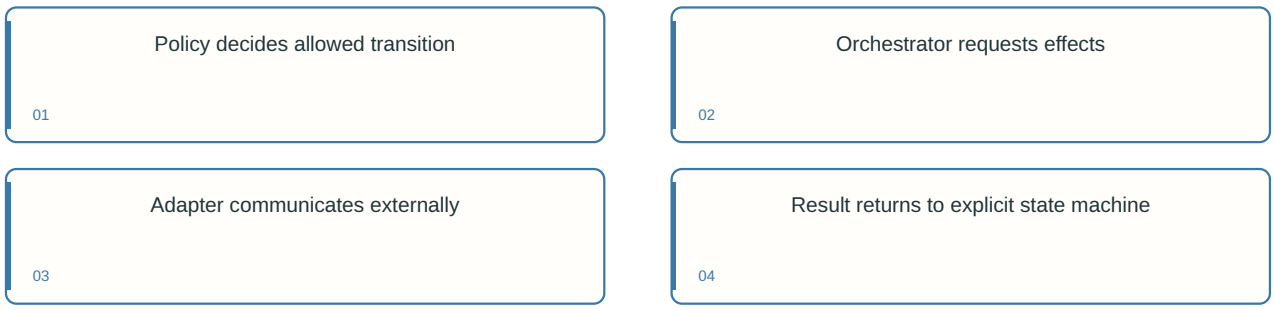


FIG. 7.3.2 **Business logic vs orchestration vs infrastructure · worked snapshots**

Policy decides allowed transition · Orchestrator requests effects · Adapter communicates externally · Result returns to explicit state machine. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.3 · Controllers, services, repositories, adapters

Controllers translate transport, services coordinate application behavior, repositories mediate data, and adapters implement boundary protocols. These roles are conventions, not mandatory suffixes on class names. An application service can be a function. A domain service has a different purpose from a network-deployed service. Review behavior rather than the directory tree: if a controller owns transaction retries and pricing rules, creating a services folder has not actually separated responsibilities.

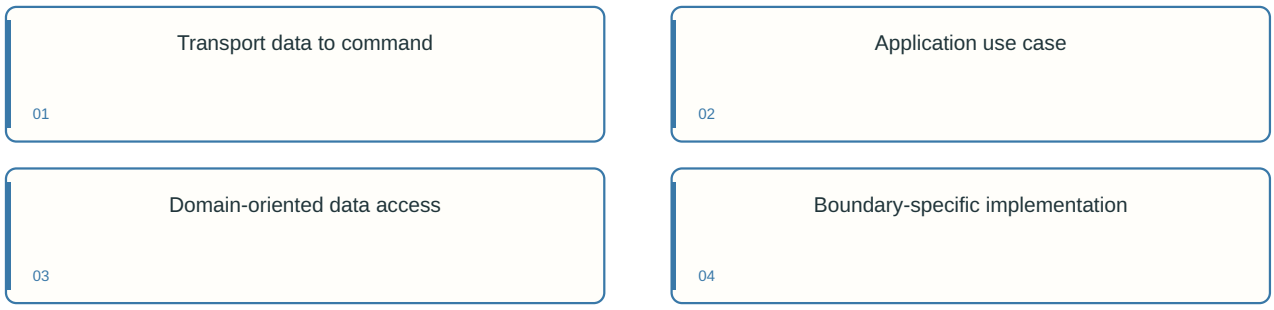


FIG. 7.3.3 **Controllers, services, repositories, adapters · worked snapshots**

Transport data to command · Application use case · Domain-oriented data access · Boundary-specific implementation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.4 · Anti-corruption layers

An anti-corruption layer translates a foreign model into the local model so external terminology and quirks do not spread. A legacy provider may call a pending authorization “settled”; the payment context must translate that carefully instead of accepting the label as its own settlement state. The layer adds mappings and reconciliation work. Preserve unmapped states explicitly and fail safely rather than silently coercing every unknown value into success.

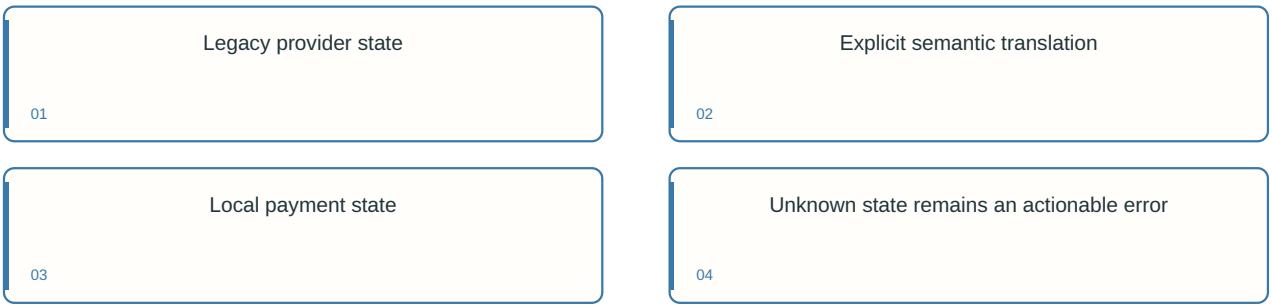


FIG. 7.3.4 **Anti-corruption layers · worked snapshots**

Legacy provider state · Explicit semantic translation · Local payment state · Unknown state remains an actionable error. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.5 · Event-driven architecture

Event-driven architecture connects components through events with independent processing. It reduces synchronous coupling in some paths while creating temporal coupling, replay, ordering, and observability obligations. An order can become accepted before a receipt is delivered, but inventory and payment requirements still determine which steps can be deferred. A broker does not supply end-to-end business reliability automatically. Make event ownership, deduplication, persistence, and reconciliation part of the architecture, not an afterthought.

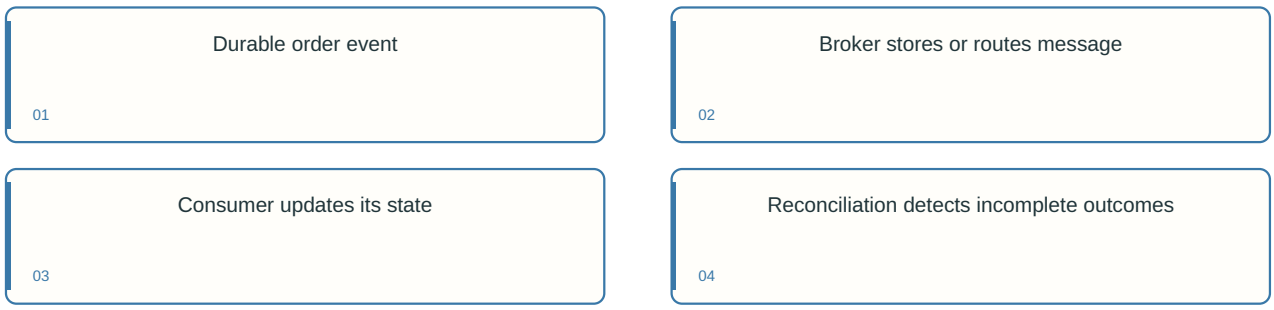


FIG. 7.3.5 **Event-driven architecture · worked snapshots**

Durable order event · Broker stores or routes message · Consumer updates its state · Reconciliation detects incomplete outcomes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which evidence would justify extracting payments from a modular monolith while retaining the same business invariants?

Let orders request a payment through a contract; disallow direct imports of the payment storage adapter or writes to its private table. A test should enforce a meaningful dependency rule rather than every incidental class name. Show how a provider adapter can change while the order rule remains stable.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A broker does not supply end-to-end business reliability automatically.

Carry forward: Durable order event · Broker stores or routes message · Consumer updates its state · Reconciliation detects incomplete outcomes.

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Evolution without a distributed monolith

Prerequisites: 7.3 · Keep business policy inside 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an online shopping application first as a modular monolith, then examine the conditions under which selected domains should become independent services.

This chapter examines evolution without a distributed monolith through concrete mechanisms, state snapshots, failure cases, and alternatives.

7.4.1 · Serverless and function-oriented architectures

Serverless functions can provide managed invocation and elastic execution, but do not remove systems design. Time limits, cold starts, concurrency caps, retries, identity, and durable state all affect behavior. Keep function-local memory as an optimization unless the platform explicitly guarantees more. A function processing a payment event must deduplicate effects across retries. Function-oriented organization can fit small independently triggered tasks; long workflows usually need an explicit durable state owner or workflow engine.

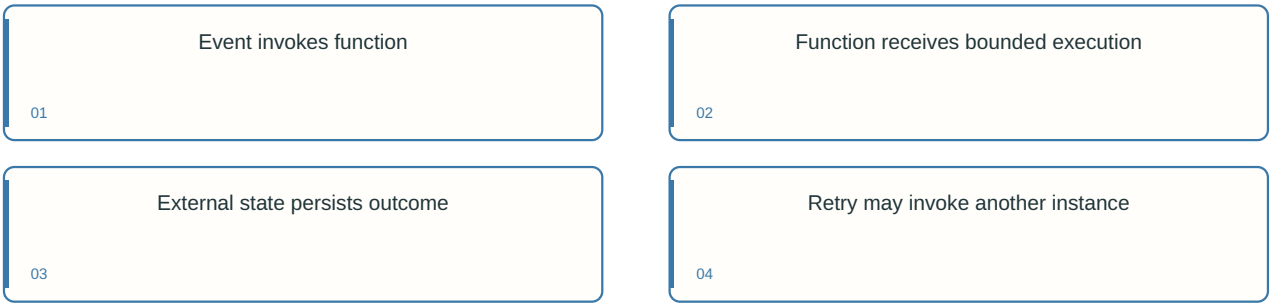


FIG. 7.4.1 **Serverless and function-oriented architectures · worked snapshots**

Event invokes function · Function receives bounded execution · External state persists outcome · Retry may invoke another instance. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.2 · Evolutionary architecture

Evolutionary architecture changes in response to evidence while protecting important properties. Add fitness checks for module boundaries, migration compatibility, latency budgets, or tenant isolation so frequent changes do not silently erode them. A check must express a useful property rather than freeze incidental implementation. An ADR records why a boundary exists and what observation should trigger reconsideration. Evolution is not constant rewriting; many successful changes remove complexity rather than introduce another platform.

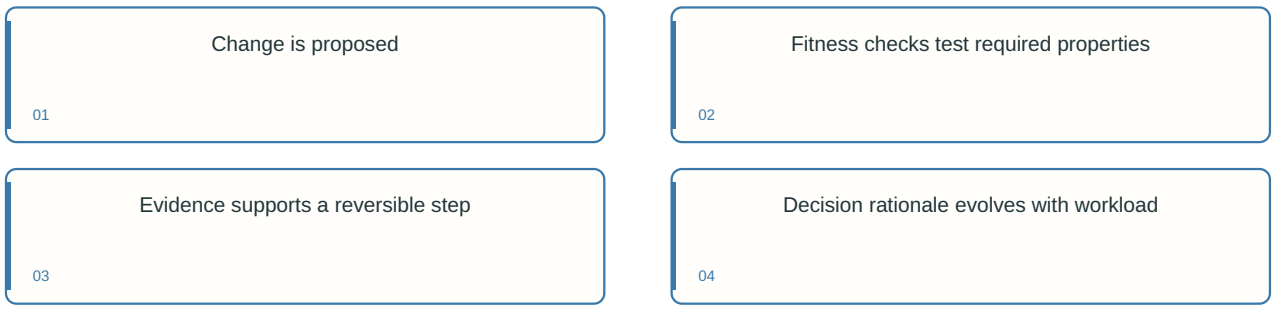


FIG. 7.4.2 **Evolutionary architecture · worked snapshots**

Change is proposed · Fitness checks test required properties · Evidence supports a reversible step · Decision rationale evolves with workload. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.3 · Technical vs business boundaries

Technical boundaries group by mechanism, such as controllers or databases; business boundaries group coherent capabilities, such as orders or fulfillment. Both can coexist at different levels. Splitting a company into an “SQL service” and “validation service” often makes every business change cross the network. Start with ownership of decisions and data, then choose internal technical layers. A bounded context is a modeling boundary; it is not automatically equivalent to a team, repository, or deployed service.

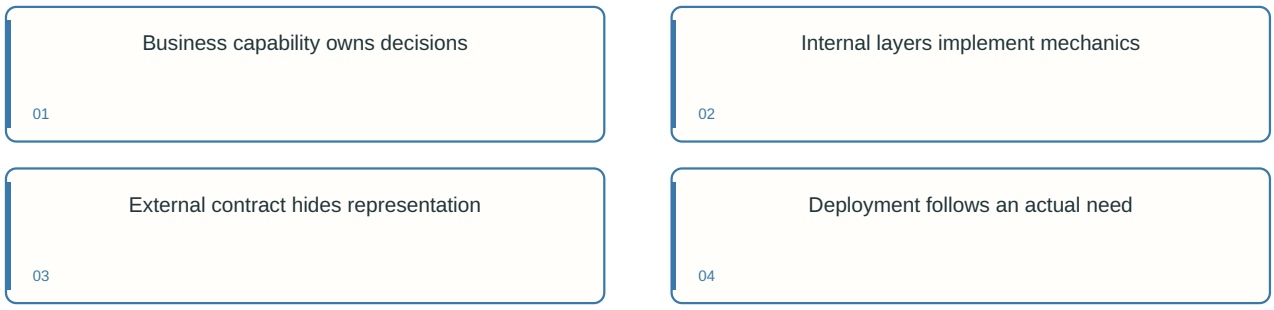


FIG. 7.4.3 **Technical vs business boundaries · worked snapshots**

Business capability owns decisions · Internal layers implement mechanics · External contract hides representation · Deployment follows an actual need. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.4 · Distributed monolith antipattern

A distributed monolith has several processes but little independent change or operation. Symptoms include synchronous chains for every action, shared writable schemas, coordinated releases, and failures cascading across services. Moving calls onto a network without clarifying ownership makes debugging harder and adds partial failure. Consolidating unstable boundaries into a modular deployment can be an improvement. Alternatively, redesign contracts, data ownership, and asynchronous coordination before claiming independent services have been achieved.

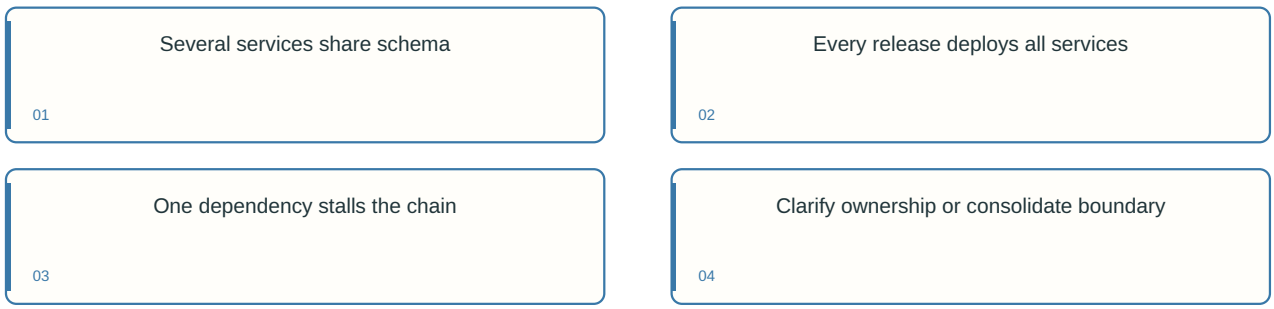


FIG. 7.4.4 **Distributed monolith antipattern · worked snapshots**

Several services share schema · Every release deploys all services · One dependency stalls the chain · Clarify ownership or consolidate boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.5 · When not to use microservices

Avoid microservices when the domain is still changing rapidly, the team cannot operate independent services safely, or scale and isolation do not require them. A modular monolith with measured bottlenecks and tested backups can be a stronger starting point. This is not a claim that microservices are inherently wrong: stable capabilities with distinct traffic and ownership can justify them. Evaluate the complete cost of transactions, deployment, telemetry, security, and incident response before extraction.

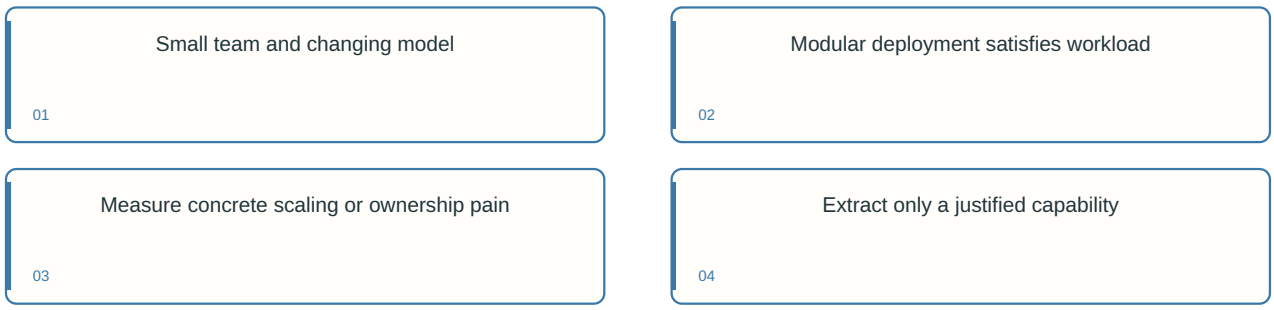


FIG. 7.4.5 **When not to use microservices · worked snapshots**

Small team and changing model · Modular deployment satisfies workload · Measure concrete scaling or ownership pain · Extract only a justified capability. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PSEUDOCODE · MODULE CONTRACT

```
place_order(command):
    priced = catalog.quote(command.items)
    reservation = inventory.reserve(command.items)
    payment = payments.authorize(command.operation_id, priced.total)
    return orders.record_outcomes(priced, reservation, payment)
# Modules expose contracts; they do not update each other's private tables.
```

This orchestration does not make three remote services one atomic transaction. In a modular monolith some local updates may share a deliberately owned transaction; external effects still need a workflow and reconciliation.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · Which evidence would justify extracting payments from a modular monolith while retaining the same business invariants?

Let orders request a payment through a contract; disallow direct imports of the payment storage adapter or writes to its private table. A test should enforce a meaningful dependency rule rather than

every incidental class name. Show how a provider adapter can change while the order rule remains stable.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

This is not a claim that microservices are inherently wrong: stable capabilities with distinct traffic and ownership can justify them.

Carry forward: Small team and changing model · Modular deployment satisfies workload · Measure concrete scaling or ownership pain · Extract only a justified capability.

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

PART 2 / UNIT 08

API Design and Communication Contracts

The running example

Design a payment API with create-payment, payment-status, refunds, idempotency protection, duplicate requests, and failed network responses.

Unit 8 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

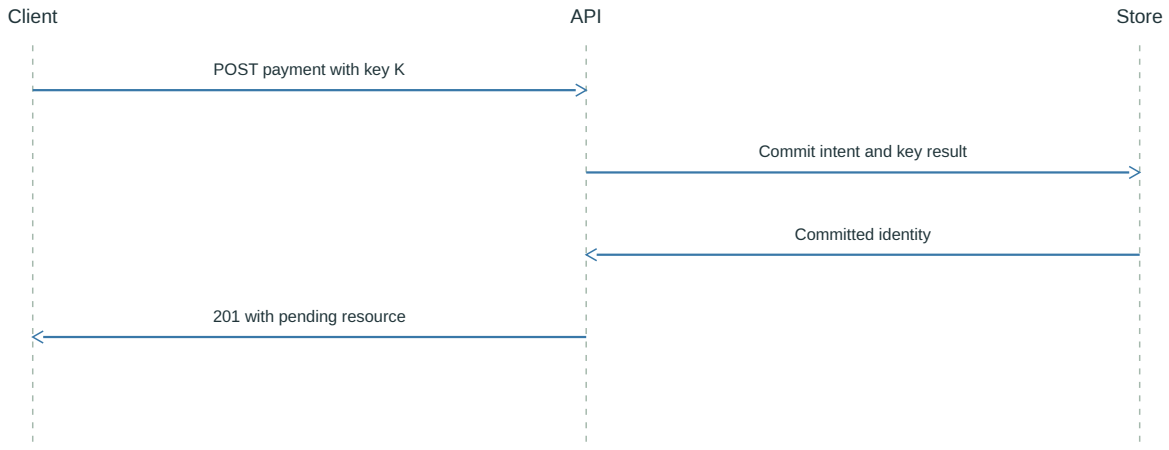
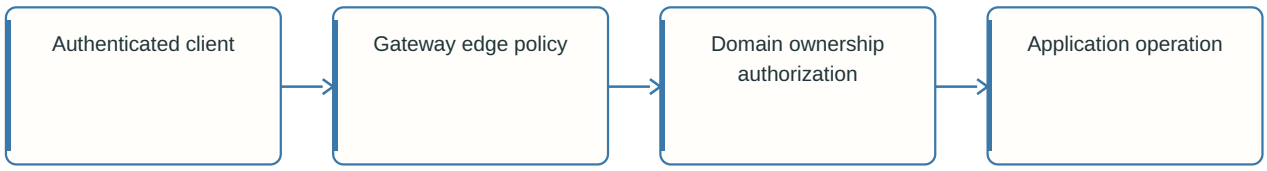


FIG. 8.0.1 **REST request flow**

HTTP creation and financial completion are different outcomes.

Why this view: Follow explicit message ordering and locate the acknowledgment or effect boundary. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 8.0.2 **API gateway topology**

An edge gateway cannot decide every resource-specific permission without the appropriate domain context.

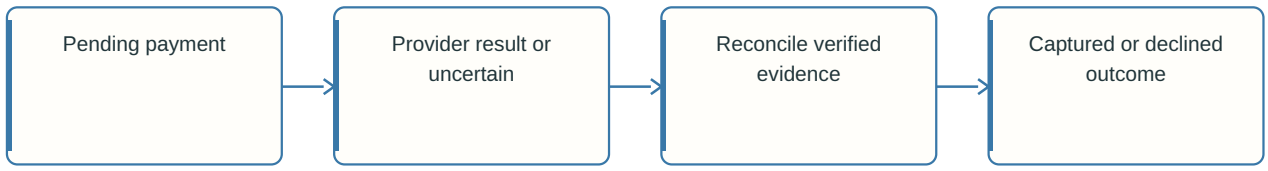
Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Contract	Caller observes	State owner
Synchronous completion	Waits for final result	Operation still may be uncertain
Asynchronous acceptance	Receives operation ID	Durable workflow
Status query	Reads current progress	Authoritative or stated-freshness view

FIG. 8.0.3 **synchronous vs asynchronous API**

An accepted operation needs durable ownership beyond the request lifetime.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 8.0.4 **payment state-machine diagram**

Uncertain is not synonymous with declined; refunds form a separately tracked lifecycle.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Model resources and messages

Prerequisites: Introduction to System Design, Networking Foundations for System Design 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a payment API with create-payment, payment-status, refunds, idempotency protection, duplicate requests, and failed network responses.

This chapter examines model resources and messages through concrete mechanisms, state snapshots, failure cases, and alternatives.

8.1.1 · API fundamentals and interface contracts

An API is a communication contract, including inputs, outputs, side effects, error semantics, identity, deadlines, and compatibility. For create-payment, specify integer minor units and currency, distinguish a declined payment from an uncertain transport outcome, and identify who may access its status. A schema alone cannot describe whether a repeated request charges twice. Write example success, validation failure, duplicate, and lost-response histories before selecting a transport framework.

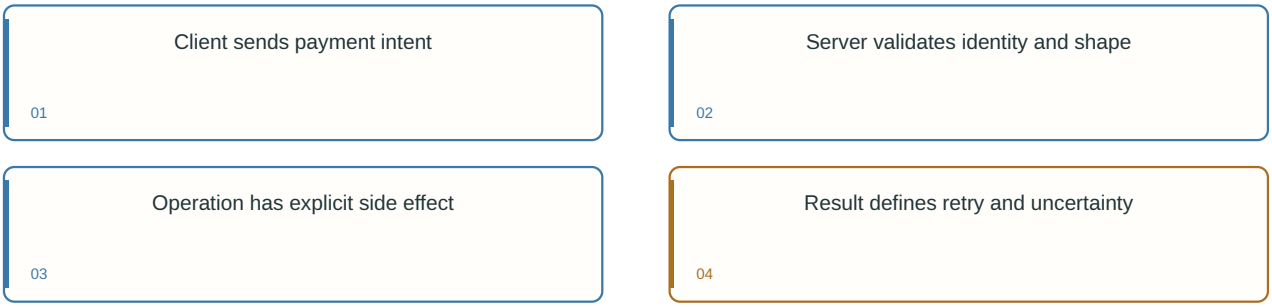


FIG. 8.1.1 **API fundamentals and interface contracts · worked snapshots**

Client sends payment intent · Server validates identity and shape · Operation has explicit side effect · Result defines retry and uncertainty. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.2 · REST, GraphQL, gRPC, RPC

REST organizes resource-oriented interactions using HTTP semantics; GraphQL exposes a typed query graph; gRPC commonly offers schema-defined RPC with streaming; RPC generally models operations. None is inherently correct for every boundary. GraphQL can reduce over-fetching but needs query-cost control and field authorization. RPC can make domain actions clear but still needs compatibility and retry semantics. Compare client needs, caching, tooling, latency, and evolution using the same operation, not only request syntax.

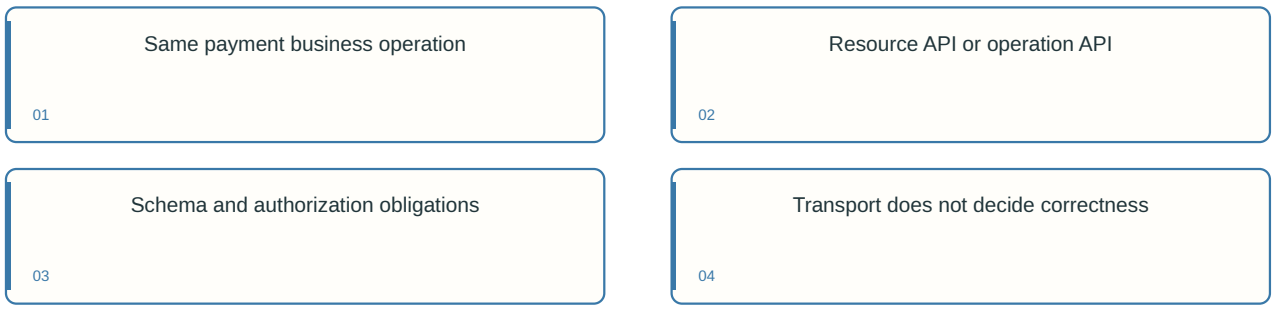


FIG. 8.1.2 **REST, GraphQL, gRPC, RPC · worked snapshots**

Same payment business operation · Resource API or operation API · Schema and authorization obligations · Transport does not decide correctness. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.3 · Resource modeling

Resources should reflect stable business identities. POST /payments creates an operation; GET /payments/{id} reports its state; POST /payments/{id}/refunds creates a separately tracked refund. Hiding every action under one generic /execute endpoint loses useful semantics, but forcing every domain action into CRUD can also obscure behavior. A refund resource needs its own amount, authorization, idempotency identity, and lifecycle. Use ownership and state transitions to guide the model.

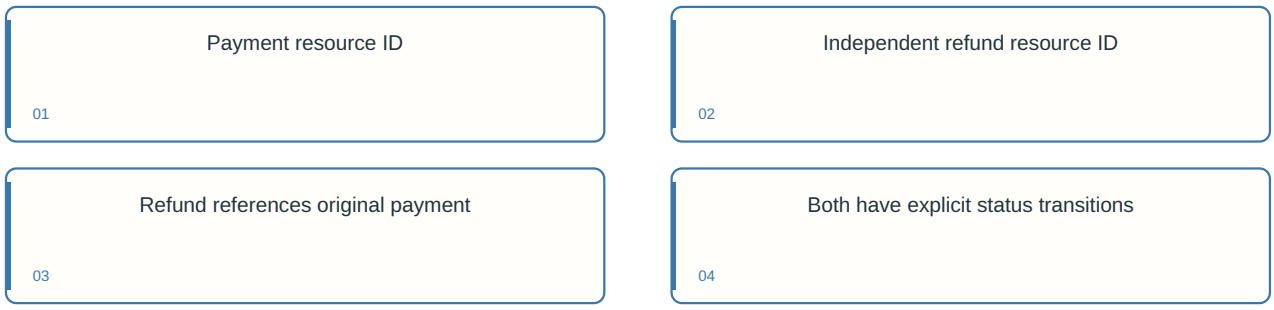


FIG. 8.1.3 **Resource modeling · worked snapshots**

Payment resource ID · Independent refund resource ID · Refund references original payment · Both have explicit status transitions. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.4 · Request/response schemas

Validate structural shape and business meaning separately. A syntactically valid amount of −100 may violate the payment operation; an unknown field can be ignored or rejected according to compatibility policy. Bound payload size, collection lengths, and accepted encodings before expensive processing. Return machine-readable error codes with safe messages and a request identifier. Do not echo secrets or provider internals in validation errors. Schema evolution must consider clients that deserialize strictly as well as those that ignore extensions.

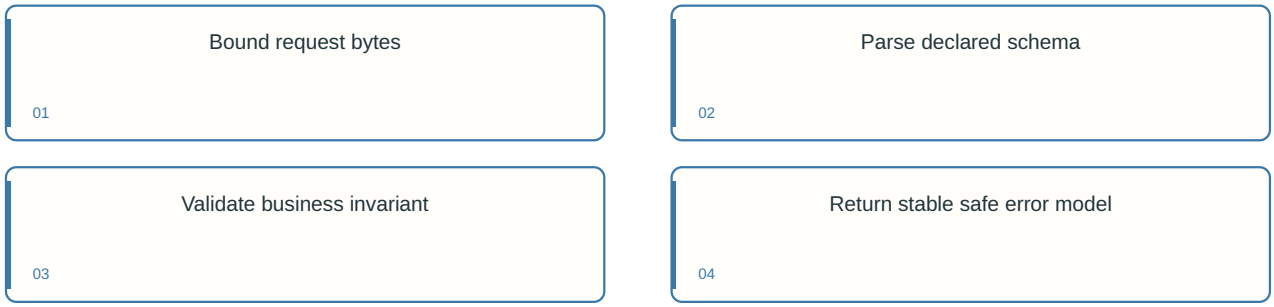


FIG. 8.1.4 **Request/response schemas · worked snapshots**

Bound request bytes · Parse declared schema · Validate business invariant · Return stable safe error model. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.5 · Status codes and error models

HTTP status codes describe the protocol outcome, while an operation body may describe asynchronous business state. 201 can identify a created payment resource that is still pending; 202 means accepted for processing, not financially approved. A timeout has no definitive business meaning. Use stable application codes for validation, conflict, authorization, and retry advice. Do not translate every provider error into a generic 500 if clients need to distinguish a correctable request from an uncertain submitted operation.

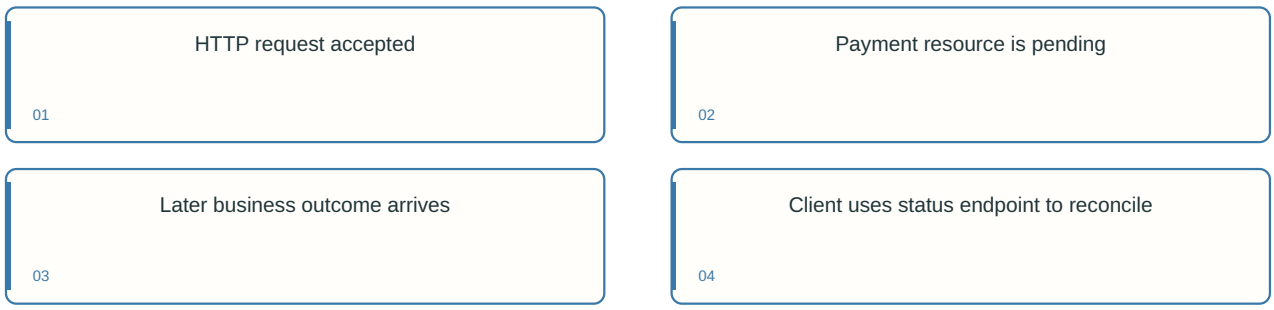


FIG. 8.1.5 **Status codes and error models · worked snapshots**

HTTP request accepted · Payment resource is pending · Later business outcome arrives · Client uses status endpoint to reconcile. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.6 · Pagination, filtering, sorting

Pagination bounds work and response size. Define stable sorting, tie-breakers, allowed filters, maximum page size, and authorization at the query boundary. Returning “all payments” can exhaust memory and expose data even when the endpoint looks simple. Sorting by created_at alone is ambiguous for equal timestamps; append a unique identifier. Offset and cursor approaches differ under concurrent insertion, and neither automatically guarantees a consistent snapshot of a changing collection.

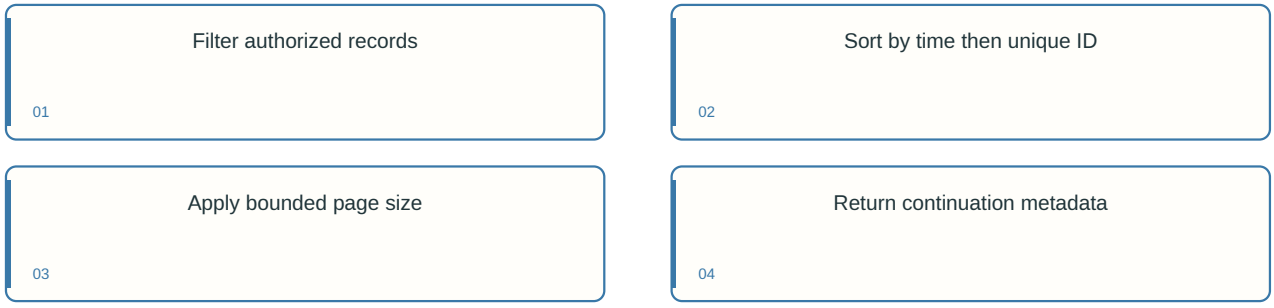


FIG. 8.1.6 **Pagination, filtering, sorting · worked snapshots**

Filter authorized records · Sort by time then unique ID · Apply bounded page size · Return continuation metadata. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can a public API offer a permanent idempotency promise with finite storage? Define a defensible contract.

Use keyset_page() for ordered pairs and test insertion before a page boundary. For idempotency, bind actor, key, fingerprint, state, and result atomically; reject a mismatched payload. A plain dictionary checked before a separate insert does not protect concurrent requests or restart recovery.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Sorting by created_at alone is ambiguous for equal timestamps; append a unique identifier.

Carry forward: Filter authorized records · Sort by time then unique ID · Apply bounded page size · Return continuation metadata.

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Compatibility & idempotency

Prerequisites: 8.1 · Model resources and messages 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a payment API with create-payment, payment-status, refunds, idempotency protection, duplicate requests, and failed network responses.

This chapter examines compatibility & idempotency through concrete mechanisms, state snapshots, failure cases, and alternatives.

8.2.1 · Cursor vs offset pagination

Offset pagination skips a number of rows and is easy to express, but inserts before the offset can create duplicates or omissions between pages. Keyset cursors encode the last ordered key, such as (created_at,id), to continue after that position. They avoid some skip cost and shift anomalies but still need a consistency contract for changing rows. Sign or validate opaque cursors and bind them to the applicable filters and tenant context.

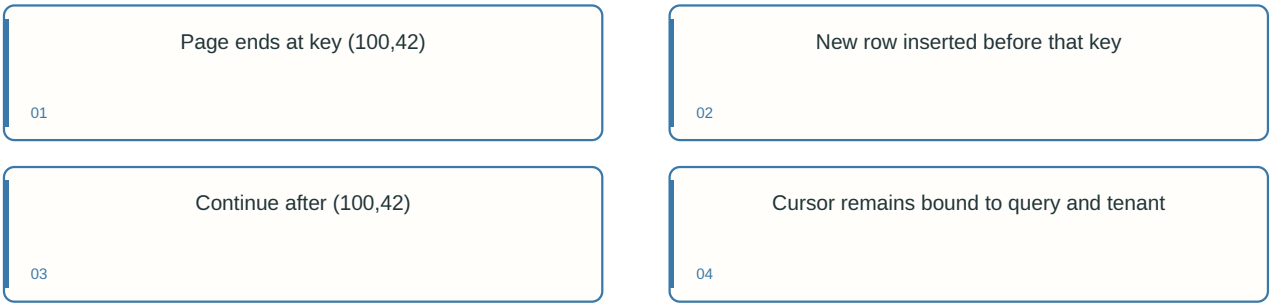


FIG. 8.2.1 **Cursor vs offset pagination · worked snapshots**

Page ends at key (100,42) · New row inserted before that key · Continue after (100,42) · Cursor remains bound to query and tenant. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.2.2 · API versioning and compatibility

Compatible evolution preserves supported client behavior. Adding an optional field may be safe for tolerant clients but break strict validators or exhaustive enum handling. Changing an amount from minor to major units is breaking even if its type remains numeric. Use versioned semantics, overlap windows, telemetry on old consumers, and contract tests. A new URL version is only one technique; it does not remove the obligation to migrate clients and preserve stored event interpretation.

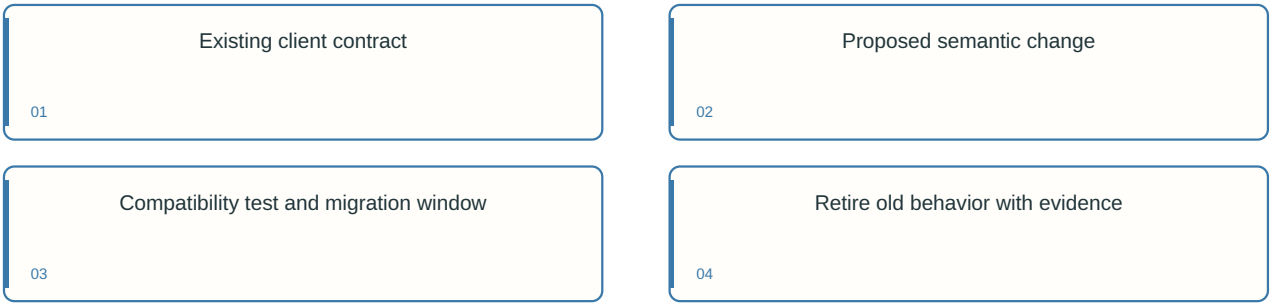


FIG. 8.2.2 **API versioning and compatibility · worked snapshots**

Existing client contract · Proposed semantic change · Compatibility test and migration window · Retire old behavior with evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.2.3 · Idempotency and idempotency keys

An idempotency key identifies one logical operation across attempts. Store its owner, request fingerprint, execution state, and committed result under a uniqueness and atomicity rule. The same key with a different payload must conflict. If the provider may have charged before a timeout, use its stable operation key or reconcile before resubmitting. Retention and key reuse rules matter. Local deduplication does not provide exactly-once external execution across an uncooperative provider boundary.

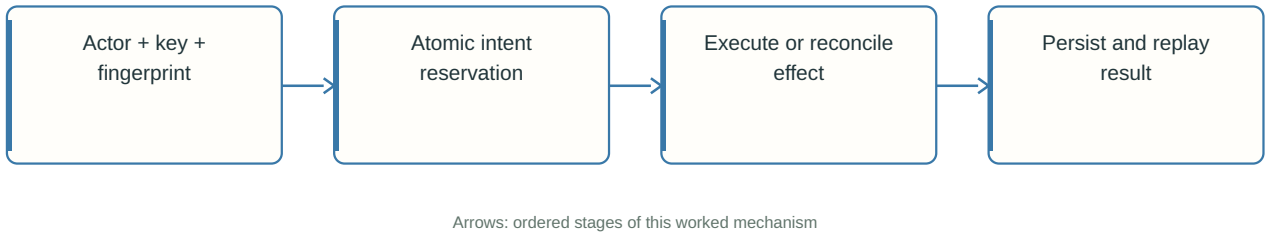


FIG. 8.2.3 **A payment retry follows durable operation identity**

The same key and payload return the established operation; a different payload conflicts. Local identity does not encompass arbitrary provider effects.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

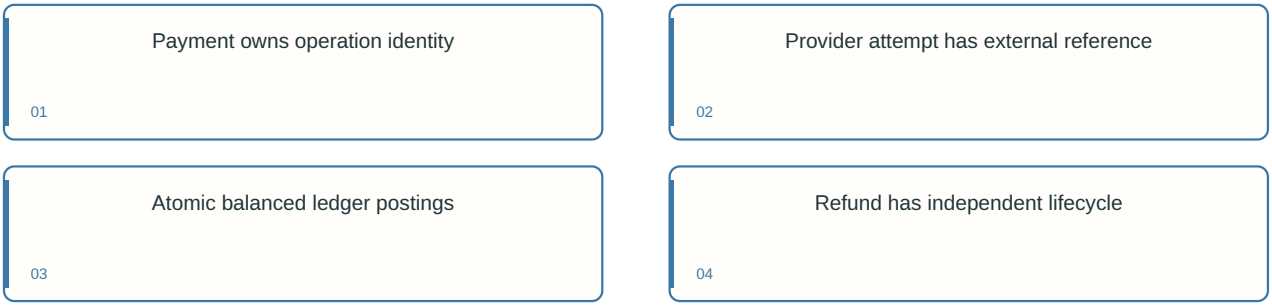


FIG. 8.2.4 **The running system · invariant and recovery review**

Review which event means financial success, who owns the ledger invariant, and what recovery does after deduplication history is restored behind provider effects. Test charge-then-crash, callback-before-response, duplicate settlement, partial refund, and restore from an older snapshot. Verify no duplicate local postings and no unsupported claim that an unknown provider outcome is safely declined.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can a public API offer a permanent idempotency promise with finite storage? Define a defensible contract.

Use keyset_page() for ordered pairs and test insertion before a page boundary. For idempotency, bind actor, key, fingerprint, state, and result atomically; reject a mismatched payload. A plain dictionary checked before a separate insert does not protect concurrent requests or restart recovery.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Retention and key reuse rules matter.

Carry forward: Key K and payload hash H · Atomically reserve operation K · Execute or reconcile provider operation · Persist and replay the same result.

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Control access, load & long operations

Prerequisites: 8.2 · Compatibility & idempotency 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a payment API with create-payment, payment-status, refunds, idempotency protection, duplicate requests, and failed network responses.

This chapter examines control access, load & long operations through concrete mechanisms, state snapshots, failure cases, and alternatives.

8.3.1 · Authentication vs authorization at API boundaries

Authentication establishes the caller's identity under a trust model; authorization decides whether that caller may perform this operation on this resource. Derive tenant context from authenticated membership, not a freely supplied tenant_id alone. GET /payments/42 must check resource ownership even if payment identifiers are hard to guess. Coarse gateway checks complement, rather than replace, domain authorization. Validate service-to-service credentials as well as user credentials when requests cross internal boundaries.

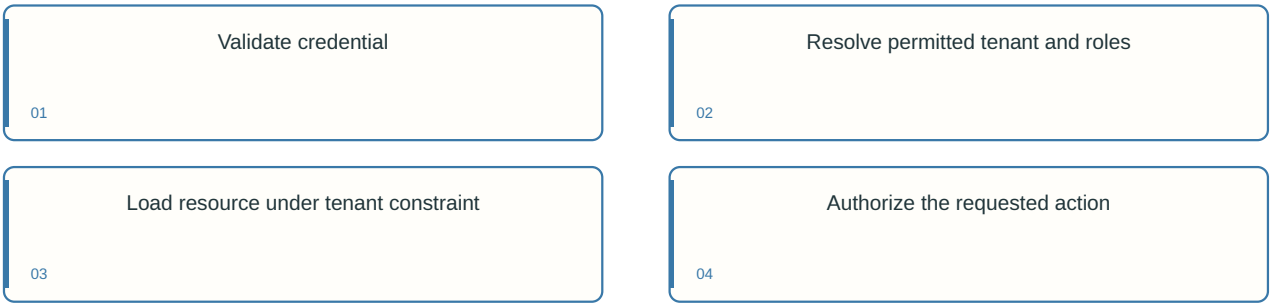


FIG. 8.3.1 **Authentication vs authorization at API boundaries · worked snapshots**

Validate credential · Resolve permitted tenant and roles · Load resource under tenant constraint · Authorize the requested action. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.2 · Rate limits and quotas

A rate limit controls request pace; a quota limits accumulated use over a period or resource dimension. A payment API might limit expensive validation attempts and separately cap daily exports. Define keys, windows, burst behavior, retry advice, and failure policy. One global limit can let a noisy tenant harm everyone, while per-user limits can miss coordinated abuse. Distributed enforcement needs atomic state or an explicit approximation and regional allocation contract.

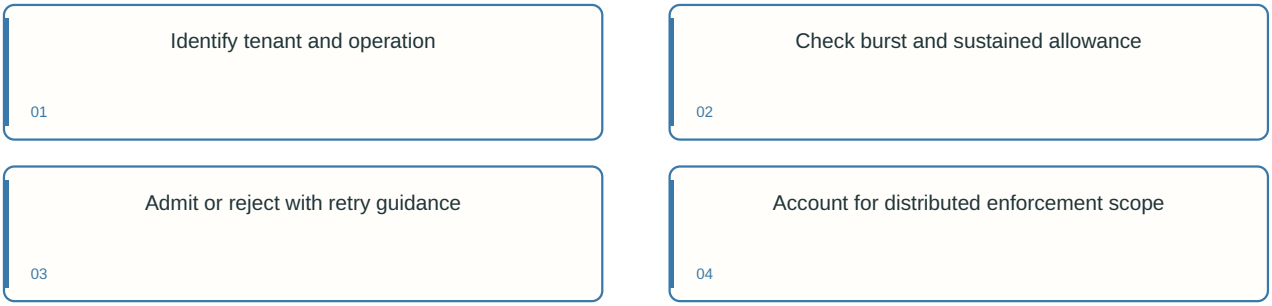


FIG. 8.3.2 **Rate limits and quotas · worked snapshots**

Identify tenant and operation · Check burst and sustained allowance · Admit or reject with retry guidance · Account for distributed enforcement scope. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.3 · Retries and timeouts

Retries should fit an overall deadline and target transient, safe-to-repeat work. Budget time for connection acquisition, attempts, and backoff; propagate remaining time downstream. Exponential backoff with jitter reduces synchronized retries but does not create capacity. Avoid retries independently at every layer: three attempts at each of four layers can amplify one intent into 81 attempts. For side effects, retries need stable identity and reconciliation of uncertain outcomes.

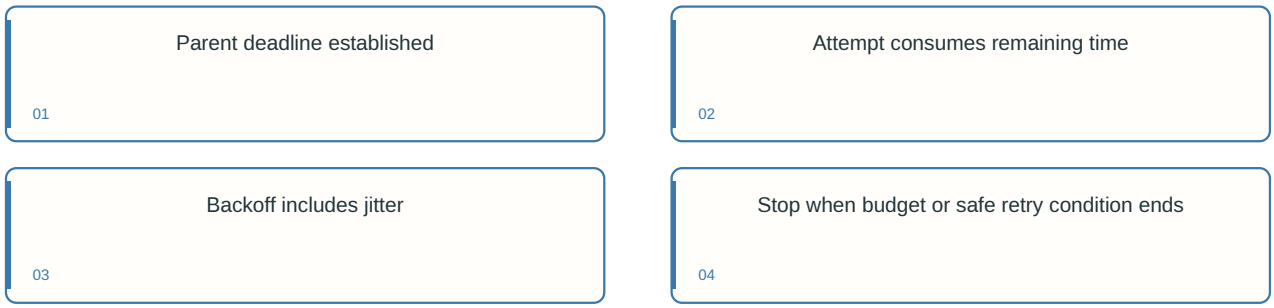


FIG. 8.3.3 **Retries and timeouts · worked snapshots**

Parent deadline established · Attempt consumes remaining time · Backoff includes jitter · Stop when budget or safe retry condition ends. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.4 · Streaming APIs and long-running operations

A long-running operation can return an operation resource and let clients poll or subscribe for progress. Streaming returns a sequence with explicit framing, cancellation, buffering, and resume policy. A disconnected client does not necessarily cancel the business job; define who owns it after submission. Persist required job state outside request-local memory. Avoid holding a database transaction open for the lifetime of a slow stream; read bounded pages or transfer a deliberate streaming resource owner.

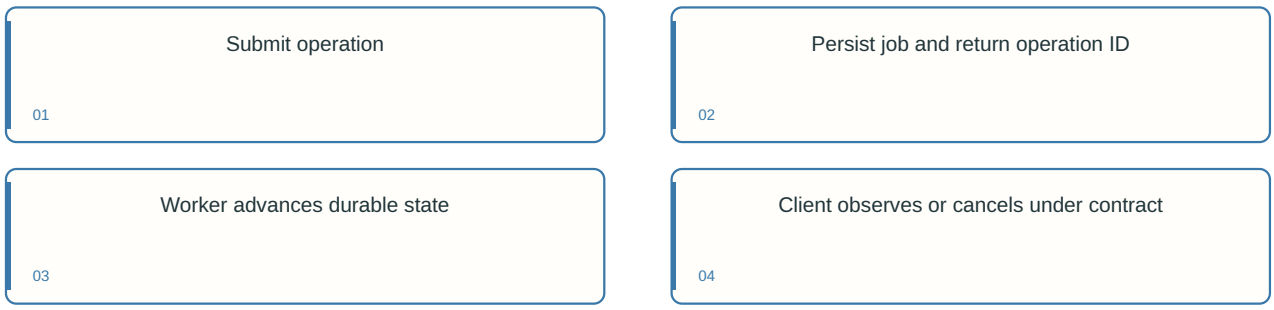


FIG. 8.3.4 **Streaming APIs and long-running operations · worked snapshots**

Submit operation · Persist job and return operation ID · Worker advances durable state · Client observes or cancels under contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.5 · Webhooks and delivery acknowledgments

A webhook is an asynchronous request to another party. Sign it, include a stable event identifier and timestamp policy, and accept acknowledgments according to a documented delivery contract. A receiver should commit its inbox or effect before acknowledging if persistence is required. A sender can retry a lost acknowledgment, so consumers must deduplicate. A 2xx response proves only what the receiving contract promises, not that every downstream business task has completed.

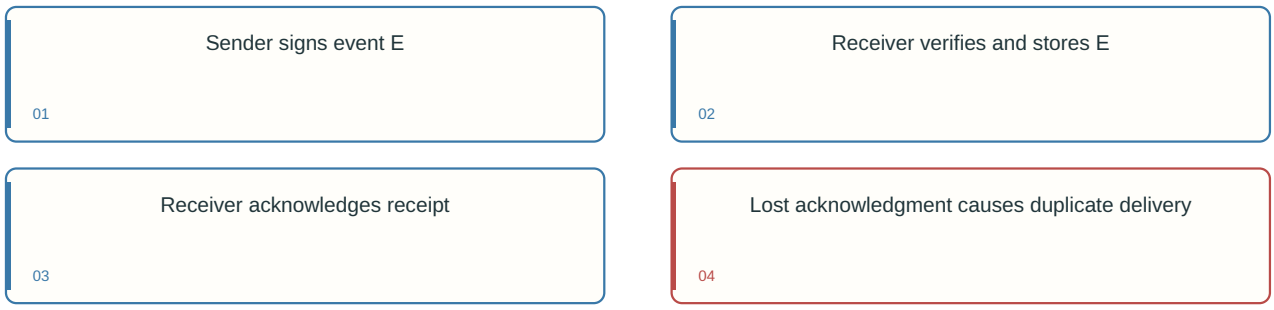


FIG. 8.3.5 **Webhooks and delivery acknowledgments · worked snapshots**

Sender signs event E · Receiver verifies and stores E · Receiver acknowledges receipt · Lost acknowledgment causes duplicate delivery. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can a public API offer a permanent idempotency promise with finite storage? Define a defensible contract.

Use keyset_page() for ordered pairs and test insertion before a page boundary. For idempotency, bind actor, key, fingerprint, state, and result atomically; reject a mismatched payload. A plain dictionary checked before a separate insert does not protect concurrent requests or restart recovery.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A sender can retry a lost acknowledgment, so consumers must deduplicate.

Carry forward: Sender signs event E · Receiver verifies and stores E · Receiver acknowledges receipt · Lost acknowledgment causes duplicate delivery.

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Evolve a public contract

Prerequisites: 8.3 · Control access, load & long operations 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a payment API with create-payment, payment-status, refunds, idempotency protection, duplicate requests, and failed network responses.

This chapter examines evolve a public contract through concrete mechanisms, state snapshots, failure cases, and alternatives.

8.4.1 · Schema evolution

Schema evolution applies to stored data and events as well as synchronous APIs. Preserve meanings, defaults, field identity, and unknown-value behavior. A consumer replaying a year of events may encounter every historical version, so deleting a decoder can break recovery even if current traffic looks healthy. Use explicit versions or upcasters with tested historical samples. An additive field is not harmless if it changes a signature calculation, a uniqueness rule, or a required business decision.

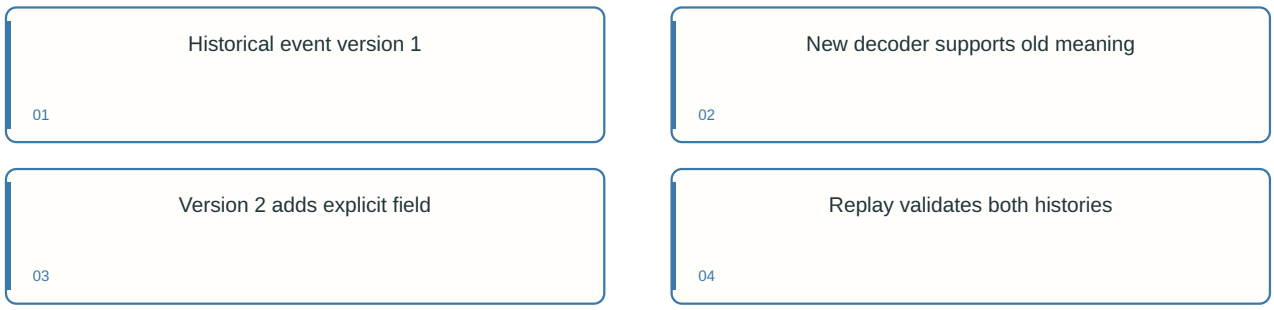


FIG. 8.4.1 **Schema evolution · worked snapshots**

Historical event version 1 · New decoder supports old meaning · Version 2 adds explicit field · Replay validates both histories. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.2 · Consumer-driven contracts

Consumer-driven contracts record assumptions that real consumers make about a provider. Run provider verification against those contracts before release, while also testing behavior that schemas miss. A fixture asserting that status is a string does not protect a client that assumes every returned status is final. Contracts can become brittle if they freeze irrelevant formatting. Focus on observable obligations and maintain representative error, authorization, pagination, and retry scenarios alongside success examples.

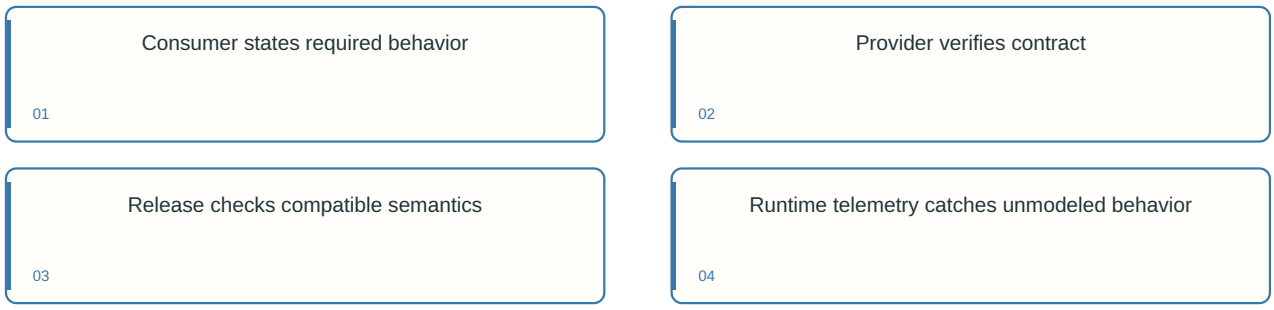


FIG. 8.4.2 **Consumer-driven contracts · worked snapshots**

Consumer states required behavior · Provider verifies contract · Release checks compatible semantics · Runtime telemetry catches unmodeled behavior. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.3 · API gateways and BFF

A gateway provides shared edge concerns; a Backend for Frontend adapts APIs for a particular client experience. A mobile BFF can aggregate necessary data and reduce chatty requests, but adds latency, capacity, and ownership concerns. Keep resource-level authorization in the systems that understand ownership. Avoid a universal aggregation service that every product must change together. Cache only representations whose key, identity scope, and freshness policy are well defined.

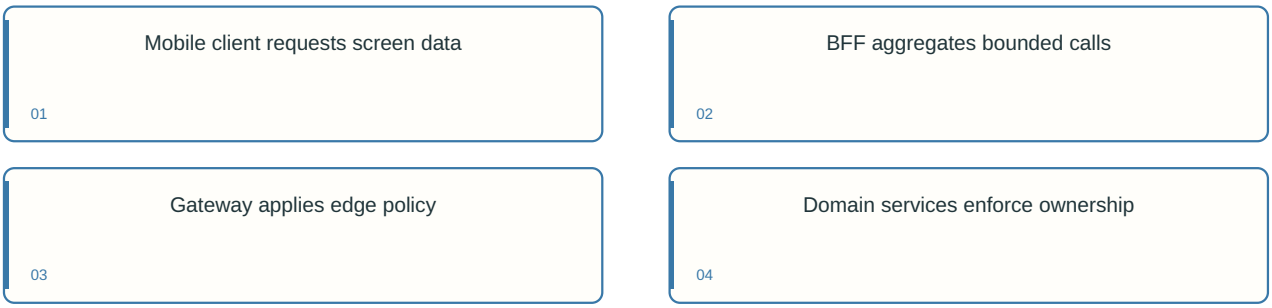


FIG. 8.4.3 **API gateways and BFF · worked snapshots**

Mobile client requests screen data · BFF aggregates bounded calls · Gateway applies edge policy · Domain services enforce ownership. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.4 · Public vs internal APIs

An internal API still crosses a trust, version, or failure boundary. Internal clients may have different release schedules and can send malformed or excessive requests. A public API generally needs stronger compatibility commitments, discoverability, abuse control, and support processes, but internal status does not justify skipping authentication or invariants. Document audience and lifetime explicitly. A service initially called “private” often becomes a dependency whose accidental behavior is difficult to change.

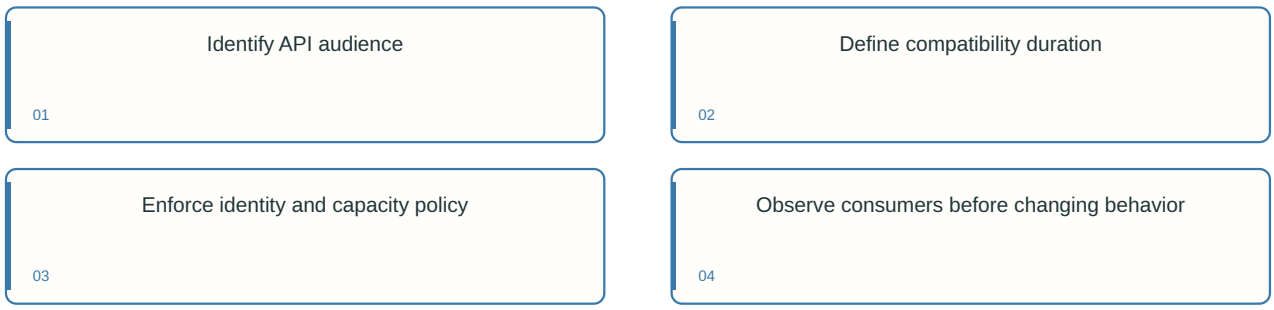


FIG. 8.4.4 **Public vs internal APIs · worked snapshots**

Identify API audience · Define compatibility duration · Enforce identity and capacity policy · Observe consumers before changing behavior. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.5 · API documentation and discoverability

Documentation should explain complete operation histories: request examples, errors, authorization, pagination, timeouts, idempotency, and asynchronous status. Machine-readable schemas help generate clients, while prose defines semantics those schemas cannot express. A runnable sandbox must distinguish simulated effects from real charges. Track documentation versions with implementation, and test key examples. Discoverability is practical: users should know how to find the right operation, determine its scope, and recover when the happy path fails.

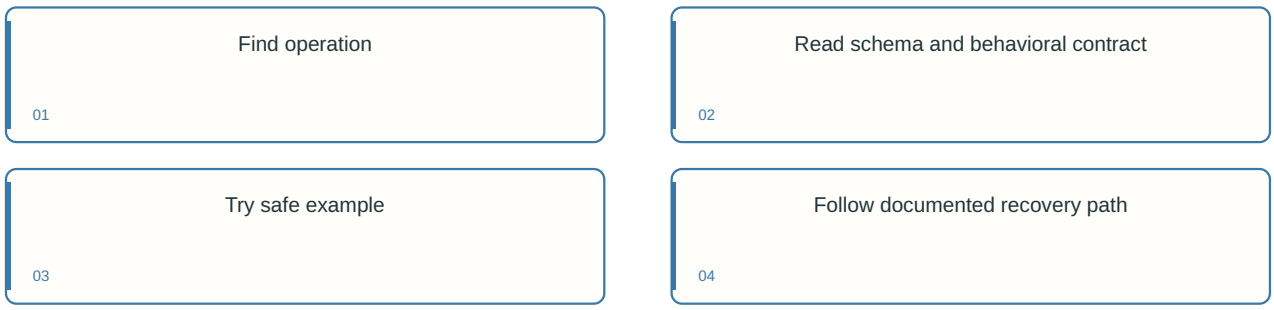


FIG. 8.4.5 **API documentation and discoverability · worked snapshots**

Find operation · Read schema and behavioral contract · Try safe example · Follow documented recovery path. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · KEYSET CONTINUATION

```
from mechanisms import keyset_page
rows = [(1,1,"A"), (1,2,"B"), (2,3,"C")]
assert keyset_page(rows, after=(1,1)) == rows[1:]
```

The ordered pair supplies a stable tie-breaker. A real cursor also needs query and tenant binding, validation, and a defined consistency policy. Keyset pagination does not create a snapshot of a changing dataset.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · Can a public API offer a permanent idempotency promise with finite storage? Define a defensible contract.

Use keyset_page() for ordered pairs and test insertion before a page boundary. For idempotency, bind actor, key, fingerprint, state, and result atomically; reject a mismatched payload. A plain dictionary checked before a separate insert does not protect concurrent requests or restart recovery.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Track documentation versions with implementation, and test key examples.

Carry forward: Find operation · Read schema and behavioral contract · Try safe example · Follow documented recovery path.

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

PART 3 / UNIT 09

Data Models and Storage Technologies

The running example

Choose storage technologies for user profiles, financial transactions, chat messages, event logs, and uploaded PDFs. Justify each decision.

Unit 9 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Access pattern	Candidate family	Important cost
Relationships and invariants	Relational	Index/write and transaction cost
Nested aggregate	Document	Cross-document rules
Direct key access	Key-value	Secondary query maintenance
Relationship traversal	Graph	Degree and traversal bounds

FIG. 9.0.1 **Database taxonomy**

Families describe useful access patterns; no category label establishes all guarantees.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Layout	Physical grouping	Suitable operation
Row	Fields of one record	Bounded point read
Column	One field across records	Analytical aggregation
Object	Named immutable bytes	Large file transfer

FIG. 9.0.2 **physical storage representations**

Logical data modeling and physical layout are separate decisions.

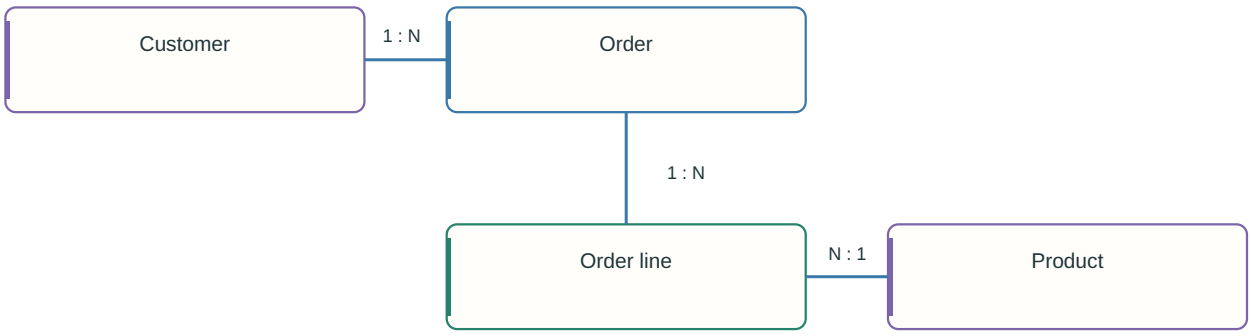
Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Fact	Normalized source	Legitimate copy
Current profile address	Customer profile	Cache with freshness policy
Purchase-time address	Order history	Historical fact, not stale profile
Display name	User record	Projection with update policy

FIG. 9.0.3 **normalized vs denormalized schema**

Not all duplication is an anomaly: distinguish historical snapshots from copies meant to track current state.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.



Order line retains purchase-time amount and address-related facts as required.

FIG. 9.0.4 **An order schema preserves relationships and history**

Customer has many Orders, Order has many Lines, and each Line references a Product identity while retaining purchase-time values.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

Storage promises & database families

Prerequisites: Introduction to System Design 9 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Choose storage technologies for user profiles, financial transactions, chat messages, event logs, and uploaded PDFs. Justify each decision.

This chapter examines storage promises & database families through concrete mechanisms, state snapshots, failure cases, and alternatives.

9.1.1 · What data storage must guarantee

Storage choices begin with promises: what is accepted, when it is committed, how it can be read, and which failures it survives. A chat history, an audit ledger, and a thumbnail cache need different retention and correctness contracts. Name the authoritative copy and recovery procedure. A database label alone does not establish transactions, isolation, backups, or acknowledgment durability; these depend on configuration and usage. Choose by access patterns and invariants before comparing product features.

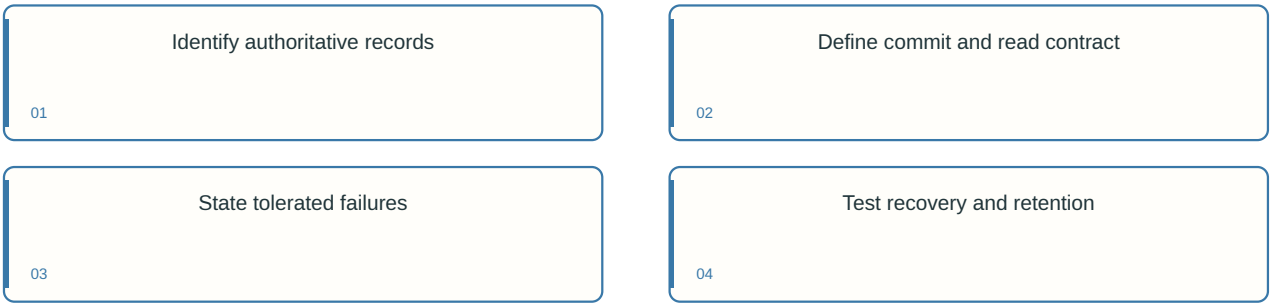


FIG. 9.1.1 **What data storage must guarantee · worked snapshots**

Identify authoritative records · Define commit and read contract · State tolerated failures · Test recovery and retention. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.2 · Relational databases and SQL

A relational model represents facts as relations with keys and constraints; SQL expresses queries and changes over that model. Foreign keys and transactions can protect relationships when configured and used appropriately. A ledger benefits from explicit account and entry relationships and atomic updates. Joins are not inherently a scaling failure; indexes, cardinality, workload, and execution plans matter. Start normalized for ownership and invariants, then denormalize specific read paths when evidence justifies maintenance cost.

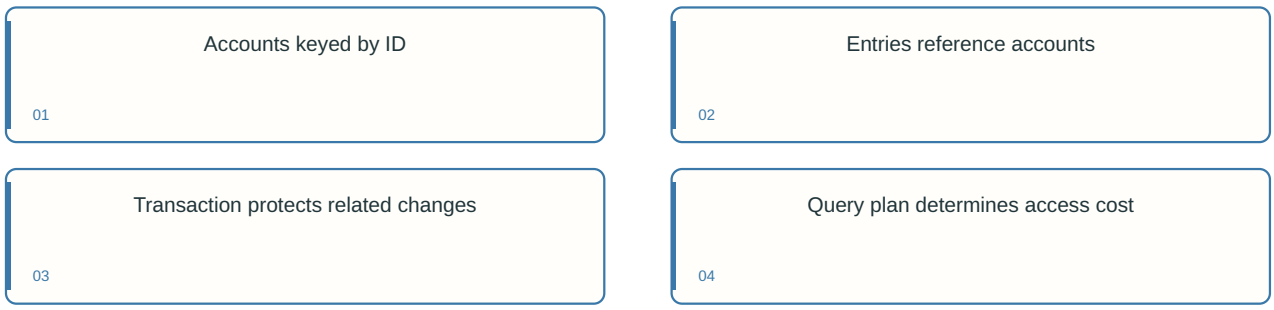


FIG. 9.1.2 **Relational databases and SQL · worked snapshots**

Accounts keyed by ID · Entries reference accounts · Transaction protects related changes · Query plan determines access cost. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.3 · Document databases

A document database groups nested data under a document identity and commonly permits flexible structures. A user profile with preferences may fit a document aggregate well. Flexibility moves some validation and migration responsibility into schema conventions and application code; it does not make schema disappear. Decide whether updates across documents need atomicity, how nested collections grow, and which fields are indexed. An unbounded array of all chat messages in one user document is usually a poor ownership and growth boundary.

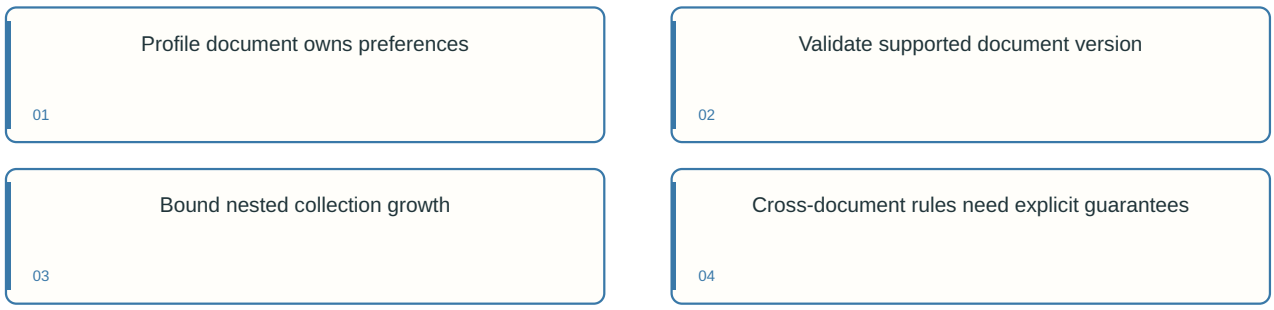


FIG. 9.1.3 **Document databases · worked snapshots**

Profile document owns preferences · Validate supported document version · Bound nested collection growth · Cross-document rules need explicit guarantees. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.4 · Key-value databases

A key-value store maps keys to values under a defined read/write contract. It suits sessions, cached objects, or direct ID lookup when operations match the key design. Querying arbitrary value fields may require secondary structures maintained separately. Do not assume all key-value stores share durability or consistency behavior. A key such as tenant:42:session:7 should encode the required identity scope, while expiration and invalidation determine whether the value remains usable after permissions change.

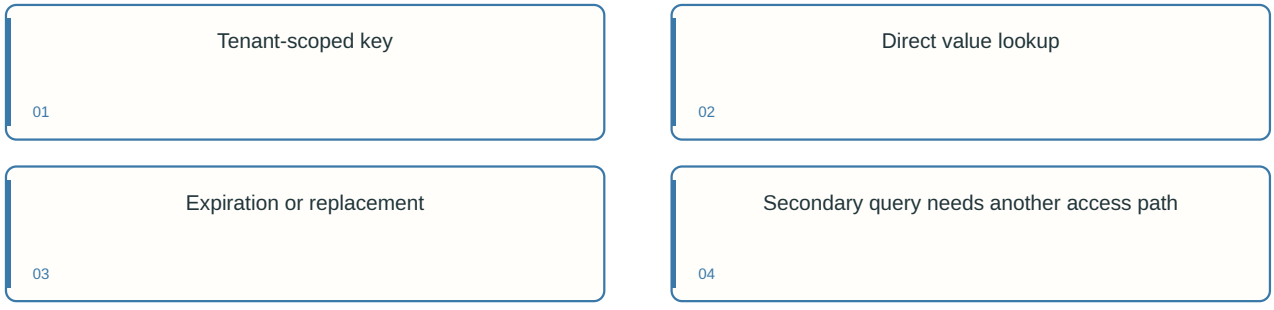


FIG. 9.1.4 **Key-value databases · worked snapshots**

Tenant-scoped key · Direct value lookup · Expiration or replacement · Secondary query needs another access path. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.5 · Wide-column stores

Wide-column systems organize data around partition keys and ordered clustering dimensions in common implementations. A conversation partition with messages ordered by sequence can support bounded history reads. Partition size and hot-key traffic must be planned: one enormous room can concentrate load despite a large cluster. Model queries first, duplicate data deliberately for other access patterns, and document which constraints or joins the application must enforce. The category is not a universal guarantee about every implementation.

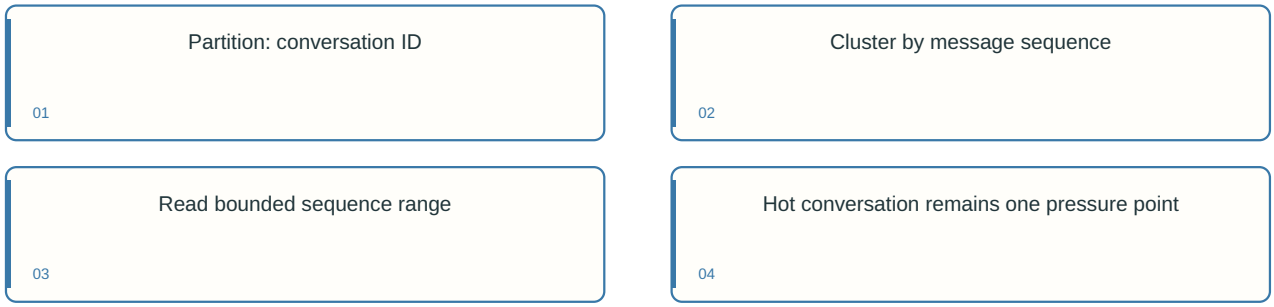


FIG. 9.1.5 **Wide-column stores · worked snapshots**

Partition: conversation ID · Cluster by message sequence · Read bounded sequence range · Hot conversation remains one pressure point. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.6 · Graph databases

Graph storage makes relationships and traversals first-class, useful when variable-depth relationships are central to queries. Fraud investigation may traverse accounts, devices, and transfers. A relational schema can also represent edges, so compare actual query complexity and locality rather than assuming graph databases alone can model relationships. Bound traversal depth and fan-out, enforce tenant or privacy boundaries during expansion, and inspect the cost of high-degree nodes. An unrestricted traversal can become an expensive data-exfiltration path.

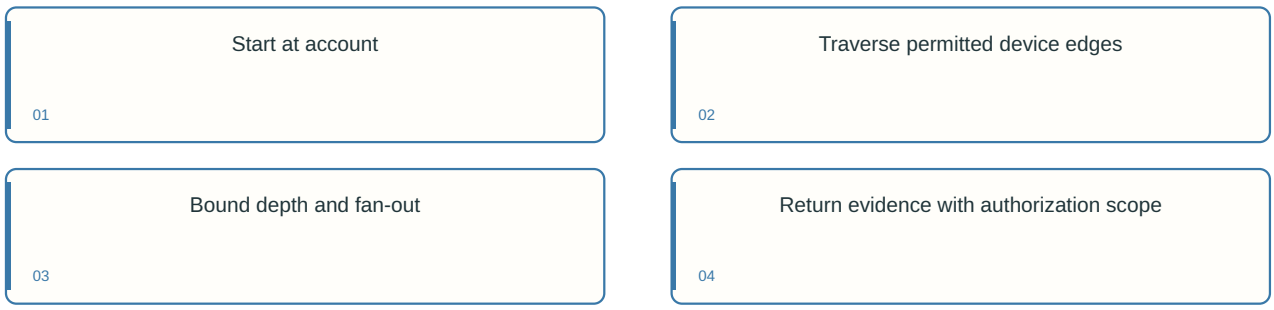


FIG. 9.1.6 **Graph databases · worked snapshots**

Start at account · Traverse permitted device edges · Bound depth and fan-out · Return evidence with authorization scope. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.7 · Time-series databases

Time-series data combines timestamps, measurements, and dimensions. Storage can exploit time locality, compression, partitioning, and retention. Device metrics with bounded labels fit this model, while adding a unique request ID as a metric label can explode cardinality. Decide handling for late data, clock skew, downsampling, and corrections. Time ordering by a device clock is not automatically causal ordering; keep ingestion time or sequence metadata when the distinction matters for investigation.

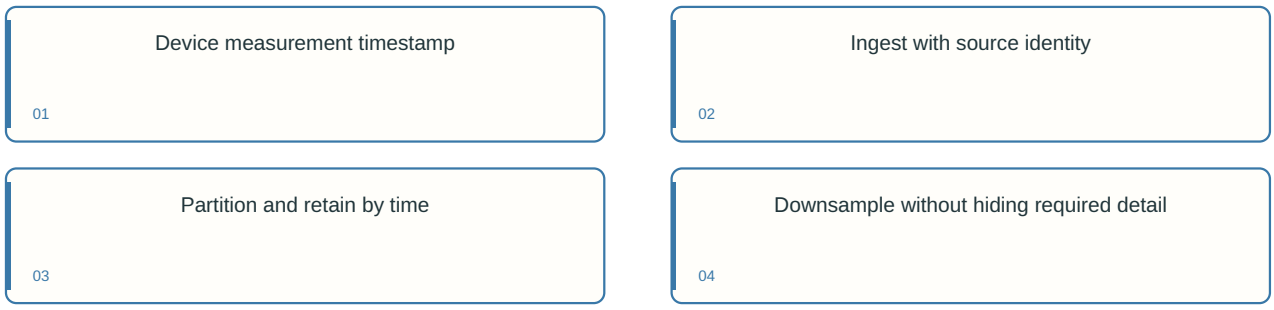


FIG. 9.1.7 **Time-series databases · worked snapshots**

Device measurement timestamp · Ingest with source identity · Partition and retain by time · Downsample without hiding required detail. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.8 · Search indexes

A search index stores representations optimized for retrieval rather than primary transactional ownership. An inverted index maps terms to postings; document scores and filters produce ranked results. Updates commonly propagate asynchronously from an authoritative store, so search may lag or temporarily retain deleted content. Authorization filtering must remain correct despite that lag. Keep enough source identity and version information to rebuild the index and reject stale updates; treating the index as the only copy makes recovery fragile.

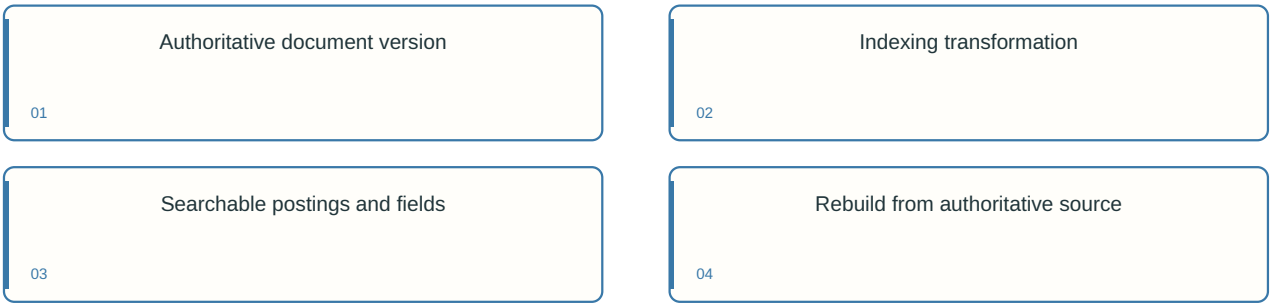


FIG. 9.1.8 **Search indexes · worked snapshots**

Authoritative document version · Indexing transformation · Searchable postings and fields · Rebuild from authoritative source. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.9 · Vector indexes: introductory architecture only

A vector index supports similarity search over numerical embeddings. It typically combines a vector representation, an approximate candidate structure, and metadata filtering. Candidate recall, update cost, memory, distance function, and authorization affect the architecture. An embedding is a derived representation, not an authoritative replacement for a document. Preserve source versions and rebuild paths; approximate retrieval can miss relevant items, and a vector score does not prove a factual answer or permission to expose the source.

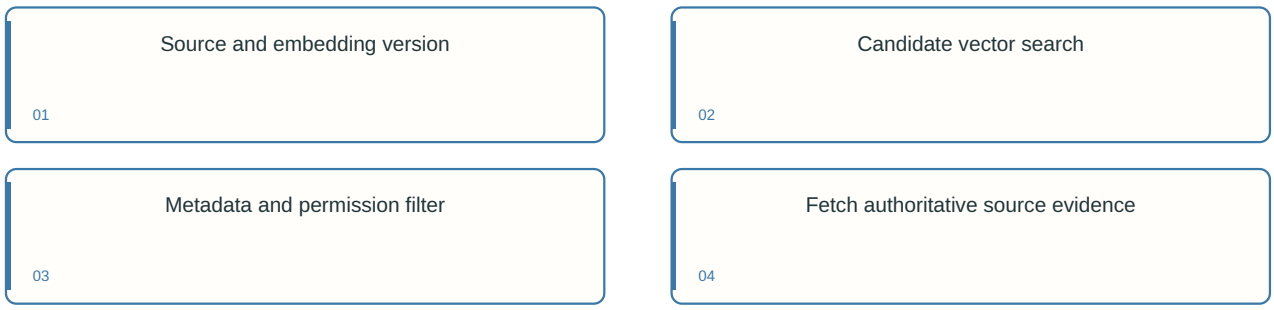


FIG. 9.1.9 **Vector indexes: introductory architecture only · worked snapshots**

Source and embedding version · Candidate vector search · Metadata and permission filter · Fetch authoritative source evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How does restoration from a historical backup interact with current privacy deletion and derived indexes?

Justify each choice through point reads, ranges, invariants, retention, and byte size. Do not choose five engines merely to cover a taxonomy. Separate PDF bytes from metadata and preserve recoverable references; explain which derived stores can be rebuilt and which facts must remain authoritative.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

An embedding is a derived representation, not an authoritative replacement for a document.

Carry forward: Source and embedding version · Candidate vector search · Metadata and permission filter · Fetch authoritative source evidence.

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Files, objects & analytical layouts

Prerequisites: 9.1 · Storage promises & database families 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Choose storage technologies for user profiles, financial transactions, chat messages, event logs, and uploaded PDFs. Justify each decision.

This chapter examines files, objects & analytical layouts through concrete mechanisms, state snapshots, failure cases, and alternatives.

9.2.1 · Object storage vs block vs file storage

Object storage exposes named blobs and metadata; block storage exposes addressable blocks; file storage exposes a filesystem namespace and file operations. Uploaded PDFs usually fit object storage plus relational metadata, while a database engine may need a block-backed filesystem with appropriate durability semantics. Protocol and implementation guarantees differ. A URL to an object is not the same as a committed metadata reference: incomplete uploads, orphan objects, and deletion require an explicit lifecycle protocol.

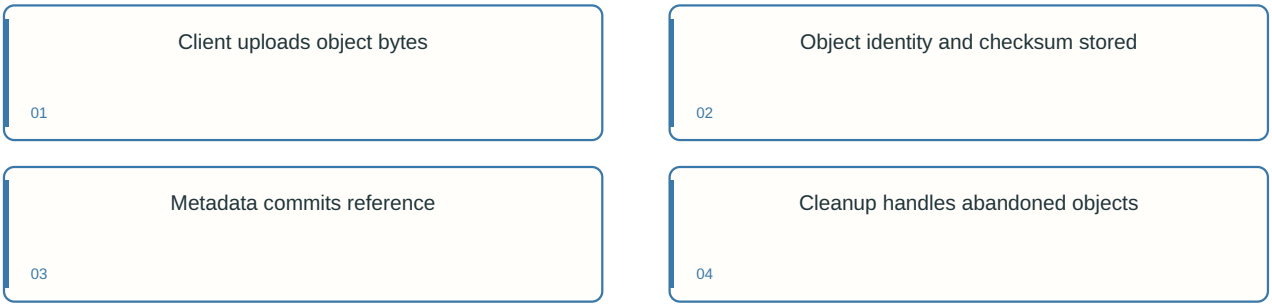


FIG. 9.2.1 **Object storage vs block vs file storage · worked snapshots**

Client uploads object bytes · Object identity and checksum stored · Metadata commits reference · Cleanup handles abandoned objects. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.2 · OLTP vs OLAP

OLTP emphasizes many short transactions over current operational state; OLAP emphasizes analytical scans and aggregation over larger histories. Running an unbounded dashboard query on the purchase database can compete with checkout for I/O and memory. Replicating or transforming operational data into analytical storage separates access patterns but creates freshness, lineage, and reconciliation obligations. A single engine can support mixed workloads to a point; measure interference before adding another system solely because two labels exist.

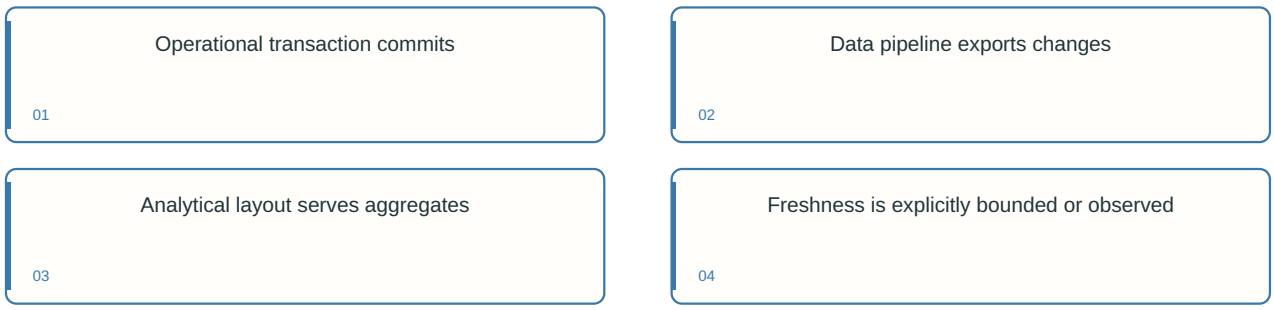


FIG. 9.2.2 **OLTP vs OLAP · worked snapshots**

Operational transaction commits · Data pipeline exports changes · Analytical layout serves aggregates · Freshness is explicitly bounded or observed. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.3 · Row-oriented vs column-oriented storage

Row layouts keep fields of a record together; column layouts keep a field's values together across many records. Fetching one customer's complete operational record often favors row-oriented access, while summing revenue across millions of rows can benefit from reading only relevant columns and compression. Modern engines mix techniques, so compare the actual plan. Layout choice affects writes, updates, locality, and compression; it does not replace indexing or a sound logical model.

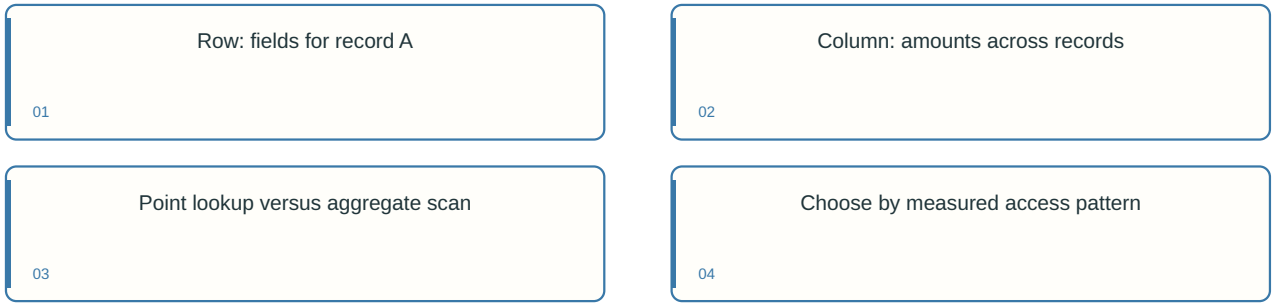


FIG. 9.2.3 **Row-oriented vs column-oriented storage · worked snapshots**

Row: fields for record A · Column: amounts across records · Point lookup versus aggregate scan · Choose by measured access pattern. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.4 · Normalization vs denormalization

Normalization avoids redundant facts and update anomalies; denormalization deliberately stores derived or repeated data to serve a read path. Store a customer's address once for their current profile, but retain the historical shipping address on a completed order as part of that order's fact. Not all duplication is an error. When a feed stores a cached author name, define whether it is a snapshot or a copy to maintain; ambiguity creates stale-data bugs.

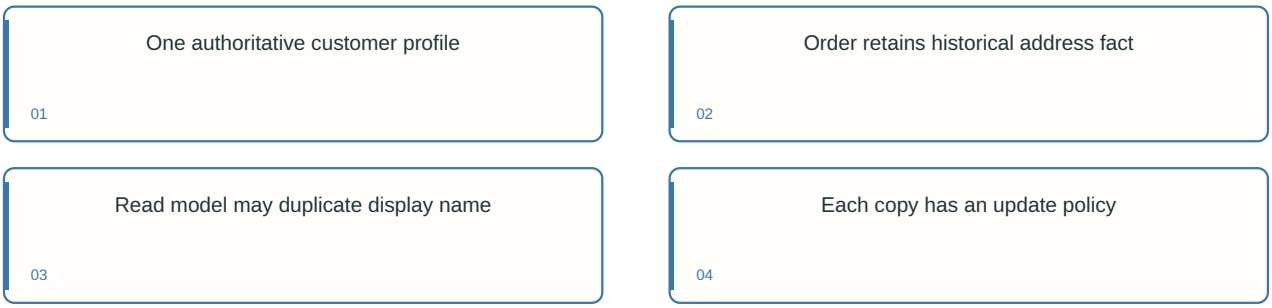


FIG. 9.2.4 **Normalization vs denormalization · worked snapshots**

One authoritative customer profile · Order retains historical address fact · Read model may duplicate display name · Each copy has an update policy. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How does restoration from a historical backup interact with current privacy deletion and derived indexes?

Justify each choice through point reads, ranges, invariants, retention, and byte size. Do not choose five engines merely to cover a taxonomy. Separate PDF bytes from metadata and preserve recoverable references; explain which derived stores can be rebuilt and which facts must remain authoritative.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Not all duplication is an error.

Carry forward: One authoritative customer profile · Order retains historical address fact · Read model may duplicate display name · Each copy has an update policy.

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Model relationships and ownership

Prerequisites: 9.2 · Files, objects & analytical layouts 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

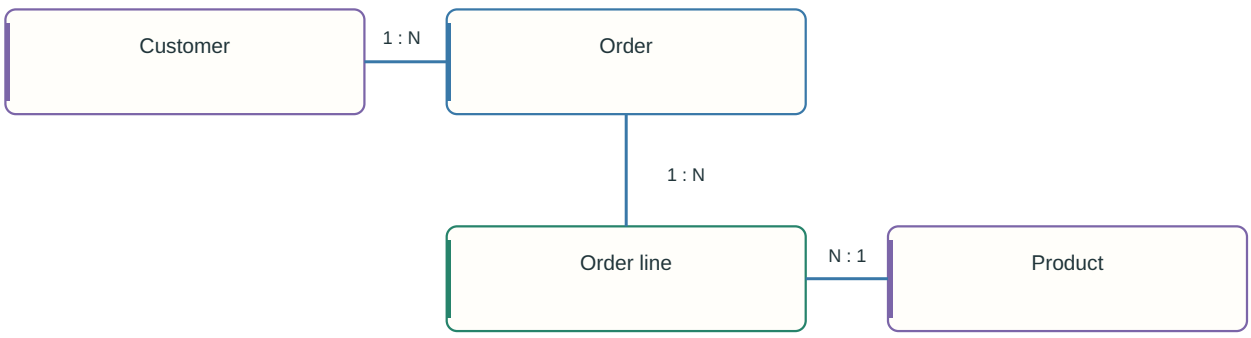
Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Choose storage technologies for user profiles, financial transactions, chat messages, event logs, and uploaded PDFs. Justify each decision.

This chapter examines model relationships and ownership through concrete mechanisms, state snapshots, failure cases, and alternatives.

9.3.1 · ER diagrams and relational modeling

An ER model identifies entities, relationships, cardinalities, and constraints. Orders have many lines; each line refers to a product identity and stores purchase-time values needed historically. A diagram should distinguish mandatory from optional relationships and name keys. The model must address deletion and lifecycle: removing a product must not erase a completed order's financial meaning. A schema is an executable part of the design when its constraints enforce the intended relationships.



Order line retains purchase-time amount and address-related facts as required.

FIG. 9.3.1 **An order schema preserves relationships and history**

Customer has many Orders, Order has many Lines, and each Line references a Product identity while retaining purchase-time values.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.2 · Referential integrity

Referential integrity ensures a stored relationship refers to an allowed target under the chosen constraint. A foreign key can prevent an order line from referencing a missing order in a relational database. Cascading deletion is a policy decision, not automatically the right cleanup. Across separately owned services, a local foreign key cannot enforce remote existence; use explicit contracts, stable identifiers, events, and reconciliation. Application checks before insertion can race unless protected by an appropriate atomic mechanism.

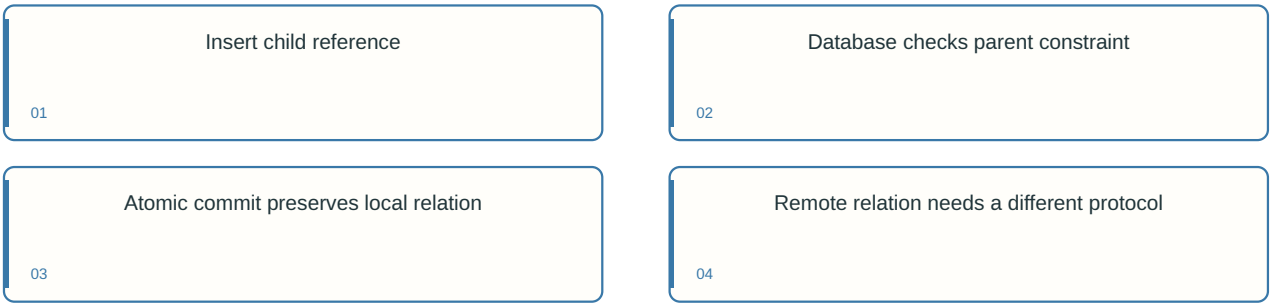


FIG. 9.3.2 **Referential integrity · worked snapshots**

Insert child reference · Database checks parent constraint · Atomic commit preserves local relation · Remote relation needs a different protocol. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.3 · Data ownership

Data ownership names who may define and change a fact. Payments owns financial outcome records; analytics consumes a derived copy. If several services directly update one table, transaction behavior and release coupling become shared even if their code repositories are separate. Publish contracts for reading and changing owned data, and give consumers the least privilege required. Ownership includes retention, incident response, schema evolution, and recovery, not only the right to create a table.

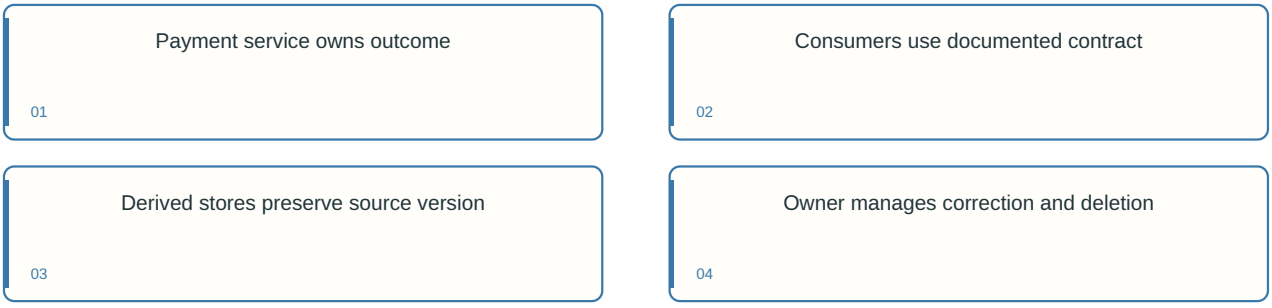


FIG. 9.3.3 **Data ownership · worked snapshots**

Payment service owns outcome · Consumers use documented contract · Derived stores preserve source version · Owner manages correction and deletion. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.4 · Choosing storage based on access patterns

Choose storage by the operations and invariants it must support. Profiles may fit relational or document storage; financial entries benefit from transaction and audit constraints; chat history needs conversation/time access; event logs need ordered append and replay; PDFs need large immutable-byte handling. These are workload arguments, not mandatory product assignments. Prefer fewer systems until a measured access pattern justifies another. Every new engine adds backup, security, migration, telemetry, and operational expertise requirements.

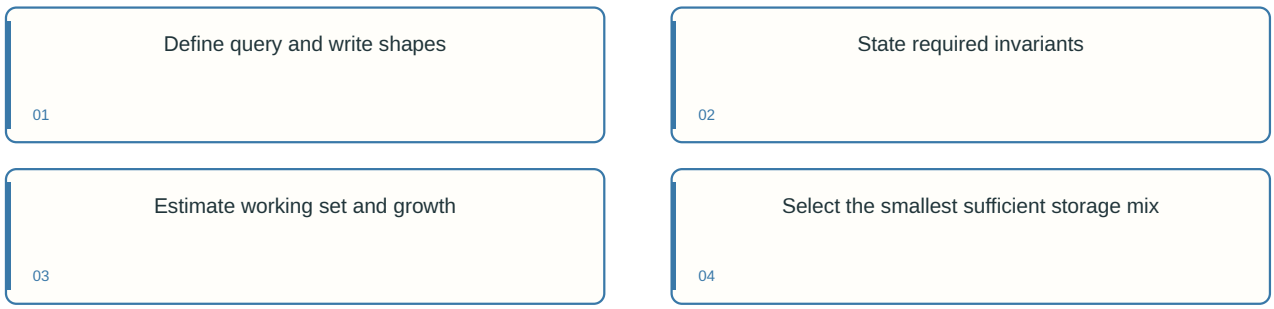


FIG. 9.3.4 **Choosing storage based on access patterns · worked snapshots**

Define query and write shapes · State required invariants · Estimate working set and growth · Select the smallest sufficient storage mix. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How does restoration from a historical backup interact with current privacy deletion and derived indexes?

Justify each choice through point reads, ranges, invariants, retention, and byte size. Do not choose five engines merely to cover a taxonomy. Separate PDF bytes from metadata and preserve recoverable references; explain which derived stores can be rebuilt and which facts must remain authoritative.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Prefer fewer systems until a measured access pattern justifies another.

Carry forward: Define query and write shapes · State required invariants · Estimate working set and growth · Select the smallest sufficient storage mix.

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Access patterns, retention & operational cost

Prerequisites: 9.3 · Model relationships and ownership 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Choose storage technologies for user profiles, financial transactions, chat messages, event logs, and uploaded PDFs. Justify each decision.

This chapter examines access patterns, retention & operational cost through concrete mechanisms, state snapshots, failure cases, and alternatives.

9.4.1 · Data retention and lifecycle

Retention is an explicit lifecycle, not a scheduled delete statement added at the end. Decide legal and business purpose, expiration, archival, backup behavior, and derived-copy deletion. Dropping old partitions may be efficient for time-bounded data, while per-user deletion needs identity-aware propagation. A tombstone may prevent reappearance during replay but still retain personal data; minimize it appropriately. Verify that recovery from an old backup reapplies required deletion decisions instead of resurrecting removed records.

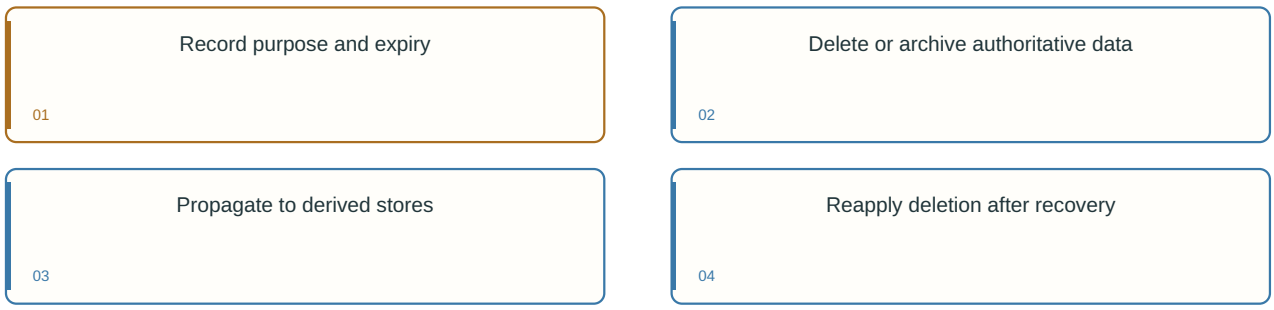


FIG. 9.4.1 **Data retention and lifecycle · worked snapshots**

Record purpose and expiry · Delete or archive authoritative data · Propagate to derived stores · Reapply deletion after recovery. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

9.4.2 · Polyglot persistence and its operational costs

Polyglot persistence gives different workloads suitable engines at the cost of more operating boundaries. A shop might add search only after relational queries fail its relevance or latency needs. It then must track indexing lag, rebuilds, schemas, permissions, and recovery across both stores. Cross-store operations are not one transaction by default. Compare the added capability with total ownership cost, and keep authoritative data and derived representations distinguishable in every diagram and runbook.

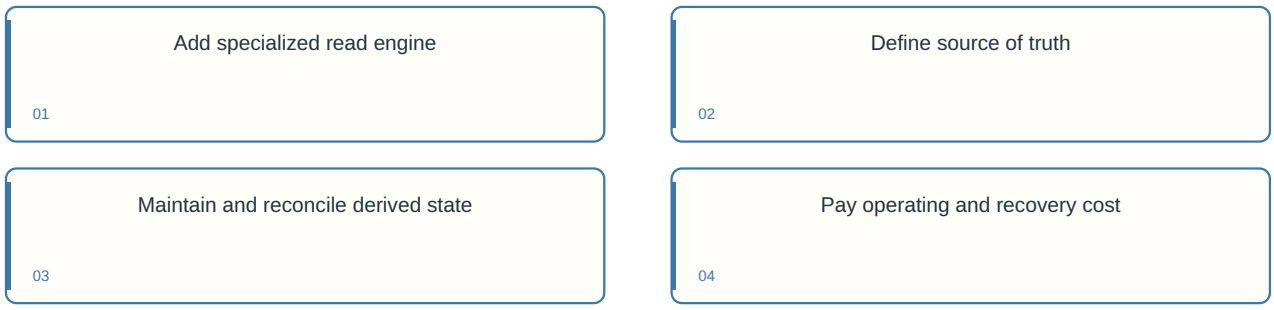
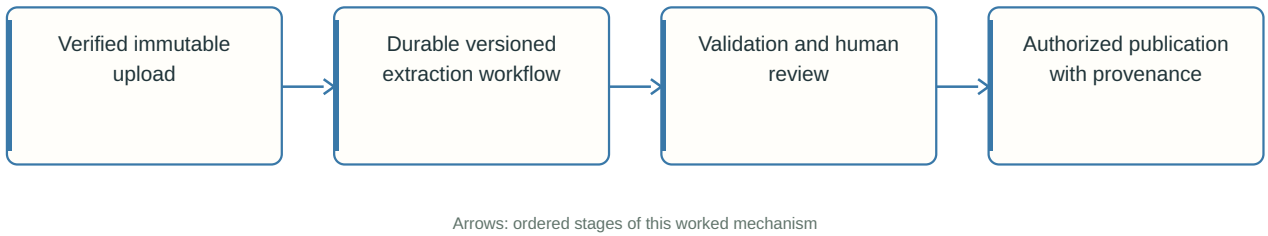


FIG. 9.4.2 **Polyglot persistence and its operational costs · worked snapshots**

Add specialized read engine · Define source of truth · Maintain and reconcile derived state · Pay operating and recovery cost. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 9.4.3 **The running system · ordered responsibility**

Verify upload and source digest, create a stable workflow, classify, extract, validate schema and semantic checks, record evidence, and either publish or request review. Human decisions create a new validated result version and publication intent. Each stage conditionally commits under its current attempt epoch. Reprocessing creates a new run over an identified source version rather than mutating old provenance. Deletion cancels or blocks inappropriate later publication.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

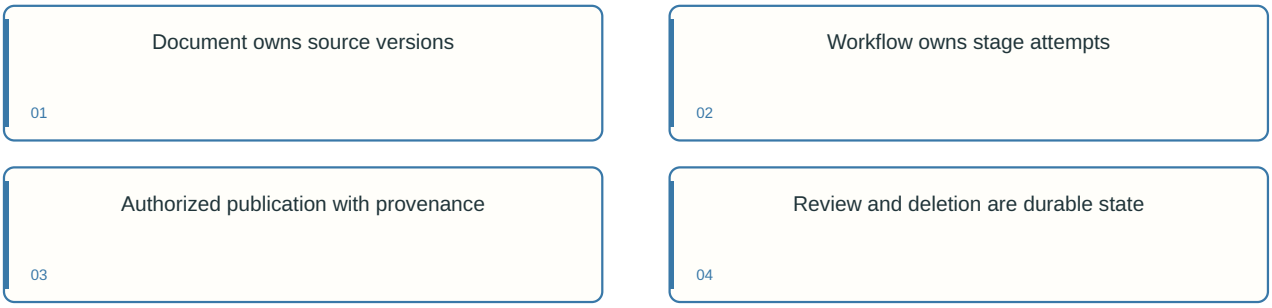


FIG. 9.4.4 **The running system · invariant and recovery review**

Review whether any stage silently changes source versions, whether an old attempt can publish after deletion, and whether human decisions are preserved during reprocessing. Test duplicate tasks, interrupted extraction, invalid output, reviewer concurrency, stale completion, missing object, index lag, and full restore. A pipeline that extracts a few fields once has not demonstrated the required recoverable, explainable platform.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PSEUDOCODE · BYTES AND METADATA

```
upload = create_authorized_upload(tenant, expected_size)
object = store_and_verify_bytes(upload)
with transaction():
    insert_document(tenant, object.version, object.digest)
    insert_processing_intent(object.version)
# Orphan-byte cleanup handles incomplete metadata handoff.
```

The authoritative metadata references a verified immutable byte version. If metadata commit fails, cleanup reconciles orphan objects. Do not claim one local transaction atomically contains a separate object-storage upload.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · How does restoration from a historical backup interact with current privacy deletion and derived indexes?

Justify each choice through point reads, ranges, invariants, retention, and byte size. Do not choose five engines merely to cover a taxonomy. Separate PDF bytes from metadata and preserve recoverable references; explain which derived stores can be rebuilt and which facts must remain authoritative.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Cross-store operations are not one transaction by default.

Carry forward: Add specialized read engine · Define source of truth · Maintain and reconcile derived state · Pay operating and recovery cost.

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

PART 3 / UNIT 10

Database Internals, Transactions, and Concurrency Control

The running example

Two users attempt to purchase the last available ticket. Demonstrate the lost-update race, show why naive reads fail, and implement valid alternatives using transactions and concurrency control.

Unit 10 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

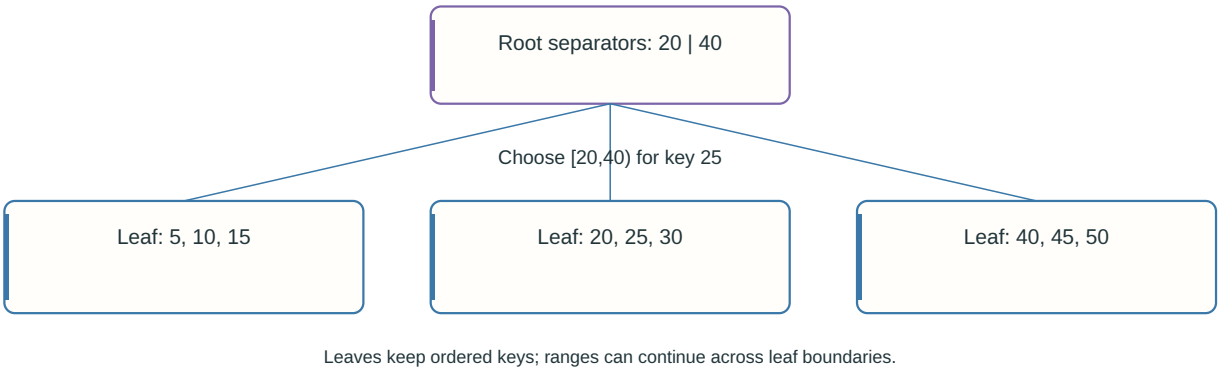


FIG. 10.0.1 **B+tree lookup narrows an ordered key range**

Illustrative keys and leaves demonstrate lookup and range locality. Exact database page layout is implementation-specific.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

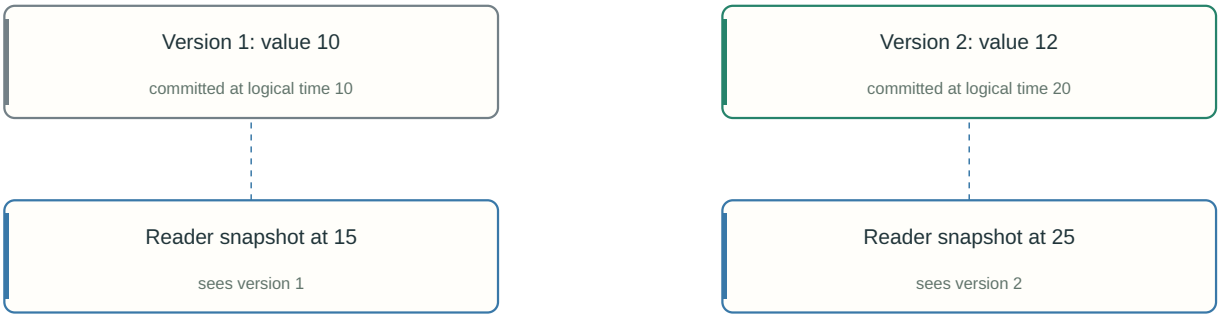


FIG. 10.0.2 **Two snapshots can legitimately see different versions**

A reader whose snapshot precedes version two can keep seeing version one while a later reader sees version two. MVCC alone does not imply serializability.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

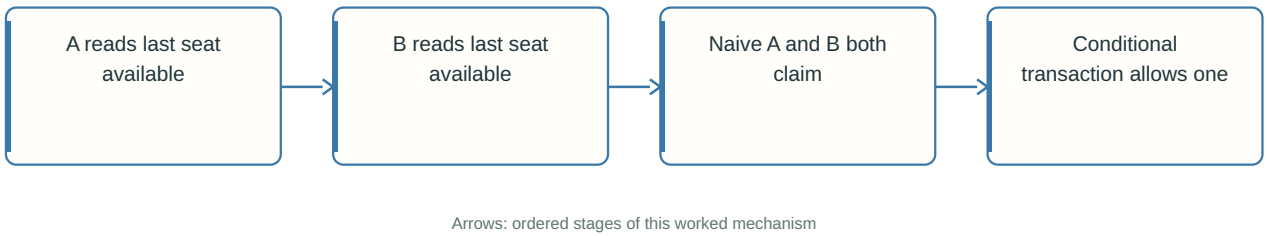


FIG. 10.0.3 **concurrent transaction interleavings**

The observable acknowledgment history, not only the final row, reveals overselling.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Holder	Owns	Waits for
Transaction A	Row X	Row Y
Transaction B	Row Y	Row X
Result	Circular wait	Abort/retry or consistent ordering

FIG. 10.0.4 **lock wait graph**

A lock-wait graph contains a cycle when each transaction waits on a resource held by the other.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

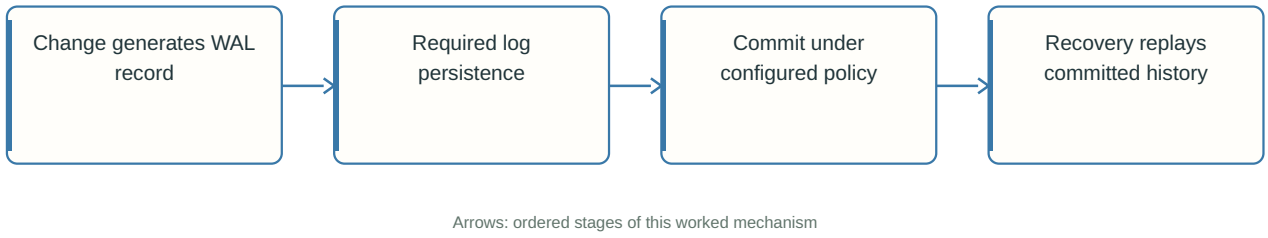


FIG. 10.0.5 **WAL and recovery**

Data pages can lag the log under a correct recovery protocol; a checkpoint is not a backup.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Pages, indexes & query plans

Prerequisites: Runtime, Processes, Threads, and Concurrency, Data Models and Storage Technologies 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two users attempt to purchase the last available ticket. Demonstrate the lost-update race, show why naive reads fail, and implement valid alternatives using transactions and concurrency control.

This chapter examines pages, indexes & query plans through concrete mechanisms, state snapshots, failure cases, and alternatives.

10.1.1 · How databases store and retrieve records

A database translates logical records into physical structures and coordinates access while preserving its promises. A lookup may consult an index, fetch pages into memory, inspect visibility, and return a row. The operating system and device still affect latency and durability. Understanding this path explains why one query may be CPU-bound while another waits on I/O. Start by measuring the plan and resource behavior; changing a database category without identifying the expensive step is guesswork.

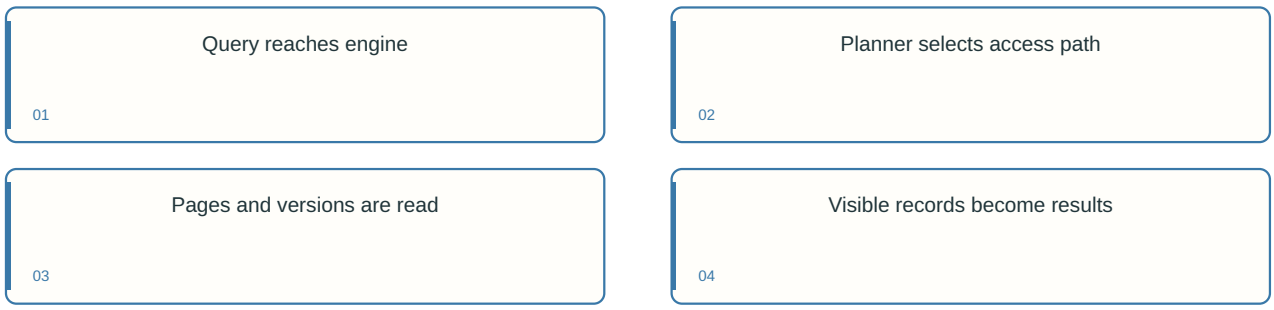


FIG. 10.1.1 **How databases store and retrieve records · worked snapshots**

Query reaches engine · Planner selects access path · Pages and versions are read · Visible records become results. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.2 · Pages, records, indexes, and buffer pools

A page is a fixed-size storage unit in many engines; records occupy pages; indexes point toward searchable values or records; a buffer pool caches useful pages. A cache hit avoids some storage I/O but still incurs parsing, execution, and synchronization. If a query touches a million scattered pages, extra application servers may only increase random reads. Working-set size and locality explain performance better than raw table row count alone. Exact page layouts and buffer policies are implementation-specific.

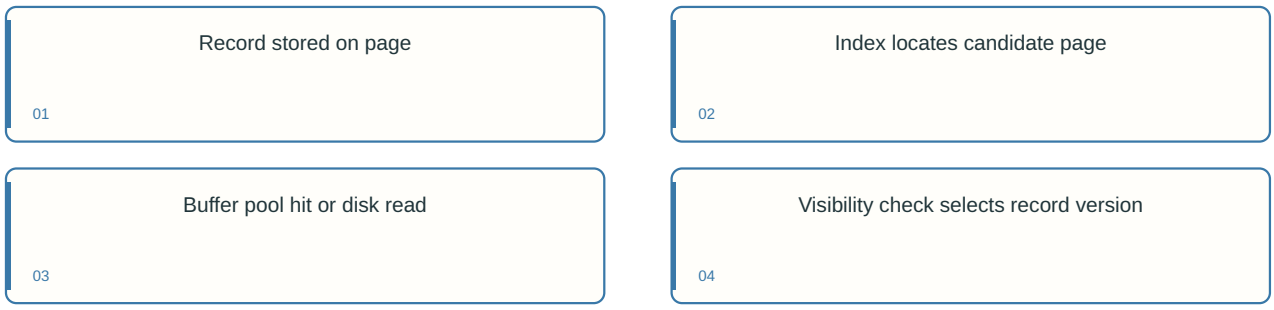


FIG. 10.1.2 **Pages, records, indexes, and buffer pools · worked snapshots**

Record stored on page · Index locates candidate page · Buffer pool hit or disk read · Visibility check selects record version. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.3 · B-tree/B+tree and hash indexing

B-tree families maintain ordered keys with bounded-height traversal; B+tree designs commonly keep data or record references in leaves linked for range scans. Hash indexing suits equality under its implementation but does not provide ordered range traversal. For keys [10,20,30,40], a range 20–35 benefits from an ordered path and neighboring leaves. Index choice depends on predicates, ordering, maintenance cost, and engine support. “An index is O(log n)” hides page I/O and result-size work.

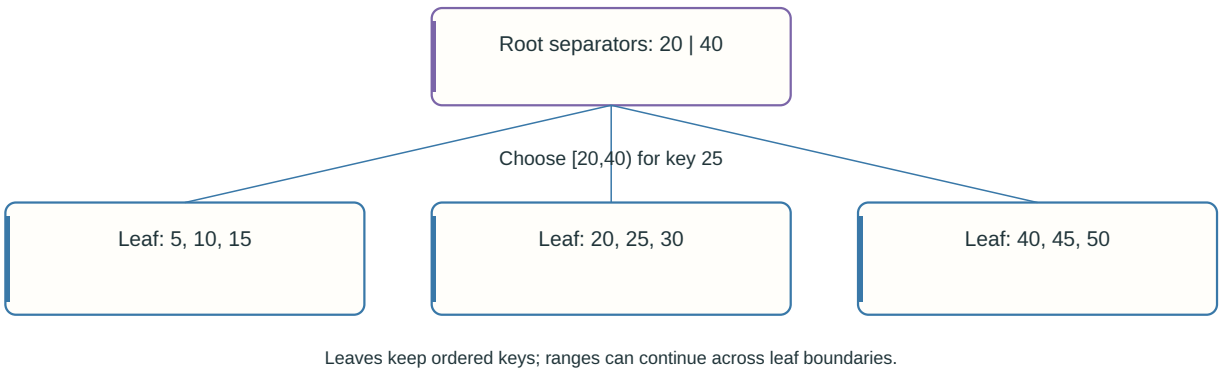


FIG. 10.1.3 **B+tree lookup narrows an ordered key range**

Illustrative keys and leaves demonstrate lookup and range locality. Exact database page layout is implementation-specific.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.4 · LSM trees and write amplification

An LSM-based engine buffers writes, flushes sorted runs, and compacts them into a searchable organization. Sequential ingestion can help write-heavy workloads, while reads may consult several structures and compaction rewrites data. Write amplification counts physical bytes written relative to logical changes; it depends on layout and workload. Compaction consumes bandwidth and can cause stalls, so provision and observe it. A write-optimized name does not mean unlimited writes or free reads; compare tail latency under sustained ingestion.

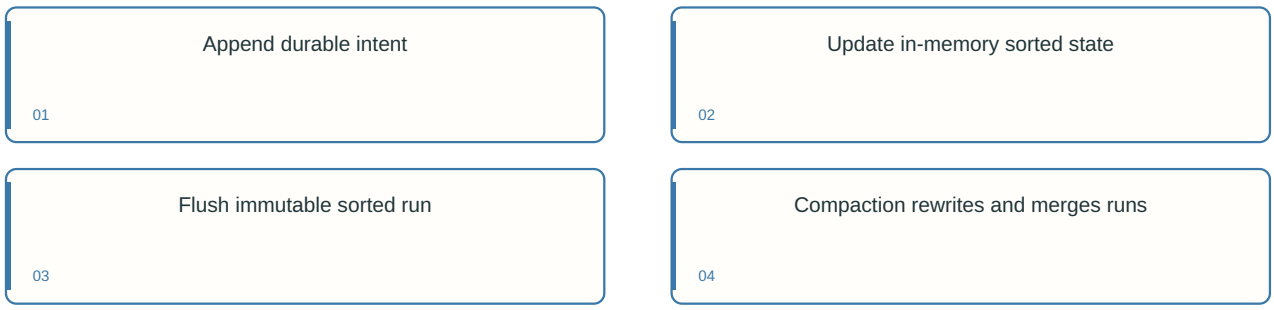


FIG. 10.1.4 **LSM trees and write amplification · worked snapshots**

Append durable intent · Update in-memory sorted state · Flush immutable sorted run · Compaction rewrites and merges runs. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.5 · Query planning and EXPLAIN

A query planner selects a physical plan from logical operations using statistics and cost estimates. EXPLAIN shows a proposed plan; execution analysis can reveal actual rows and time but may execute side effects depending on the command and engine. A join estimated at ten rows but producing a million can choose an unsuitable strategy. Inspect cardinality estimates, filters, scan types, sorts, and buffers. Update statistics or rewrite the query before blindly adding hardware or indexes.

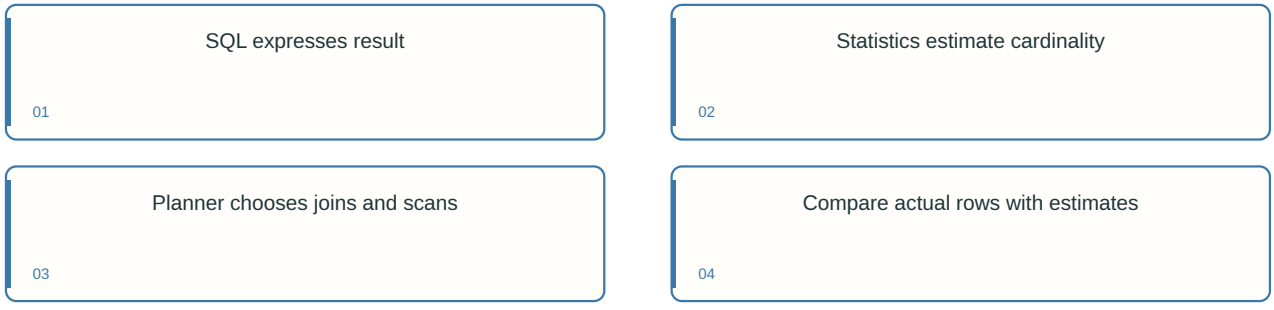


FIG. 10.1.5 **Query planning and EXPLAIN · worked snapshots**

SQL expresses result · Statistics estimate cardinality · Planner chooses joins and scans · Compare actual rows with estimates. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.6 · Index selection and query optimization

An index speeds selected access paths while consuming space and write work. A composite index on (tenant_id,created_at,id) can support a tenant-scoped time-ordered page when the predicates and ordering fit. Unused or redundant indexes slow writes without helping reads. Covering an operation can avoid extra fetches in some engines, but larger indexes reduce cache locality. Test with representative data distribution and parameters; a small uniform fixture may conceal skew and poor selectivity.

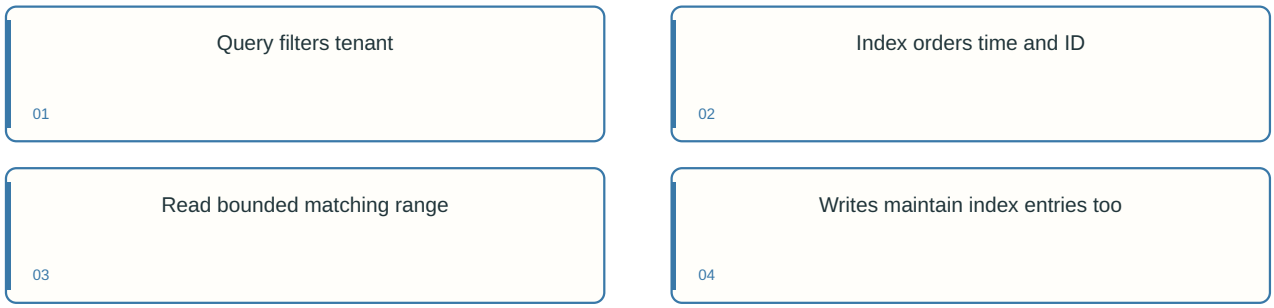


FIG. 10.1.6 **Index selection and query optimization · worked snapshots**

Query filters tenant · Index orders time and ID · Read bounded matching range · Writes maintain index entries too. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which invariant becomes harder when a local multi-statement transaction is split across independently owned databases?

Use claim_seat() against the same store and assert one owner wins while the other receives conflict. Explain which local transaction protects the invariant. Add rollback and crash histories; do not claim a local seat claim makes a remote payment part of the same transaction.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Covering an operation can avoid extra fetches in some engines, but larger indexes reduce cache locality.

Carry forward: Query filters tenant · Index orders time and ID · Read bounded matching range · Writes maintain index entries too.

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

Commit, logging & versions

Prerequisites: 10.1 · Pages, indexes & query plans 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two users attempt to purchase the last available ticket. Demonstrate the lost-update race, show why naive reads fail, and implement valid alternatives using transactions and concurrency control.

This chapter examines commit, logging & versions through concrete mechanisms, state snapshots, failure cases, and alternatives.

10.2.1 · ACID properties

ACID separates atomicity, consistency, isolation, and durability. Atomicity prevents partial transaction outcomes; consistency means the transaction preserves declared invariants when the application and constraints are correct; isolation governs concurrent observations; durability protects committed state under configuration and failure assumptions. The C is not a promise that all business rules are automatically inferred. A transaction that atomically transfers the wrong amount can satisfy engine mechanics while violating the intended business rule.

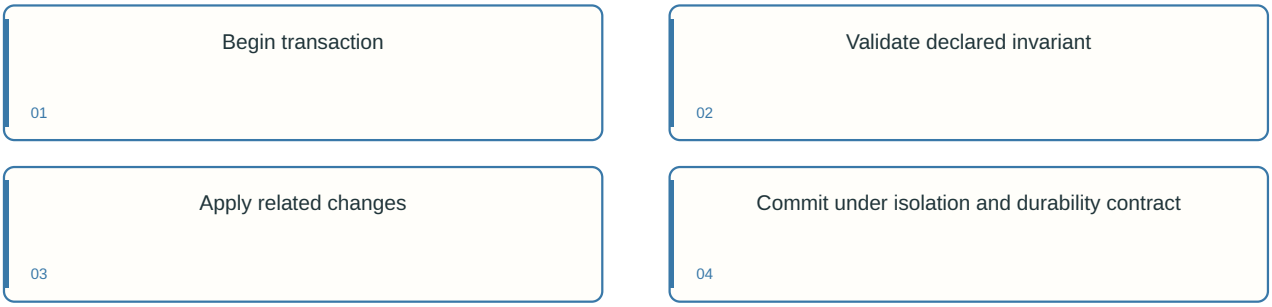


FIG. 10.2.1 **ACID properties · worked snapshots**

Begin transaction · Validate declared invariant · Apply related changes · Commit under isolation and durability contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.2 · Transaction commit and rollback

Commit establishes a transaction's accepted outcome according to the engine's protocol; rollback abandons its uncommitted work. Do not send a business success before commit completes if success requires persistence. Commit can fail or leave the client uncertain after a connection loss. Retrying the transaction may duplicate an external effect unless identity and coordination are explicit. Keep remote provider calls outside long-held local transactions where possible, and reconcile their results rather than pretend rollback reverses a remote charge.

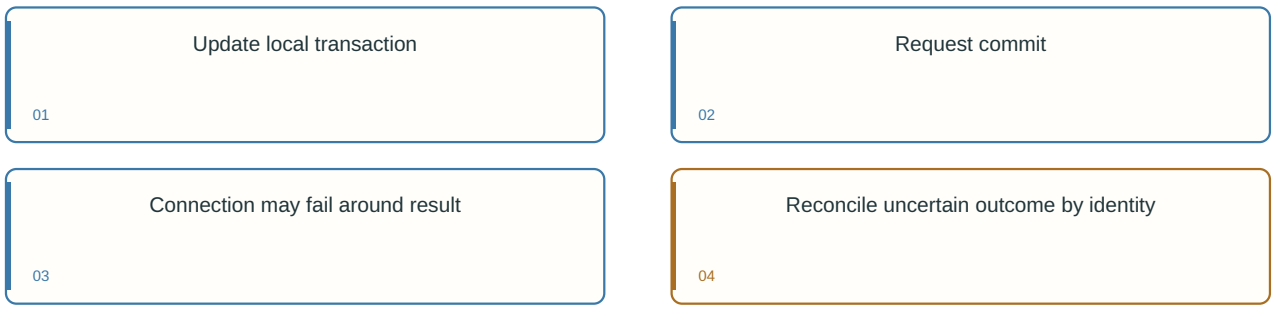


FIG. 10.2.2 **Transaction commit and rollback · worked snapshots**

Update local transaction · Request commit · Connection may fail around result · Reconcile uncertain outcome by identity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.3 · Write-ahead logging

Write-ahead logging records recovery information before the corresponding dirty data pages are considered safely persisted. Recovery can use durable log records to reconstruct committed work even when data pages lag. Checkpointing limits recovery effort; it is not the same as a backup. Acknowledgment behavior depends on sync and replication configuration. Log durability alone cannot protect against loss of the only device containing both log and data, or an application committing logically wrong changes.

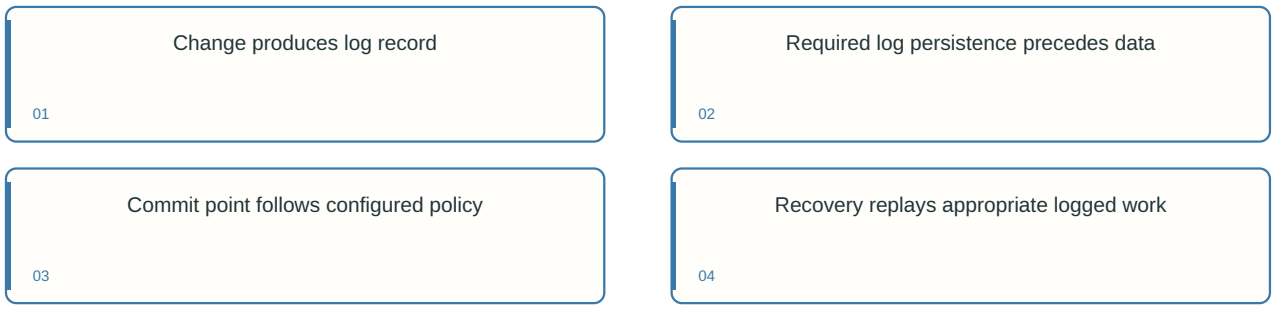


FIG. 10.2.3 **Write-ahead logging · worked snapshots**

Change produces log record · Required log persistence precedes data · Commit point follows configured policy · Recovery replays appropriate logged work. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.4 · MVCC

MVCC maintains versions so readers can observe a defined snapshot while writers create new versions. Visibility depends on transaction rules, not simply the newest timestamp. A long-running reader can retain old versions and delay cleanup. Snapshot reads do not inherently prevent write skew: two transactions may read the same invariant and update different rows. Understand the engine's isolation level and conflict detection rather than treating “uses MVCC” as synonymous with serializable execution.

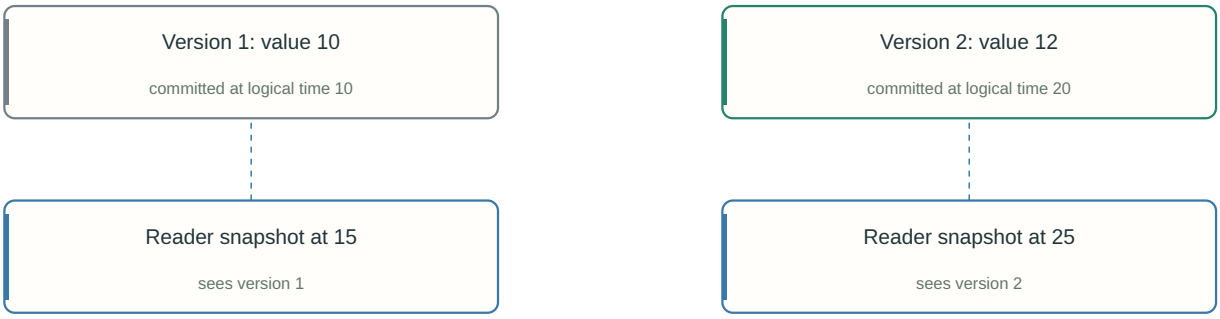


FIG. 10.2.4 **Two snapshots can legitimately see different versions**

A reader whose snapshot precedes version two can keep seeing version one while a later reader sees version two. MVCC alone does not imply serializability.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which invariant becomes harder when a local multi-statement transaction is split across independently owned databases?

Use claim_seat() against the same store and assert one owner wins while the other receives conflict. Explain which local transaction protects the invariant. Add rollback and crash histories; do not claim a local seat claim makes a remote payment part of the same transaction.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Snapshot reads do not inherently prevent write skew: two transactions may read the same invariant and update different rows.

Carry forward: Reader snapshot established · Writer creates new row version · Reader sees its allowed version · Cleanup waits for relevant readers.

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   `Access reviewed · 21 October 2026`

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   `Access reviewed · 21 October 2026`

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   `Access reviewed · 21 October 2026`

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   `Access reviewed · 21 October 2026`

Races, anomalies & isolation

Prerequisites: 10.2 · Commit, logging & versions 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two users attempt to purchase the last available ticket. Demonstrate the lost-update race, show why naive reads fail, and implement valid alternatives using transactions and concurrency control.

This chapter examines races, anomalies & isolation through concrete mechanisms, state snapshots, failure cases, and alternatives.

10.3.1 · Locking and optimistic concurrency control

Pessimistic locking coordinates access before conflicting work; optimistic concurrency validates that assumptions still hold before committing. A versioned update such as UPDATE seat SET owner=A,version=8 WHERE id=7 AND version=7 succeeds only if its expected version remains current. Zero affected rows means conflict, not success. Retrying must re-read and reconsider business conditions. Locks can wait or deadlock, while optimistic retries can waste work under contention; choose based on conflict rate and transaction cost.

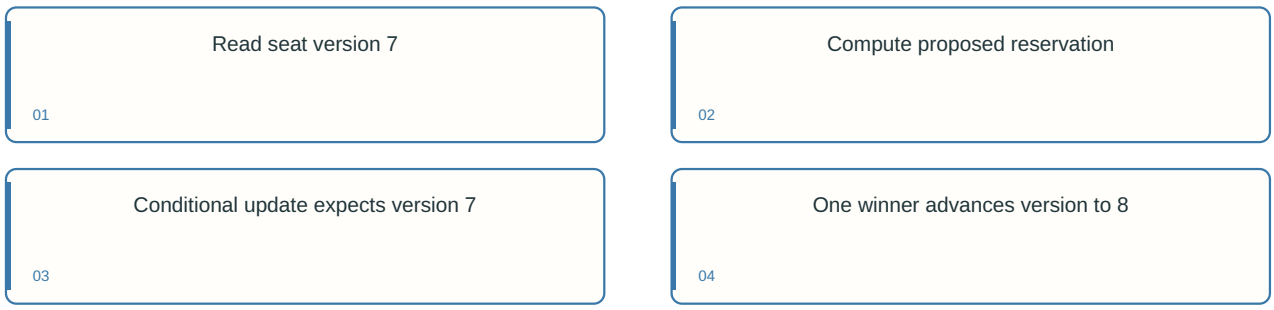


FIG. 10.3.1 **Locking and optimistic concurrency control · worked snapshots**

Read seat version 7 · Compute proposed reservation · Conditional update expects version 7 · One winner advances version to 8. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.2 · Dirty reads, nonrepeatable reads, phantom reads

A dirty read observes uncommitted changes; a nonrepeatable read sees a previously read row change; a phantom changes the set matching a predicate. Starting with three open reservations, another transaction inserts a fourth before a repeated predicate query. The anomaly's occurrence depends on isolation and implementation. Do not assume that an isolation level's name implies identical behavior in every database. Define the problematic history and test the selected engine's documented guarantees.

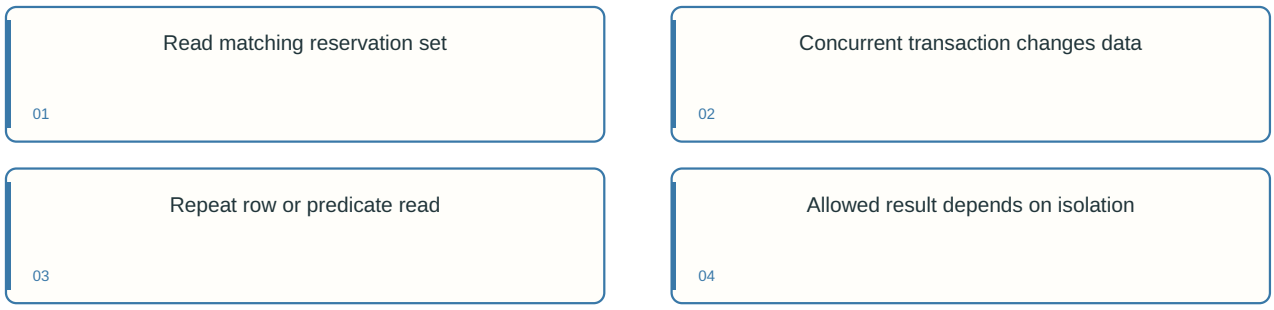


FIG. 10.3.2 **Dirty reads, nonrepeatable reads, phantom reads · worked snapshots**

Read matching reservation set · Concurrent transaction changes data · Repeat row or predicate read · Allowed result depends on isolation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.3 · Lost updates, write skew, serialization anomalies

Lost updates overwrite another writer's result; write skew violates a multi-row invariant after transactions update different rows; serialization anomalies are histories without an equivalent serial ordering. If two doctors each see the other on call and both mark themselves off, a snapshot can preserve each read yet violate “at least one on call.” Fix the invariant with appropriate serialization, locking, or a redesigned shared conflict point. Protecting only each individual row is insufficient.

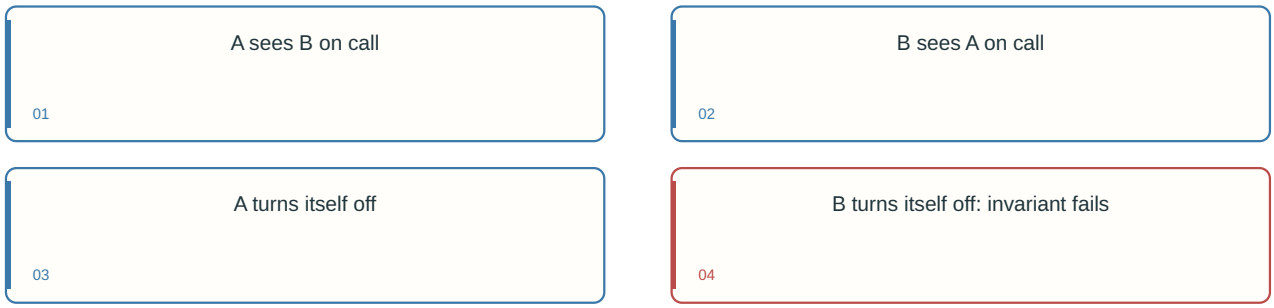


FIG. 10.3.3 **Lost updates, write skew, serialization anomalies · worked snapshots**

A sees B on call · B sees A on call · A turns itself off · B turns itself off: invariant fails. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.4 · Isolation levels and implementation differences

Isolation levels specify allowed concurrency behavior, but engines implement them differently. PostgreSQL 18 Read Committed uses statement-level snapshots for ordinary reads; its Repeatable Read provides a transaction snapshot yet can permit serialization anomalies; Serializable adds conflict handling that can abort transactions. Other engines and statement forms differ. Applications must handle retryable failures by restarting the whole affected transaction with bounded attempts. Do not retry only the last statement while retaining assumptions from an invalidated snapshot.

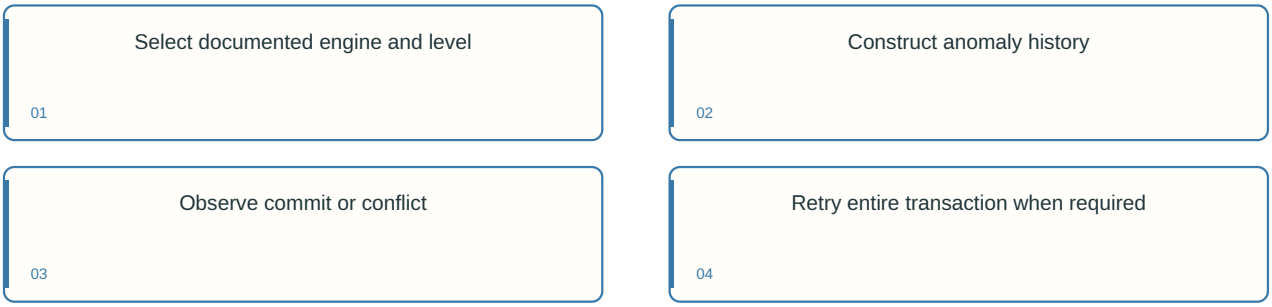


FIG. 10.3.4 **Isolation levels and implementation differences · worked snapshots**

Select documented engine and level · Construct anomaly history · Observe commit or conflict · Retry entire transaction when required. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.5 · Serializable execution and serializability

Serializable means the committed effect is equivalent to some serial transaction order, not necessarily wall-clock order. Linearizability is a distinct real-time single-operation property; strict serializability combines a transactional serial order with real-time constraints. A transaction may need retry after serialization failure. Keep external side effects out of blindly replayed transaction bodies or protect them with stable identity. Serialization makes concurrency reasoning stronger but does not validate incorrect business logic or solve every availability requirement.

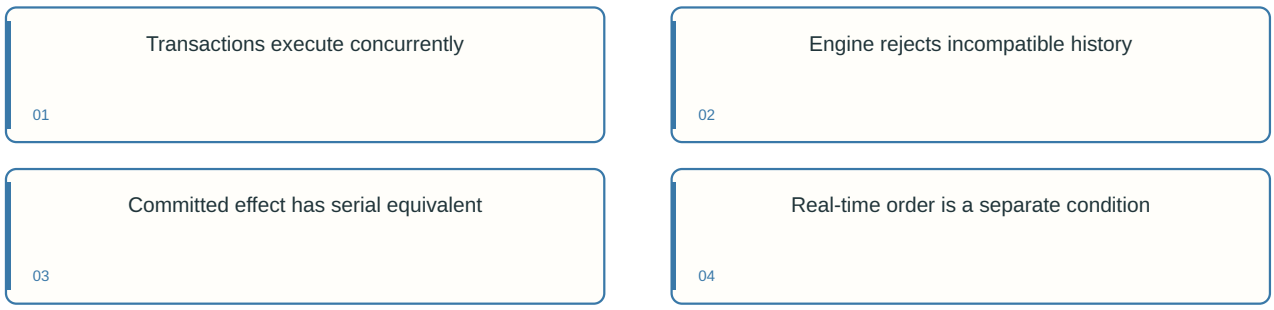


FIG. 10.3.5 **Serializable execution and serializability · worked snapshots**

Transactions execute concurrently · Engine rejects incompatible history · Committed effect has serial equivalent · Real-time order is a separate condition. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which invariant becomes harder when a local multi-statement transaction is split across independently owned databases?

Use claim_seat() against the same store and assert one owner wins while the other receives conflict. Explain which local transaction protects the invariant. Add rollback and crash histories; do not claim a local seat claim makes a remote payment part of the same transaction.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Keep external side effects out of blindly replayed transaction bodies or protect them with stable identity.

Carry forward: Transactions execute concurrently · Engine rejects incompatible history · Committed effect has serial equivalent · Real-time order is a separate condition.

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

Pools, recovery & ticket correctness

Prerequisites: 10.3 · Races, anomalies & isolation 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two users attempt to purchase the last available ticket. Demonstrate the lost-update race, show why naive reads fail, and implement valid alternatives using transactions and concurrency control.

This chapter examines pools, recovery & ticket correctness through concrete mechanisms, state snapshots, failure cases, and alternatives.

10.4.1 · Database connection pooling

A database pool caps active connections and queues acquisition. A larger pool can increase contention and memory rather than throughput. Measure connection hold time, lock waits, idle-in-transaction sessions, and queue age. In an illustrative pool of ten connections with 100 ms mean hold time, the ideal fully busy bound is 100 operations/s before other limits. If callers hold connections while waiting on external HTTP calls, shorten the ownership scope instead of only increasing the pool.

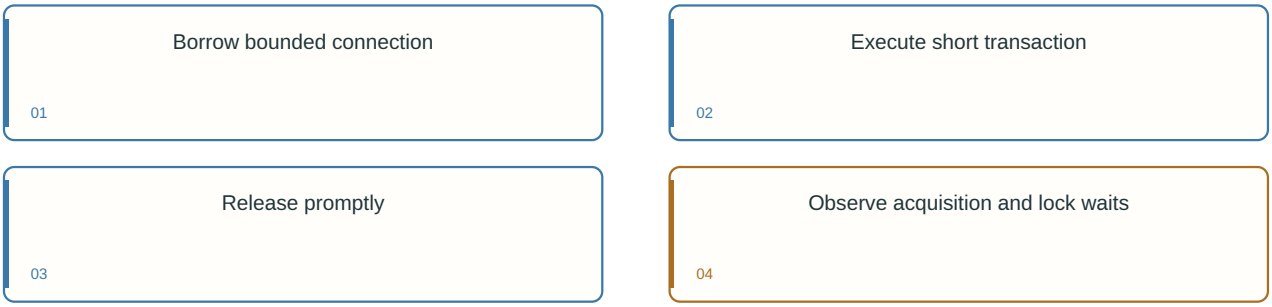


FIG. 10.4.1 **Database connection pooling · worked snapshots**

Borrow bounded connection · Execute short transaction · Release promptly · Observe acquisition and lock waits. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.4.2 · Transactions across multiple operations

A business operation spanning several statements needs one deliberate atomic boundary if its invariant requires all-or-nothing changes. Reserving inventory and recording a local reservation should commit together. An ORM save call is not proof that every related action shares the transaction. Across separate stores or external services, use an explicit distributed protocol or workflow with reconciliation. Transaction scope should be as small as possible while preserving the invariant, and must not leak through lazy results after disposal.

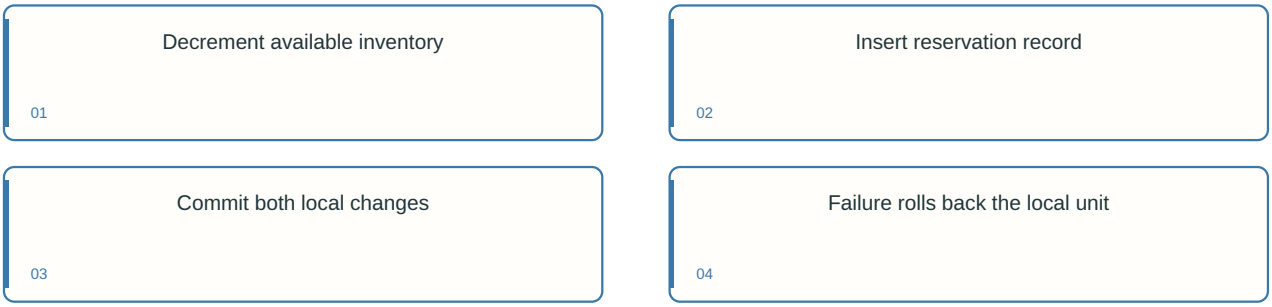


FIG. 10.4.2 **Transactions across multiple operations · worked snapshots**

Decrement available inventory · Insert reservation record · Commit both local changes · Failure rolls back the local unit. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

10.4.3 · Backup, recovery, and durability

Backups provide recoverable historical state; replication keeps live copies and can replicate mistakes. Test restores into a usable environment and verify application invariants, credentials, schema versions, and recovery time. A backup job reporting success does not prove that its archive is complete or decryptable. Specify point-in-time recovery requirements and how logs are retained. Durability is conditional on acknowledged persistence and independent failure protection, not established by merely owning a backup file.

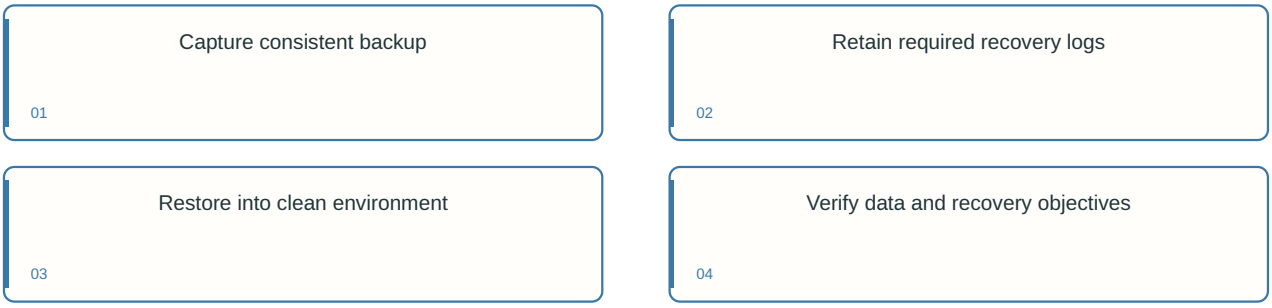


FIG. 10.4.3 **Backup, recovery, and durability · worked snapshots**

Capture consistent backup · Retain required recovery logs · Restore into clean environment · Verify data and recovery objectives. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

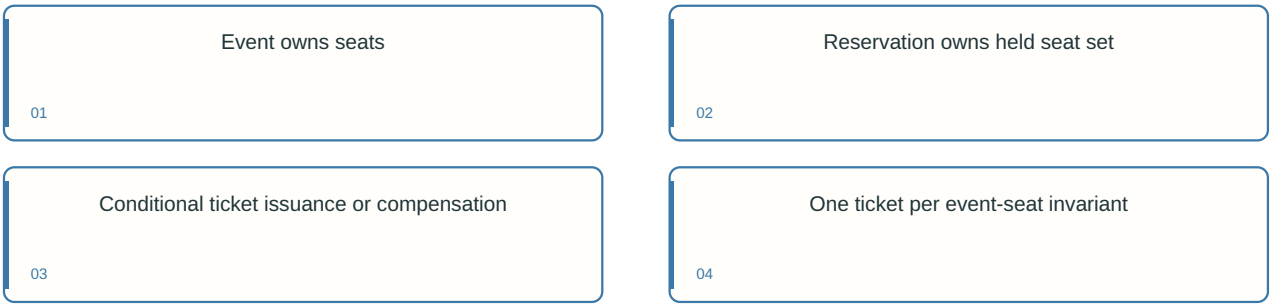


FIG. 10.4.4 **The running system · invariant and recovery review**

Review whether expiry and payment use the same reservation version, whether late payment is resolved safely, and whether the display is confused with authority. Test two buyers for one seat, duplicate callbacks, crash after commit, stale expiry after reassignment, and payment success after expiry. A correct happy-path lock does not answer those workflow histories.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · CONDITIONAL SEAT CLAIM

```
from mechanisms import open_demo_store, claim_seat
db = open_demo_store()
db.execute("INSERT INTO seats VALUES (1,NULL)")
db.commit()
assert claim_seat(db, 1, "A")
assert not claim_seat(db, 1, "B")
db.close()
```

The conditional update owns the local invariant. The verification suite also uses two connections to one store. External payment and expiry must preserve reservation identity and need their own workflow contract.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

Replay the last-seat interleaving

1. A reads available.
2. B reads available.
3. A writes owner A and reports success.
4. B overwrites owner B and reports success.

Two callers believe they won. The final row alone hides the broken acknowledgment history.

Deterministic two-caller interleaving, not a production transaction engine. The executable SQLite fixture verifies the local conditional-update invariant separately.

▼ Architecture review · Which invariant becomes harder when a local multi-statement transaction is split across independently owned databases?

Use claim_seat() against the same store and assert one owner wins while the other receives conflict. Explain which local transaction protects the invariant. Add rollback and crash histories; do not claim a local seat claim makes a remote payment part of the same transaction.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Specify point-in-time recovery requirements and how logs are retained.

Carry forward: Capture consistent backup · Retain required recovery logs · Restore into clean environment · Verify data and recovery objectives.

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

PART 3 / UNIT 11

Caching and Content Delivery

The running example

A popular product page receives enormous traffic. Introduce caching gradually and examine stale prices, thundering herds, inventory correctness, and failure behavior.

Unit 11 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

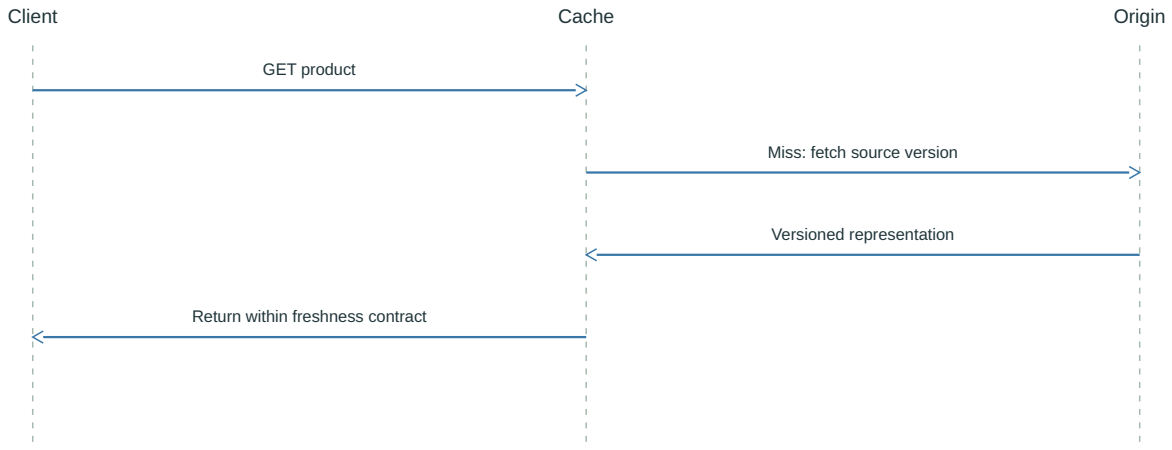
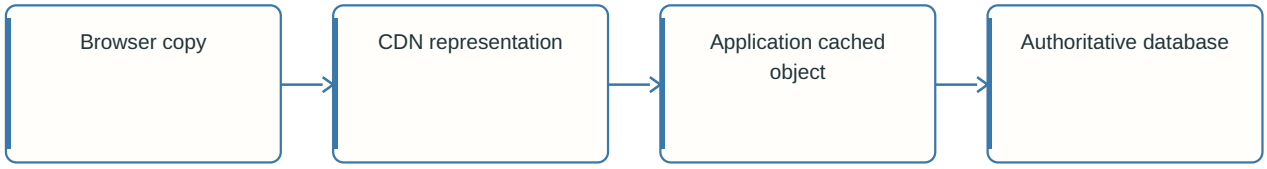


FIG. 11.0.1 **Cache hit/miss sequence**

On a hit, the origin exchange is omitted. Cache-aside population still needs race reasoning.

Why this view: Follow explicit message ordering and locate the acknowledgment or effect boundary. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 11.0.2 **cache hierarchy**

These are distinct copies with separate freshness, identity, and invalidation policies.

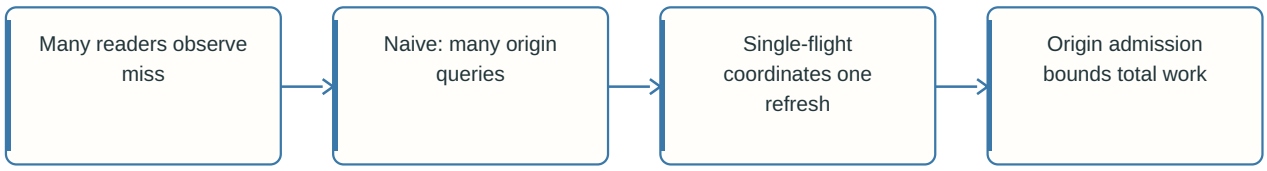
Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Policy	After A,B,C,A	Insert D: evicted
Ideal LRU	Order B,C,A	B
FIFO	Insertion A,B,C	A

FIG. 11.0.3 **LRU and FIFO disagree after the same accesses**

Capacity is three. Both policies saw the same requests; policy metadata changes the decision.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

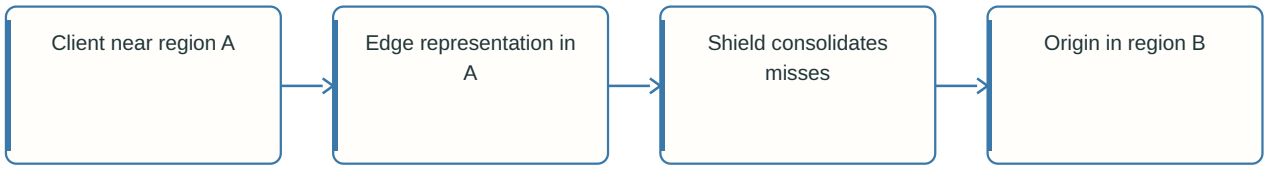


Arrows: ordered stages of this worked mechanism

FIG. 11.0.4 **stampede timeline**

A cache outage or synchronized expiry must not create an unbounded origin reconstruction storm.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 11.0.5 **geographic CDN delivery**

Geographic delivery reduces some distance and origin work while adding copies and freshness obligations.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Locality and cache placement

Prerequisites: Networking Foundations for System Design, Data Models and Storage Technologies 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A popular product page receives enormous traffic. Introduce caching gradually and examine stale prices, thundering herds, inventory correctness, and failure behavior.

This chapter examines locality and cache placement through concrete mechanisms, state snapshots, failure cases, and alternatives.

11.1.1 · Why caching works

Caching works when requests reuse data or nearby data often enough to avoid more expensive work. A product description read thousands of times between edits is a candidate; a unique one-use computation may not be. Compare hit latency, miss cost, maintenance, and correctness. With assumed 1 ms hits, 50 ms misses, and 90% hits, the simple weighted service-time average is 5.9 ms, excluding queues and other work. A hit ratio alone does not describe tail latency or cost.

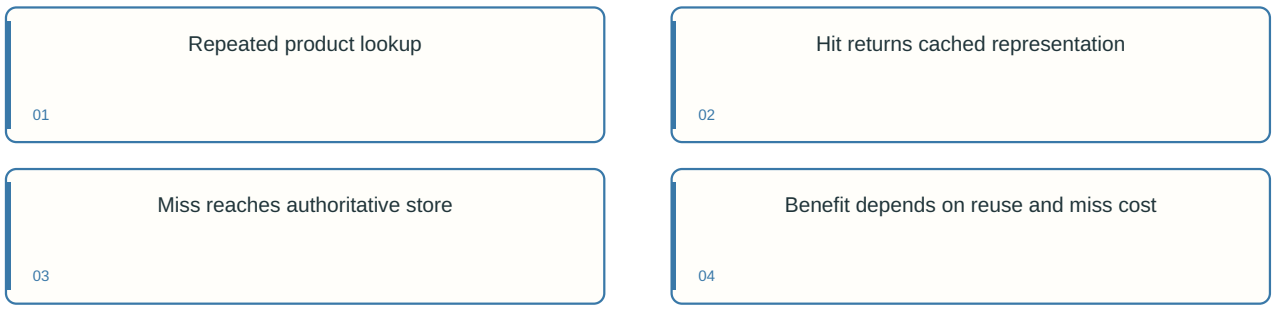


FIG. 11.1.1 **Why caching works · worked snapshots**

Repeated product lookup · Hit returns cached representation · Miss reaches authoritative store · Benefit depends on reuse and miss cost. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.2 · Cache locality and the working set

Temporal locality is reuse over time; spatial locality is reuse of nearby data. The working set is the data actively needed over a relevant interval. A cache too small for that set may churn even if total historical data is enormous. Scan traffic can evict useful entries under some policies. Measure bytes, object sizes, skew, and admission behavior, not only key count. Caching every object eagerly can waste memory on cold data while crowding out hot content.

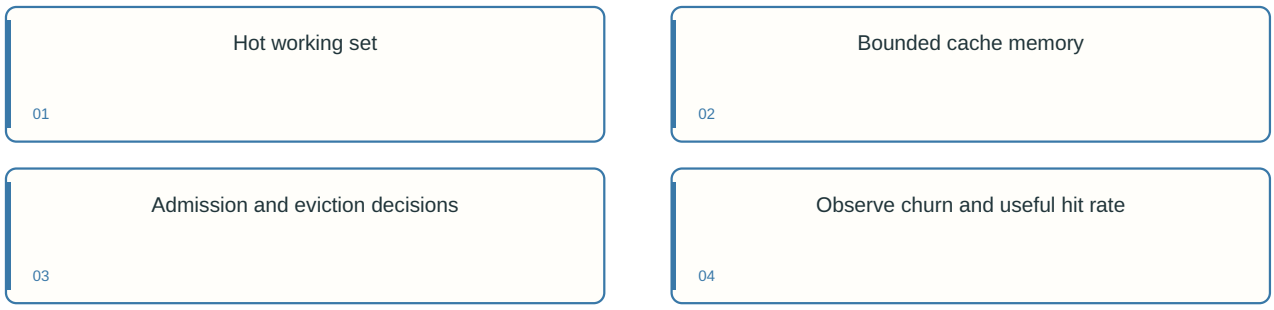


FIG. 11.1.2 **Cache locality and the working set · worked snapshots**

Hot working set · Bounded cache memory · Admission and eviction decisions · Observe churn and useful hit rate. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.3 · In-process vs distributed caches

An in-process cache is fast and simple but creates independent copies per application instance. A distributed cache shares a network-accessible state layer, adding serialization, connection, and availability costs. If two app instances cache price 10 and 12, reads differ until policy resolves freshness. Do not place an invariant such as inventory allocation solely in an ordinary stale cache. Choose the location by reuse, memory duplication, required coordination, and failure behavior.

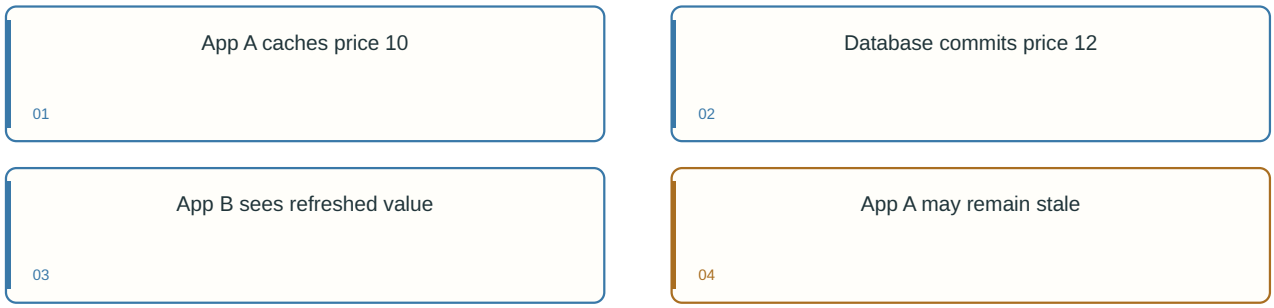


FIG. 11.1.3 **In-process vs distributed caches · worked snapshots**

App A caches price 10 · Database commits price 12 · App B sees refreshed value · App A may remain stale. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.4 · Browser, CDN, application, and database caches

Browser, CDN, application, and database caches sit at different ownership and trust boundaries. A personalized response cached at a public edge without the right key can leak data. A database buffer cache stores pages, while an application cache stores computed representations; invalidating one does not invalidate the other. Draw every copy and define freshness at each layer. Diagnosing a stale response requires knowing which layer answered and which version it held.

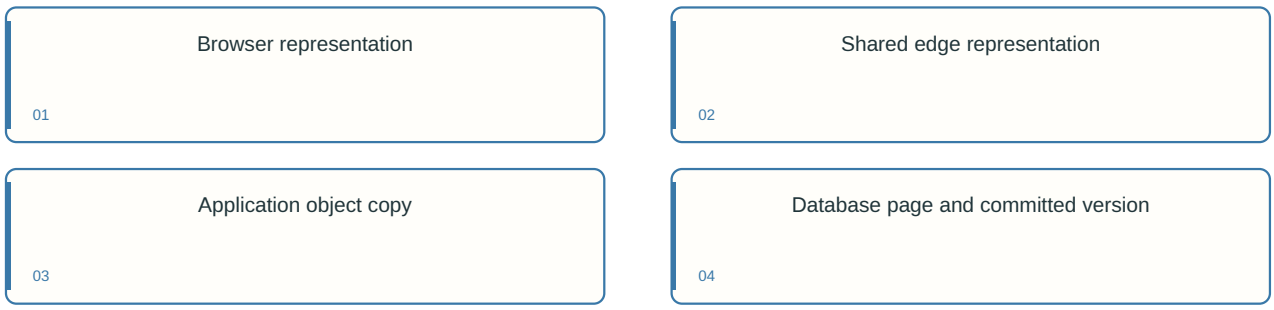


FIG. 11.1.4 **Browser, CDN, application, and database caches · worked snapshots**

Browser representation · Shared edge representation · Application object copy · Database page and committed version. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · When would a product choose lower availability rather than serve a stale cached value?

At capacity three, A,B,C,A,D evicts B under ideal LRU. Then pause a reader holding version one, commit version two and invalidate, and resume the stale population. Version checks or another justified protocol must prevent an invalid freshness claim; a short TTL only changes exposure.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Draw every copy and define freshness at each layer.

Carry forward: Browser representation · Shared edge representation · Application object copy · Database page and committed version.

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

Copies, freshness & eviction

Prerequisites: 11.1 · Locality and cache placement 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A popular product page receives enormous traffic. Introduce caching gradually and examine stale prices, thundering herds, inventory correctness, and failure behavior.

This chapter examines copies, freshness & eviction through concrete mechanisms, state snapshots, failure cases, and alternatives.

11.2.1 · Cache-aside, read-through, write-through, write-behind

Cache-aside loads after a miss; read-through delegates loading to the cache interface; write-through updates a backing store in the write path; write-behind defers backing persistence. Name the actual ordering and failure semantics because implementations vary. In write-behind, acknowledging only a volatile cache can lose accepted changes. Cache-aside still has races: a delayed reader can populate an old value after a newer write invalidated the key. Use versioning or stronger coordination when freshness requires it.

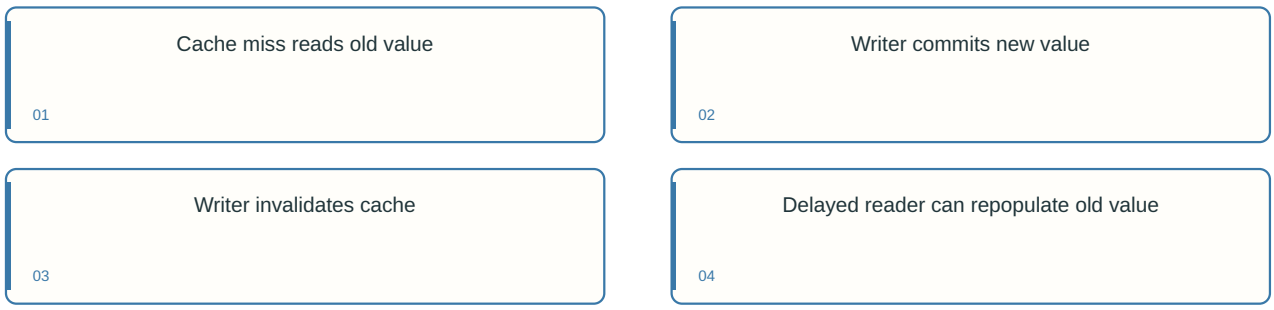


FIG. 11.2.1 **Cache-aside, read-through, write-through, write-behind · worked snapshots**

Cache miss reads old value · Writer commits new value · Writer invalidates cache · Delayed reader can repopulate old value. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.2 · TTL and cache expiration

A TTL bounds intended residence after insertion, not necessarily the age of the underlying fact when inserted. A cached value already five minutes old with a one-minute TTL can remain six minutes behind its source. Expiration is distinct from eviction, and clients may retain their own copies. Use jitter to avoid synchronized expirations. Choose TTL from tolerated staleness and refresh behavior; correctness-critical actions should validate authoritative state rather than trust a display cache.

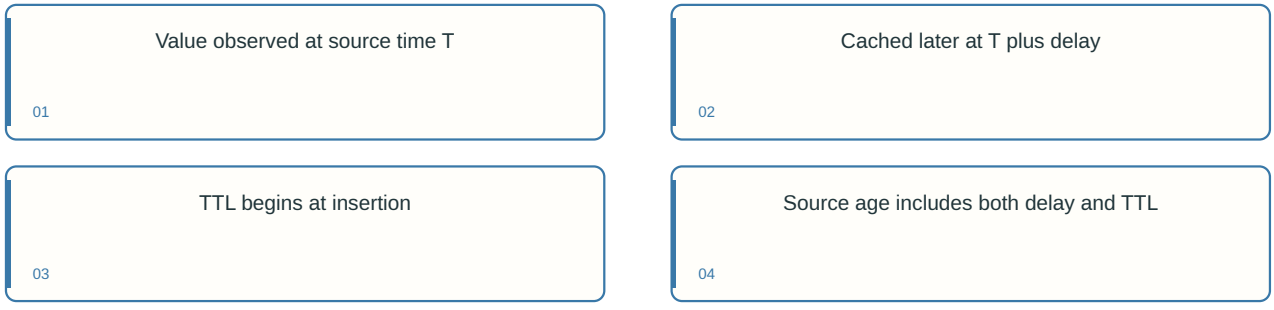


FIG. 11.2.2 **TTL and cache expiration · worked snapshots**

Value observed at source time T · Cached later at T plus delay · TTL begins at insertion · Source age includes both delay and TTL. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.3 · Cache consistency and invalidation

Invalidation coordinates derived copies with authoritative changes. Delete-after-write is common but cannot by itself prevent every stale repopulation race. Versioned cache entries, conditional population, or source-version checks can strengthen semantics. Events for invalidation can lag or repeat, so observe version and propagation time. For prices, stale display may be acceptable while checkout revalidates the current price; for inventory, accepting stale availability as authority can oversell. Match the cache contract to the operation's invariant.

Location	Version and value	Authority
Database	v2: price 12	current authoritative value
App cache A	v1: price 10	stale representation
Checkout	revalidates database	does not trust display cache

FIG. 11.2.3 **A stale cache is a separate state copy**

Invalidation and propagation policy decide when copies agree. The critical checkout operation uses the required authoritative contract.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.4 · Eviction: LRU, LFU, FIFO, and alternatives

LRU favors recent access; LFU favors frequent access; FIFO evicts insertion order. Their precise implementations may be approximate. For capacity three and accesses A,B,C,A,D, ideal LRU evicts B; FIFO evicts A; a frequency-based policy also needs tie-breaking. Compare policies with representative traffic and object sizes. A clever eviction algorithm cannot fix a working set vastly beyond memory or a cache that stores low-value objects indiscriminately. Admission and eviction are separate choices.

Policy	After A,B,C,A	Insert D: evicted
Ideal LRU	Order B,C,A	B
FIFO	Insertion A,B,C	A

FIG. 11.2.4 **LRU and FIFO disagree after the same accesses**

Capacity is three. Both policies saw the same requests; policy metadata changes the decision.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · When would a product choose lower availability rather than serve a stale cached value?

At capacity three, A,B,C,A,D evicts B under ideal LRU. Then pause a reader holding version one, commit version two and invalidate, and resume the stale population. Version checks or another justified protocol must prevent an invalid freshness claim; a short TTL only changes exposure.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A clever eviction algorithm cannot fix a working set vastly beyond memory or a cache that stores low-value objects indiscriminately.

Carry forward: Capacity: 3 entries · Access A, B, C, A · Insert D · LRU removes B; FIFO removes A.

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

Hot keys, stampedes & negative results

Prerequisites: 11.2 · Copies, freshness & eviction 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A popular product page receives enormous traffic. Introduce caching gradually and examine stale prices, thundering herds, inventory correctness, and failure behavior.

This chapter examines hot keys, stampedes & negative results through concrete mechanisms, state snapshots, failure cases, and alternatives.

11.3.1 · Hot keys

A hot key concentrates traffic on one shard or owner. More cache nodes do not distribute a single key automatically. Options include local read copies, replication where supported, request coalescing, or deliberate partitioning of commutative aggregates. Each affects freshness or aggregation cost. For a stock count, blindly splitting into independent counters can violate allocation correctness. First identify whether the pressure is read bandwidth, write serialization, object size, or repeated recomputation.

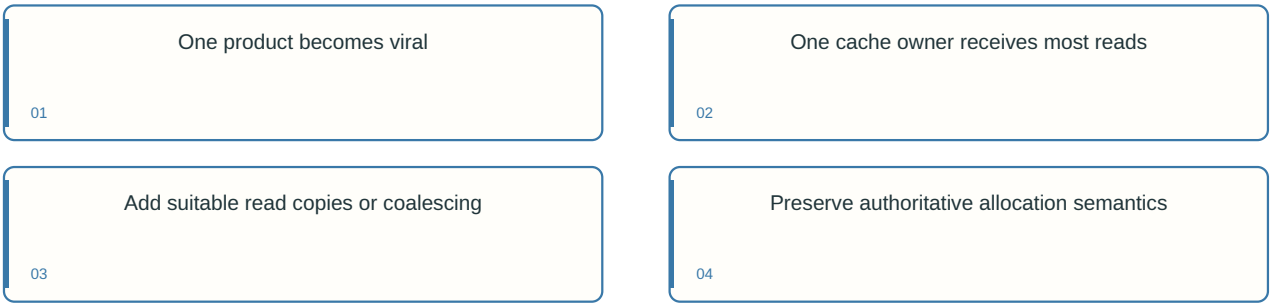


FIG. 11.3.1 **Hot keys · worked snapshots**

One product becomes viral · One cache owner receives most reads · Add suitable read copies or coalescing · Preserve authoritative allocation semantics. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.2 · Cache stampedes, penetration, and avalanche

A stampede occurs when many misses trigger duplicate expensive work. Penetration repeatedly requests absent keys; an avalanche is a broad synchronized loss or expiry of cached entries. Use single-flight loading, bounded refresh workers, negative caching, TTL jitter, and origin admission limits as appropriate. Locks used for refresh also need expiration and failure handling. A cache outage should not let every application instance independently overwhelm the database while attempting to reconstruct the entire working set.

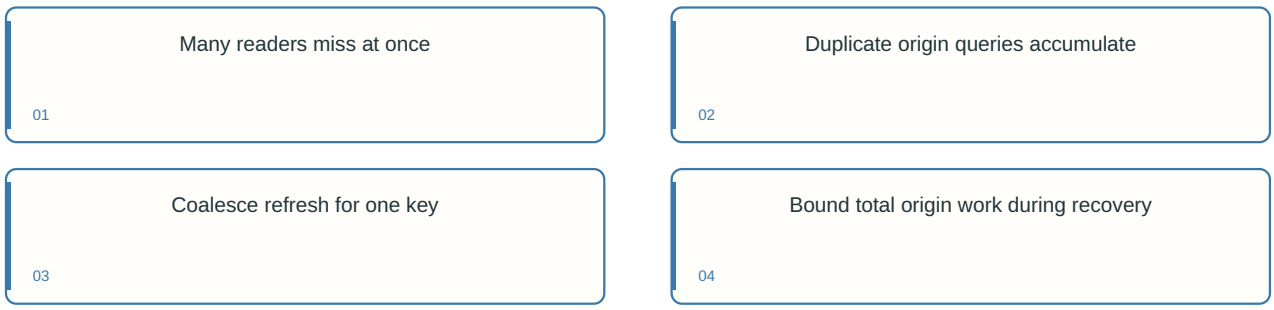


FIG. 11.3.2 **Cache stampedes, penetration, and avalanche · worked snapshots**

Many readers miss at once · Duplicate origin queries accumulate · Coalesce refresh for one key · Bound total origin work during recovery. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.3 · Negative caching

Negative caching stores an absence or error under a narrow policy. Caching “product not found” for a short interval can protect against repeated nonexistent lookups. Do not cache a temporary database failure as permanent absence, and do not let one tenant's authorization failure poison another tenant's key. A newly created product may remain hidden until negative-entry expiry or invalidation. Use separate error types, short appropriate lifetimes, and correctly scoped keys.

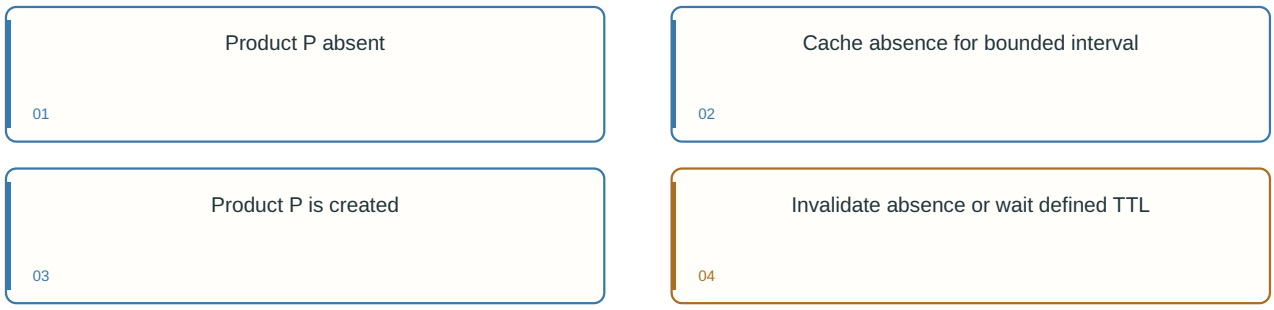


FIG. 11.3.3 **Negative caching · worked snapshots**

Product P absent · Cache absence for bounded interval · Product P is created · Invalidate absence or wait defined TTL. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.4 · Stale-while-revalidate

Stale-while-revalidate serves an allowed stale value while one refresh updates it. This trades freshness for latency and origin protection; it is not suitable for every operation. A cached description can be stale, while payment authorization cannot be inferred from an old response. Define fresh, stale-allowed, and unusable age limits. If refresh repeatedly fails, do not extend stale serving forever unless that is an explicit degradation contract visible to users and operators.

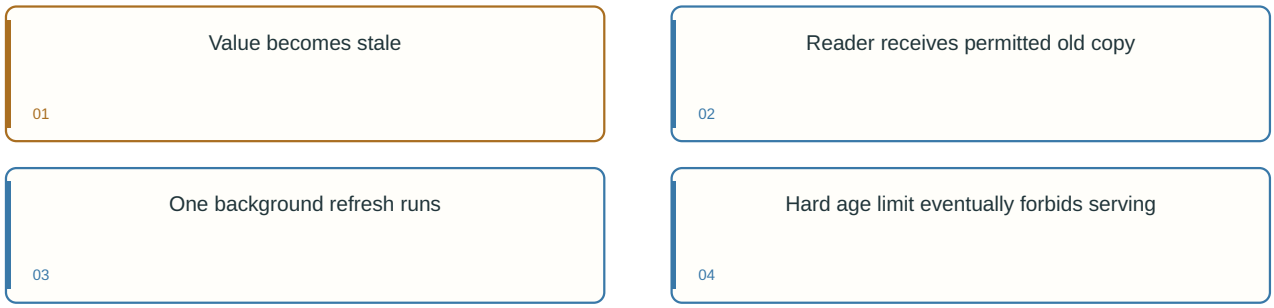


FIG. 11.3.4 **Stale-while-revalidate · worked snapshots**

Value becomes stale · Reader receives permitted old copy · One background refresh runs · Hard age limit eventually forbids serving. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.5 · Cache warming and prefetching

Warming preloads likely useful entries; prefetching predicts near-future requests. Both consume bandwidth and may evict more valuable content. Warm a measured working set gradually after deployment rather than issuing a full-database scan from every new instance. Track whether prefetched entries are actually used and cancel obsolete work. A cold-start problem may be better addressed through staggered rollout, shared cache layers, or request admission than an increasingly elaborate prediction system.

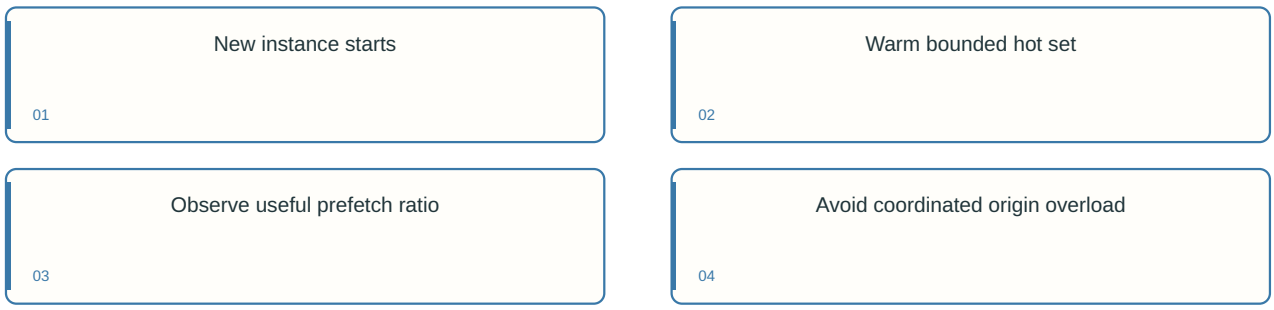


FIG. 11.3.5 **Cache warming and prefetching · worked snapshots**

New instance starts · Warm bounded hot set · Observe useful prefetch ratio · Avoid coordinated origin overload. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · When would a product choose lower availability rather than serve a stale cached value?

At capacity three, A,B,C,A,D evicts B under ideal LRU. Then pause a reader holding version one, commit version two and invalidate, and resume the stale population. Version checks or another justified protocol must prevent an invalid freshness claim; a short TTL only changes exposure.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Track whether prefetched entries are actually used and cancel obsolete work.

Carry forward: New instance starts · Warm bounded hot set · Observe useful prefetch ratio · Avoid coordinated origin overload.

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

Edges, partitions & the price of a cache

Prerequisites: 11.3 · Hot keys, stampedes & negative results 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A popular product page receives enormous traffic. Introduce caching gradually and examine stale prices, thundering herds, inventory correctness, and failure behavior.

This chapter examines edges, partitions & the price of a cache through concrete mechanisms, state snapshots, failure cases, and alternatives.

11.4.1 · Redis-like data structures

Redis-like structures include strings, hashes, lists, sets, sorted sets, and streams, each with different operations and cost models. A sorted set can track a time-ordered window; a set can deduplicate identifiers within a lifecycle. Atomicity of one command is not automatically atomicity of a multi-command workflow. Server-side scripts or transactions may provide the needed local boundary, but durability, replication, cluster key placement, and expiry still matter. Do not assume every structure is a durable queue by default.

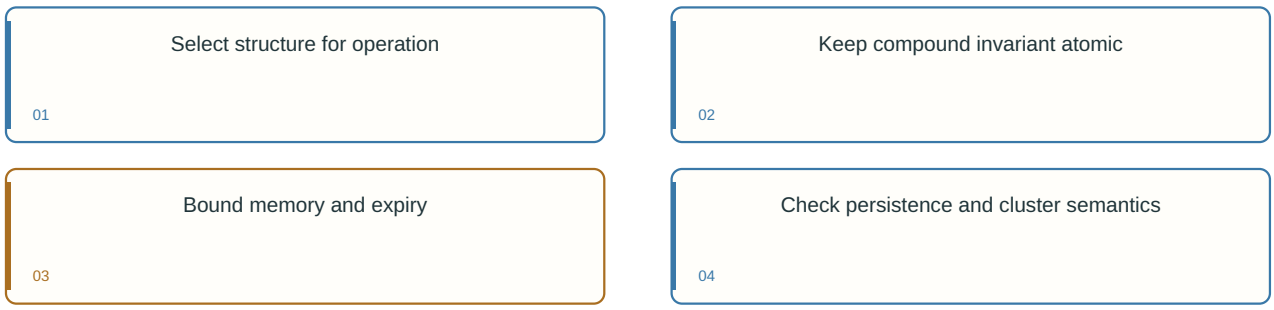


FIG. 11.4.1 **Redis-like data structures · worked snapshots**

Select structure for operation · Keep compound invariant atomic · Bound memory and expiry · Check persistence and cluster semantics. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.2 · CDN architecture and origin shielding

An edge cache serves representations near clients, while an origin shield can consolidate misses before the origin. Signed URLs and cache keys need policy: query parameters, headers, tenant identity, and content versions can change what is safe to share. A purge request propagates under the provider's contract and may not invalidate every client-held copy. Use immutable versioned asset URLs when practical. Delivery improves bandwidth and latency for suitable content but does not authorize serving private data globally.

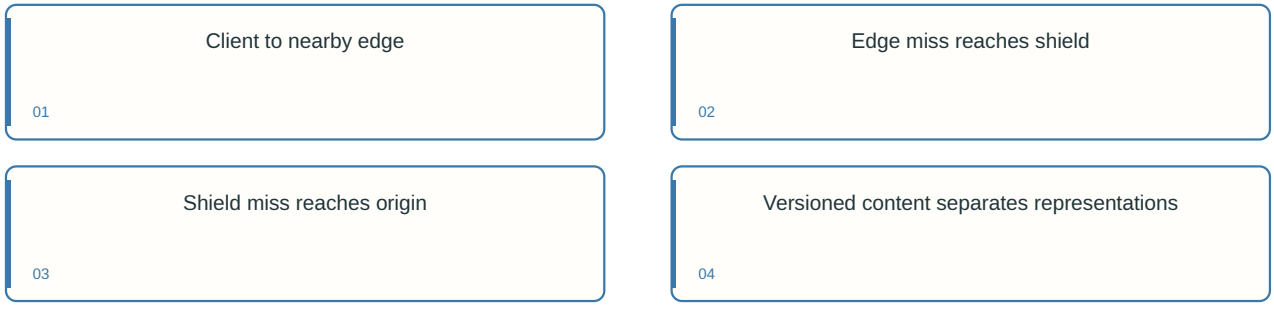


FIG. 11.4.2 **CDN architecture and origin shielding · worked snapshots**

Client to nearby edge · Edge miss reaches shield · Shield miss reaches origin · Versioned content separates representations. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.3 · Cache partitioning and replication

Partitioning distributes keys; replication creates copies for resilience or reads under a protocol. A cache may lose or evict entries during failover, and replicas may lag. The application must decide whether to fall back, serve allowed stale data, or reject. Consistent hashing reduces some remapping during membership changes but cannot prevent hot-key skew. Keep cache and authoritative-store failure domains and recovery capacity explicit; rebuilding after a cluster event can be the real peak workload.

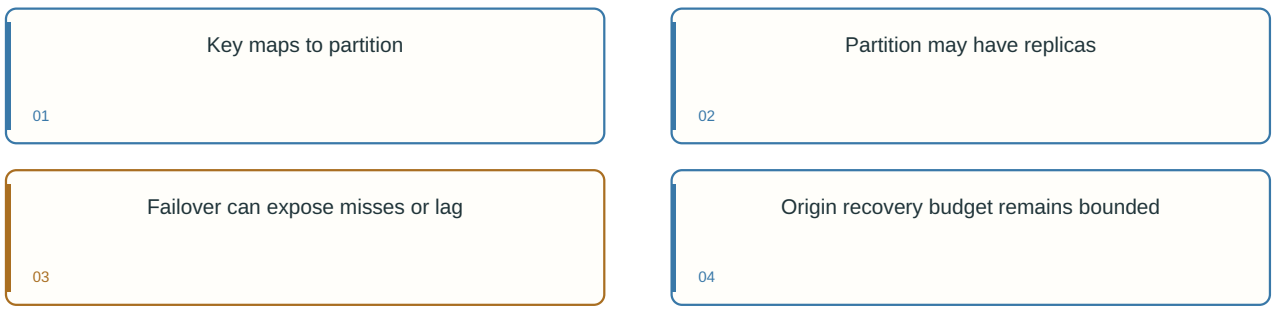


FIG. 11.4.3 **Cache partitioning and replication · worked snapshots**

Key maps to partition · Partition may have replicas · Failover can expose misses or lag · Origin recovery budget remains bounded. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.4 · When caching is harmful

Caching is harmful when maintenance and correctness cost exceed saved work. A low-reuse result cache may consume memory and serialization time without many hits. A cache with missing authorization scope can leak data; one that masks a slow database during normal operation can hide an origin capacity deficit until outage. Test the system with the cache empty or unavailable. Prefer query/index improvements or simpler data access when they address the bottleneck with fewer state copies.

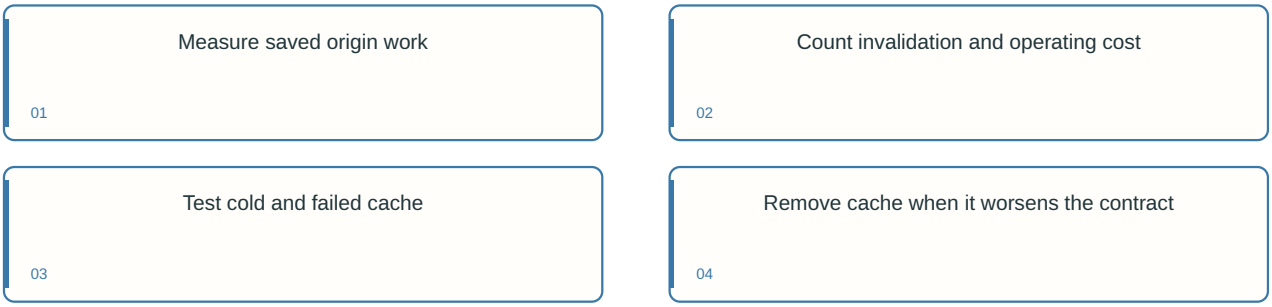


FIG. 11.4.4 **When caching is harmful · worked snapshots**

Measure saved origin work · Count invalidation and operating cost · Test cold and failed cache · Remove cache when it worsens the contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · IDEAL LRU

```
from mechanisms import LRU
cache = LRU(3)
for key in "ABC": cache.put(key, key)
cache.get("A")
assert cache.put("D", "D") == ("B", "B")
```

The ideal LRU example is deterministic. Production engines can approximate policy, include byte limits, and treat expiration separately. Eviction behavior alone says nothing about source freshness or safe cache-aside population.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

Inspect exact LRU state after each access

1. A: miss · oldest → newest: A
2. B: miss · oldest → newest: A, B
3. C: miss · oldest → newest: A, B, C
4. A: hit · oldest → newest: B, C, A
5. D: miss, evict B · oldest → newest: C, A, D

Ideal entry-count LRU; each access loads or refreshes a key. Real engines may approximate eviction and have byte-size and expiration policies. No source freshness guarantee follows.

▼ Architecture review · When would a product choose lower availability rather than serve a stale cached value?

At capacity three, A,B,C,A,D evicts B under ideal LRU. Then pause a reader holding version one, commit version two and invalidate, and resume the stale population. Version checks or another justified protocol must prevent an invalid freshness claim; a short TTL only changes exposure.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Test the system with the cache empty or unavailable.

Carry forward: Measure saved origin work · Count invalidation and operating cost · Test cold and failed cache · Remove cache when it worsens the contract.

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

PART 3 / UNIT 12

Scalability, Load Balancing, and Data Distribution

The running example

Scale a URL shortener from 100 requests per second to 100,000. Introduce new components only when justified by a demonstrated bottleneck.

Unit 12 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

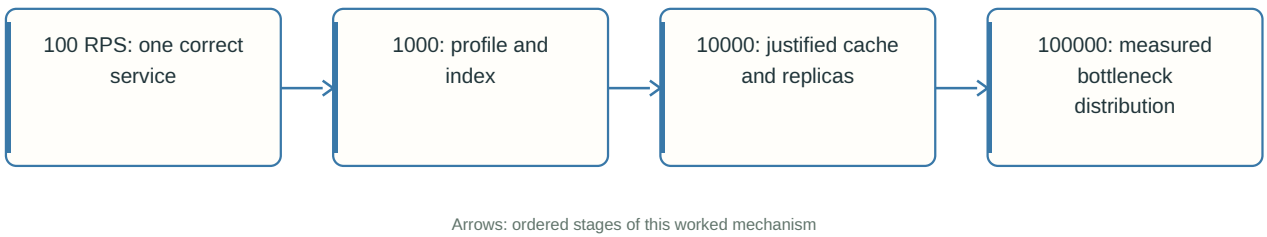
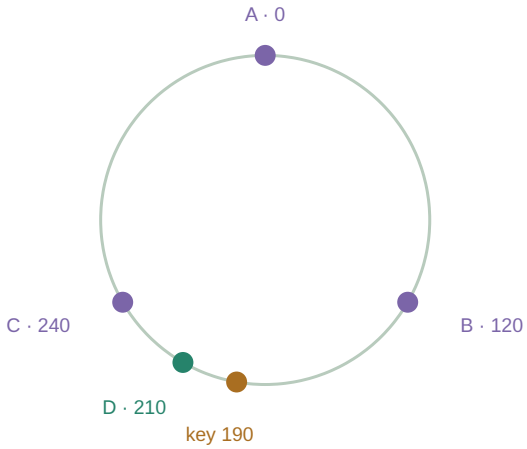


FIG. 12.0.1 **Architecture evolution across four load levels**

These are conceptual teaching checkpoints, not hardware benchmark thresholds.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.



Clockwise successor: key 190 moves from C to new owner D.

FIG. 12.0.2 **A new owner takes only part of the hash space**

With clockwise-successor ownership, key 190 belongs to C at 240. Adding D at 210 moves it to D; the affected interval is (120,210]. Hash coordinates are illustrative.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

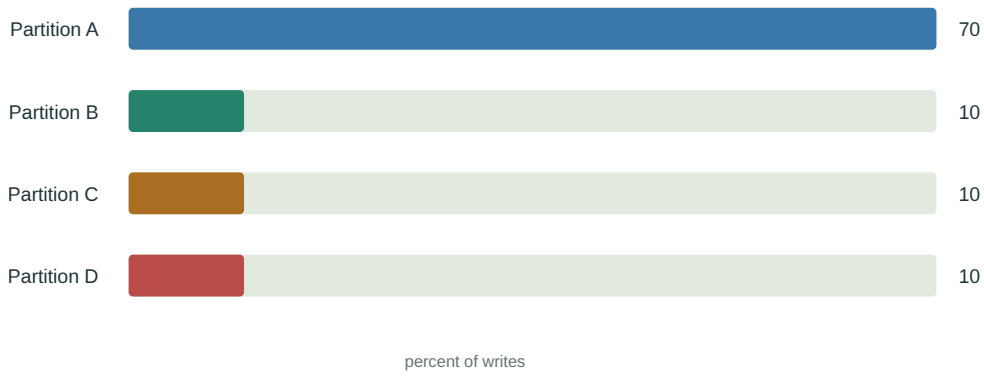
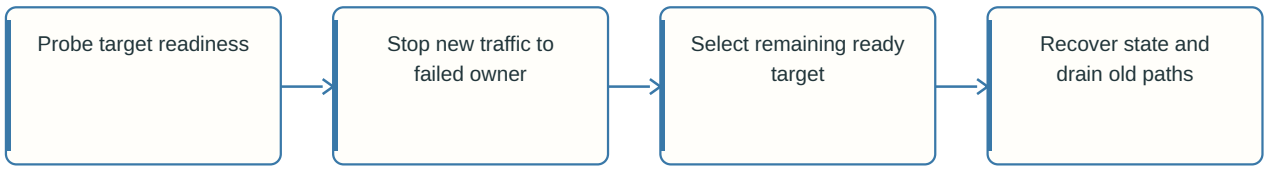


FIG. 12.0.3 **Averages hide a hot partition**

Seventy percent of writes go to A. Adding cold capacity does not split one hot ownership key.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 12.0.4 **load-balancer failover**

Routing failover does not by itself establish fresh state or preserve uncommitted work.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Scale compute & route requests

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Networking Foundations for System Design, Data Models and Storage Technologies, Database Internals, Transactions, and Concurrency Control, Caching and Content Delivery

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Scale a URL shortener from 100 requests per second to 100,000. Introduce new components only when justified by a demonstrated bottleneck.

This chapter examines scale compute & route requests through concrete mechanisms, state snapshots, failure cases, and alternatives.

12.1.1 · Vertical vs horizontal scaling

Vertical scaling increases resources of one node; horizontal scaling adds nodes. Neither guarantees proportional throughput. A larger database host may buy time without distributed data management, while extra stateless application instances can distribute independent compute. Shared locks, disks, or downstream services can remain limiting. Compare failure impact, scaling delay, cost, and operational complexity. Choose the smallest change supported by profiling, then rerun the workload because the limiting resource can move.

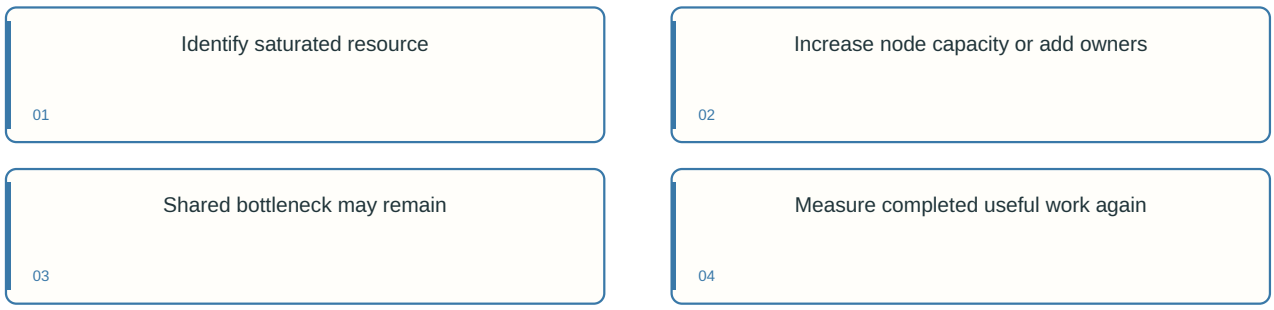


FIG. 12.1.1 **Vertical vs horizontal scaling · worked snapshots**

Identify saturated resource · Increase node capacity or add owners · Shared bottleneck may remain · Measure completed useful work again. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.2 · Stateless vs stateful services

A stateless service retains no required per-client conversational state locally between requests; it still operates on state elsewhere. This makes instance replacement and traffic distribution easier. A server-local session or uncommitted job can prevent transparent failover. Move required state to an appropriate owner, or make affinity and recovery explicit. Sending every request to any instance is safe only if identity, state access, configuration, and side-effect handling are compatible across those instances.

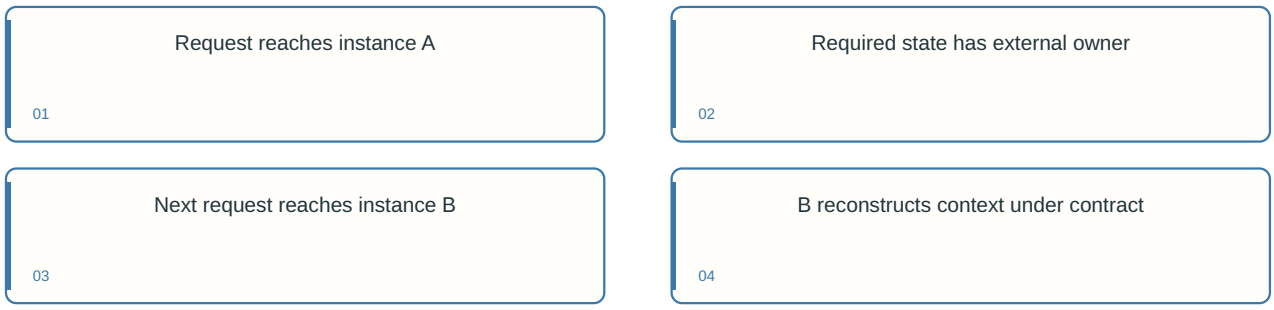


FIG. 12.1.2 **Stateless vs stateful services · worked snapshots**

Request reaches instance A · Required state has external owner · Next request reaches instance B · B reconstructs context under contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.3 · Load distribution algorithms

Round-robin spreads selections, least-connections approximates active load, weighted policies reflect capacity, and hashing can preserve affinity. Their effectiveness depends on request cost and connection behavior. One expensive request can occupy a backend much longer than many cheap ones. Observe queueing and per-instance saturation, not only request counts. Client-side or server-side balancing also affects discovery, retries, and failure detection. A distribution algorithm cannot create capacity at a saturated shared database.

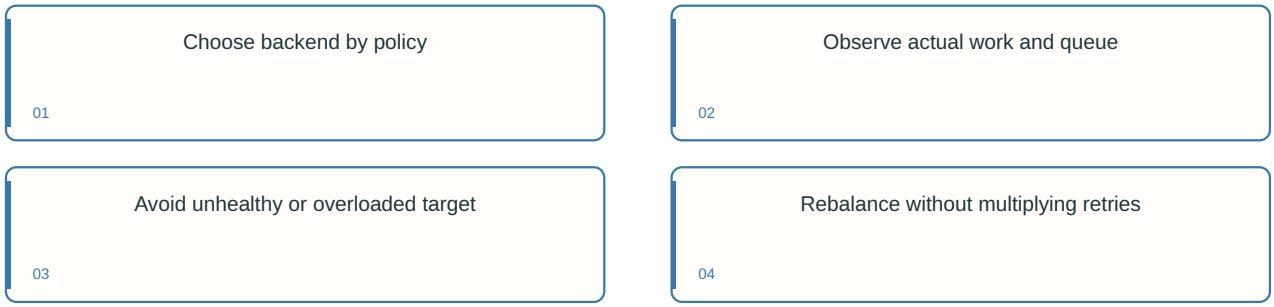


FIG. 12.1.3 **Load distribution algorithms · worked snapshots**

Choose backend by policy · Observe actual work and queue · Avoid unhealthy or overloaded target · Rebalance without multiplying retries. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.4 · Health checks and failover

Health checks approximate readiness, not omniscient correctness. Liveness asks whether a process should be restarted; readiness asks whether it should receive traffic under the chosen policy. A check that requires every downstream service can remove all instances during a shared dependency outage and worsen recovery. Use meaningful local signals, bounded checks, and deliberate degradation policy. Failover also requires state and connection behavior that support the switch; routing alone does not recover uncommitted work.

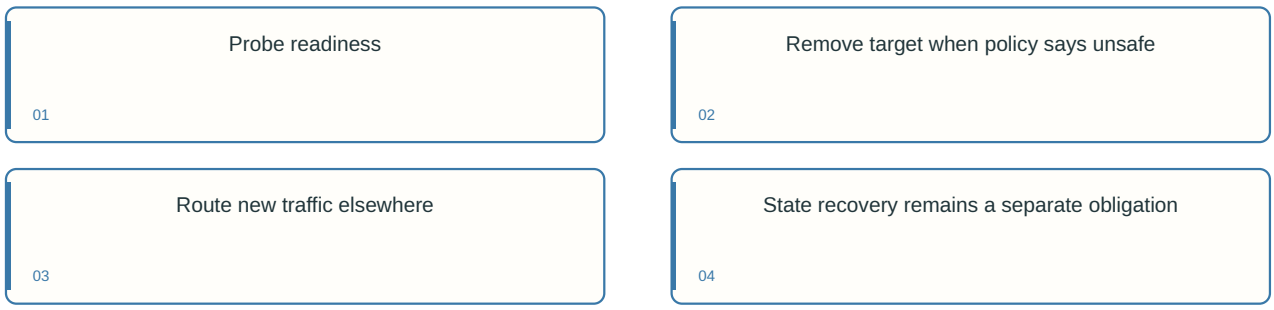


FIG. 12.1.4 **Health checks and failover · worked snapshots**

Probe readiness · Remove target when policy says unsafe · Route new traffic elsewhere · State recovery remains a separate obligation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.5 · Sticky sessions

Sticky sessions keep a client associated with an instance and can support local state or reduce cache misses. They also create uneven load and complicate failover. A user's connection may outlive the instance, so required state needs recovery or an explicit reconnection protocol. Prefer external session ownership for simple replaceable web instances when feasible. Affinity is an optimization or a deliberate constraint, not a substitute for deciding where authoritative state lives.

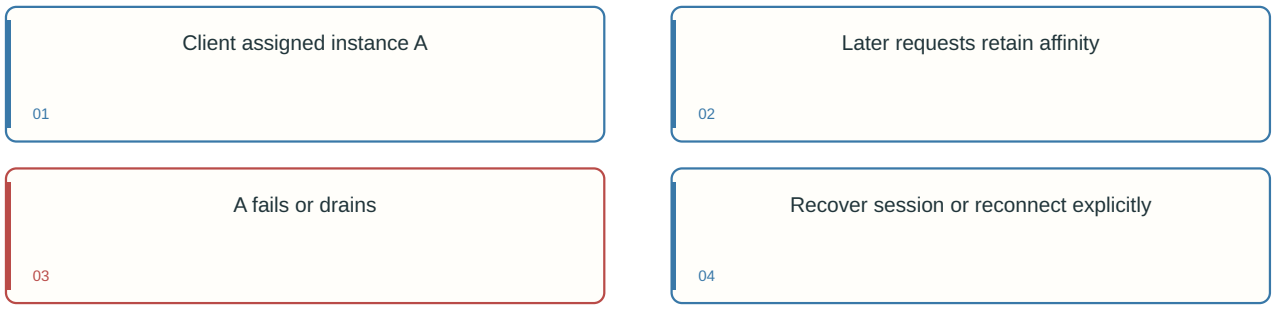


FIG. 12.1.5 **Sticky sessions · worked snapshots**

Client assigned instance A · Later requests retain affinity · A fails or drains · Recover session or reconnect explicitly. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.6 · Autoscaling metrics

Autoscaling reacts to a measured signal, with delay and limits. CPU may suit compute-heavy workers, queue age may better represent background latency, and connection saturation may expose an I/O bottleneck. Scaling on request count alone assumes similar cost per request. Use stabilization and maximums to avoid oscillation or runaway spend. Provision startup headroom and admission control so a sudden spike does not exhaust the system before new capacity becomes ready.

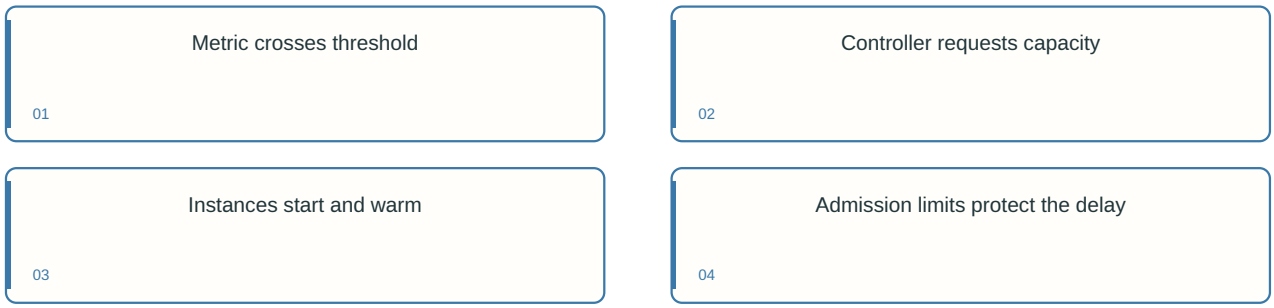


FIG. 12.1.6 **Autoscaling metrics · worked snapshots**

Metric crosses threshold · Controller requests capacity · Instances start and warm · Admission limits protect the delay. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can a single globally hot invariant be scaled merely by adding more partitions? Explain the constraint.

For many deterministic keys, a key either keeps its old owner or moves to the new owner in this model. That property does not guarantee balanced request traffic. Make one key extremely popular and show why adding cold owners cannot split that single-key pressure without a new serving strategy.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Use stabilization and maximums to avoid oscillation or runaway spend.

Carry forward: Metric crosses threshold · Controller requests capacity · Instances start and warm · Admission limits protect the delay.

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

Replicate and partition data

Prerequisites: 12.1 · Scale compute & route requests 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Scale a URL shortener from 100 requests per second to 100,000. Introduce new components only when justified by a demonstrated bottleneck.

This chapter examines replicate and partition data through concrete mechanisms, state snapshots, failure cases, and alternatives.

12.2.1 · Replication

Replication maintains copies under an acknowledgment and reconciliation protocol. It can improve availability or read distribution, but may introduce lag, conflict, and acknowledged-write-loss risks depending on configuration. Async replication can acknowledge before a replica has the write; sync replication can increase latency or block during failure. State which replicas are required for acknowledgment and which failures promotion tolerates. A second copy is not a proof of durability without independent failure domains and recovery rules.

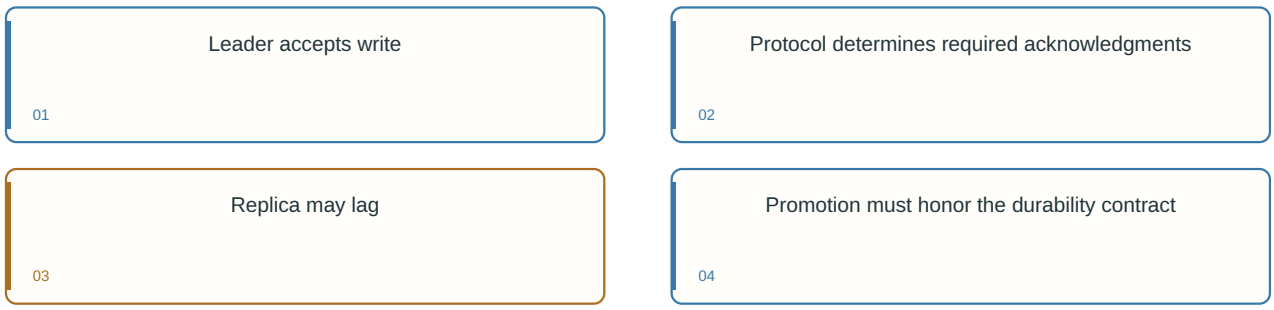


FIG. 12.2.1 **Replication · worked snapshots**

Leader accepts write · Protocol determines required acknowledgments · Replica may lag · Promotion must honor the durability contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.2 · Partitioning and sharding

Partitioning divides a dataset; sharding commonly distributes those partitions across separate storage owners. Choose a key that supports important queries and distributes work. Sharding by tenant can isolate some traffic but leave one large tenant hot. Cross-shard joins, uniqueness, transactions, and migrations become more complex. Keep a routing model and relocation procedure. Add partitioning when a measured storage or write bottleneck justifies it, rather than assuming every large user count requires it.

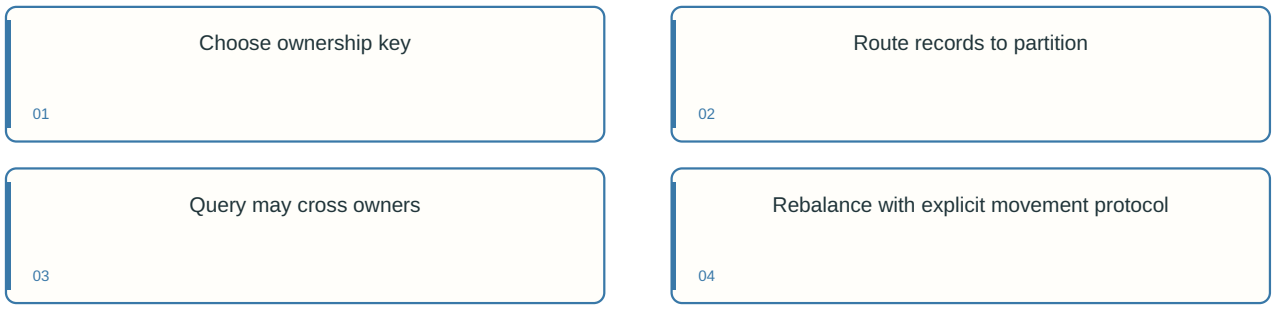


FIG. 12.2.2 **Partitioning and sharding · worked snapshots**

Choose ownership key · Route records to partition · Query may cross owners · Rebalance with explicit movement protocol. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.3 · Range partitioning vs hash partitioning

Range partitioning preserves ordered locality and can make time-based retention efficient, but a newest range may receive most writes. Hash partitioning spreads keys more evenly while making ordered scans and range locality harder. Neither avoids skew if one key dominates. For event ingestion, partitioning only by current day concentrates all writers; combining a time bucket with a spread dimension can help at the cost of multi-partition reads. Define the resulting query fan-out.

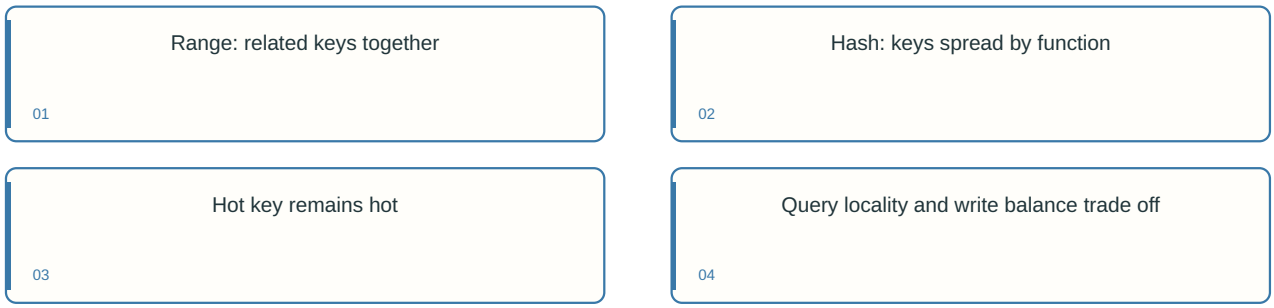


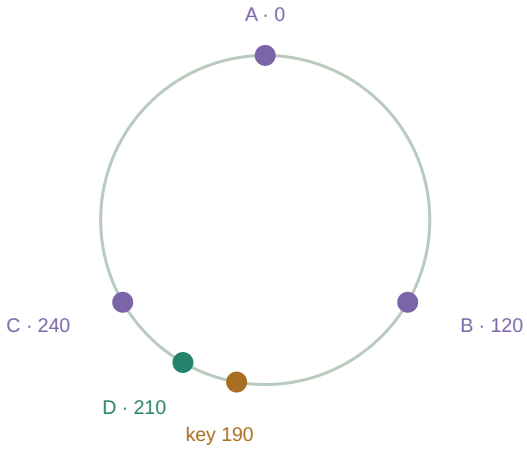
FIG. 12.2.3 **Range partitioning vs hash partitioning · worked snapshots**

Range: related keys together · Hash: keys spread by function · Hot key remains hot · Query locality and write balance trade off. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.4 · Consistent hashing and rendezvous hashing

Consistent hashing places keys and owners in a mapping that changes only part of the assignment when membership changes; virtual nodes can improve distribution. Rendezvous hashing scores each key-owner pair and chooses the highest eligible score, offering a different implementation of stable assignment. Neither guarantees equal traffic because request popularity is not uniform. Membership agreement, migration, replication, and hot-key handling remain required. Avoid changing hash functions without a plan for old data and client routing.



Clockwise successor: key 190 moves from C to new owner D.

FIG. 12.2.4 **A new owner takes only part of the hash space**

With clockwise-successor ownership, key 190 belongs to C at 240. Adding D at 210 moves it to D; the affected interval is (120,210]. Hash coordinates are illustrative.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.5 · Rebalancing

Rebalancing moves data or ownership while serving traffic. A robust move copies a snapshot, captures concurrent changes, validates catch-up, and changes routing under an ownership epoch or equivalent protocol. Readers and writers must not disagree indefinitely about the owner. Throttle movement so it does not consume all production I/O. Keep a rollback and reconciliation path; deleting the old copy before the new owner is authoritative can turn a routine capacity change into data loss.

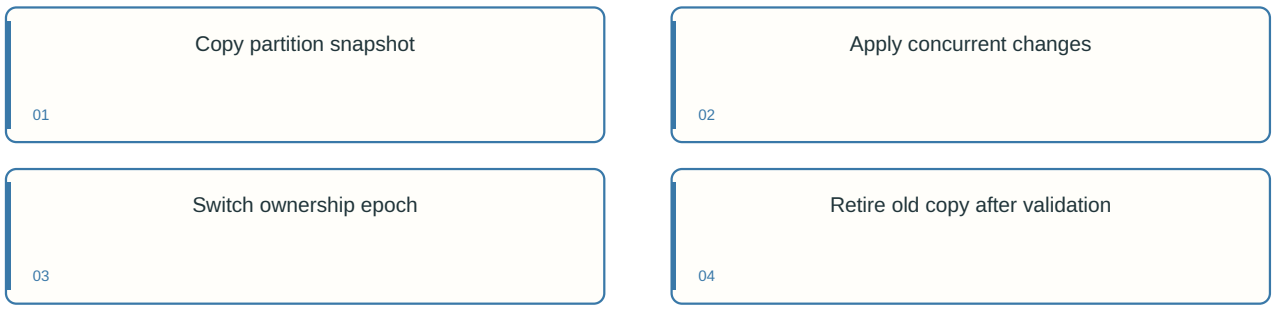


FIG. 12.2.5 **Rebalancing · worked snapshots**

Copy partition snapshot · Apply concurrent changes · Switch ownership epoch · Retire old copy after validation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.6 · Hot partitions and skew

Hot partitions arise from uneven key popularity, data size, or write concentration. A heatmap of requests and bytes per partition reveals skew hidden by fleet averages. Splitting a range may help if traffic divides; a single hot entity may need read copies, batching, or a redesigned invariant owner. Randomizing writes can distribute append pressure but makes reads aggregate more partitions. Match the fix to the actual pressure rather than repeatedly adding cold capacity.

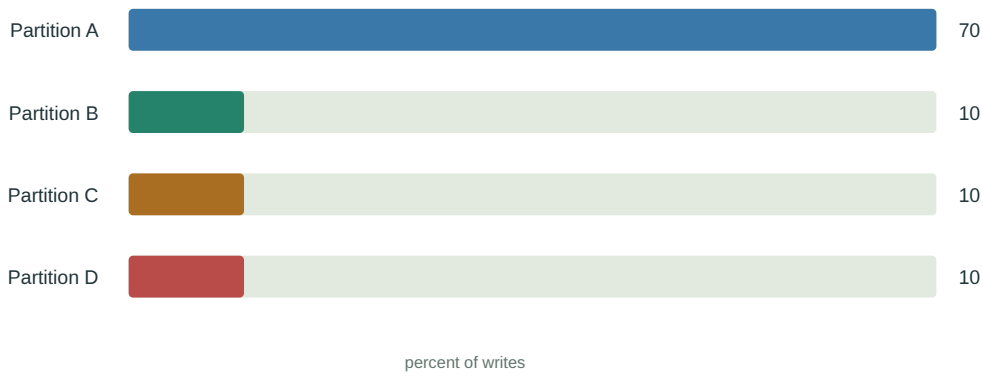


FIG. 12.2.6 **Averages hide a hot partition**

Seventy percent of writes go to A. Adding cold capacity does not split one hot ownership key.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can a single globally hot invariant be scaled merely by adding more partitions? Explain the constraint.

For many deterministic keys, a key either keeps its old owner or moves to the new owner in this model. That property does not guarantee balanced request traffic. Make one key extremely popular and show why adding cold owners cannot split that single-key pressure without a new serving strategy.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Randomizing writes can distribute append pressure but makes reads aggregate more partitions.

Carry forward: Fleet average appears healthy · One partition receives 70% of writes · Its queue determines tail latency · Split or redesign the specific hot path.

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon's highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

Lag, proxies & saturated pools

Prerequisites: 12.2 · Replicate and partition data 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Scale a URL shortener from 100 requests per second to 100,000. Introduce new components only when justified by a demonstrated bottleneck.

This chapter examines lag, proxies & saturated pools through concrete mechanisms, state snapshots, failure cases, and alternatives.

12.3.1 · Read replicas and read/write splitting

Read replicas can distribute suitable reads while the primary retains writes. A read immediately after an update can observe stale data if routed to a lagging replica. Route critical reads to a sufficiently current owner, use session position tracking where supported, or accept an explicit freshness contract. Read/write splitting must classify operations by required semantics, not only by SQL verb: a SELECT may participate in a consistency-critical transaction or require a lock.

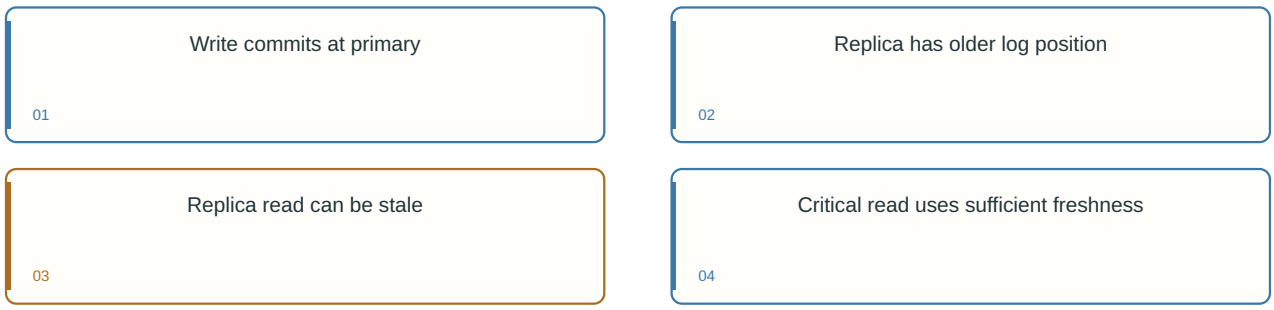


FIG. 12.3.1 **Read replicas and read/write splitting · worked snapshots**

Write commits at primary · Replica has older log position · Replica read can be stale · Critical read uses sufficient freshness. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.2 · Replication lag

Replication lag is the distance between source and replica state, measured by time, positions, or application version. A small average lag can hide a replica stalled for a particular workload. Expose source progress and replica apply progress, and distinguish transport lag from apply lag. Users may need read-your-writes even when global reads tolerate eventual freshness. Failover decisions must account for lag and acknowledged-write policy; the fastest promotion is not necessarily the safest promotion.

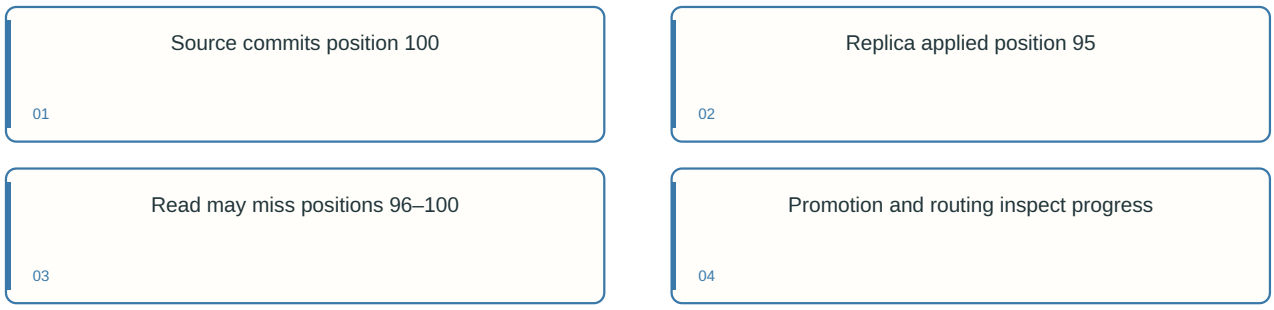


FIG. 12.3.2 **Replication lag · worked snapshots**

Source commits position 100 · Replica applied position 95 · Read may miss positions 96–100 · Promotion and routing inspect progress. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.3 · Database proxying

A database proxy can pool, route, or multiplex connections according to implementation. It does not remove database transaction limits and can change session-state assumptions. A client using session variables, temporary tables, or long transactions may not be safe under arbitrary multiplexing. Document where authentication, routing, and failover decisions happen. The proxy itself needs capacity and failure handling; placing it in a diagram does not eliminate connection saturation or guarantee transparent retries.

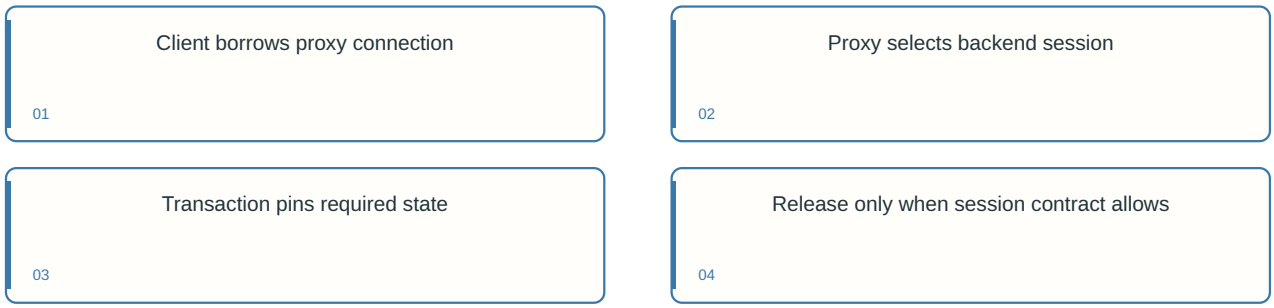


FIG. 12.3.3 **Database proxying · worked snapshots**

Client borrows proxy connection · Proxy selects backend session · Transaction pins required state · Release only when session contract allows. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.4 · Connection pool saturation

When a pool is full, waiting callers accumulate outside the database as well as inside it. More application replicas can multiply total database connections beyond the safe limit. Bound the aggregate across instances, reserve recovery capacity, and include acquisition time in deadlines. A query waiting on a lock may hold its connection while doing no CPU work. Fix hold time, contention, and admission before expanding every pool and increasing the pressure that caused the queue.

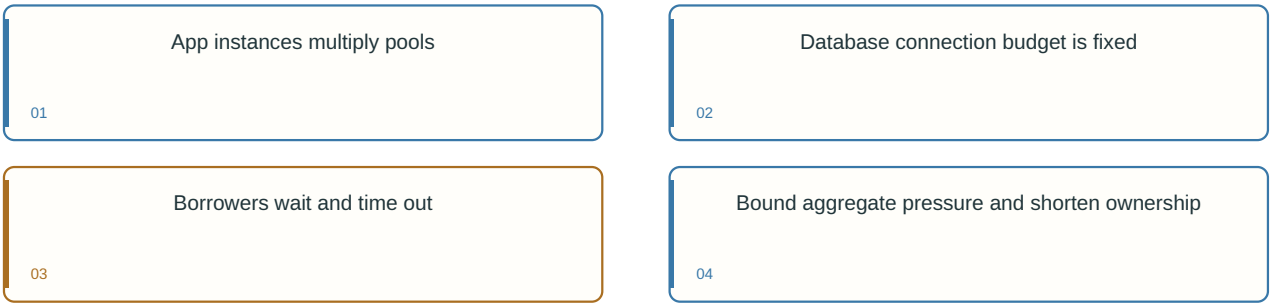


FIG. 12.3.4 **Connection pool saturation · worked snapshots**

App instances multiply pools · Database connection budget is fixed · Borrowers wait and time out · Bound aggregate pressure and shorten ownership. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.5 · Bottleneck identification

Bottleneck diagnosis follows the critical path and resource saturation. Compare offered load, good completions, queues, CPU, memory, disk, network, connection waits, and lock contention. If a single database query dominates, caching or indexing may help more than extra web instances; if CPU is saturated with independent work, compute scaling may help. Change one material variable and measure again. A component's high utilization is a clue, not proof that it limits the business outcome.

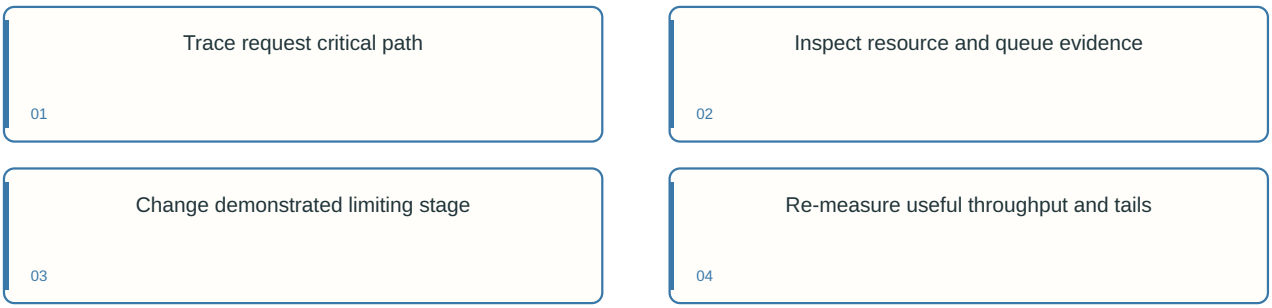


FIG. 12.3.5 **Bottleneck identification · worked snapshots**

Trace request critical path · Inspect resource and queue evidence · Change demonstrated limiting stage · Re-measure useful throughput and tails. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can a single globally hot invariant be scaled merely by adding more partitions? Explain the constraint.

For many deterministic keys, a key either keeps its old owner or moves to the new owner in this model. That property does not guarantee balanced request traffic. Make one key extremely popular and show why adding cold owners cannot split that single-key pressure without a new serving strategy.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Change one material variable and measure again.

Carry forward: Trace request critical path · Inspect resource and queue evidence · Change demonstrated limiting stage · Re-measure useful throughput and tails.

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

Regions, write paths & scale economics

Prerequisites: 12.3 · Lag, proxies & saturated pools 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Scale a URL shortener from 100 requests per second to 100,000. Introduce new components only when justified by a demonstrated bottleneck.

This chapter examines regions, write paths & scale economics through concrete mechanisms, state snapshots, failure cases, and alternatives.

12.4.1 · Queueing and overload

A queue absorbs temporary mismatch, not sustained overload. If arrivals exceed completion, the queue grows until latency, memory, or a limit fails. Bound queue size and age, reject or shed excess work, and prioritize essential operations. Waiting longer can make a request useless while still consuming resources. With a two-second deadline, a ten-second backlog should not be silently accepted as if work were on time. Return an explicit overload outcome and avoid automatic retry storms.

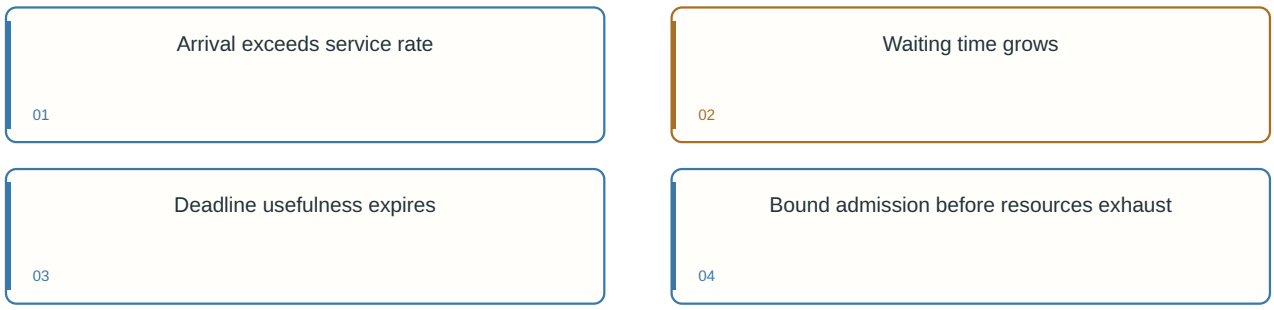


FIG. 12.4.1 **Queueing and overload · worked snapshots**

Arrival exceeds service rate · Waiting time grows · Deadline usefulness expires · Bound admission before resources exhaust. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.2 · Single points of failure

A single point of failure can be a machine, database, credentials service, router, operator action, or shared dependency. Count independent failure domains rather than only process replicas. Two application containers on one host do not survive that host's loss. Redundancy also needs failover, capacity, detection, and state correctness. Inspect control-plane dependencies during recovery: the data path may be redundant while a missing configuration service prevents every replacement instance from starting.

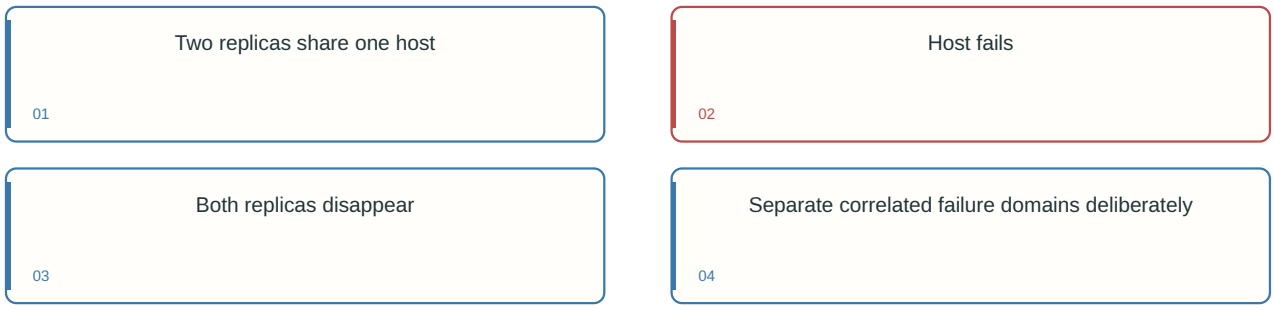


FIG. 12.4.2 **Single points of failure · worked snapshots**

Two replicas share one host · Host fails · Both replicas disappear · Separate correlated failure domains deliberately. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.3 · Multi-region strategies

Multi-region designs trade latency, data placement, conflict, cost, and failure behavior. Active-passive can simplify write ownership but requires tested promotion and recovery targets. Active-active may improve locality or continuity but needs a precise conflict or coordination model. Cross-region acknowledgment adds propagation delay; asynchronous copies can lose recent acknowledged writes on failover. Start with business requirements for locality, tolerated interruption, and data loss, then choose write authority and read contracts per operation.

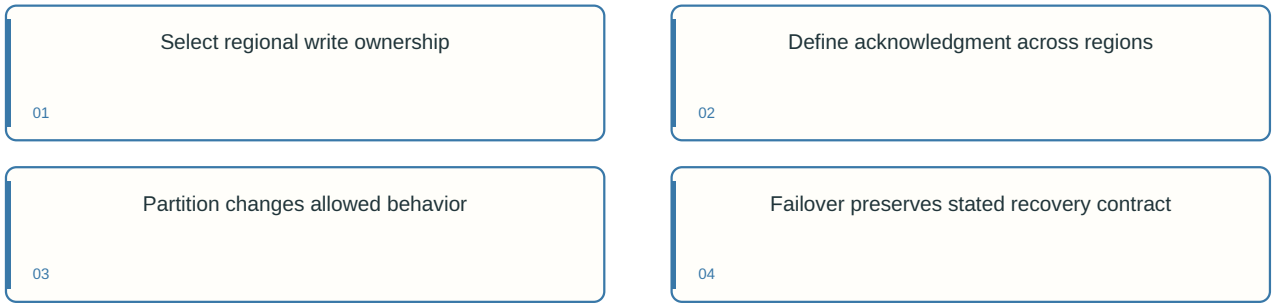


FIG. 12.4.3 **Multi-region strategies · worked snapshots**

Select regional write ownership · Define acknowledgment across regions · Partition changes allowed behavior · Failover preserves stated recovery contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.4 · Scaling writes vs scaling reads

Read scaling often replicates or caches immutable or sufficiently stale-tolerant data. Write scaling must distribute, batch, or reduce ownership work while preserving invariants. A globally unique sequential allocator or one hot account can remain serialized across many shards. Reduce unnecessary indexes or coordination before distributing more components. Aggregate counters may allow commutative updates, while seat allocation needs exclusive ownership. Different business operations can legitimately use different scaling and consistency strategies.

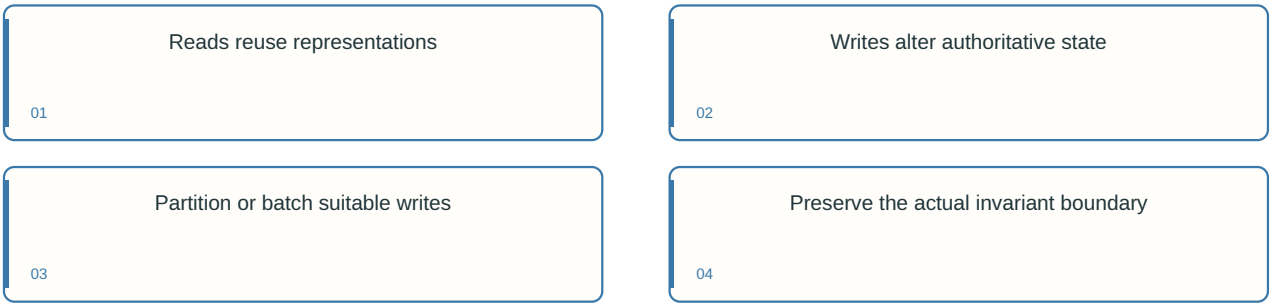


FIG. 12.4.4 **Scaling writes vs scaling reads · worked snapshots**

Reads reuse representations · Writes alter authoritative state · Partition or batch suitable writes · Preserve the actual invariant boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.5 · Scale-out economics

Scale-out economics includes duplicate baseline capacity, coordination, networking, telemetry, cache warmup, and operational work. Ten small nodes are not automatically cheaper or faster than one larger node. Compare equal service objectives and failure headroom under the same workload; include the capacity required during a node or zone loss. Managed systems can trade provider charges against staffing and recovery work. Use cost per successful business outcome and investigate bottlenecks before purchasing capacity that cannot help.

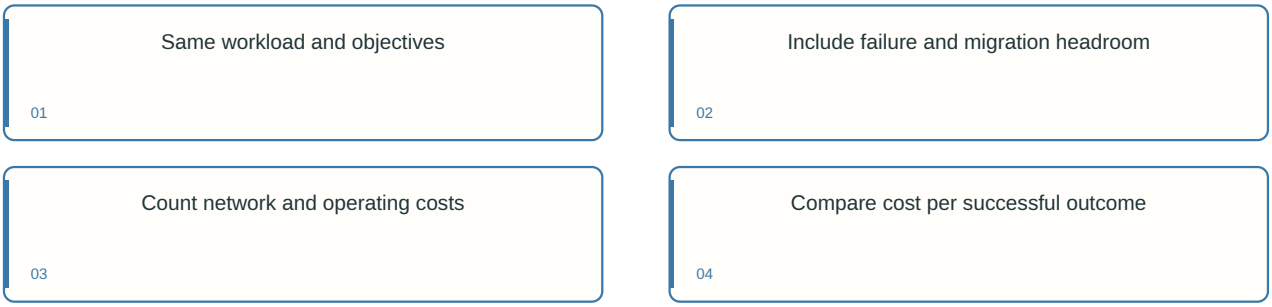


FIG. 12.4.5 **Scale-out economics · worked snapshots**

Same workload and objectives · Include failure and migration headroom · Count network and operating costs · Compare cost per successful outcome. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · STABLE RENDEZVOUS MAPPING

```
from mechanisms import rendezvous
for key in range(1000):
    old = rendezvous(key, ["A","B","C"])
    new = rendezvous(key, ["A","B","C","D"])
    assert old == new or new == "D"
```

Adding an owner moves only keys now won by that owner in this model. It does not transfer data, synchronize membership, or distribute one hot key. Those protocols and workload effects remain explicit design work.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

Compare owners before and after adding D

Key 190: old owner **B**; new owner **B**.

Across keys 0–999, 246 change owners. Counts after adding D: A=228, B=252, C=274, D=246.

Changed keys must move to D in this model. Popularity and data movement are separate concerns.

Deterministic illustrative rendezvous scores. Stable assignment is not data transfer, membership agreement, traffic balance, or protection against one hot key.

▼ Architecture review · Can a single globally hot invariant be scaled merely by adding more partitions? Explain the constraint.

For many deterministic keys, a key either keeps its old owner or moves to the new owner in this model. That property does not guarantee balanced request traffic. Make one key extremely popular and show why adding cold owners cannot split that single-key pressure without a new serving strategy.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Managed systems can trade provider charges against staffing and recovery work.

Carry forward: Same workload and objectives · Include failure and migration headroom · Count network and operating costs · Compare cost per successful outcome.

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

PART 3 / UNIT 13

File Systems, Search Infrastructure, and Data Pipelines

The running example

Design a document-processing system accepting millions of files, tracking processing state, extracting metadata, maintaining provenance, and exposing searchable results.

Unit 13 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

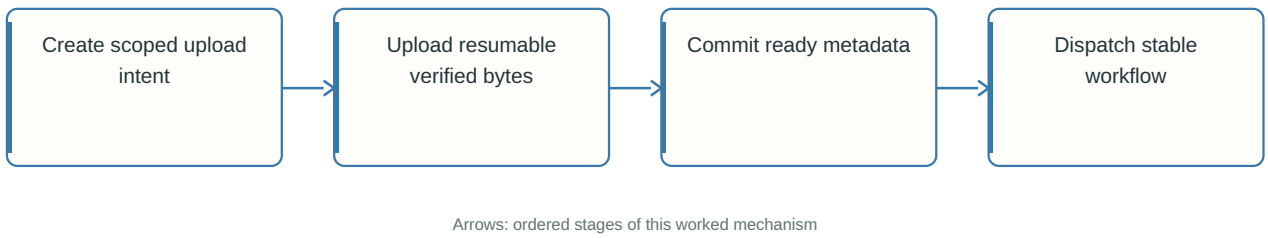


FIG. 13.0.1 **Upload pipeline**

Input bytes and control metadata have different owners and failure windows.

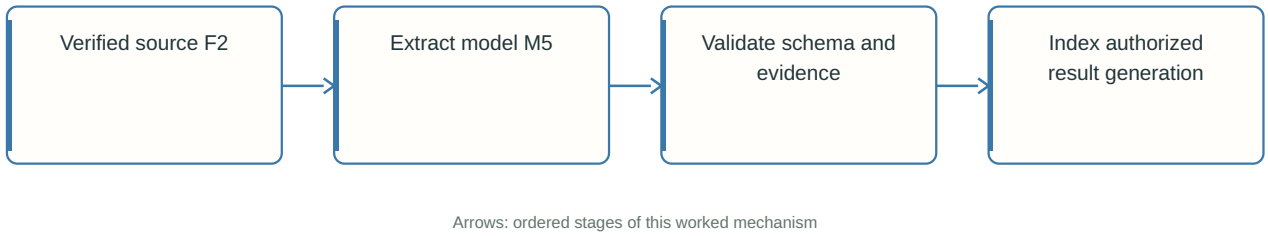
Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Source / term	Representation	Meaning
D1	blue car	document text
D2	blue bike	document text
blue	D1, D2	posting list
car	D1	posting list

FIG. 13.0.2 **An inverted index is a term-to-document map**

Token analysis creates the lookup representation. Source versions and permissions remain separately owned.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 13.0.3 **ingestion DAG**

A stage’s versioned artifact and durable transition make replay inspectable.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Property	Batch	Streaming
Input	Bounded set	Continuing events
Freshness	Scheduled recomputation	Progress and lateness policy
Recovery	Replay bounded batch	Restore state and offsets

FIG. 13.0.4 **batch vs stream timeline**

Neither mode is automatically correct; output semantics and replay identity must be explicit.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

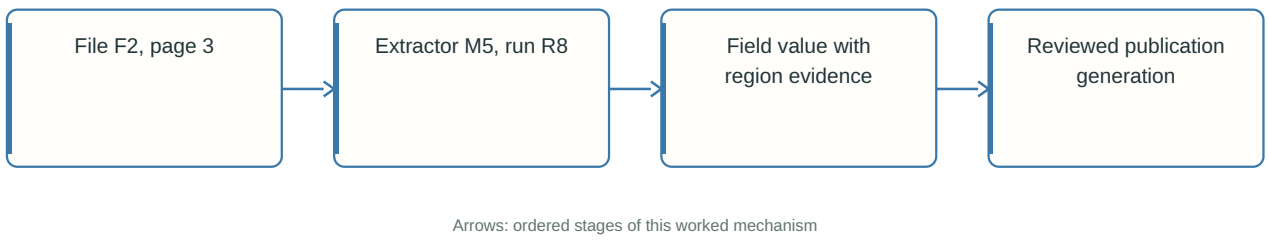


FIG. 13.0.5 **lineage graph**

Lineage connects exact source and transform identities through derived artifacts.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Upload bytes without losing identity

Prerequisites: API Design and Communication Contracts, Data Models and Storage Technologies, Database Internals, Transactions, and Concurrency Control

5 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a document-processing system accepting millions of files, tracking processing state, extracting metadata, maintaining provenance, and exposing searchable results.

This chapter examines upload bytes without losing identity through concrete mechanisms, state snapshots, failure cases, and alternatives.

13.1.1 · Binary/object storage

Store large binary content separately from mutable processing metadata when their access and lifecycle needs differ. An object has a stable identity, content version or checksum, size, and ownership metadata; a job record references it. A database row pointing at a missing or incomplete object is a broken workflow even if both services individually succeed. Use explicit upload states and cleanup for abandoned bytes. Treat object-store-specific consistency and durability claims as implementation contracts to verify, not category-wide assumptions.

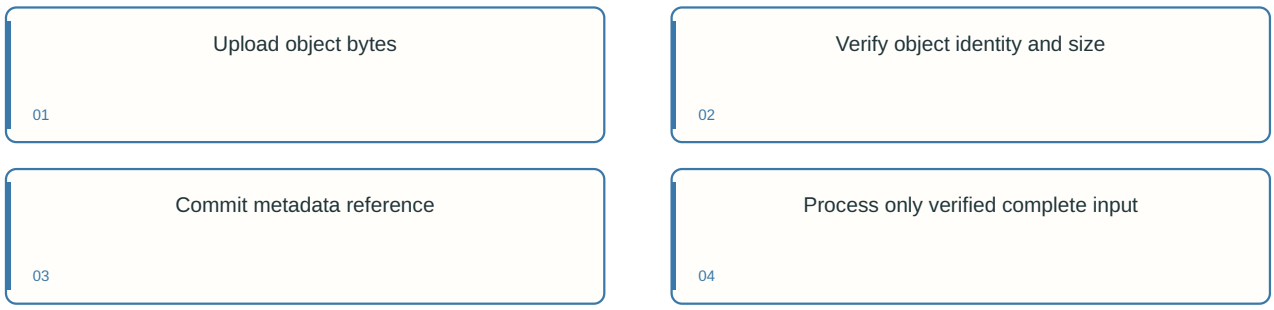


FIG. 13.1.1 **Binary/object storage · worked snapshots**

Upload object bytes · Verify object identity and size · Commit metadata reference · Process only verified complete input. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.2 · Large-file upload architecture

A large-file architecture can issue a scoped upload authorization so the client streams directly to storage, while the application owns metadata and authorization. This avoids proxying all bytes through web workers, but the client may never complete the upload. Validate ownership, quotas, content size, and completion evidence before making a file available for processing. Expire abandoned intents and reconcile unreferenced objects. A signed upload URL is a time-bounded capability that must not grant arbitrary bucket access.

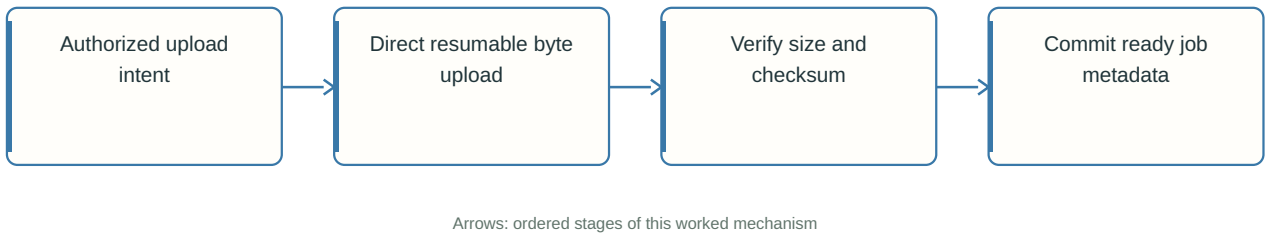


FIG. 13.1.2 **The upload pipeline separates bytes from control**

Incomplete uploads never enter processing as verified input. Metadata failure leaves cleanup/reconciliation work rather than a fictitious cross-service transaction.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.3 · Multipart and resumable uploads

Multipart upload splits a file into identifiable parts with explicit completion. Resumability tracks confirmed offsets or parts so a client can retry without restarting all bytes. Duplicate part submission and changed source content need defined behavior. Verify the completed object rather than assuming every acknowledged part implies a correct final file. Multipart identifiers and checksums can be provider-specific; a convenient entity tag is not universally a content hash. Clean up incomplete sessions to prevent silent storage growth.

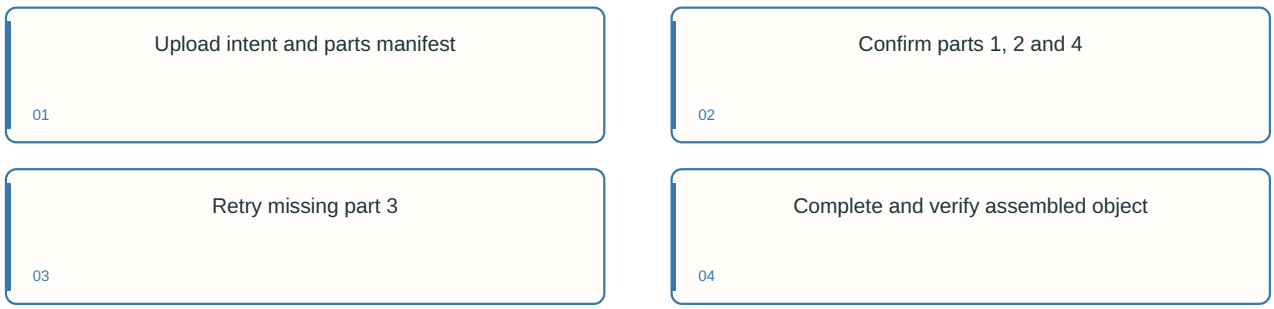


FIG. 13.1.3 **Multipart and resumable uploads · worked snapshots**

Upload intent and parts manifest · Confirm parts 1, 2 and 4 · Retry missing part 3 · Complete and verify assembled object. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.4 · Checksums and content addressing

A checksum detects accidental corruption under its algorithm's properties; a cryptographic digest can identify content with collision assumptions and appropriate verification. Content addressing uses a digest-derived identity, enabling deduplication or immutable references. It does not authorize access or prove the content is safe. Scope deduplication carefully across tenants to avoid information leakage, and verify downloaded bytes when integrity matters. A hash of a client-provided claim is not a checksum of the bytes actually stored.

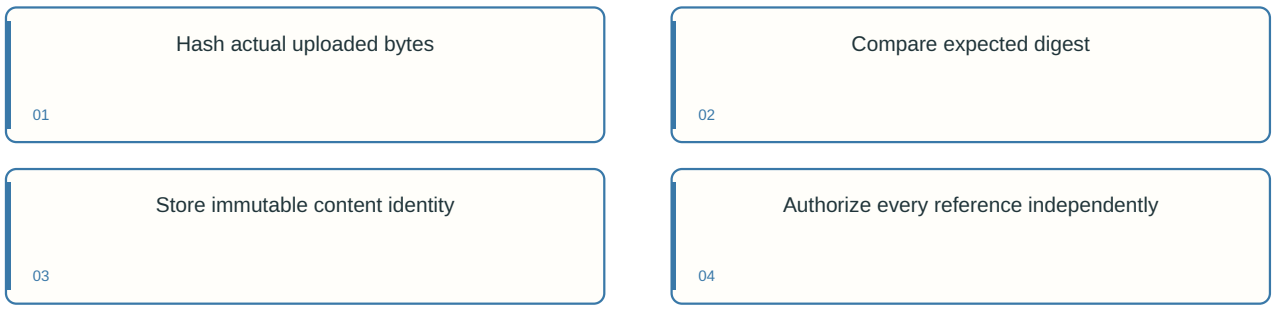


FIG. 13.1.4 **Checksums and content addressing · worked snapshots**

Hash actual uploaded bytes · Compare expected digest · Store immutable content identity · Authorize every reference independently. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.5 · Metadata vs binary data

Binary bytes and metadata have different update patterns. Keep file identity and immutable byte versions distinct from classification status, user annotations, and processing attempts. An extractor should record exactly which content and model version produced an output. Mutating a file in place under the same identifier can make old results impossible to explain. If the input changes, create a new version and schedule appropriate reprocessing while retaining or deleting historical artifacts according to policy.

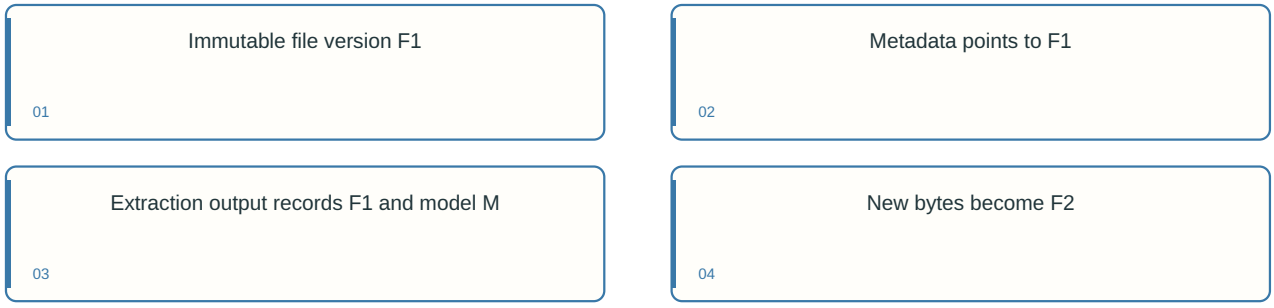


FIG. 13.1.5 **Metadata vs binary data · worked snapshots**

Immutable file version F1 · Metadata points to F1 · Extraction output records F1 and model M · New bytes become F2. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would you reproduce a disputed extracted field after both the source and model have new versions?

Use Workflow to reject stale epochs and wrong input versions. Retain exact source and transform versions. Build a new index generation, account for concurrent changes and deletions, validate, and switch a serving pointer. Matching record counts alone is not sufficient evidence of correct versions or authorization.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Mutating a file in place under the same identifier can make old results impossible to explain.

Carry forward: Immutable file version F1 · Metadata points to F1 · Extraction output records F1 and model M · New bytes become F2.

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Build and rebuild a search index

Prerequisites: 13.1 · Upload bytes without losing identity 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a document-processing system accepting millions of files, tracking processing state, extracting metadata, maintaining provenance, and exposing searchable results.

This chapter examines build and rebuild a search index through concrete mechanisms, state snapshots, failure cases, and alternatives.

13.2.1 · Search engines and inverted indexes

An inverted index maps a term to postings describing documents or positions containing it. For documents D1='blue car' and D2='blue bike', blue maps to D1,D2 while car maps only to D1. Analysis choices such as tokenization and normalization affect matching. Search updates need document identity and version to prevent an old extraction from replacing a newer one. Keep source records rebuildable; the index is a derived serving structure with its own freshness and permission obligations.

Source / term	Representation	Meaning
D1	blue car	document text
D2	blue bike	document text
blue	D1, D2	posting list
car	D1	posting list

FIG. 13.2.1 **An inverted index is a term-to-document map**

Token analysis creates the lookup representation. Source versions and permissions remain separately owned.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.2.2 · Indexing pipelines

An indexing pipeline transforms authoritative content into searchable representations. Separate extraction, validation, enrichment, indexing, and publication states so a partial failure is diagnosable. Retry a stage using stable input and output identities; avoid creating duplicate documents on every retry. Poison files need bounded attempts and a review path. Check that index permissions derive from current ownership rather than stale public flags embedded in an old output. Completion should refer to a precise index version and source version.

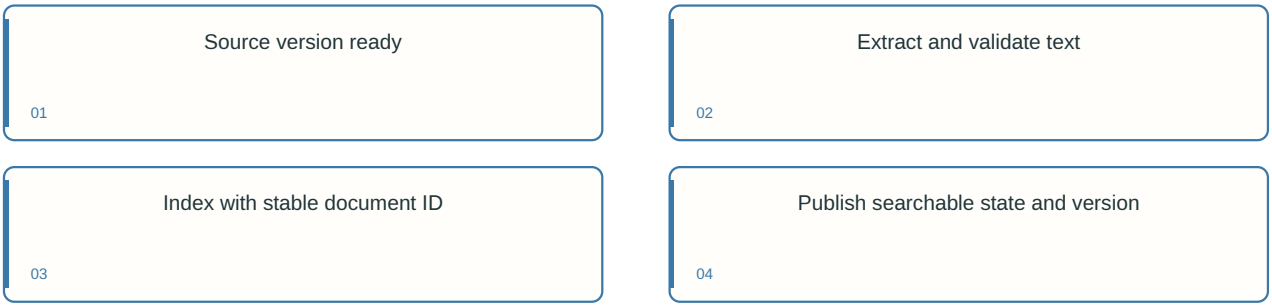


FIG. 13.2.2 **Indexing pipelines · worked snapshots**

Source version ready · Extract and validate text · Index with stable document ID · Publish searchable state and version. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.2.3 · Search ingestion and reindexing

Reindexing builds a new representation while preserving serving behavior. Take a snapshot or defined source position, process its records, catch up subsequent changes, validate, and switch a serving alias or routing epoch. Do not overwrite the only working index before the new one is verified. Deletions and updates during the build need replay or reconciliation. Rollback must preserve schema compatibility and any new writes; counting indexed documents alone cannot establish that the right versions or permissions were indexed.

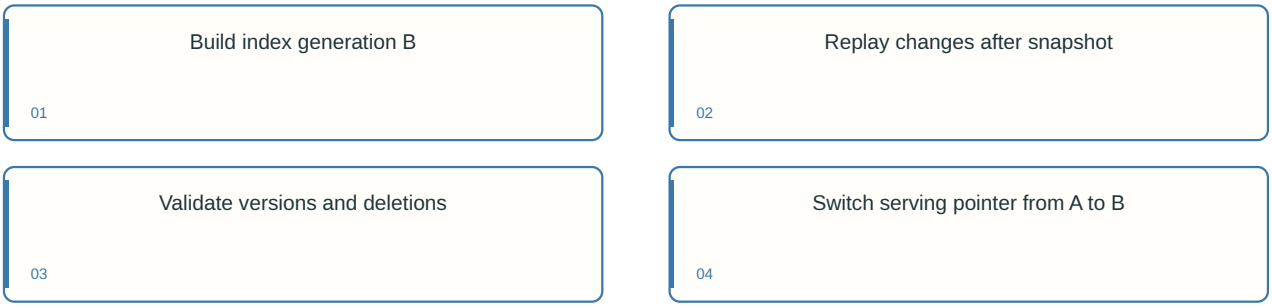


FIG. 13.2.3 **Search ingestion and reindexing · worked snapshots**

Build index generation B · Replay changes after snapshot · Validate versions and deletions · Switch serving pointer from A to B. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

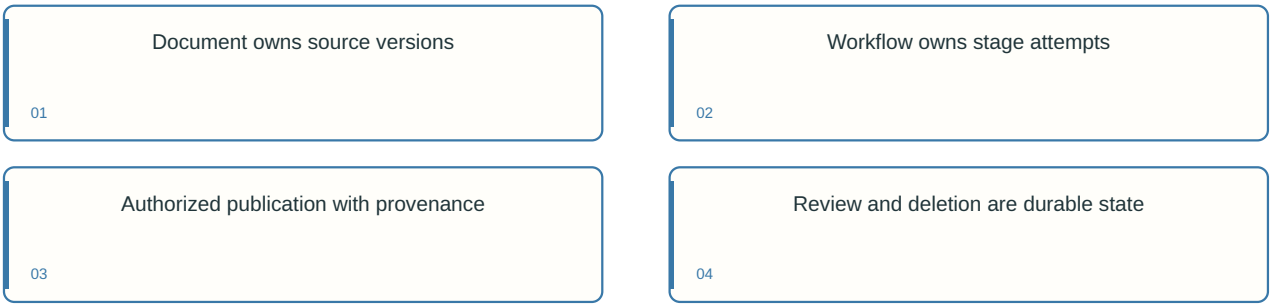


FIG. 13.2.4 **The running system · invariant and recovery review**

Review whether any stage silently changes source versions, whether an old attempt can publish after deletion, and whether human decisions are preserved during reprocessing. Test duplicate tasks, interrupted extraction, invalid output, reviewer concurrency, stale completion, missing object, index lag, and full restore. A pipeline that extracts a few fields once has not demonstrated the required recoverable, explainable platform.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would you reproduce a disputed extracted field after both the source and model have new versions?

Use Workflow to reject stale epochs and wrong input versions. Retain exact source and transform versions. Build a new index generation, account for concurrent changes and deletions, validate, and

switch a serving pointer. Matching record counts alone is not sufficient evidence of correct versions or authorization.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Deletions and updates during the build need replay or reconciliation.

Carry forward: Build index generation B · Replay changes after snapshot · Validate versions and deletions · Switch serving pointer from A to B.

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Move data through batch and streams

Prerequisites: 13.2 · Build and rebuild a search index 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a document-processing system accepting millions of files, tracking processing state, extracting metadata, maintaining provenance, and exposing searchable results.

This chapter examines move data through batch and streams through concrete mechanisms, state snapshots, failure cases, and alternatives.

13.3.1 · Batch vs streaming pipelines

Batch processing operates on bounded sets; stream processing operates on continuing input with explicit progress and state. A nightly export may be simpler when hours of freshness delay are acceptable; fraud signals may require lower-latency updates. Streaming introduces late data, state recovery, partitioning, and checkpoint concerns. Batch can still be expensive and failure-prone. Choose based on freshness and recomputation needs, and specify whether results are provisional or final when late events can change them.

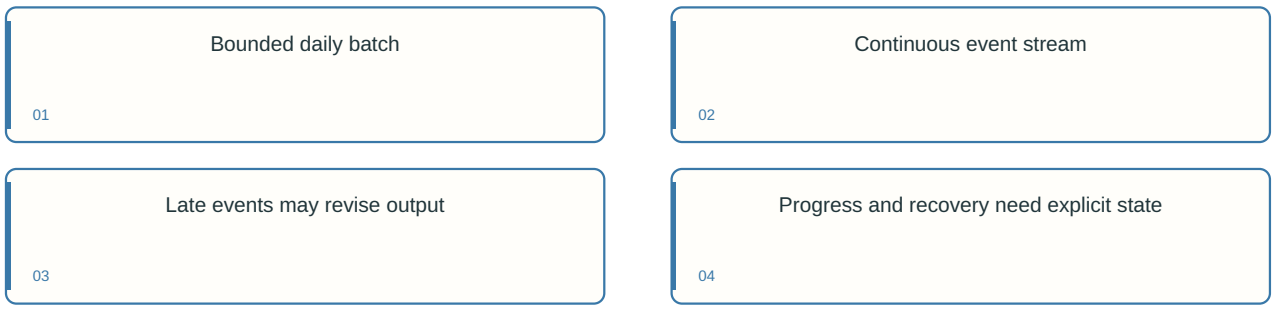


FIG. 13.3.1 **Batch vs streaming pipelines · worked snapshots**

Bounded daily batch · Continuous event stream · Late events may revise output · Progress and recovery need explicit state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.2 · ETL vs ELT

ETL transforms before loading into the target; ELT loads and transforms in the target environment. ETL can minimize sensitive fields before they reach an analytical store; ELT can preserve flexible raw inputs but increases governance and storage obligations. Either can lose lineage or silently change semantics if transformations are undocumented. Version transformations, validate schemas, and define ownership of raw and derived datasets. The acronym does not determine quality, privacy, or repeatability.

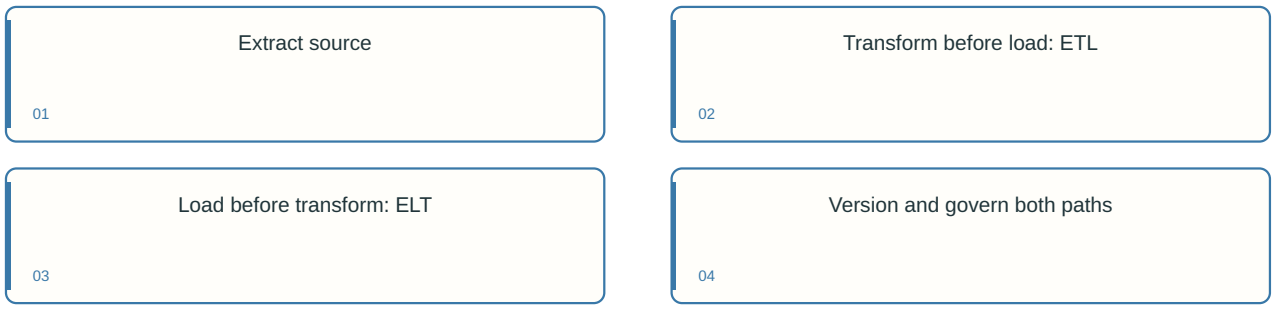


FIG. 13.3.2 **ETL vs ELT · worked snapshots**

Extract source · Transform before load: ETL · Load before transform: ELT · Version and govern both paths. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.3 · Change Data Capture (CDC)

Change Data Capture emits committed changes from a source through a supported mechanism such as a transaction log. It can keep search or analytics current without polling entire tables. A snapshot plus a change stream needs a consistent handoff position so writes are neither missed nor applied incorrectly. Events can repeat, and schema changes affect decoding. Preserve source identity and ordering metadata, deduplicate, and monitor lag. CDC does not automatically make the destination transactionally consistent with the source for every query.

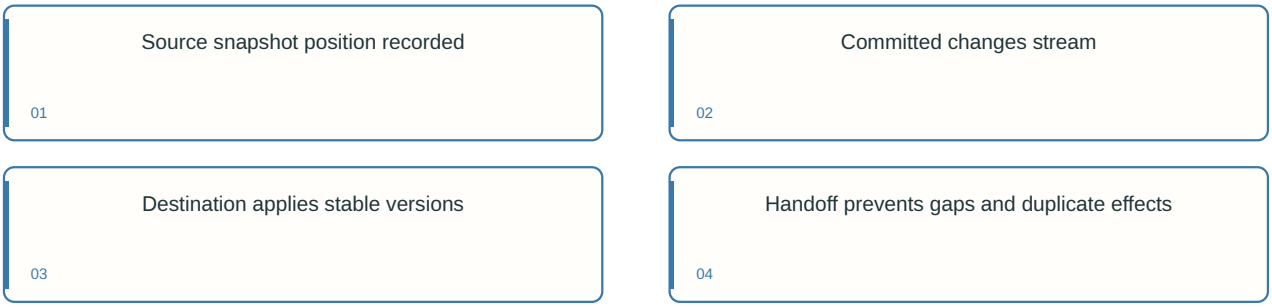


FIG. 13.3.3 **Change Data Capture (CDC) · worked snapshots**

Source snapshot position recorded · Committed changes stream · Destination applies stable versions · Handoff prevents gaps and duplicate effects. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.4 · Data lakes and warehouses

A data lake stores flexible datasets, often in object storage; a warehouse provides curated analytical structures and serving capabilities. Boundaries vary by product. Raw retention can help replay but also accumulates privacy, quality, and discovery problems. A lake without schema, ownership, and lineage becomes difficult to trust. Model immutable inputs, curated outputs, catalog metadata, access policies, and lifecycle separately. Choose formats and partitioning for the queries and recovery operations that actually need to run.

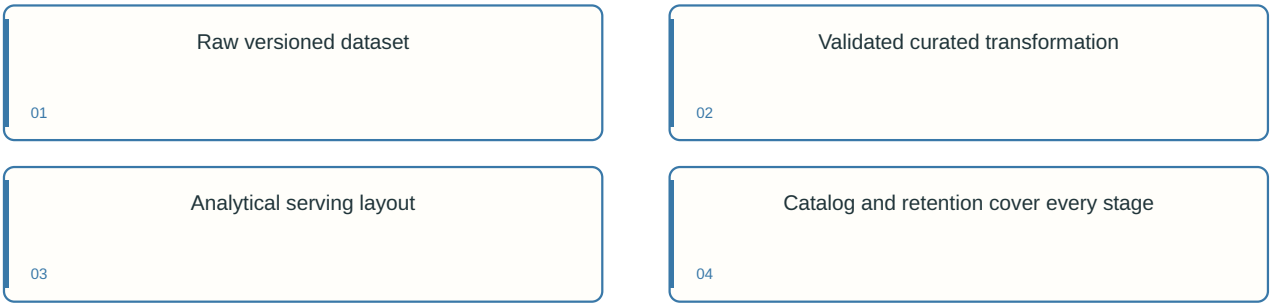


FIG. 13.3.4 **Data lakes and warehouses · worked snapshots**

Raw versioned dataset · Validated curated transformation · Analytical serving layout · Catalog and retention cover every stage. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.5 · Stream processing fundamentals

Stream processing maintains results over ongoing events using windows, keys, and state. Event time describes when a source event occurred; processing time describes when it was handled. Watermarks estimate progress under assumptions about late arrivals; they are not proof that no earlier event can ever arrive. Define allowed lateness, correction behavior, checkpoint ownership, and effect boundaries. A windowed count can be updated after a late event, so consumers need to understand whether it is provisional or final.

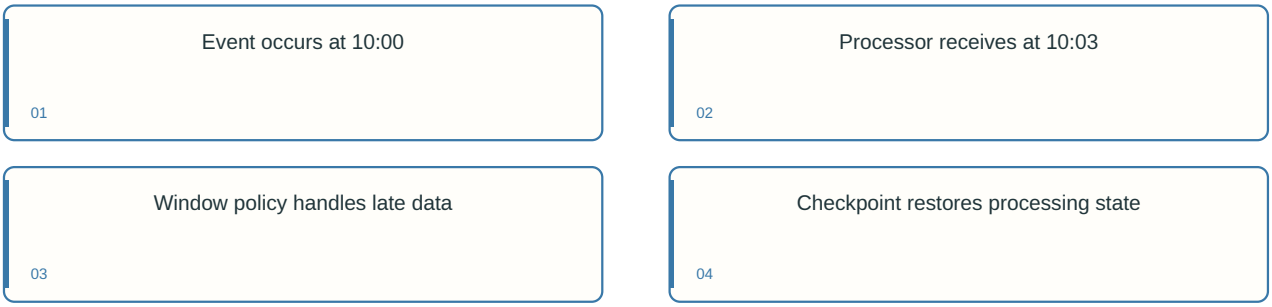


FIG. 13.3.5 **Stream processing fundamentals · worked snapshots**

Event occurs at 10:00 · Processor receives at 10:03 · Window policy handles late data · Checkpoint restores processing state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would you reproduce a disputed extracted field after both the source and model have new versions?

Use Workflow to reject stale epochs and wrong input versions. Retain exact source and transform versions. Build a new index generation, account for concurrent changes and deletions, validate, and switch a serving pointer. Matching record counts alone is not sufficient evidence of correct versions or authorization.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Define allowed lateness, correction behavior, checkpoint ownership, and effect boundaries.

Carry forward: Event occurs at 10:00 · Processor receives at 10:03 · Window policy handles late data · Checkpoint restores processing state.

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Quality, provenance & replay

Prerequisites: 13.3 · Move data through batch and streams 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a document-processing system accepting millions of files, tracking processing state, extracting metadata, maintaining provenance, and exposing searchable results.

This chapter examines quality, provenance & replay through concrete mechanisms, state snapshots, failure cases, and alternatives.

13.4.1 · Data provenance and lineage

Provenance records where an artifact came from; lineage connects transformations between artifacts. An extracted amount should link to file version, page or region, extractor version, parameters, and review decisions. This supports debugging and correction without claiming the extraction is always right. Store stable references, not only a text explanation that cannot be checked. Privacy and retention apply to provenance too; an audit trail containing raw personal data can outlive the source if lifecycle policies are incomplete.

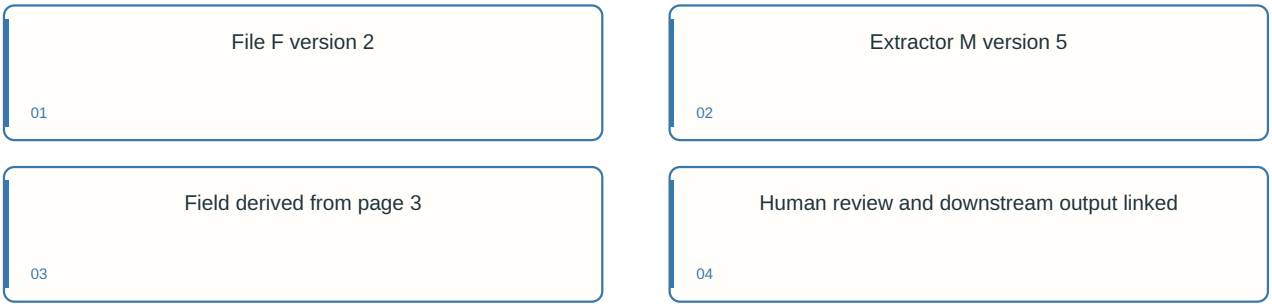


FIG. 13.4.1 **Data provenance and lineage · worked snapshots**

File F version 2 · Extractor M version 5 · Field derived from page 3 · Human review and downstream output linked. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.2 · Data quality, schema validation, and replay

Data quality checks validate shape, ranges, identity, relationships, and completeness against explicit contracts. Schema-valid records can still be semantically wrong: a currency field may be present but incompatible with the amount units. Quarantine invalid inputs with reasons and preserve enough evidence for repair. Replay must be versioned and deduplicated so fixing a transformation does not double side effects. Compare counts, versions, and business invariants at source and sink rather than trusting an empty error queue.

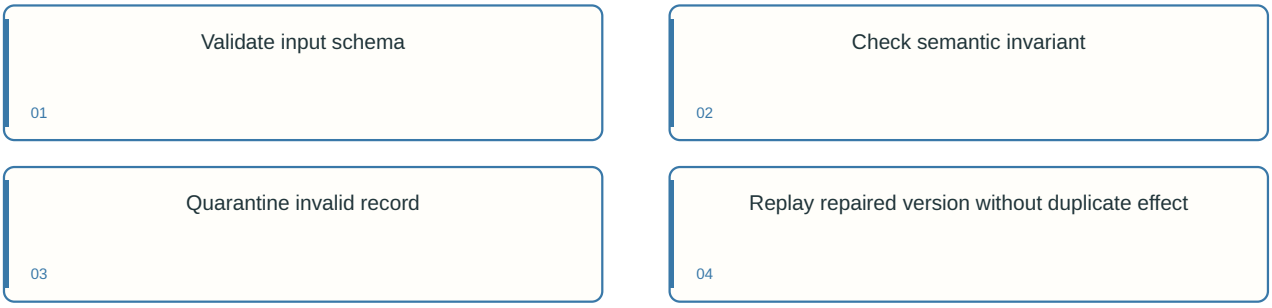


FIG. 13.4.2 **Data quality, schema validation, and replay · worked snapshots**

Validate input schema · Check semantic invariant · Quarantine invalid record · Replay repaired version without duplicate effect. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.3 · Lifecycle policies and retention

Lifecycle policies should follow every copy: raw files, extracted text, thumbnails, indexes, logs, and backups. A retention timer on the original PDF alone leaves derived personal data behind. Use deletion workflows with durable progress and reconciliation; maintain narrowly scoped tombstones where required to prevent replay resurrection. Large object collections need efficient listing or ownership indexes for deletion. Test recovery from old snapshots against deletion history so a restore does not silently reverse required privacy actions.

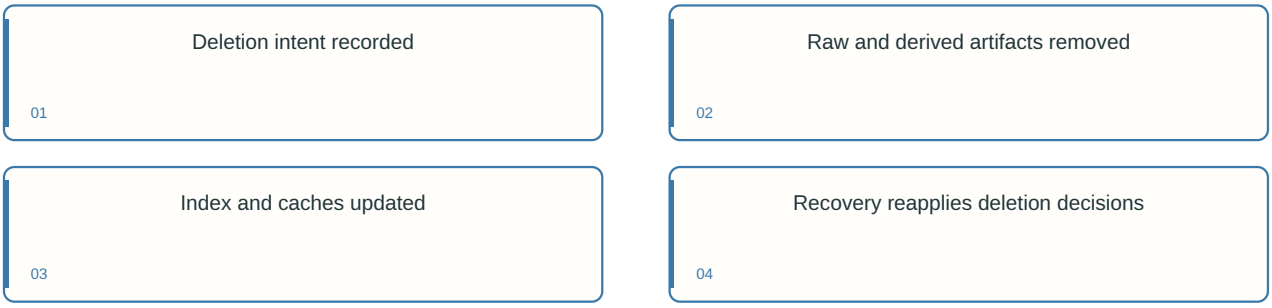


FIG. 13.4.3 **Lifecycle policies and retention · worked snapshots**

Deletion intent recorded · Raw and derived artifacts removed · Index and caches updated · Recovery reapplies deletion decisions. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.4 · Designing data-intensive applications

A data-intensive design is a network of state transformations, not a collection of product logos. For every edge, define source authority, version, ordering, retry identity, validation, ownership, and recovery. Keep immutable source versions so derived structures can be rebuilt, but do not retain everything indefinitely without purpose. End-to-end reliability comes from coordinating those contracts and observing progress across the pipeline. A scalable queue cannot compensate for an extractor that loses provenance or an index that serves unauthorized copies.

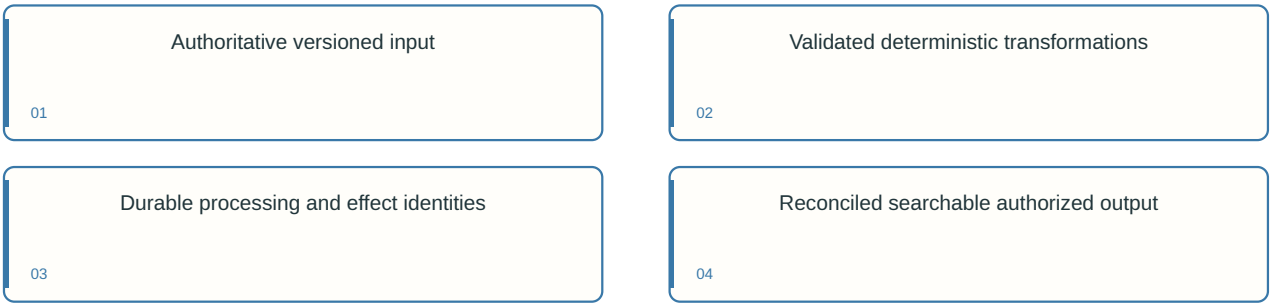


FIG. 13.4.4 **Designing data-intensive applications · worked snapshots**

Authoritative versioned input · Validated deterministic transformations · Durable processing and effect identities · Reconciled searchable authorized output. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · SOURCE AND ATTEMPT FENCING

```
from mechanisms import Workflow
run = Workflow("file-version-2", epoch=8)
assert not run.complete(7, "file-version-2", "old")
assert not run.complete(8, "file-version-1", "wrong-input")
assert run.complete(8, "file-version-2", "current")
```

The miniature checks local identity and epoch. Persisting it safely requires an atomic storage transition and output validation. Provenance must also record transformation versions and relevant source regions, not just an artifact name.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

Reject stale or deleted workflow completion

Reject completion

Epoch 7 has lost authority to epoch 8.

One local conditional-transition model. Persisting and applying the condition atomically is required in a real store. External model calls and object writes are outside this local transition.

▼ Architecture review · How would you reproduce a disputed extracted field after both the source and model have new versions?

Use Workflow to reject stale epochs and wrong input versions. Retain exact source and transform versions. Build a new index generation, account for concurrent changes and deletions, validate, and switch a serving pointer. Matching record counts alone is not sufficient evidence of correct versions or authorization.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

End-to-end reliability comes from coordinating those contracts and observing progress across the pipeline.

Carry forward: Authoritative versioned input · Validated deterministic transformations · Durable processing and effect identities · Reconciled searchable authorized output.

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

PART 4 / UNIT 14

Distributed Systems Fundamentals

The running example

A social application writes a new post in one region while a friend reads it from another. Examine different consistency contracts and their user-visible effects.

Unit 14 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

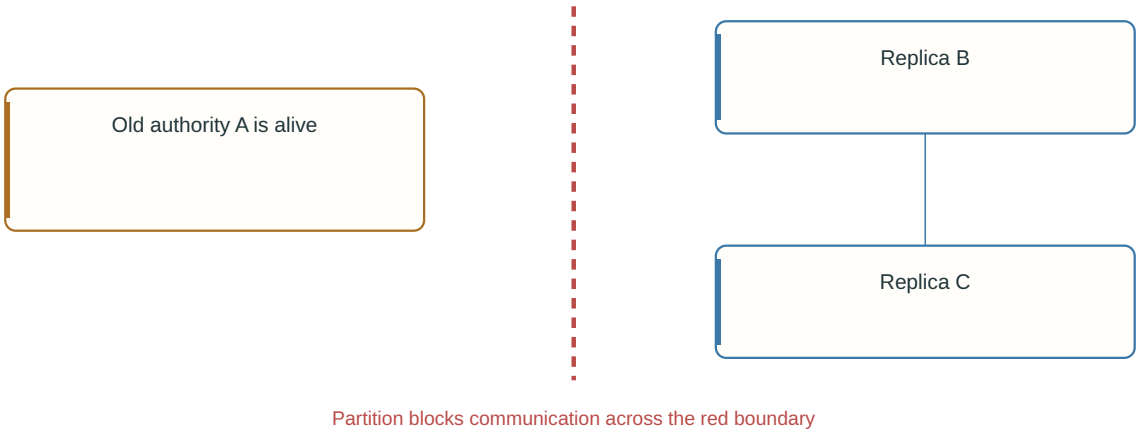
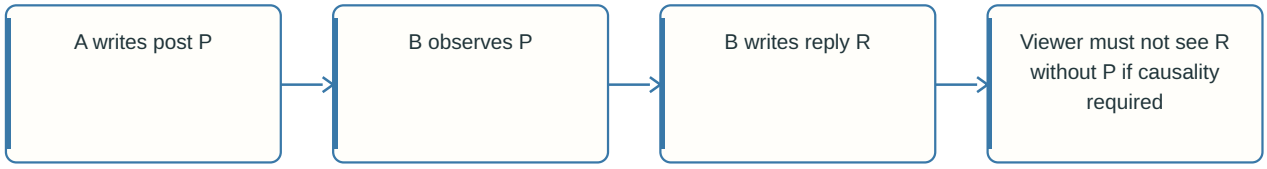


FIG. 14.0.1 **A partition can leave two live but disconnected sides**

Region A remains running while B and C form another reachable side. Correct authority and fencing determine who may make which writes; a missing heartbeat cannot stop A.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 14.0.2 **causality timeline**

Causality follows program and message dependencies, not arbitrary wall-clock ordering.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Model	Required constraint	What is not implied
Linearizable	Real-time atomic operations	External-effect transaction
Serializable	Equivalent serial transactions	Real-time order
Causal	Preserve dependencies	One total global order
Eventual	Converge under assumptions	Bounded freshness by itself

FIG. 14.0.3 **consistency model comparisons**

Name the model rather than treating strong and eventual as complete application descriptions.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Set	Members	Intersection
Write W=2	A, B	B
Read R=2	B, C	B
Fixed N=3	R + W = 4 > 3	one shared member

FIG. 14.0.4 **Intersecting quorums still need a protocol**

Intersection preserves a possible witness. Version, membership, persistence, and concurrency rules determine how that witness is used.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

Partial failure & the meaning of time

Prerequisites: Networking Foundations for System Design, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control

7 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social application writes a new post in one region while a friend reads it from another. Examine different consistency contracts and their user-visible effects.

This chapter examines partial failure & the meaning of time through concrete mechanisms, state snapshots, failure cases, and alternatives.

14.1.1 · What makes a distributed system difficult?

Distribution removes the ability to observe one coherent machine state. A request may succeed on one node while another is unreachable, and a caller may not know whether a timed-out write committed. Designing a distributed system means specifying allowed histories under those uncertainties. Begin with one authoritative owner and explicit operations; add replication or coordination to meet a concrete need. Additional nodes create failure combinations as well as capacity, so correctness must be a protocol rather than a diagram of reachable boxes.

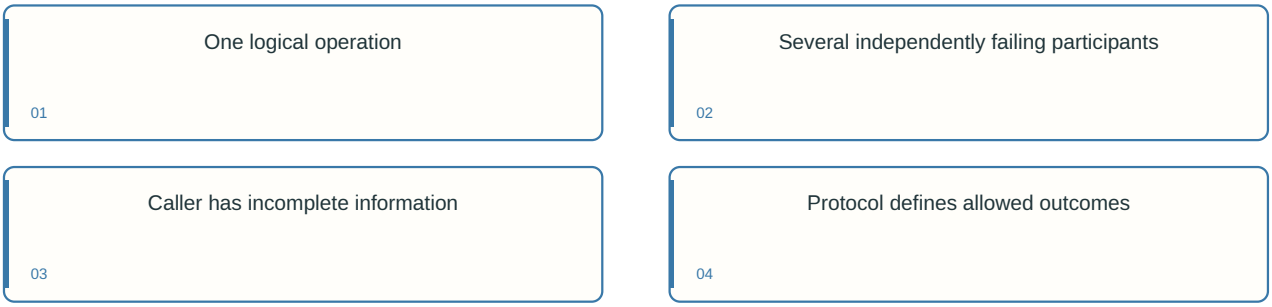


FIG. 14.1.1 **What makes a distributed system difficult · worked snapshots**

One logical operation · Several independently failing participants · Caller has incomplete information · Protocol defines allowed outcomes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.2 · Partial failures

A partial failure affects some participants or paths while others keep operating. The payment provider can charge successfully while the order service loses the reply. Rolling back the local order cannot reverse that external event. Track an uncertain state with a stable provider reference and reconcile it. Design compensations only for effects that can actually be compensated, and distinguish an operation not attempted from one attempted with unknown outcome. Treating every exception as a clean failure creates duplicate business effects.

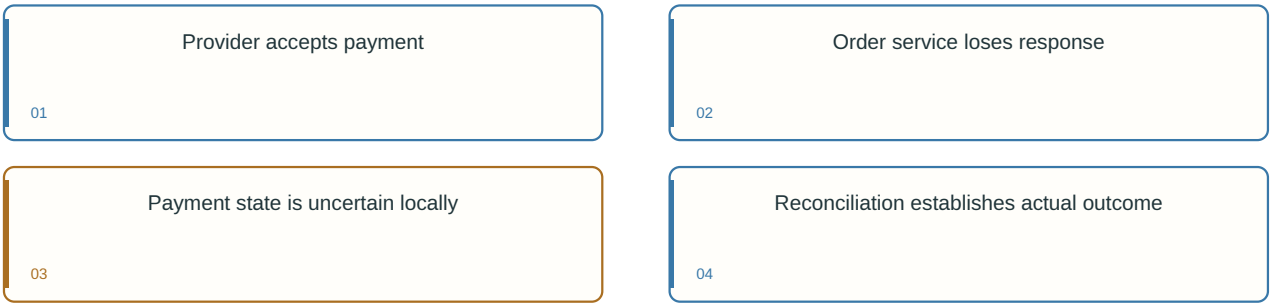


FIG. 14.1.2 **Partial failures · worked snapshots**

Provider accepts payment · Order service loses response · Payment state is uncertain locally · Reconciliation establishes actual outcome. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.3 · Network uncertainty

A slow network and a failed node can look the same to a participant for a while. Timeouts are practical failure suspicions, not perfect failure detectors. A retry can reach another node after the original is merely delayed. Use deadlines and bounded retries to protect resources, but let the protocol prevent conflicting owners or duplicate effects. Availability decisions must state when the system prefers rejecting work to risking an invariant, rather than assuming every missing heartbeat proves the old owner stopped.

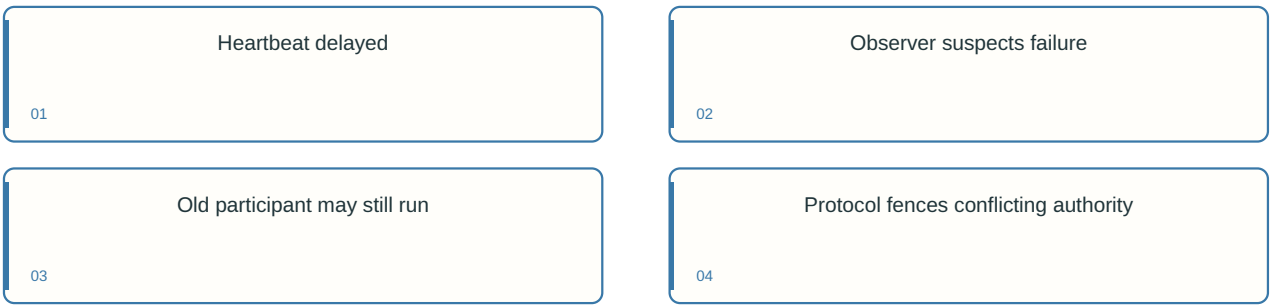


FIG. 14.1.3 **Network uncertainty · worked snapshots**

Heartbeat delayed · Observer suspects failure · Old participant may still run · Protocol fences conflicting authority. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.4 · Fault models

A fault model states what kinds of failure a protocol tolerates: crash-stop, crash-recovery, message loss, partitions, or arbitrary Byzantine behavior. A majority algorithm designed for non-malicious crash failures does not automatically tolerate participants lying about their logs. Durability also depends on whether persistent state survives restart and on failure independence. Name these assumptions before counting nodes. Three replicas on one failing device do not provide the same protection as independent durable replicas.

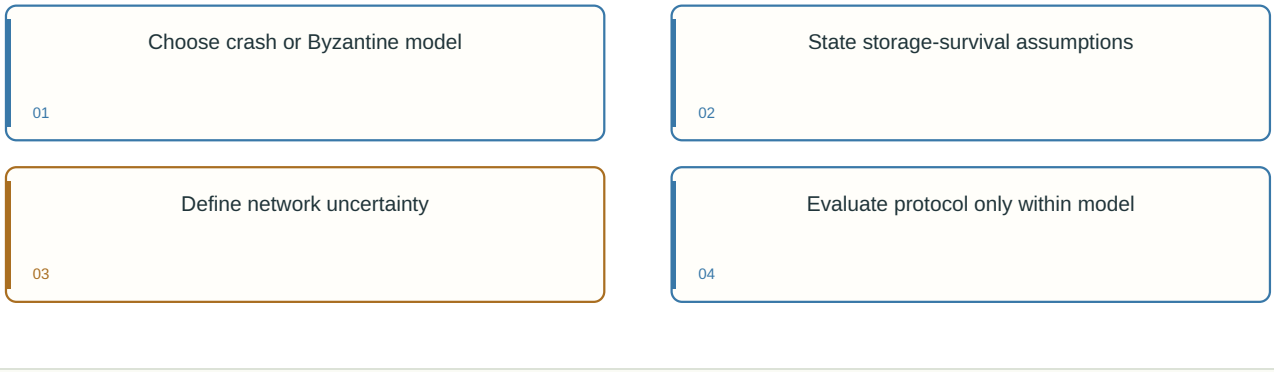


FIG. 14.1.4 **Fault models · worked snapshots**

Choose crash or Byzantine model · State storage-survival assumptions · Define network uncertainty · Evaluate protocol only within model. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.5 · Time, clocks, and event ordering

Wall clocks support timestamps and scheduling but can drift, jump, or be adjusted. A timestamp earlier than another does not necessarily establish causality across machines. Duration measurement should use an appropriate monotonic clock where available. If a reservation expires based on time, define the authoritative clock and tolerated uncertainty. Ordering requirements often need a logical sequence or consensus log rather than trusting client-generated timestamps. Record both source and observed time when their distinction matters for debugging.

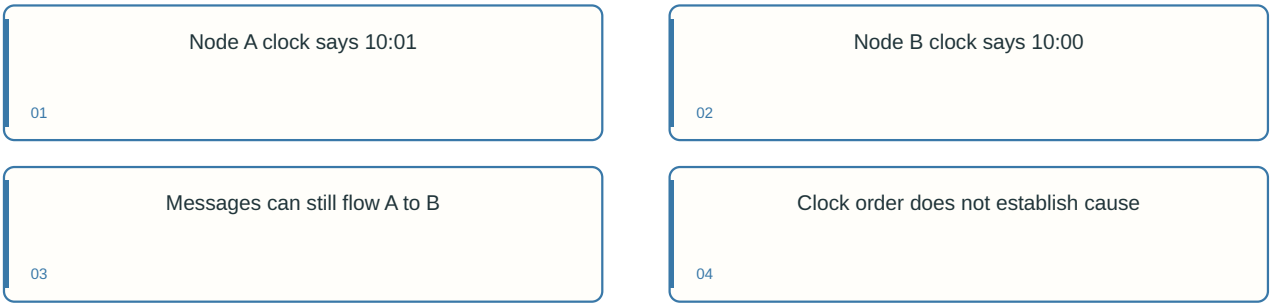


FIG. 14.1.5 **Time, clocks, and event ordering · worked snapshots**

Node A clock says 10:01 · Node B clock says 10:00 · Messages can still flow A to B · Clock order does not establish cause. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.6 · Physical vs logical clocks

Physical clocks estimate real time; logical clocks represent ordering constraints. A logical counter can advance without representing seconds, while a physical timestamp can compare unrelated events without proving causal order. Use physical time for user-facing dates and appropriate deadline policies, and logical order for protocol relationships where needed. Combining them requires a precise model. Do not subtract two arbitrary hosts' timestamps and call the result network latency without accounting for clock synchronization error.

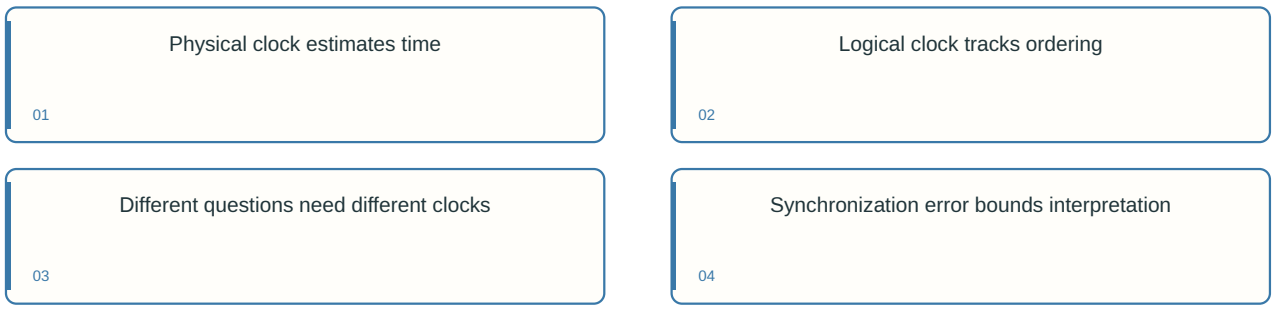


FIG. 14.1.6 **Physical vs logical clocks · worked snapshots**

Physical clock estimates time · Logical clock tracks ordering · Different questions need different clocks · Synchronization error bounds interpretation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.7 · Lamport clocks and vector clocks

Lamport clocks advance locally and on receive using max(local,received)+1. If event A happens before B through program or message order, its Lamport value is smaller; the converse need not hold. Vector clocks track per-participant causal progress and can identify concurrent versions under their assumptions, at metadata cost. An example with vectors [1,0] and [0,1] is concurrent. Neither clock by itself resolves a conflict or establishes a global real-time commit order.

Event	Vector	Relation
A only	[1,0]	not before B
B only	[0,1]	not before A
After observing both	[1,1]	dominates both

FIG. 14.1.7 **Vector comparison exposes concurrent events**

Componentwise comparison identifies causality under the chosen participant model; it does not choose a business merge.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · During a partition, which operations could degrade while exclusive allocation must wait or reject?

lamport_receive(5,9) returns 10; vectors [1,0] and [0,1] are concurrent. For three fixed members, R=2,W=2 intersects. Explain why these calculations do not independently resolve concurrent business updates or prove linearizability. The missing protocol rules are part of the answer.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

An example with vectors [1,0] and [0,1] is concurrent.

Carry forward: A event has vector [1,0] · B independent event has [0,1] · Neither vector dominates · Merge policy must handle concurrency.

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Consistency as a reader-visible contract

Prerequisites: 14.1 · Partial failure & the meaning of time 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social application writes a new post in one region while a friend reads it from another. Examine different consistency contracts and their user-visible effects.

This chapter examines consistency as a reader-visible contract through concrete mechanisms, state snapshots, failure cases, and alternatives.

14.2.1 · Consistency models

A consistency model defines legal observation histories. Ask what a user may read after a write, after switching replicas, and after seeing another user's action. A post body and a feed count can have different needs. Stronger constraints can require coordination or reduce allowed serving behavior during partitions. “Strong” is ambiguous without naming the model, and “eventual” alone omits conflict and freshness details. Write concrete histories and identify which observations are acceptable before selecting a database setting.

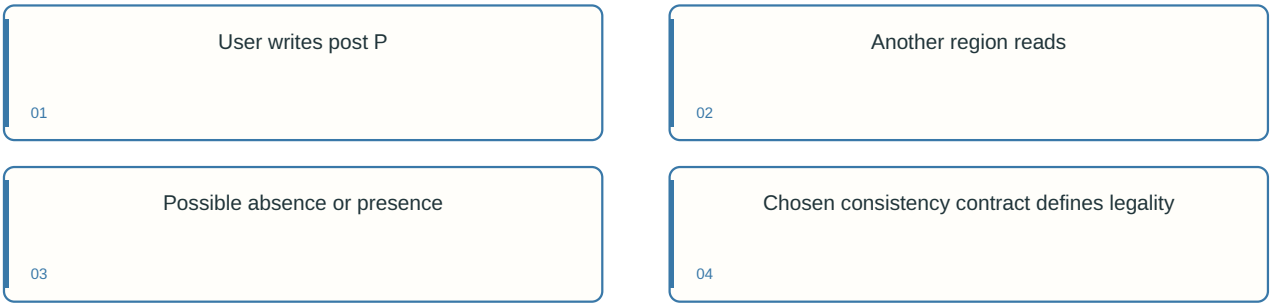


FIG. 14.2.1 **Consistency models · worked snapshots**

User writes post P · Another region reads · Possible absence or presence · Chosen consistency contract defines legality. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.2 · Strong consistency and eventual consistency

Strong consistency is often used informally for linearizable behavior, but must be defined in context. Eventual convergence says replicas should agree after updates cease and communication and repair assumptions hold; it does not promise a fixed staleness bound or excuse lost acknowledged writes. A display feed may accept temporary delay while the author's profile reads its own post promptly. Specify per-operation behavior and conflict resolution rather than labeling the whole application with one vague adjective.

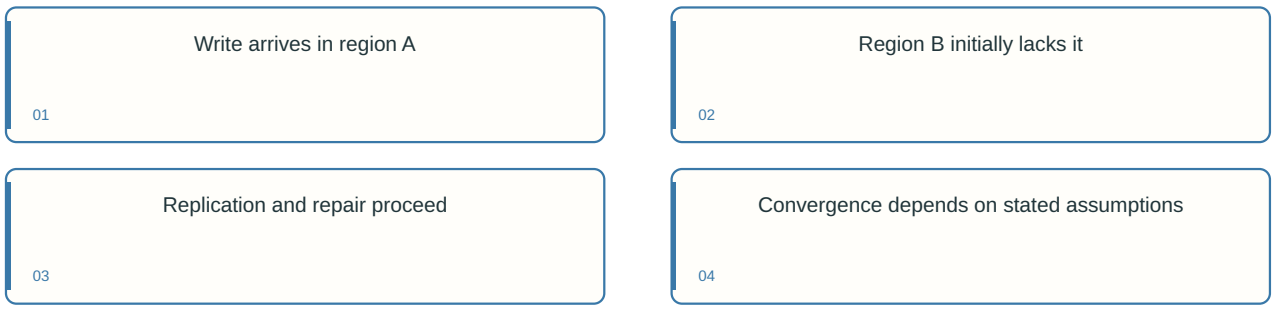


FIG. 14.2.2 **Strong consistency and eventual consistency · worked snapshots**

Write arrives in region A · Region B initially lacks it · Replication and repair proceed · Convergence depends on stated assumptions. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.3 · Linearizability

Linearizability makes completed operations appear to take effect atomically at points between invocation and response, respecting real-time precedence. If a write of 2 completes before a read starts, the read cannot return the overwritten 1 in a linearizable register. Concurrent operations can admit more than one valid ordering. Implementing the property requires a suitable protocol, not only majority counts or wall-clock timestamps. A timed-out operation may still have taken effect because it lacks a completed response.

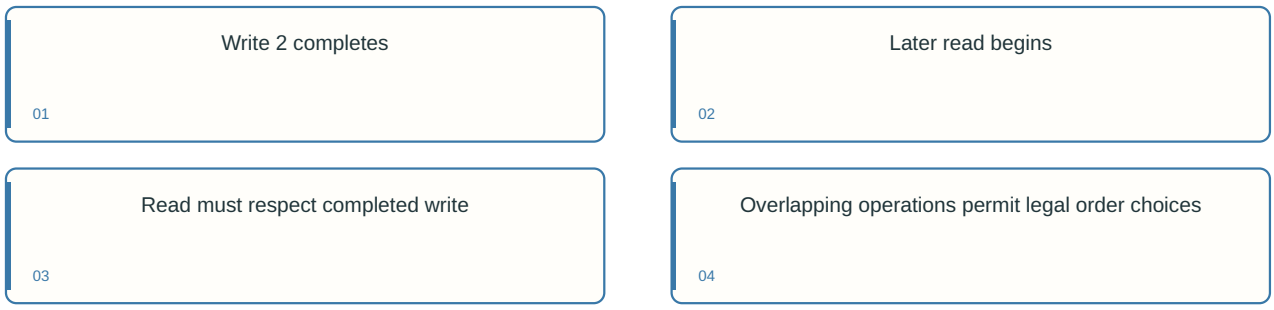


FIG. 14.2.3 **Linearizability · worked snapshots**

Write 2 completes · Later read begins · Read must respect completed write · Overlapping operations permit legal order choices. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.4 · Serializability vs linearizability

Serializability concerns equivalence of transactions to a serial execution; linearizability concerns real-time atomic behavior of operations. A serial order need not follow observed wall-clock order unless a stronger contract requires it. Strict serializability combines serial transaction behavior with real-time precedence. These distinctions matter when a system mixes transactions, replica reads, and external messages. A serializable database transaction does not automatically make a payment provider call part of one real-time atomic operation.

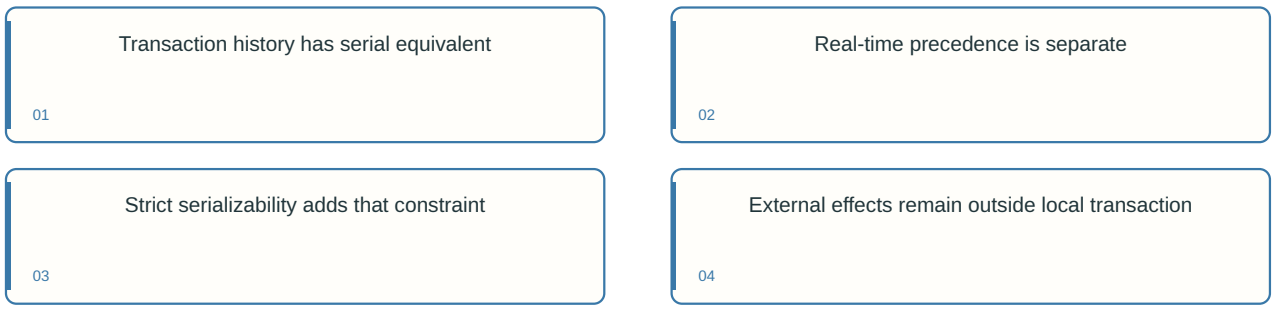


FIG. 14.2.4 **Serializability vs linearizability · worked snapshots**

Transaction history has serial equivalent · Real-time precedence is separate · Strict serializability adds that constraint · External effects remain outside local transaction. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.5 · Sequential and causal consistency

Sequential consistency preserves each participant's program order in some shared ordering without necessarily honoring real-time order across participants. Causal consistency preserves cause-and-effect dependencies while allowing unrelated concurrent events to be observed in different orders. If a reply depends on seeing a post, exposing the reply before its parent violates that causal relationship. Metadata and dependency tracking can enforce it, but an ordinary replication delay setting alone does not establish a causal contract.

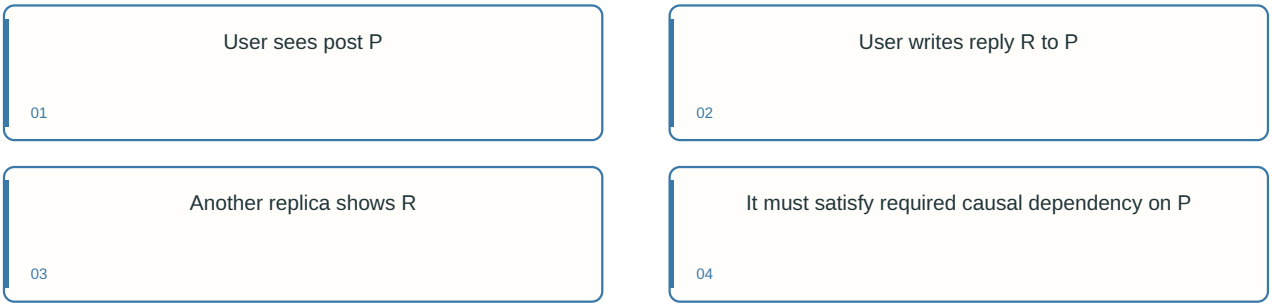


FIG. 14.2.5 **Sequential and causal consistency · worked snapshots**

User sees post P · User writes reply R to P · Another replica shows R · It must satisfy required causal dependency on P. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.6 · Read-your-writes and monotonic reads

Read-your-writes keeps a session from losing visibility of its own completed updates; monotonic reads prevent that session from moving backward in observed progress. They can be implemented through routing affinity, replication-position tracking, or suitable protocols. They are session guarantees, not necessarily global linearizability. A user switching devices needs a policy for carrying session progress. Sticky routing alone can fail after an instance change, so define how the guarantee survives failover or when a read must wait.

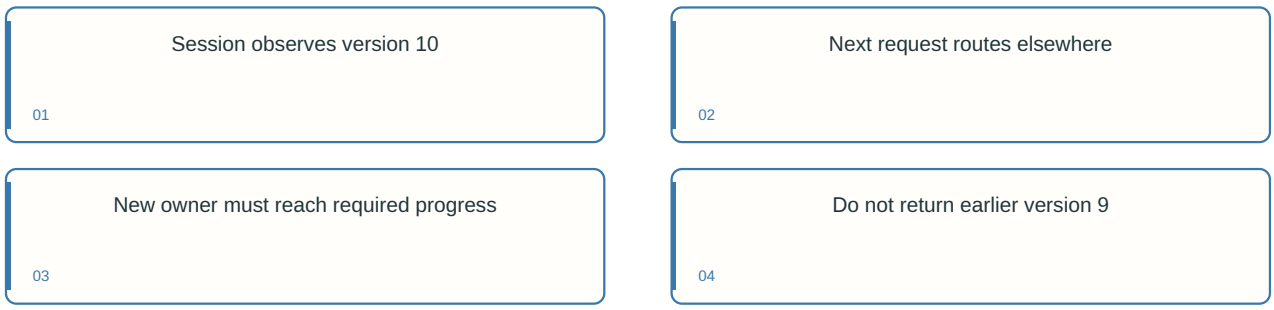


FIG. 14.2.6 **Read-your-writes and monotonic reads · worked snapshots**

Session observes version 10 · Next request routes elsewhere · New owner must reach required progress · Do not return earlier version 9. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · During a partition, which operations could degrade while exclusive allocation must wait or reject?

lamport_receive(5,9) returns 10; vectors [1,0] and [0,1] are concurrent. For three fixed members, R=2,W=2 intersects. Explain why these calculations do not independently resolve concurrent business updates or prove linearizability. The missing protocol rules are part of the answer.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A user switching devices needs a policy for carrying session progress.

Carry forward: Session observes version 10 · Next request routes elsewhere · New owner must reach required progress · Do not return earlier version 9.

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   `Access reviewed · 21 October 2026`

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   `Access reviewed · 21 October 2026`

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   `Access reviewed · 21 October 2026`

Partitions, CAP, PACELC & quorums

Prerequisites: 14.2 · Consistency as a reader-visible contract 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social application writes a new post in one region while a friend reads it from another. Examine different consistency contracts and their user-visible effects.

This chapter examines partitions, cap, pacelc & quorums through concrete mechanisms, state snapshots, failure cases, and alternatives.

14.3.1 · CAP theorem, with its actual assumptions

CAP's impossibility applies to a defined distributed read/write model under partitions: its formal atomic consistency and availability cannot both be guaranteed for every allowed execution. It is not a menu where a system permanently picks any two attractive qualities. When replicas cannot communicate, an application may reject some operations to protect invariants or serve bounded weaker behavior for others. State the affected operation, partition assumption, and required property. Normal nonpartitioned latency trade-offs require additional reasoning.

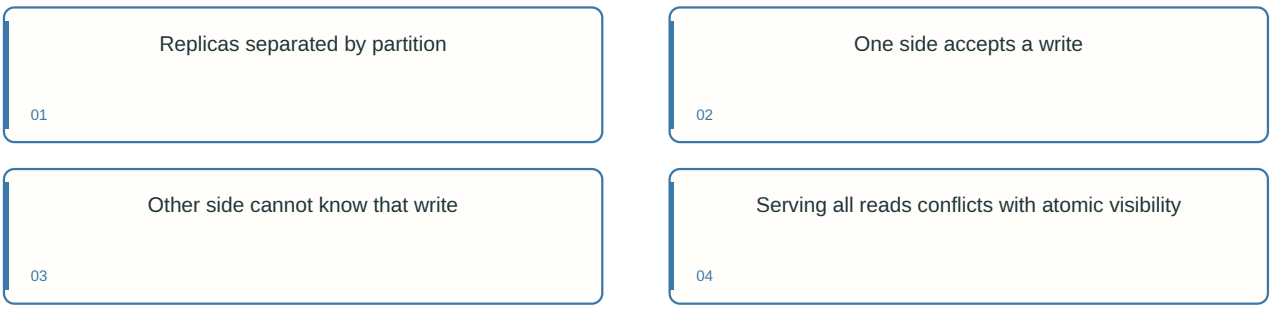


FIG. 14.3.1 **CAP theorem, with its actual assumptions · worked snapshots**

Replicas separated by partition · One side accepts a write · Other side cannot know that write · Serving all reads conflicts with atomic visibility. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.2 · PACELC as a useful extension

PACELC highlights two questions: during a partition, availability versus consistency; otherwise, latency versus consistency under the considered design. It is a useful classification lens, not a universal theorem replacing a protocol specification. Waiting for a geographically distant acknowledgment can increase write latency even when the network is healthy. Different operations can choose different paths. Quantify propagation and coordination costs with explicit assumptions, and avoid assigning a permanent one-letter label to a product without checking its configuration.

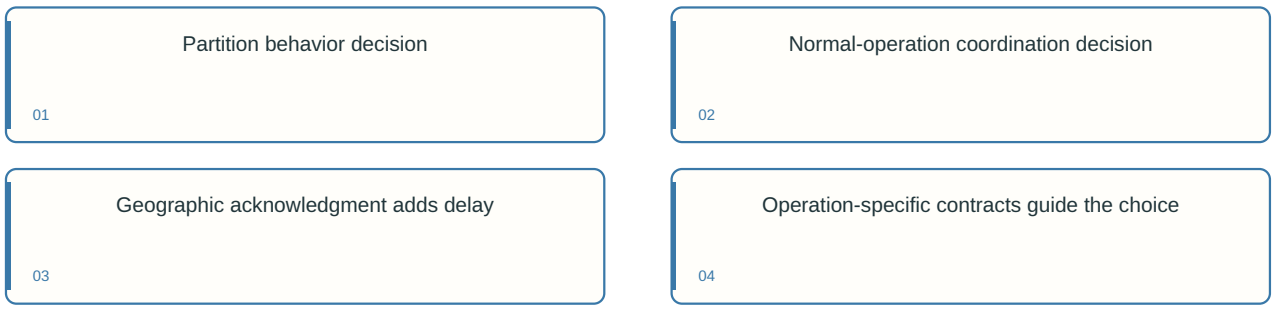


FIG. 14.3.2 **PACELC as a useful extension · worked snapshots**

Partition behavior decision · Normal-operation coordination decision · Geographic acknowledgment adds delay · Operation-specific contracts guide the choice. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.3 · Quorums and read/write trade-offs

With fixed N replicas, R+W>N ensures a read set intersects a write set; W>N/2 ensures two write quorums intersect. These are set properties, not a complete proof of linearizability. Version ordering, concurrent writes, durable acknowledgments, membership, and read repair matter. Sloppy quorums or changing membership can invalidate naive assumptions. In N=3,R=2,W=2, a read sees at least one participant in a completed write set, but still needs a correct rule for interpreting returned versions.

Set	Members	Intersection
Write W=2	A, B	B
Read R=2	B, C	B
Fixed N=3	R + W = 4 > 3	one shared member

FIG. 14.3.3 **Intersecting quorums still need a protocol**

Intersection preserves a possible witness. Version, membership, persistence, and concurrency rules determine how that witness is used.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.4 · Split-brain situations

Split brain means participants act as conflicting authorities. A partitioned old leader may keep running after another is elected. A failure detector cannot directly stop it. Use an ownership protocol and enforce epochs or fencing at the resource that accepts effects. Merely expiring a client-side lock variable is insufficient if an old worker can still write to storage. Test paused workers and delayed messages, not only clean process termination.

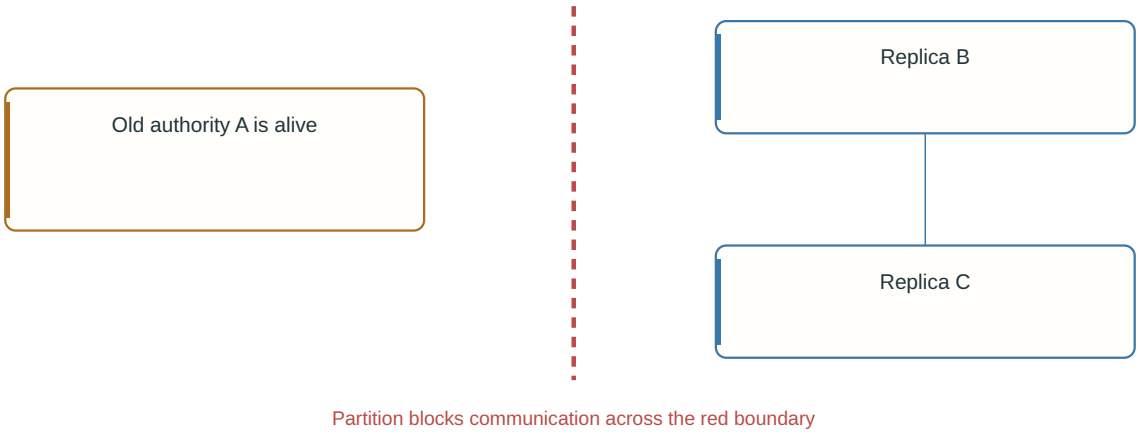


FIG. 14.3.4 **A partition can leave two live but disconnected sides**

Region A remains running while B and C form another reachable side. Correct authority and fencing determine who may make which writes; a missing heartbeat cannot stop A.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · During a partition, which operations could degrade while exclusive allocation must wait or reject?

lamport_receive(5,9) returns 10; vectors [1,0] and [0,1] are concurrent. For three fixed members, R=2,W=2 intersects. Explain why these calculations do not independently resolve concurrent business updates or prove linearizability. The missing protocol rules are part of the answer.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Merely expiring a client-side lock variable is insufficient if an old worker can still write to storage.

Carry forward: Old leader pauses · New leader obtains higher epoch · Old leader resumes · Resource rejects stale epoch effects.

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Coordination, safety & liveness

Prerequisites: 14.3 · Partitions, CAP, PACELC & quorums 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A social application writes a new post in one region while a friend reads it from another. Examine different consistency contracts and their user-visible effects.

This chapter examines coordination, safety & liveness through concrete mechanisms, state snapshots, failure cases, and alternatives.

14.4.1 · Distributed coordination

Coordination establishes shared decisions such as ownership, configuration, or ordered metadata. Use it for invariants that genuinely need agreement, while avoiding a coordination round for independent operations that can safely commute. A coordinator must itself have a failure and recovery protocol; making one service named coordinator does not make it reliable. Bound how clients behave when it is unavailable, and distinguish cached configuration serving from issuing new exclusive ownership.

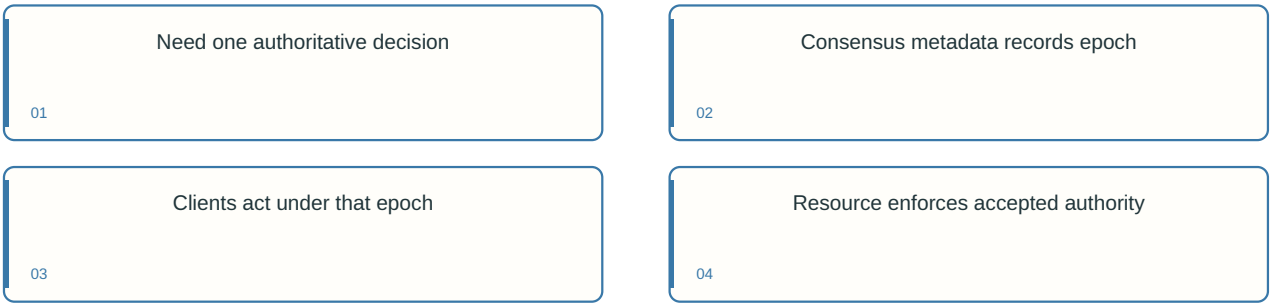


FIG. 14.4.1 **Distributed coordination · worked snapshots**

Need one authoritative decision · Consensus metadata records epoch · Clients act under that epoch · Resource enforces accepted authority. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.4.2 · The fallacies of distributed computing

The fallacies of distributed computing warn against assuming reliable networks, zero latency, infinite bandwidth, static topology, free transport, homogeneous infrastructure, automatic security, or one administrator. Translate each into a concrete design check. A serialized payload crossing a team boundary needs version and authorization policy; a large fan-out needs latency and bandwidth accounting. These warnings are engineering prompts rather than formal guarantees. Use them to discover hidden assumptions in a diagram before an outage discovers them for you.

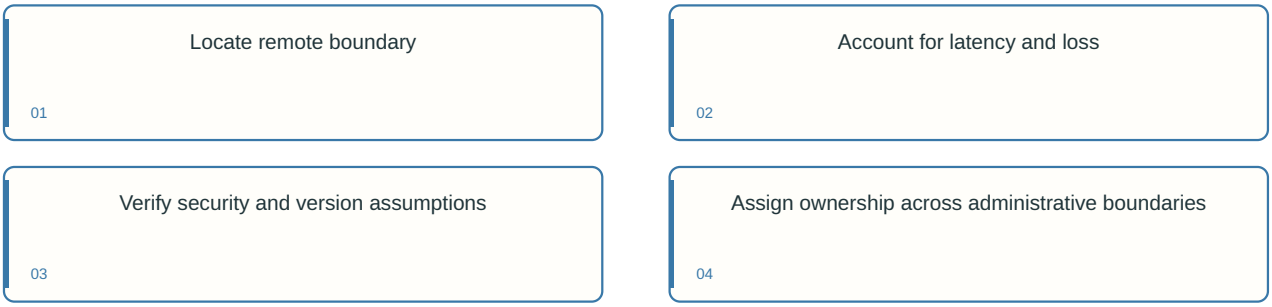


FIG. 14.4.2 **The fallacies of distributed computing · worked snapshots**

Locate remote boundary · Account for latency and loss · Verify security and version assumptions · Assign ownership across administrative boundaries. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

14.4.3 · Safety vs liveness

Safety excludes bad outcomes; liveness requires progress under stated conditions. “No seat is sold twice” is safety. “Every accepted valid booking eventually resolves” is a liveness-oriented obligation with environmental assumptions. A system that rejects all requests may protect the first but fail the second. Network models and failure thresholds can prevent unconditional progress guarantees. Write both properties, state what failures may block completion, and expose stuck work so operators can restore progress without violating safety.

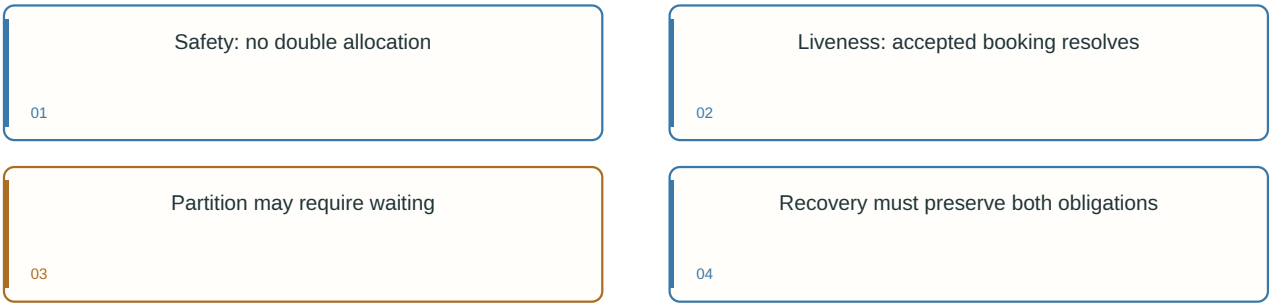


FIG. 14.4.3 **Safety vs liveness · worked snapshots**

Safety: no double allocation · Liveness: accepted booking resolves · Partition may require waiting · Recovery must preserve both obligations. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

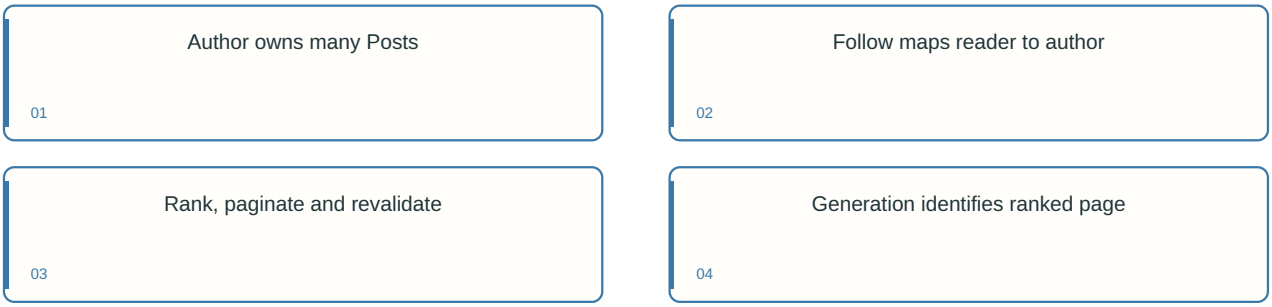


FIG. 14.4.4 **The running system · invariant and recovery review**

Review the assumptions about inactive readers, celebrity skew, and access revocation. Test a deleted or private post already present in several cached candidate pages, duplicate fan-out, a paused worker, and an unavailable feature service. Verify the fallback feed remains authorized. A queue drained to zero does not prove that every intended follower received exactly one usable candidate.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · ORDERING MODELS

```
from mechanisms import vector_relation, lamport_receive
assert vector_relation([1,0], [0,1]) == "concurrent"
assert vector_relation([1,0], [1,2]) == "before"
assert lamport_receive(5, 9) == 10
```

The first pair has no componentwise causal dominance; the second does. A Lamport timestamp smaller than another is not sufficient to prove causality in reverse. These values do not establish a consensus decision or real-time register behavior.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

Inspect fixed-set quorum intersection

Write set example: A, B. Read set example: B, C. Shared members: B.

Every such fixed-set read/write pair must intersect. R + W = 4; N = 3.

This does not establish linearizability or a safe membership transition.

Only set intersection is calculated. Membership, version rules, concurrent writes, persistence, and repair determine the actual consistency protocol. R + W > N alone does not prove linearizability.

▼ Architecture review · During a partition, which operations could degrade while exclusive allocation must wait or reject?

lamport_receive(5,9) returns 10; vectors [1,0] and [0,1] are concurrent. For three fixed members, R=2,W=2 intersects. Explain why these calculations do not independently resolve concurrent business updates or prove linearizability. The missing protocol rules are part of the answer.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Network models and failure thresholds can prevent unconditional progress guarantees.

Carry forward: Safety: no double allocation · Liveness: accepted booking resolves · Partition may require waiting · Recovery must preserve both obligations.

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

PART 4 / UNIT 15

Replication, Leader Election, and Consensus

The running example

Three database replicas, one leader, and a network partition. Walk through elections, writes, acknowledgement rules, recovery, and potential data loss.

Unit 15 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

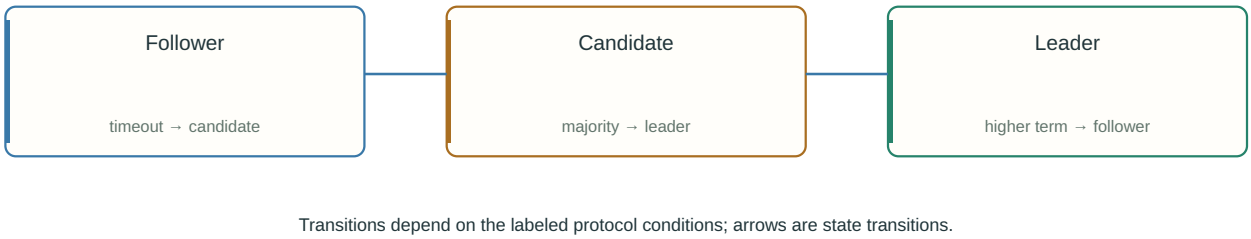
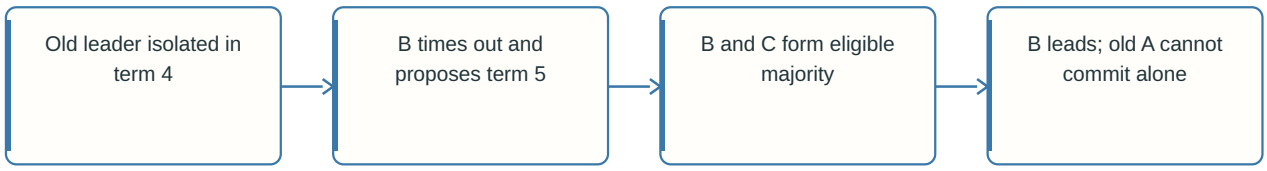


FIG. 15.0.1 **Raft election changes authority under protocol rules**

Election timeout can make a follower a candidate; a valid majority can elect an eligible leader. Observing a higher term returns a participant to follower. This is the role state machine, not the complete protocol.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.0.2 **election timeline**

The example assumes correct persistent voting/log rules and fixed non-Byzantine membership.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Participant	Durable log suffix	Meaning
Leader term 5	8:t4, 9:t5	current-term entry 9
Follower A	8:t4, 9:t5	required durable replica
Follower B	8:t4	lagging replica

FIG. 15.0.3 **Presence in a log is not the same as commitment**

Within the stated fixed-membership Raft protocol, majority replication of the current-term entry can advance commit and protect the prefix. This table is not a generic quorum proof.

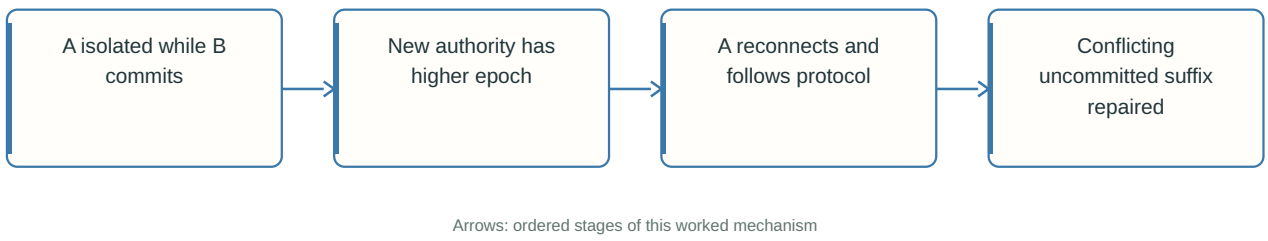
Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

Majority	Members	Shared witness
Election quorum 1	A, B	B
Election quorum 2	B, C	B
Required rules	Voting and durable log evidence	Intersection alone is insufficient

FIG. 15.0.4 **quorum intersection**

Safety comes from protocol rules applied to intersecting decision sets.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.0.5 **network partition recovery**

Recovery does not authorize an old leader to resume independent writes or ignore current term and committed history.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Choose a replication model

Prerequisites: Distributed Systems Fundamentals 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Three database replicas, one leader, and a network partition. Walk through elections, writes, acknowledgement rules, recovery, and potential data loss.

This chapter examines choose a replication model through concrete mechanisms, state snapshots, failure cases, and alternatives.

15.1.1 · Primary/replica architectures

A primary/replica design gives one participant authority for a class of writes and copies its state elsewhere. It simplifies conflict ownership but requires promotion and protection against stale primaries. State who may accept writes and which failures preserve acknowledged data. Read replicas may provide older observations. A topology with one primary and two replicas is not a consensus specification; acknowledgment, persistence, election, membership, and recovery rules determine its actual guarantee.

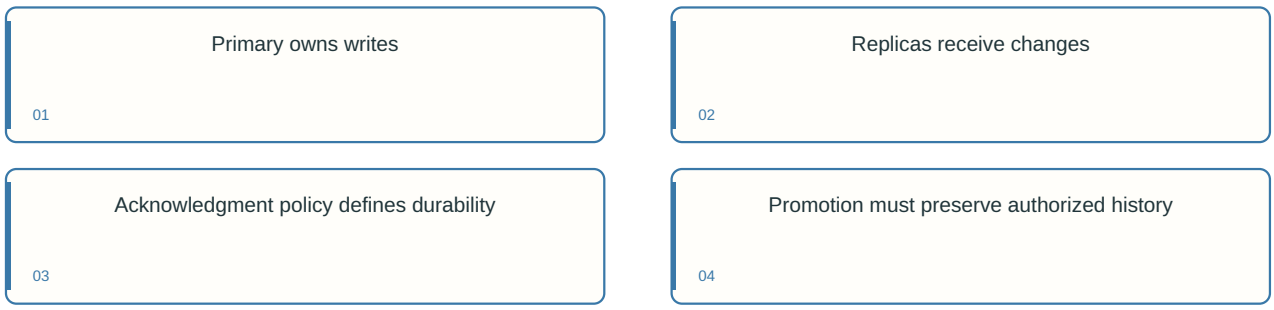


FIG. 15.1.1 **Primary/replica architectures · worked snapshots**

Primary owns writes · Replicas receive changes · Acknowledgment policy defines durability · Promotion must preserve authorized history. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.2 · Leader/follower replication

Leader/follower replication sends an ordered stream from one leader to followers. Followers apply progress and can serve suitable reads or participate in failover under the protocol. Async followers may lag, while sync acknowledgment can increase latency. A follower that has received bytes but not durably stored the required record may not provide the intended crash-recovery protection. Track receive, persist, commit, and apply positions separately when the implementation distinguishes them.

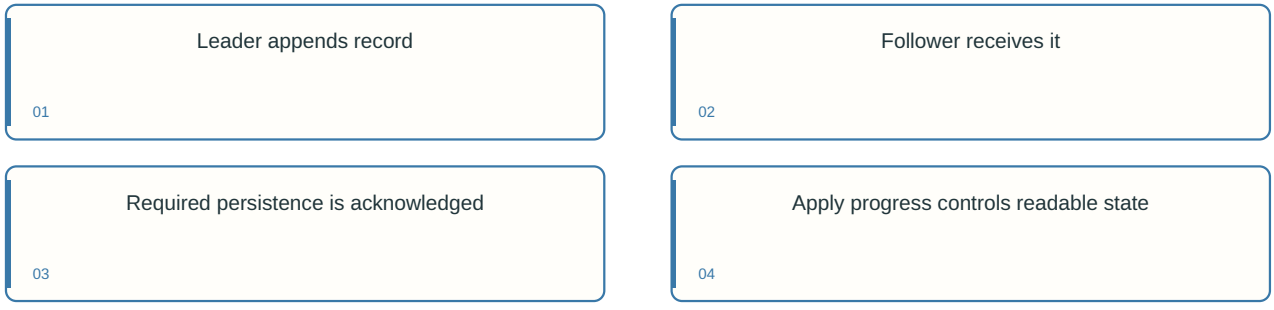


FIG. 15.1.2 **Leader/follower replication · worked snapshots**

Leader appends record · Follower receives it · Required persistence is acknowledged · Apply progress controls readable state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.3 · Multi-leader replication

Multi-leader replication allows writes at several authorities, often reducing local write latency or supporting disconnected regions. Concurrent writes can conflict and need resolution. Last-writer-wins based on unsynchronized timestamps can discard a meaningful update; deterministic convergence is not necessarily correct business merging. Use it for domains with justified conflict semantics, not an invariant that demands a single winner unless additional coordination exists. Observe replication loops, delete propagation, and recovery from prolonged disconnection.

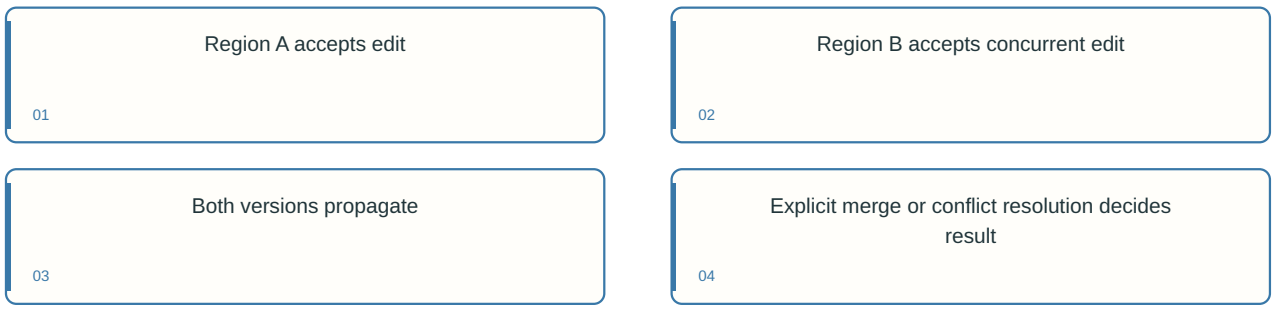


FIG. 15.1.3 **Multi-leader replication · worked snapshots**

Region A accepts edit · Region B accepts concurrent edit · Both versions propagate · Explicit merge or conflict resolution decides result. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.4 · Leaderless replication

Leaderless designs let clients or coordinators contact several replicas and reconcile returned versions. Read repair, hinted handoff, anti-entropy, and quorum configuration are separate mechanisms with implementation-specific effects. A successful response from two of three nodes does not establish linearizability without correct version and membership rules. Model concurrent writes and stale replica recovery. Choose this family when its availability, conflict, and operational contract fits the data; do not describe it as coordination-free for every possible operation.

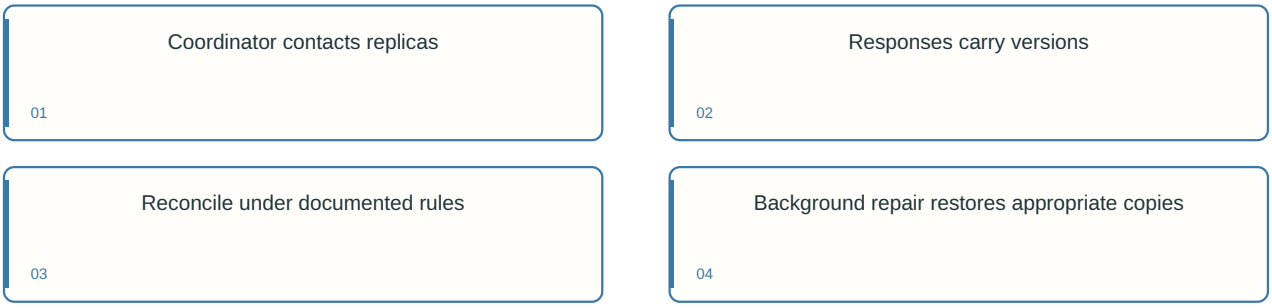


FIG. 15.1.4 **Leaderless replication · worked snapshots**

Coordinator contacts replicas · Responses carry versions · Reconcile under documented rules · Background repair restores appropriate copies. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.5 · Synchronous vs asynchronous replication

Synchronous replication waits for specified acknowledgments before completing a write; asynchronous replication allows acknowledgment earlier. “Synchronous” must name whether an acknowledgment means received, durably stored, or applied. With async copying, loss of the only up-to-date owner can lose recent acknowledged writes. Waiting for independent durable replicas strengthens protection within the fault model but increases latency and may block during failure. Configuration is part of the application contract and must be tested during promotion.

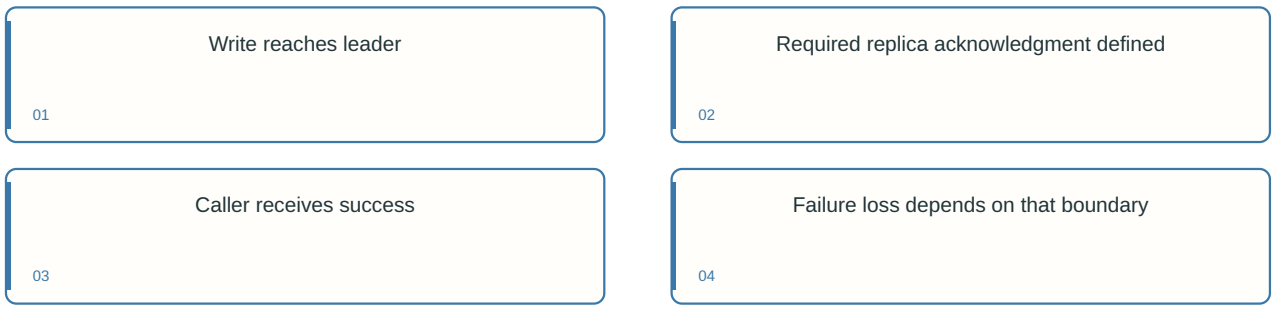


FIG. 15.1.5 **Synchronous vs asynchronous replication · worked snapshots**

Write reaches leader · Required replica acknowledgment defined · Caller receives success · Failure loss depends on that boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.6 · Quorum-based systems

A quorum is a permitted decision set under a protocol. Majority intersection can preserve evidence across elections or writes when persistence, voting, and membership rules hold. In a three-node fixed group, any two majorities share at least one node. That node must also obey rules about what it can acknowledge and later support. Quorum arithmetic without durable state or correct election eligibility is insufficient; a quorum is a component of a protocol, not a universal consistency switch.

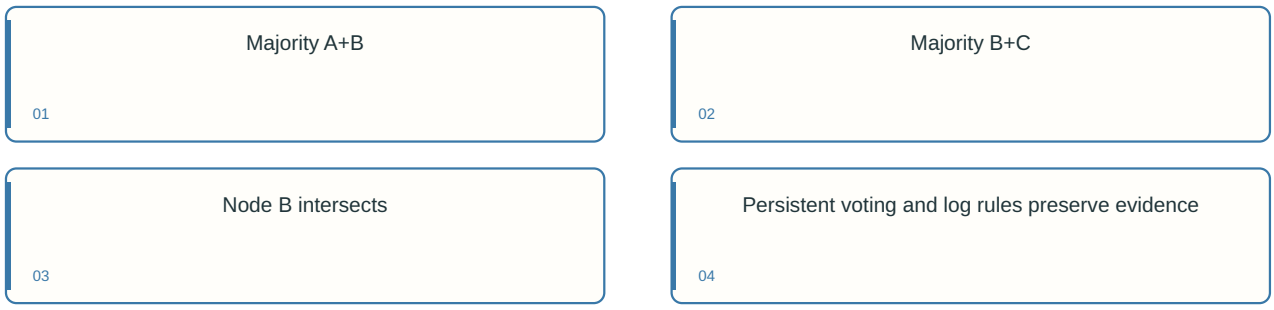


FIG. 15.1.6 **Quorum-based systems · worked snapshots**

Majority A+B · Majority B+C · Node B intersects · Persistent voting and log rules preserve evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What does a region returning after failover have to prove before it can accept authoritative writes again?

Use Workflow epoch checks for the local demonstration: epoch seven cannot complete after epoch eight owns the attempt. Explain why external storage must enforce its own fencing if old workers can still call it. A client-side lease flag cannot stop a resumed remote process.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

That node must also obey rules about what it can acknowledge and later support.

Carry forward: Majority A+B · Majority B+C · Node B intersects · Persistent voting and log rules preserve evidence.

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Raft and replicated state machines

Prerequisites: 15.1 · Choose a replication model 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Three database replicas, one leader, and a network partition. Walk through elections, writes, acknowledgement rules, recovery, and potential data loss.

This chapter examines raft and replicated state machines through concrete mechanisms, state snapshots, failure cases, and alternatives.

15.2.1 · Consensus and replicated state machines

Consensus makes nonfaulty participants agree on a decision sequence under specified assumptions. A replicated state machine applies the same ordered commands deterministically to maintain equivalent state. Nondeterministic choices such as current time or random IDs must be supplied or reconciled consistently rather than recomputed differently on each replica. Consensus does not automatically make external HTTP effects atomic with state-machine application. Keep those effects explicitly identified and deduplicated outside the replicated decision boundary.

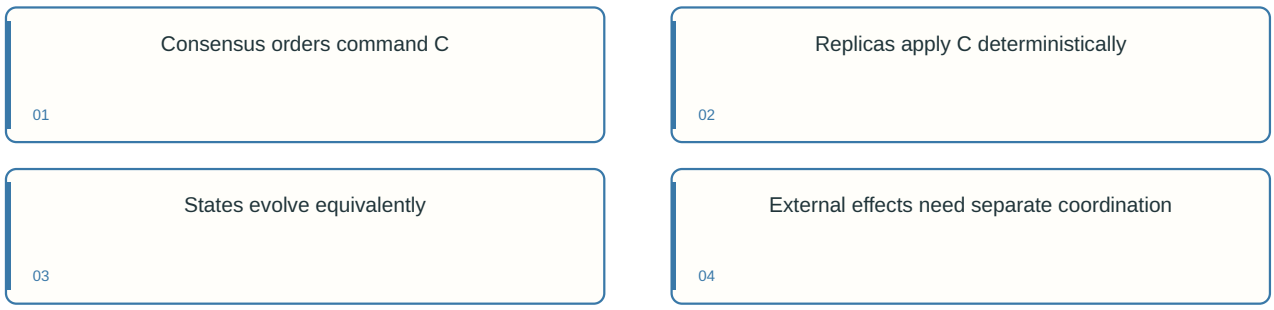


FIG. 15.2.1 **Consensus and replicated state machines · worked snapshots**

Consensus orders command C · Replicas apply C deterministically · States evolve equivalently · External effects need separate coordination. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.2 · Raft: terms, elections, log replication, commit

Raft separates leader election, log replication, and safety. Terms identify election generations; a candidate needs a majority of votes and appropriate log freshness. Followers persist the protocol's required state. The leader replicates entries and advances commitment using Raft's rules, including the restriction on directly committing by majority counting to entries from its current term. This protects prior history through the protocol, not a generic “two acknowledgments always suffice” rule. Reads need a justified freshness mechanism too.

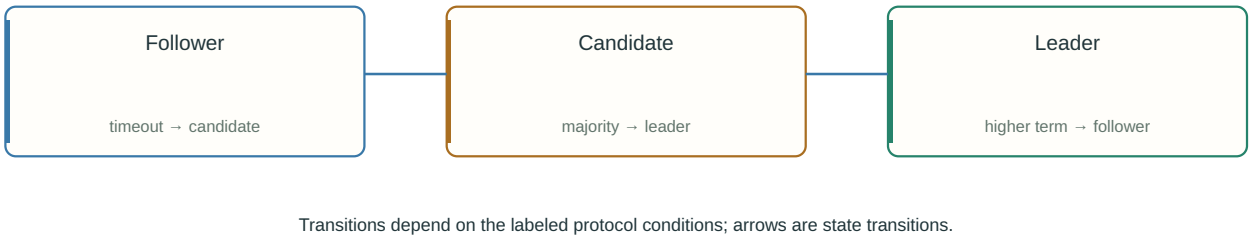


FIG. 15.2.2 **Raft election changes authority under protocol rules**

Election timeout can make a follower a candidate; a valid majority can elect an eligible leader. Observing a higher term returns a participant to follower. This is the role state machine, not the complete protocol.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.3 · Paxos: conceptual foundations

Paxos uses numbered proposals and intersecting majorities to ensure that once a value is chosen, later successful proposals preserve it under the protocol. Prepare and accept phases gather and carry forward required prior evidence. Multi-Paxos extends the approach for ordered decisions, often optimizing around a stable leader. This is a conceptual introduction, not executable consensus code. Production correctness depends on persistence, membership, retries, and liveness details; use a reviewed implementation rather than assembling a lock from the analogy.

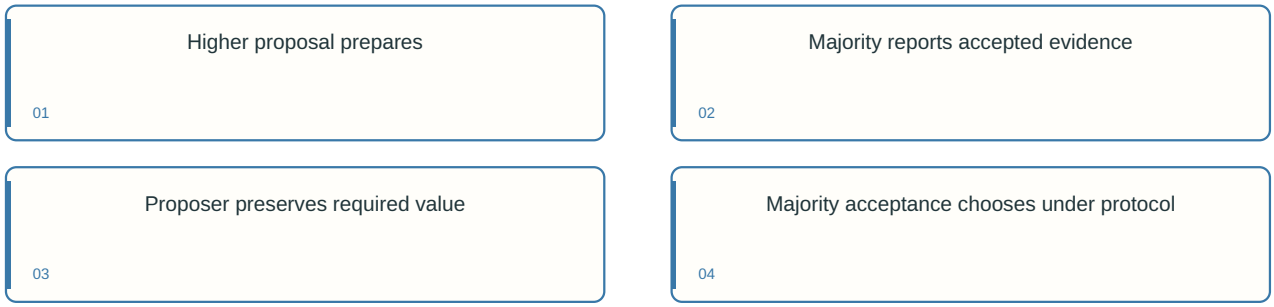


FIG. 15.2.3 **Paxos: conceptual foundations · worked snapshots**

Higher proposal prepares · Majority reports accepted evidence · Proposer preserves required value · Majority acceptance chooses under protocol. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.4 · Log ordering and durability

A replicated log needs both an ordering rule and a durability rule. An entry present on one node is not necessarily committed, and an applied local state is not necessarily safe to expose as globally authoritative. Recovery must distinguish committed prefixes from conflicting uncommitted suffixes. Do not return success because bytes reached a follower unless the protocol says that event establishes the promised outcome. Observe commit and apply progress, and make snapshot installation preserve the same history assumptions.

Participant	Durable log suffix	Meaning
Leader term 5	8:t4, 9:t5	current-term entry 9
Follower A	8:t4, 9:t5	required durable replica
Follower B	8:t4	lagging replica

FIG. 15.2.4 **Presence in a log is not the same as commitment**

Within the stated fixed-membership Raft protocol, majority replication of the current-term entry can advance commit and protect the prefix. This table is not a generic quorum proof.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.5 · Membership changes

Membership changes alter which groups can make decisions. Replacing an old majority with a disjoint new majority in one step can allow conflicting histories. Protocols such as Raft use a coordinated transition, including overlapping quorum requirements in joint consensus. Learners may catch up before becoming voting members. A cluster-size setting is not merely an operations toggle: apply the supported membership protocol, preserve persistent identities, and test interruption during reconfiguration.

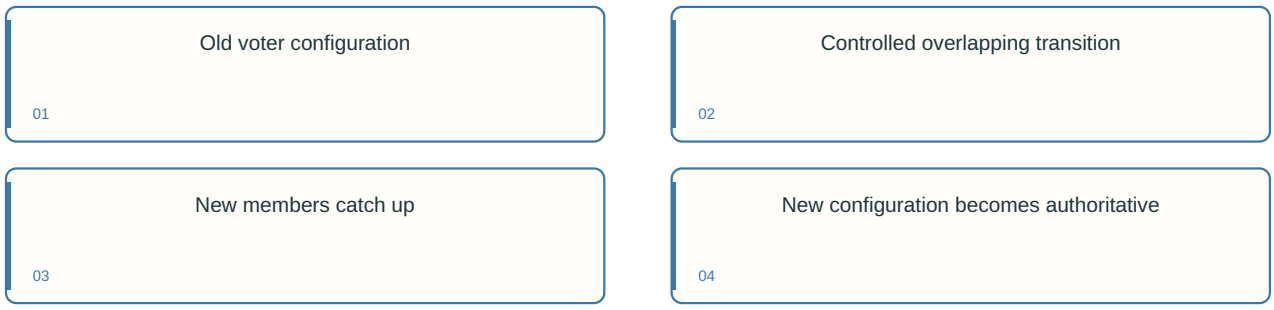


FIG. 15.2.5 **Membership changes · worked snapshots**

Old voter configuration · Controlled overlapping transition · New members catch up · New configuration becomes authoritative. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What does a region returning after failover have to prove before it can accept authoritative writes again?

Use Workflow epoch checks for the local demonstration: epoch seven cannot complete after epoch eight owns the attempt. Explain why external storage must enforce its own fencing if old workers can still call it. A client-side lease flag cannot stop a resumed remote process.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Learners may catch up before becoming voting members.

Carry forward: Old voter configuration · Controlled overlapping transition · New members catch up · New configuration becomes authoritative.

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Fences, leases & failure detection

Prerequisites: 15.2 · Raft and replicated state machines 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Three database replicas, one leader, and a network partition. Walk through elections, writes, acknowledgement rules, recovery, and potential data loss.

This chapter examines fences, leases & failure detection through concrete mechanisms, state snapshots, failure cases, and alternatives.

15.3.1 · Split-brain prevention and fencing tokens

A fencing token is a monotonically ordered ownership epoch checked by the resource receiving an effect. Worker A holding token 7 can pause past its lease while B obtains token 8. When A resumes, the protected store must reject token 7. A token generated but never enforced at the side-effect boundary does not fence anything. The receiving resource must atomically compare and apply, and the design must account for operations already accepted before the newer owner appeared.

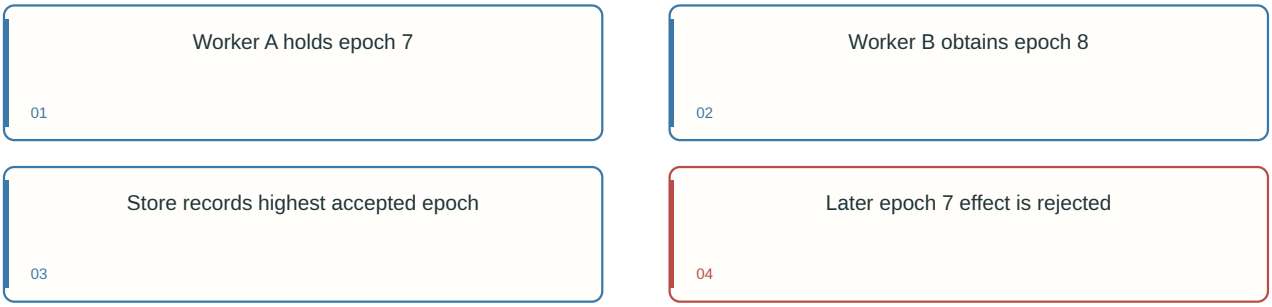


FIG. 15.3.1 **Split-brain prevention and fencing tokens · worked snapshots**

Worker A holds epoch 7 · Worker B obtains epoch 8 · Store records highest accepted epoch · Later epoch 7 effect is rejected. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.2 · Failure detection

Failure detection observes missing or delayed evidence and produces suspicion under timing assumptions. Short detection intervals can reduce failover delay but increase false suspicions during pauses or network congestion. The safety protocol must remain correct when suspicion is wrong. Separate detector tuning from authority transfer and resource fencing. A successful heartbeat also does not prove that the node can meet every business operation's deadline, so readiness and capacity signals remain useful alongside protocol heartbeats.

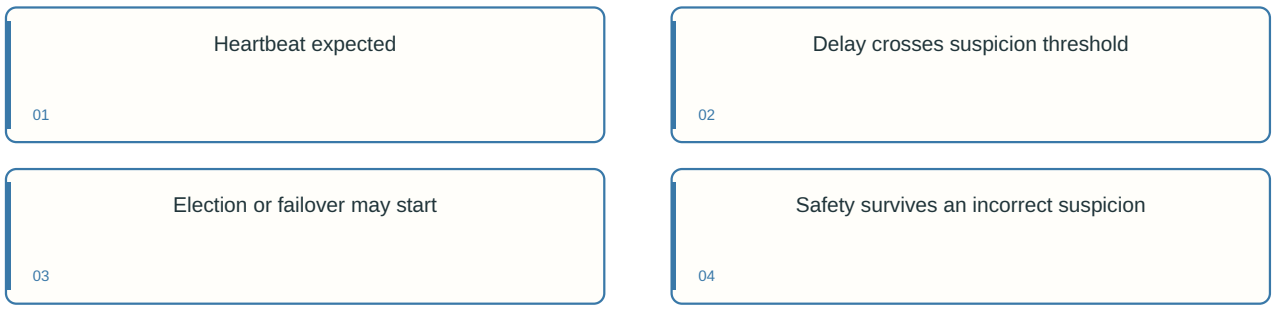


FIG. 15.3.2 **Failure detection · worked snapshots**

Heartbeat expected · Delay crosses suspicion threshold · Election or failover may start · Safety survives an incorrect suspicion. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.3 · Strongly consistent metadata services

Strongly consistent metadata services can coordinate configuration, ownership epochs, and small control-plane decisions. They are usually unsuitable as the high-volume store for every application payload. Cache safe configuration for serving where appropriate, and specify behavior when new ownership cannot be issued. A metadata outage can freeze deployments or leases while existing data paths continue; that distinction should appear in the failure model. Avoid making every request synchronously depend on a control plane unless its cost is justified.

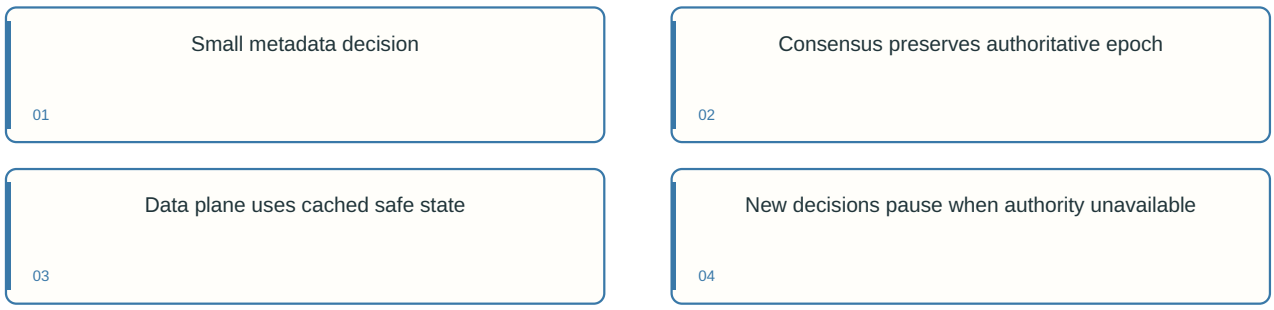


FIG. 15.3.3 **Strongly consistent metadata services · worked snapshots**

Small metadata decision · Consensus preserves authoritative epoch · Data plane uses cached safe state · New decisions pause when authority unavailable. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.4 · Distributed locks and leases

A lock grants a form of exclusive permission; a lease grants it for a time under clock and expiry assumptions. A paused owner may act after its lease ends, so resource-side fencing is often needed for correctness-critical effects. Releasing must verify owner identity rather than blindly delete another owner's lock. Lock acquisition and protected work are separate events. Prefer a single authoritative data transaction for a local invariant when it avoids an unnecessary distributed locking protocol.

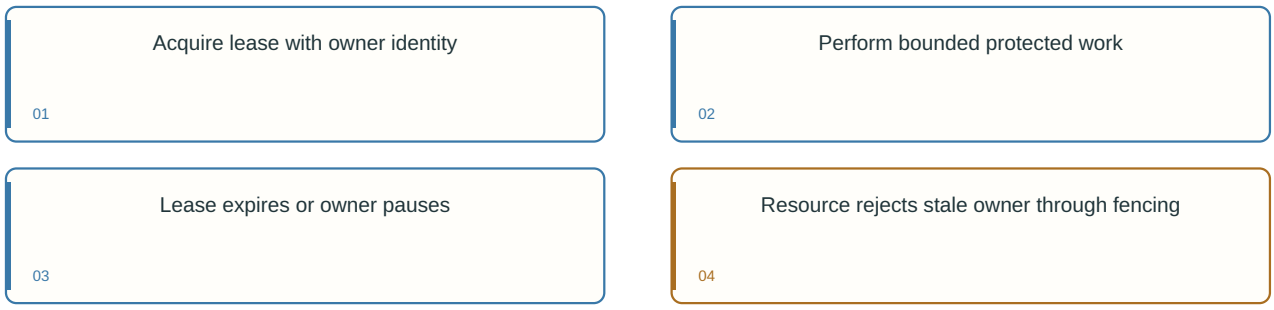


FIG. 15.3.4 **Distributed locks and leases · worked snapshots**

Acquire lease with owner identity · Perform bounded protected work · Lease expires or owner pauses · Resource rejects stale owner through fencing. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What does a region returning after failover have to prove before it can accept authoritative writes again?

Use Workflow epoch checks for the local demonstration: epoch seven cannot complete after epoch eight owns the attempt. Explain why external storage must enforce its own fencing if old workers can still call it. A client-side lease flag cannot stop a resumed remote process.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Lock acquisition and protected work are separate events.

Carry forward: Acquire lease with owner identity · Perform bounded protected work · Lease expires or owner pauses · Resource rejects stale owner through fencing.

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Conflicts, CRDTs & regional trade-offs

Prerequisites: 15.3 · Fences, leases & failure detection 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Three database replicas, one leader, and a network partition. Walk through elections, writes, acknowledgement rules, recovery, and potential data loss.

This chapter examines conflicts, crdts & regional trade-offs through concrete mechanisms, state snapshots, failure cases, and alternatives.

15.4.1 · Conflict resolution and CRDT introduction

CRDTs arrange updates and merges so replicas converge under stated algebraic and delivery assumptions. A grow-only set can merge by union; an observed-remove set needs metadata to distinguish observed additions from concurrent ones. Convergence does not enforce arbitrary invariants such as “stock never below zero.” Choose a type whose semantics match the domain, preserve required metadata, and bound tombstone growth or compaction safely. An explicit conflict review may be better for business edits without a natural automatic merge.

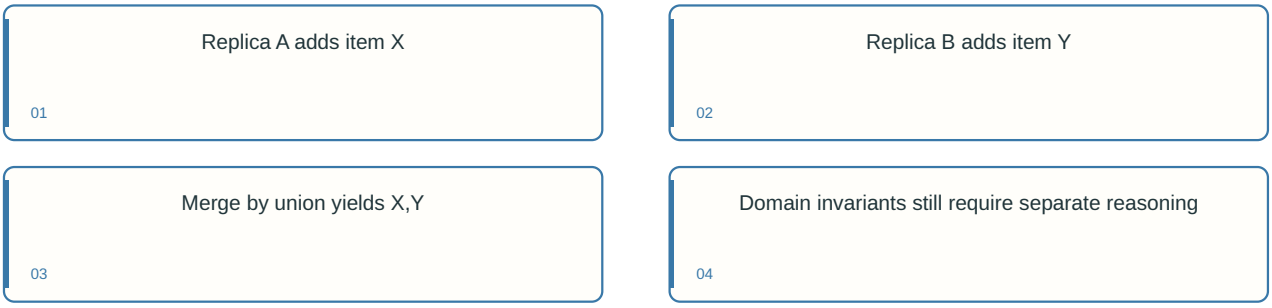


FIG. 15.4.1 **Conflict resolution and CRDT introduction · worked snapshots**

Replica A adds item X · Replica B adds item Y · Merge by union yields X,Y · Domain invariants still require separate reasoning. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

15.4.2 · Multi-region replication trade-offs

Regional replication must define write authority, conflict behavior, acknowledgment geography, and disaster recovery. A nearby async acknowledgment has different data-loss exposure from a cross-region durable quorum. During a partition, some operations may wait or reject while stale-tolerant reads continue. Reconnection requires catch-up and reconciliation before arbitrary old owners resume. Test failback as well as failover: restoring an old primary without epoch and history control can reintroduce conflicting state after the visible outage ends.

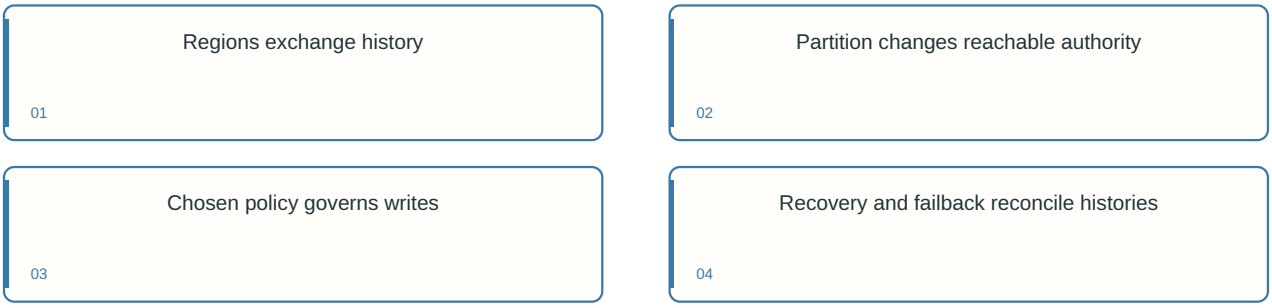
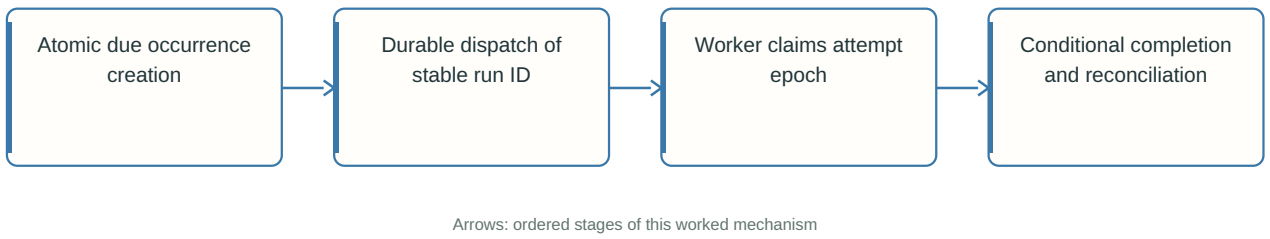


FIG. 15.4.2 **Multi-region replication trade-offs · worked snapshots**

Regions exchange history · Partition changes reachable authority · Chosen policy governs writes · Recovery and failback reconcile histories. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.4.3 **The running system · ordered responsibility**

Scanner locks or conditionally claims a due schedule, creates the unique occurrence and dispatch intent, advances next_due, and commits. Worker claims an attempt epoch, executes the stable run identity, renews its lease if supported, and conditionally records completion. An expired attempt can be replaced, but resource fencing or effect deduplication must prevent harmful stale work. Reconcile runs without terminal progress.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

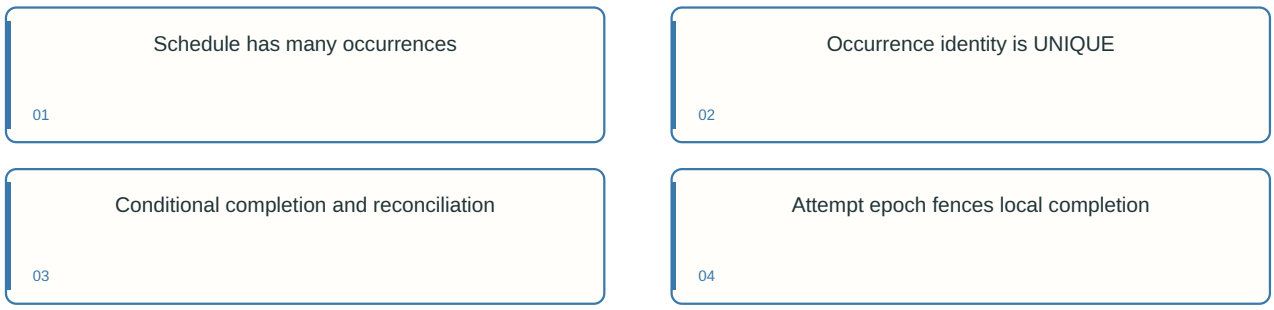


FIG. 15.4.4 **The running system · invariant and recovery review**

Review the stable run identity, atomic schedule advancement, stale-worker behavior, and recovery from a historical snapshot. Test scanner crash at commit boundaries, worker pause beyond lease, cancellation during execution, timezone transition, and catch-up after a long outage. A scheduler that fires a callback on time once has not demonstrated durable scheduling or duplicate-safe effects.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PSEUDOCODE · THREE REPLICAS, ONE PARTITION

```
A,B,C contain committed index 8
partition isolates old leader A
B and C elect eligible B in term 5
B appends index 9, term 5
B and C durably replicate index 9
Raft commit rule establishes committed prefix
A cannot acknowledge a new quorum-committed write alone
rejoin: A follows term 5 and repairs uncommitted suffix
```

This trace assumes non-Byzantine members, correct durable protocol state, and a fixed three-voter configuration. It is not a replacement for the full Raft election and log safety rules or a production consensus implementation.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · What does a region returning after failover have to prove before it

can accept authoritative writes again?

Use Workflow epoch checks for the local demonstration: epoch seven cannot complete after epoch eight owns the attempt. Explain why external storage must enforce its own fencing if old workers can still call it. A client-side lease flag cannot stop a resumed remote process.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Reconnection requires catch-up and reconciliation before arbitrary old owners resume.

Carry forward: Regions exchange history · Partition changes reachable authority · Chosen policy governs writes · Recovery and failback reconcile histories.

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   `Access reviewed · 21 October 2026`

PART 4 / UNIT 16

Message Queues, Events, and Distributed Workflows

The running example

Design an e-commerce checkout that coordinates payment, inventory reservation, shipment creation, and refund compensation.

Unit 16 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

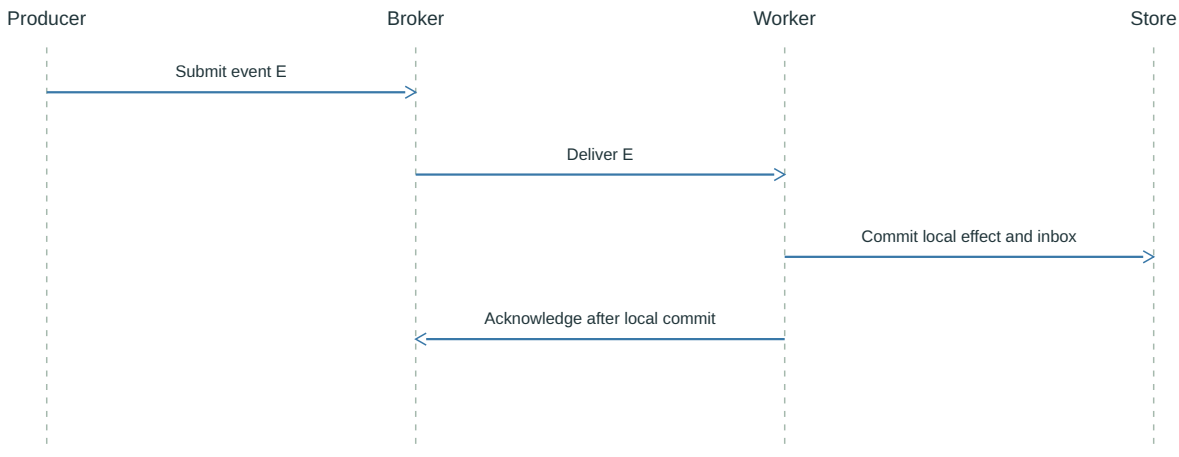


FIG. 16.0.1 **Queue lifecycle**

A lost final acknowledgment can cause redelivery; the local effect contract must handle it.

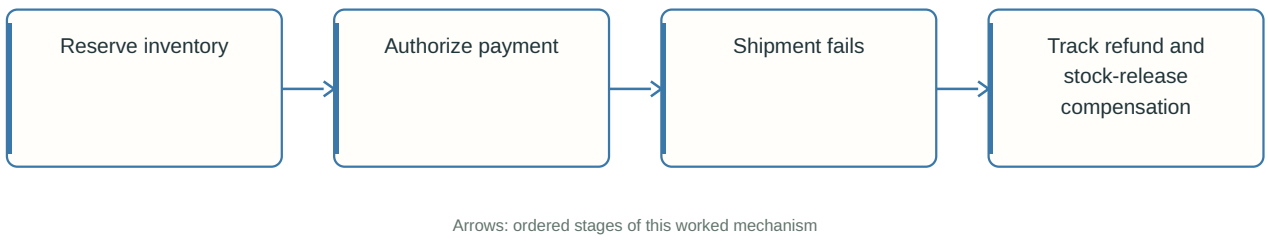
Why this view: Follow explicit message ordering and locate the acknowledgment or effect boundary. Author-created illustration. Numbers are illustrative unless stated otherwise.

Partition	Owner before	Owner after rebalance
P0	Consumer A	Consumer B
P1	Consumer B	Consumer B
Progress	Last committed offset	In-flight work may replay

FIG. 16.0.2 **consumer partition assignment**

Ordering is per partition under this model, and rebalancing does not make in-flight side effects disappear.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

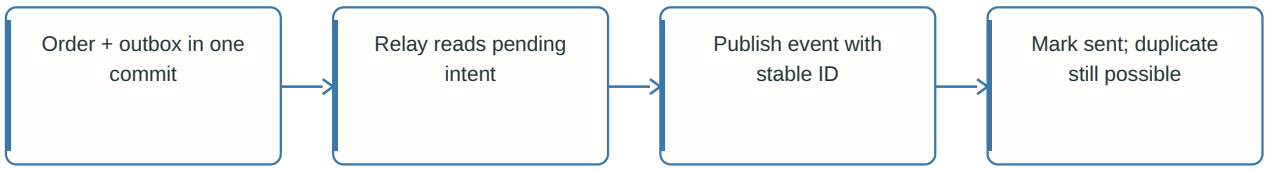


Arrows: ordered stages of this worked mechanism

FIG. 16.0.3 **Saga happy/failure paths**

Compensation has its own retry, uncertainty, and final-state obligations.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

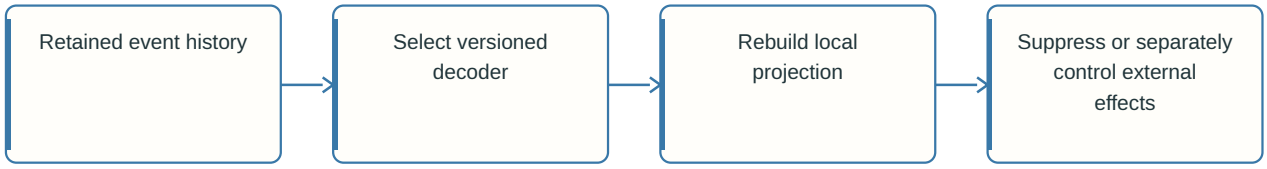


Arrows: ordered stages of this worked mechanism

FIG. 16.0.4 **The outbox closes a local publication gap**

The relay can crash after publish but before marking. A consumer must handle the repeated event; the database transaction does not include the entire network.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 16.0.5 **event replay**

Replaying state must not blindly repeat emails, charges, or other external business effects.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Queues, logs & delivery boundaries

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus

9 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an e-commerce checkout that coordinates payment, inventory reservation, shipment creation, and refund compensation.

This chapter examines queues, logs & delivery boundaries through concrete mechanisms, state snapshots, failure cases, and alternatives.

16.1.1 · Synchronous vs asynchronous communication

Synchronous communication makes the caller wait for the result; asynchronous communication records or sends work whose completion is observed later. A receipt email can be deferred after an order commits, while payment acceptance may require a different response contract. Asynchrony moves waiting and failure handling into durable state and delivery protocols. A queue can absorb a short burst, but sustained arrival above completion still grows backlog. Define who owns accepted work after the initiating request ends.

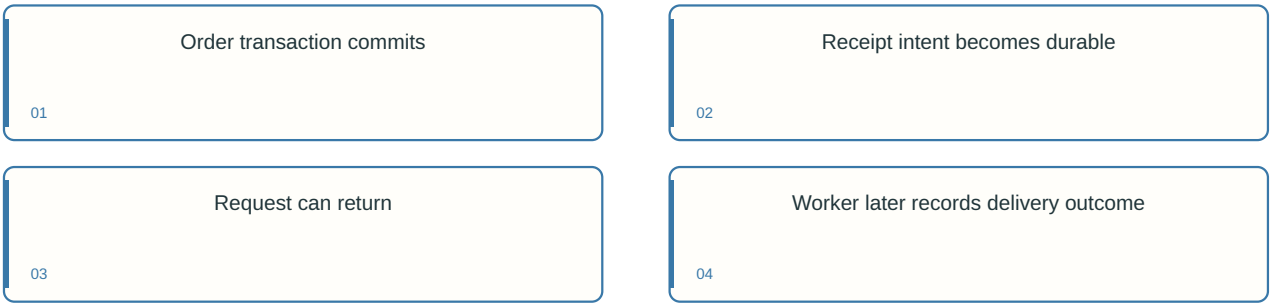


FIG. 16.1.1 **Synchronous vs asynchronous communication · worked snapshots**

Order transaction commits · Receipt intent becomes durable · Request can return · Worker later records delivery outcome. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.2 · Queues vs logs vs pub/sub

A queue commonly distributes work among consumers; a retained log preserves an ordered append history for replay; pub/sub distributes publications to subscribers under its specific retention and acknowledgment semantics. Products combine these features, so check the actual contract. A consumer that needs replay after an outage cannot rely on an ephemeral subscription alone. Choose retention, partitioning, and consumer progress rules from recovery requirements rather than assuming every broker is a durable event history.

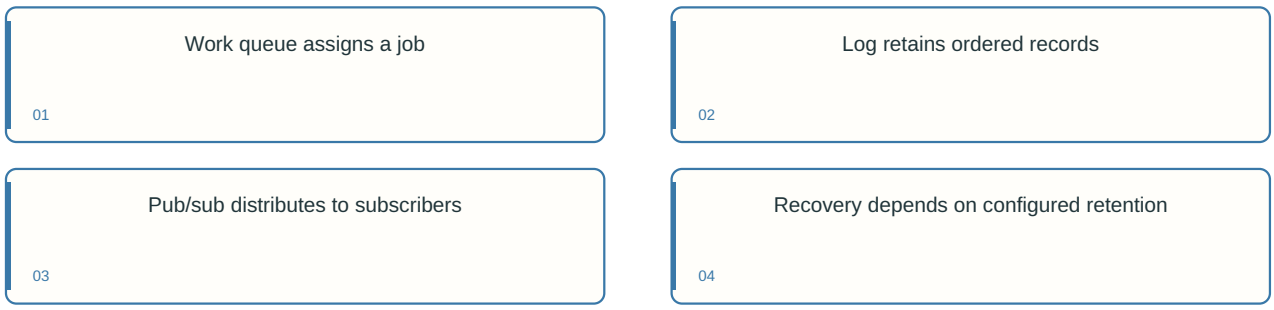


FIG. 16.1.2 **Queues vs logs vs pub/sub · worked snapshots**

Work queue assigns a job · Log retains ordered records · Pub/sub distributes to subscribers · Recovery depends on configured retention. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.3 · Producers, consumers, brokers

Producers create messages, brokers accept or route them, and consumers interpret them. There are several distinct acknowledgments: producer acceptance, durable broker replication, consumer receipt, and completion of a business effect. Naming only “acknowledged” hides the boundary. Include stable message identity, schema version, ownership, and correlation metadata. A broker can be healthy while consumers repeatedly fail, so monitor end-to-end age and outcome rather than only broker uptime and enqueue rate.

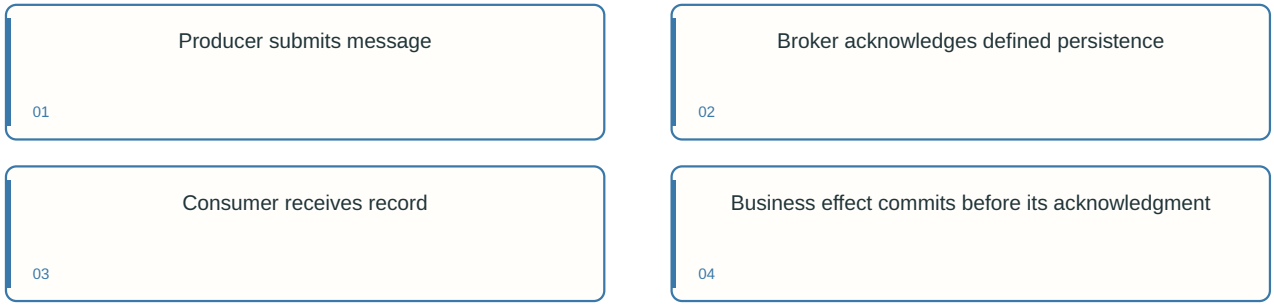


FIG. 16.1.3 **Producers, consumers, brokers · worked snapshots**

Producer submits message · Broker acknowledges defined persistence · Consumer receives record · Business effect commits before its acknowledgment. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.4 · Partitions, consumer groups, offsets

Partitions provide ordered lanes in systems such as Kafka; consumer groups commonly distribute partitions among members, and offsets represent progress within a lane. Order is typically per partition, not across all partitions. Rebalancing can move ownership and cause work to repeat around an acknowledgment boundary. Commit offsets according to the effect contract and preserve stable keys for required ordering. One very hot partition can limit a group despite many idle consumers.

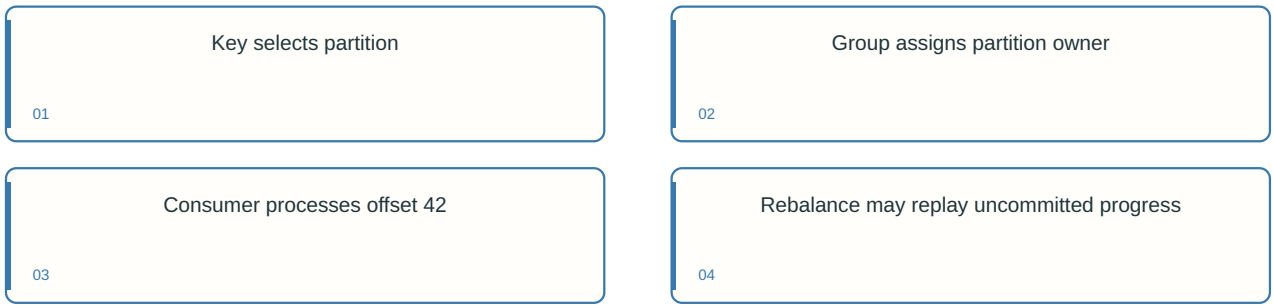


FIG. 16.1.4 **Partitions, consumer groups, offsets · worked snapshots**

Key selects partition · Group assigns partition owner · Consumer processes offset 42 · Rebalance may replay uncommitted progress. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.5 · Delivery guarantees

Delivery terminology must identify where messages and effects are counted. A broker can deliver at least once while a transactional consumer produces one local database effect through deduplication. An external provider can still receive repeated calls. Describe the full path: source acceptance, retention, retry, effect commit, and progress acknowledgment. Strong guarantees at one stage do not automatically compose into an end-to-end guarantee across independent systems and failures.

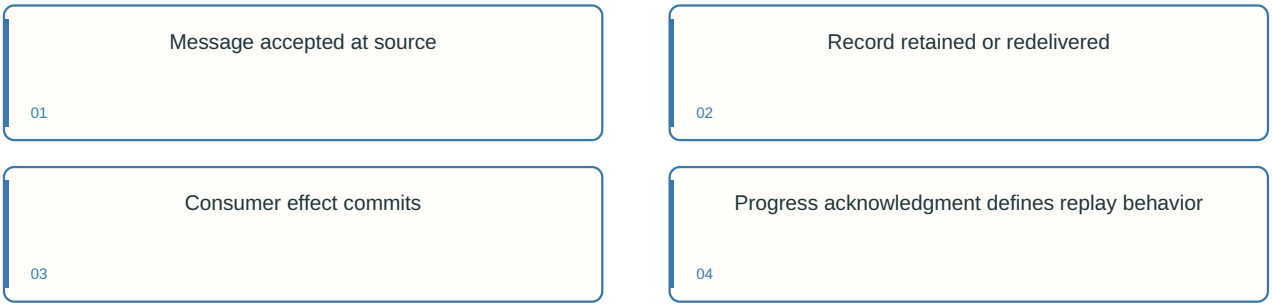


FIG. 16.1.5 **Delivery guarantees · worked snapshots**

Message accepted at source · Record retained or redelivered · Consumer effect commits · Progress acknowledgment defines replay behavior. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.6 · At-most-once and at-least-once processing

At-most-once processing can acknowledge before an effect and lose work after a crash. It avoids certain duplicates at the cost of missing effects. It may suit disposable telemetry under a deliberate loss budget, but not an order that has been promised durable processing. The term does not imply “always one”; zero is allowed. Specify whether the guarantee concerns delivery attempts, processing starts, or committed results because different boundaries can produce different counts.

At-least-once processing retries until acknowledged under retention and recovery assumptions, so duplicates are possible. Commit a durable effect before acknowledging and make repetition safe. A crash after commit but before acknowledgment replays the record. A uniqueness constraint on event identity within the same local transaction as the effect can prevent duplicate local changes. If the transaction marks processed before making an unprotected remote call, it can lose that call; the effect boundary matters.

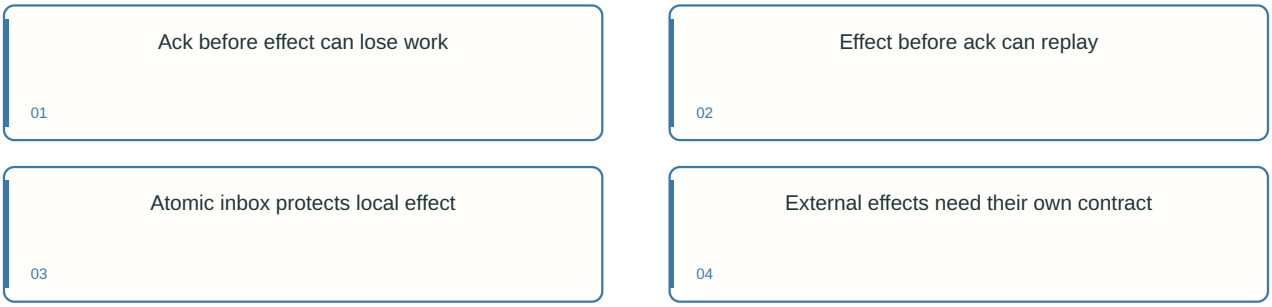


FIG. 16.1.6 **At-most-once and at-least-once processing · worked snapshots**

Ack before effect can lose work · Effect before ack can replay · Atomic inbox protects local effect · External effects need their own contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.7 · Exactly-once processing within defined boundaries

Exactly-once claims require a precise boundary. A transactional stream processor may atomically couple consumed progress and produced records within its supported system. That does not include an arbitrary email server or payment provider unless another protocol covers the side effect. Idempotency can give a single relevant effect for repeated attempts under a retention and identity contract; it does not mean execution started only once. Name failures and recovery assumptions instead of using the phrase as a universal broker property.

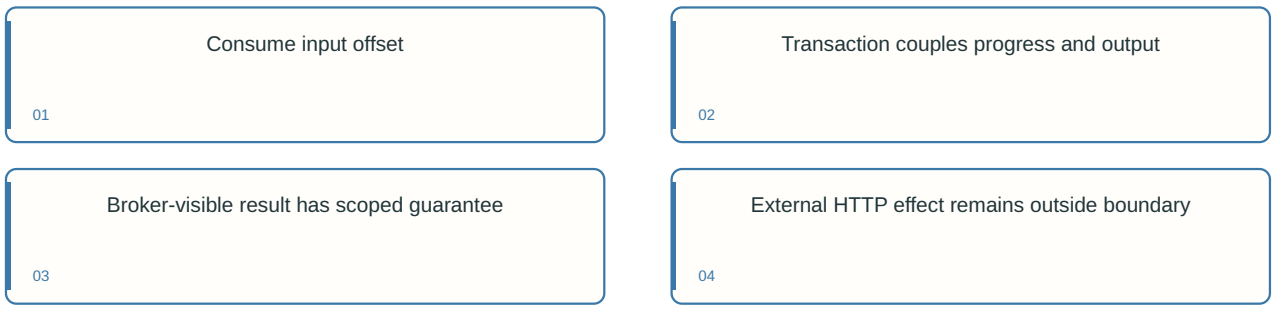


FIG. 16.1.7 **Exactly-once processing within defined boundaries · worked snapshots**

Consume input offset · Transaction couples progress and output · Broker-visible result has scoped guarantee · External HTTP effect remains outside boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.8 · Ordering guarantees

Ordering can mean publication order, per-key log order, consumer start order, or effect-commit order. Parallel processing can preserve one while changing another. If account operations must apply sequentially, partition or coordinate by account and do not acknowledge later changes as if earlier ones completed. Poison messages require a policy: skipping can restore progress while breaking the business sequence. Use sequence numbers and explicit gap handling where the domain depends on ordered effects.

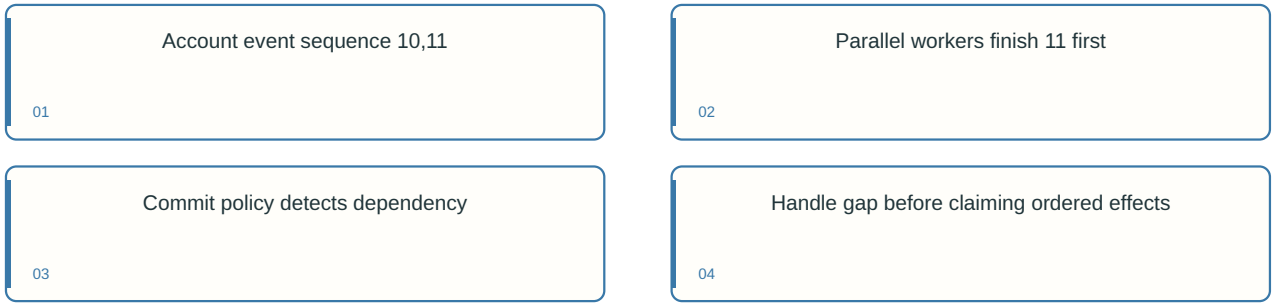


FIG. 16.1.8 **Ordering guarantees · worked snapshots**

Account event sequence 10,11 · Parallel workers finish 11 first · Commit policy detects dependency · Handle gap before claiming ordered effects. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.9 · Consumer acknowledgements

A consumer acknowledgment permits the broker to advance or remove delivery state according to the protocol. If acknowledgment precedes a required effect, a crash can lose work; if it follows, a crash can repeat work. There is no magical placement that atomically covers an unrelated external system. Use local transactions, inbox/outbox records, provider operation keys, and reconciliation as appropriate. Keep acknowledgment deadlines compatible with processing time and lease extension rules so healthy long jobs are not repeatedly redelivered.

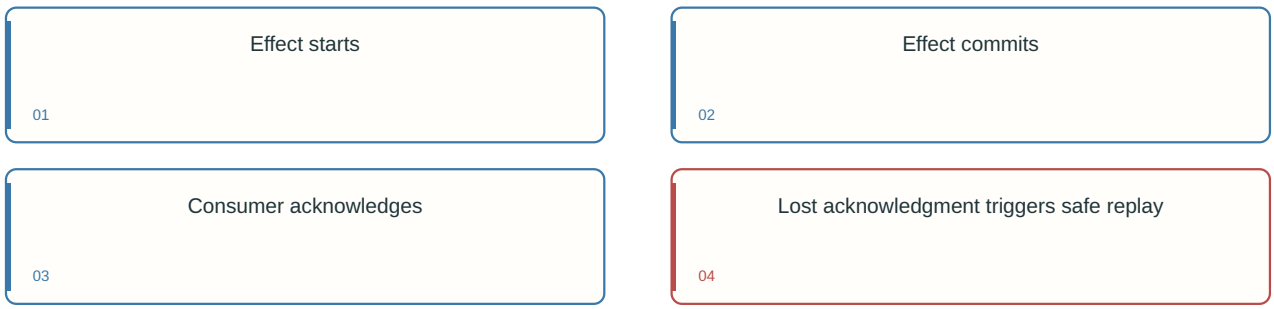


FIG. 16.1.9 **Consumer acknowledgements · worked snapshots**

Effect starts · Effect commits · Consumer acknowledges · Lost acknowledgment triggers safe replay. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can one workflow be exactly once at a broker boundary but still send two emails? Specify the boundaries.

The example records two deliveries but one counter effect. Deduplication identity and effect commit share a transaction. If an external email or payment is substituted for the counter, the local inbox no longer establishes one external effect; use the provider contract and reconciliation.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Use local transactions, inbox/outbox records, provider operation keys, and reconciliation as appropriate.

Carry forward: Effect starts · Effect commits · Consumer acknowledges · Lost acknowledgment triggers safe replay.

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Retries, poison messages & idempotence

Prerequisites: 16.1 · Queues, logs & delivery boundaries 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an e-commerce checkout that coordinates payment, inventory reservation, shipment creation, and refund compensation.

This chapter examines retries, poison messages & idempotence through concrete mechanisms, state snapshots, failure cases, and alternatives.

16.2.1 · Retries and dead-letter queues

Retry transient failures with a bounded policy and delay; dead-letter persistent failures with context and a path for repair. A dead-letter queue is not a completed business outcome. Include original identity, attempt count, failure reason, and source version, then alert or reconcile unresolved important work. Replaying a repaired record must preserve deduplication and ordering rules. An unbounded retry loop can block a partition or overwhelm a permanently invalid downstream request.

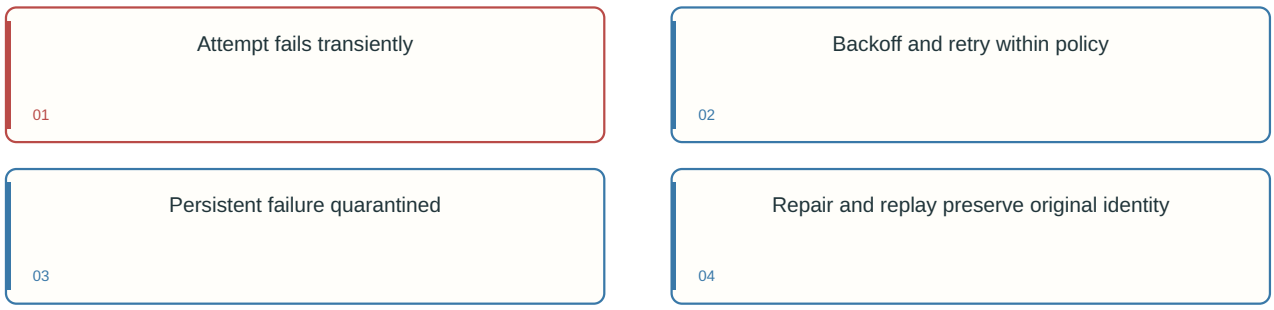


FIG. 16.2.1 **Retries and dead-letter queues · worked snapshots**

Attempt fails transiently · Backoff and retry within policy · Persistent failure quarantined · Repair and replay preserve original identity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.2 · Poison messages

A poison message repeatedly fails because of malformed data, incompatible schema, or an invariant violation. Distinguish it from a temporary dependency failure so retries do not waste the same resources forever. Quarantine it with evidence and decide whether dependent messages must wait. A parser bug can make an entire valid historical schema look poisonous; inspect error distributions and replay samples before discarding records. Privacy rules apply to diagnostic payloads retained in quarantine.

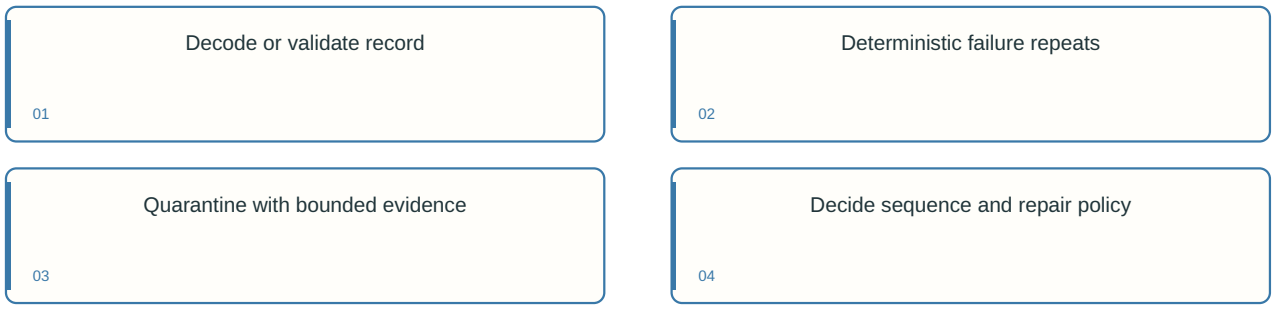


FIG. 16.2.2 **Poison messages · worked snapshots**

Decode or validate record · Deterministic failure repeats · Quarantine with bounded evidence · Decide sequence and repair policy. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.3 · Idempotent consumers

An idempotent consumer records event identity and its relevant local effect atomically. INSERT processed(event_id) under uniqueness and the corresponding state update must share one transaction if they protect the same invariant. A duplicate then returns the already established outcome. Retain identity long enough for permitted replay and delayed delivery. This protects local effects only; calling an external provider still requires its operation key or reconciliation. Deduplicating only in volatile memory fails after restart.

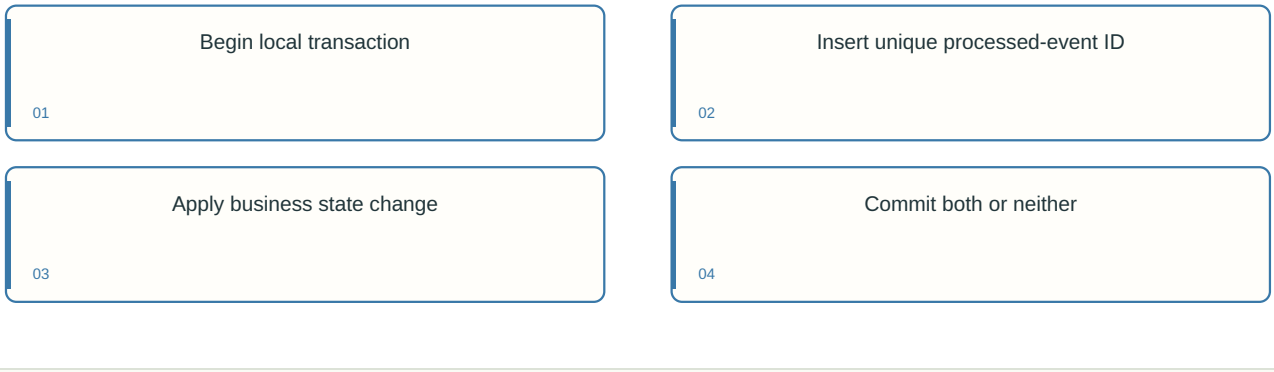


FIG. 16.2.3 **Idempotent consumers · worked snapshots**

Begin local transaction · Insert unique processed-event ID · Apply business state change · Commit both or neither. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.4 · Event sourcing

Event sourcing stores domain events as the authoritative history from which state is derived. It supports audit and reconstruction when event semantics remain interpretable, but adds schema evolution, correction, replay, and privacy obligations. A snapshot accelerates reconstruction and must identify its event position. Do not replay external side effects while merely rebuilding state. Append-only history is not automatically a legally sufficient ledger or an authorization model; domain and operational constraints still need design.

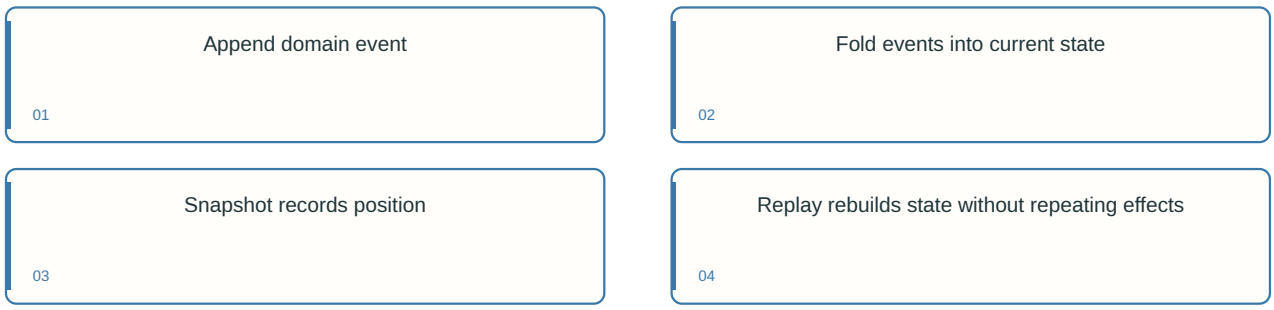


FIG. 16.2.4 **Event sourcing · worked snapshots**

Append domain event · Fold events into current state · Snapshot records position · Replay rebuilds state without repeating effects. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can one workflow be exactly once at a broker boundary but still send two emails? Specify the boundaries.

The example records two deliveries but one counter effect. Deduplication identity and effect commit share a transaction. If an external email or payment is substituted for the counter, the local inbox no longer establishes one external effect; use the provider contract and reconciliation.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Do not replay external side effects while merely rebuilding state.

Carry forward: Append domain event · Fold events into current state · Snapshot records position · Replay rebuilds state without repeating effects.

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author’s explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Event sourcing, CQRS & the outbox

Prerequisites: 16.2 · Retries, poison messages & idempotence 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an e-commerce checkout that coordinates payment, inventory reservation, shipment creation, and refund compensation.

This chapter examines event sourcing, cqrs & the outbox through concrete mechanisms, state snapshots, failure cases, and alternatives.

16.3.1 · CQRS

CQRS separates command and query responsibilities or models when their requirements differ. An order command model can enforce transitions while a read projection provides a convenient customer history. The projection may lag, so clients need a read-after-write policy. CQRS does not require event sourcing, independent databases, or microservices. Its additional models and synchronization are justified when they simplify real complexity, not when a straightforward CRUD application already satisfies its access patterns.

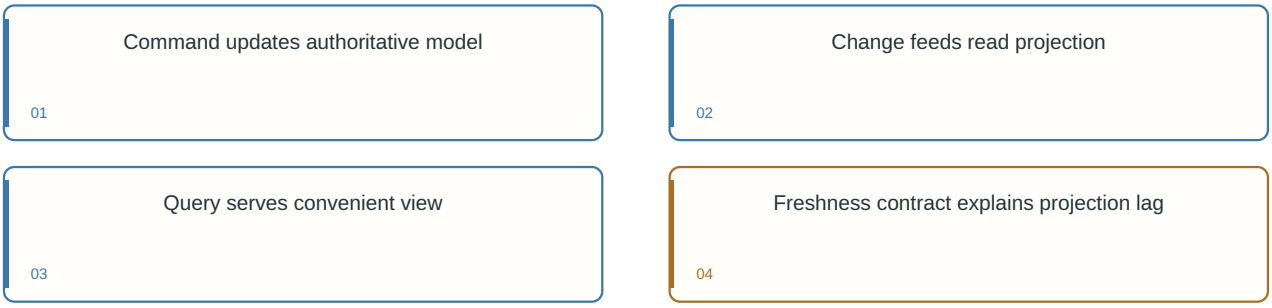


FIG. 16.3.1 **CQRS · worked snapshots**

Command updates authoritative model · Change feeds read projection · Query serves convenient view · Freshness contract explains projection lag. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.3.2 · Transactional outbox

A transactional outbox records state changes and publication intent in the same local transaction. A relay later publishes pending intents and marks progress. If it publishes then crashes before marking, it can publish again, so consumers still deduplicate. The pattern closes a local database-to-publication gap; it does not make arbitrary subscribers part of that transaction. Use stable event IDs, retryable relay progress, retention, and reconciliation for intents that remain unprocessed too long.

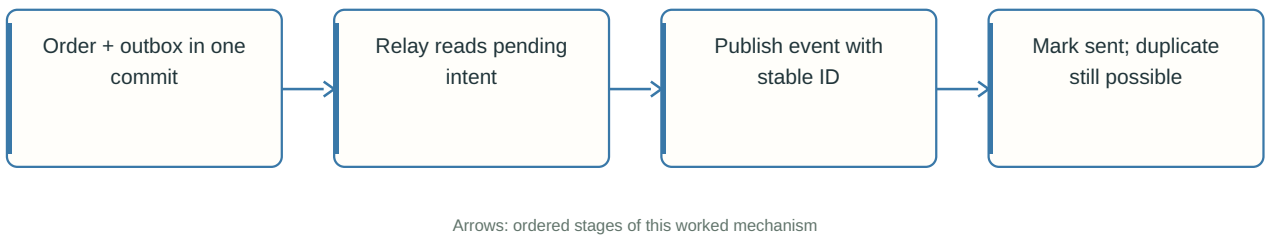


FIG. 16.3.2 **The outbox closes a local publication gap**

The relay can crash after publish but before marking. A consumer must handle the repeated event; the database transaction does not include the entire network.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.3.3 · Sagas and compensating transactions

A saga coordinates local transactions and compensating actions across ownership boundaries. Inventory reservation, payment authorization, and shipment creation cannot be treated as one ordinary local transaction. On failure, compensation might release stock or request a refund, but it can fail and may not perfectly reverse the original action. Persist workflow state and compensation progress. Distinguish recoverable intermediate states from final success or failure, and reconcile with external systems after uncertain responses.

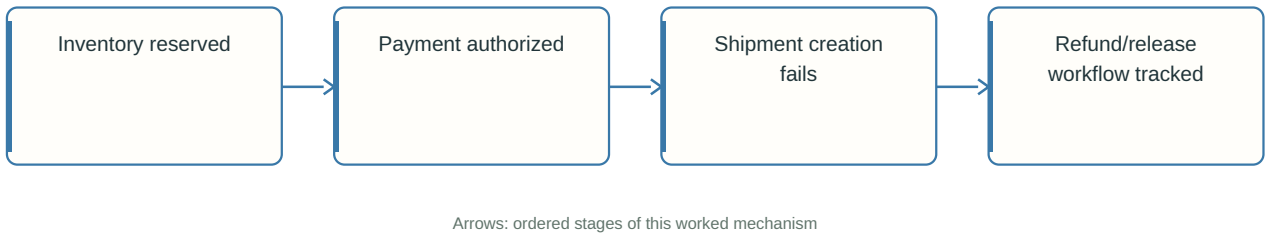


FIG. 16.3.3 **A failed shipment starts tracked compensation**

Each step is a local outcome. Compensation can fail or remain uncertain and must have durable progress and reconciliation.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

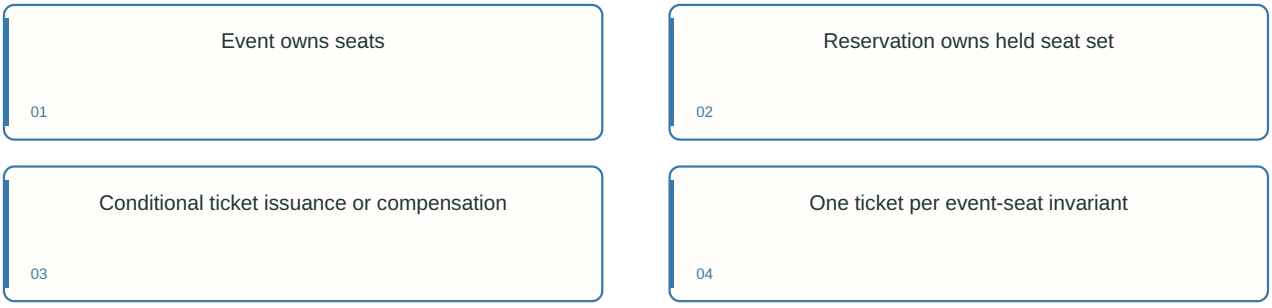


FIG. 16.3.4 **The running system · invariant and recovery review**

Review whether expiry and payment use the same reservation version, whether late payment is resolved safely, and whether the display is confused with authority. Test two buyers for one seat, duplicate callbacks, crash after commit, stale expiry after reassignment, and payment success after expiry. A correct happy-path lock does not answer those workflow histories.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Can one workflow be exactly once at a broker boundary but still send two emails? Specify the boundaries.

The example records two deliveries but one counter effect. Deduplication identity and effect commit share a transaction. If an external email or payment is substituted for the counter, the local inbox no longer establishes one external effect; use the provider contract and reconciliation.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Persist workflow state and compensation progress.

Carry forward: Reserve inventory · Authorize payment · Shipment creation fails · Compensate and track unresolved actions.

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Sagas, durable work & reconciliation

Prerequisites: 16.3 · Event sourcing, CQRS & the outbox 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design an e-commerce checkout that coordinates payment, inventory reservation, shipment creation, and refund compensation.

This chapter examines sagas, durable work & reconciliation through concrete mechanisms, state snapshots, failure cases, and alternatives.

16.4.1 · Choreography vs orchestration

Choreography lets participants react to events without one central workflow coordinator; orchestration stores and directs the workflow explicitly. Choreography can reduce centralized coupling but makes end-to-end progress harder to see. Orchestration clarifies ownership and retries while introducing a coordinator and workflow schema to operate. Neither eliminates participant failures. Choose based on workflow complexity, auditability, change ownership, and compensation needs, and maintain a correlation identity through every step.

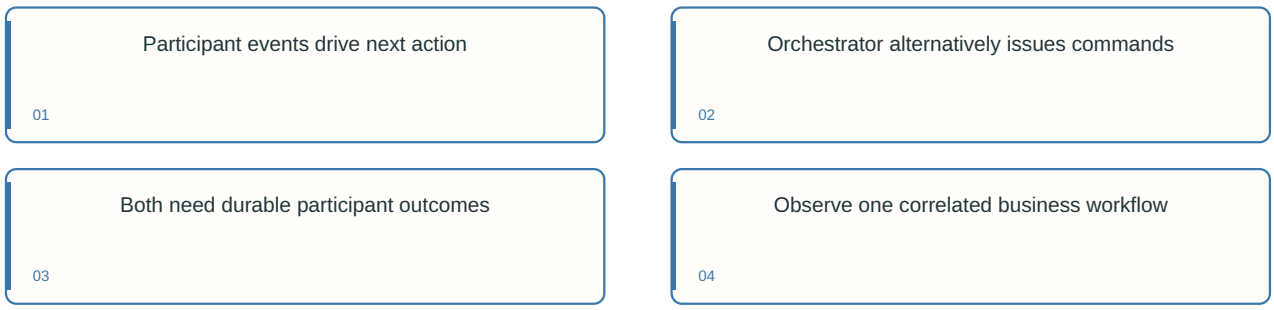


FIG. 16.4.1 **Choreography vs orchestration · worked snapshots**

Participant events drive next action · Orchestrator alternatively issues commands · Both need durable participant outcomes · Observe one correlated business workflow. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.2 · Workflow engines

A workflow engine persists execution progress and resumes work after failures under its programming model. It can manage timers, retries, and human waits, but activities calling external systems can still execute more than once around acknowledgment gaps. Understand deterministic replay, versioning, activity identity, and cancellation rules of the chosen implementation. An engine is a tool for durable orchestration, not proof that every external side effect or business invariant has become exactly once.



FIG. 16.4.2 **Workflow engines · worked snapshots**

Workflow state persisted · Activity dispatched · Result may be uncertain after worker loss · Resume with stable activity identity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.3 · Durable job execution

Durable jobs have a persistent identity, input reference, state, attempt history, and ownership protocol. A worker should record completion under the current attempt or fencing epoch so a stale worker cannot overwrite a newer result. Store large payloads outside the queue and validate input version before work. Retry does not mean unlimited recomputation: set budgets and surface terminal or review states. A process-local background task is not durable merely because the request returned 202.

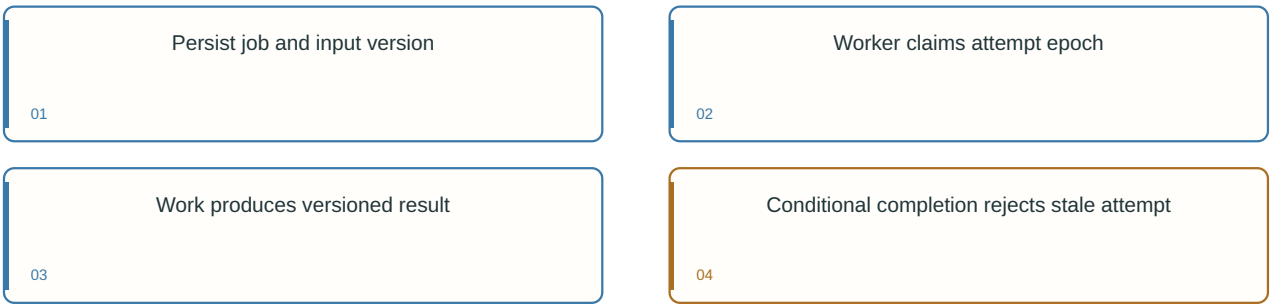


FIG. 16.4.3 **Durable job execution · worked snapshots**

Persist job and input version · Worker claims attempt epoch · Work produces versioned result · Conditional completion rejects stale attempt. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.4 · Scheduling, leases, and worker coordination

Scheduling combines a durable due-time record with worker coordination. A scheduler claims due work using an atomic transition or lease and creates a stable run identity such as (schedule_id,occurrence). Clock changes, missed periods, overlapping runs, and failover need explicit policy. A lease can expire while the old worker runs, so protect effects with fencing or idempotency at their resource. Exactly one scheduler process is not a substitute for a recoverable occurrence protocol.

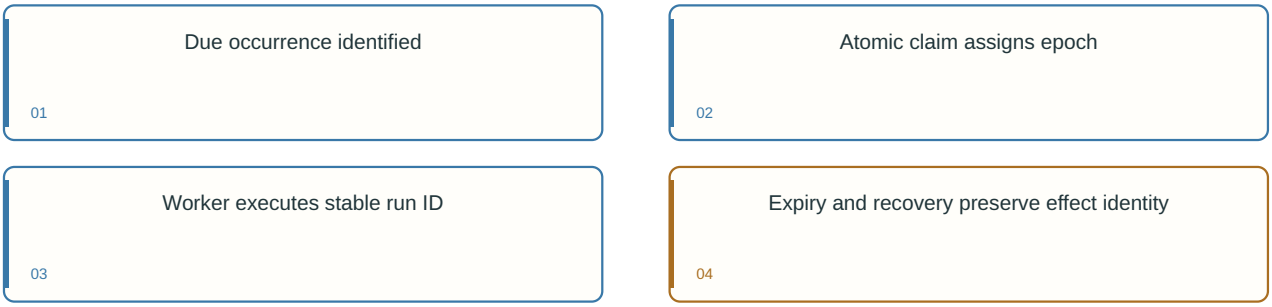


FIG. 16.4.4 **Scheduling, leases, and worker coordination · worked snapshots**

Due occurrence identified · Atomic claim assigns epoch · Worker executes stable run ID · Expiry and recovery preserve effect identity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.5 · Backpressure and replay

Backpressure controls admission or producer progress when consumers cannot keep up; replay deliberately revisits retained input. Replaying at full historical speed can overload live dependencies and delay current work. Give replay its own rate and capacity budget, preserve source identity, and isolate side effects. Retention must cover the recovery horizon. Queue length alone is not enough: monitor oldest age, per-key blockage, and meaningful completion rates so a seemingly busy pipeline does not conceal stuck business outcomes.

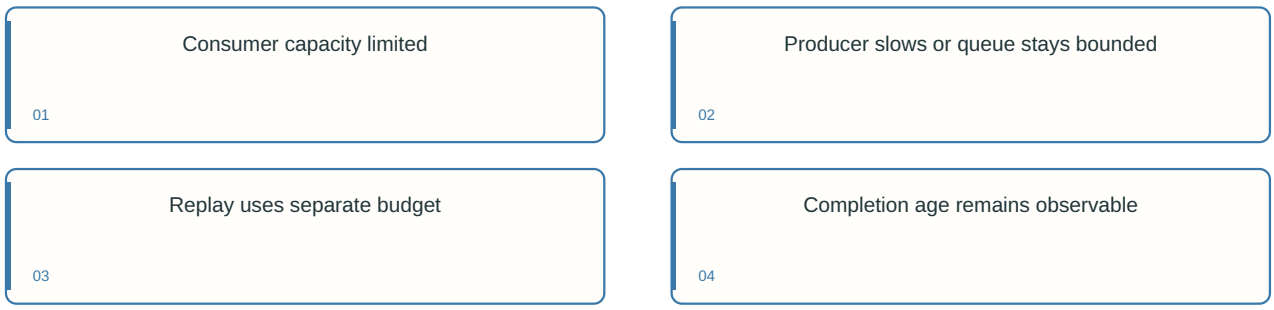


FIG. 16.4.5 **Backpressure and replay · worked snapshots**

Consumer capacity limited · Producer slows or queue stays bounded · Replay uses separate budget · Completion age remains observable. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.6 · Reconciliation after partial failure

Reconciliation compares intended and observed outcomes after partial failure. A payment marked pending locally may already be charged at the provider; query by stable operation identity and repair the local state rather than blindly charging again. Reconciliation must itself be idempotent, authorized, and observable. Run it periodically and during incident recovery, tracking unresolved discrepancies with age and ownership. It is an essential protocol stage, not an admission that the normal path can ignore persistence or validation.

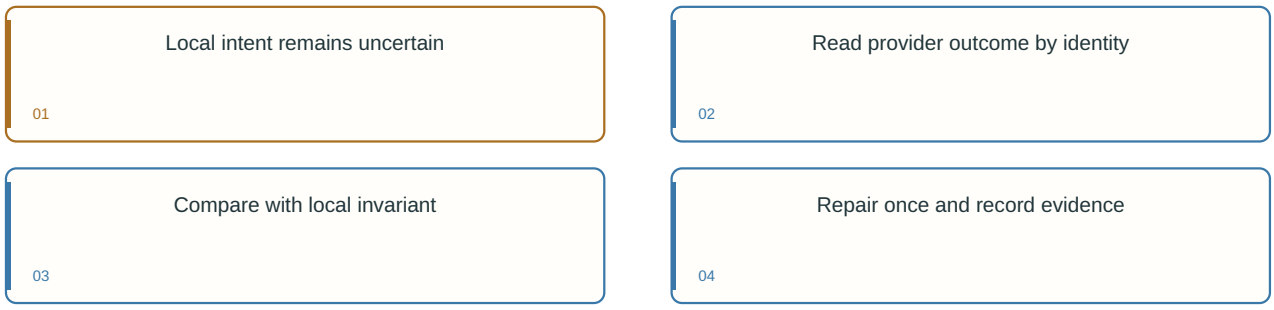


FIG. 16.4.6 **Reconciliation after partial failure · worked snapshots**

Local intent remains uncertain · Read provider outcome by identity · Compare with local invariant · Repair once and record evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · DUPLICATE DELIVERY, ONE LOCAL EFFECT

```
from mechanisms import open_demo_store, consume_local_event
db = open_demo_store()
assert consume_local_event(db, "E1", "orders")
assert not consume_local_event(db, "E1", "orders")
assert db.execute("SELECT value FROM counters").fetchone()[0] == 1
db.close()
```

The event ID and local effect share a transaction. Replacing the counter with an external provider call invalidates this one-local-effect argument unless the external boundary has its own protocol.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · Can one workflow be exactly once at a broker boundary but still send two emails? Specify the boundaries.

The example records two deliveries but one counter effect. Deduplication identity and effect commit share a transaction. If an external email or payment is substituted for the counter, the local inbox no longer establishes one external effect; use the provider contract and reconciliation.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Run it periodically and during incident recovery, tracking unresolved discrepancies with age and ownership.

Carry forward: Local intent remains uncertain · Read provider outcome by identity · Compare with local invariant · Repair once and record evidence.

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author’s explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

PART 4 / UNIT 17

Reliability, Fault Tolerance, and Disaster Recovery

The running example

A downstream payment provider becomes unavailable during peak traffic. Demonstrate the failure cascade and progressively introduce resilience mechanisms.

Unit 17 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

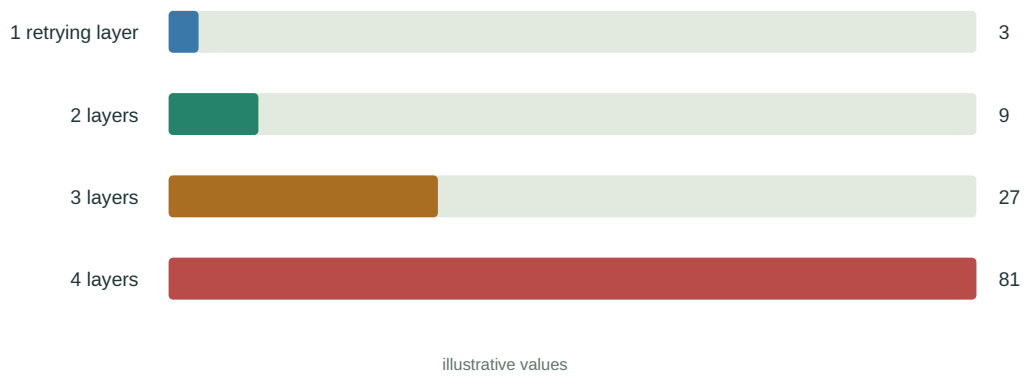
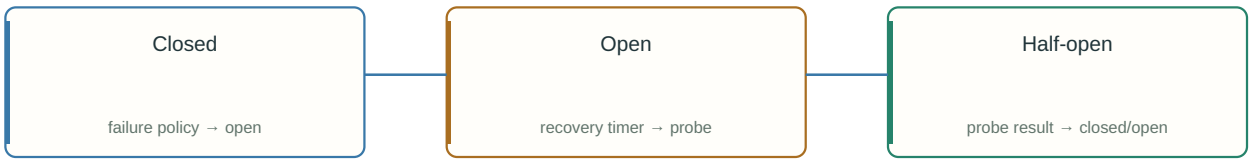


FIG. 17.0.1 **Retry amplification timeline**

Three attempts at every independent layer can amplify an intent geometrically; jitter alone does not reduce the attempt budget.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.



Transitions depend on the labeled protocol conditions; arrows are state transitions.

FIG. 17.0.2 **A circuit breaker bounds futile calls**

Closed permits calls, an observed failure policy opens the breaker, and bounded half-open probes test recovery. The breaker does not fabricate a successful business result.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

Pool	Criticality	Failure isolation
Payments	Critical	Reserved bounded capacity
Receipts	Optional	Own queue and workers
Shared limits	DB, memory, network	Still require aggregate budgets

FIG. 17.0.3 **bulkhead isolation**

Separate pools limit some failure propagation, but shared resources remain correlated.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

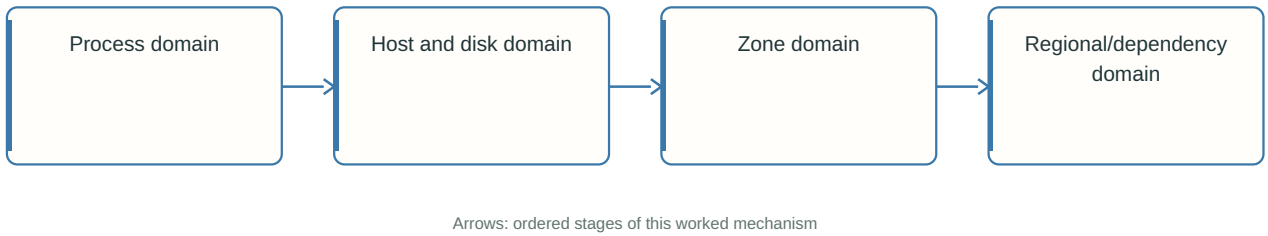


FIG. 17.0.4 **failure-domain map**

Each larger failure domain can remove several smaller replicas at once; independent placement is deliberate.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.



FIG. 17.0.5 **Recovery time and recoverable data are different**

Last recoverable data at 11:55, incident at 12:00, service restored at 12:20: five minutes of data exposure and twenty minutes of recovery in this example.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

Failure domains & redundancy

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Scalability, Load Balancing, and Data Distribution, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A downstream payment provider becomes unavailable during peak traffic. Demonstrate the failure cascade and progressively introduce resilience mechanisms.

This chapter examines failure domains & redundancy through concrete mechanisms, state snapshots, failure cases, and alternatives.

17.1.1 · Failure domains and fault isolation

A failure domain groups elements exposed to a common cause: a process, host, zone, dependency, or administrative action. Isolate critical functions so one overload or crash cannot consume every resource. Two workers sharing one disk and credential source may fail together even in different processes. Map correlations and test them. Isolation has a cost in duplicate capacity, network calls, and operating work, so apply it according to the harm and recovery objectives of each failure.

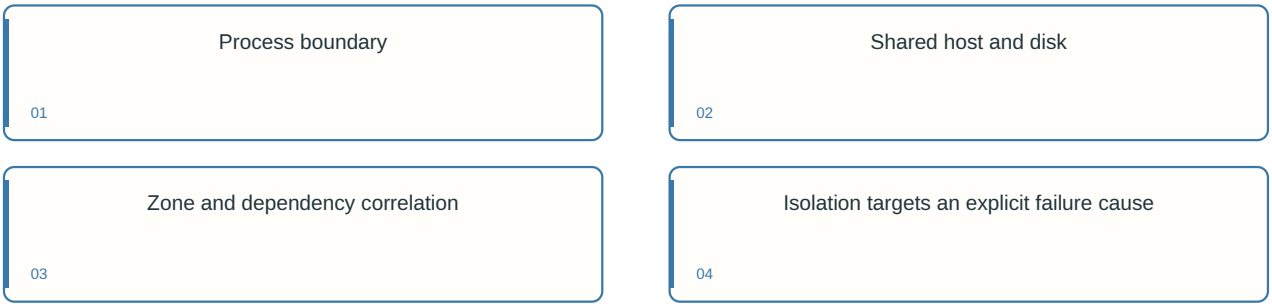


FIG. 17.1.1 **Failure domains and fault isolation · worked snapshots**

Process boundary · Shared host and disk · Zone and dependency correlation · Isolation targets an explicit failure cause. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.2 · Availability vs durability

Availability concerns useful serving behavior; durability concerns retained committed state under stated failures. A database can remain durable while unavailable during recovery. A stateless application can return 200 while silently dropping writes. Specify and observe both properties independently. Backups, replication, timeouts, and failover affect different parts of the contract. Do not promise a “reliable service” without explaining eligible requests, required outcomes, data-loss tolerance, and the time window or failure model being evaluated.

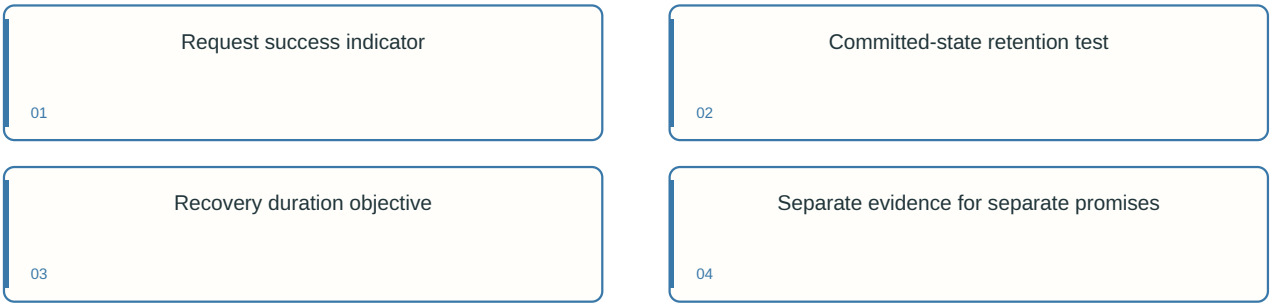


FIG. 17.1.2 **Availability vs durability · worked snapshots**

Request success indicator · Committed-state retention test · Recovery duration objective · Separate evidence for separate promises. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.3 · Redundancy

Redundancy adds spare components or copies, but their usefulness depends on independence, capacity, detection, and state correctness. If two instances each run at 70% and one fails, the other cannot absorb all traffic without overload. Plan reduced-capacity behavior or reserve appropriate headroom. Shared configuration mistakes can disable all replicas simultaneously. Redundancy should be paired with tested traffic movement and recovery, rather than counted as a number of boxes in an architecture picture.

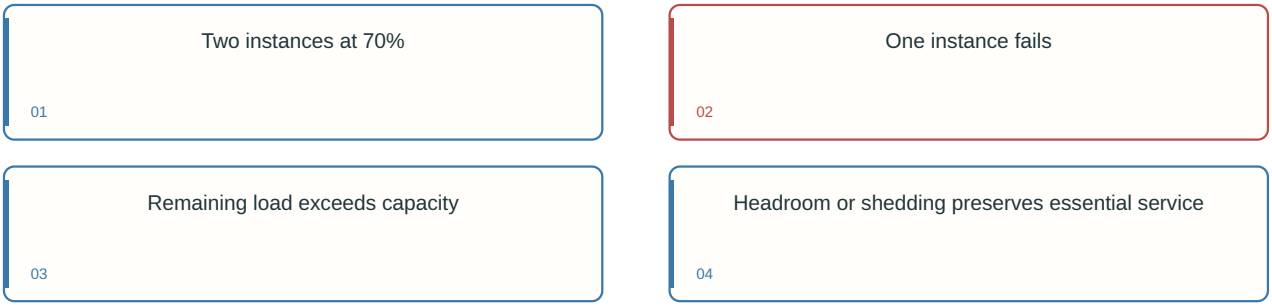


FIG. 17.1.3 **Redundancy · worked snapshots**

Two instances at 70% · One instance fails · Remaining load exceeds capacity · Headroom or shedding preserves essential service. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.4 · Active-active vs active-passive

Active-active serves from multiple locations or owners; active-passive keeps a standby for takeover. The choice concerns a particular function: applications may be active-active while a database has one write leader. Active-passive simplifies some authority decisions but can reveal cold-start and drift problems during failover. Active-active adds routing, coordination, or conflict complexity. Define failure detection, promotion, fencing, capacity, and failback for either model; a standby that has never restored usable state is unproven protection.

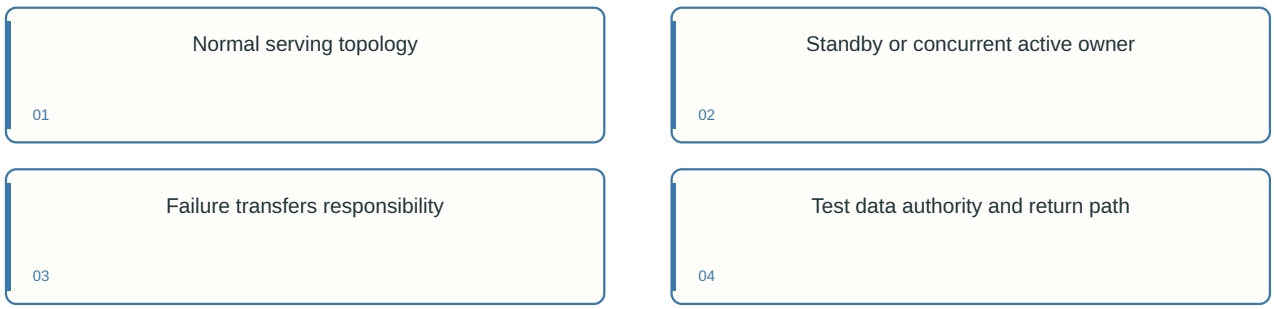


FIG. 17.1.4 **Active-active vs active-passive · worked snapshots**

Normal serving topology · Standby or concurrent active owner · Failure transfers responsibility · Test data authority and return path. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How can a disaster exercise pass infrastructure checks yet fail the business recovery requirement?

Compute 3^4 = 81 potential attempts for four independently retrying layers. Introduce one end-to-end retry budget, bounded waits, admission, and channel isolation. Do not use a false payment approval as a fallback. Verify recovery behavior and unresolved business state after the provider returns.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Active-active adds routing, coordination, or conflict complexity.

Carry forward: Normal serving topology · Standby or concurrent active owner · Failure transfers responsibility · Test data authority and return path.

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   `Access reviewed · 21 October 2026`

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   `Access reviewed · 21 October 2026`

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   `Access reviewed · 21 October 2026`

Retries, circuit breakers & isolation

Prerequisites: 17.1 · Failure domains & redundancy 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A downstream payment provider becomes unavailable during peak traffic. Demonstrate the failure cascade and progressively introduce resilience mechanisms.

This chapter examines retries, circuit breakers & isolation through concrete mechanisms, state snapshots, failure cases, and alternatives.

17.2.1 · Timeouts

Timeouts bound waiting and resource ownership. Use an overall request deadline plus sensible stage limits for connection, pool acquisition, and operations. A timeout should trigger cleanup or cancellation where possible, but does not prove a remote action failed. Choosing every timeout independently can let downstream work outlive the caller's useful window. Propagate remaining time and avoid beginning expensive work that cannot finish within it. Record timeout stage to distinguish queueing from execution failures.

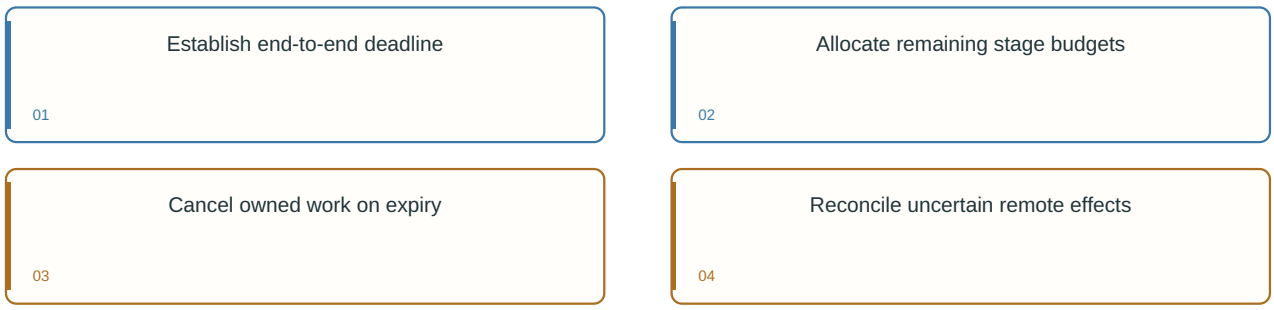


FIG. 17.2.1 **Timeouts · worked snapshots**

Establish end-to-end deadline · Allocate remaining stage budgets · Cancel owned work on expiry · Reconcile uncertain remote effects. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.2 · Retries, exponential backoff, jitter

Retries can recover transient faults but spend capacity. Use bounded attempts, exponential backoff, jitter, and error classification, with a stable identity for effects. A permanent validation failure should not be retried unchanged. Respect downstream retry guidance where applicable, but do not let a large Retry-After exceed the business operation's lifecycle unnoticed. Retry policies belong to the end-to-end design; one safe retry layer is often better than many independently aggressive layers.

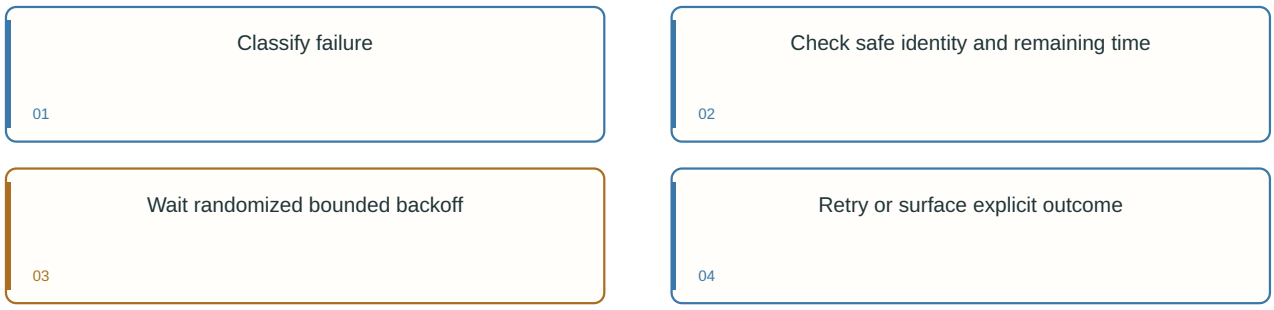


FIG. 17.2.2 **Retries, exponential backoff, jitter · worked snapshots**

Classify failure · Check safe identity and remaining time · Wait randomized bounded backoff · Retry or surface explicit outcome. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.3 · Retry storms

A retry storm amplifies an outage when many callers repeat work against a struggling dependency. Three attempts at four nested layers can produce 81 downstream attempts per original intent. Random delay reduces synchronization, but an attempt budget and admission control are also needed. Preserve goodput by rejecting excess work and suppressing redundant attempts. Monitor original business intents separately from attempts so apparent traffic growth during an outage is not mistaken for healthy user demand.

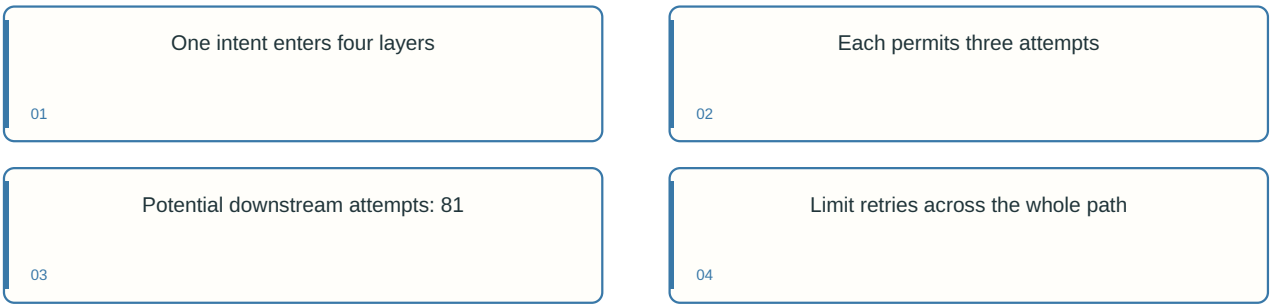


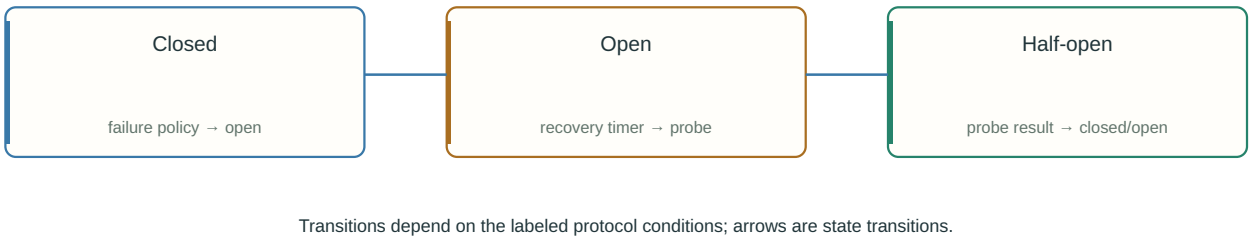
FIG. 17.2.3 **Retry storms · worked snapshots**

One intent enters four layers · Each permits three attempts · Potential downstream attempts: 81 · Limit retries across the whole path. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.4 · Circuit breakers

A circuit breaker tracks dependency failures and temporarily rejects or redirects calls under a policy. Common states are closed, open, and half-open, with bounded probes to test recovery. It prevents futile waiting but does not repair the dependency or decide domain correctness. A fallback that returns a fabricated payment approval is unsafe. Make failure classification, probe concurrency, and recovery thresholds explicit, and avoid every instance probing simultaneously after the same interval.



Transitions depend on the labeled protocol conditions; arrows are state transitions.

FIG. 17.2.4 **A circuit breaker bounds futile calls**

Closed permits calls, an observed failure policy opens the breaker, and bounded half-open probes test recovery. The breaker does not fabricate a successful business result.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.5 · Bulkheads

Bulkheads partition resources so one class of work cannot consume everything. Separate pools for payment authorization and optional receipt delivery can keep checkout viable when email stalls. Pool sizing must include shared limits such as database connections and host memory; separate thread pools alone do not isolate every dependency. Over-isolation can waste spare capacity. Choose boundaries by failure harm and observe saturation and rejection in each partition, not only fleet-level averages.

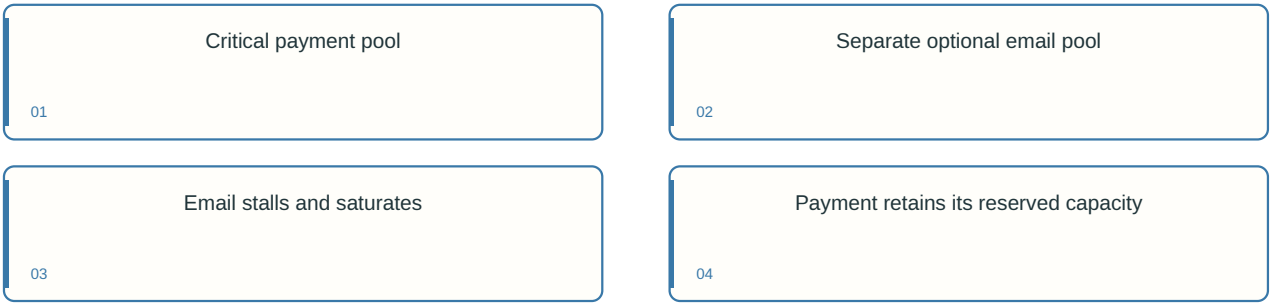


FIG. 17.2.5 **Bulkheads · worked snapshots**

Critical payment pool · Separate optional email pool · Email stalls and saturates · Payment retains its reserved capacity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.6 · Rate limiting

Rate limiting protects a boundary from excessive traffic under a defined identity and burst policy. A token bucket permits bounded bursts while enforcing a refill rate; a queue-based shaper changes timing rather than simply rejecting. For multi-tenant services, combine per-tenant fairness with global capacity protection. A limiter depending on an unavailable remote store needs a deliberate fail-open, fail-closed, or local-budget policy. Each choice affects abuse risk and availability, so test the failure mode explicitly.

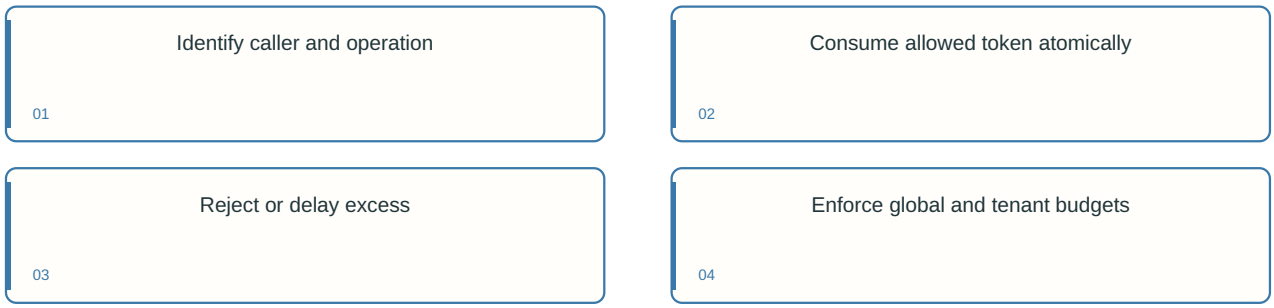


FIG. 17.2.6 **Rate limiting · worked snapshots**

Identify caller and operation · Consume allowed token atomically · Reject or delay excess · Enforce global and tenant budgets. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How can a disaster exercise pass infrastructure checks yet fail the business recovery requirement?

Compute 3^4 = 81 potential attempts for four independently retrying layers. Introduce one end-to-end retry budget, bounded waits, admission, and channel isolation. Do not use a false payment approval as a fallback. Verify recovery behavior and unresolved business state after the provider returns.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A limiter depending on an unavailable remote store needs a deliberate fail-open, fail-closed, or local-budget policy.

Carry forward: Identify caller and operation · Consume allowed token atomically · Reject or delay excess · Enforce global and tenant budgets.

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Shed load and degrade deliberately

Prerequisites: 17.2 · Retries, circuit breakers & isolation 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A downstream payment provider becomes unavailable during peak traffic. Demonstrate the failure cascade and progressively introduce resilience mechanisms.

This chapter examines shed load and degrade deliberately through concrete mechanisms, state snapshots, failure cases, and alternatives.

17.3.1 · Load shedding

Load shedding rejects lower-value or excessive work before resources are exhausted. A service may stop recommendations while preserving order status and payment reconciliation. Decide priorities from business meaning, and avoid spending expensive computation only to reject at the end. Shedding should return a clear outcome with bounded retry behavior. It improves useful completion under overload, but cannot make an unsustainable workload sustainable indefinitely; scaling, workload reduction, or a changed objective may still be needed.

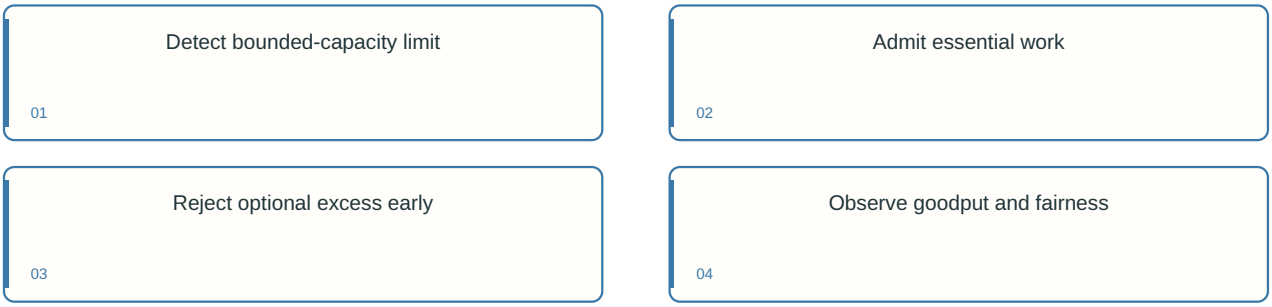


FIG. 17.3.1 **Load shedding · worked snapshots**

Detect bounded-capacity limit · Admit essential work · Reject optional excess early · Observe goodput and fairness. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.2 · Graceful degradation

Graceful degradation preserves a useful subset of behavior under failure. Showing product descriptions without recommendations can be acceptable; displaying stale available inventory as permission to sell may not be. Define which data may be stale, which actions must pause, and how users see the limitation. Test degraded paths because rarely used fallbacks can fail when needed. Degradation is a product and correctness decision, not simply catching every exception and returning a default value.

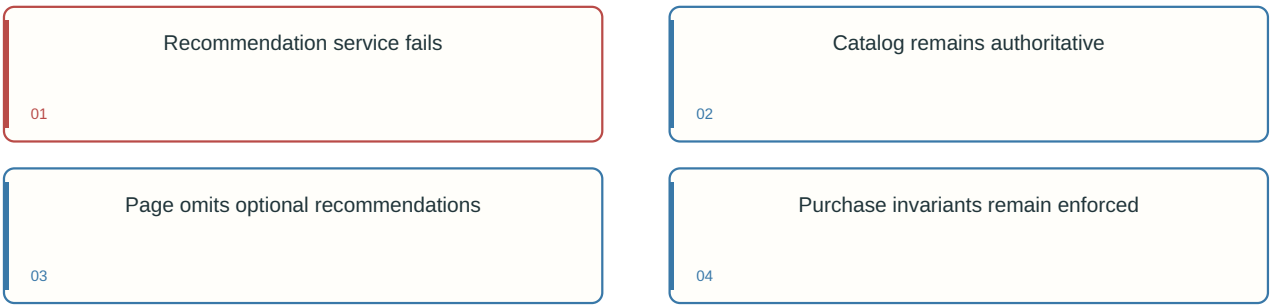


FIG. 17.3.2 **Graceful degradation · worked snapshots**

Recommendation service fails · Catalog remains authoritative · Page omits optional recommendations · Purchase invariants remain enforced. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.3 · Fallbacks

A fallback substitutes an alternative behavior with an explicit contract. A cached help page can replace a failed content request; a default account balance cannot replace a failed ledger lookup. Fallbacks can create load elsewhere and can mask a prolonged incident. Bound their freshness and duration, signal degraded operation, and check security scope. A second provider is not transparent if its semantics, operation identities, or reconciliation process differ from the first.

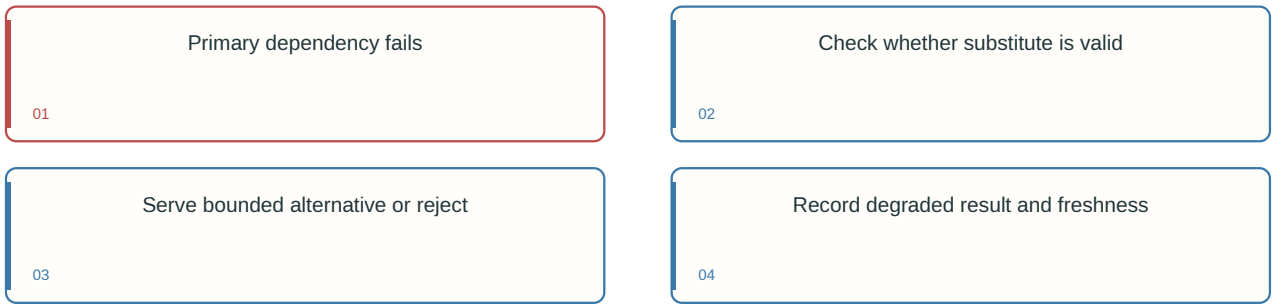


FIG. 17.3.3 **Fallbacks · worked snapshots**

Primary dependency fails · Check whether substitute is valid · Serve bounded alternative or reject · Record degraded result and freshness. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.4 · Health checking

Health checks should answer a useful local question and be bounded. Restarting a process because a shared provider is down can create a restart storm without repairing the provider. Readiness may stop new work while allowing in-flight operations to finish. Liveness should avoid external dependencies that can fail independently. Include startup and warmup behavior, and distinguish “process responds to probe” from “critical business operations meet their objectives.” Probe success is evidence, not a complete reliability guarantee.

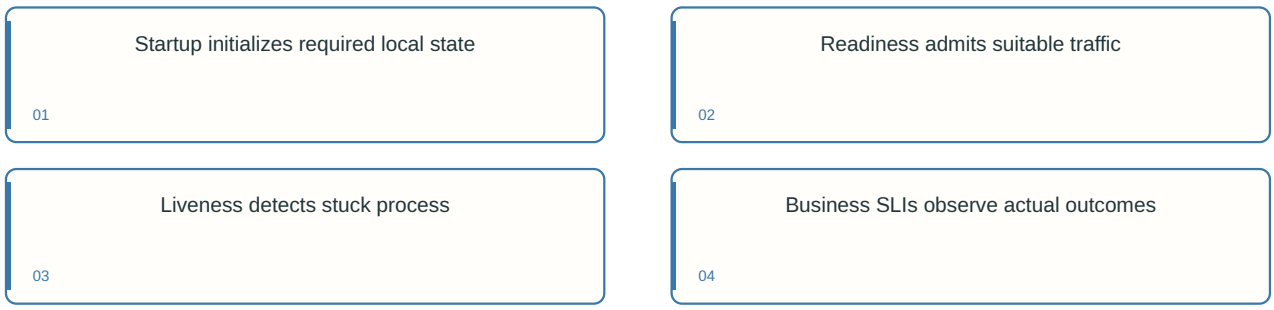


FIG. 17.3.4 **Health checking · worked snapshots**

Startup initializes required local state · Readiness admits suitable traffic · Liveness detects stuck process · Business SLIs observe actual outcomes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How can a disaster exercise pass infrastructure checks yet fail the business recovery requirement?

Compute 3^4 = 81 potential attempts for four independently retrying layers. Introduce one end-to-end retry budget, bounded waits, admission, and channel isolation. Do not use a false payment approval as a fallback. Verify recovery behavior and unresolved business state after the provider returns.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Liveness should avoid external dependencies that can fail independently.

Carry forward: Startup initializes required local state · Readiness admits suitable traffic · Liveness detects stuck process · Business SLIs observe actual outcomes.

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Recover, test & learn from failure

Prerequisites: 17.3 · Shed load and degrade deliberately 8 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A downstream payment provider becomes unavailable during peak traffic. Demonstrate the failure cascade and progressively introduce resilience mechanisms.

This chapter examines recover, test & learn from failure through concrete mechanisms, state snapshots, failure cases, and alternatives.

17.4.1 · RTO and RPO

RTO is the recovery time objective; RPO is the acceptable data-loss exposure expressed in time. If an incident begins at 12:00, service restoration at 12:20 gives a twenty-minute interruption; restoring data only through 11:55 creates five minutes of exposure relative to the incident point. These are separate objectives and measurements. Set them per business function, and verify detection, decision, restoration, validation, and traffic return in drills rather than timing only the database copy operation.



FIG. 17.4.1 **Recovery time and recoverable data are different**

Last recoverable data at 11:55, incident at 12:00, service restored at 12:20: five minutes of data exposure and twenty minutes of recovery in this example.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.2 · Backups and restore testing

A backup must be restorable, authorized, decryptable, consistent enough for the application, and retained separately from the failure it protects against. Test actual restoration and verify business invariants, not just checksum or job status. Replication can copy corruption or accidental deletion, so historical recovery serves a different purpose. Document keys, credentials, schema versions, and log dependencies needed during a disaster. A backup whose only decryption key was on the lost host is unusable protection.

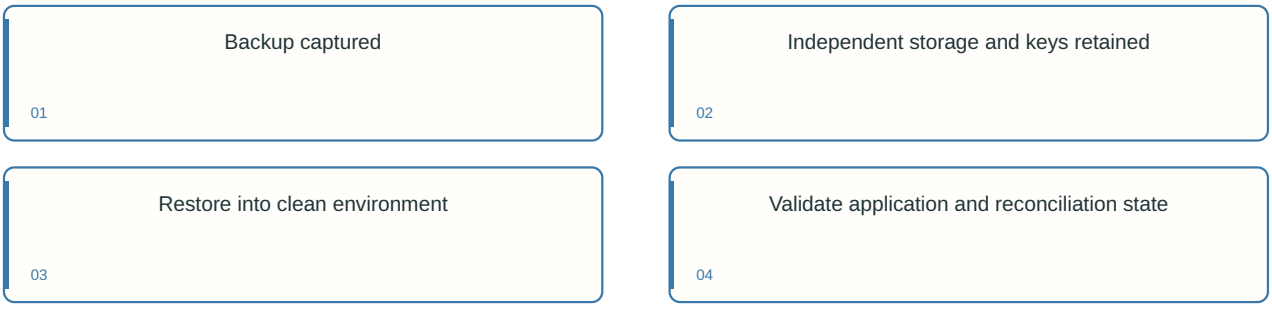


FIG. 17.4.2 **Backups and restore testing · worked snapshots**

Backup captured · Independent storage and keys retained · Restore into clean environment · Validate application and reconciliation state. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.3 · Failover and failback

Failover transfers service or authority to a replacement; failback returns responsibility after repair. Both require state freshness, fencing, routing, and capacity decisions. Promote only under the chosen durability contract and prevent the old owner from accepting conflicting writes. Rejoining an old primary may require rebuilding its state from the new history. DNS, connection pools, caches, and client retries can prolong the transition even after a database promotion succeeds. Test the complete user-visible sequence.

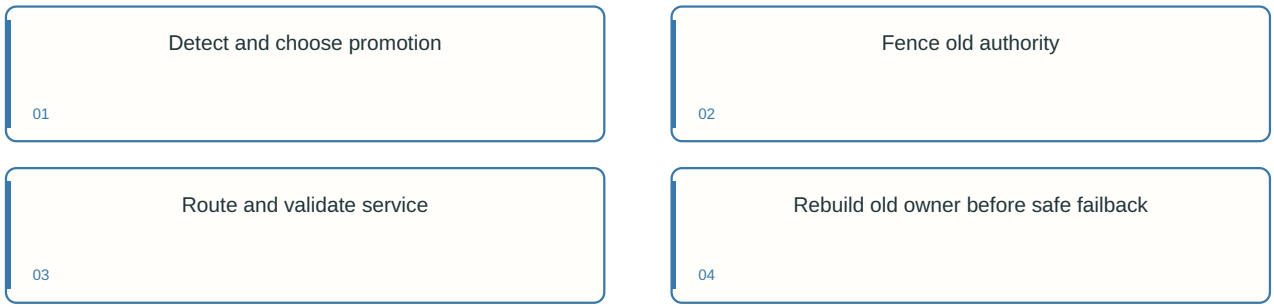


FIG. 17.4.3 **Failover and failback · worked snapshots**

Detect and choose promotion · Fence old authority · Route and validate service · Rebuild old owner before safe failback. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.4 · Multi-zone and multi-region deployments

Multi-zone and multi-region deployments protect against different correlated failures. Cross-zone copies may survive a host or zone incident but not a regional control-plane or operator error. Cross-region replication adds latency, network charges, data placement concerns, and recovery complexity. Avoid claiming region independence when both rely on one global credential or configuration component. Select topology from explicit failure goals, and reserve enough surviving capacity for the degraded traffic and catch-up work during recovery.

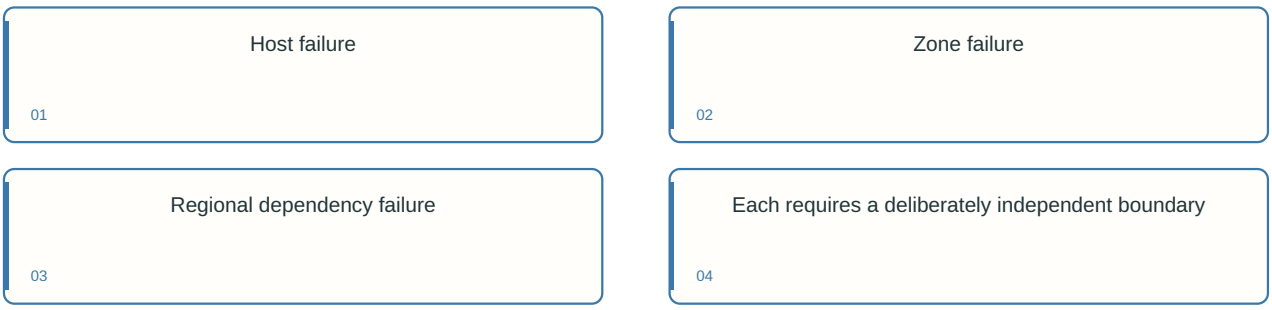


FIG. 17.4.4 **Multi-zone and multi-region deployments · worked snapshots**

Host failure · Zone failure · Regional dependency failure · Each requires a deliberately independent boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.5 · Chaos engineering

Chaos engineering tests hypotheses about behavior under controlled faults. Start with an explicit claim, bounded blast radius, observability, stop conditions, and recovery ownership. For example, pause one worker and verify that its expired lease cannot overwrite a newer result. Randomly killing processes without a hypothesis can create incidents without learning much. Prefer safe environments and progressively justified scope; interpret a passing drill as evidence for the tested scenario, not proof against all future failures.

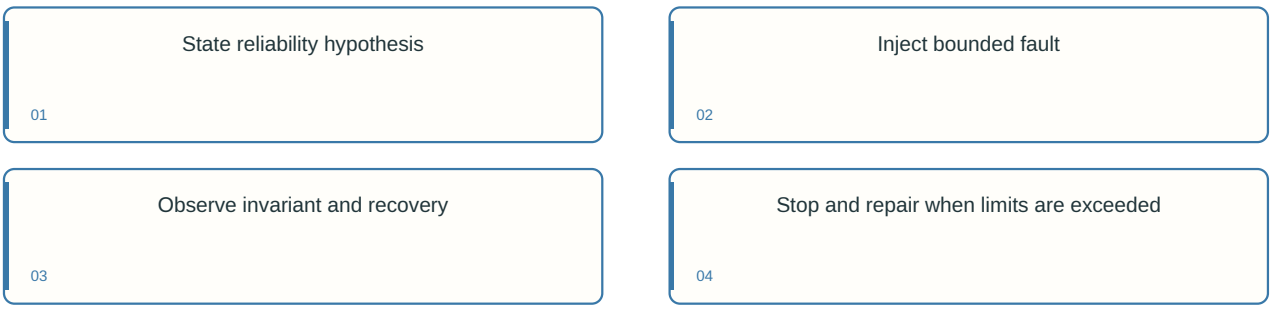


FIG. 17.4.5 **Chaos engineering · worked snapshots**

State reliability hypothesis · Inject bounded fault · Observe invariant and recovery · Stop and repair when limits are exceeded. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.6 · SLO-based reliability decisions

Reliability effort should follow user-visible objectives and error-budget evidence. If a small fraction of expensive optional calls causes most deadline failures, removing them from the critical path may help more than another region. Avoid maximizing redundancy without considering cost and complexity. An objective that ignores critical low-volume operations can hide serious harm behind a large denominator. Segment indicators where necessary and use the budget as a decision aid, not permission to dismiss particular users' failures.

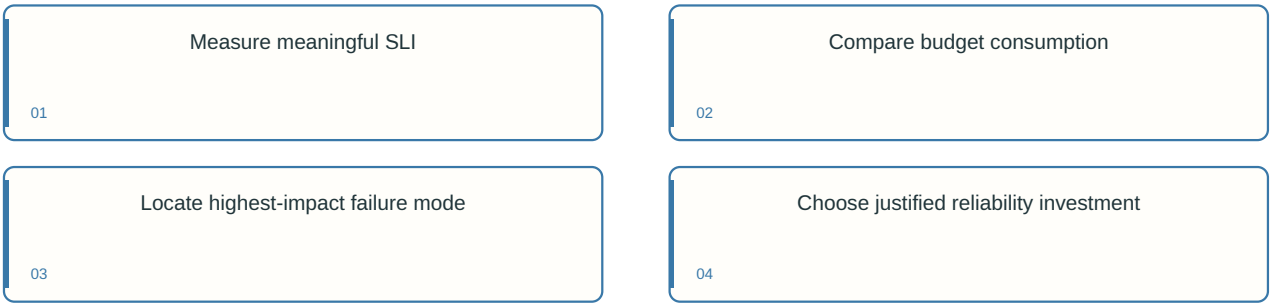


FIG. 17.4.6 **SLO-based reliability decisions · worked snapshots**

Measure meaningful SLI · Compare budget consumption · Locate highest-impact failure mode · Choose justified reliability investment. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.7 · Cascading failures

Cascading failure occurs when one fault creates additional pressure and failures elsewhere: a slow provider holds connections, queues grow, callers retry, and formerly healthy dependencies become overloaded. Break the feedback with deadlines, bounded admission, retry budgets, bulkheads, and meaningful degradation. Capacity must remain available for recovery work. A circuit breaker alone can still trigger a stampede against a fallback. Trace the complete resource path and test correlated failures rather than assuming each component fails in isolation.

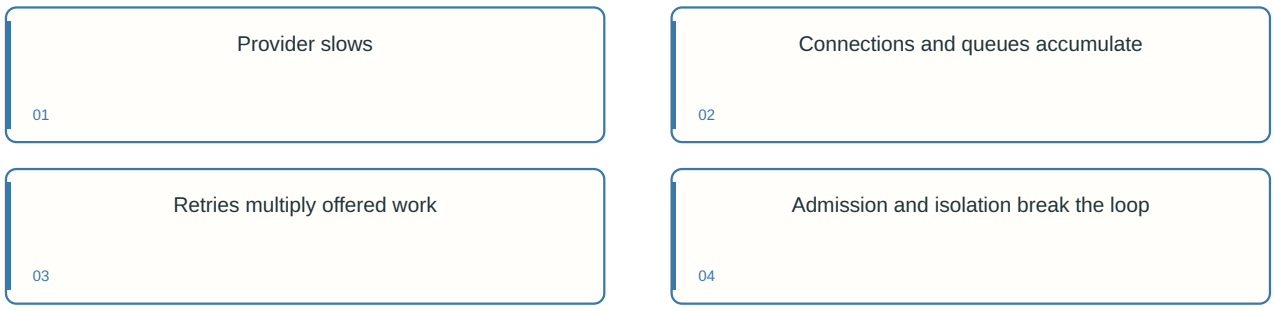


FIG. 17.4.7 **Cascading failures · worked snapshots**

Provider slows · Connections and queues accumulate · Retries multiply offered work · Admission and isolation break the loop. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.8 · Incident analysis and prevention

Incident analysis reconstructs the timeline, contributing conditions, detection gaps, and recovery decisions without reducing the explanation to individual blame. “Human error” does not explain why one action could disable every replica without a safe recovery path. Link corrective work to observed mechanisms and assign verification evidence. Prefer changes that reduce recurrence or harm, such as scoped credentials, tested restores, and bounded retries, rather than adding alerts that do not create a useful response action.

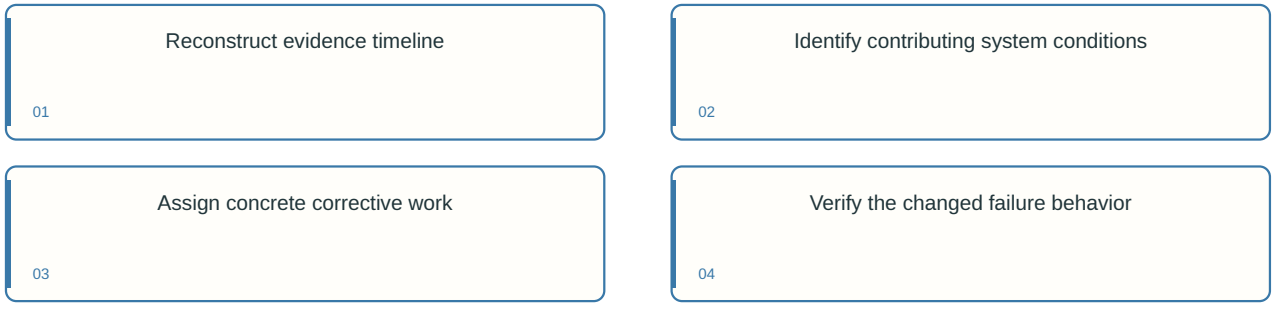


FIG. 17.4.8 **Incident analysis and prevention · worked snapshots**

Reconstruct evidence timeline · Identify contributing system conditions · Assign concrete corrective work · Verify the changed failure behavior. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · RETRY AMPLIFICATION

```
attempts_per_layer = 3
independent_retry_layers = 4
assert attempts_per_layer ** independent_retry_layers == 81
```

This is a worst-case attempt-count model for nested policies, not a measured outage. Bound retries end to end, propagate deadlines, protect goodput, and preserve stable identity for effects. Jitter alone does not reduce the total permitted attempts.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · How can a disaster exercise pass infrastructure checks yet fail the business recovery requirement?

Compute 3^4 = 81 potential attempts for four independently retrying layers. Introduce one end-to-end retry budget, bounded waits, admission, and channel isolation. Do not use a false payment approval as a fallback. Verify recovery behavior and unresolved business state after the provider returns.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Link corrective work to observed mechanisms and assign verification evidence.

Carry forward: Reconstruct evidence timeline · Identify contributing system conditions · Assign concrete corrective work · Verify the changed failure behavior.

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

PART 5 / UNIT 18

Security Architecture and Privacy

The running example

Design a multi-tenant SaaS application where an administrator must never gain access to another organization's data.

Unit 18 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

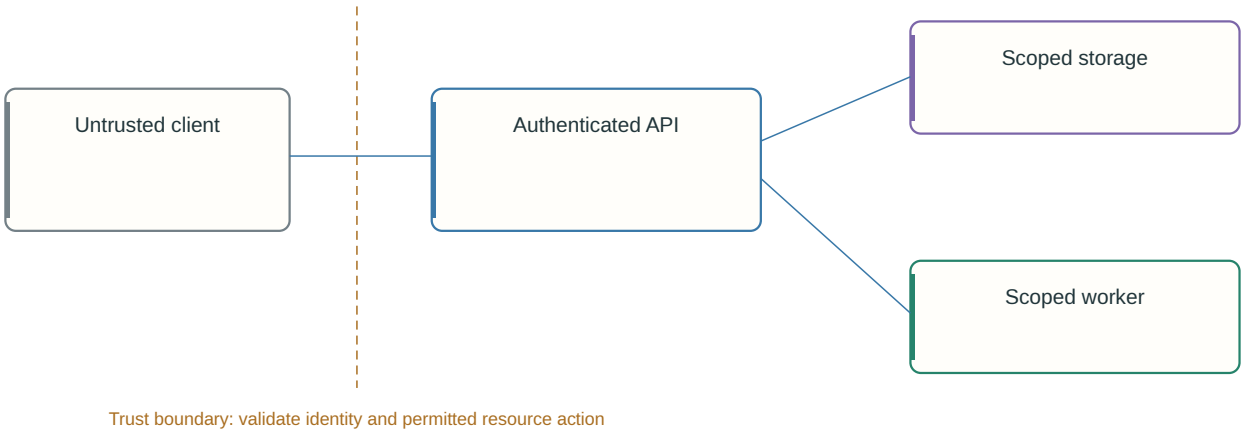


FIG. 18.0.1 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

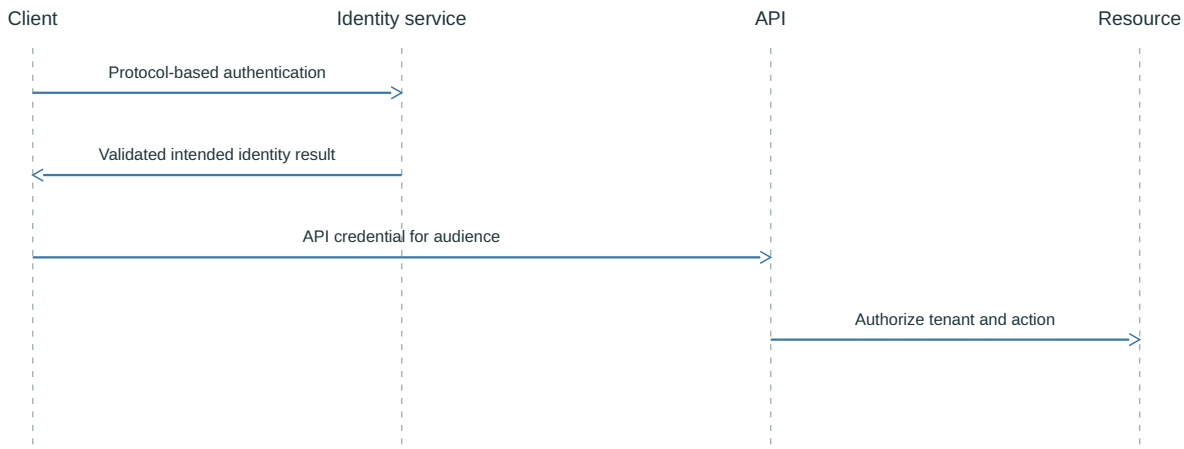


FIG. 18.0.2 **authentication sequence**

Authentication, token validation, and resource authorization are distinct protocol and enforcement decisions.

Why this view: Follow explicit message ordering and locate the acknowledgment or effect boundary. Author-created illustration. Numbers are illustrative unless stated otherwise.

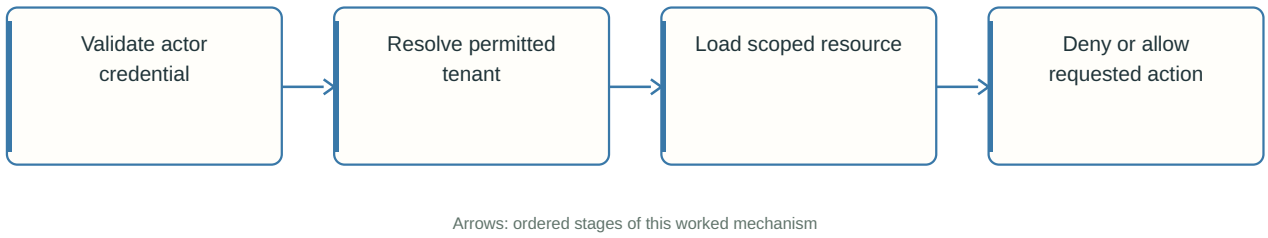


FIG. 18.0.3 **authorization decision flow**

A valid token never grants arbitrary access to a guessed resource identifier.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

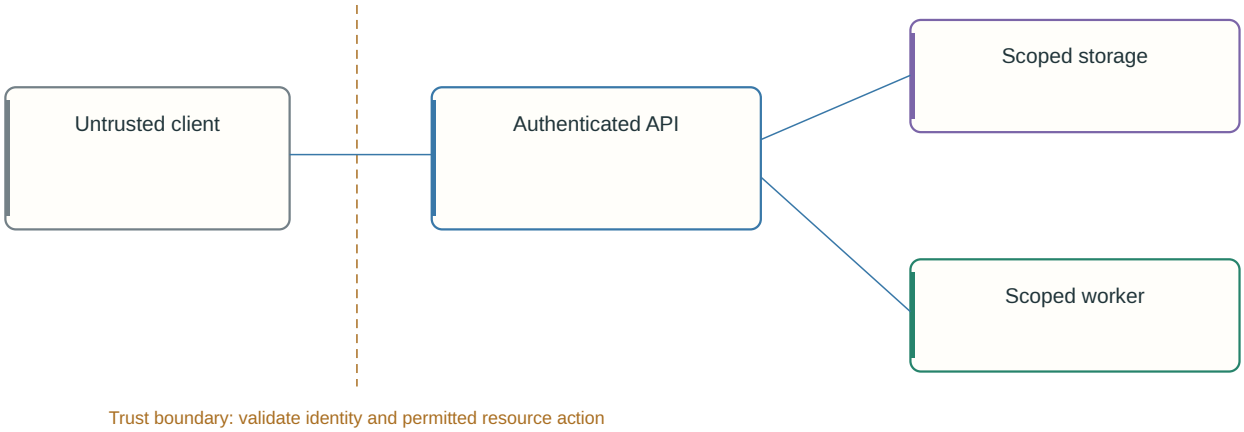


FIG. 18.0.4 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

Threats, identity & authorization

Prerequisites: Networking Foundations for System Design, Application Architecture and Domain-Driven Design, API Design and Communication Contracts, Data Models and Storage Technologies

7 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a multi-tenant SaaS application where an administrator must never gain access to another organization's data.

This chapter examines threats, identity & authorization through concrete mechanisms, state snapshots, failure cases, and alternatives.

18.1.1 · Threat modeling

Threat modeling starts from assets, actors, trust boundaries, and abuse goals. For a multi-tenant document service, identify confidential files, credentials, processing workers, administrators, and storage capabilities. Ask how an attacker could read another tenant's file, forge a callback, exhaust extraction capacity, or alter an audit record. Prioritize by impact and plausible paths, then attach mitigations and tests. A diagram without a data inventory or actor privileges is an incomplete threat model.

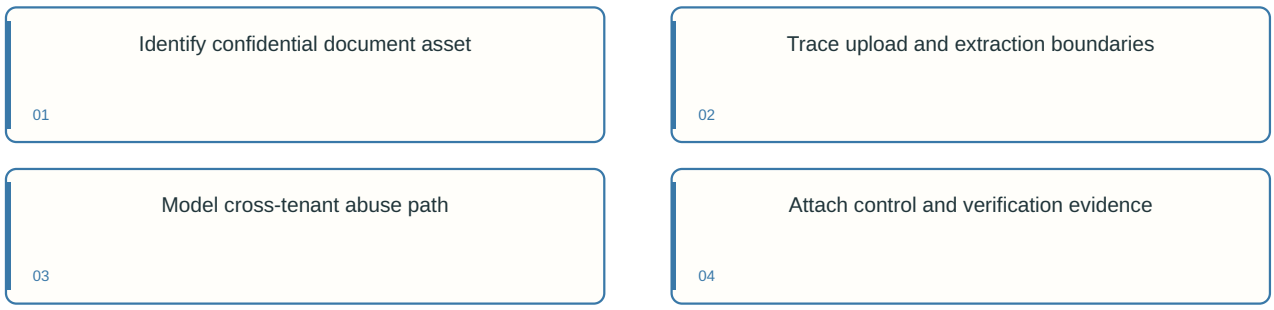


FIG. 18.1.1 **Threat modeling · worked snapshots**

Identify confidential document asset · Trace upload and extraction boundaries · Model cross-tenant abuse path · Attach control and verification evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.2 · Trust boundaries

A trust boundary changes the confidence or authority granted to input. Browser-to-server, server-to-storage, worker-to-model, and operator-to-production are distinct boundaries. Validate identity and relevant data each time a boundary is crossed; do not treat internal network origin as proof of legitimacy. Headers forwarded by a trusted proxy require a specific trust configuration. Uploaded files remain untrusted after storage and must be processed with appropriate resource and execution isolation.

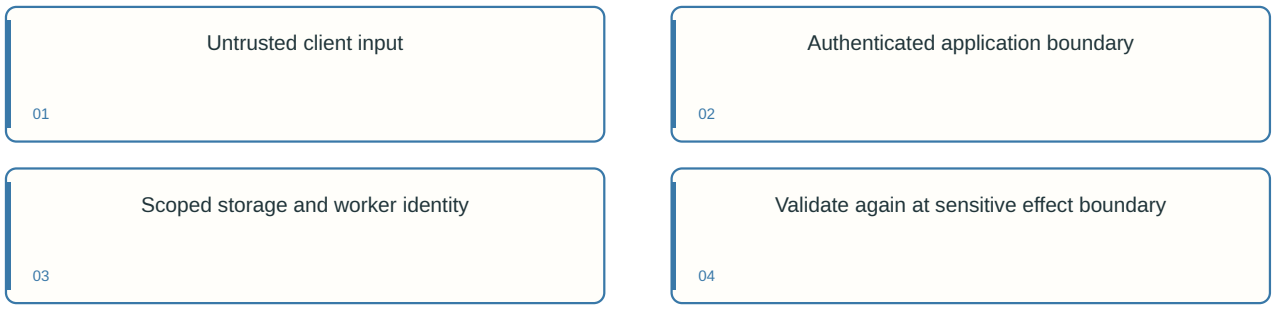


FIG. 18.1.2 **Trust boundaries · worked snapshots**

Untrusted client input · Authenticated application boundary · Scoped storage and worker identity · Validate again at sensitive effect boundary. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.3 · Authentication vs authorization

Authentication establishes identity; authorization decides permitted action on a particular resource in context. An organization administrator may manage their own tenant without any right to another tenant's files. Derive tenancy from authenticated membership and enforce it on every read, update, export, and background operation. Guess-resistant IDs reduce enumeration but do not supply authorization. Deny by default, keep policy decisions explicit, and test both allowed access and cross-tenant denial paths.

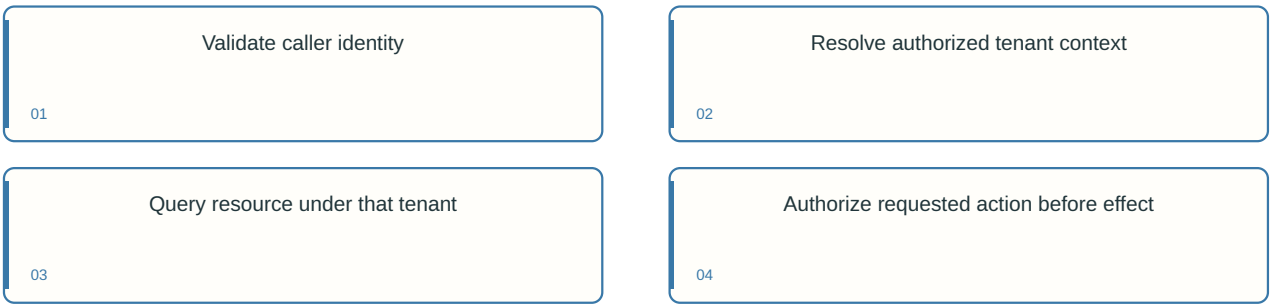


FIG. 18.1.3 **Authentication vs authorization · worked snapshots**

Validate caller identity · Resolve authorized tenant context · Query resource under that tenant · Authorize requested action before effect. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.4 · Sessions, cookies, tokens

Sessions commonly store server-side state referenced by a credential; tokens can carry claims verified under a protocol. Cookies are a browser transport/storage mechanism with security attributes, not an authentication model by themselves. Protect credentials with appropriate secure transport, cookie settings, CSRF defenses where relevant, and token validation. Logout and revocation behavior differ between stateful sessions and self-contained tokens. A signed token is not automatically current, audience-correct, or authorized for a resource.

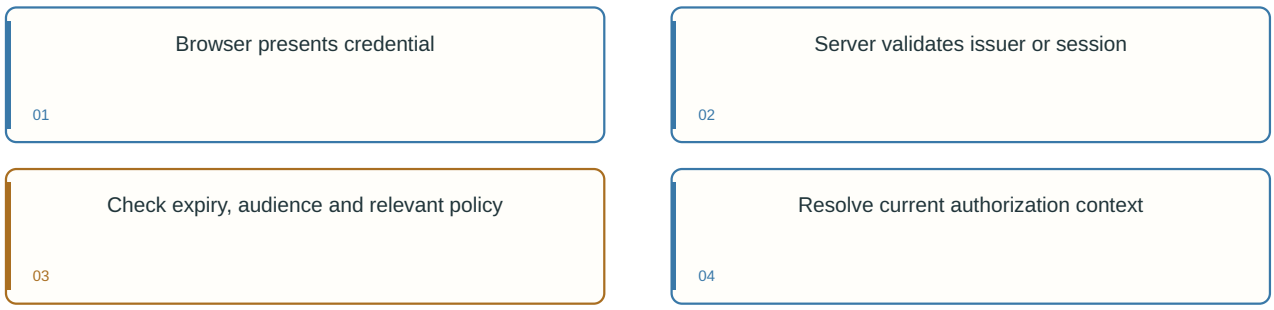


FIG. 18.1.4 **Sessions, cookies, tokens · worked snapshots**

Browser presents credential · Server validates issuer or session · Check expiry, audience and relevant policy · Resolve current authorization context. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.5 · OAuth 2.0 and OpenID Connect

OAuth 2.0 concerns delegated authorization; OpenID Connect adds an identity layer. Do not use an access token as an identity assertion without the intended protocol and validation. For appropriate browser/native flows, use current best-practice guidance such as authorization code with PKCE, exact redirect handling, and library-based validation. Issuer, audience, signature, nonce or state requirements, and token type depend on the flow. Never invent a custom shortcut that confuses login identity with permission to a target API.

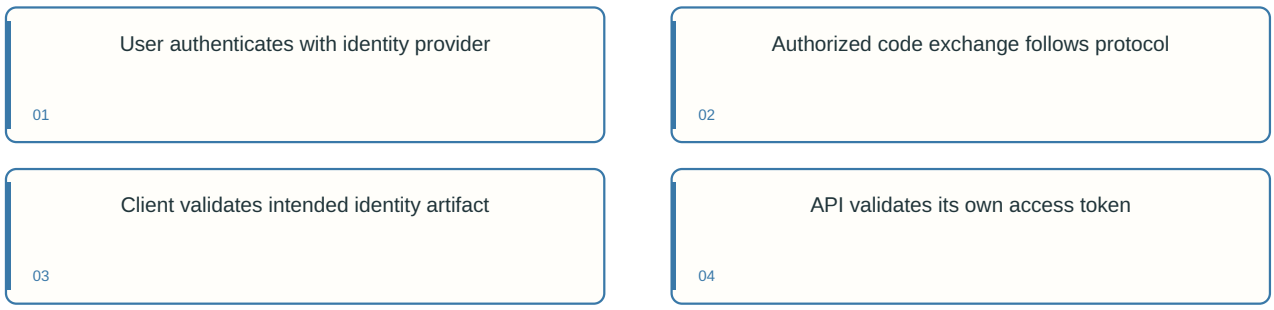


FIG. 18.1.5 **OAuth 2.0 and OpenID Connect · worked snapshots**

User authenticates with identity provider · Authorized code exchange follows protocol · Client validates intended identity artifact · API validates its own access token. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.6 · Role-based and attribute-based access control

RBAC grants permissions through roles; ABAC evaluates attributes such as tenant, resource classification, ownership, and context. Roles simplify administration but can become too broad; attribute policies can express finer rules while becoming harder to explain and test. A tenant-admin role must be bound to the tenant, not treated as global administration. Keep policy inputs trustworthy and test revocation and background jobs. A policy engine centralizes evaluation only if enforcement points actually call it correctly.

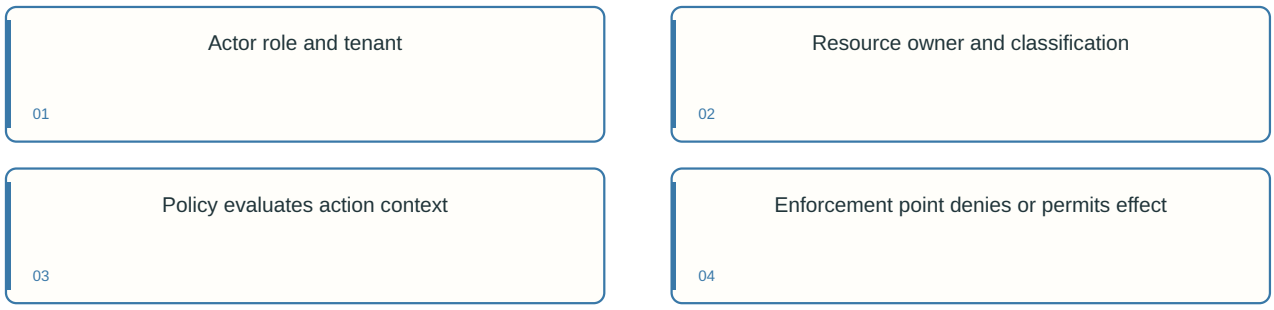


FIG. 18.1.6 **Role-based and attribute-based access control · worked snapshots**

Actor role and tenant · Resource owner and classification · Policy evaluates action context · Enforcement point denies or permits effect. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.7 · Least privilege

Least privilege limits identities to the resources and operations required for their work. An extraction worker may read one input object and write its output without listing every tenant's bucket. Separate deployment, runtime, and human administrative identities where useful. Short-lived scoped credentials reduce exposure, but also require reliable refresh and failure handling. Review effective permissions, inherited roles, and emergency access rather than only the intended configuration in a design document.

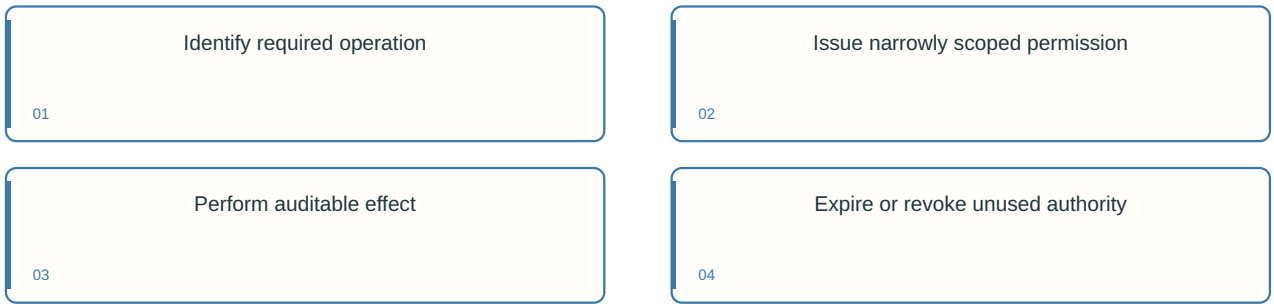


FIG. 18.1.7 **Least privilege · worked snapshots**

Identify required operation · Issue narrowly scoped permission · Perform auditable effect · Expire or revoke unused authority. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would tenant revocation affect an already issued object capability and a queued export?

permit_demo() denies tenant A reading tenant B even when the action name is allowed. This synthetic predicate is not credential validation. Extend tests to cache keys, queued jobs, exports, and object capabilities; a correct predicate unused by a worker does not enforce anything.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Short-lived scoped credentials reduce exposure, but also require reliable refresh and failure handling.

Carry forward: Identify required operation · Issue narrowly scoped permission · Perform auditable effect · Expire or revoke unused authority.

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   `Access reviewed · 21 October 2026`

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   `Access reviewed · 21 October 2026`

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   `Access reviewed · 21 October 2026`

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   `Access reviewed · 21 October 2026`

Secrets, encryption & trust boundaries

Prerequisites: 18.1 · Threats, identity & authorization 7 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a multi-tenant SaaS application where an administrator must never gain access to another organization's data.

This chapter examines secrets, encryption & trust boundaries through concrete mechanisms, state snapshots, failure cases, and alternatives.

18.2.1 · Defense in depth

Defense in depth places complementary controls around a risk: tenant-scoped queries, database permissions, signed object access, worker isolation, and audit evidence can all reduce cross-tenant harm. Repeating the same ineffective check at three layers is not three independent defenses. Avoid controls whose failures share one mistaken assumption, such as trusting client tenant_id everywhere. Understand fallback behavior so a failed security dependency does not silently bypass the protections it was meant to enforce.

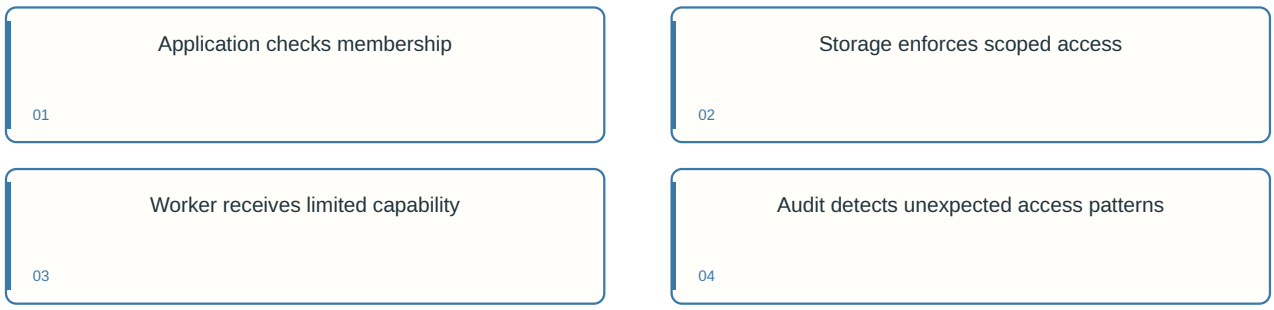


FIG. 18.2.1 **Defense in depth · worked snapshots**

Application checks membership · Storage enforces scoped access · Worker receives limited capability · Audit detects unexpected access patterns. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.2 · Secrets management

Secrets need an owner, distribution mechanism, access policy, rotation, and revocation procedure. Keep credentials out of source, logs, command histories, and client bundles. Prefer managed identity or short-lived credentials when supported, while planning bootstrap and outage behavior. Rotation may require overlapping valid versions and verified retirement. Encrypting a secret file does little if its decryption key is accessible to every process that can read the file; key and permission boundaries matter.

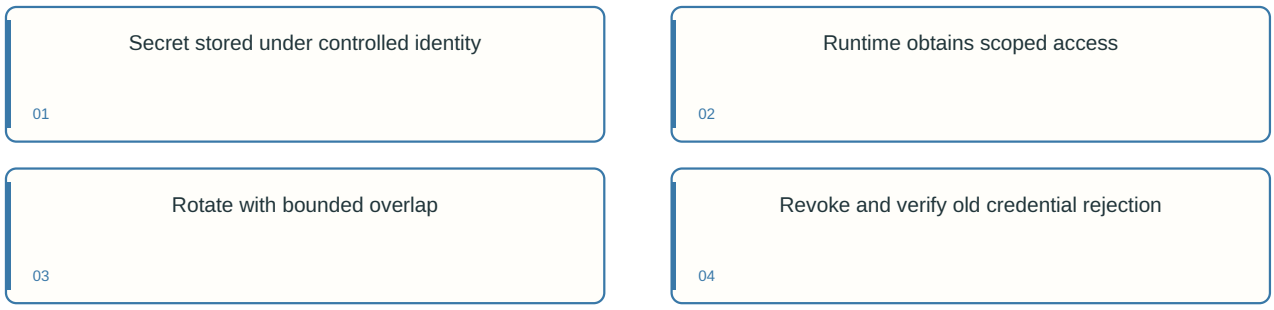


FIG. 18.2.2 **Secrets management · worked snapshots**

Secret stored under controlled identity · Runtime obtains scoped access · Rotate with bounded overlap · Revoke and verify old credential rejection. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.3 · Encryption in transit and at rest

Encryption in transit protects communication under a protocol; encryption at rest protects stored bytes under key and access assumptions. Neither prevents an authorized-but-compromised application from reading data it is allowed to decrypt. Identify termination points, key custody, backups, logs, and derived copies. A storage provider's at-rest encryption does not automatically encrypt an exported CSV sent elsewhere. Match controls to the threat model and use reviewed protocols and libraries instead of custom cryptography.

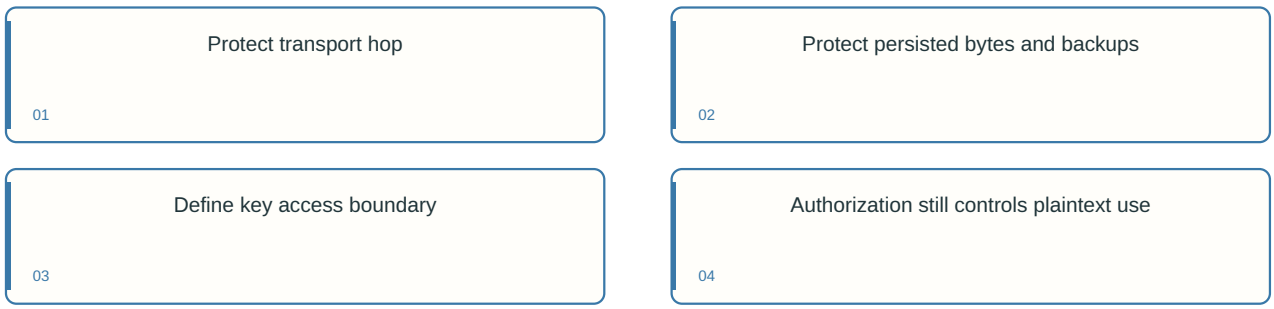


FIG. 18.2.3 **Encryption in transit and at rest · worked snapshots**

Protect transport hop · Protect persisted bytes and backups · Define key access boundary · Authorization still controls plaintext use. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.4 · Password hashing

Passwords require purpose-built password hashing with salt and an appropriate work factor, not fast general-purpose hashing or reversible encryption. Use a maintained implementation and current OWASP guidance, choosing parameters that balance offline-guessing resistance with service capacity. Store the encoded algorithm and parameters for migration, and rehash when policy changes. Rate-limit verification and protect recovery flows. Do not invent fixed universal parameters in a textbook; hardware, libraries, and current guidance must be checked during deployment.

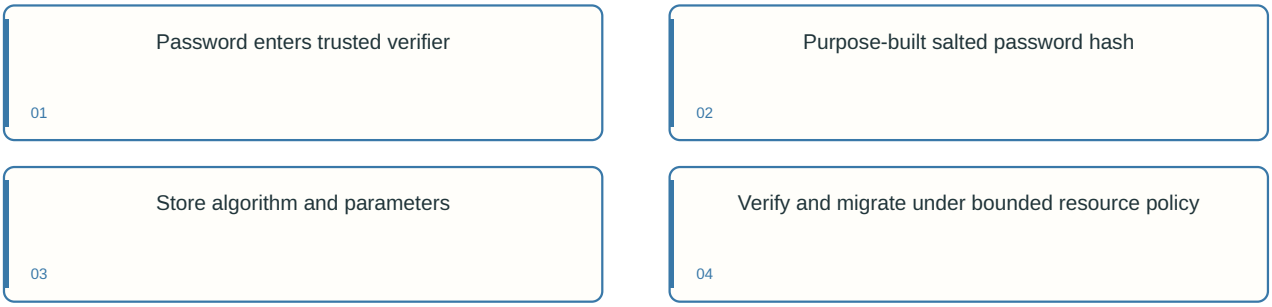


FIG. 18.2.4 **Password hashing · worked snapshots**

Password enters trusted verifier · Purpose-built salted password hash · Store algorithm and parameters · Verify and migrate under bounded resource policy. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.5 · Key management and rotation

Key management includes generation, storage, access, rotation, versioning, backup, and destruction. Envelope encryption can separate data-encryption keys from a key-encryption authority, but the design must retain the right key versions to read permitted historical data. Deleting a key before verifying retention and recovery needs can make backups unusable. Conversely, retaining every key indefinitely may defeat deletion goals. Record which data and services depend on each key version and test rotation and restore paths.

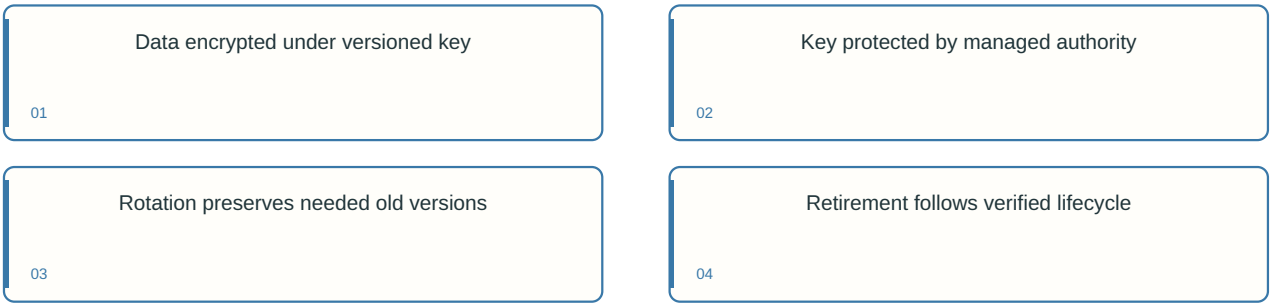


FIG. 18.2.5 **Key management and rotation · worked snapshots**

Data encrypted under versioned key · Key protected by managed authority · Rotation preserves needed old versions · Retirement follows verified lifecycle. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.6 · Network segmentation

Network segmentation limits reachable paths and reduces some lateral movement. A worker network may reach storage and a model service but not the administrative database interface. Egress policy matters for uploaded-file processing and remote URL fetching because server-side requests can target internal services. Network controls complement application identity and authorization; an allowed connection should not grant every operation. Test actual reachability and DNS behavior, not only firewall rule descriptions.

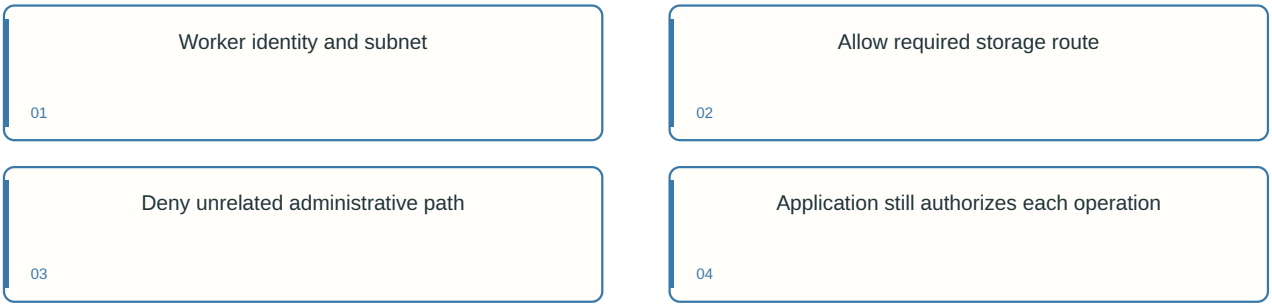


FIG. 18.2.6 **Network segmentation · worked snapshots**

Worker identity and subnet · Allow required storage route · Deny unrelated administrative path · Application still authorizes each operation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.7 · Zero-trust principles

Zero-trust principles reject implicit authorization based solely on network location. Evaluate identity, resource, action, and relevant context at explicit boundaries, with limited privileges and observable policy decisions. The term is not a single product or a requirement to synchronously contact a central server for every packet. Cached credentials and policy need expiry and revocation decisions. Make trust assumptions inspectable rather than claiming that a service mesh or VPN automatically supplies the whole security architecture.

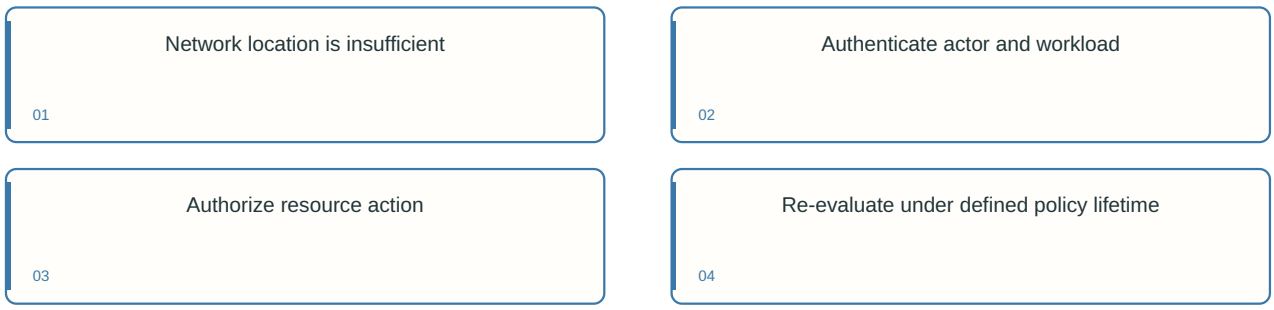


FIG. 18.2.7 **Zero-trust principles · worked snapshots**

Network location is insufficient · Authenticate actor and workload · Authorize resource action · Re-evaluate under defined policy lifetime. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would tenant revocation affect an already issued object capability and a queued export?

permit_demo() denies tenant A reading tenant B even when the action name is allowed. This synthetic predicate is not credential validation. Extend tests to cache keys, queued jobs, exports, and object capabilities; a correct predicate unused by a worker does not enforce anything.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Cached credentials and policy need expiry and revocation decisions.

Carry forward: Network location is insufficient · Authenticate actor and workload · Authorize resource action · Re-evaluate under defined policy lifetime.

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

Tenant isolation, audit & deletion

Prerequisites: 18.2 · Secrets, encryption & trust boundaries 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a multi-tenant SaaS application where an administrator must never gain access to another organization's data.

This chapter examines tenant isolation, audit & deletion through concrete mechanisms, state snapshots, failure cases, and alternatives.

18.3.1 · API security and common vulnerabilities

API risks include broken object authorization, excessive data exposure, unsafe resource consumption, injection, and server-side request abuse. Validate inputs but also constrain how they are used. Parameterized SQL protects a query boundary; it does not authorize a tenant's access to the selected row. Bound expensive exports and validate callback destinations. Maintain negative tests for object IDs, property changes, quotas, and privileged endpoints so a syntactically valid request cannot bypass business security.

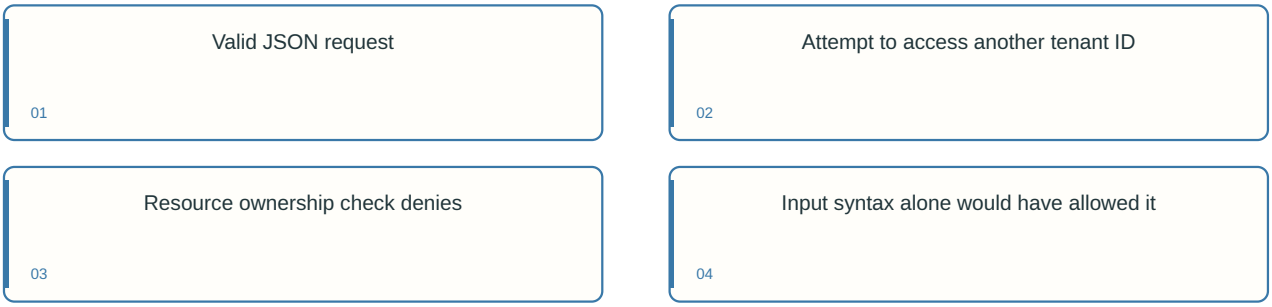


FIG. 18.3.1 **API security and common vulnerabilities · worked snapshots**

Valid JSON request · Attempt to access another tenant ID · Resource ownership check denies · Input syntax alone would have allowed it. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.2 · Multi-tenant isolation

Tenant isolation must survive every path: API lookup, cache key, job queue, object URL, search filter, export, telemetry, and administrative action. Carry trusted tenant context and revalidate resource relationships rather than accepting arbitrary tenant IDs in messages. Composite keys or database policies can add enforcement when correctly configured. A global cache keyed only by document ID can bypass an otherwise correct database query. Test two tenants with overlapping local IDs and adversarial cross-access attempts.

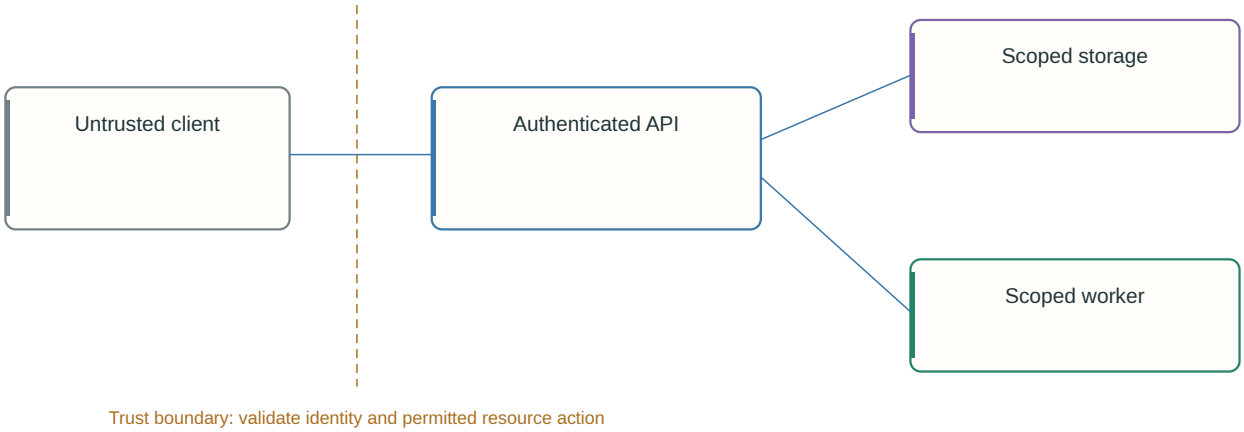


FIG. 18.3.2 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.3 · Data minimization

Data minimization retains only information required for a defined purpose. An observability event often needs an operation ID and error class, not a raw document or access token. Reducing retained sensitive data lowers exposure and makes deletion and recovery simpler. Avoid collecting fields merely because future analysis might find them useful. Document purpose, access, retention, and derived uses; privacy is affected by architecture choices long before an organization writes a retention policy.

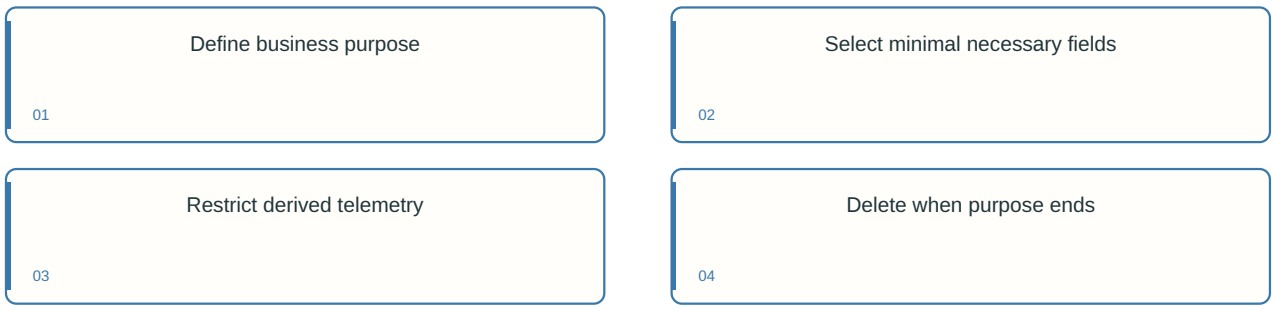


FIG. 18.3.3 **Data minimization · worked snapshots**

Define business purpose · Select minimal necessary fields · Restrict derived telemetry · Delete when purpose ends. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.4 · Audit trails

Audit trails record security-relevant actions with actor, scope, action, target, outcome, and relevant time or sequence. Protect integrity and access; do not let ordinary operators silently rewrite the evidence of their own privileged actions. Audit logs can themselves expose sensitive data, so store identifiers and necessary context rather than entire payloads. Logging “request received” is not evidence that an action committed. Record the actual outcome boundary and reconcile interrupted privileged workflows.

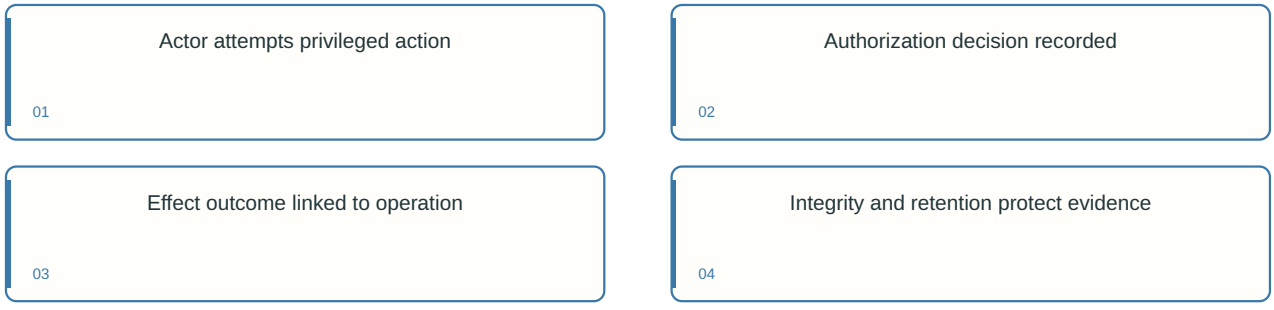


FIG. 18.3.4 **Audit trails · worked snapshots**

Actor attempts privileged action · Authorization decision recorded · Effect outcome linked to operation · Integrity and retention protect evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.5 · Privacy, retention, and deletion

Privacy lifecycle includes collection, access, correction, retention, and deletion across raw and derived copies. A delete API returning success before all required deletion work is durable creates an ambiguous promise. Model deletion as a tracked workflow when it spans stores, backups, indexes, and caches. Legal obligations vary and need appropriate organizational review; this design discussion does not invent jurisdiction-specific rules. Test that restoring historical data does not silently resurrect deleted content without applying the deletion history.

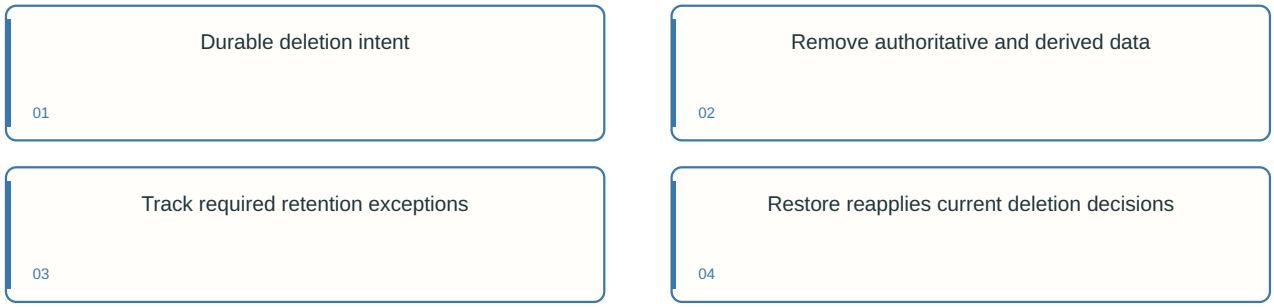


FIG. 18.3.5 **Privacy, retention, and deletion · worked snapshots**

Durable deletion intent · Remove authoritative and derived data · Track required retention exceptions · Restore reapplies current deletion decisions. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How would tenant revocation affect an already issued object capability and a queued export?

permit_demo() denies tenant A reading tenant B even when the action name is allowed. This synthetic predicate is not credential validation. Extend tests to cache keys, queued jobs, exports, and object capabilities; a correct predicate unused by a worker does not enforce anything.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Legal obligations vary and need appropriate organizational review; this design discussion does not invent jurisdiction-specific rules.

Carry forward: Durable deletion intent · Remove authoritative and derived data · Track required retention exceptions · Restore reapplies current deletion decisions.

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   `Access reviewed · 21 October 2026`

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   `Access reviewed · 21 October 2026`

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   `Access reviewed · 21 October 2026`

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   `Access reviewed · 21 October 2026`

Supply chains and security reviews

Prerequisites: 18.3 · Tenant isolation, audit & deletion 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Design a multi-tenant SaaS application where an administrator must never gain access to another organization's data.

This chapter examines supply chains and security reviews through concrete mechanisms, state snapshots, failure cases, and alternatives.

18.4.1 · Supply-chain security

Supply-chain security covers dependencies, build tools, artifacts, deployment permissions, and update paths. Pin or otherwise control dependency resolution, verify artifact identity, and keep build and runtime privileges appropriately narrow. A signed artifact proves a signing relationship, not that its code is harmless. Review provenance, vulnerability response, and the ability to revoke a compromised release. Avoid letting an untrusted pull request access deployment secrets merely because the normal branch build is trusted.

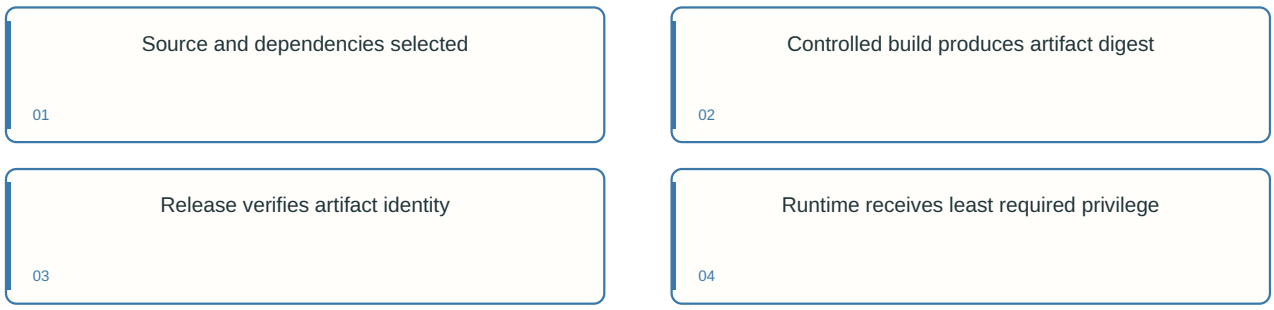


FIG. 18.4.1 **Supply-chain security · worked snapshots**

Source and dependencies selected · Controlled build produces artifact digest · Release verifies artifact identity · Runtime receives least required privilege. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.4.2 · Security vs usability trade-offs

Security and usability interact through real user behavior. A protection that makes legitimate recovery impossible may drive unsafe workarounds; a convenience shortcut that treats every administrator as global creates cross-tenant risk. Evaluate the concrete journey, threat, and fallback. Progressive friction, scoped administration, clear recovery, and safe defaults can improve both. Do not reduce the trade-off to a score that permits violating a mandatory isolation rule; some constraints must remain hard requirements.

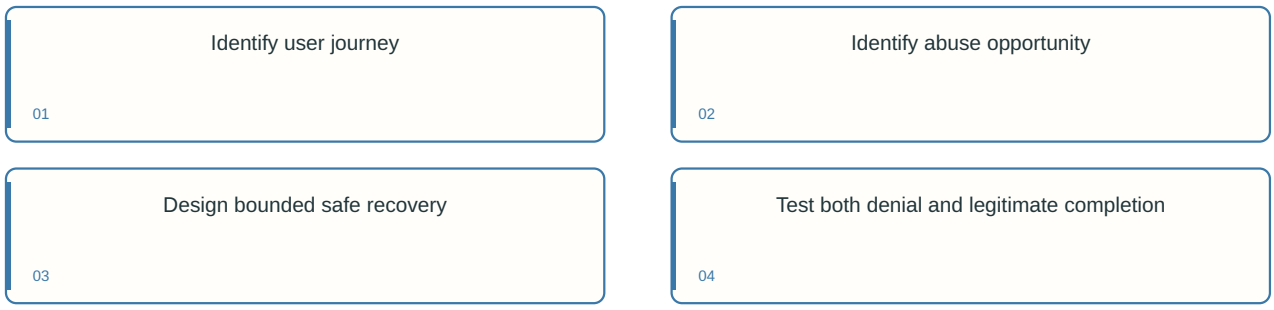


FIG. 18.4.2 **Security vs usability trade-offs · worked snapshots**

Identify user journey · Identify abuse opportunity · Design bounded safe recovery · Test both denial and legitimate completion. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

18.4.3 · Secure-by-design reviews

A secure-by-design review follows one sensitive operation end to end. For an export, verify actor and tenant, resource filters, object permissions, queue context, signed download lifetime, audit outcome, and deletion. Include failures such as revoked membership after enqueue and a worker retry after cancellation. Record assumptions and ownership of controls. A checklist helps coverage, but its evidence should be concrete tests and reviewed configurations rather than a declaration that the system is secure.

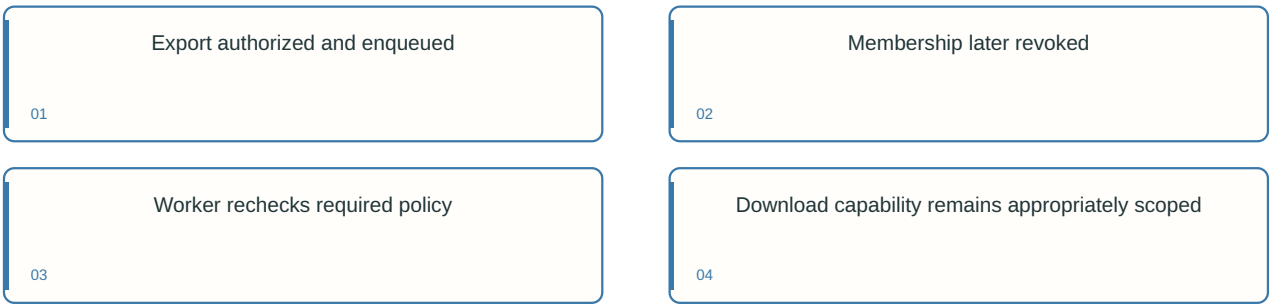


FIG. 18.4.3 **Secure-by-design reviews · worked snapshots**

Export authorized and enqueued · Membership later revoked · Worker rechecks required policy · Download capability remains appropriately scoped. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

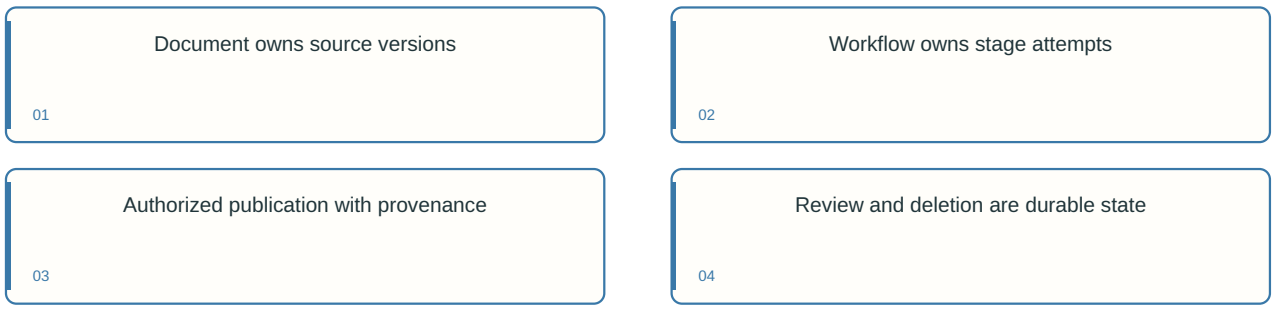


FIG. 18.4.4 **The running system · invariant and recovery review**

Review whether any stage silently changes source versions, whether an old attempt can publish after deletion, and whether human decisions are preserved during reprocessing. Test duplicate tasks, interrupted extraction, invalid output, reviewer concurrency, stale completion, missing object, index lag, and full restore. A pipeline that extracts a few fields once has not demonstrated the required recoverable, explainable platform.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · SYNTHETIC TENANT POLICY

```
from mechanisms import permit_demo
actor = {"tenant":"A", "active":True, "actions":["read"]}
assert permit_demo(actor, {"tenant":"A"}, "read")
assert not permit_demo(actor, {"tenant":"B"}, "read")
```

This is an authorization predicate, not an authentication implementation. Every enforcement point needs trustworthy identity and resource context. A correctly denied API cannot compensate for an unprotected object link or worker.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · How would tenant revocation affect an already issued object capability and a queued export?

permit_demo() denies tenant A reading tenant B even when the action name is allowed. This synthetic predicate is not credential validation. Extend tests to cache keys, queued jobs, exports, and object capabilities; a correct predicate unused by a worker does not enforce anything.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Record assumptions and ownership of controls.

Carry forward: Export authorized and enqueued · Membership later revoked · Worker rechecks required policy · Download capability remains appropriately scoped.

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

PART 5 / UNIT 19

Observability, Monitoring, and Incident Engineering

The running example

An API's p99 latency increases from 200 ms to 5 seconds. Use traces, metrics, logs, and database analysis to localize the bottleneck.

Unit 19 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.



FIG. 19.0.1 **This slow request is dominated by waiting**

The synthetic five-second path spends 92% in pool acquisition. More CPU is not an evidence-based first fix for this execution.

Why this view: Separate stage waits and execution rather than attach one unexplained number to the entire request. Author-created illustration. Numbers are illustrative unless stated otherwise.

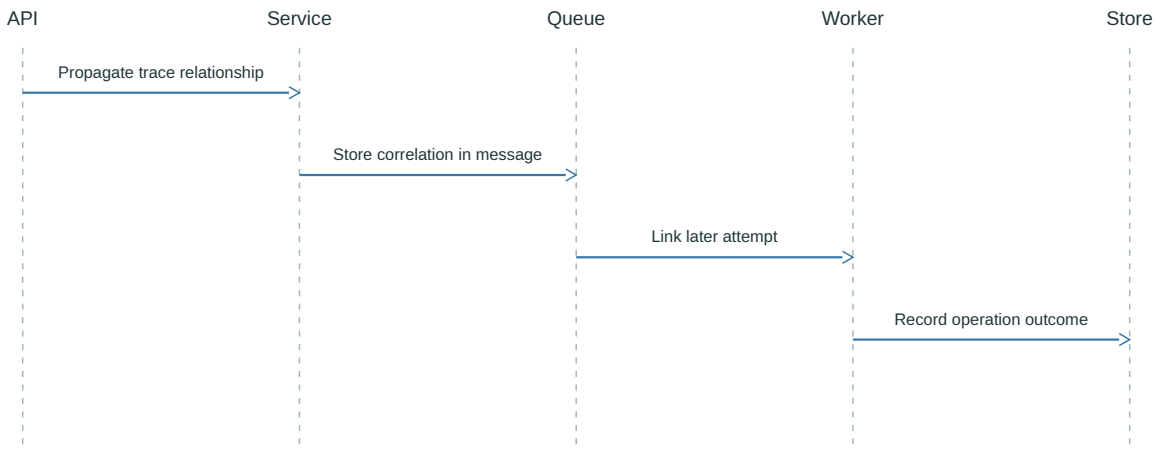


FIG. 19.0.2 **request ID propagation**

Trace IDs relate evidence; they are not authorization or proof that the entire workflow is complete.

Why this view: Follow explicit message ordering and locate the acknowledgment or effect boundary. Author-created illustration. Numbers are illustrative unless stated otherwise.

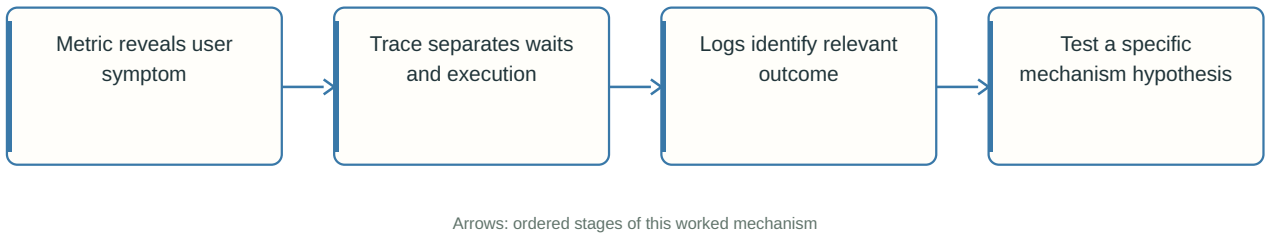


FIG. 19.0.3 **metrics-to-root-cause workflow**

No one signal substitutes for a causal investigation with explicit counterevidence.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.



FIG. 19.0.4 **error-budget burn chart**

Percentages: at a 99.9% event objective, 1% bad events burns the 0.1% allowance ten times as fast.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

Observe behavior with three signals

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Networking Foundations for System Design, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control

6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An API's p99 latency increases from 200 ms to 5 seconds. Use traces, metrics, logs, and database analysis to localize the bottleneck.

This chapter examines observe behavior with three signals through concrete mechanisms, state snapshots, failure cases, and alternatives.

19.1.1 · Monitoring vs observability

Monitoring tracks known signals and conditions; observability supports explaining system behavior from outputs and instrumentation. The distinction is a useful emphasis, not a universal product boundary. A dashboard that says CPU is normal may not explain a five-second payment request. Start from a user-visible symptom and connect operations, waits, dependencies, and state transitions. Instrumentation is valuable when it helps answer questions and act, not simply because more data is being collected.

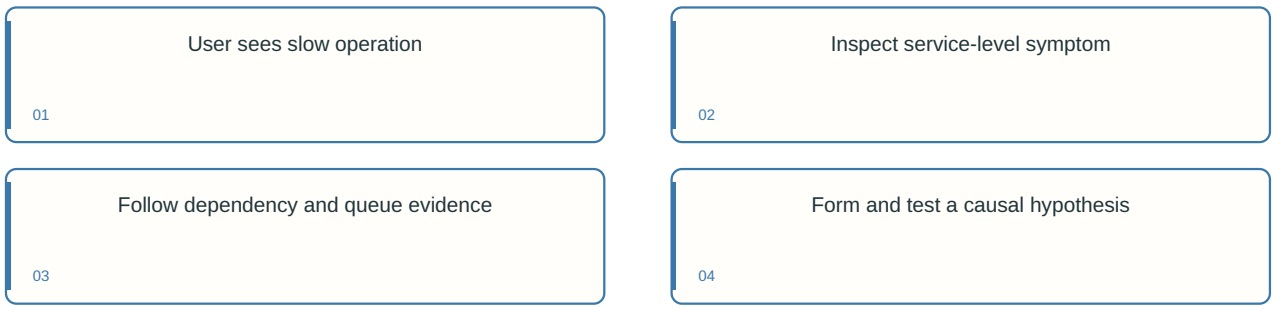


FIG. 19.1.1 **Monitoring vs observability · worked snapshots**

User sees slow operation · Inspect service-level symptom · Follow dependency and queue evidence · Form and test a causal hypothesis. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.2 · Logs, metrics, and traces

Logs record discrete events, metrics aggregate measurements, and traces relate operations along a path. They complement one another: a latency histogram reveals the population tail, a trace localizes one slow request, and structured logs explain a particular error. None alone guarantees a complete account. Sampling, dropped telemetry, inconsistent clocks, and privacy limits affect the evidence. Use stable operation and resource identities to connect signals without making unbounded identifiers metric labels.

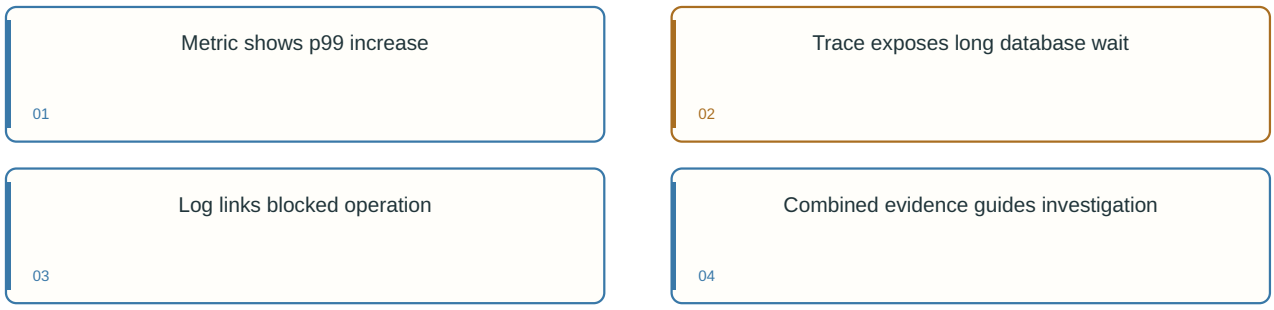


FIG. 19.1.2 **Logs, metrics, and traces · worked snapshots**

Metric shows p99 increase · Trace exposes long database wait · Log links blocked operation · Combined evidence guides investigation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.3 · Structured logging

Structured logging uses named fields with stable meanings rather than relying only on free-form messages. Include event type, operation ID, tenant-safe scope, outcome, and relevant duration. Do not log credentials or raw sensitive documents as a debugging shortcut. A request-start log and request-finish log describe different boundaries; missing finish events can indicate interruption but not a specific business result. Bound volume and field sizes so a failure does not create a logging-induced outage.

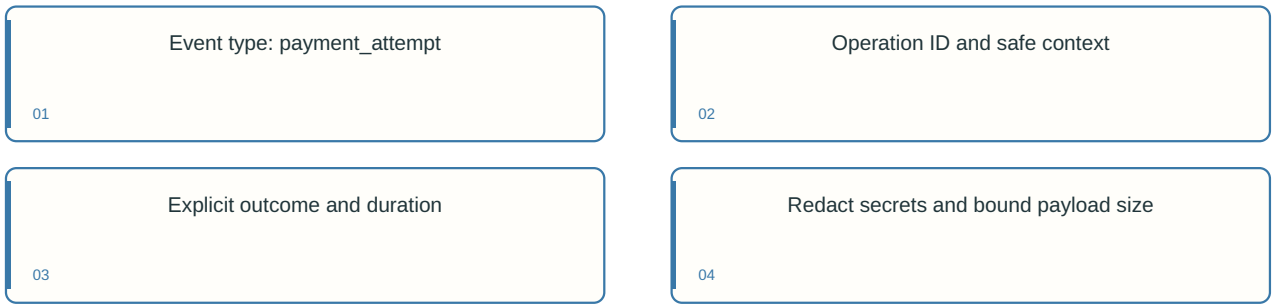


FIG. 19.1.3 **Structured logging · worked snapshots**

Event type: payment_attempt · Operation ID and safe context · Explicit outcome and duration · Redact secrets and bound payload size. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.4 · Distributed tracing

A distributed trace links spans representing work and waits across components. Parent-child relationships describe instrumentation structure; asynchronous work may need links rather than a misleading synchronous parent lifetime. A five-second root span with 4.6 seconds in pool acquisition suggests a different intervention from 4.6 seconds of database execution. Instrument queue wait and connection wait explicitly. A trace is one sampled execution, so compare distributions and errors before generalizing its cause to every slow request.

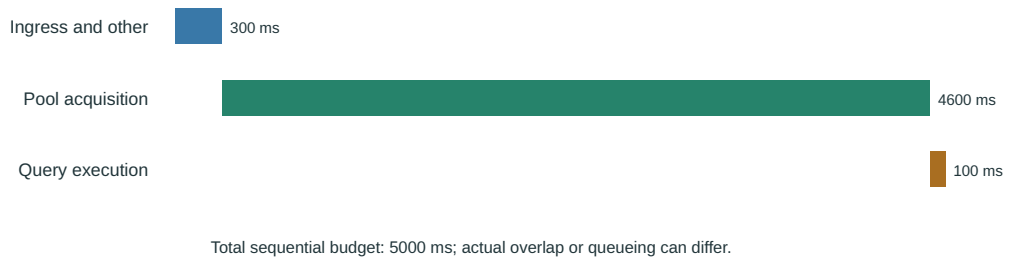


FIG. 19.1.4 **This slow request is dominated by waiting**

The synthetic five-second path spends 92% in pool acquisition. More CPU is not an evidence-based first fix for this execution.

Why this view: Separate stage waits and execution rather than attach one unexplained number to the entire request. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.5 · Trace IDs, span IDs, context propagation

Trace IDs identify a trace, span IDs identify individual spans, and propagation carries the relevant context across boundaries. Validate or regenerate untrusted incoming context according to policy and avoid treating trace identity as authorization. For queues, propagate correlation or links so a later worker can be connected to the originating operation without implying it ran during the HTTP request. Standard propagation formats improve interoperability; they do not ensure every library or message transport is instrumented automatically.

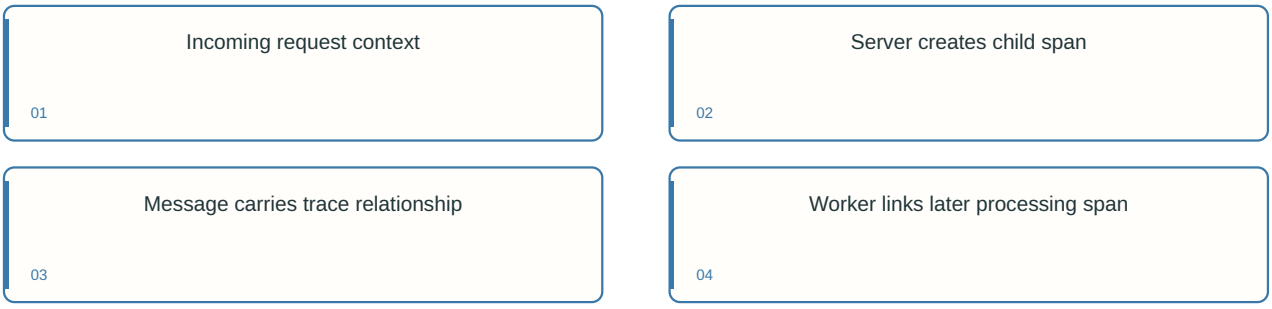


FIG. 19.1.5 **Trace IDs, span IDs, context propagation · worked snapshots**

Incoming request context · Server creates child span · Message carries trace relationship · Worker links later processing span. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.6 · Golden signals

Golden signals commonly include latency, traffic, errors, and saturation. They describe user serving and resource pressure together. A service can have low error rate while latency grows because work queues rather than fails immediately. Break down outcomes and routes where needed so one high-volume cheap endpoint does not hide a critical low-volume failure. Definitions must identify eligible requests and useful results; counting every HTTP 200 as success can conceal a wrong or empty business response.

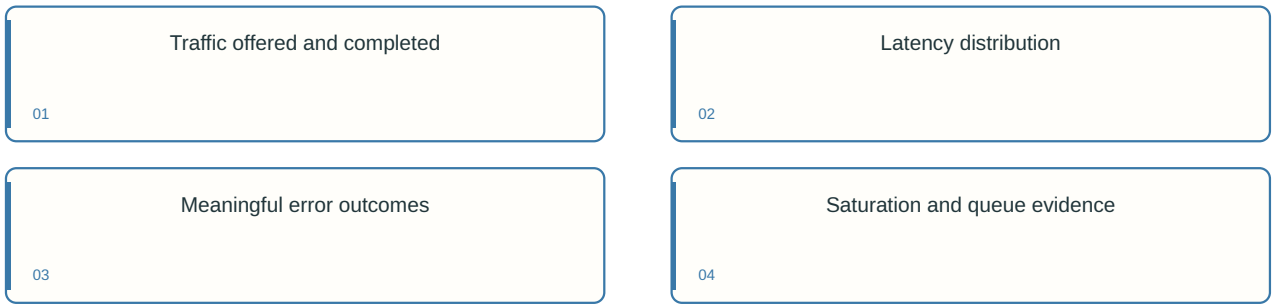


FIG. 19.1.6 **Golden signals · worked snapshots**

Traffic offered and completed · Latency distribution · Meaningful error outcomes · Saturation and queue evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which evidence would make a latency hypothesis convincing rather than merely correlated with a release?

Use the five-second path with 4.6 seconds in pool acquisition. Separate wait from query execution and compare population evidence. Track submission-to-terminal job age rather than only HTTP acceptance. Avoid raw document content or high-cardinality request IDs as ordinary metric labels.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Break down outcomes and routes where needed so one high-volume cheap endpoint does not hide a critical low-volume failure.

Carry forward: Traffic offered and completed · Latency distribution · Meaningful error outcomes · Saturation and queue evidence.

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

Choose indicators, budgets & alerts

Prerequisites: 19.1 · Observe behavior with three signals 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An API's p99 latency increases from 200 ms to 5 seconds. Use traces, metrics, logs, and database analysis to localize the bottleneck.

This chapter examines choose indicators, budgets & alerts through concrete mechanisms, state snapshots, failure cases, and alternatives.

19.2.1 · RED and USE methods

RED focuses on rate, errors, and duration for request-serving components; USE focuses on utilization, saturation, and errors for resources. Apply the lens to the question. A database connection pool can be saturated while host CPU is low, explaining why increasing CPU does not help. Averages hide bursts, so examine time resolution and per-owner skew. These methods organize investigation rather than replace workload-specific indicators such as oldest unprocessed payment age.

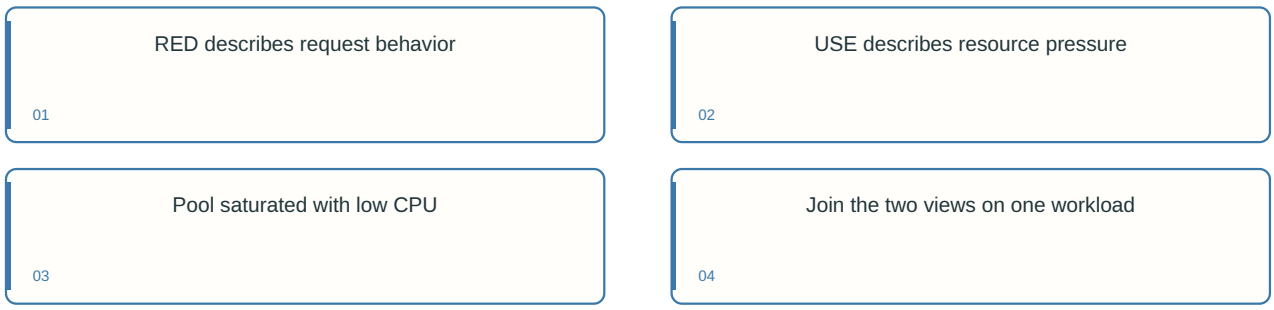


FIG. 19.2.1 **RED and USE methods · worked snapshots**

RED describes request behavior · USE describes resource pressure · Pool saturated with low CPU · Join the two views on one workload. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.2 · SLIs, SLOs, error budgets

An SLI measures a chosen service behavior; an SLO sets a target; an error budget quantifies the allowed shortfall over a window. Define both good events and eligible events, including relevant failures. For a 99.9% target, a 1% bad-event rate consumes budget ten times as fast as the allowed 0.1% rate. Segment critical operations appropriately and ensure low-volume denominators are handled thoughtfully. An SLO is not a guarantee about every individual request.

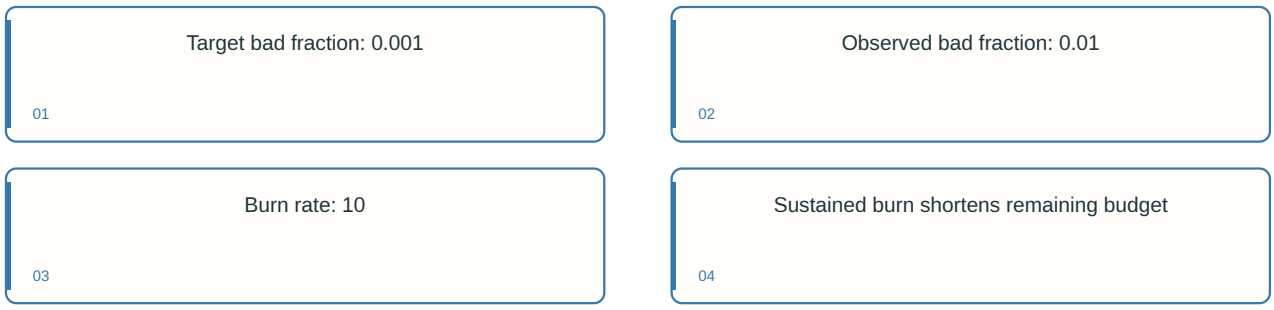


FIG. 19.2.2 **SLIs, SLOs, error budgets · worked snapshots**

Target bad fraction: 0.001 · Observed bad fraction: 0.01 · Burn rate: 10 · Sustained burn shortens remaining budget. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.3 · Alert design and alert fatigue

Alerts should identify actionable user harm or imminent loss of an objective with a clear owner and response. Paging on every isolated CPU spike creates noise; paging on sustained fast error-budget burn can align response with impact. Use appropriate windows and thresholds, and pair urgent alerts with runbooks. Avoid alerts that fire constantly during known degradation without changing the action. Evaluate false positives, missed incidents, and the operational load imposed on responders.

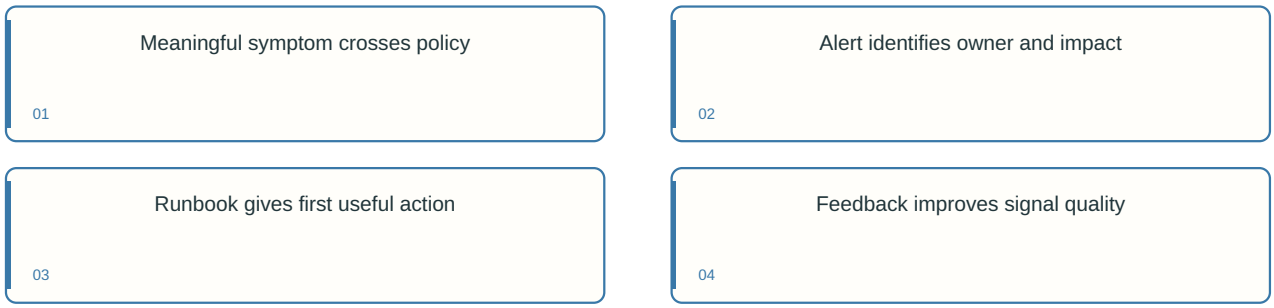


FIG. 19.2.3 **Alert design and alert fatigue · worked snapshots**

Meaningful symptom crosses policy · Alert identifies owner and impact · Runbook gives first useful action · Feedback improves signal quality. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.4 · Sampling and telemetry cost

Sampling and retention control telemetry cost but affect which questions can be answered. Head sampling decides early; tail sampling can select traces based on later observed behavior but needs buffering and capacity. High-cardinality labels such as unique request IDs can create unbounded time-series growth. Keep detailed identities in suitable logs or traces and use bounded dimensions for metrics. Protect telemetry ingestion during outages so error bursts do not consume all application or observability capacity.

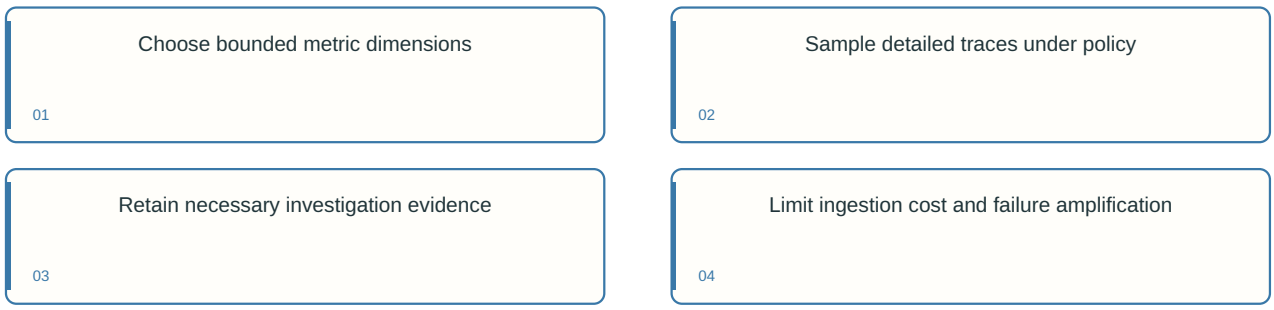


FIG. 19.2.4 **Sampling and telemetry cost · worked snapshots**

Choose bounded metric dimensions · Sample detailed traces under policy · Retain necessary investigation evidence · Limit ingestion cost and failure amplification. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.5 · Dashboards

A dashboard should answer a decision question: is the service meeting its objective, what changed, and where is capacity constrained? Put user outcomes before infrastructure vanity metrics. Show denominators, time windows, units, data gaps, and deployment markers. A chart without a stable definition can mislead during an incident. Include drill-down paths and ownership, but avoid hundreds of panels that obscure the one signal requiring action. Verify dashboards against known synthetic or historical scenarios.

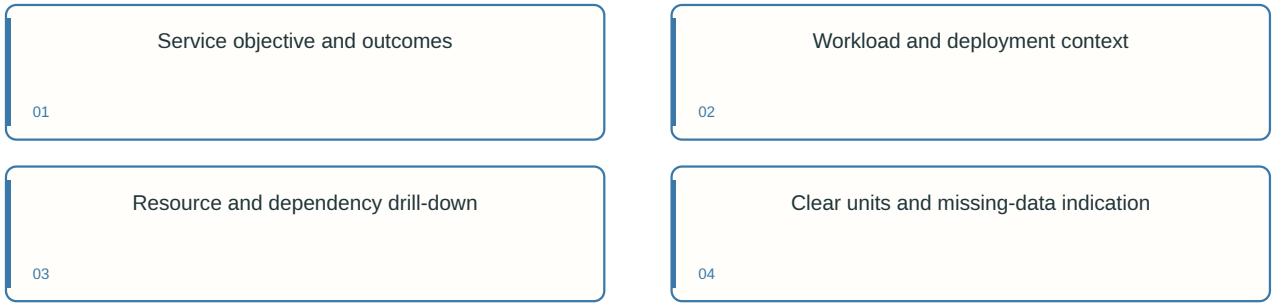


FIG. 19.2.5 **Dashboards · worked snapshots**

Service objective and outcomes · Workload and deployment context · Resource and dependency drill-down · Clear units and missing-data indication. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which evidence would make a latency hypothesis convincing rather than merely correlated with a release?

Use the five-second path with 4.6 seconds in pool acquisition. Separate wait from query execution and compare population evidence. Track submission-to-terminal job age rather than only HTTP acceptance. Avoid raw document content or high-cardinality request IDs as ordinary metric labels.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Include drill-down paths and ownership, but avoid hundreds of panels that obscure the one signal requiring action.

Carry forward: Service objective and outcomes · Workload and deployment context · Resource and dependency drill-down · Clear units and missing-data indication.

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

Profile and investigate production

Prerequisites: 19.2 · Choose indicators, budgets & alerts 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An API's p99 latency increases from 200 ms to 5 seconds. Use traces, metrics, logs, and database analysis to localize the bottleneck.

This chapter examines profile and investigate production through concrete mechanisms, state snapshots, failure cases, and alternatives.

19.3.1 · Profiling

Profiling samples or instruments CPU, allocation, blocking, and other runtime behavior. A profile of one representative workload can reveal expensive parsing or lock contention hidden by request metrics. Sampling overhead and production privacy need consideration. A function using most CPU in a profile is not necessarily on the latency-critical path; I/O waiting may dominate response time. Compare profile findings with traces and completion rate, and re-measure after a focused optimization.

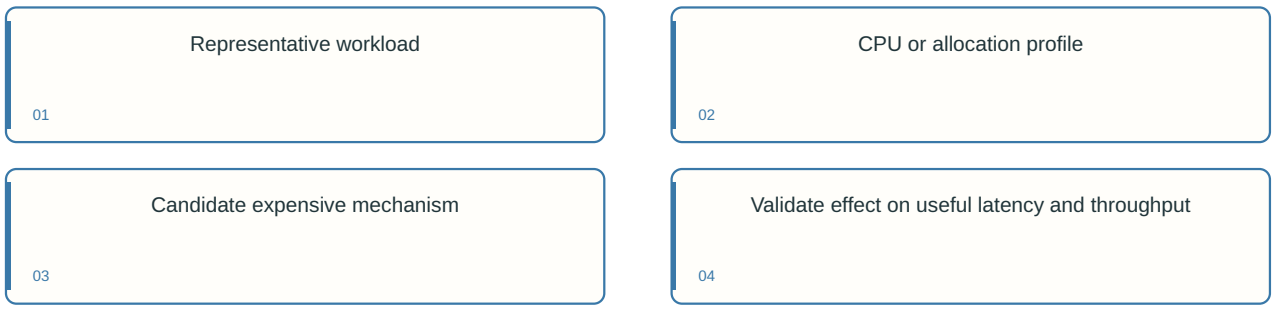


FIG. 19.3.1 **Profiling · worked snapshots**

Representative workload · CPU or allocation profile · Candidate expensive mechanism · Validate effect on useful latency and throughput. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.2 · Query and application performance diagnosis

Diagnose a slow application by splitting waiting from execution. Inspect queue time, pool acquisition, lock wait, query plan, rows processed, network, and serialization. If p99 rises from 200 ms to five seconds while CPU remains modest, a saturated pool or lock can be plausible but requires evidence. Compare slow and normal paths and correlate with changes. Do not restart every component before preserving useful evidence and understanding whether the action could worsen an overload.

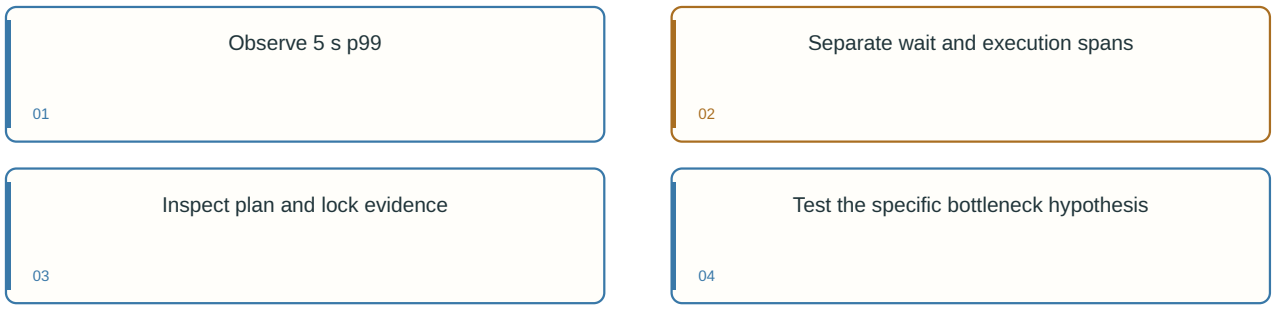


FIG. 19.3.2 **Query and application performance diagnosis · worked snapshots**

Observe 5 s p99 · Separate wait and execution spans · Inspect plan and lock evidence · Test the specific bottleneck hypothesis. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.3 · Incident response

Incident response coordinates mitigation, communication, evidence, and decision ownership. Assign an incident lead for sufficiently complex events, maintain a timeline, and reduce harmful changes. Restore useful service with a justified reversible intervention while preserving data correctness. A rollback is not safe if it misreads a migrated schema. Track uncertain business outcomes such as pending charges for reconciliation after serving recovers; ending an infrastructure alert does not mean the complete incident outcome is resolved.

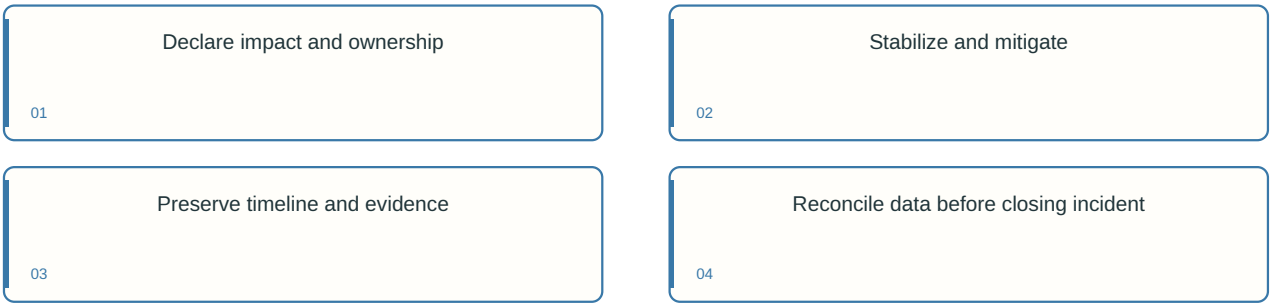


FIG. 19.3.3 **Incident response · worked snapshots**

Declare impact and ownership · Stabilize and mitigate · Preserve timeline and evidence · Reconcile data before closing incident. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.4 · On-call and runbooks

On-call needs actionable alerts, access, runbooks, escalation, and sustainable ownership. A runbook should specify diagnosis, safe actions, stop conditions, and verification, not merely “restart the service.” Rehearse recovery paths and ensure emergency access is audited. Assign service owners who can improve recurring failure mechanisms. Operational burden is architecture evidence: repeated manual reconciliation or night-time cache rebuilds may justify changing the design rather than making the instructions longer.

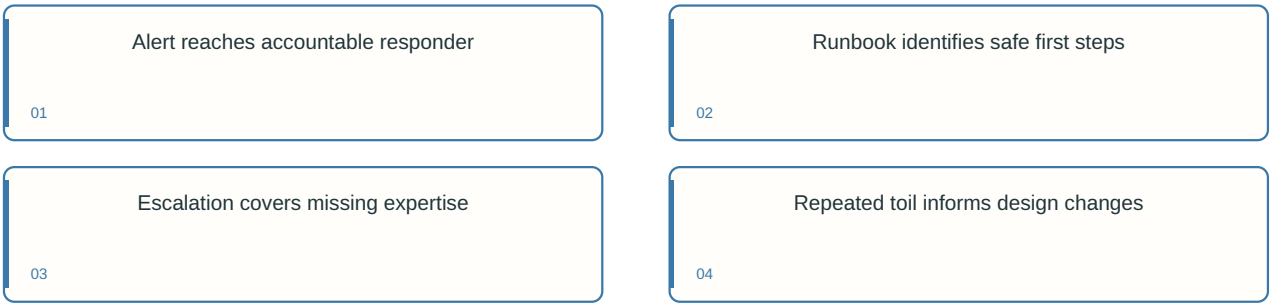


FIG. 19.3.4 **On-call and runbooks · worked snapshots**

Alert reaches accountable responder · Runbook identifies safe first steps · Escalation covers missing expertise · Repeated toil informs design changes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.5 · Root-cause investigation

Root-cause investigation tests explanations against evidence and distinguishes triggers from contributing conditions. A deployment may trigger a failure because an unbounded queue or incompatible migration made the system fragile. Correlation is a starting hypothesis, not proof. Ask what changed, which requests failed, what resource path explains the effect, and whether the proposed fix changes that mechanism. Several interacting causes are common; a single short label can hide the conditions needed to prevent recurrence.

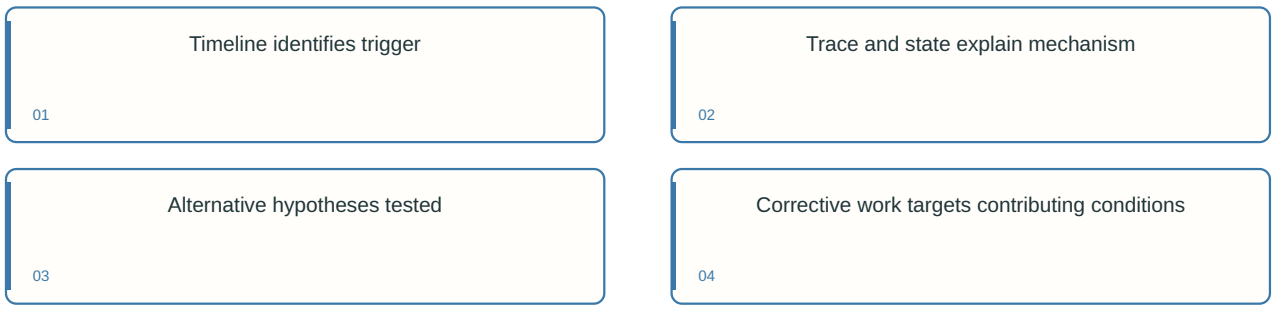


FIG. 19.3.5 **Root-cause investigation · worked snapshots**

Timeline identifies trigger · Trace and state explain mechanism · Alternative hypotheses tested · Corrective work targets contributing conditions. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · Which evidence would make a latency hypothesis convincing rather than merely correlated with a release?

Use the five-second path with 4.6 seconds in pool acquisition. Separate wait from query execution and compare population evidence. Track submission-to-terminal job age rather than only HTTP acceptance. Avoid raw document content or high-cardinality request IDs as ordinary metric labels.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Ask what changed, which requests failed, what resource path explains the effect, and whether the proposed fix changes that mechanism.

Carry forward: Timeline identifies trigger · Trace and state explain mechanism · Alternative hypotheses tested · Corrective work targets contributing conditions.

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

Learn from incidents and trace async work

Prerequisites: 19.3 · Profile and investigate production 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An API's p99 latency increases from 200 ms to 5 seconds. Use traces, metrics, logs, and database analysis to localize the bottleneck.

This chapter examines learn from incidents and trace async work through concrete mechanisms, state snapshots, failure cases, and alternatives.

19.4.1 · Blameless postmortems

A blameless postmortem records impact, timeline, detection, mitigation, and contributing system conditions without pretending decisions were made with hindsight. Include what helped as well as what failed, then assign specific changes and evidence. “Be more careful” is not an architectural mitigation. A postmortem should explain why safeguards did not prevent or limit harm and how the system will behave differently next time. Protect sensitive customer and employee information in the resulting record.

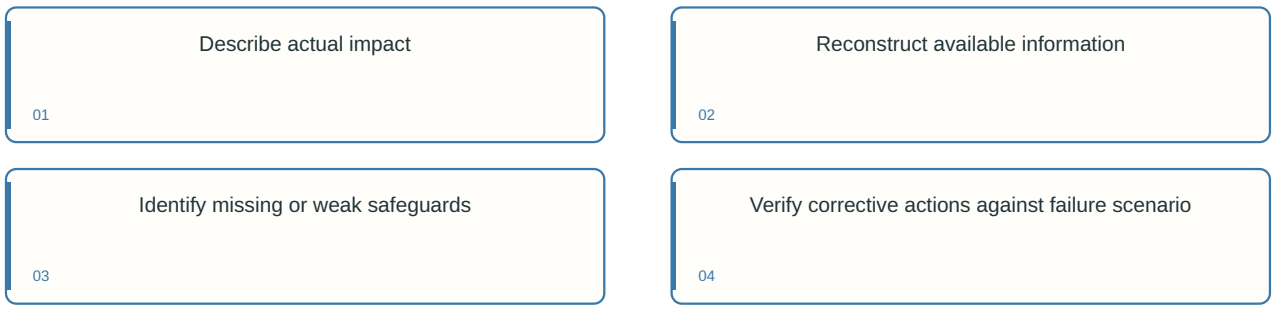


FIG. 19.4.1 **Blameless postmortems · worked snapshots**

Describe actual impact · Reconstruct available information · Identify missing or weak safeguards · Verify corrective actions against failure scenario. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.4.2 · Capacity monitoring

Capacity monitoring observes rates, resource pressure, queue age, growth, and surviving capacity during failure. A low fleet CPU average can hide one saturated shard or a full connection pool. Alert on meaningful approach to a limit with enough lead time for the available response. Track storage growth and compaction or backup overhead as well as foreground traffic. Forecasts should retain uncertainty and be updated when access patterns, release behavior, or business events change.

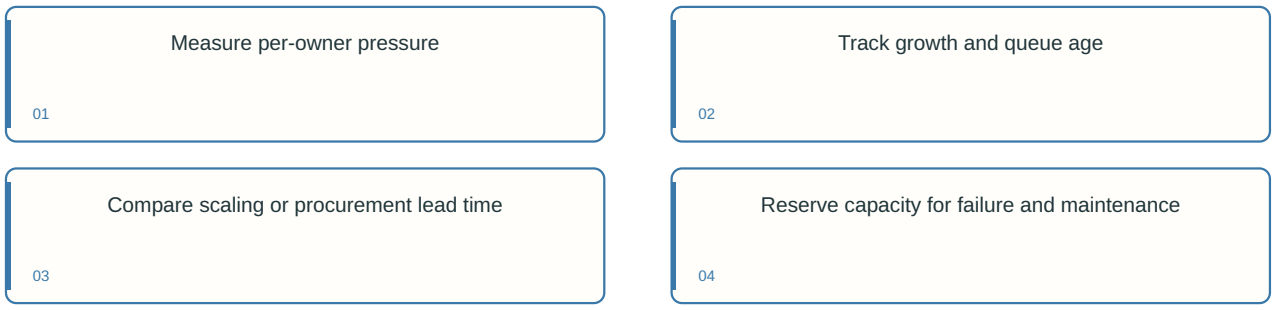


FIG. 19.4.2 **Capacity monitoring · worked snapshots**

Measure per-owner pressure · Track growth and queue age · Compare scaling or procurement lead time · Reserve capacity for failure and maintenance. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

19.4.3 · Observability across asynchronous workflows

Asynchronous observability follows a business operation beyond the original request. Link submission, queue delivery, attempts, stage commits, retries, and terminal outcomes using stable identities. Separate queue wait from processing duration and record duplicate or stale-attempt rejection. A trace ending at HTTP 202 says little about whether an extraction or refund eventually finished. Define end-to-end age and completion indicators, and maintain a way to locate stuck operations across brokers, workers, databases, and external providers.

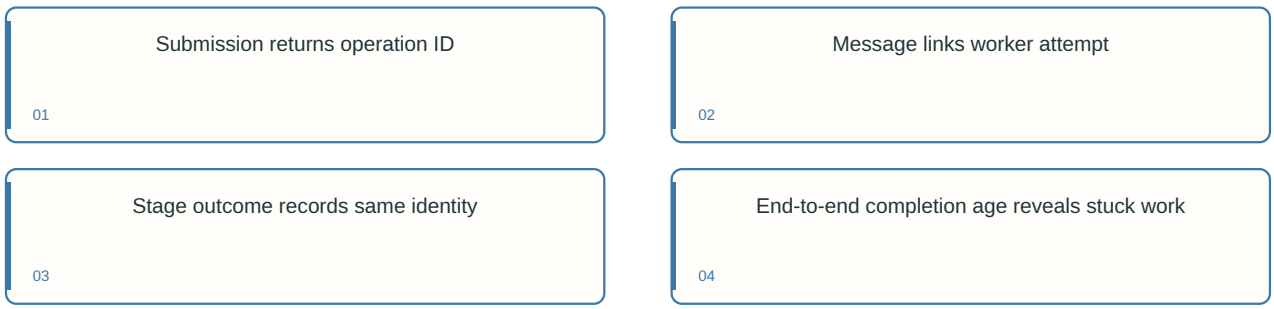


FIG. 19.4.3 **Observability across asynchronous workflows · worked snapshots**

Submission returns operation ID · Message links worker attempt · Stage outcome records same identity · End-to-end completion age reveals stuck work. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

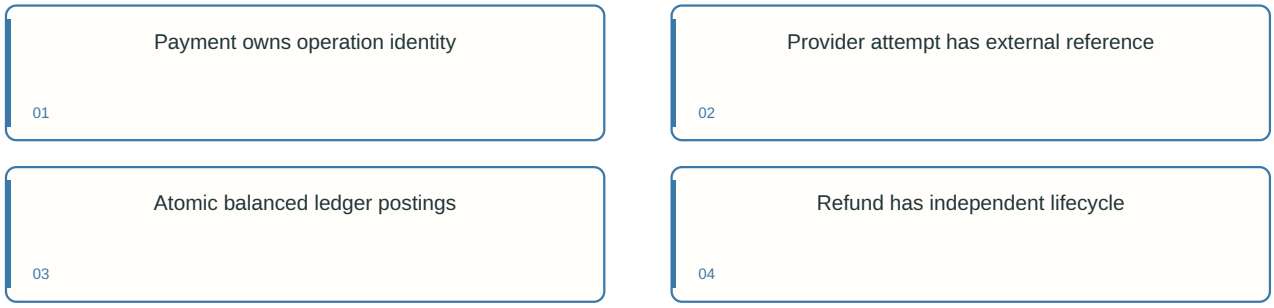


FIG. 19.4.4 **The running system · invariant and recovery review**

Review which event means financial success, who owns the ledger invariant, and what recovery does after deduplication history is restored behind provider effects. Test charge-then-crash, callback-before-response, duplicate settlement, partial refund, and restore from an older snapshot. Verify no duplicate local postings and no unsupported claim that an unknown provider outcome is safely declined.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · ONE TRACE ACCOUNTING MODEL

```
stages_ms = {"other":300, "pool_wait":4600, "query":100}
assert sum(stages_ms.values()) == 5000
assert stages_ms["pool_wait"] / sum(stages_ms.values()) == .92
```

The trace spends 92% of its elapsed time waiting for a pool. Compare population traces, saturation, and lock evidence before generalizing the cause. The arithmetic does not identify why the pool is full by itself.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · Which evidence would make a latency hypothesis convincing rather than merely correlated with a release?

Use the five-second path with 4.6 seconds in pool acquisition. Separate wait from query execution and compare population evidence. Track submission-to-terminal job age rather than only HTTP acceptance. Avoid raw document content or high-cardinality request IDs as ordinary metric labels.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A trace ending at HTTP 202 says little about whether an extraction or refund eventually finished.

Carry forward: Submission returns operation ID · Message links worker attempt · Stage outcome records same identity · End-to-end completion age reveals stuck work.

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

PART 5 / UNIT 20

Deployment, Cloud Infrastructure, and Platform Architecture

The running example

Take a single-process application and deploy it into a reliable, observable, multi-instance production environment with safe releases.

Unit 20 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

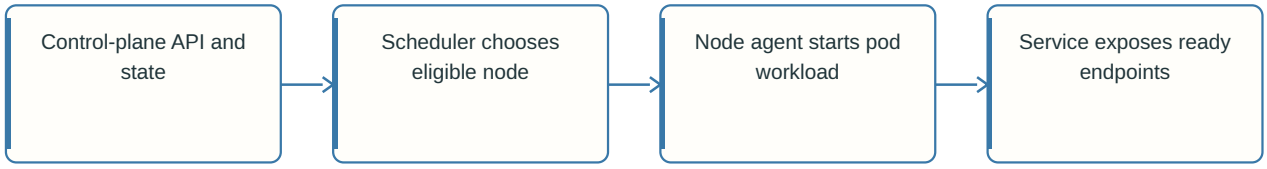
Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Layer	Virtual machine	Container
Operating environment	Guest OS per VM	Shared host kernel
Process resources	Guest-managed processes	Runtime constraints
Persistent state	Explicit attached owner	Not ephemeral writable layer

FIG. 20.0.1 **VMs and containers isolate at different layers**

These are common architecture distinctions, not a claim that one option universally provides sufficient security or reliability.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

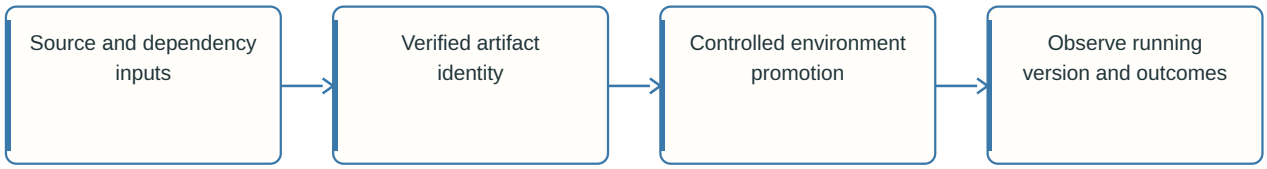


Arrows: ordered stages of this worked mechanism

FIG. 20.0.2 **Kubernetes cluster**

A simplified cluster control/data relationship; scheduling does not supply application data consistency.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 20.0.3 **CI/CD pipeline**

CI/CD preserves evidence and identity rather than merely copying files.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

Stage	Blue	Green
Before switch	Serving existing version	Candidate verified
During switch	Drain under policy	Receives new traffic
Data	Shared or migrated state	Rollback compatibility required

FIG. 20.0.4 **blue-green deployment**

Parallel runtime environments do not automatically make shared schema changes reversible.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

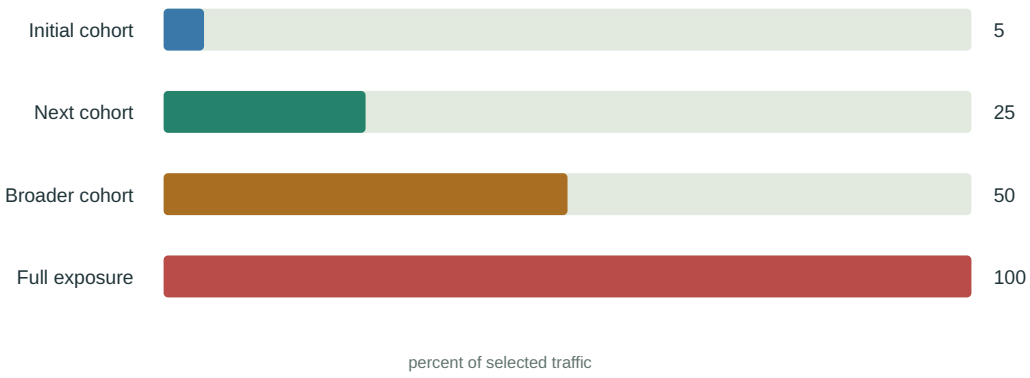


FIG. 20.0.5 **Canary exposure expands only after verification**

Percentages are an illustrative rollout plan, not an automatic safety rule. Each increase depends on meaningful representative checks and stop conditions.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

Environments, VMs & containers

Prerequisites: Networking Foundations for System Design, Application Architecture and Domain-Driven Design, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Take a single-process application and deploy it into a reliable, observable, multi-instance production environment with safe releases.

This chapter examines environments, vms & containers through concrete mechanisms, state snapshots, failure cases, and alternatives.

20.1.1 · Development vs staging vs production

Development, staging, and production serve different purposes but need deliberate compatibility. Staging should exercise relevant protocols, migrations, identity, and failure paths without using uncontrolled production secrets or personal data. Exact scale parity may be impractical, so document which behaviors it cannot validate. Configuration differences are a common source of surprises; keep environment-specific values separate from application semantics. A passing local test does not establish production connection limits, disk behavior, or network policy.

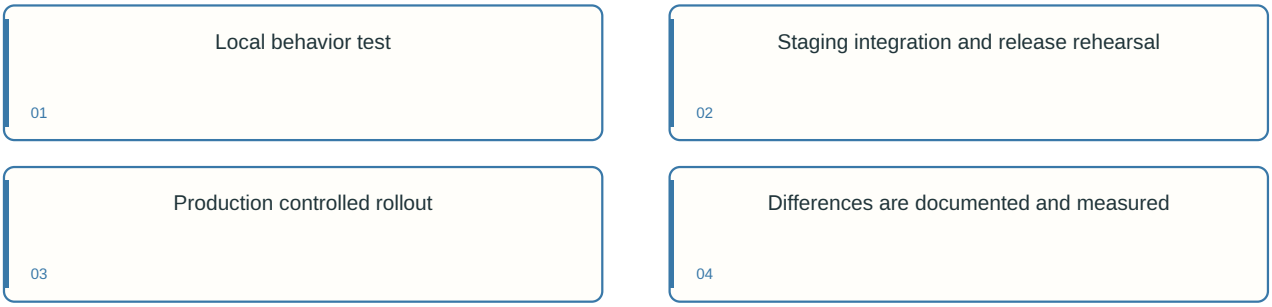


FIG. 20.1.1 **Development vs staging vs production · worked snapshots**

Local behavior test · Staging integration and release rehearsal · Production controlled rollout · Differences are documented and measured. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.2 · Virtual machines vs containers

A virtual machine presents virtualized hardware with its guest operating system; containers commonly isolate processes while sharing a host kernel. Both need image lifecycle, resource controls, identity, network, and storage design. A container is not a tiny VM with all the same isolation guarantees. Choose based on workload, operating expertise, startup and isolation needs. Multiple containers on one host share failure and resource domains, while VM boundaries still do not remove shared infrastructure risks.

Layer	Virtual machine	Container
Operating environment	Guest OS per VM	Shared host kernel
Process resources	Guest-managed processes	Runtime constraints
Persistent state	Explicit attached owner	Not ephemeral writable layer

FIG. 20.1.2 **VMs and containers isolate at different layers**

These are common architecture distinctions, not a claim that one option universally provides sufficient security or reliability.

Why this view: Make the internal structure or state relation visible before claiming its behavior. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.3 · Container fundamentals

A container packages an executable environment while runtime controls constrain CPU, memory, filesystem, and network access. Keep persistent state outside an ephemeral writable layer when it must survive replacement. Image reproducibility and configuration matter more than assuming packaging alone makes deployments deterministic. Resource limits can terminate or throttle work, so applications need graceful shutdown and durable progress. Running as a privileged user or mounting broad host access can weaken the intended boundary.

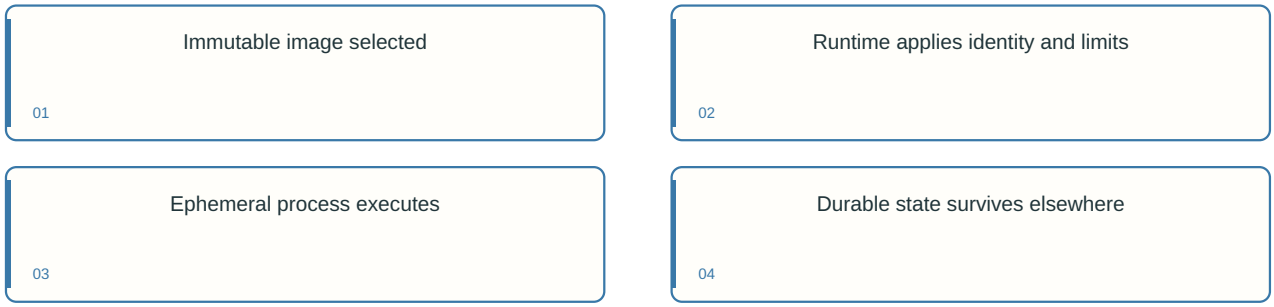


FIG. 20.1.3 **Container fundamentals · worked snapshots**

Immutable image selected · Runtime applies identity and limits · Ephemeral process executes · Durable state survives elsewhere. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.4 · Kubernetes fundamentals

Kubernetes reconciles declared objects with cluster state through its control plane and node agents. Pods, services, controllers, and scheduling provide useful primitives, but they do not automatically supply correct application transactions, backups, tenant isolation, or SLOs. Understand the platform responsibilities you operate or delegate. Start with simpler deployment machinery when it already satisfies the workload; adopting an orchestrator creates a control plane, configuration surface, and operational dependency that must be justified.

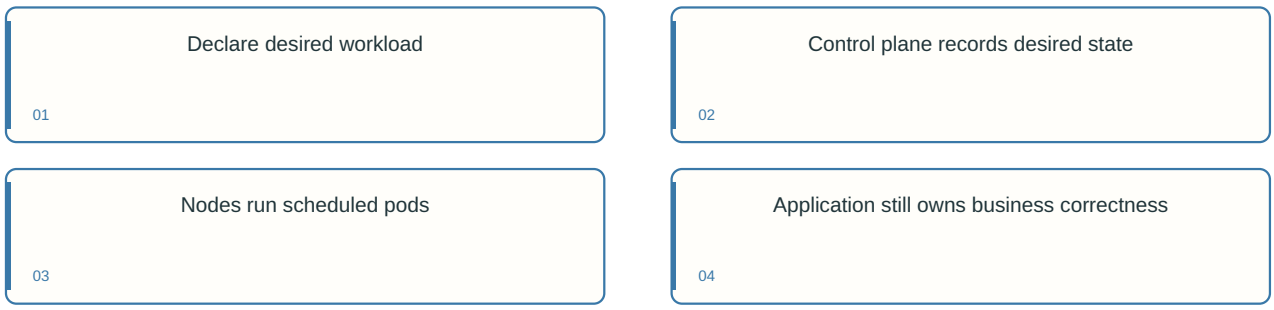


FIG. 20.1.4 **Kubernetes fundamentals · worked snapshots**

Declare desired workload · Control plane records desired state · Nodes run scheduled pods · Application still owns business correctness. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How could a rollback restore old code while making stored data or external effects less correct?

Add a compatible field, deploy readers that tolerate both versions, backfill under a bounded budget, verify, and remove old structure only after old readers retire. Test rollback before contraction and explicitly identify when rollback ceases to be safe. A canary must receive representative traffic.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Understand the platform responsibilities you operate or delegate.

Carry forward: Declare desired workload · Control plane records desired state · Nodes run scheduled pods · Application still owns business correctness.

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Control planes, scheduling & configuration

Prerequisites: 20.1 · Environments, VMs & containers 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Take a single-process application and deploy it into a reliable, observable, multi-instance production environment with safe releases.

This chapter examines control planes, scheduling & configuration through concrete mechanisms, state snapshots, failure cases, and alternatives.

20.2.1 · Control plane vs data plane

The control plane decides configuration and placement; the data plane serves application traffic. A control-plane outage can stop new scheduling while already running workloads continue under some conditions. Avoid unnecessary synchronous control-plane dependencies in every request. Cached configuration needs an explicit validity and update policy, especially for security-sensitive changes. Draw the two paths separately so a claimed serving redundancy does not conceal a single configuration or credential dependency needed for recovery.

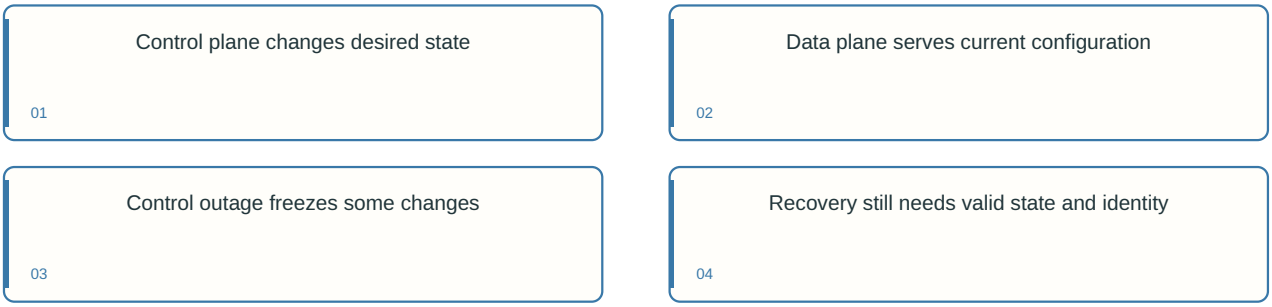


FIG. 20.2.1 **Control plane vs data plane · worked snapshots**

Control plane changes desired state · Data plane serves current configuration · Control outage freezes some changes · Recovery still needs valid state and identity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.2 · Scheduling and service discovery

A scheduler assigns work to eligible resources according to constraints; service discovery exposes reachable service endpoints. Placement decisions should include capacity, affinity, anti-affinity, and failure domains where required. A scheduled process may not yet be ready to receive traffic. Discovery can lag termination or startup, so clients and proxies still need bounded failure handling. Do not infer state consistency from successful scheduling: two replicas can run correctly while pointing at independent local databases.

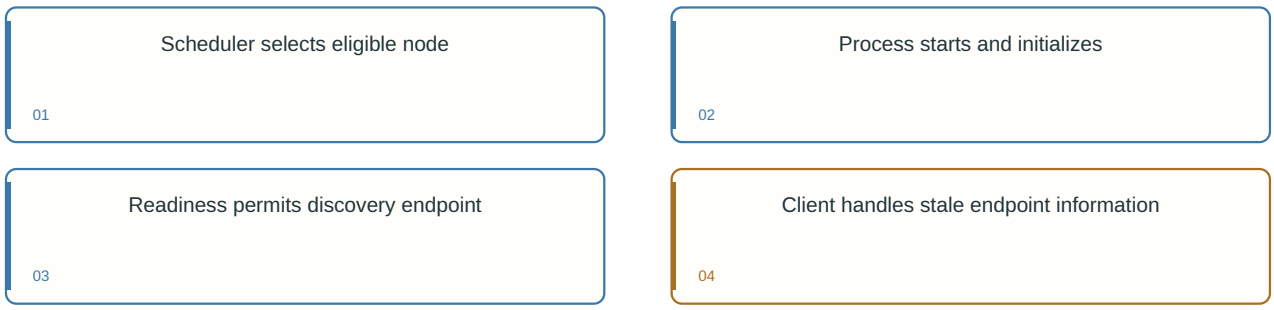


FIG. 20.2.2 **Scheduling and service discovery · worked snapshots**

Scheduler selects eligible node · Process starts and initializes · Readiness permits discovery endpoint · Client handles stale endpoint information. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.3 · Autoscaling

Autoscaling can change instances, worker concurrency, or provisioned capacity. Select signals related to the workload, preserve safe minimums, and include warmup and draining. Scaling down must stop admission, respect deadlines, and transfer durable work rather than killing unrecorded jobs. Database and provider limits can cap useful growth even if platform capacity is available. Protect total downstream concurrency and spend; an autoscaler responding to retries can otherwise amplify an incident.

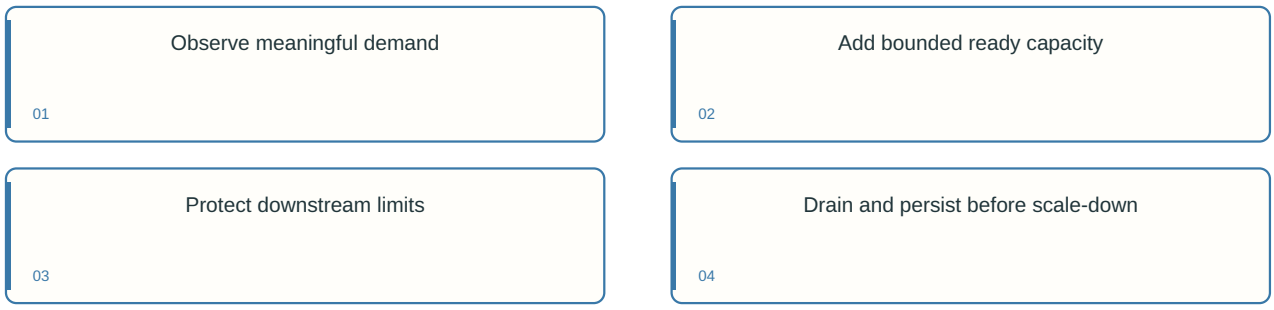


FIG. 20.2.3 **Autoscaling · worked snapshots**

Observe meaningful demand · Add bounded ready capacity · Protect downstream limits · Drain and persist before scale-down. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.4 · Configuration and secret delivery

Configuration and secrets need versioned, observable delivery with validation. A bad configuration can break every healthy replica without a code release. Validate new values, roll out gradually where appropriate, and retain a recovery path. Secrets require stronger access and logging discipline than ordinary configuration. Reload semantics matter: some values can change in place, while others require a new process or connection. Record the effective configuration version on telemetry so incidents can be correlated to actual behavior.

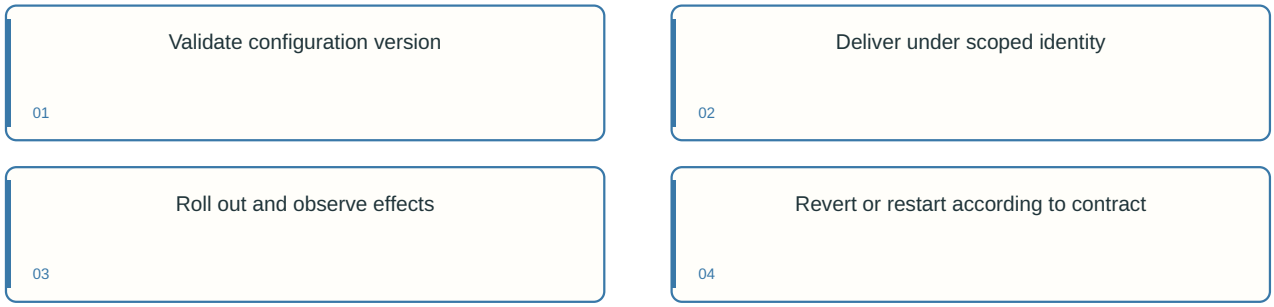


FIG. 20.2.4 **Configuration and secret delivery · worked snapshots**

Validate configuration version · Deliver under scoped identity · Roll out and observe effects · Revert or restart according to contract. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.5 · Immutable infrastructure

Immutable infrastructure replaces instances from defined artifacts rather than relying on undocumented manual changes. It improves repeatability when artifacts, configuration, and provisioning are controlled. It does not mean runtime data is immutable or that rollback is universally safe. Keep persistent state and migration strategy explicit. Emergency changes should be captured into the source of configuration or they disappear at replacement and create drift. Verify what an instance actually runs, not only what deployment intended.

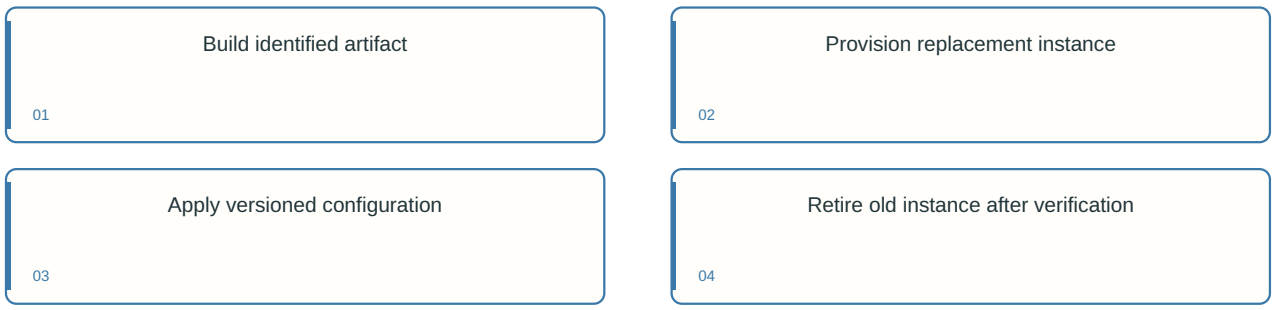


FIG. 20.2.5 **Immutable infrastructure · worked snapshots**

Build identified artifact · Provision replacement instance · Apply versioned configuration · Retire old instance after verification. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How could a rollback restore old code while making stored data or external effects less correct?

Add a compatible field, deploy readers that tolerate both versions, backfill under a bounded budget, verify, and remove old structure only after old readers retire. Test rollback before contraction and explicitly identify when rollback ceases to be safe. A canary must receive representative traffic.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Emergency changes should be captured into the source of configuration or they disappear at replacement and create drift.

Carry forward: Build identified artifact · Provision replacement instance · Apply versioned configuration · Retire old instance after verification.

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Releases, flags & schema migrations

Prerequisites: 20.2 · Control planes, scheduling & configuration 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Take a single-process application and deploy it into a reliable, observable, multi-instance production environment with safe releases.

This chapter examines releases, flags & schema migrations through concrete mechanisms, state snapshots, failure cases, and alternatives.

20.3.1 · Infrastructure as Code

Infrastructure as Code describes desired resources and policy in reviewable form. Plans help expose changes, but provider behavior, state management, privileges, and drift still matter. A harmless-looking resource replacement can delete a stateful service unless lifecycle and backups are understood. Separate secrets from ordinary tracked configuration, lock or coordinate concurrent updates, and verify applied state. The code is one input to operations; recovery also needs access to its state and credentials when infrastructure is damaged.

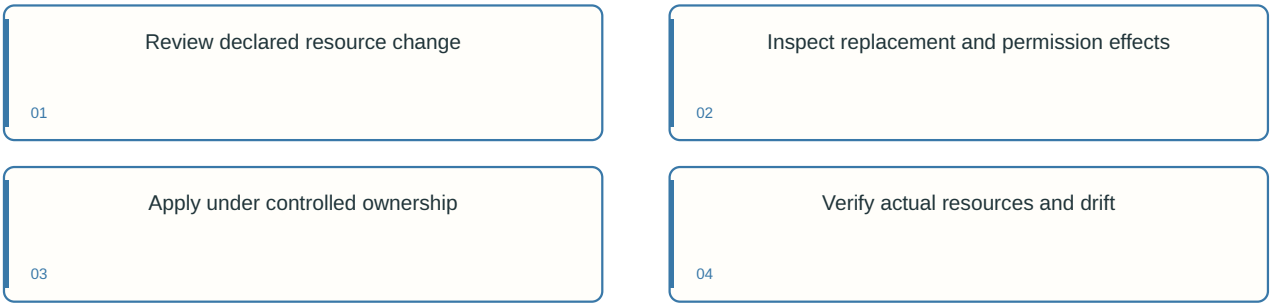


FIG. 20.3.1 **Infrastructure as Code · worked snapshots**

Review declared resource change · Inspect replacement and permission effects · Apply under controlled ownership · Verify actual resources and drift. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.2 · CI/CD

CI checks changes; delivery prepares releasable artifacts; deployment moves an artifact into an environment. A pipeline should preserve identity from source through built artifact to running version, run meaningful tests, and apply scoped permissions. An integration test passing in one environment does not prove a migration is safe during mixed-version traffic. Include the checks that address real release risks and avoid untrusted build inputs gaining release credentials. Repeatable automation reduces manual variation, but its own failures need a recovery procedure.

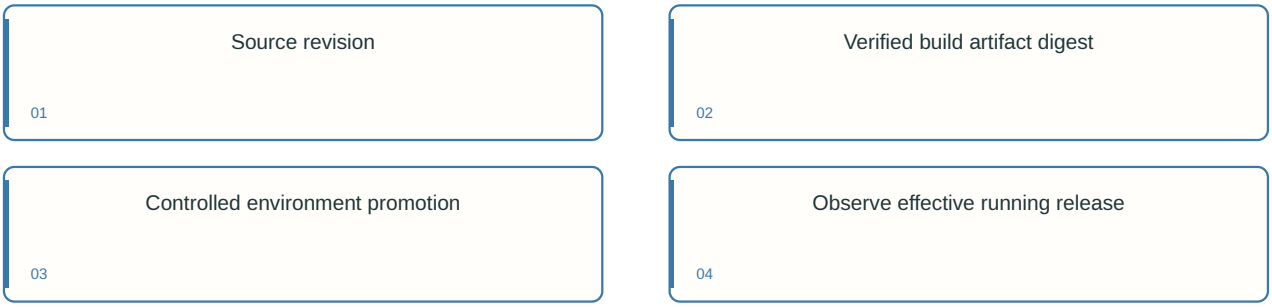


FIG. 20.3.2 **CI/CD · worked snapshots**

Source revision · Verified build artifact digest · Controlled environment promotion · Observe effective running release. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.3 · Rolling, blue-green, and canary deployments

Rolling deployment replaces instances gradually; blue-green prepares a parallel environment and switches traffic; canary exposes a controlled subset before wider rollout. All need compatible schemas, traffic drain, capacity, and verification. A canary that receives only cheap requests can miss a critical-path regression. Define cohorts and stop conditions using meaningful indicators. Blue-green duplicates runtime capacity but does not automatically duplicate or safely roll back shared mutable data. Select by failure risk and operating cost.

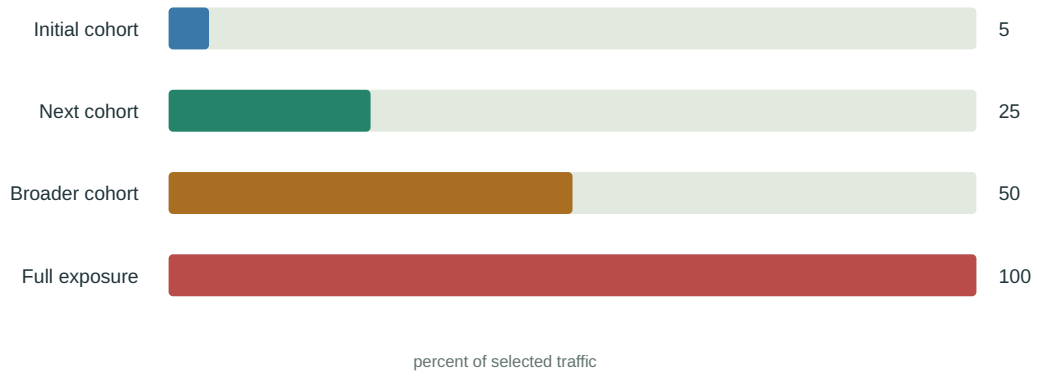


FIG. 20.3.3 **Canary exposure expands only after verification**

Percentages are an illustrative rollout plan, not an automatic safety rule. Each increase depends on meaningful representative checks and stop conditions.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.4 · Feature flags

Feature flags separate some release and exposure decisions. They need ownership, safe defaults, scope, observability, and removal policy. A flag does not replace authorization: letting the browser enable a privileged server operation is unsafe. Combinations create a testing surface, and stale flags accumulate complexity. When disabling a feature after it has written new state, ensure old code understands that state or provide a migration path. Runtime configuration is itself part of the release risk.

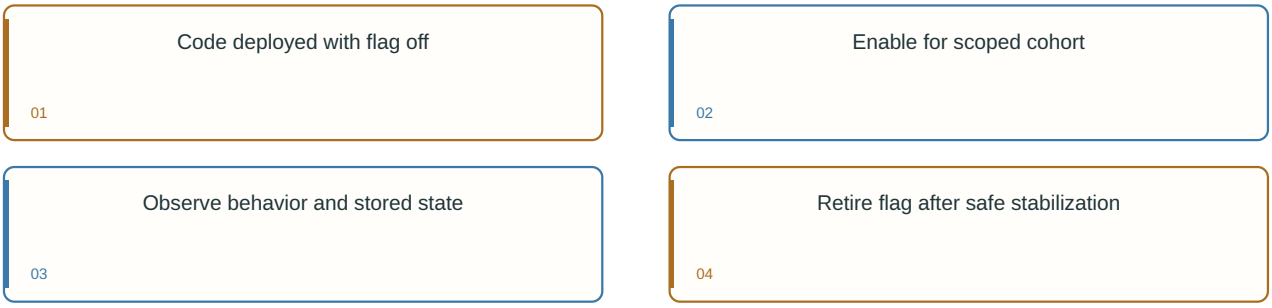


FIG. 20.3.4 **Feature flags · worked snapshots**

Code deployed with flag off · Enable for scoped cohort · Observe behavior and stored state · Retire flag after safe stabilization. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.5 · Rollbacks and database migrations

Rollback restores code or configuration only when state and contracts remain compatible. Use expand-and-contract database changes: add compatible structures, deploy code that can operate across the transition, backfill and verify, then remove obsolete fields after old readers are gone. Destructive schema changes can make old code unusable. Long backfills need rate control and resumable progress. A rollback button should identify the affected data and protocol assumptions rather than imply time can be reversed for every side effect.

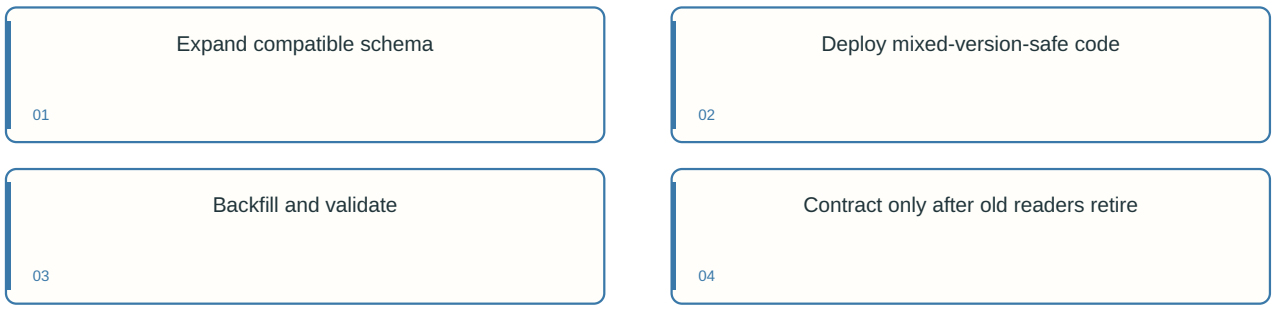


FIG. 20.3.5 **Rollbacks and database migrations · worked snapshots**

Expand compatible schema · Deploy mixed-version-safe code · Backfill and validate · Contract only after old readers retire. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.6 · Multi-environment configuration

Environment configuration should change endpoints, credentials, capacity, and declared policy without silently changing business semantics. Validate types and required fields at startup or controlled reload. Avoid a pile of environment-specific conditionals that no test environment reproduces. Use explicit configuration versions and safe defaults, especially for security and data retention. A missing value should fail clearly when it would otherwise route data to the wrong tenant, region, or storage account.

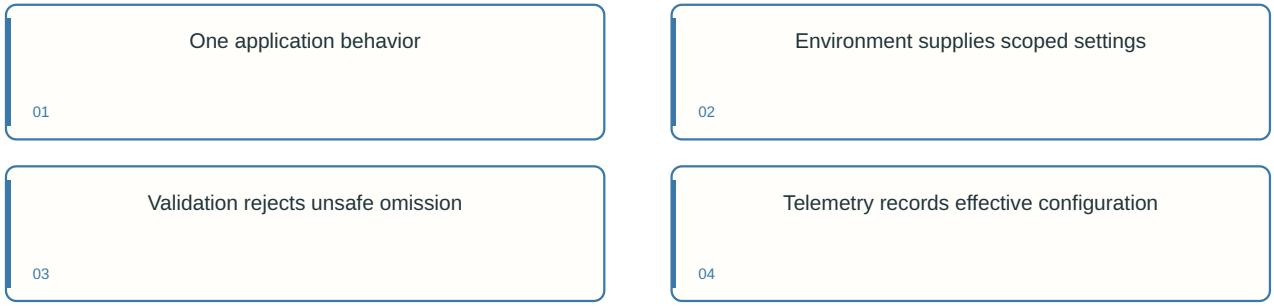


FIG. 20.3.6 **Multi-environment configuration · worked snapshots**

One application behavior · Environment supplies scoped settings · Validation rejects unsafe omission · Telemetry records effective configuration. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · How could a rollback restore old code while making stored data or external effects less correct?

Add a compatible field, deploy readers that tolerate both versions, backfill under a bounded budget, verify, and remove old structure only after old readers retire. Test rollback before contraction and explicitly identify when rollback ceases to be safe. A canary must receive representative traffic.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Use explicit configuration versions and safe defaults, especially for security and data retention.

Carry forward: One application behavior · Environment supplies scoped settings · Validation rejects unsafe omission · Telemetry records effective configuration.

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Platforms, serverless & ownership

Prerequisites: 20.3 · Releases, flags & schema migrations 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Take a single-process application and deploy it into a reliable, observable, multi-instance production environment with safe releases.

This chapter examines platforms, serverless & ownership through concrete mechanisms, state snapshots, failure cases, and alternatives.

20.4.1 · Twelve-Factor App principles

The Twelve-Factor App is methodology for service-oriented application operation: explicit dependencies and configuration, backing services, stateless processes where appropriate, logs as event streams, and related deployment practices. It is guidance rather than a universal law for every application. Stateful workloads and modern platforms require additional decisions about persistence, identity, resource limits, and recovery. Apply the intent to improve repeatability and ownership; do not claim that meeting twelve labels proves reliability or security.

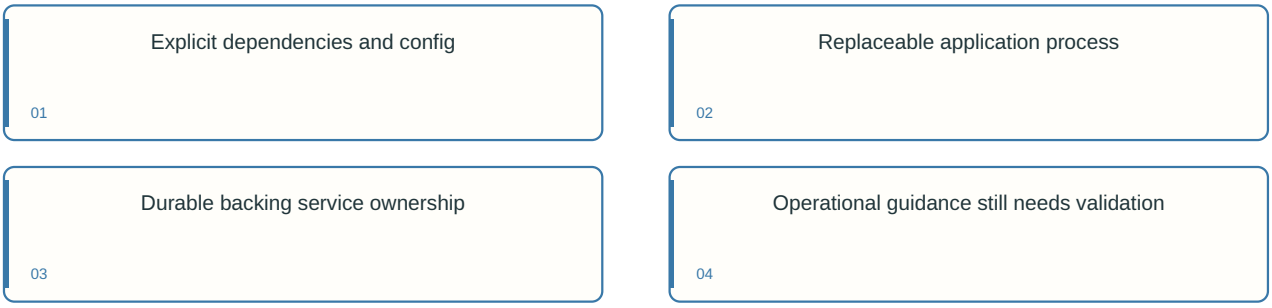


FIG. 20.4.1 **Twelve-Factor App principles · worked snapshots**

Explicit dependencies and config · Replaceable application process · Durable backing service ownership · Operational guidance still needs validation. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.2 · Serverless compute

Serverless compute delegates some provisioning and execution management to a platform while preserving limits and failure semantics. Functions can be retried or interrupted, and scaling may create too many concurrent calls to a database. Externalize durable state, bound downstream concurrency, and use stable effect identity. Compare sustained cost, startup latency, portability, and operational responsibility with container or VM options. The serverless label does not imply zero servers, infinite capacity, or freedom from observability and security design.

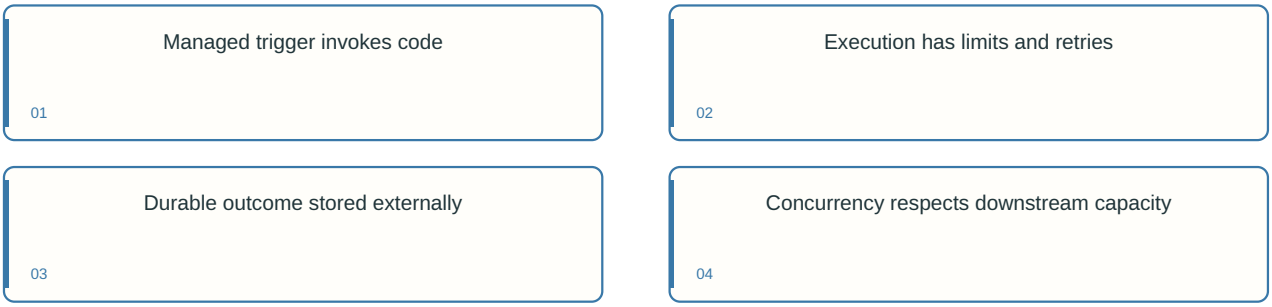


FIG. 20.4.2 **Serverless compute · worked snapshots**

Managed trigger invokes code · Execution has limits and retries · Durable outcome stored externally · Concurrency respects downstream capacity. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.3 · Cloud vs on-premises trade-offs

Cloud and on-premises choices distribute ownership differently. Compare procurement, data placement, network, managed capabilities, staff expertise, compliance needs, recovery, and exit cost under the same workload. Cloud elasticity is useful only within provisioning and quota constraints; on-premises capacity can be efficient for some predictable needs but requires equipment lifecycle and disaster planning. Avoid generic price claims without current rates and assumptions. This atlas models illustrative unit costs rather than claiming a provider is universally cheapest.

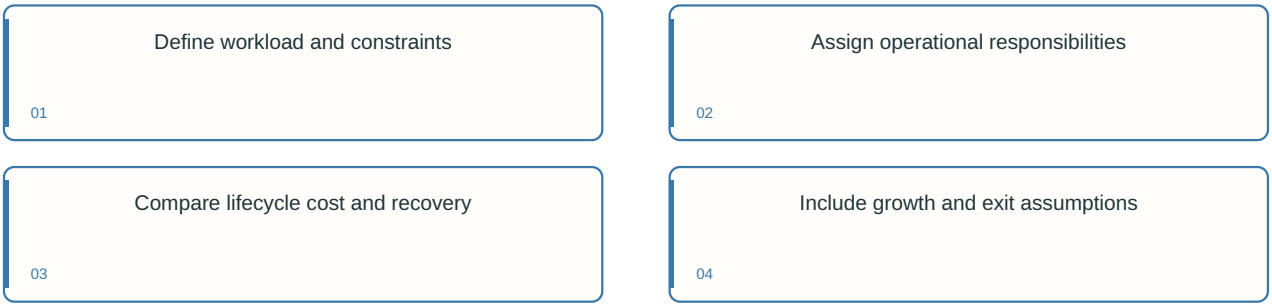


FIG. 20.4.3 **Cloud vs on-premises trade-offs · worked snapshots**

Define workload and constraints · Assign operational responsibilities · Compare lifecycle cost and recovery · Include growth and exit assumptions. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.4 · Infrastructure ownership and managed services

A managed service transfers selected tasks, not business responsibility. The provider may operate patches or replication while the application still chooses acknowledgment settings, schema, access, retention, backup validation, and regional recovery. Read the actual service contract and failure boundaries. Self-management gives control while requiring expertise and on-call work. Compare what remains your responsibility in either choice, and test client behavior under maintenance, throttling, failover, and credential changes.

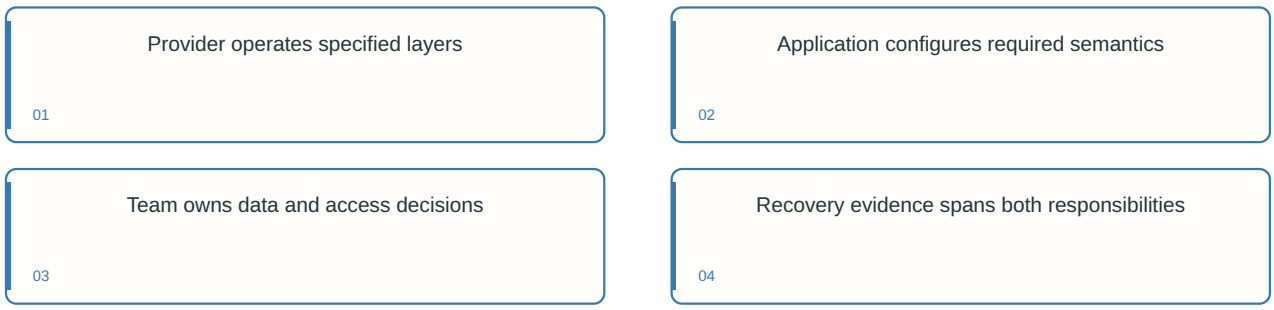


FIG. 20.4.4 **Infrastructure ownership and managed services · worked snapshots**

Provider operates specified layers · Application configures required semantics · Team owns data and access decisions · Recovery evidence spans both responsibilities. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.5 · Deployment topologies and networking

Deployment topology places processes, state, trust boundaries, routes, and failure domains on actual infrastructure. A logical module diagram cannot show whether two replicas share a host, zone, or database. Include ingress, egress, TLS termination, private paths, and operational access. A topology should explain what happens when one placement disappears and how replacement obtains state and configuration. Keep control and data flow meanings clear; every arrow should identify the protocol or dependency it represents.

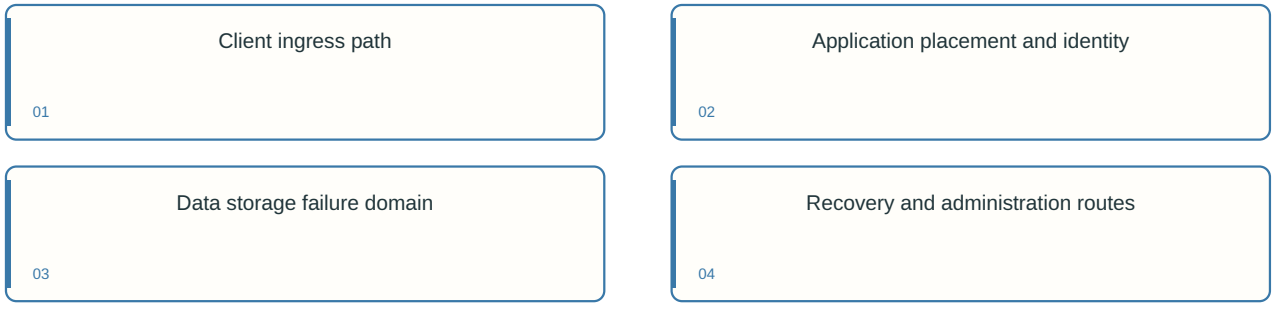


FIG. 20.4.5 **Deployment topologies and networking · worked snapshots**

Client ingress path · Application placement and identity · Data storage failure domain · Recovery and administration routes. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.6 · Platform engineering basics

Platform engineering provides reusable paths for teams to build and operate services safely. Useful capabilities include artifact delivery, identity, telemetry, resource policy, and tested deployment patterns. Treat internal developers as users and provide escape hatches with clear ownership rather than an opaque mandatory framework for every workload. Platform abstractions must expose relevant limits and failure behavior. Measure adoption, reliability, and operating burden; another layer is valuable only when it simplifies work without hiding essential decisions.

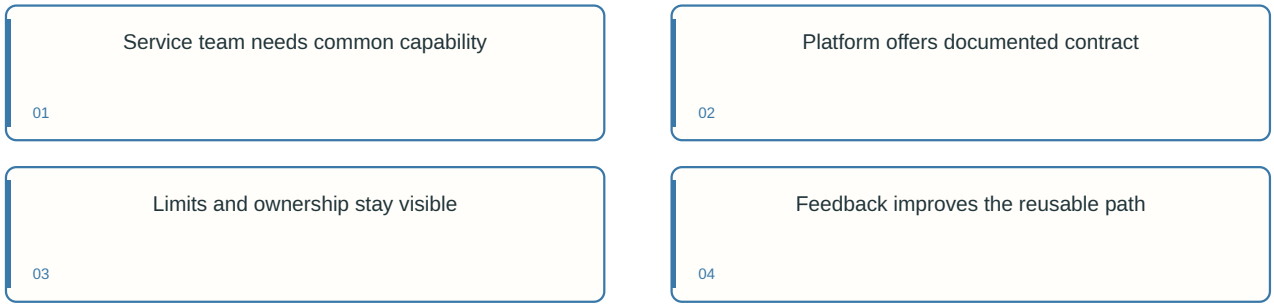


FIG. 20.4.6 **Platform engineering basics · worked snapshots**

Service team needs common capability · Platform offers documented contract · Limits and ownership stay visible · Feedback improves the reusable path. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PSEUDOCODE · SAFE RELEASE PROGRESSION

```
add_compatible_schema_field()
deploy_mixed_version_safe_code()
backfill_with_bounded_progress()
verify_old_and_new_readers()
retire_old_readers()
remove_obsolete_schema_only_when_safe()
```

Code rollback is safe only while its stored data and protocol assumptions still hold. Destructive contraction and external effects may need a forward repair instead. Include configuration, artifact identity, and runtime

readiness in the release evidence.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Architecture review · How could a rollback restore old code while making stored data or external effects less correct?

Add a compatible field, deploy readers that tolerate both versions, backfill under a bounded budget, verify, and remove old structure only after old readers retire. Test rollback before contraction and explicitly identify when rollback ceases to be safe. A canary must receive representative traffic.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Platform abstractions must expose relevant limits and failure behavior.

Carry forward: Service team needs common capability · Platform offers documented contract · Limits and ownership stay visible · Feedback improves the reusable path.

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

PART 5 / UNIT 21

Performance, Cost Optimization, and System Validation

The running example

Compare three production architectures for a service with variable traffic. Model monthly costs, p99 latency, availability, and operational complexity under explicit assumptions.

Unit 21 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

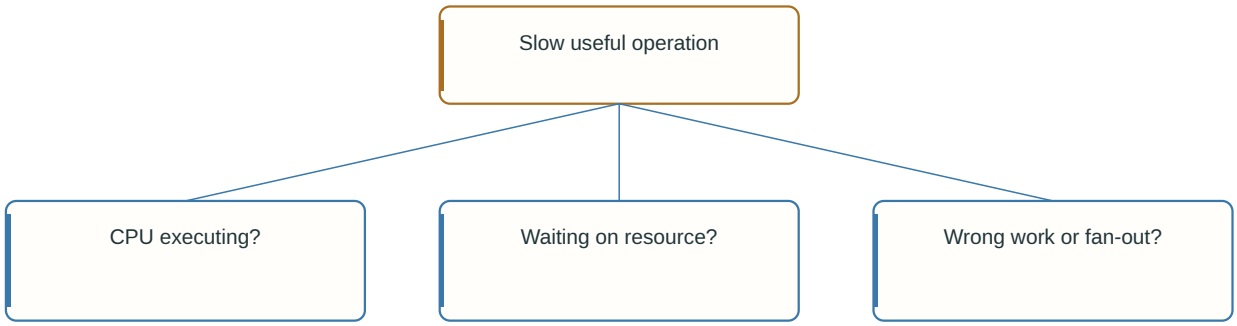


FIG. 21.0.1 **Performance bottleneck tree**

Start with a symptom, distinguish execution from waiting, then inspect the resource on the critical path.

Why this view: Each branch is a hypothesis requiring profiling, traces, and workload evidence. Author-created illustration. Numbers are illustrative unless stated otherwise.

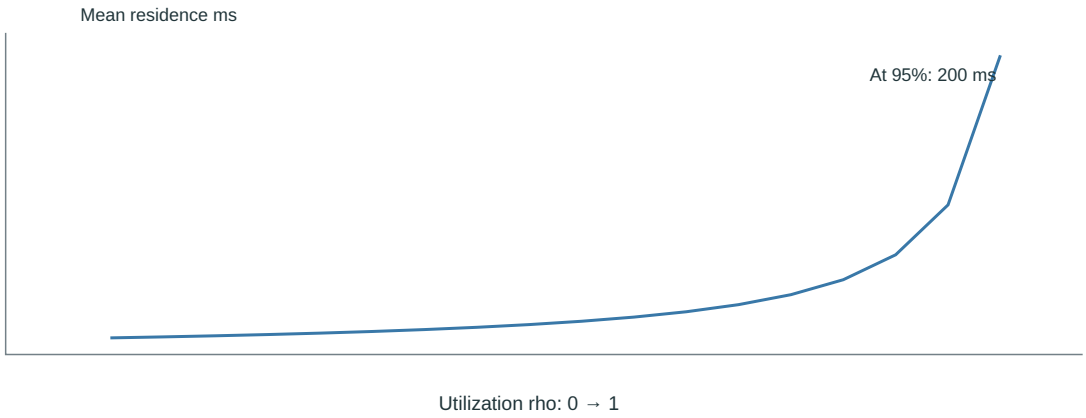


FIG. 21.0.2 **throughput-latency curve**

Stable M/M/1 model illustrates why mean residence rises near saturation; real workloads and p99 require measurement.

Why this view: Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

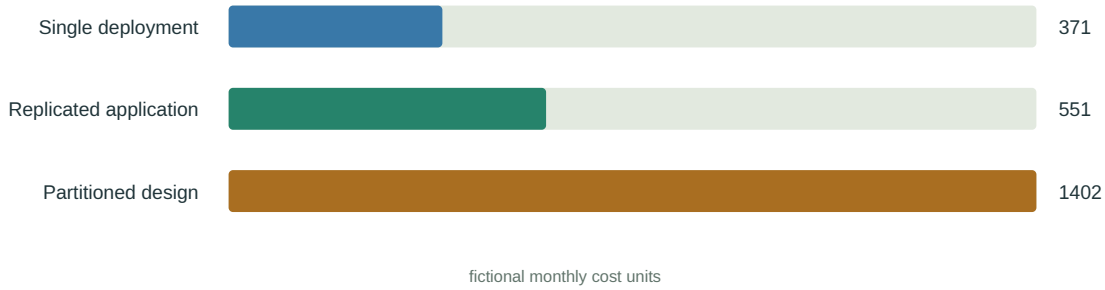


FIG. 21.0.3 **Architecture costs are not capacity guarantees**

The complete cost worksheet uses fictional monthly units. Each candidate still needs independent checks for correctness, performance, and failure requirements.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

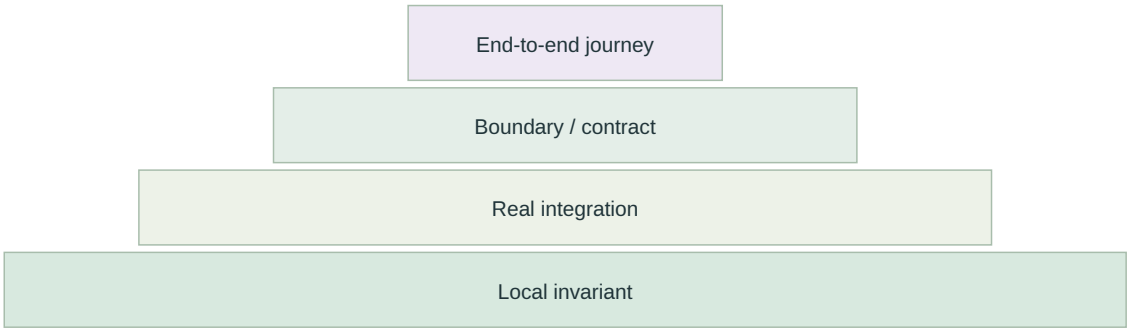


FIG. 21.0.4 **test pyramid**

The conceptual pyramid distinguishes feedback scope and cost, not a mandatory numerical ratio of tests.

Why this view: Test scope and feedback cost differ; choose by the failure evidence needed. Author-created illustration. Numbers are illustrative unless stated otherwise.

Criterion	Hard constraint?	Evidence
No duplicate local posting	Yes	Transactional invariant test
p99 target	Required objective	Representative load distribution
Recovery target	Required objective	Restore/failover drill
Monthly cost	Preference or budget	Explicit unit-cost model

FIG. 21.0.5 **design decision matrix**

Feasibility precedes preference scoring. An unmeasured objective must not be reported as achieved.

Why this view: Compare the stated dimensions under one explicit contract. Author-created illustration. Numbers are illustrative unless stated otherwise.

Budgets, bottlenecks & honest benchmarks

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Compare three production architectures for a service with variable traffic. Model monthly costs, p99 latency, availability, and operational complexity under explicit assumptions.

This chapter examines budgets, bottlenecks & honest benchmarks through concrete mechanisms, state snapshots, failure cases, and alternatives.

21.1.1 · Performance budgets

A performance budget allocates acceptable latency, resource, and cost across an operation under a named workload. An assumed 200 ms budget might reserve 30 ms for edge and network, 20 ms for queues, 100 ms for data access, and 50 ms for computation and response. These are design assumptions to validate, not measured benchmark facts. Include tails and failure cases. A budget that ignores pool wait or serialization can be exhausted before the supposed main operation begins.

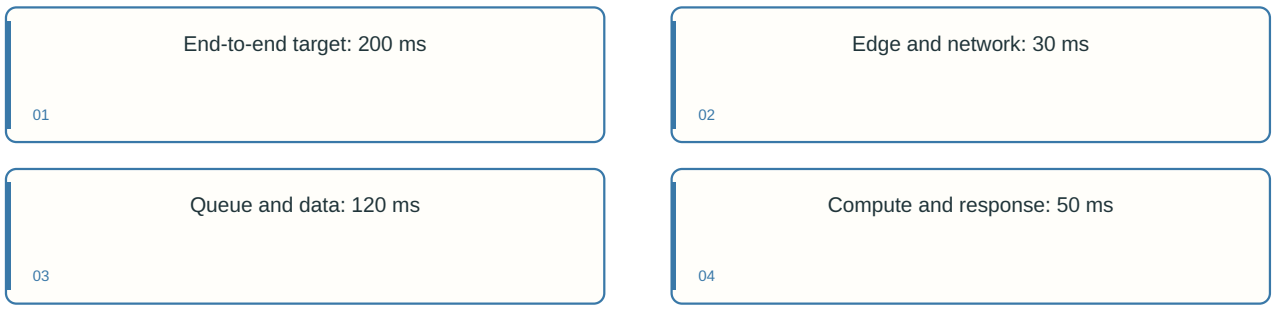


FIG. 21.1.1 **Performance budgets · worked snapshots**

End-to-end target: 200 ms · Edge and network: 30 ms · Queue and data: 120 ms · Compute and response: 50 ms. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.2 · CPU, memory, disk, network, and database bottlenecks

CPU, memory, disk, network, and databases can each limit useful work. High CPU can indicate computation or contention; low CPU can coexist with long lock or I/O waits. Memory pressure can trigger allocation stalls or termination. Follow the critical path and compare throughput with resource and queue evidence. A bottleneck tree is a set of hypotheses to test, not a justification for changing every component. Fixing one resource often exposes the next limit.

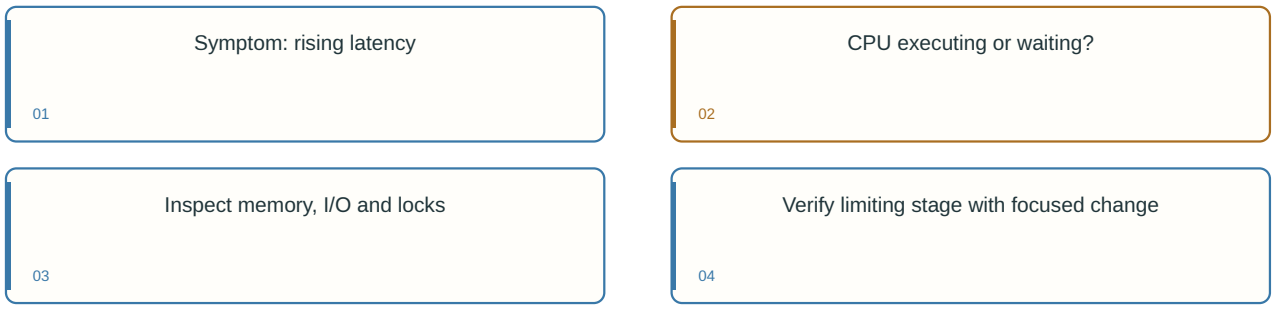


FIG. 21.1.2 **CPU, memory, disk, network, and database bottlenecks · worked snapshots**

Symptom: rising latency · CPU executing or waiting? · Inspect memory, I/O and locks · Verify limiting stage with focused change. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.3 · Profiling and benchmarking

Profiling identifies where runtime work occurs; benchmarking compares behavior under a controlled workload. Define hardware or capacity, data distribution, warmup, concurrency, duration, success criteria, and versions. A microbenchmark of one function does not establish application throughput. Include variability and resource cost, and compare like-for-like service promises. If a result changes after a dependency or schema change, preserve enough setup information to reproduce the explanation rather than presenting a single impressive number.

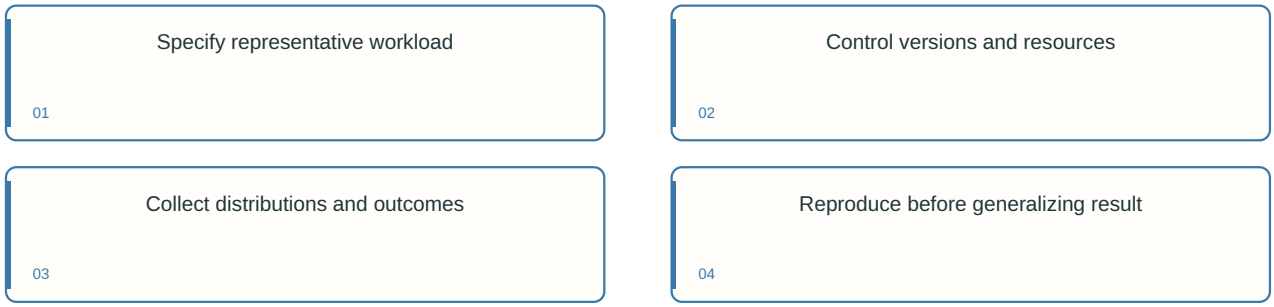


FIG. 21.1.3 **Profiling and benchmarking · worked snapshots**

Specify representative workload · Control versions and resources · Collect distributions and outcomes · Reproduce before generalizing result. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.4 · Load, stress, spike, and soak testing

Load tests exercise expected demand; stress tests explore limits; spike tests expose response to abrupt change; soak tests reveal long-duration issues such as leaks or compaction. Each answers a different question. Use a load generator that does not silently reduce offered load when the system slows unless that closed-loop behavior matches real clients. Check generator capacity and include errors, abandoned requests, and queue age. A high request count with wrong business results is not a successful performance test.

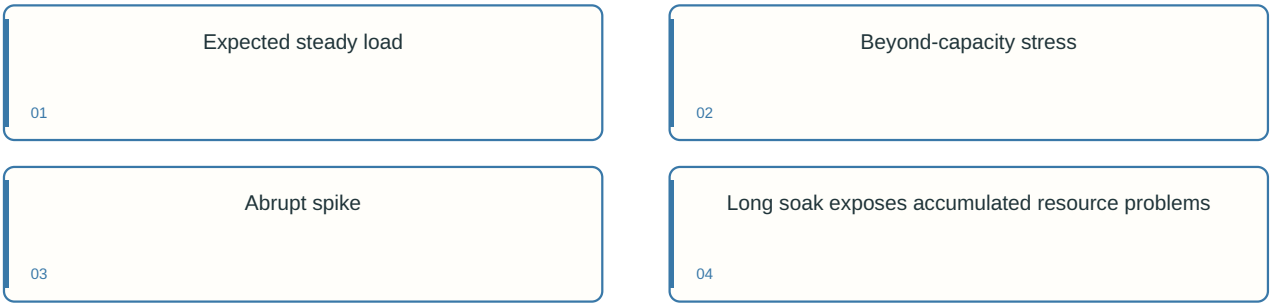


FIG. 21.1.4 **Load, stress, spike, and soak testing · worked snapshots**

Expected steady load · Beyond-capacity stress · Abrupt spike · Long soak exposes accumulated resource problems. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.5 · Benchmark methodology and statistical pitfalls

Benchmark pitfalls include unrealistic data, warmed caches without disclosure, coordinated omission, tiny samples, missing failures, and incompatible versions. Coordinated omission can occur when the generator waits on a slow response before issuing work that should have arrived independently, understating delay. Report distributions and methodology, not fabricated precision. Change one factor or use a justified experimental design. A result is evidence for the tested conditions and does not automatically extrapolate to a new traffic mix or failure state.

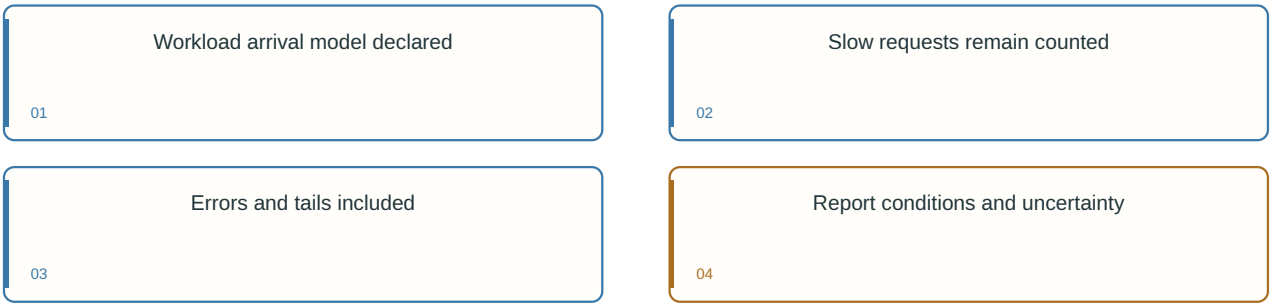


FIG. 21.1.5 **Benchmark methodology and statistical pitfalls · worked snapshots**

Workload arrival model declared · Slow requests remain counted · Errors and tails included · Report conditions and uncertainty. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.6 · Tail latency analysis

Tail analysis examines distributions and critical dependencies rather than only means. Distinguish a rare expensive route from a broad slowdown, and separate queue wait, service time, network, and fan-out. Merging percentile summaries incorrectly can hide fleet tails. Percentile estimates require enough samples for the question being asked. A p99 regression from 200 ms to five seconds should be investigated with population metrics and representative traces, not explained from one anecdotal slow request.

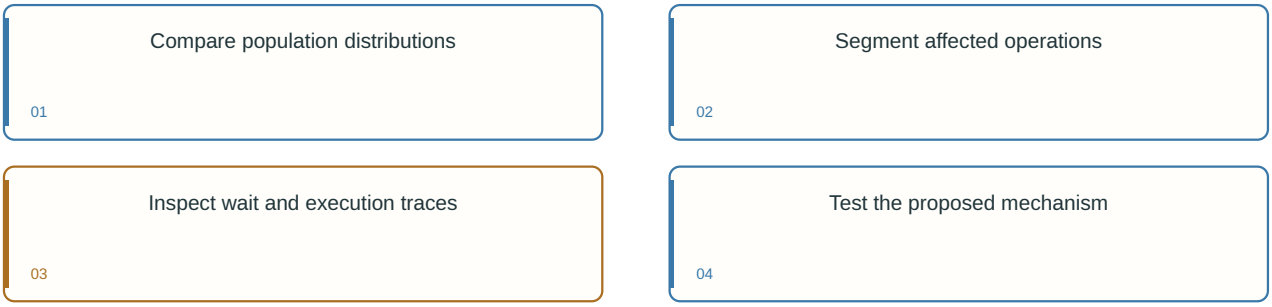


FIG. 21.1.6 **Tail latency analysis · worked snapshots**

Compare population distributions · Segment affected operations · Inspect wait and execution traces · Test the proposed mechanism. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What evidence would justify accepting a more expensive architecture as the simpler complete operating system?

cost_options() totals 371, 551, and 1402 fictional monthly units. These are cost inputs only, not measured capacities or reliability. Compare each with original latency, failure, correctness, and operating constraints; reject an infeasible cheap option before ranking cost efficiency.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Percentile estimates require enough samples for the question being asked.

Carry forward: Compare population distributions · Segment affected operations · Inspect wait and execution traces · Test the proposed mechanism.

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Capacity, architecture & cost per outcome

Prerequisites: 21.1 · Budgets, bottlenecks & honest benchmarks 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Compare three production architectures for a service with variable traffic. Model monthly costs, p99 latency, availability, and operational complexity under explicit assumptions.

This chapter examines capacity, architecture & cost per outcome through concrete mechanisms, state snapshots, failure cases, and alternatives.

21.2.1 · Queue saturation and throughput ceilings

Throughput approaches a ceiling when a limiting resource cannot complete work faster. Additional offered traffic then becomes rejection or queue growth. A queue is not extra service capacity. Find the goodput knee by observing successful outcomes, latency, and saturation under increasing demand, then reserve margin for variability and failure. A measured maximum in a short benchmark is not a safe production target. Include maintenance, background work, and the surviving capacity of the promised failure scenario.

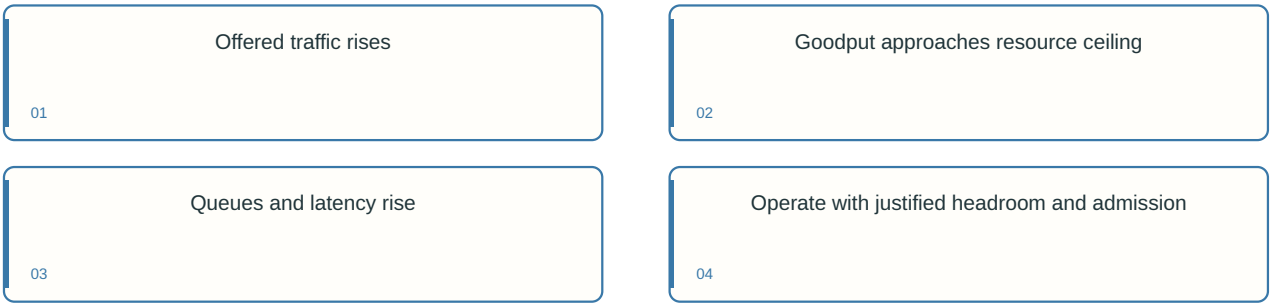


FIG. 21.2.1 **Queue saturation and throughput ceilings · worked snapshots**

Offered traffic rises · Goodput approaches resource ceiling · Queues and latency rise · Operate with justified headroom and admission. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.2 · Performance tuning vs architecture changes

Tuning improves an implementation within its architecture; an architectural change alters ownership, placement, or major interaction. An index, batch size, or shorter transaction may resolve a bottleneck with less migration risk than sharding. Conversely, an unavoidably centralized invariant can require a different business or partitioning model. Measure before and after, and count correctness, operations, and migration cost alongside speed. The most sophisticated topology is not automatically the most defensible intervention.

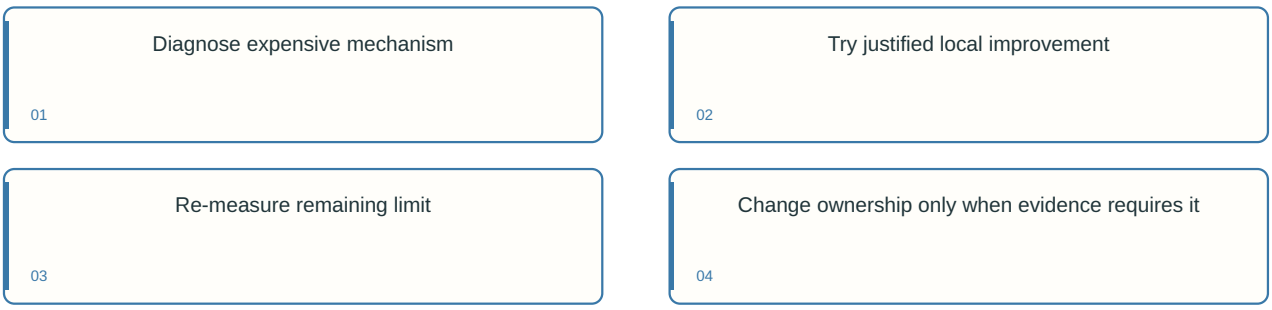


FIG. 21.2.2 **Performance tuning vs architecture changes · worked snapshots**

Diagnose expensive mechanism · Try justified local improvement · Re-measure remaining limit · Change ownership only when evidence requires it. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.3 · Horizontal scaling economics

Horizontal scaling adds coordination, baseline replicas, discovery, and often network traffic. Its cost curve depends on utilization and shared bottlenecks. Compare a larger single deployment, a replicated stateless tier with one data owner, and a partitioned architecture under identical workload and objectives. The last may cost more at small scale while extending the eventual ceiling. Use explicit illustrative prices if actual quotes are unavailable, and keep failure headroom and operational labor assumptions visible.

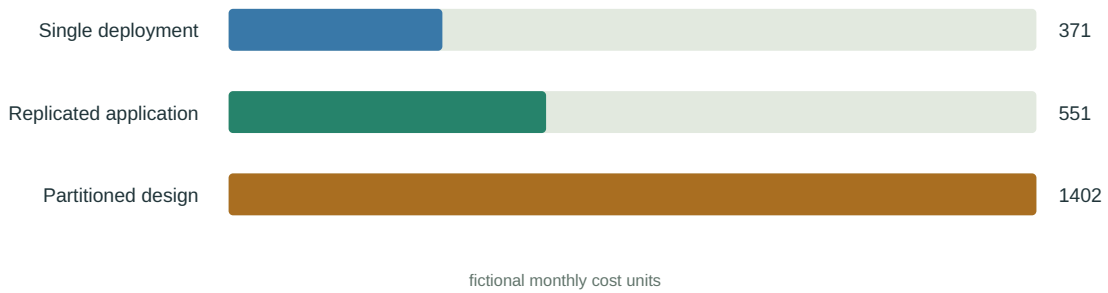


FIG. 21.2.3 **Architecture costs are not capacity guarantees**

The complete cost worksheet uses fictional monthly units. Each candidate still needs independent checks for correctness, performance, and failure requirements.

Why this view: Compare calculated magnitudes under the same explicit model; bar length is not a benchmark. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.4 · Compute, storage, networking, and telemetry costs

Total cost includes compute, storage, replicas, indexes, backups, networking, requests, telemetry, and operational work. A large response sent frequently can dominate egress despite cheap computation. Trace the cost path just as you trace latency. A cache may reduce origin compute while increasing memory and invalidation work. Do not quote live provider pricing without current verification; this atlas's worked prices are hypothetical arithmetic inputs and should be replaced with real quotes for a procurement decision.

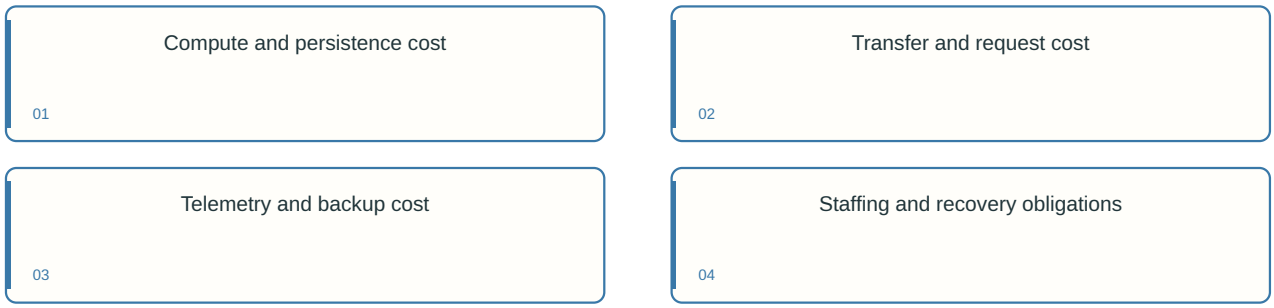


FIG. 21.2.4 **Compute, storage, networking, and telemetry costs · worked snapshots**

Compute and persistence cost · Transfer and request cost · Telemetry and backup cost · Staffing and recovery obligations. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.5 · Cost per request or transaction

Cost per request can obscure the business outcome if retries or failures inflate the denominator. Prefer cost per successful order, delivered document, or another meaningful unit, while still analyzing technical costs. With an assumed monthly cost of 3000 units and 150000 successful outcomes, unit cost is 0.02. If half the attempts fail, dividing by all attempts understates the cost of useful work. Define period, allocation, and included operating expenses.

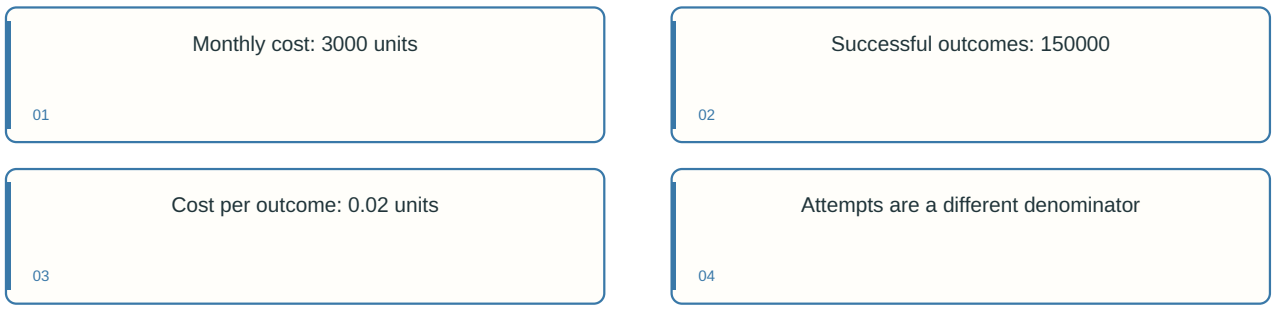


FIG. 21.2.5 **Cost per request or transaction · worked snapshots**

Monthly cost: 3000 units · Successful outcomes: 150000 · Cost per outcome: 0.02 units · Attempts are a different denominator. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.6 · Capacity reservation vs elastic capacity

Reserved capacity can lower some unit rates in exchange for commitment and utilization risk; elastic capacity handles variable demand with provisioning delay and possible premiums. Compare baseline demand, peak duration, growth uncertainty, and failure capacity. A reservation sized for an occasional peak can sit idle, while purely reactive elasticity can miss a short burst. Mixed strategies often fit, but the correct balance requires measured traffic and current commercial terms rather than a universal recommendation.

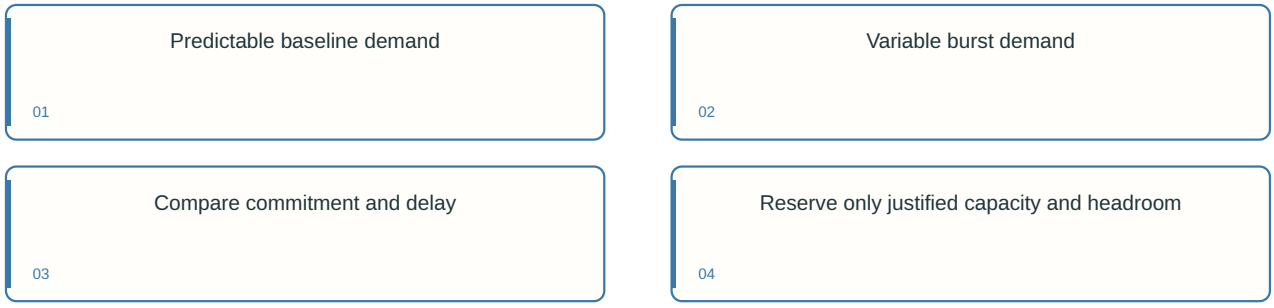


FIG. 21.2.6 **Capacity reservation vs elastic capacity · worked snapshots**

Predictable baseline demand · Variable burst demand · Compare commitment and delay · Reserve only justified capacity and headroom. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

A complete cost worksheet with fictional inputs

Monthly rates below are invented teaching inputs, not provider quotations. Capacity, p99, and availability are evaluation dimensions whose results must be measured; this cost table cannot establish them.

Fictional monthly cost units; compare only after checking requirements

Architecture	Compute	Data	Cache	Storage	Transfer	Telemetry	Total
Single deployment	40	300	0	1	20	10	371
Replicated application	160	300	50	1	20	20	551
Partitioned design	320	900	100	2	40	40	1402

Compare latency, availability, and operating work under stated assumptions

Construct a 100-request trace for each option: single deployment has 98 observations at 190 ms, one at 900 ms, and one at 1,400 ms; replicated application has 98 at 140 ms, one at 450 ms, and one at 800 ms; partitioned design has 98 at 110 ms, one at 600 ms, and one at 1,600 ms. Nearest-rank p99 is observation 99 after sorting. These synthetic traces deliberately show that lower common-case latency can coexist with worse tail latency. They predict no real deployment.

For a separate steady-state availability model, assume independent app-instance availability of 0.99 and data-owner availability of 0.999. An interchangeable pool of k apps is available with probability 1 − 0.01^k. The single and replicated options require one data owner. The partitioned operation in this example requires all three independent owners, so multiply by 0.999^3. Shared outages, overload, failover time, repair, and correlated failures are omitted; these probabilities are not SLO commitments.

Illustrative comparison; latency traces, availability probabilities, and operating judgments are assumptions

Architecture	Monthly cost units	Constructed p99	Modeled availability	Operational complexity judgment
Single: 1 app, 1 data owner	371	900 ms	98.901%	Lower: one app release and one data owner

Architecture	Monthly cost units	Constructed p99	Modeled availability	Operational complexity judgment
Replicated: 4 apps, 1 data owner	551	450 ms	99.899999%	Medium: rolling releases, shared cache, pool capacity and failover
Partitioned: 8 apps, 3 required data owners	1,402	600 ms	99.700300%	Higher: routing, rebalancing, multiple owners and partial recovery

For a per-key request touching only one shard, the all-three-owners availability expression does not apply. Replication with an actual quorum or failover protocol also needs a different model. The calculation is executable as architecture_comparison() in the mechanism collection. Choose a candidate against the workload and targets, then test tail latency, degraded capacity, restoration, and operating burden. Cost arithmetic alone cannot establish those properties.

▼ Architecture review · What evidence would justify accepting a more expensive architecture as the simpler complete operating system?

cost_options() totals 371, 551, and 1402 fictional monthly units. These are cost inputs only, not measured capacities or reliability. Compare each with original latency, failure, correctness, and operating constraints; reject an infeasible cheap option before ranking cost efficiency.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

A reservation sized for an occasional peak can sit idle, while purely reactive elasticity can miss a short burst.

Carry forward: Predictable baseline demand · Variable burst demand · Compare commitment and delay · Reserve only justified capacity and headroom.

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Efficiency and a useful test portfolio

Prerequisites: 21.2 · Capacity, architecture & cost per outcome 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Compare three production architectures for a service with variable traffic. Model monthly costs, p99 latency, availability, and operational complexity under explicit assumptions.

This chapter examines efficiency and a useful test portfolio through concrete mechanisms, state snapshots, failure cases, and alternatives.

21.3.1 · Cost-performance optimization

Cost-performance optimization seeks a feasible point under service, security, and maintainability constraints. A cheaper design that misses the latency objective or loses acknowledged payments is not equivalent. Vary cache size, batching, compression, placement, and instance shape with measurements. Beware local improvements that increase downstream cost, such as compressed writes requiring excessive decompression on every read. Record the workload and objective so future changes do not reuse an optimization outside the conditions that made it sensible.

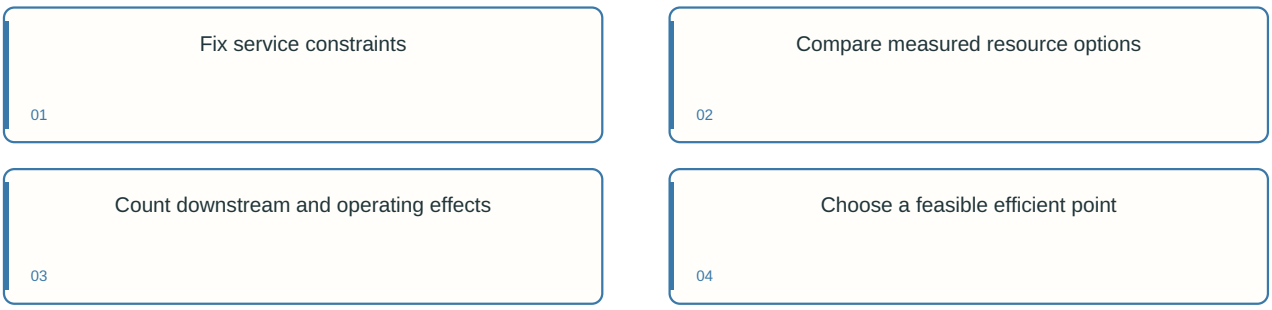


FIG. 21.3.1 **Cost-performance optimization · worked snapshots**

Fix service constraints · Compare measured resource options · Count downstream and operating effects · Choose a feasible efficient point. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.2 · Sustainability and resource efficiency

Resource efficiency includes avoiding idle work, unnecessary transfer, redundant computation, and over-retention. Sustainability depends on more than a single energy or carbon estimate; hardware utilization, region, time, lifecycle, and data quality affect claims. Do not invent environmental measurements. Useful engineering steps include bounding retries, deleting unused data, right-sizing capacity, and batching appropriate background work without violating deadlines. Evaluate the complete effect so saving CPU does not create much larger network or storage demand.

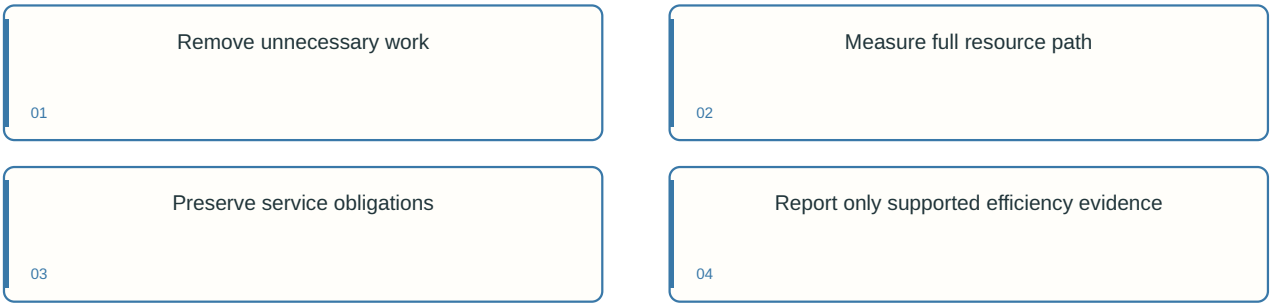


FIG. 21.3.2 **Sustainability and resource efficiency · worked snapshots**

Remove unnecessary work · Measure full resource path · Preserve service obligations · Report only supported efficiency evidence. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.3 · Unit, integration, contract, end-to-end, and performance tests

Unit tests isolate local behavior; integration tests verify real collaborators; contract tests verify boundary expectations; end-to-end tests exercise complete journeys. A test pyramid is guidance about feedback cost and scope, not a mandated numerical ratio. Use a fake provider for business decline logic and real database tests for atomicity. A large suite that mirrors private method calls can impede refactoring while missing the lost-response history. Choose tests by the failure each would detect and the evidence it provides.

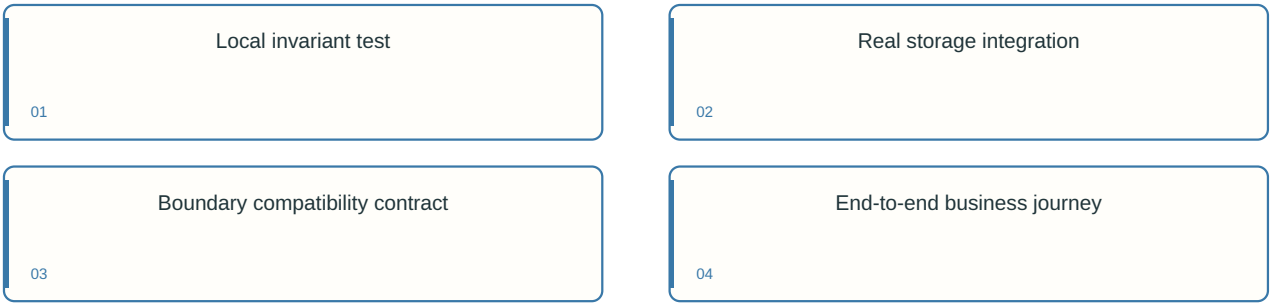


FIG. 21.3.3 **Unit, integration, contract, end-to-end, and performance tests · worked snapshots**

Local invariant test · Real storage integration · Boundary compatibility contract · End-to-end business journey. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.4 · Fault-injection tests

Fault injection tests behavior when a dependency is slow, unavailable, inconsistent, or interrupted. Place faults at meaningful boundaries: after database commit but before acknowledgment, during partition transfer, or after a worker lease expires. Killing a process only before work starts misses the dangerous windows. Define acceptable outcomes and stop conditions. Combine fault evidence with data reconciliation; an application that recovers request serving while silently losing jobs has not passed the full business contract.

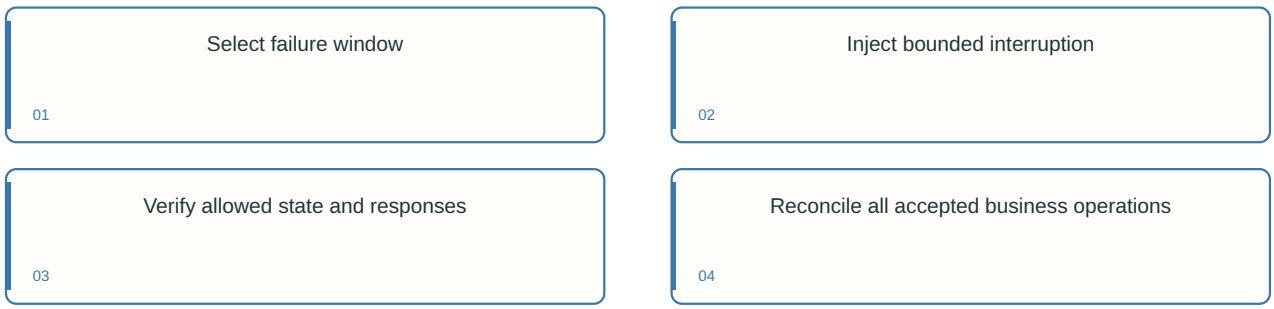


FIG. 21.3.4 **Fault-injection tests · worked snapshots**

Select failure window · Inject bounded interruption · Verify allowed state and responses · Reconcile all accepted business operations. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Architecture review · What evidence would justify accepting a more expensive architecture as the simpler complete operating system?

cost_options() totals 371, 551, and 1402 fictional monthly units. These are cost inputs only, not measured capacities or reliability. Compare each with original latency, failure, correctness, and operating constraints; reject an infeasible cheap option before ranking cost efficiency.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

Define acceptable outcomes and stop conditions.

Carry forward: Select failure window · Inject bounded interruption · Verify allowed state and responses · Reconcile all accepted business operations.

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Fitness functions and design validation

Prerequisites: 21.3 · Efficiency and a useful test portfolio 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Compare three production architectures for a service with variable traffic. Model monthly costs, p99 latency, availability, and operational complexity under explicit assumptions.

This chapter examines fitness functions and design validation through concrete mechanisms, state snapshots, failure cases, and alternatives.

21.4.1 · Data consistency validation

Data validation compares source and destination identities, versions, aggregates, and domain invariants. Row counts alone cannot detect swapped ownership, stale updates, or equal-count omissions and duplicates. For a ledger, validate balanced postings and unique external operation identity; for a search index, compare source version and deletion state. Run checks after migration, replay, and recovery. Sampling can reduce cost but weakens the claim, so state whether evidence covers all records or only a sample.

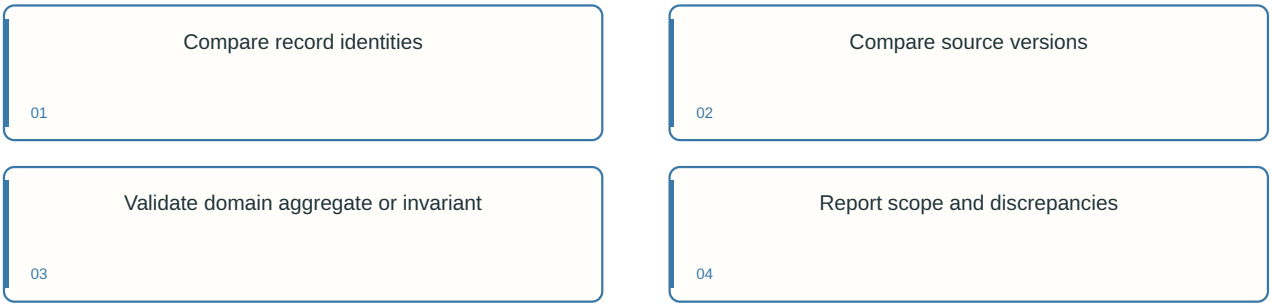


FIG. 21.4.1 **Data consistency validation · worked snapshots**

Compare record identities · Compare source versions · Validate domain aggregate or invariant · Report scope and discrepancies. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.2 · Architecture fitness functions

Architecture fitness functions continuously test required properties such as allowed imports, tenant-scoped queries, API compatibility, or recovery time. A useful function protects a reasoned architectural characteristic without freezing every incidental class relationship. Some properties need runtime drills or manual review rather than a static rule. Keep tests aligned with current decisions and update them when an ADR legitimately changes the architecture. Passing a collection of checks is evidence for their scope, not a universal architecture certification.

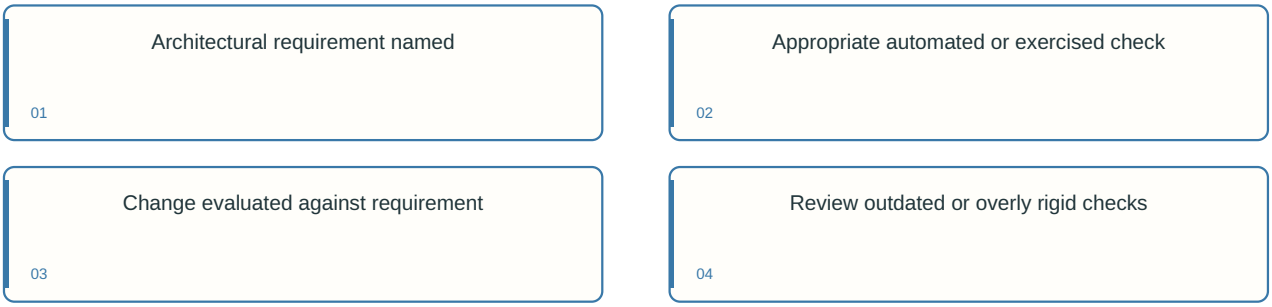


FIG. 21.4.2 **Architecture fitness functions · worked snapshots**

Architectural requirement named · Appropriate automated or exercised check · Change evaluated against requirement · Review outdated or overly rigid checks. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.3 · Regression testing

Regression testing preserves important behavior as implementations change. Include error, concurrency, compatibility, and recovery histories that have caused harm, not only the original happy path. Use representative data and maintain test meaning during refactoring. A performance regression test needs stable enough conditions and tolerances to avoid noise-driven failures. When a requirement changes, update the expected behavior explicitly rather than deleting a failing test without explaining which contract is now different.

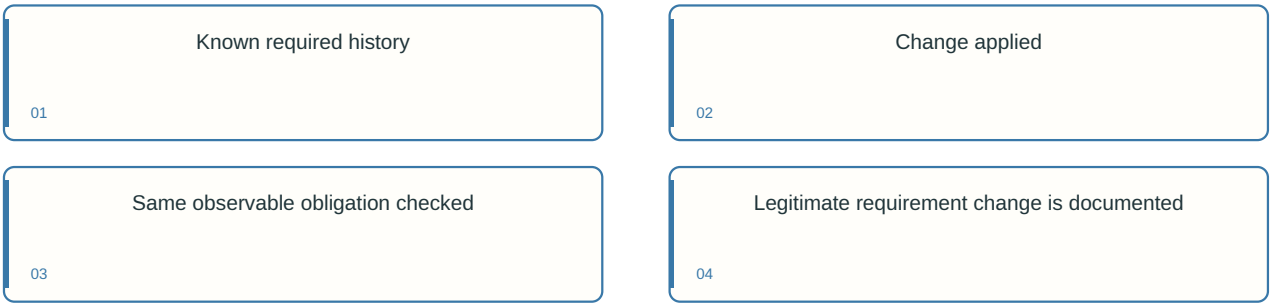


FIG. 21.4.3 **Regression testing · worked snapshots**

Known required history · Change applied · Same observable obligation checked · Legitimate requirement change is documented. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.4 · Evaluating a design against its original requirements

Validate the design against its original requirements using a traceable evidence matrix. Every promise should identify a test, measurement, review, or unresolved assumption. Compare functional behavior, latency distributions, capacity, recovery, security, cost, and operational burden under the stated workload. A diagram review and a load test provide different evidence. If a target remains unmeasured, label it as an objective rather than reporting it as achieved. This closes the loop from design hypothesis to defensible engineering decision.

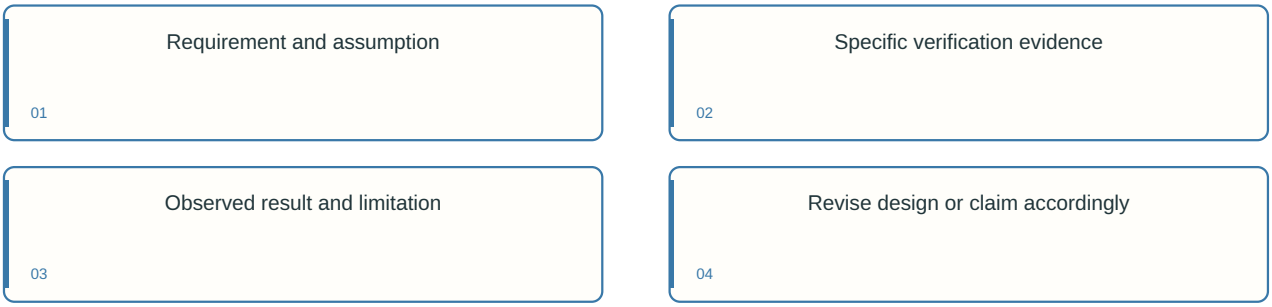


FIG. 21.4.4 **Evaluating a design against its original requirements · worked snapshots**

Requirement and assumption · Specific verification evidence · Observed result and limitation · Revise design or claim accordingly. These labeled observations explain the mechanism discussed above; cards do not imply independent processes or network calls.

Why this view: Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name. Author-created illustration. Numbers are illustrative unless stated otherwise.

Worked implementation · connect the unit

PYTHON · FICTIONAL COMPARISON

```
from mechanisms import cost_options
totals = {name:sum(parts.values()) for name,parts in cost_options().items()}
assert totals == {"single":371, "replicated_app":551, "partitioned":1402}
```

These fictional monthly totals do not establish equivalent reliability or capacity. Filter infeasible architectures before cost ranking. The complete case studies show the latency, consistency, security, and recovery checks required alongside the arithmetic.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · CALCULATED TEACHING MODEL

Compare cost per useful outcome

- Single deployment: 371 monthly units / 150,000 successful outcomes = **0.002473** units/outcome.
- Replicated application: 551 monthly units / 150,000 successful outcomes = **0.003673** units/outcome.
- Partitioned design: 1402 monthly units / 150,000 successful outcomes = **0.009347** units/outcome.

Check feasibility and comparable objectives before ranking costs.

Fictional monthly totals: single 371, replicated application 551, partitioned 1402. Capacity, p99, availability, and invariant feasibility are not established by these totals. This is not live provider pricing.

▼ Architecture review · What evidence would justify accepting a more expensive architecture as the simpler complete operating system?

cost_options() totals 371, 551, and 1402 fictional monthly units. These are cost inputs only, not measured capacities or reliability. Compare each with original latency, failure, correctness, and operating constraints; reject an infeasible cheap option before ranking cost efficiency.

Use the failure histories and assumptions above to defend an answer. Distinguish the local mechanism demonstrated here from the full system boundary in the linked complete design.

CHECK THE BOUNDARY

If a target remains unmeasured, label it as an objective rather than reporting it as achieved.

Carry forward: Requirement and assumption · Specific verification evidence · Observed result and limitation · Revise design or claim accordingly.

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

PART 6 / UNIT 22

End-to-End Design, Architecture Reviews, and Evolution

The running example

Twelve complete architectural designs with a common 22-point review protocol.

Unit 22 assessment → Explained solutions →

The complete review protocol

1. Functional requirements
2. Nonfunctional requirements
3. Explicit assumptions and workload numbers
4. Naive initial architecture
5. Core API contracts
6. Data model and schema
7. Low-level component organization
8. High-level system architecture
9. Data and control flows
10. Capacity estimates
11. End-to-end latency estimates
12. Failure scenarios
13. Scalability improvements
14. Consistency requirements
15. Security model
16. Deployment approach
17. Observability design
18. Infrastructure cost considerations
19. Alternative architectures and trade-offs
20. Architecture evolution from small scale to large scale
21. Design review identifying weaknesses and improvements
22. A short implementation exercise

Each of the twelve designs above contains every section, including the implementation challenge. Optional next subjects: Search engine. · Cloud storage service. · Distributed cache. · API gateway. · Metrics collection platform. · Multi-tenant SaaS. · Real-time fraud detection. · IoT ingestion platform..

URL Shortener

Prerequisites: API Design and Communication Contracts, Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a short link for an authorized user, redirect a known code, expire or revoke a link, and expose owner-only management. Preserve the original target and creation identity. Abuse review and redirect correctness are core behavior; analytics is an asynchronous secondary outcome that must not make a valid redirect wait for an analytics database.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.1.1 · Functional requirements

Create a short link for an authorized user, redirect a known code, expire or revoke a link, and expose owner-only management. Preserve the original target and creation identity. Abuse review and redirect correctness are core behavior; analytics is an asynchronous secondary outcome that must not make a valid redirect wait for an analytics database.

22.1.2 · Nonfunctional requirements

Target p99 redirect latency below 100 ms under the assumed supported peak, retain acknowledged mappings through a single storage-node failure under the selected replication contract, and reject unsafe management access. Expiry and revocation need a specified freshness bound; a long-lived cached redirect cannot simultaneously promise immediate revocation without an additional invalidation policy.

22.1.3 · Explicit assumptions and workload numbers

Assume 100 creations/s, 10000 redirects/s average, a tenfold redirect peak, and 200 stored bytes per mapping before indexes and replicas. New raw mappings consume 100×86400×200 = 1.728 GB/day. Assume a 500-byte redirect response for bandwidth modeling. These assumptions describe this example only; measure actual record and response sizes before provisioning.

22.1.4 · Naive initial architecture

Begin with one web process, one relational mapping table, a uniqueness constraint on code, and a background analytics job. This satisfies a small workload and makes uniqueness easy to enforce locally. Its limits are application-host failure, authoritative lookup capacity, and potentially database write or index pressure. Do not add shards before measuring which resource limits useful traffic.

22.1.5 · Core API contracts

POST /links accepts target, expiry, and an idempotency key; GET /{code} returns the documented redirect or expiry result; DELETE /links/{id} requires owner authorization. Restrict allowed URL schemes and validate syntax without fetching arbitrary internal URLs. Reusing an operation key with a different target conflicts; a lost creation reply can be recovered by the same key.

22.1.6 · Data model and schema

Store link(id,code UNIQUE,target,owner_id,created_at,expires_at,state,version) and operation(owner_id,key UNIQUE,fingerprint,result_id). Code uniqueness is global within the chosen namespace. If data is later sharded, route by code or use an allocation protocol so uniqueness is not merely local to unrelated shards. Keep revocation and expiry authoritative and versioned.

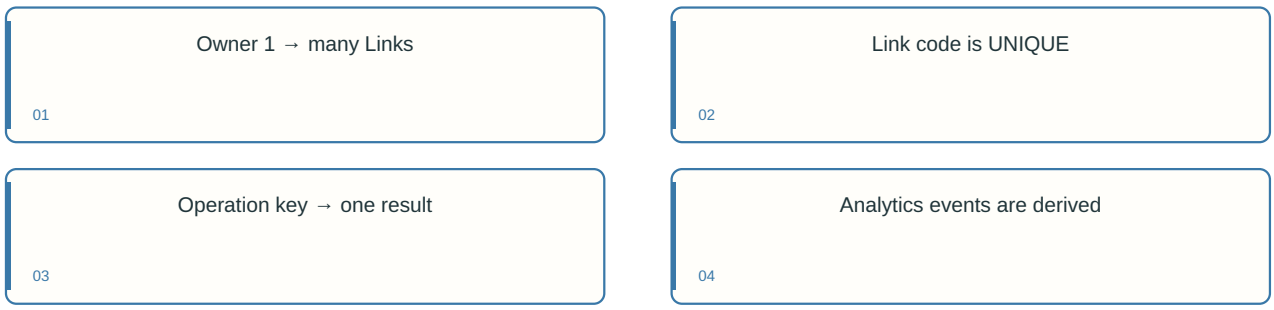


FIG. 22.1.1 **Data ownership and relationships**

Store link(id,code UNIQUE,target,owner_id,created_at,expires_at,state,version) and operation(owner_id,key UNIQUE,fingerprint,result_id). Code uniqueness is global within the chosen namespace. If data is later sharded, route by code or use an allocation protocol so uniqueness is not merely local to unrelated shards. Keep revocation and expiry authoritative and versioned.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.7 · Low-level component organization

LinkService validates target and ownership, CodeAllocator proposes a code, LinkRepository atomically inserts the mapping and operation result, RedirectResolver checks usable state, and AnalyticsPublisher records noncritical events. The service depends on application-owned contracts. Use integer IDs encoded in base62 when an appropriate unique allocator is available, or random codes with atomic conflict detection and bounded retry.

22.1.8 · High-level system architecture

Place an edge or reverse proxy before replaceable application instances. A shared cache serves permitted mapping representations; the authoritative replicated store owns writes and recovery. A queue or log receives analytics independently. The cache is optional for correctness and must have bounded fallback traffic. Storage placement and acknowledgment determine durability; multiple web instances do not protect the mapping history.

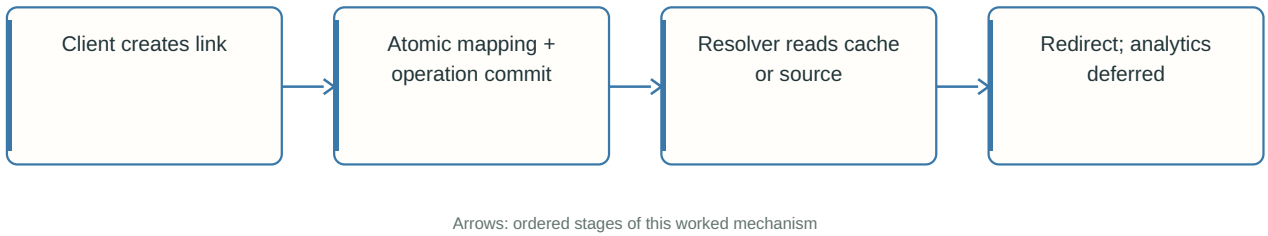


FIG. 22.1.2 **The high-level path and responsibility boundaries**

Place an edge or reverse proxy before replaceable application instances. A shared cache serves permitted mapping representations; the authoritative replicated store owns writes and recovery. A queue or log receives analytics independently. The cache is optional for correctness and must have bounded fallback traffic. Storage placement and acknowledgment determine durability; multiple web instances do not protect the mapping history.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.9 · Data and control flows

Creation validates, atomically commits mapping plus deduplication result, then replies. Redirect checks expiry and revocation policy, resolves a cached or authoritative target, emits a permitted redirect, and records analytics without blocking the response. A cache miss coalesces per code where useful. Do not cache an authorization-dependent management representation in a shared public redirect cache.

22.1.10 · Capacity estimates

At 10000 redirects/s and an assumed 90% cache hit ratio, nominal authoritative reads are 1000/s before coalescing and retries. Peak 100000 redirects/s implies 10000 misses/s if the ratio holds. Peak response traffic at 500 bytes is 50 MB/s before transport overhead. Failure planning must include a cold-cache case, not only this nominal miss rate.

22.1.11 · End-to-end latency estimates

An illustrative warm redirect budget is edge/network 20 ms, application 10 ms, cache 5 ms, and return-path work 20 ms: 55 ms total. This arithmetic is an assumed path budget, not measured p99. A miss adds authoritative lookup and queue time. Validate the tail under hot keys, cache loss, storage failover, and revocation checks before claiming the 100 ms objective is achieved.

22.1.12 · Failure scenarios

A code collision must cause an atomic uniqueness failure and retry, not overwrite the prior target. A cache outage must not flood storage without admission control. A lost creation reply must replay the committed operation result. A stale cache may serve a revoked link unless its policy limits that exposure. Analytics failure should reduce analytics freshness rather than redirect availability.

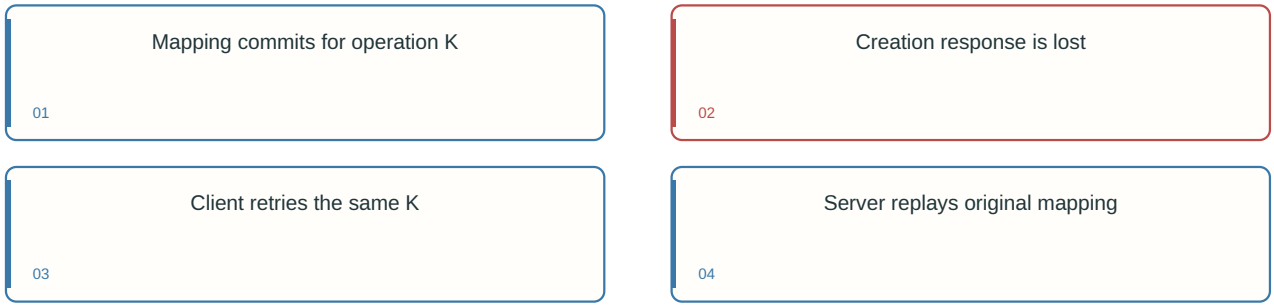


FIG. 22.1.3 **Naive path, failure, improved path, review**

A code collision must cause an atomic uniqueness failure and retry, not overwrite the prior target. A cache outage must not flood storage without admission control. A lost creation reply must replay the committed operation result. A stale cache may serve a revoked link unless its policy limits that exposure. Analytics failure should reduce analytics freshness rather than redirect availability.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.13 · Scalability improvements

Scale the stateless tier for measured independent compute, add shared caching for demonstrated read pressure, then partition authoritative mappings only if a real storage bottleneck remains. A globally hot code may need edge or local read copies rather than more cold shards. Versioned invalidation and expiry checks accompany those copies. Preserve routing and recovery during any key-space migration.

22.1.14 · Consistency requirements

Creation and code ownership require an atomic uniqueness boundary; management reads should meet the chosen read-after-write contract. Redirects may permit bounded stale targets only if revocation and expiry requirements allow it. Analytics can be eventually processed with stable event identities. Do not describe the whole system as simply eventual: the mapping insertion and the click count have different obligations.

22.1.15 · Security model

Enforce owner-scoped management, rate-limit creation, permit only supported destination schemes, and assess abuse without allowing server-side fetches to arbitrary private addresses. If preview generation fetches targets, isolate its egress and validate every redirect hop. Protect management credentials and avoid exposing private original targets through analytics or public enumeration. A hard-to-guess code is not an access-control system for confidential content.

22.1.16 · Deployment approach

Deploy the application as a small rolling-compatible service with a replicated authoritative store and tested backups. New code must understand existing mapping versions. Drain connections and preserve operation identities across releases. Start in one region with a clear failure contract; introduce another only when latency or disaster requirements justify its data-loss and authority decisions.

22.1.17 · Observability design

Measure redirect success and latency, cache hit and miss rates, origin admission, hot-key concentration, creation conflicts, storage progress, and revocation propagation. Trace misses separately from hits. Alert on useful redirect failures and rapid error-budget burn; track analytics lag independently. A healthy cache dashboard does not prove newly created or revoked mappings are being served correctly.

22.1.18 · Infrastructure cost considerations

Use symbolic monthly costs: compute = instance_hours×compute_unit_rate; storage = retained_bytes×copy_factor×storage_rate; transfer = response_bytes×transfer_rate. With 100000 redirects/s sustained for 30 days, the assumed 500-byte response totals 129.6 TB before overhead. This is a deliberately different scenario from a short peak. Do not purchase sustained-peak capacity from an average-load estimate without defining the traffic duration.

22.1.19 · Alternative architectures and trade-offs

Sequential allocated IDs simplify collision handling but reveal ordering and need an allocator strategy. Random codes distribute guesses but still need atomic uniqueness and retry. Permanent redirects can help client caching but complicate later target changes; temporary redirects can preserve more server control at a traffic cost. Compare these alternatives against revocation, privacy, and mutation requirements rather than choosing by habit.

22.1.20 · Architecture evolution from small scale to large scale

At roughly 100 requests/s, use one application and one transactional store. At 1000, profile and index before distribution. At 10000, add justified cache and application capacity. At 100000, partition or expand the measured bottleneck, protect cold-origin capacity, and test degraded operation. These levels are teaching checkpoints, not benchmark-proven hardware thresholds.

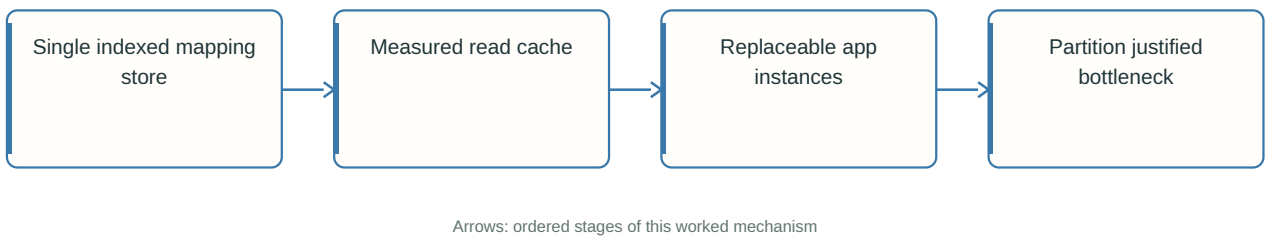


FIG. 22.1.4 **Evolution follows demonstrated requirements**

At roughly 100 requests/s, use one application and one transactional store. At 1000, profile and index before distribution. At 10000, add justified cache and application capacity. At 100000, partition or expand the measured bottleneck, protect cold-origin capacity, and test degraded operation. These levels are teaching checkpoints, not benchmark-proven hardware thresholds.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.21 · Design review identifying weaknesses and improvements

The weakest assumptions are cache freshness during revocation, cold-cache origin capacity, and global uniqueness after partitioning. Review them with a revoke-under-load test, a full cache-loss exercise, and a forced code-collision test spanning the actual ownership boundary. Prefer an explicit bounded-degradation policy over a claim that cache replication automatically solves all three concerns.

22.1.22 · A short implementation exercise

Implement base62 encoding and a repository that rejects duplicate codes atomically. Force two candidates to use the same code and prove the first target is preserved. Then inject a crash after mapping commit but before reply and verify that the same owner/key returns the original mapping rather than creating another.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def allocate(target, operation_key):
    prior = operations.find(operation_key)
    if prior: return prior
    while attempts_remaining():
        code = propose_code()
        try:
            with transaction():
                link = insert_unique_code(code, target)
                save_operation_result(operation_key, link.id)
            return link
        except CodeConflict:
            continue
        except OperationConflict:
            return load_and_validate_prior_result(operation_key, target)
    raise AllocationUnavailable
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

3. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

Rate Limiter

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Admit or reject an operation according to a named caller, resource, cost, and policy. Expose useful retry guidance and operator policy controls. Distinguish rate, burst, and cumulative quota. For distributed use, define whether the promise is a strict global limit or an explicit bounded approximation; silently applying a full limit independently in every region is not the same contract.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.2.1 · Functional requirements

Admit or reject an operation according to a named caller, resource, cost, and policy. Expose useful retry guidance and operator policy controls. Distinguish rate, burst, and cumulative quota. For distributed use, define whether the promise is a strict global limit or an explicit bounded approximation; silently applying a full limit independently in every region is not the same contract.

22.2.2 · Nonfunctional requirements

Target a small admission overhead under the assumed load while protecting an authoritative dependency from excess work. Preserve per-tenant fairness and a deliberate failure policy. If an exact global cap is mandatory, some partitioned operations may need to reject or wait. A limiter that fails open during storage outage cannot simultaneously promise a strict cap unless safe local budgets already cover that outage.

22.2.3 · Explicit assumptions and workload numbers

Assume 100000 admission checks/s, one million active keys, a token bucket capacity of ten and refill of two tokens/s for one sample policy. Assume 64 bytes of raw state per key before engine overhead: 64 MB. TTL, replication, allocator overhead, indexes, and network requests increase real memory and cost. These are illustrative quantities, not Redis or other product benchmarks.

22.2.4 · Naive initial architecture

Begin with one in-process dictionary and a lock around each token update. It provides a correct local boundary for one process, but two processes each grant a full bucket unless state or budget ownership is shared. A process restart can reset allowances. This is appropriate only when those scope and restart effects fit the required protection contract.

22.2.5 · Core API contracts

check(key,cost,policy_version) returns allowed, remaining allowance, and appropriate retry delay. Administrative policy updates require authorization and effective-version handling. The caller must not choose a different key to bypass its identity. Cost must be positive and bounded. Return overload distinctly from business validation; document whether rejected attempts themselves consume another abuse or request budget.

22.2.6 · Data model and schema

Store bucket(scope_key,policy_version,tokens,last_refill,expiry) under an atomic update boundary. Track cumulative quotas separately if required. Integer or fixed-point units can avoid unnecessary floating-point drift. Policy changes need a rule for existing tokens and capacities. A distributed clock value supplied by an untrusted client must not let that client refill its own bucket early.

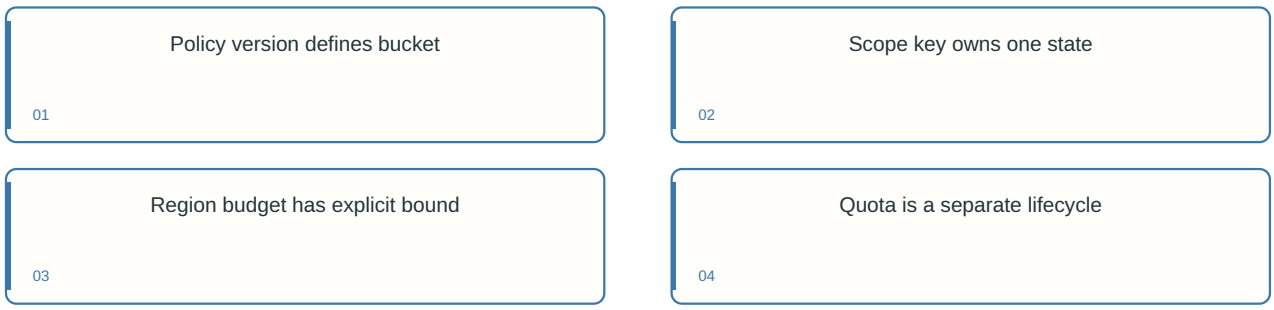


FIG. 22.2.1 **Data ownership and relationships**

Store bucket(scope_key,policy_version,tokens,last_refill,expiry) under an atomic update boundary. Track cumulative quotas separately if required. Integer or fixed-point units can avoid unnecessary floating-point drift. Policy changes need a rule for existing tokens and capacities. A distributed clock value supplied by an untrusted client must not let that client refill its own bucket early.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.7 · Low-level component organization

PolicyResolver selects validated limits, KeyResolver derives trusted identity, Clock supplies the authoritative elapsed-time basis, BucketStore executes refill/check/debit atomically, and AdmissionService maps the result. Keep these contracts independently testable. A stateless calculation outside an atomic store update is insufficient when several requests act on the same bucket concurrently.

22.2.8 · High-level system architecture

Put coarse admission near the edge and resource-specific protection near expensive operations. For strict shared buckets, route updates to one appropriate atomic owner or a reviewed distributed mechanism. For regional approximation, allocate bounded regional budgets and state the overshoot behavior. Keep a global emergency cap and protect the limiter's own capacity; an overloaded protection service can otherwise become the first failure point.

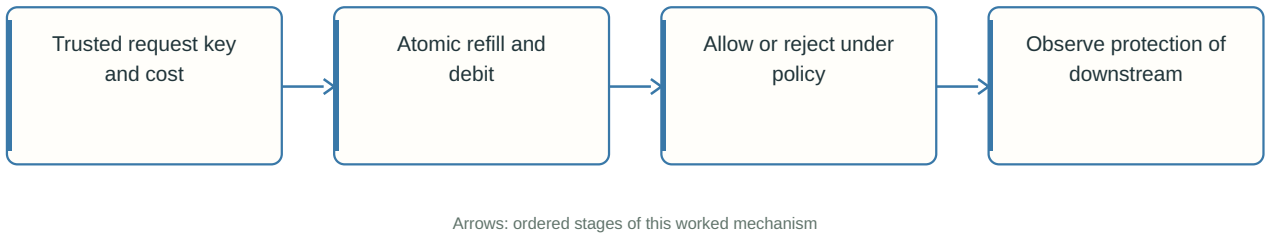


FIG. 22.2.2 **The high-level path and responsibility boundaries**

Put coarse admission near the edge and resource-specific protection near expensive operations. For strict shared buckets, route updates to one appropriate atomic owner or a reviewed distributed mechanism. For regional approximation, allocate bounded regional budgets and state the overshoot behavior. Keep a global emergency cap and protect the limiter's own capacity; an overloaded protection service can otherwise become the first failure point.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.9 · Data and control flows

For each request, derive scope, load the current bucket under coordination, compute max(0,elapsed), refill up to capacity, test cost, and debit only if admitted. Return a delay based on the deficit and refill rate where meaningful. Record metrics using bounded dimensions. Do not split read and write into separate unprotected calls, which can let concurrent requests spend the same tokens.

22.2.10 · Capacity estimates

A million assumed 64-byte states is 64 MB raw. At 100000 checks/s, a one-round-trip remote check creates 100000 limiter requests/s; batching or hierarchical policies may reduce that burden with changed semantics. One hot tenant can concentrate writes despite key sharding. Size by checks, active state, hot-key contention, and replicated write cost rather than memory alone.

22.2.11 · End-to-end latency estimates

Suppose the application has a 100 ms budget and allocates 5 ms to admission. That is an objective to validate under contention and failover, not a measured guarantee. A synchronous strict global check across distant regions may violate it because coordination adds propagation and queue time. Local preallocated budgets reduce latency but require bounded allocation and replenishment behavior.

22.2.12 · Failure scenarios

Race: two requests both read one token and both pass. Fix with an atomic transition. Clock rollback must not create extra refill; expiry should not unexpectedly grant an unsafe fresh burst. Store outage triggers the documented local-budget, fail-closed, or bounded fail-open policy. Policy inconsistency and key spoofing can bypass intended limits even when the bucket arithmetic is correct.

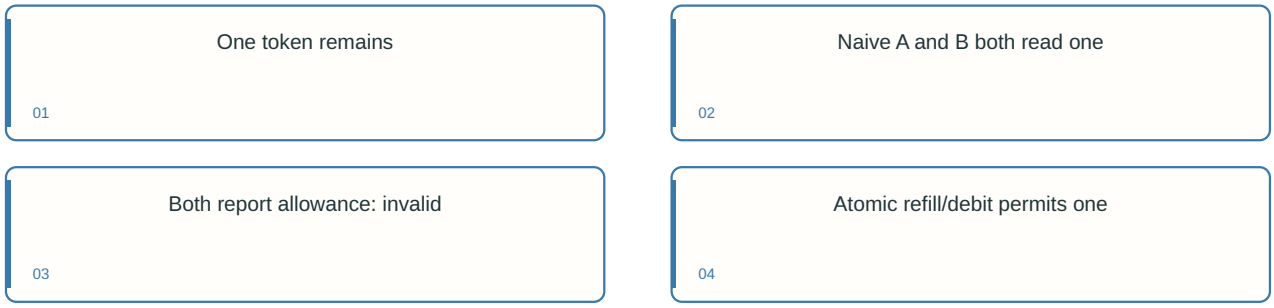


FIG. 22.2.3 **Naive path, failure, improved path, review**

Race: two requests both read one token and both pass. Fix with an atomic transition. Clock rollback must not create extra refill; expiry should not unexpectedly grant an unsafe fresh burst. Store outage triggers the documented local-budget, fail-closed, or bounded fail-open policy. Policy inconsistency and key spoofing can bypass intended limits even when the bucket arithmetic is correct.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.13 · Scalability improvements

Partition independent keys, co-locate a bucket's atomic update, and control hot-key access with suitable local or hierarchical budgets. Cache policy definitions with version and expiry while keeping spending authoritative. Scaling keys does not split a single strict bucket without a new allocation protocol. Consider grouping or weighted admission when the protected resource cost varies by operation.

22.2.14 · Consistency requirements

A strict bucket needs an atomic refill-and-debit under one policy and time model. Distributed approximation must identify the maximum outstanding regional allowance or other bound. Token bucket permits a burst; fixed-window counters can permit boundary bursts; sliding-window logs or counters approximate different histories. Consistency here is an admission invariant, not merely whether replicated metric counters eventually converge.

22.2.15 · Security model

Derive keys from authenticated tenant or service identity, guard policy administration, bound key cardinality to avoid memory abuse, and avoid exposing detailed shared usage to unauthorized callers. Do not trust user-supplied timestamps, policy names, or arbitrary tenant headers. Resource-level authorization remains necessary after admission; having tokens never grants permission to another tenant's data.

22.2.16 · Deployment approach

Deploy the policy and bucket ownership layers with defined failover and state-loss semantics. Roll out policy version changes compatibly and validate that old clients understand responses. A stateful failover that resets every bucket changes the protection promise. Test restart, time behavior, and loss of the shared owner. Keep a simple local fallback only when its bounded behavior is acceptable.

22.2.17 · Observability design

Measure admitted and rejected cost, check latency, store errors, active keys, hot-key saturation, token anomalies, and policy-version disagreement. Alert on protection failure as well as excessive rejection of valid traffic. Track downstream saturation to verify that the limiter is protecting the actual resource. A high rejection count alone can reflect either effective defense or an incorrectly configured policy.

22.2.18 · Infrastructure cost considerations

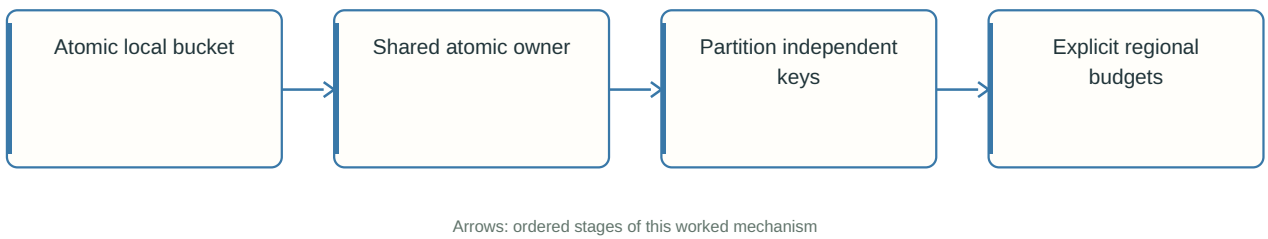
Cost is state memory plus checks, replicated updates, transfer, and operating work. A strict remote check for every request can be more expensive than the business operation it protects. Use symbolic unit prices and compare local approximate budgets with strict shared admission at equal stated guarantees. If the guarantee changes, show that trade-off rather than presenting the cheaper design as equivalent.

22.2.19 · Alternative architectures and trade-offs

Token bucket allows controlled bursts; leaky-bucket shaping smooths output while adding waiting; fixed windows are simple but have boundary effects; sliding windows better represent recent history at state and computation cost. Distributed local budgets trade precision for latency and availability. Choose the algorithm from the protected workload, not because one name is universally best.

22.2.20 · Architecture evolution from small scale to large scale

Start with a correct local bucket and deterministic tests. Move to an atomic shared owner when multiple processes require a shared cap. Add key partitioning for measured limiter load. Introduce bounded regional allocation only when latency or partition behavior justifies its extra protocol. Preserve the original policy meaning through every move, or version it explicitly.



Arrows: ordered stages of this worked mechanism

FIG. 22.2.4 **Evolution follows demonstrated requirements**

Start with a correct local bucket and deterministic tests. Move to an atomic shared owner when multiple processes require a shared cap. Add key partitioning for measured limiter load. Introduce bounded regional allocation only when latency or partition behavior justifies its extra protocol. Preserve the original policy meaning through every move, or version it explicitly.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.21 · Design review identifying weaknesses and improvements

Review whether the system really enforces one global contract, whether administrative changes create extra allowance, and how a hot key behaves. Test simultaneous spends of the last token, backward elapsed time, policy replacement, storage outage, and regional reconnection. Reject a design that advertises exact global admission while independently resetting a full bucket at every replica.

22.2.22 · A short implementation exercise

Implement a deterministic token bucket with an injected monotonic clock. Spend ten initial tokens, advance three seconds, and prove six more are available under capacity ten. Reject an eleventh immediate spend, negative costs, and time rollback. Then demonstrate the lost-token race with separate read/write operations and repair it with an atomic owner.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def spend(bucket, now, cost):
    require(cost > 0)
    elapsed = max(0, now - bucket.last)
    available = min(bucket.capacity,
                    bucket.tokens + elapsed * bucket.rate)
    allowed = available >= cost
    bucket.tokens = available - cost if allowed else available
    bucket.last = max(bucket.last, now)
    return allowed
# Entire transition executes atomically for this bucket.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

THE WORKBENCH · CALCULATED TEACHING MODEL

Spend a bounded burst, then advance time

Virtual time: 0s · tokens: **10** of 10 · admitted: 0 · rejected: 0.

Refill is two tokens/s and cannot exceed capacity. This state has one browser-local owner.

Capacity ten, refill two tokens per second, one local owner, virtual monotonic time. A real shared bucket needs an atomic transition; distributed global admission requires an explicit protocol.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 2697: A Single Rate Three Color Marker, 1999 ↗

   Original protocol description · Informational RFC. Token-based traffic metering example; atlas admission bucket is a simpler model.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

Chat Application

Prerequisites: Networking Foundations for System Design, Data Models and Storage Technologies, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Send a message to an authorized conversation, retain it durably, deliver online updates, reconnect and fetch missed messages, and expose a meaningful receipt model. Support one-to-one and bounded group conversations. Presence is approximate and ephemeral. Define sent, persisted, delivered-to-device, and read as different states rather than promising that opening a WebSocket makes every message read by a person.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.3.1 · Functional requirements

Send a message to an authorized conversation, retain it durably, deliver online updates, reconnect and fetch missed messages, and expose a meaningful receipt model. Support one-to-one and bounded group conversations. Presence is approximate and ephemeral. Define sent, persisted, delivered-to-device, and read as different states rather than promising that opening a WebSocket makes every message read by a person.

22.3.2 · Nonfunctional requirements

Protect conversation privacy, preserve acknowledged messages under the chosen storage failure contract, and provide per-conversation ordering or an explicitly weaker model. Target online delivery below one second under the illustrative supported workload, with eventual catch-up after disconnect. Do not make

presence freshness an inventory-style correctness invariant. Offline history and moderation or deletion policies need deliberate retention semantics.

22.3.3 · Explicit assumptions and workload numbers

Assume 100000 concurrent connections, 10000 accepted messages/s, 500 stored bytes per message, and average five recipient deliveries. Raw message growth is 432 GB/day before indexes and replicas. Delivery payload is about 25 MB/s before framing and transport overhead. Connections, reconnect storms, device multiplicity, and large groups create different pressures from the average message rate.

22.3.4 · Naive initial architecture

Start with one connection server and one transactional message store. Persist messages before returning a persistence receipt, then attempt online fan-out. This makes ownership simple but loses live connections on process failure and cannot infer device receipt from a successful database commit. Clients reconnect and read from durable history, so lost transient delivery does not mean lost acknowledged messages.

22.3.5 · Core API contracts

POST or a WebSocket send command includes conversation_id, client_message_id, and bounded content. GET /conversations/{id}/messages?after=sequence fetches catch-up pages. Delivery and read receipts identify the device or user and highest relevant sequence under the policy. Membership is checked for both writes and reads, and repeated client_message_id within the sender scope returns the existing message.

22.3.6 · Data model and schema

Store conversation, membership, message(conversation_id,sequence,sender,client_message_id,body,version), and receipt(conversation_id,user_or_device,delivered_seq,read_seq). Enforce an appropriate unique client-message scope. Presence lives separately with heartbeat expiry. Conversation ordering needs one sequence authority or a reviewed conflict model; wall-clock timestamps alone do not establish a stable total order for concurrent sends.

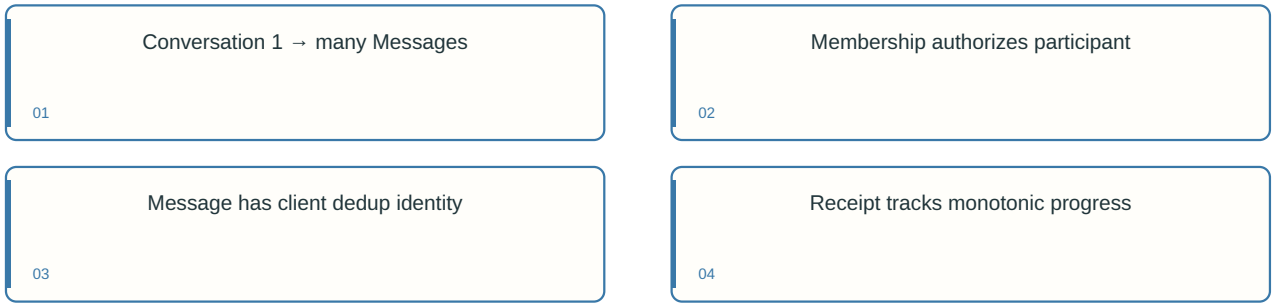


FIG. 22.3.1 **Data ownership and relationships**

Store conversation, membership, message(conversation_id,sequence,sender,client_message_id,body,version), and receipt(conversation_id,user_or_device,delivered_seq,read_seq). Enforce an appropriate unique client-message scope. Presence lives separately with heartbeat expiry. Conversation ordering needs one sequence authority or a reviewed conflict model; wall-clock timestamps alone do not establish a stable total order for concurrent sends.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.7 · Low-level component organization

ConnectionGateway owns sessions, MembershipPolicy authorizes access, MessageService coordinates deduplication and persistence, SequenceOwner defines ordering, HistoryRepository serves bounded pages, and DeliveryWorker fans out committed messages. ReceiptAggregator maintains monotonic progress according to device semantics. Large-group distribution is an explicit operation, not a synchronous loop hidden inside a database transaction.

22.3.8 · High-level system architecture

Use replaceable connection gateways, a conversation-routed durable message path, a delivery log or outbox, recipient-routing metadata, and history storage. Gateways hold ephemeral connection state while the store owns messages. A delivery route can be stale, so clients must recover by durable sequence. A broker outage can delay online delivery without losing committed messages if publication intent is recorded atomically.

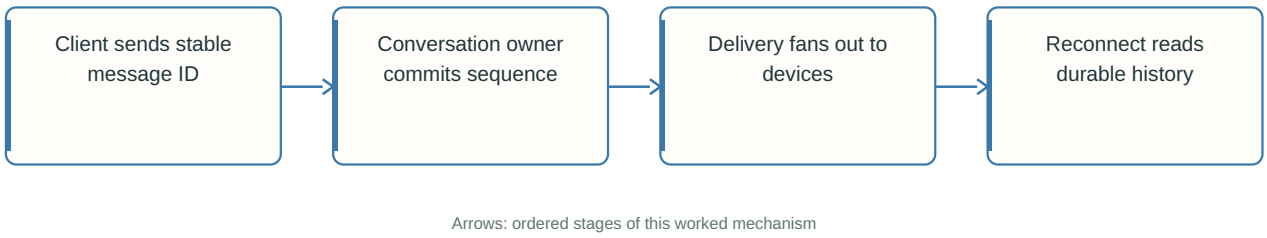


FIG. 22.3.2 **The high-level path and responsibility boundaries**

Use replaceable connection gateways, a conversation-routed durable message path, a delivery log or outbox, recipient-routing metadata, and history storage. Gateways hold ephemeral connection state while the store owns messages. A delivery route can be stale, so clients must recover by durable sequence. A broker outage can delay online delivery without losing committed messages if publication intent is recorded atomically.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.9 · Data and control flows

Client sends stable message identity; service verifies membership, assigns sequence, commits message and delivery intent, and replies with persistence status. Delivery workers send to connected devices and record allowed receipts. On reconnect, client supplies last observed sequence and fetches missing history with gap handling. A lost live frame is repaired through catch-up, not by pretending the transport is durable storage.

22.3.10 · Capacity estimates

The assumed 10000 messages/s at 500 bytes produce 5 MB/s raw ingestion and 432 GB/day. Five recipient deliveries give 50000 deliveries/s and about 25 MB/s body traffic. Connection memory must be measured: at an illustrative 20 kB per connection it is 2 GB before process overhead and queued data. Bound outbound buffers for slow clients.

22.3.11 · End-to-end latency estimates

An illustrative online budget is 20 ms ingress, 30 ms commit coordination, 20 ms delivery queue, and 80 ms recipient path: 150 ms before client rendering. These numbers are assumptions. Long tails from hot conversations, offline devices, congestion, or slow-client buffers require separate measurement. Persistence receipt latency and human read latency are fundamentally different indicators.

22.3.12 · Failure scenarios

Lost send response can duplicate messages unless stable client identity is deduplicated. Gateway failure drops connections; history remains authoritative. A slow client must be disconnected or degraded before its buffer exhausts the gateway. A membership revocation can race with cached authorization, so enforce the chosen revocation policy for delivery and history. Reconnect storms need admission and staggered retry.

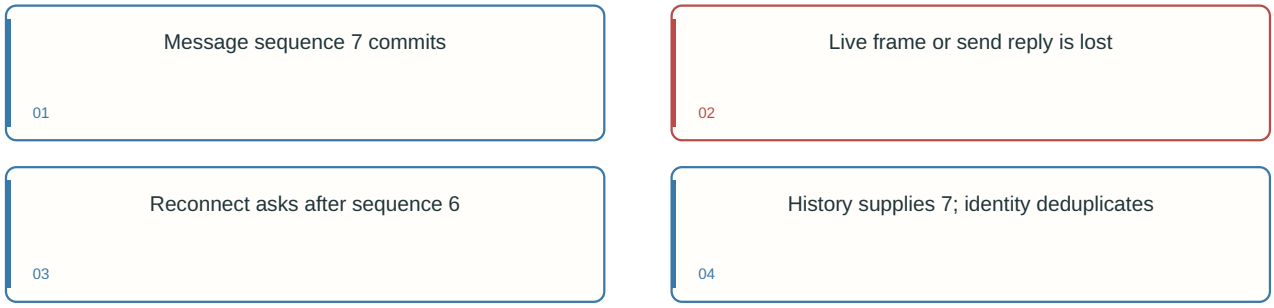


FIG. 22.3.3 **Naive path, failure, improved path, review**

Lost send response can duplicate messages unless stable client identity is deduplicated. Gateway failure drops connections; history remains authoritative. A slow client must be disconnected or degraded before its buffer exhausts the gateway. A membership revocation can race with cached authorization, so enforce the chosen revocation policy for delivery and history. Reconnect storms need admission and staggered retry.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.13 · Scalability improvements

Partition history and sequence ownership by conversation, distribute independent gateway connections, and move large-group fan-out to bounded workers. A single extremely hot conversation remains a serialization or distribution challenge despite many cold partitions. Consider separate large-channel semantics rather than forcing every group into one low-latency per-recipient acknowledgment workflow. Preserve source sequence and deduplication across migration.

22.3.14 · Consistency requirements

Provide monotonic per-conversation sequence under the selected authority, read-after-write for sender confirmation, and replayable catch-up. Cross-conversation ordering is unnecessary unless a product requirement says otherwise. Device delivery is at least retryable and can duplicate frames; clients

deduplicate by message identity. Presence and read indicators may be eventually updated but should not move backward under their defined receipt model.

22.3.15 · Security model

Check membership server-side on all APIs, message fan-out, search, and media access. Rate-limit abusive sends, bound payloads and group membership, and avoid leaking content in telemetry. If end-to-end encryption is required, define device keys, recovery, membership changes, and what server-side features become impossible or different; do not imply TLS supplies that product guarantee.

22.3.16 · Deployment approach

Roll out gateways with connection draining and reconnect support. Evolve message and receipt schemas compatibly across old clients. Preserve history during region or store failover under an explicit acknowledgment and promotion contract. Load-test reconnection and catch-up together with normal traffic. A deployment that restarts every gateway simultaneously can cause a larger load event than ordinary messaging.

22.3.17 · Observability design

Measure persistence acknowledgment latency, live-delivery lag, catch-up gaps, reconnect rate, fan-out backlog, per-conversation skew, slow-client buffer limits, and membership-denial outcomes. Correlate client_message_id with server message identity and delivery attempts without using unbounded IDs as metric labels. Synthetic clients can verify send, reconnect, history, and authorization histories end to end.

22.3.18 · Infrastructure cost considerations

Model connection-serving capacity, message retention, fan-out transfer, indexes, replicas, media storage, and telemetry separately. Retaining 432 GB/day raw for thirty days is 12.96 TB before replication and indexing under the assumptions. Media can dominate both storage and egress, so it needs independent quotas and lifecycle. No live provider prices are assumed.

22.3.19 · Alternative architectures and trade-offs

WebSockets suit bidirectional low-latency exchange but require connection operations; SSE plus HTTP sends can simplify some client paths; polling is simple but creates idle traffic. Per-conversation sequence ownership simplifies ordering at a hot-key cost. A conflict-tolerant multi-region model reduces some

coordination but changes ordering semantics. Select the smallest mechanism matching the product's actual receipts and offline needs.

22.3.20 · Architecture evolution from small scale to large scale

Begin with persistent history and reconnect correctness. Add connection replicas and explicit routing. Introduce durable fan-out for higher traffic or groups. Partition conversation ownership when storage or sequence pressure justifies it. Finally add regional placement only with an explicit message-order, membership, and failover contract. At every step, preserve stable message identity so retries remain safe.

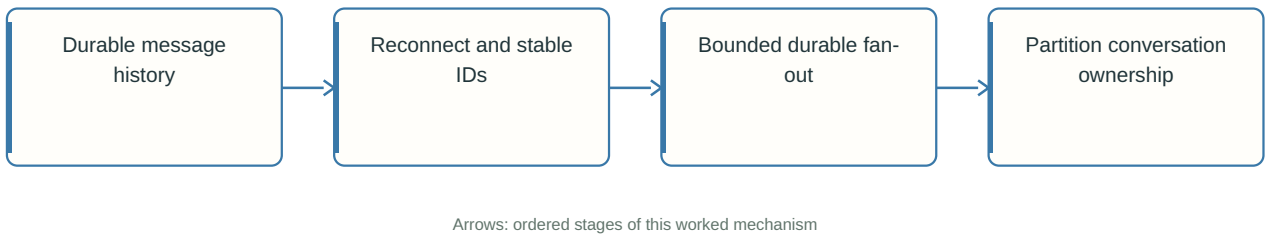


FIG. 22.3.4 **Evolution follows demonstrated requirements**

Begin with persistent history and reconnect correctness. Add connection replicas and explicit routing. Introduce durable fan-out for higher traffic or groups. Partition conversation ownership when storage or sequence pressure justifies it. Finally add regional placement only with an explicit message-order, membership, and failover contract. At every step, preserve stable message identity so retries remain safe.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.21 · Design review identifying weaknesses and improvements

Review the ambiguous words “sent” and “delivered,” hot conversation behavior, slow-client memory, and membership revocation. Test a commit followed by lost send reply, a dropped live frame, gateway termination, a stale connection route, and history access after revocation. A demo that works only with two permanently connected clients has not validated the required system.

22.3.22 · A short implementation exercise

Build an in-memory client that deduplicates messages by server ID and detects sequence gaps. Persist a message, suppress its live frame, reconnect with the previous sequence, and prove catch-up restores it

once in the visible list. Then replay the same client_message_id and show that no second history record is created.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
with transaction():
    existing = find_by_client_id(sender, client_message_id)
    if existing: return existing
    require_member(sender, conversation)
    seq = allocate_conversation_sequence(conversation)
    msg = insert_message(conversation, seq, body)
    insert_outbox(msg.id, "deliver")
return persisted_receipt(msg)
# Device delivery and read receipt happen later.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 6455: The WebSocket Protocol, 2011 ↗

   Original standard. Bidirectional transport framing; not durable chat delivery.

   Access reviewed · 21 October 2026

2. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

Notification Platform

Prerequisites: Design Patterns, Dependency Injection, and Object Lifecycles, API Design and Communication Contracts, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Accept a notification intent, evaluate recipient preferences and templates, deliver through email, SMS, or push, and track outcomes. Support scheduled sending, cancellation within a defined window, and delivery callbacks. Distinguish provider acceptance from final delivery and human viewing. Product notifications and legally or operationally required messages may have different preference policies; encode the rule explicitly rather than letting a channel adapter decide.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.4.1 · Functional requirements

Accept a notification intent, evaluate recipient preferences and templates, deliver through email, SMS, or push, and track outcomes. Support scheduled sending, cancellation within a defined window, and delivery callbacks. Distinguish provider acceptance from final delivery and human viewing. Product notifications and legally or operationally required messages may have different preference policies; encode the rule explicitly rather than letting a channel adapter decide.

22.4.2 · Nonfunctional requirements

Preserve accepted intents, prevent duplicate relevant effects where provider contracts permit, isolate channels, and make delayed or permanently failed delivery visible. Set a business expiry for time-sensitive

messages. A password-reset message sent hours late may be harmful even if eventually delivered. The system should protect critical traffic when a marketing batch or one provider consumes capacity.

22.4.3 · Explicit assumptions and workload numbers

Assume ten million intents/day: about 115.7/s average, with a tenfold peak of roughly 1157/s. Assume 1 kB of stored intent and attempt metadata initially, giving 10 GB/day before additional attempt history, indexes, and replicas. Channel quotas are hypothetical configured inputs; they must be verified with the actual provider and contract, not inferred from these workload averages.

22.4.4 · Naive initial architecture

Start with an application function directly sending through one provider after a user action. It is easy to implement but ties request latency and success to the provider and can lose notification intent if the process crashes after committing the business action. Move accepted intent into a durable local transaction or outbox so delivery can resume without replaying the original business operation blindly.

22.4.5 · Core API contracts

POST /notifications includes recipient identity, template version, variables, channel policy, expiry, and idempotency key. GET /notifications/{id} reports accepted, pending, submitted, delivered, failed, expired, or cancelled under the chosen state machine. Cancellation does not promise to recall a message already accepted by a provider. Callback endpoints verify authenticity and process repeated provider events safely.

22.4.6 · Data model and schema

Store intent, recipient preference version, template version, delivery task per channel or recipient, attempt, and provider-event inbox. Uniqueness binds the logical intent and channel identity according to policy. Record provider references and uncertain outcomes. Keep message content retention separate from minimal audit metadata. A later preference change needs a rule for already queued messages, particularly where cancellation or consent matters.

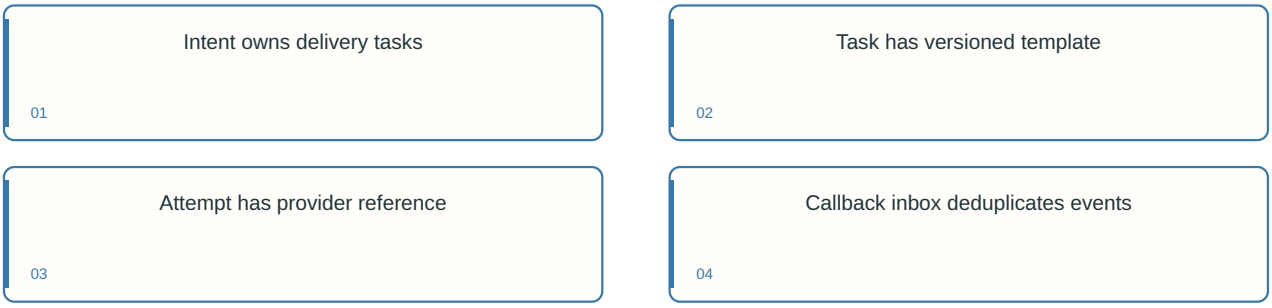


FIG. 22.4.1 **Data ownership and relationships**

Store intent, recipient preference version, template version, delivery task per channel or recipient, attempt, and provider-event inbox. Uniqueness binds the logical intent and channel identity according to policy. Record provider references and uncertain outcomes. Keep message content retention separate from minimal audit metadata. A later preference change needs a rule for already queued messages, particularly where cancellation or consent matters.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.7 · Low-level component organization

NotificationService owns intent, PreferencePolicy decides admissible channels, TemplateRenderer produces a bounded message, SenderStrategy represents email/SMS/push behavior, ProviderAdapter translates APIs, and DeliveryRepository owns attempts and state. Decorators can add metrics or an appropriate retry wrapper without changing result meaning. A factory selects channel implementations at composition or runtime, with clear resource lifetimes.

22.4.8 · High-level system architecture

Use a durable ingress path, preference/template access, channel-specific queues or partitions, bounded workers, provider adapters, callback inboxes, and an outcome store. Separate critical and bulk capacity where necessary. A provider outage should saturate only its allowed resources. Queueing is not reliability by itself: delivery intent, worker attempts, acknowledgments, and reconciliation must agree about ownership.

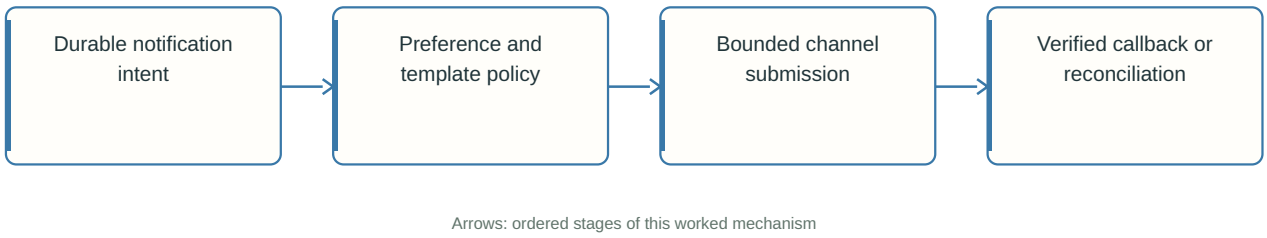


FIG. 22.4.2 **The high-level path and responsibility boundaries**

Use a durable ingress path, preference/template access, channel-specific queues or partitions, bounded workers, provider adapters, callback inboxes, and an outcome store. Separate critical and bulk capacity where necessary. A provider outage should saturate only its allowed resources. Queueing is not reliability by itself: delivery intent, worker attempts, acknowledgments, and reconciliation must agree about ownership.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.9 · Data and control flows

Accept and deduplicate intent, resolve permitted channel and template, persist delivery task, and dispatch under channel capacity. Before sending, verify expiry and relevant current policy. Submit with stable provider identity when supported, persist result or uncertain state, and acknowledge the job. Apply signed callbacks idempotently and prevent an older callback from regressing a terminal delivery state improperly.

22.4.10 · Capacity estimates

Average 115.7 intents/s understates a tenfold peak. If an assumed SMS route can submit only 40/s while arrival is 100/s, backlog grows by 60/s until policy limits or alternative capacity intervene. A ten-minute spike adds 36000 waiting submissions before completion changes. Model channel-specific arrival, capacity, retries, expiry, and recipient fan-out separately.

22.4.11 · End-to-end latency estimates

Notification acceptance can target a short API budget, while delivery latency has a separate queue-plus-provider objective. With an illustrative 50 ms acceptance, 2 s queue wait, and 500 ms submission, provider acceptance occurs around 2.55 s, not 50 ms. Human receipt depends on additional systems and cannot be claimed from an HTTP 200 returned by the provider.

22.4.12 · Failure scenarios

A process can crash after sending but before recording the provider result, producing an uncertain outcome and a possible duplicate on retry. Use provider idempotency or lookup where available; otherwise expose the residual duplicate risk under the channel contract. Poison templates require quarantine, provider outage needs backoff and expiry, and forged callbacks must not mark messages delivered. Reconcile old submitted or uncertain records.

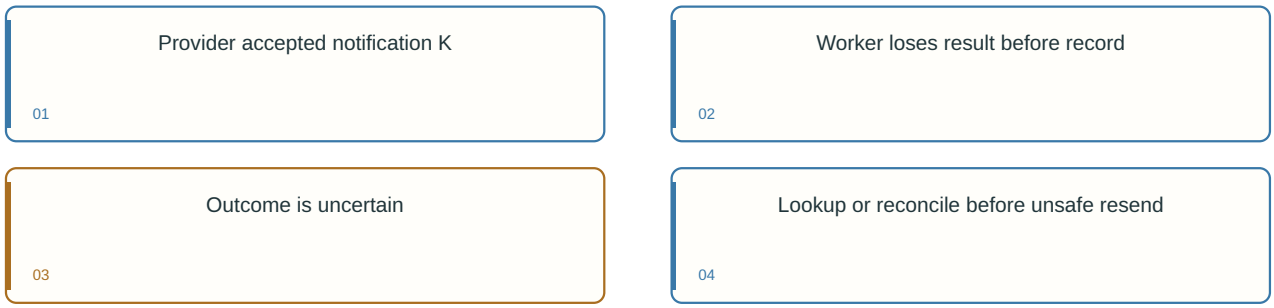


FIG. 22.4.3 **Naive path, failure, improved path, review**

A process can crash after sending but before recording the provider result, producing an uncertain outcome and a possible duplicate on retry. Use provider idempotency or lookup where available; otherwise expose the residual duplicate risk under the channel contract. Poison templates require quarantine, provider outage needs backoff and expiry, and forged callbacks must not mark messages delivered. Reconcile old submitted or uncertain records.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.13 · Scalability improvements

Scale independent recipients and channels through partitioned tasks and bounded workers, while preserving any required per-recipient ordering. Add provider routes only with explicit selection, rate, and duplicate policies. A failover to another SMS provider can send a second message if the first accepted it before timing out. More providers improve some availability paths but increase reconciliation and content-consistency obligations.

22.4.14 · Consistency requirements

The durable intent and its deduplication result need an atomic boundary. Task delivery can be at least once with idempotent local state transitions. Provider acceptance and final delivery remain separate effects under provider-specific contracts. Preferences may require a fresh check at send time; template rendering

should use a retained version so retries do not silently change message content. Avoid a blanket exactly-once claim across all channels.

22.4.15 · Security model

Authorize who may send which template to whom, protect recipient data, validate template variables, sign or verify callbacks, and prevent header or content injection at adapters. Scope provider credentials per use and environment. Redact sensitive content and tokens from logs, and rate-limit intent creation to prevent abuse. Signed links in messages need bounded lifetime and appropriate audience binding.

22.4.16 · Deployment approach

Deploy workers with graceful draining, durable attempts, and compatible task schemas. Roll out template changes as new versions rather than rewriting content under an existing identity. Preserve queue ownership during scale-down and credential rotation. Test an unavailable provider while deploying, and verify that channel isolation keeps critical notifications and acceptance behavior within their stated policies.

22.4.17 · Observability design

Measure accepted intents, per-channel queue age, expired tasks, provider submission latency, uncertain outcomes, callback delay, duplicate suppression, and critical-vs-bulk fairness. Track business completion indicators separately from enqueue success. Alert on old unresolved critical notifications and rapid budget consumption. A full queue with no worker errors can still represent a serious delivery incident.

22.4.18 · Infrastructure cost considerations

Cost includes stored intents and attempts, provider submissions including retries, message segments or payload sizes, worker capacity, callback processing, transfer, and operating work. Use symbolic channel_unit_price×submitted_units rather than inventing provider rates. Unicode or long SMS content may change billable units according to the provider, so verify its contract. Count cost per successful relevant delivery as well as accepted intent.

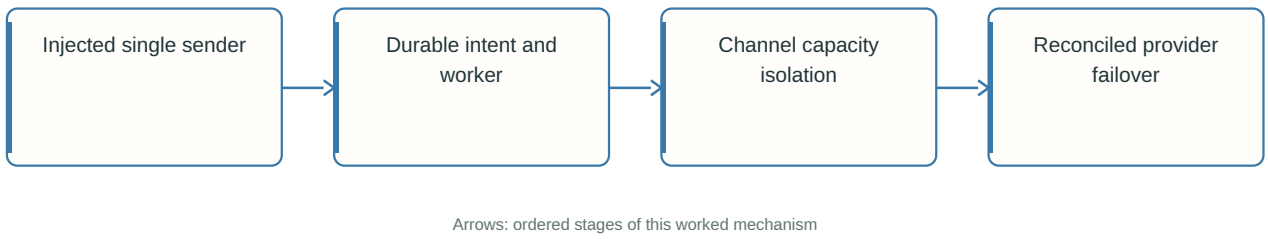
22.4.19 · Alternative architectures and trade-offs

Direct synchronous sending is simple for a noncritical low-volume path but couples latency and failure. A durable queue improves recovery but introduces delay and state. Orchestration makes multi-channel escalation visible; choreography can reduce central coordination but complicates outcome tracking. A

backup provider improves some outages while making duplicates harder. Evaluate alternatives against expiry, criticality, and provider evidence rather than only throughput.

22.4.20 · Architecture evolution from small scale to large scale

Start with one sender behind an injected contract. Add durable intent and a worker when loss or latency matters. Add channel strategies, factories, and metrics decorators for real variation. Partition and isolate critical and bulk delivery when traffic justifies it. Finally add provider failover and escalation only after uncertain-outcome and duplicate policies are explicit and tested.



Arrows: ordered stages of this worked mechanism

FIG. 22.4.4 **Evolution follows demonstrated requirements**

Start with one sender behind an injected contract. Add durable intent and a worker when loss or latency matters. Add channel strategies, factories, and metrics decorators for real variation. Partition and isolate critical and bulk delivery when traffic justifies it. Finally add provider failover and escalation only after uncertain-outcome and duplicate policies are explicit and tested.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.21 · Design review identifying weaknesses and improvements

Review whether “delivered” is being overstated, whether expired work can still send, and whether provider failover duplicates effects. Test send-then-crash, forged and repeated callbacks, preference revocation after enqueue, template version changes, and channel saturation. A dashboard of successful enqueue operations cannot certify that users received the intended messages under the stated policy.

22.4.22 · A short implementation exercise

Implement a recording sender and an adapter that returns an uncertain result after recording submission. Process the same task twice and verify local attempt identity and state do not produce a false delivery

claim. Add expiry and preference checks, then demonstrate why changing providers after uncertainty requires a distinct policy rather than an automatic retry.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def deliver(task, now):
    require_current_attempt(task)
    if now >= task.expires_at: return mark_expired(task)
    if not permitted_now(task): return mark_cancelled(task)
    sender = sender_factory.for_channel(task.channel)
    result = sender.send(task.stable_provider_key, render(task))
    if result.uncertain:
        return persist_uncertain(task, result.reference)
    return persist_submission_result(task, result)
# A provider acknowledgment is not a human-read receipt.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Social Media Feed

Prerequisites: Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Publish posts, follow accounts, read a ranked home feed, paginate consistently enough for the product, and apply deletion, privacy, and blocking rules. Ranking infrastructure selects among authorized candidates; it must not reintroduce posts a user cannot see. Counts and engagement are secondary representations whose freshness can differ from the source post.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.5.1 · Functional requirements

Publish posts, follow accounts, read a ranked home feed, paginate consistently enough for the product, and apply deletion, privacy, and blocking rules. Ranking infrastructure selects among authorized candidates; it must not reintroduce posts a user cannot see. Counts and engagement are secondary representations whose freshness can differ from the source post.

22.5.2 · Nonfunctional requirements

Target a useful feed within an assumed 300 ms p99 budget under supported load, retain accepted posts, and make access-control changes effective under a stated policy. Feed order may evolve, but pagination should not produce unexplained duplicates or unbounded skips. Define whether a refresh starts a new ranking snapshot and how stale cached candidates are filtered.

22.5.3 · Explicit assumptions and workload numbers

Assume ten million daily readers, twenty feed reads per reader, one million posts/day, and one hundred followers per ordinary author on average for the simplified model. Feed reads average about 2315/s. With 32-byte materialized candidate references, ordinary fan-out would create 3.2 GB/day raw references. A celebrity with millions of followers is not described by that average.

22.5.4 · Naive initial architecture

Begin with relational posts and follow edges, assembling a feed on read through indexed access. This avoids writing every follower feed and is simple at small scale. As follow counts and traffic rise, query fan-out and ranking work can exceed the latency budget. Do not denormalize the complete feed before measuring those reads and the skewed follower distribution.

22.5.5 · Core API contracts

POST /posts creates a versioned post; POST /follows changes a relationship; GET /feed returns a bounded cursor and ranking generation. Cursors bind filters, identity, and snapshot or continuation semantics. DELETE /posts/{id} records authoritative deletion. A feed cursor is not permission to read a post after access has been revoked.

22.5.6 · Data model and schema

Store post(id,author,version,visibility,state,created_at), follow(follower,author), and optional feed_candidate(reader,post_id,source_version,rank_features). Keep ranking model version and feed generation where reproducibility matters. Candidate references are derived; the post and authorization relationships remain authoritative. Rebuild and deletion propagation need stable source identity and progress.

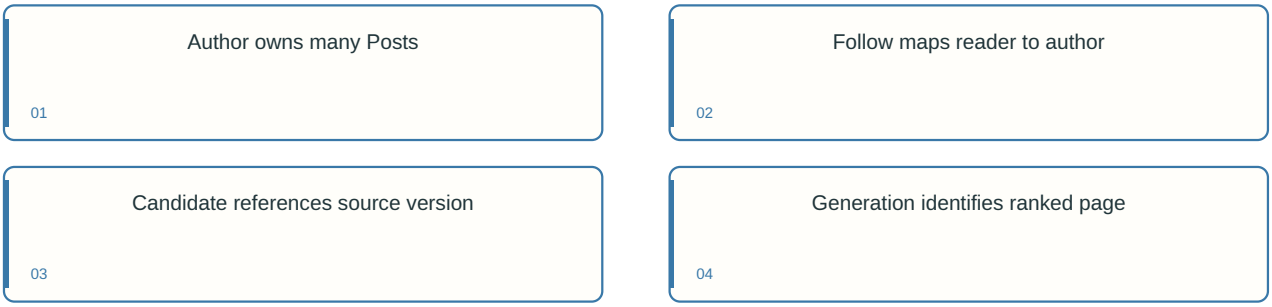


FIG. 22.5.1 **Data ownership and relationships**

Store post(id,author,version,visibility,state,created_at), follow(follower,author), and optional feed_candidate(reader,post_id,source_version,rank_features). Keep ranking model version and feed generation where reproducibility matters. Candidate references are derived; the post and authorization relationships remain authoritative. Rebuild and deletion propagation need stable source identity and progress.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.7 · Low-level component organization

PostService owns publication, FollowService owns relationship changes, CandidateGenerator retrieves or materializes references, FeatureFetcher supplies bounded ranking data, Ranker scores candidates, and VisibilityFilter enforces current policy. FeedAssembler deduplicates and paginates. Keep authorization filtering independent from ranking desirability so a high score can never override a deny decision.

22.5.8 · High-level system architecture

Use a durable publication path and outbox, fan-out workers for appropriate authors, a candidate store, pull-on-read retrieval for high-fan-out authors, ranking compute, and cached permitted results. A hybrid keeps ordinary writes manageable while avoiding synchronous fan-out to every celebrity follower. Control-plane model deployment is distinct from the feed-serving data path.

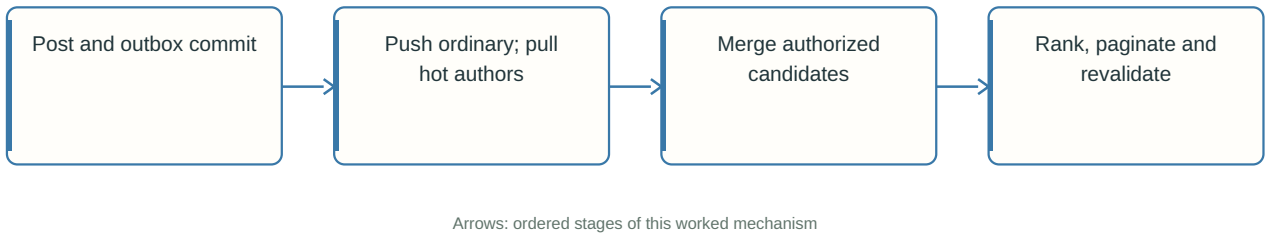


FIG. 22.5.2 **The high-level path and responsibility boundaries**

Use a durable publication path and outbox, fan-out workers for appropriate authors, a candidate store, pull-on-read retrieval for high-fan-out authors, ranking compute, and cached permitted results. A hybrid keeps ordinary writes manageable while avoiding synchronous fan-out to every celebrity follower. Control-plane model deployment is distinct from the feed-serving data path.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.9 · Data and control flows

Publish commits the post and fan-out intent. Workers append candidate references under stable identity. A feed read combines materialized and pulled candidates, checks current source state and permissions, fetches bounded features, ranks, deduplicates, and returns a cursor. Track which generation produced the page. A deleted candidate can remain physically present while being rejected at serving.

22.5.10 · Capacity estimates

Two hundred million reads/day gives about 2315/s average; an assumed fivefold peak is about 11574/s. One million posts times one hundred followers creates one hundred million references/day at the simplified average. A single author with ten million followers creates a separate burst and distribution problem. Estimate ranking candidates per read and feature lookup amplification independently.

22.5.11 · End-to-end latency estimates

Assume 20 ms edge, 50 ms candidate retrieval, 80 ms feature access, 70 ms ranking, and 30 ms response work: 250 ms before additional tail and queue margin. This is a budget, not a p99 measurement. Fan-out completeness and ranking freshness have separate indicators. One slow mandatory feature dependency can dominate a read, so define safe missing-feature behavior.

22.5.12 · Failure scenarios

Fan-out can lag or duplicate after worker restart; stable candidate identity makes replay safe. Celebrity publication can overload queues; classify it into the pull path or a controlled distribution plan. Privacy and deletion can lag candidate cleanup, so serving must validate authoritative policy. Ranking failure may fall back to an authorized chronological feed rather than fabricated or unauthorized content.

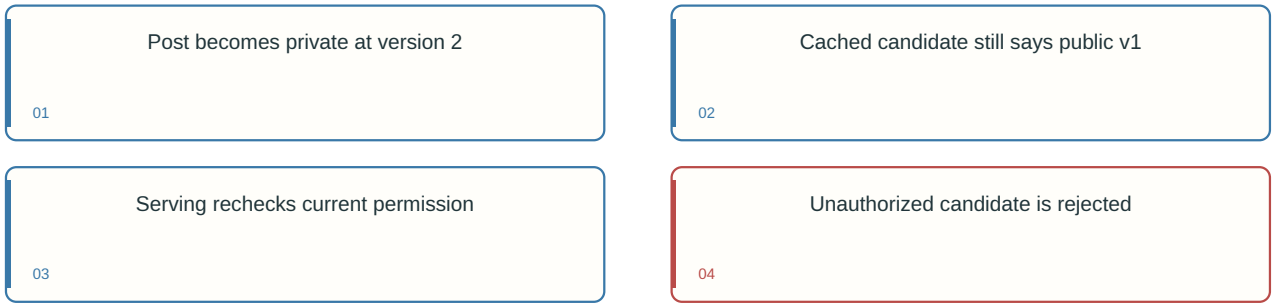


FIG. 22.5.3 **Naive path, failure, improved path, review**

Fan-out can lag or duplicate after worker restart; stable candidate identity makes replay safe. Celebrity publication can overload queues; classify it into the pull path or a controlled distribution plan. Privacy and deletion can lag candidate cleanup, so serving must validate authoritative policy. Ranking failure may fall back to an authorized chronological feed rather than fabricated or unauthorized content.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.13 · Scalability improvements

Scale candidate storage and workers by reader or another measured ownership key, isolate hot authors, cache safe features, and batch reads. Bound candidate counts and use separate capacity for rebuilds. Do not increase fan-out workers beyond storage or provider capacity. A hybrid threshold should be based on measured write amplification and read cost, with migration between strategies preserving duplicate suppression.

22.5.14 · Consistency requirements

Accepted post identity and deletion state need strong local transactional handling. Candidate projections and engagement counts can be eventually refreshed. Read-your-writes for an author's own post may require a direct-source overlay. Authorization is not allowed to become weaker merely because candidates are stale. State which parts of the feed are snapshots and which are revalidated at every serving operation.

22.5.15 · Security model

Enforce author and relationship permissions, privacy scope, blocks, moderation policy, and secure cursor validation. Do not allow client-supplied ranking features to bypass visibility. Bound scraped feed access and feature-query amplification. Minimize personal data in ranking logs, and propagate deletions to feature and analytical stores under the retention policy.

22.5.16 · Deployment approach

Release rankers by model version and canary cohorts with rollback compatible with stored features. Deploy fan-out schema changes with mixed-version readers. Separate rebuild jobs from live publication and feed serving. Multi-region distribution needs explicit source and privacy update behavior before it can claim local low-latency serving with consistent access decisions.

22.5.17 · Observability design

Observe publication-to-candidate lag, candidate count, authorization rejection, feed latency and errors, feature dependency tails, ranker version, duplicate rate, and hot-author backlog. Evaluate user-visible quality separately from infrastructure latency. A low feed p99 can still serve empty, repetitive, or stale content; correctness and relevance checks should accompany performance indicators.

22.5.18 · Infrastructure cost considerations

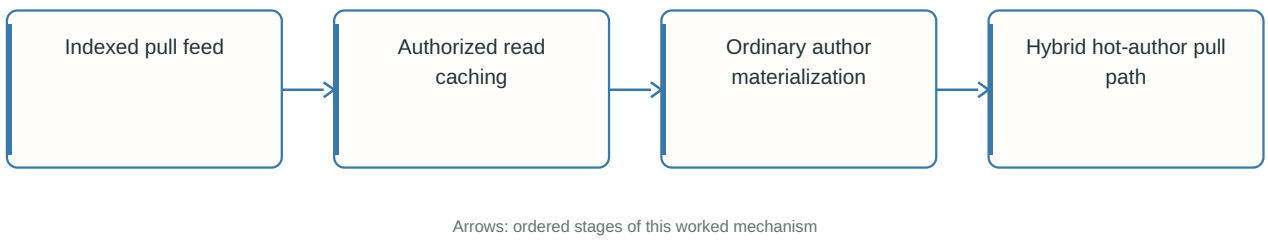
Cost grows through candidate-write amplification, feature storage and reads, ranking CPU, transfer, and rebuilds. Under the simplified model, 3.2 GB/day of raw references excludes indexes and metadata. Compare push and pull using expected active-reader fraction; writing feeds for inactive users may waste most work. Use hypothetical rates or current verified quotes rather than implying a universal cheap architecture.

22.5.19 · Alternative architectures and trade-offs

Fan-out-on-write gives fast reads at write amplification and hot-author cost. Fan-out-on-read reduces unused materialization but increases read work. Hybrid models handle different author traffic classes but add strategy and reconciliation complexity. Precomputed ranking reduces serving compute while increasing freshness lag. Compare alternatives with the same visibility and deletion guarantees, not only with average latency.

22.5.20 · Architecture evolution from small scale to large scale

Start with indexed pull reads. Add caching and bounded ranking. Materialize ordinary-author candidates when repeated read work dominates. Add celebrity pull and partitioned distribution after observing skew. Introduce regional copies only with explicit source-version and privacy rules. Recheck the threshold as reading activity, posting rates, and follower distribution evolve.



Arrows: ordered stages of this worked mechanism

FIG. 22.5.4 **Evolution follows demonstrated requirements**

Start with indexed pull reads. Add caching and bounded ranking. Materialize ordinary-author candidates when repeated read work dominates. Add celebrity pull and partitioned distribution after observing skew. Introduce regional copies only with explicit source-version and privacy rules. Recheck the threshold as reading activity, posting rates, and follower distribution evolve.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.21 · Design review identifying weaknesses and improvements

Review the assumptions about inactive readers, celebrity skew, and access revocation. Test a deleted or private post already present in several cached candidate pages, duplicate fan-out, a paused worker, and an unavailable feature service. Verify the fallback feed remains authorized. A queue drained to zero does not prove that every intended follower received exactly one usable candidate.

22.5.22 · A short implementation exercise

Implement push and pull candidate generation over a small follow graph with one high-degree author. Count writes and reads under explicit journeys. Delete a post after materialization, then verify the serving filter removes it. Replay one fan-out event and prove stable candidate identity prevents visible duplicates.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def feed(reader, cursor):
    candidates = materialized(reader) + pull_hot_authors(reader)
    unique = deduplicate_by_post_id(candidates)
    permitted = [p for p in unique if visible_now(reader, p)]
    features = fetch_bounded_features(permitted)
    ranked = rank(permitted, features, model_version)
    return page(ranked, validated_cursor(cursor, reader))
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Video Streaming Platform

Prerequisites: Networking Foundations for System Design, Caching and Content Delivery, File Systems, Search Infrastructure, and Data Pipelines, Deployment, Cloud Infrastructure, and Platform Architecture

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Upload videos, validate ownership and media input, transcode into an appropriate rendition set, publish a manifest, and deliver authorized playback through an edge network. Support resumable upload, processing status, and deletion. Adaptive playback chooses renditions according to client and network conditions; it cannot compensate for missing or incorrectly published segments.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.6.1 · Functional requirements

Upload videos, validate ownership and media input, transcode into an appropriate rendition set, publish a manifest, and deliver authorized playback through an edge network. Support resumable upload, processing status, and deletion. Adaptive playback chooses renditions according to client and network conditions; it cannot compensate for missing or incorrectly published segments.

22.6.2 · Nonfunctional requirements

Keep original and required derived media under a defined durability and retention contract. Target smooth playback and bounded startup for supported clients using measured delivery indicators. Processing can be asynchronous but must expose failed and review states. Private videos must remain protected through manifests, segments, thumbnails, and download links, not only the page that embeds a player.

22.6.3 · Explicit assumptions and workload numbers

Assume one thousand uploads/day, one GB per original, and thirty minutes average duration. Raw input is one TB/day. Assume five total rendition-output GB per source GB for the model, giving five TB/day derived output. Ten thousand concurrent viewers at an average three Mb/s imply thirty Gb/s, or 3.75 GB/s, before protocol overhead.

22.6.4 · Naive initial architecture

Start with one upload API and a single encoding worker writing one playback file. This is useful for a small prototype but ties large bytes and CPU to a fragile process and lacks adaptive delivery. Direct object upload and durable encoding tasks address those concrete constraints. Keep a simple supported rendition set rather than assuming every input requires many expensive formats immediately.

22.6.5 · Core API contracts

POST /videos creates an upload intent; multipart upload uses scoped authorization; POST /videos/{id}/complete validates stored content; GET /videos/{id}/status reports processing; GET /videos/{id}/playback returns an authorized manifest reference. Deletion tracks cleanup. Playback capability lifetime and cache behavior must be explicit so possession of an old page URL does not grant indefinite access.

22.6.6 · Data model and schema

Store video(id,owner,source_version,state), upload session, encoding job(input_digest,profile_version,attempt_epoch), rendition(video,profile,version,object_key,checksum), and manifest generation. Record required artifacts before publishing a generation. Never mutate segments behind an immutable published manifest without a version change. Deletion links every derived object and cacheable representation to ownership.

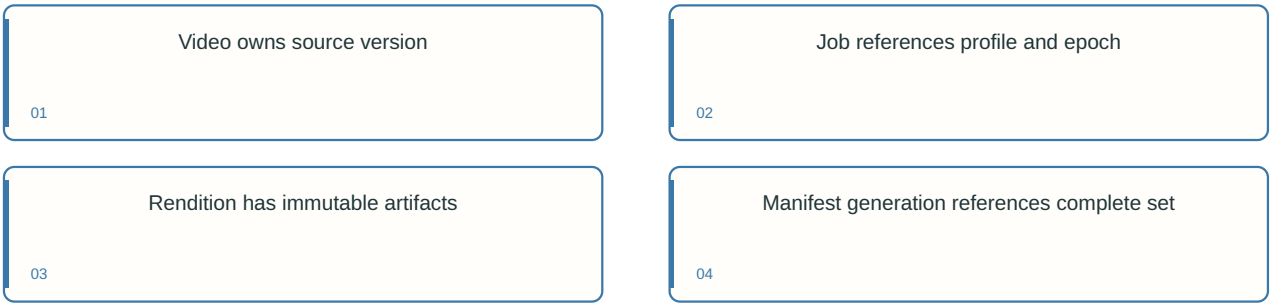


FIG. 22.6.1 **Data ownership and relationships**

Store video(id,owner,source_version,state), upload session, encoding job(input_digest,profile_version,attempt_epoch), rendition(video,profile,version,object_key,checksum), and manifest generation. Record required artifacts before publishing a generation. Never mutate segments behind an immutable published manifest without a version change. Deletion links every derived object and cacheable representation to ownership.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.7 · Low-level component organization

UploadService owns input authorization; MediaValidator checks bounded supported content; EncodingPlanner selects profiles; WorkerLease coordinates jobs; TranscoderAdapter runs isolated compute; ArtifactPublisher atomically advances published metadata when required outputs exist. PlaybackService authorizes access and issues scoped delivery references. Keep byte transformation separate from the metadata transition that declares a video playable.

22.6.8 · High-level system architecture

Use object storage for sources and immutable segments, a durable processing state machine and task queue, isolated transcoding workers, metadata storage, and CDN delivery. Source and derived artifacts have independent lifecycle policy. The application handles control and authorization while large media bytes flow directly through storage and edge paths. Workers need resource limits and restricted egress for untrusted input.

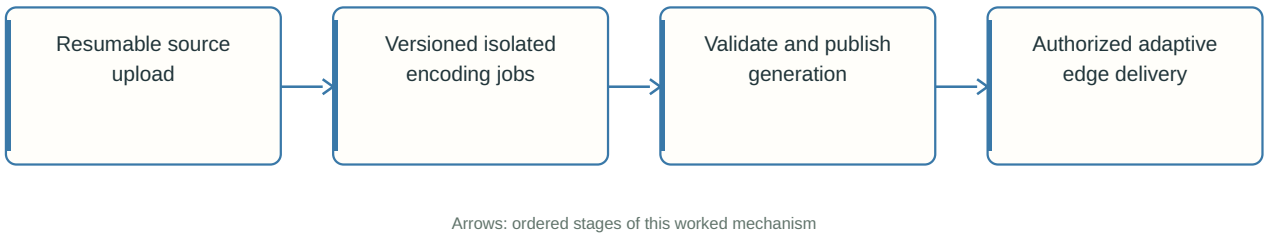


FIG. 22.6.2 **The high-level path and responsibility boundaries**

Use object storage for sources and immutable segments, a durable processing state machine and task queue, isolated transcoding workers, metadata storage, and CDN delivery. Source and derived artifacts have independent lifecycle policy. The application handles control and authorization while large media bytes flow directly through storage and edge paths. Workers need resource limits and restricted egress for untrusted input.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.9 · Data and control flows

Complete upload, verify content identity, plan versioned encoding tasks, produce segments and manifests under an attempt identity, validate artifacts, then publish the playable generation. A failed attempt leaves unpublished artifacts for cleanup. Clients fetch an authorized manifest and adapt among supported renditions. Do not mark ready after only the first segment or one partial output becomes available unless progressive publication is an explicit product contract.

22.6.10 · Capacity estimates

At one TB/day input and assumed fivefold output, thirty-day retention is thirty TB source plus 150 TB derived before replication and overhead. Thirty Gb/s viewer delivery dominates application API bandwidth. Encoding demand depends on source duration, codec, profiles, and measured compute ratio. A hypothetical two compute-hours per source-hour yields one thousand compute-hours/day for the assumed five hundred source-hours/day.

22.6.11 · End-to-end latency estimates

Playback startup includes manifest authorization, edge fetches, initial segment download, and player buffering. Assume 50 ms authorization, 30 ms manifest delivery, and 800 ms initial buffering: 880 ms under that illustrative path. A cache miss or constrained client changes it. Measure startup, rebuffering, rendition switches, and failures separately; a low metadata-API p99 does not establish playback quality.

22.6.12 · Failure scenarios

Uploads may stop midway, media may be malformed, encoding workers can fail after writing partial artifacts, and manifests can refer to missing segments. Track each state and publish only verified generations. Edge failure needs a delivery policy; a fallback origin can be overwhelmed by video traffic. Deletion must remove or invalidate playable references and derived copies according to the chosen revocation contract.

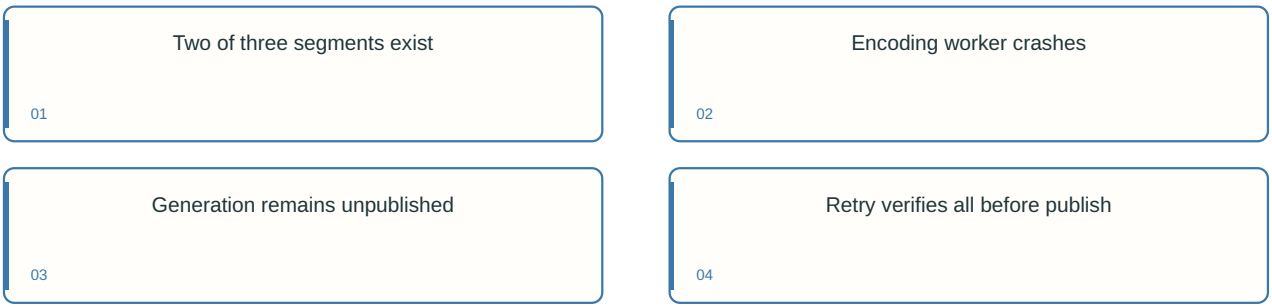


FIG. 22.6.3 **Naive path, failure, improved path, review**

Uploads may stop midway, media may be malformed, encoding workers can fail after writing partial artifacts, and manifests can refer to missing segments. Track each state and publish only verified generations. Edge failure needs a delivery policy; a fallback origin can be overwhelmed by video traffic. Deletion must remove or invalidate playable references and derived copies according to the chosen revocation contract.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.13 · Scalability improvements

Scale uploads by direct byte paths, encoding by bounded independent jobs, and playback by cacheable immutable segments and appropriate edge capacity. Batch or prioritize encoding based on business deadlines. A popular video is a hot object distribution workload rather than a reason to shard metadata blindly. Keep origin protection and regional replication cost explicit, especially during cache loss or a viral event.

22.6.14 · Consistency requirements

Metadata transitions need atomicity: a published generation identifies a complete usable artifact set. Encoding tasks can retry with idempotent output identity and conditional completion. Playback representations may be cacheable for a declared interval, but private access and deletion remain separate

correctness constraints. Cross-region object copies must satisfy the chosen publication and recovery protocol before they are used as authoritative failover sources.

22.6.15 · Security model

Authorize upload and playback, isolate parsers and transcoders handling untrusted media, restrict worker capabilities, validate file size and type, and protect signed URLs. Manifests and segments must not accidentally become public while metadata is private. Avoid logging sensitive access capabilities. Token expiry, CDN key construction, and revoked membership require end-to-end tests through actual delivery paths.

22.6.16 · Deployment approach

Roll out encoding profiles as versions and keep old outputs usable until migration is validated. Deploy workers with durable claims and graceful shutdown. Separate foreground playback from expensive backfills. Test rebuilding metadata and artifact inventories after recovery, including the case where source bytes survive but the only metadata database is restored to an earlier point.

22.6.17 · Observability design

Measure upload completion, encoding queue age, compute-to-media ratio, failed profiles, ready-state integrity, edge hit ratio, origin bandwidth, startup delay, rebuffering, and authorization outcomes. Correlate playback generation with source and profile version. Alert on missing required segments and old pending videos, not only worker exceptions. Verify a synthetic private-video journey including revocation.

22.6.18 · Infrastructure cost considerations

Model storage, repeated rendition generation, encoding compute, CDN requests, transfer, metadata, and telemetry. The assumed 3.75 GB/s sustained for one hour is 13.5 TB delivered. Peak duration matters more than a single concurrency label. Compare a smaller rendition set and retention policy against user quality needs, with explicit fictional or verified unit rates rather than fabricated provider bills.

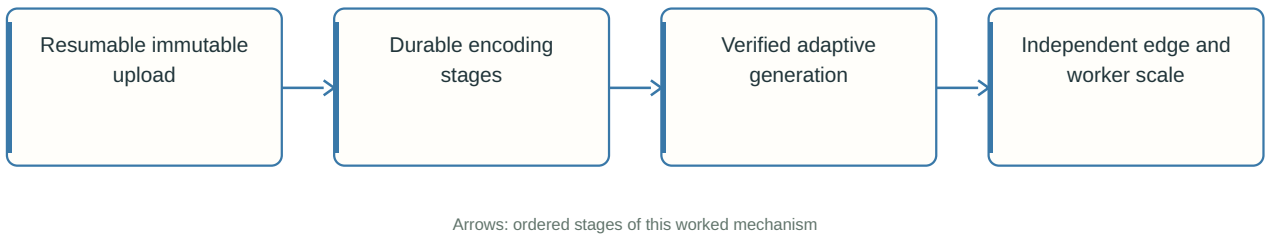
22.6.19 · Alternative architectures and trade-offs

A single downloadable file is simple but adapts poorly to variable networks; segmented adaptive delivery adds manifests and artifact management. Pre-encoding every profile improves readiness at cost and storage overhead; on-demand encoding reduces unused work but adds startup and burst risk. Managed

transcoding transfers selected operational tasks without removing publication, privacy, and reconciliation obligations.

22.6.20 · Architecture evolution from small scale to large scale

Begin with resumable source upload and durable job state. Add a tested small adaptive rendition set, then immutable versioned publication and CDN delivery. Scale encoding and delivery independently. Add regional artifact copies and advanced profiles only for demonstrated locality, recovery, or quality needs. Preserve source identity so a profile change can be explained and replayed.



Arrows: ordered stages of this worked mechanism

FIG. 22.6.4 **Evolution follows demonstrated requirements**

Begin with resumable source upload and durable job state. Add a tested small adaptive rendition set, then immutable versioned publication and CDN delivery. Scale encoding and delivery independently. Add regional artifact copies and advanced profiles only for demonstrated locality, recovery, or quality needs. Preserve source identity so a profile change can be explained and replayed.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.21 · Design review identifying weaknesses and improvements

Review whether ready means every required artifact is actually usable, whether private segments remain protected, and whether a cache outage overloads origin. Test corrupt input, worker interruption between segments and publish, repeated completion, missing segment, expired playback capability, and deletion after caching. A video that played once from a warm public URL does not establish the required platform behavior.

22.6.22 · A short implementation exercise

Build a toy publisher with three required segments. Write two, interrupt, and prove the manifest remains unpublished. Resume with the same job identity, add the third, validate checksums, and advance one

generation atomically. Then remove a segment deliberately and verify an integrity check detects the broken publication.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def publish(job):
    require_current_epoch(job)
    artifacts = expected_artifacts(job.source, job.profile)
    require(all(exists_and_valid(a) for a in artifacts))
    with transaction():
        assert_current_epoch(job)
        generation = record_complete_manifest(artifacts)
        mark_playable(job.video, generation)
# Partial worker output is never silently declared ready.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. RFC 8216: HTTP Live Streaming, 2017 ↗

   Original protocol description · Informational RFC. One adaptive streaming protocol; do not confuse Informational RFC with Standards Track.

   Access reviewed · 21 October 2026

2. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

3. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

Ride-Hailing System

Prerequisites: Data Models and Storage Technologies, Scalability, Load Balancing, and Data Distribution, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Receive driver location updates, let a rider request a trip, find plausible nearby available drivers, offer and accept an assignment, track trip state, and handle cancellation. Proximity is a candidate query; it is not permission to assign one driver to two trips. Define offer expiry and what a rider sees while matching is uncertain.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.7.1 · Functional requirements

Receive driver location updates, let a rider request a trip, find plausible nearby available drivers, offer and accept an assignment, track trip state, and handle cancellation. Proximity is a candidate query; it is not permission to assign one driver to two trips. Define offer expiry and what a rider sees while matching is uncertain.

22.7.2 · Nonfunctional requirements

Provide bounded dispatch latency under the assumed operating region, prevent conflicting active assignments, and protect precise location privacy. Freshness and accuracy of positions are explicit uncertainties. A location update may be delayed or inaccurate. A trip state machine must survive retries and temporary disconnection without treating a missing heartbeat as proof that a trip ended or payment failed.

22.7.3 · Explicit assumptions and workload numbers

Assume one hundred thousand online drivers reporting every five seconds: 20000 updates/s. At 100 bytes of raw position data that is 2 MB/s and 172.8 GB/day if every update is retained. Assume 100 new trip requests/s. Retention, map traffic, geographic skew, and dispatch candidate fan-out are separate planning dimensions.

22.7.4 · Naive initial architecture

Start with one service storing latest driver positions and sequentially selecting an available nearby driver. It is easy to test but can race as request workers multiply, and stale positions can produce poor candidates. Separate a location-serving projection from authoritative driver and trip assignment state. Improve candidate search only after preserving the exclusive assignment invariant.

22.7.5 · Core API contracts

PUT /drivers/me/location includes position, source sequence, and observed time; POST /trips creates a stable request; POST /offers/{id}/accept claims an offer; GET /trips/{id} reports state; cancellation is an explicit transition. Server derives driver identity and validates plausible updates. Accepting an expired or already superseded offer must return a defined conflict rather than overwrite a current trip.

22.7.6 · Data model and schema

Store driver_state(driver_id,availability,assignment_id,version), trip(id,rider,state,driver_id,version), offer(id,trip,driver,expiry,state), and latest_position(driver,sequence,cell,observed_at). Historical locations have separate restricted retention. A unique active-assignment rule or conditional ownership transition protects each driver. The spatial index is a derived candidate structure, not authoritative availability.

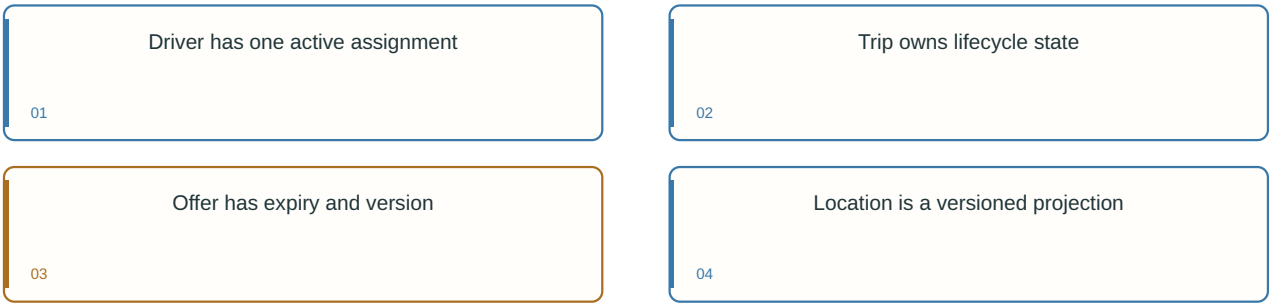


FIG. 22.7.1 **Data ownership and relationships**

Store driver_state(driver_id,availability,assignment_id,version), trip(id,rider,state,driver_id,version), offer(id,trip,driver,expiry,state), and latest_position(driver,sequence,cell,observed_at). Historical locations have separate restricted retention. A unique active-assignment rule or conditional ownership transition protects each driver. The spatial index is a derived candidate structure, not authoritative availability.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.7 · Low-level component organization

LocationIngestor validates and orders updates, SpatialIndex finds nearby candidates, Matcher ranks bounded candidates, OfferService coordinates expiry, AssignmentRepository atomically claims driver ownership, and TripStateMachine enforces transitions. Separate map and ETA adapters because their timeout should not corrupt assignment. Stable operation identities protect trip creation and offer acceptance across client retries.

22.7.8 · High-level system architecture

Use a regional location stream and spatial serving projection, a durable trip and assignment store, matching workers, offer delivery, and trip event publication. Partition geography for query locality while providing a protocol for drivers crossing cell boundaries. A global control plane can distribute configuration, but a regional assignment must not depend on stale independent copies of driver availability.

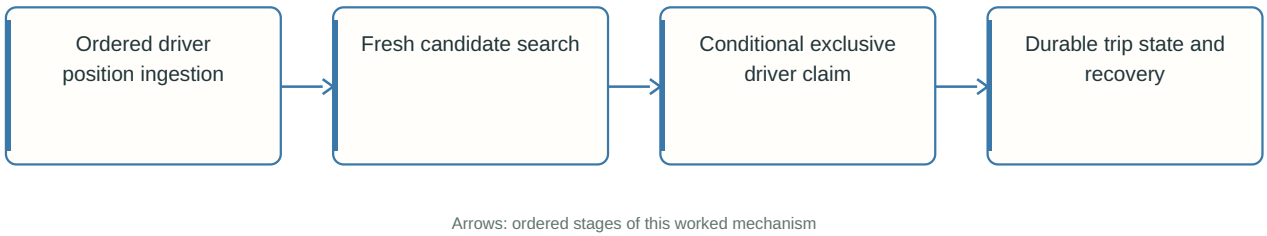


FIG. 22.7.2 **The high-level path and responsibility boundaries**

Use a regional location stream and spatial serving projection, a durable trip and assignment store, matching workers, offer delivery, and trip event publication. Partition geography for query locality while providing a protocol for drivers crossing cell boundaries. A global control plane can distribute configuration, but a regional assignment must not depend on stale independent copies of driver availability.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.9 · Data and control flows

A rider request commits one trip intent. Matcher reads fresh-enough location candidates, rechecks authoritative availability, creates an offer with expiry, and accepts only a valid conditional claim. Driver acceptance atomically binds the assignment and trip transition under the chosen ownership model. Delayed or repeated offers are rejected or replayed safely. Losing the acceptance reply creates an uncertain client view resolved by trip status.

22.7.10 · Capacity estimates

The assumed location rate is 20000/s, while dispatch is 100/s. If each dispatch evaluates twenty candidates, that is 2000 candidate evaluations/s before retries and ETA calls. Avoid calling a slow route service for every location update. Hot city centers and events can create localized overload even when global capacity looks ample; size partitions by regional peak and cross-cell movement.

22.7.11 · End-to-end latency estimates

Assume 30 ms request ingress, 40 ms spatial candidate retrieval, 80 ms bounded ETA evaluation, and 30 ms assignment work: 180 ms machine-side planning budget. Human offer acceptance is a separate interval and objective. Expiry, missing drivers, network delays, and candidate retries can extend matching. Measure those states explicitly rather than reporting a single optimistic “dispatch latency.”

22.7.12 · Failure scenarios

Late location updates can move a driver backward unless sequence or version checks reject them. Two requests can select one apparent driver; only one authoritative claim may win. A worker paused past offer expiry must not complete stale acceptance. Lost notifications require status recovery. A map-provider outage should use a justified degraded ranking or pause offers, not create duplicate assignments.

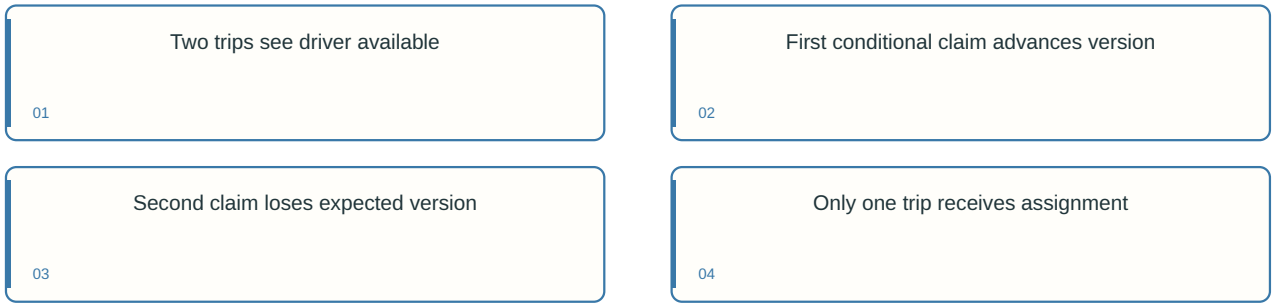


FIG. 22.7.3 **Naive path, failure, improved path, review**

Late location updates can move a driver backward unless sequence or version checks reject them. Two requests can select one apparent driver; only one authoritative claim may win. A worker paused past offer expiry must not complete stale acceptance. Lost notifications require status recovery. A map-provider outage should use a justified degraded ranking or pause offers, not create duplicate assignments.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.13 · Scalability improvements

Scale location ingestion and spatial queries independently from trip transactions. Partition by region or cell with a controlled handoff, retain current driver assignment authority, and batch map queries where appropriate. Introduce candidate caches only with freshness metadata. More matchers cannot solve a hot serialized assignment owner unless the state model changes while preserving exclusive claims.

22.7.14 · Consistency requirements

Location projections can be eventually refreshed with bounded freshness handling. Active assignment must have one authoritative conditional transition per driver and trip. Cross-region movement needs ownership transfer, not independent acceptance at both locations. Trip status should provide read-after-accept behavior where required. Timestamps help judge age but do not replace sequence or fencing for stale writes.

22.7.15 · Security model

Treat location as sensitive. Authenticate driver updates, restrict rider and driver visibility to appropriate trip context, protect operations and support access, and retain only justified history. Bound geographic scraping and map-query amplification. A rider must not choose an arbitrary driver identity in an acceptance request. Audit privileged location access and avoid placing raw coordinates in broad telemetry unnecessarily.

22.7.16 · Deployment approach

Deploy within justified regional failure domains, preserving durable trip and assignment state during rolling releases. Maintain compatible offer and trip state transitions across old clients. Draining must stop new assignment work while safely preserving in-flight offers. Disaster recovery must explicitly reconcile active trips and provider outcomes; restoring a stale snapshot without fencing can reoffer a currently assigned driver.

22.7.17 · Observability design

Measure location age, rejected stale updates, regional cell skew, candidate count, assignment conflicts, offer expiry, successful match rate, dispatch-state durations, and map-provider errors. Track trip creation and acceptance identities across retries. Alert on unresolved assignments and invariant violations, not only API errors. Simulated drivers should test movement, disconnection, expiry, and concurrent claims.

22.7.18 · Infrastructure cost considerations

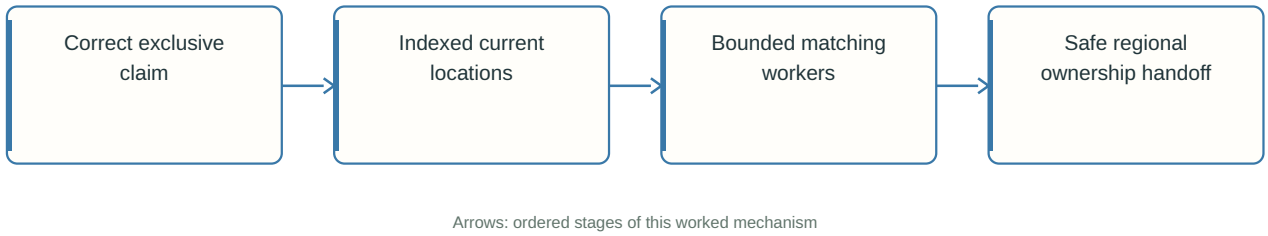
Costs include location ingestion, hot serving state, maps and ETA calls, assignment transactions, notification attempts, retained history, transfer, and operations. One ETA call per location update can dominate an otherwise efficient system. Model calls per dispatch candidate and compare pruning strategies. Use current verified commercial terms or explicit fictional units; do not infer an actual ride-hailing company's cost structure.

22.7.19 · Alternative architectures and trade-offs

Simple distance ranking is cheap but ignores road travel; route-based ETA can improve matching at higher latency and external dependence. Push offers one at a time to reduce conflicting acceptance, or use a controlled multi-offer protocol with exactly one claim winner. Fine spatial cells improve locality but add cross-boundary queries and movement work. Compare fairness and user outcomes as well as computational cost.

22.7.20 · Architecture evolution from small scale to large scale

Start with a correct regional assignment transaction and basic proximity query. Add location streaming and indexed candidates when query cost grows. Introduce independent matching workers and bounded ETA work. Partition regions or cells with explicit authority transfer only after observing hot areas. Add multi-region serving with a documented partition policy for active assignments rather than copying a stateless topology.



Arrows: ordered stages of this worked mechanism

FIG. 22.7.4 **Evolution follows demonstrated requirements**

Start with a correct regional assignment transaction and basic proximity query. Add location streaming and indexed candidates when query cost grows. Introduce independent matching workers and bounded ETA work. Partition regions or cells with explicit authority transfer only after observing hot areas. Add multi-region serving with a documented partition policy for active assignments rather than copying a stateless topology.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.21 · Design review identifying weaknesses and improvements

Review the distinction between nearby, recently observed, available, offered, and assigned. Test two riders selecting one driver, out-of-order positions, acceptance after expiry, worker pause, and region handoff. Verify one claim wins and the loser receives a meaningful state. The spatial index should improve candidate quality without becoming a second conflicting assignment authority.

22.7.22 · A short implementation exercise

Implement an optimistic driver claim with an expected version and an offer expiry. Start two requests from the same version and prove exactly one claim succeeds locally. Then deliver a stale location and stale acceptance after the winner commits. Verify neither can overwrite the active assignment, and document which part is a local invariant rather than an external end-to-end guarantee.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def accept_offer(offer, driver, now):
    with transaction():
        current = lock_offer(offer.id)
        require(current.pending and now < current.expires_at)
        won = claim_driver_if_available(driver, current.expected_version)
        if not won: raise AssignmentConflict
        bind_trip_and_offer(offer.trip, driver, offer.id)
    return current_trip_state(offer.trip)
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. PostGIS: ST_DWithin ↗

   Official implementation documentation · accessed 21 Oct 2026. Coordinate-system-aware proximity predicate, not exclusive dispatch assignment.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

3. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

Online Ticket Booking

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Security Architecture and Privacy

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Browse events and availability, reserve one or more seats for a defined interval, pay against a reservation, issue tickets, and cancel or refund under policy. Availability display is advisory; authoritative reservation determines ownership. A seat must not become sold to two buyers even when retries, expiry, and payment callbacks overlap.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.8.1 · Functional requirements

Browse events and availability, reserve one or more seats for a defined interval, pay against a reservation, issue tickets, and cancel or refund under policy. Availability display is advisory; authoritative reservation determines ownership. A seat must not become sold to two buyers even when retries, expiry, and payment callbacks overlap.

22.8.2 · Nonfunctional requirements

Prevent duplicate confirmed seat ownership, retain accepted financial and ticket records under the stated failure contract, and resolve pending reservations through payment or expiry. Set explicit objectives for booking latency and waiting-room fairness during launches. A 100% acceptance goal during a fixed-seat spike is incoherent; the service should reject or queue excess demand clearly rather than oversell.

22.8.3 · Explicit assumptions and workload numbers

Assume an event has fifty thousand seats and a launch reaches one hundred thousand reservation attempts/s for a brief peak. Assume 200 bytes per seat record before indexes: 10 MB raw for that event. Request concurrency and contention, rather than raw seat bytes, dominate the launch. Assume a two-minute reservation period and bounded per-user seat count.

22.8.4 · Naive initial architecture

Begin with a transactional seat table and a conditional update from available to reserved. A read followed by an unconditional write is incorrect under concurrency even on one database. Add a reservation record in the same atomic boundary. This can handle a smaller workload and makes ownership clear; flash traffic, hot-event contention, payment uncertainty, and expiry races then motivate admission and workflow improvements.

22.8.5 · Core API contracts

POST /reservations includes event, seat IDs, and idempotency key; GET /reservations/{id} reports state and expiry; POST /reservations/{id}/payments initiates the payment workflow; GET /tickets lists authorized issued tickets. Repeat keys replay an established result. A payment response must distinguish authorization, capture, decline, and uncertainty according to the provider contract.

22.8.6 · Data model and schema

Store seat(event_id,seat_id,state,reservation_id,version), reservation(id,owner,state,expires_at,version), reservation_seat, payment_operation, and ticket(unique_event_seat,owner,state). Enforce one confirmed active ticket per seat under the chosen model. Expiry and confirmation both check reservation identity and current version; an old expiry task must not release a seat assigned by a later reservation.

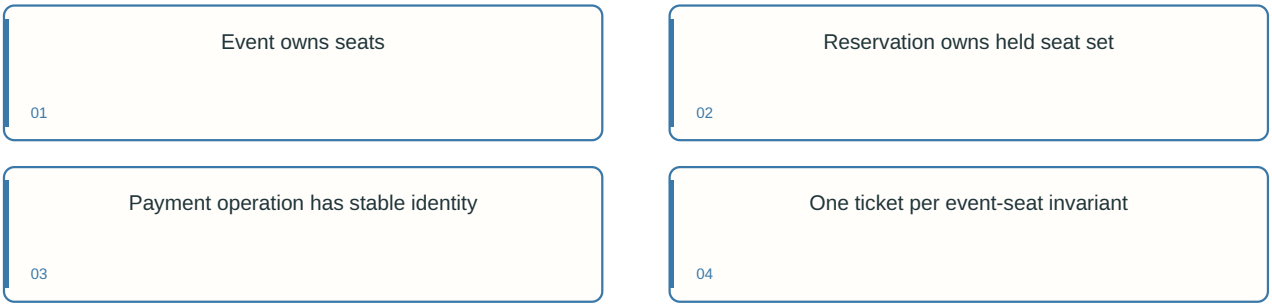


FIG. 22.8.1 **Data ownership and relationships**

Store seat(event_id,seat_id,state,reservation_id,version), reservation(id,owner,state,expires_at,version), reservation_seat, payment_operation, and ticket(unique_event_seat,owner,state). Enforce one confirmed active ticket per seat under the chosen model. Expiry and confirmation both check reservation identity and current version; an old expiry task must not release a seat assigned by a later reservation.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.7 · Low-level component organization

SeatRepository provides conditional claims and locks, ReservationService owns expiry and selection, PaymentWorkflow coordinates external effects, TicketIssuer commits authorized issuance, and ExpiryWorker applies version-checked release. Pricing and refund policy are domain logic with explicit values and units. Do not make the HTTP handler the only place that enforces ownership; workers and callbacks need the same invariant boundary.

22.8.8 · High-level system architecture

Use a waiting room or bounded admission before reservation work, an authoritative event/seat ownership store, durable reservation state, payment adapter, outbox or workflow engine, and ticket read model. Cache event descriptions and advisory maps separately from authoritative allocation. Partitioning by event can preserve local seat transactions but a hot event remains a concentrated workload needing admission and contention control.

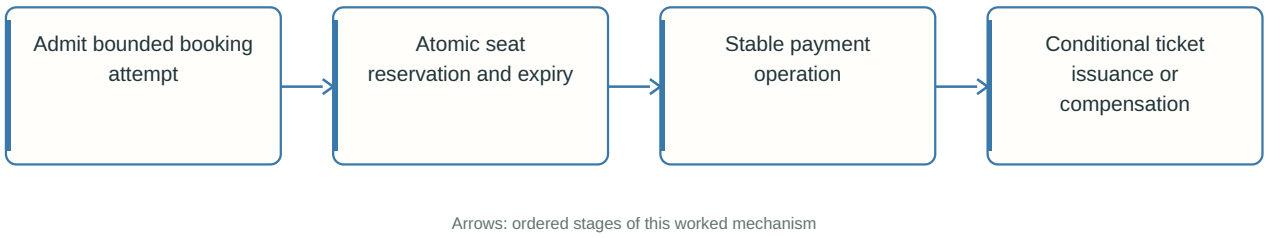


FIG. 22.8.2 **The high-level path and responsibility boundaries**

Use a waiting room or bounded admission before reservation work, an authoritative event/seat ownership store, durable reservation state, payment adapter, outbox or workflow engine, and ticket read model. Cache event descriptions and advisory maps separately from authoritative allocation. Partitioning by event can preserve local seat transactions but a hot event remains a concentrated workload needing admission and contention control.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.9 · Data and control flows

Claim seats atomically with the reservation, commit, and return its deadline. Payment workflow submits a stable provider operation. Confirmation checks a still-valid authorized transition and commits ticket issuance plus reservation/seat state. If payment succeeds after expiry, follow a defined recovery policy such as reacquisition if safe or refund; do not silently issue a ticket for a now-owned seat. Expiry uses conditional versioned release.

22.8.10 · Capacity estimates

At one hundred thousand attempts/s, admitting an illustrative two thousand/s caps authoritative pressure, while the remaining callers receive waiting or rejection according to policy. The rate is a hypothetical configured limit to validate. A two-minute hold creates at most seat-count-bounded held inventory, but retries can still generate enormous control traffic. Estimate storage writes per attempt and per-seat contention, not only total users.

22.8.11 · End-to-end latency estimates

An illustrative admitted reservation path budgets 20 ms edge, 30 ms transaction acquisition, 40 ms conditional seat and reservation work, and 20 ms response: 110 ms. Contended locks can exceed it, so bound waiting and reject stale demand. Payment completion is asynchronous with its own expiry-sensitive objective. Report time in waiting room separately from the server's admitted-request latency.

22.8.12 · Failure scenarios

Two users reading available must not both win. A process can crash after reservation commit but before reply; idempotency must recover it. Payment can succeed after the hold expires, and stale expiry work can arrive after a renewal or new reservation. Version-checked transitions and reconciliation prevent these from corrupting ownership. Email delivery failure must not imply ticket issuance failed if the ticket already committed.

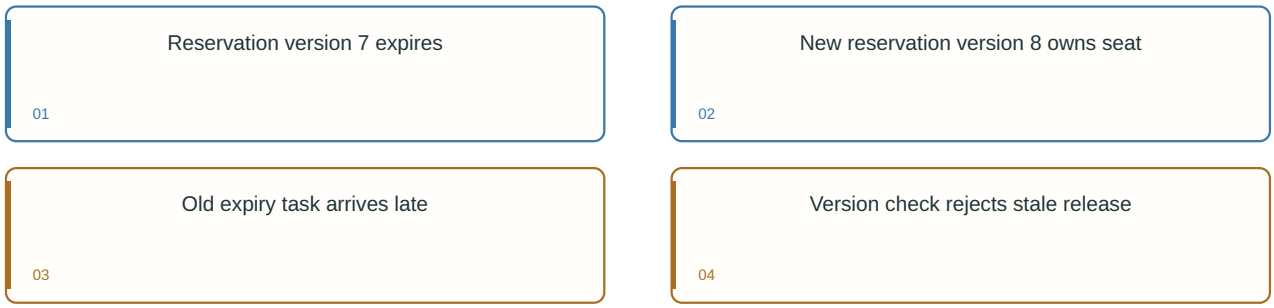


FIG. 22.8.3 **Naive path, failure, improved path, review**

Two users reading available must not both win. A process can crash after reservation commit but before reply; idempotency must recover it. Payment can succeed after the hold expires, and stale expiry work can arrive after a renewal or new reservation. Version-checked transitions and reconciliation prevent these from corrupting ownership. Email delivery failure must not imply ticket issuance failed if the ticket already committed.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.13 · Scalability improvements

Protect hot launches with bounded admission, cache only advisory data, optimize atomic seat transactions, and partition independent events where appropriate. Multi-seat reservations need a deliberate atomicity boundary and consistent lock order to avoid deadlock. Splitting one event across shards introduces cross-shard group-booking complexity; do not assume more shards are automatically useful for one hot seat.

22.8.14 · Consistency requirements

Seat ownership and issuance require a strong atomic invariant in the authoritative boundary. Availability display can lag and must be labeled advisory. Payment and email are separate external effects managed through stable operation identity and workflow state. A local serializable transaction is one possible tool, not

a universal guarantee covering the payment provider. Handle serialization failures through bounded whole-transaction retries.

22.8.15 · Security model

Authorize reservations and ticket views, bound bots and bulk attempts, validate prices server-side, protect provider credentials, and verify callbacks. An idempotency key must be scoped to a trusted actor so one user's replay cannot disclose another's result. Use short-lived capabilities for ticket download if needed, and audit transfers or refunds. A waiting room is capacity protection, not proof of authorization or fair identity.

22.8.16 · Deployment approach

Deploy with compatible reservation states and safe migration of expiry semantics. Persist scheduled expiry intent so replacing workers does not forget held seats. Test database promotion under the promised acknowledgment policy and reconcile provider outcomes after recovery. During uncertain authority or recovery, pausing allocation can protect the seat invariant even if read-only event pages remain available.

22.8.17 · Observability design

Measure admitted attempts, claim conflicts, lock waits, reservation age, expiry backlog, payment uncertainty, late successes, ticket uniqueness, and user-visible outcome latency. Track each logical booking through retries and callbacks. Periodic invariant scans should detect duplicate or inconsistent seat ownership. An empty error log does not prove seats and payments reconcile correctly.

22.8.18 · Infrastructure cost considerations

Cost includes flash-traffic admission, transactional contention, state retention, external payment operations, delivery, telemetry, and recovery capacity. Compare a short launch peak with baseline traffic instead of pricing the peak as permanent demand. Overprovisioning web workers cannot fix one hot transactional seat. Use explicit fictional or verified unit rates and include abandoned hold and retry processing.

22.8.19 · Alternative architectures and trade-offs

Pessimistic locks provide explicit serialization but can queue and deadlock; conditional optimistic claims fail conflicts quickly but require retry and user feedback. A single owner per event can simplify ordering at a

capacity ceiling. Distributed inventory allocation can reduce central pressure while complicating multi-seat booking and fairness. Choose the mechanism that meets the actual seat and reservation contract.

22.8.20 · Architecture evolution from small scale to large scale

Start with a correct transaction and repeat-key behavior. Add durable expiry and payment reconciliation. Introduce admission and advisory caching for launches. Partition independent events after measurement, and consider more complex seat allocation only if one-event constraints justify it. Verify the same no-double-ticket invariant at every architectural stage.

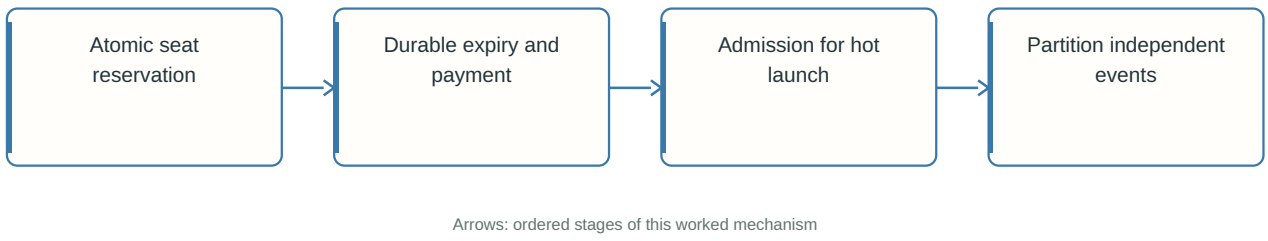


FIG. 22.8.4 **Evolution follows demonstrated requirements**

Start with a correct transaction and repeat-key behavior. Add durable expiry and payment reconciliation. Introduce admission and advisory caching for launches. Partition independent events after measurement, and consider more complex seat allocation only if one-event constraints justify it. Verify the same no-double-ticket invariant at every architectural stage.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.21 · Design review identifying weaknesses and improvements

Review whether expiry and payment use the same reservation version, whether late payment is resolved safely, and whether the display is confused with authority. Test two buyers for one seat, duplicate callbacks, crash after commit, stale expiry after reassignment, and payment success after expiry. A correct happy-path lock does not answer those workflow histories.

22.8.22 · A short implementation exercise

Implement the last-seat race with two connections or an explicit interleaving. Show the naive read/write oversell, then use a conditional update plus reservation transaction and check affected rows. Add versioned expiry, confirm or reassign a reservation, and prove an old expiry task cannot release the new owner's seat.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
with transaction():
    insert_reservation(reservation_id, owner, expires_at)
    updated = reserve_if_available(event, seat, reservation_id)
    if updated != 1: raise SeatUnavailable
# Expiry later releases only the same current reservation version.
# Provider payment is coordinated outside this local transaction.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

3. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

Payment System

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create a payment intent, submit authorization or capture under explicit semantics, report status, record balanced ledger entries, reconcile provider evidence, and support tracked refunds. Separate business order state, provider operation state, and accounting state. Money has currency and integer minor units with an explicit rounding policy; a database row labeled paid is not a complete payment or ledger design.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.9.1 · Functional requirements

Create a payment intent, submit authorization or capture under explicit semantics, report status, record balanced ledger entries, reconcile provider evidence, and support tracked refunds. Separate business order state, provider operation state, and accounting state. Money has currency and integer minor units with an explicit rounding policy; a database row labeled paid is not a complete payment or ledger design.

22.9.2 · Nonfunctional requirements

Preserve financial invariants, prevent duplicate relevant provider effects where supported, retain auditable state, and resolve uncertain outcomes. Favor safe pending states over fabricated success during a provider outage. Target a short intent-acceptance latency separately from provider completion. The system must

explain every settled amount through unique, balanced postings and authorized corrections under the domain's accounting rules.

22.9.3 · Explicit assumptions and workload numbers

Assume one million payment intents/day, averaging about 11.6/s, with an assumed peak of 200/s. Assume six one-kB operational/audit records per intent for an initial raw 6 GB/day model. Ledger entries, indexes, replicas, provider events, and retention add storage. Use a synthetic currency unit model; no actual provider fees, financial approvals, or institution-specific internal implementation is claimed.

22.9.4 · Naive initial architecture

Begin with one service and one transactional database. A naive flow charges the provider, updates an order flag, and returns success. It fails if the provider charges but the local update is lost, or if a retry charges again. Introduce a durable operation identity and state machine before scaling. Local transaction correctness and external provider reconciliation solve different parts of the problem.

22.9.5 · Core API contracts

POST /payment-intents accepts amount_minor, currency, business_reference, and idempotency key. GET /payment-intents/{id} reports pending, authorized, captured, declined, uncertain, or other explicitly supported states. POST /payments/{id}/refunds creates an independently keyed refund. Webhooks verify provider authenticity and do not trust client-supplied approval fields. Reusing a key with a different financial payload conflicts.

22.9.6 · Data model and schema

Store payment_intent, operation(actor,key,fingerprint,state,result), provider_attempt(reference,key,state), provider_event inbox, ledger_transaction, and ledger_entry(transaction,account,currency,signed_amount). A transaction's entries must balance per relevant currency and accounting scope. Enforce uniqueness of external operation linkage and correction identity. A ledger correction is a new authorized accounting event rather than an untracked rewrite of history.

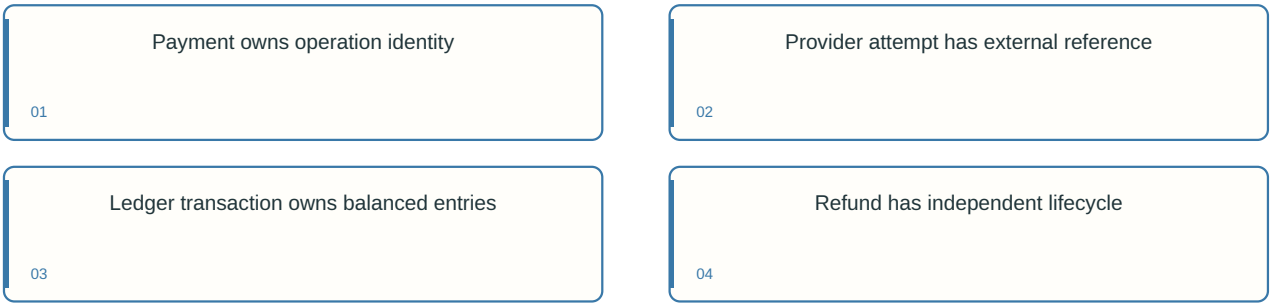


FIG. 22.9.1 **Data ownership and relationships**

Store payment_intent, operation(actor,key,fingerprint,state,result), provider_attempt(reference,key,state), provider_event inbox, ledger_transaction, and ledger_entry(transaction,account,currency,signed_amount). A transaction's entries must balance per relevant currency and accounting scope. Enforce uniqueness of external operation linkage and correction identity. A ledger correction is a new authorized accounting event rather than an untracked rewrite of history.

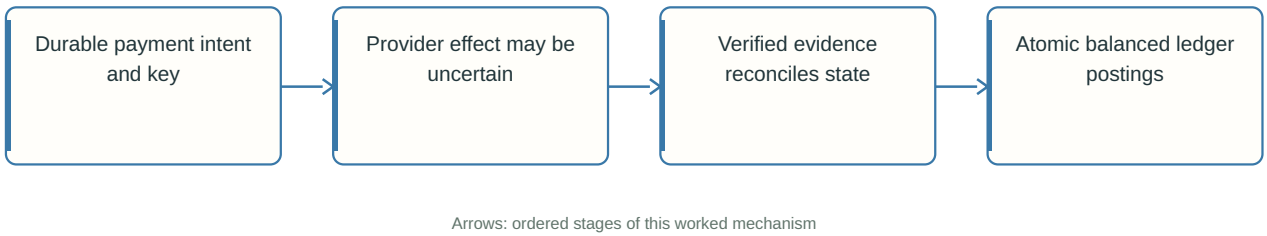
Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.7 · Low-level component organization

PaymentService owns business transitions, Money and policy values validate units, ProviderAdapter maps explicit outcomes, OperationRepository owns deduplication, LedgerService validates and commits postings, Reconciler compares external evidence, and RefundWorkflow tracks compensation. A provider timeout yields uncertainty, not decline. Keep these responsibilities separate without claiming each must be a separate microservice.

22.9.8 · High-level system architecture

Use an authenticated API, durable intent/operation store, bounded payment workers, provider adapters, verified callback inbox, ledger boundary, and reconciliation scheduler. Persist publication intent with state changes where notifications or downstream orders need reliable propagation. Protect critical reconciliation capacity from marketing or analytics work. A regional deployment choice must preserve one authoritative ledger and provider-operation contract.



Arrows: ordered stages of this worked mechanism

FIG. 22.9.2 **The high-level path and responsibility boundaries**

Use an authenticated API, durable intent/operation store, bounded payment workers, provider adapters, verified callback inbox, ledger boundary, and reconciliation scheduler. Persist publication intent with state changes where notifications or downstream orders need reliable propagation. Protect critical reconciliation capacity from marketing or analytics work. A regional deployment choice must preserve one authoritative ledger and provider-operation contract.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.9 · Data and control flows

Commit intent and operation identity, dispatch a stable provider key, then record authoritative response or uncertainty. Apply settlement or capture evidence through an idempotent transition that creates balanced ledger postings once. Verified callbacks can arrive late or before a synchronous response and must not regress state incorrectly. Reconciliation queries stable provider references and repairs discrepancies without blindly resubmitting a charge.

22.9.10 · Capacity estimates

One million intents/day is about 11.6/s average; the assumed peak 200/s drives concurrency planning. At a hypothetical 500 ms provider wait, Little's Law implies about one hundred in-flight calls at stable 200/s before other stages. This is not a safe pool size recommendation; include variance, deadlines, provider limits, and failure behavior. The raw operational-record model is 6 GB/day before overhead.

22.9.11 · End-to-end latency estimates

Assume 20 ms ingress, 20 ms local intent commit, 500 ms provider exchange, and 20 ms final local update: 560 ms for a successful synchronous path. It is an arithmetic scenario, not measured p99 or a completion guarantee. Returning a durable pending intent in 40 ms changes the API contract and separates completion latency. Timeouts require status and reconciliation, not an automatic failure interpretation.

22.9.12 · Failure scenarios

Critical histories include commit followed by lost reply, provider charge followed by process loss, duplicate or out-of-order callbacks, refund timeout, and ledger commit failure. Persist uncertainty and retry only under stable provider identity or established safe evidence. An unavailable ledger should not be bypassed by marking the order paid. Recovery must reconcile provider state and postings before asserting the business outcome is complete.

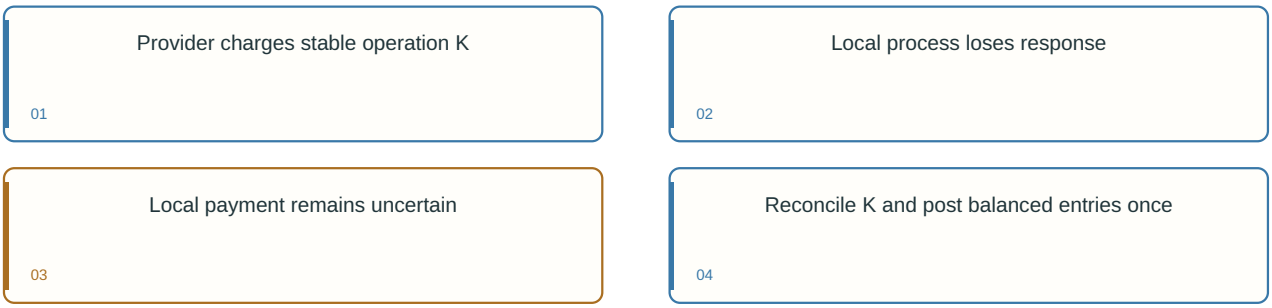


FIG. 22.9.3 **Naive path, failure, improved path, review**

Critical histories include commit followed by lost reply, provider charge followed by process loss, duplicate or out-of-order callbacks, refund timeout, and ledger commit failure. Persist uncertainty and retry only under stable provider identity or established safe evidence. An unavailable ledger should not be bypassed by marking the order paid. Recovery must reconcile provider state and postings before asserting the business outcome is complete.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.13 · Scalability improvements

Scale stateless acceptance and independent operation processing while preserving per-payment and per-account invariants. Partitioning ledger data requires deliberate transaction and balance scope; a hot account can remain a serialization point. Use bounded provider concurrency and isolate reconciliation. Read projections can scale status views but must expose freshness where important. Do not shard solely from a daily-intent count that is modest in average rate.

22.9.14 · Consistency requirements

Deduplication, payment-state changes, and required local postings need appropriate atomicity. Provider effects are outside that local boundary and rely on provider identity and reconciliation contracts. Exactly-once local posting can be enforced with uniqueness and transaction semantics; exactly-once external

execution is not implied. Refunds have their own lifecycle and may not perfectly reverse timing, fees, or downstream business consequences.

22.9.15 · Security model

Authorize payment and refund scope, validate financial values on the server, protect keys and callbacks, minimize retained sensitive data, and maintain auditable privileged actions. Use appropriate reviewed provider tokenization or integration rather than storing unnecessary raw payment credentials. An internal operator role is not unlimited authority to rewrite ledger history. Keep tenant and account identity binding explicit on every effect and query.

22.9.16 · Deployment approach

Deploy with backward-compatible state and event schemas, controlled provider configuration, and safe key rotation. Preserve immutable operation identities across releases and disaster recovery. A stale database restore can forget deduplication records for effects already executed externally, so reconciliation and replay protection must be part of recovery. Test old callbacks arriving after a new release and failback after a provider outage.

22.9.17 · Observability design

Measure acceptance and completion latency separately, pending and uncertain age, provider attempt counts, duplicate suppression, callback verification failures, ledger imbalance, unmatched settlements, refunds, and reconciliation progress. Alert on invariant violations and old unresolved money movement, not only HTTP failures. Audit evidence must distinguish attempted, accepted, and posted operations with stable identities.

22.9.18 · Infrastructure cost considerations

Cost includes provider operations and retries, secure storage, ledger transactions, audit retention, telemetry, reconciliation compute, and operating controls. Use symbolic fees and explicit hypothetical infrastructure rates. A lower-cost design that cannot explain an uncertain charge is not equivalent. Cost per reconciled successful business payment is more meaningful than cost per HTTP attempt, especially during retry-heavy incidents.

22.9.19 · Alternative architectures and trade-offs

Synchronous completion is easier for some clients but couples them to provider latency and uncertainty. Asynchronous intents provide explicit pending state at workflow cost. One local transactional service simplifies invariants; separate provider and ledger services can isolate responsibilities but add coordination. Compare append-only ledger history with mutable operational projections as complementary structures rather than assuming every payment record must serve all purposes.

22.9.20 · Architecture evolution from small scale to large scale

Start with explicit states, integer money, local transactions, and stable provider operation identity. Add durable callback ingestion and reconciliation before scale. Introduce projections and bounded workers for actual load. Partition only a demonstrated bottleneck with a preserved invariant boundary. A larger topology should improve a measured need while keeping every prior payment explainable through its operation and posting history.

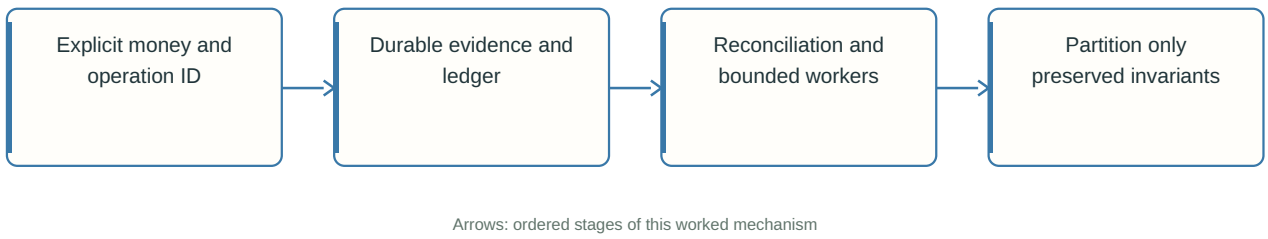


FIG. 22.9.4 **Evolution follows demonstrated requirements**

Start with explicit states, integer money, local transactions, and stable provider operation identity. Add durable callback ingestion and reconciliation before scale. Introduce projections and bounded workers for actual load. Partition only a demonstrated bottleneck with a preserved invariant boundary. A larger topology should improve a measured need while keeping every prior payment explainable through its operation and posting history.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.21 · Design review identifying weaknesses and improvements

Review which event means financial success, who owns the ledger invariant, and what recovery does after deduplication history is restored behind provider effects. Test charge-then-crash, callback-before-response, duplicate settlement, partial refund, and restore from an older snapshot. Verify no duplicate local postings and no unsupported claim that an unknown provider outcome is safely declined.

22.9.22 · A short implementation exercise

Implement a synthetic provider with a stable operation key and a local balanced-entry ledger. Inject loss after provider execution but before local recording. Reconcile by key and prove one logical operation yields one balanced local posting set. Then replay the provider event and reject a mismatched idempotency payload without altering the established result.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def apply_settlement(event):
    with transaction():
        if inbox_contains(event.id): return prior_outcome(event.id)
        payment = lock_payment(event.payment_id)
        validate_transition_and_amount(payment, event)
        entries = [debit_entry(event), credit_entry(event)]
        require_balanced_by_currency(entries)
        insert_unique_posting_set(event.operation_id, entries)
        update_payment_and_insert_inbox(payment, event)
# Provider execution is reconciled separately from this transaction.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author’s explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

Distributed Job Scheduler

Prerequisites: Runtime, Processes, Threads, and Concurrency, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create durable schedules, determine due occurrences, execute jobs on workers, retry under policy, cancel future work, and expose history. Define fixed-time versus interval semantics, timezone handling, missed-run behavior, and whether overlapping runs are allowed. A schedule and a particular occurrence are distinct identities; worker restarts must not invent new occurrences for the same logical due time.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.10.1 · Functional requirements

Create durable schedules, determine due occurrences, execute jobs on workers, retry under policy, cancel future work, and expose history. Define fixed-time versus interval semantics, timezone handling, missed-run behavior, and whether overlapping runs are allowed. A schedule and a particular occurrence are distinct identities; worker restarts must not invent new occurrences for the same logical due time.

22.10.2 · Nonfunctional requirements

Retain accepted schedules and runs, avoid losing due work under supported failures, and prevent stale attempts from corrupting newer outcomes. Execution may start more than once around failures, so effects need stable identity or fencing. Target bounded scheduling delay under the assumed load. Cancellation must specify whether it affects only future dispatch, pending runs, or best-effort interruption of an already running external effect.

22.10.3 · Explicit assumptions and workload numbers

Assume one million schedules, with one hundred thousand occurrences/hour on average: about 27.8 runs/s. Assume 2 kB of run state initially, giving 200 MB/hour raw. Ten-second average work implies about 278 in-flight executions at a stable average rate under Little's Law. Bursts at round minutes, long jobs, retries, and retention increase the requirements substantially.

22.10.4 · Naive initial architecture

Begin with one process scanning a table for due schedules and starting local functions. It can miss work if schedule advancement and run creation are not atomic, or repeat work after a restart if identity is unstable. Add a durable occurrence record and transactional due-state transition. A leader process can simplify scanning, but leadership alone does not make worker effects exactly once.

22.10.5 · Core API contracts

POST /schedules defines timing, timezone, payload reference, retry and overlap policy; POST /schedules/{id}/cancel changes future behavior; GET /runs/{id} exposes attempt and outcome. A run-once request has its own idempotency key. Bound schedule expression and payload complexity. Return accepted durable identity separately from execution completion and external effect status.

22.10.6 · Data model and schema

Store schedule(id,version,next_due,timing_policy,state), run(schedule_id,occurrence UNIQUE,state,input_version), attempt(run_id,epoch,lease_until,worker,result), and relevant outbox or effect records. Make occurrence identity stable through leadership change. Keep original scheduled time and observed dispatch time distinct. Attempts are not interchangeable with runs, and an old attempt must not overwrite a newer attempt's final state.

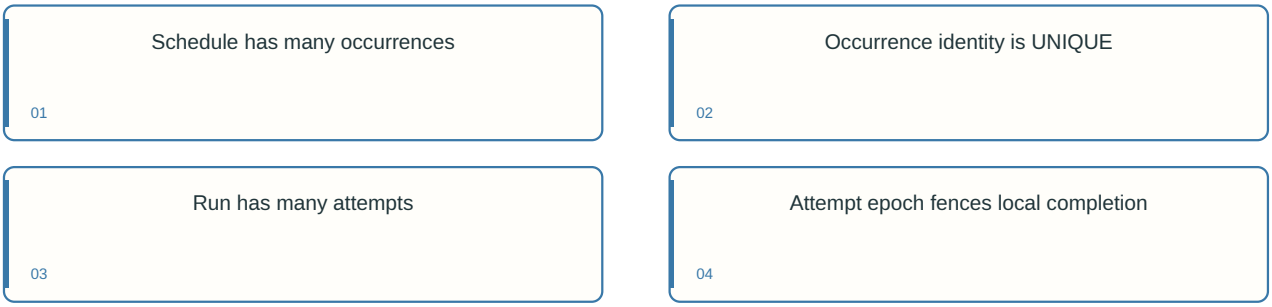


FIG. 22.10.1 **Data ownership and relationships**

Store schedule(id,version,next_due,timing_policy,state), run(schedule_id,occurrence UNIQUE,state,input_version), attempt(run_id,epoch,lease_until,worker,result), and relevant outbox or effect records. Make occurrence identity stable through leadership change. Keep original scheduled time and observed dispatch time distinct. Attempts are not interchangeable with runs, and an old attempt must not overwrite a newer attempt's final state.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.7 · Low-level component organization

ScheduleParser and TimePolicy determine occurrences, DueScanner finds candidates, RunRepository atomically advances schedule and inserts run, Dispatcher routes work, LeaseManager grants attempt epochs, WorkerAdapter executes, and Reconciler detects stuck or uncertain effects. The scheduler owns timing decisions; workers own bounded execution under a claim. Domain effect adapters must state their idempotency or fencing contract.

22.10.8 · High-level system architecture

Use durable schedule storage, coordinated scanners or partition owners, a dispatch log or queue, bounded worker pools, lease/attempt state, and reconciliation. Separate timing control from execution capacity. A control-plane outage can delay new dispatch while running jobs continue; worker loss should leave a recoverable run. Store large payloads as versioned objects rather than duplicating them in every retry message.

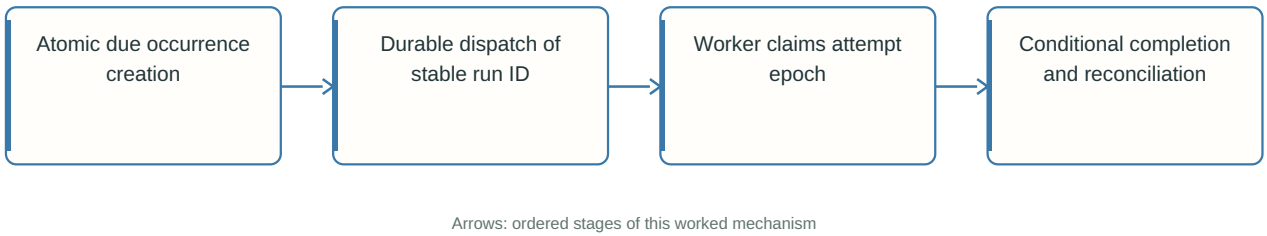


FIG. 22.10.2 **The high-level path and responsibility boundaries**

Use durable schedule storage, coordinated scanners or partition owners, a dispatch log or queue, bounded worker pools, lease/attempt state, and reconciliation. Separate timing control from execution capacity. A control-plane outage can delay new dispatch while running jobs continue; worker loss should leave a recoverable run. Store large payloads as versioned objects rather than duplicating them in every retry message.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.9 · Data and control flows

Scanner locks or conditionally claims a due schedule, creates the unique occurrence and dispatch intent, advances next_due, and commits. Worker claims an attempt epoch, executes the stable run identity, renews its lease if supported, and conditionally records completion. An expired attempt can be replaced, but resource fencing or effect deduplication must prevent harmful stale work. Reconcile runs without terminal progress.

22.10.10 · Capacity estimates

At 27.8 average runs/s and ten seconds mean execution, average active work is about 278. If a worker handles four such concurrent I/O jobs, the arithmetic minimum is about seventy workers before headroom and actual bottlenecks. CPU jobs need measured CPU capacity instead. A top-of-hour burst can exceed that average dramatically, so model due-time distribution and queue delay rather than only total schedules.

22.10.11 · End-to-end latency estimates

Scheduling delay is due-time to dispatch or start, distinct from runtime. An illustrative objective is 95% of eligible runs starting within five seconds under supported load. Scanner interval, ownership, queueing, cold starts, and concurrency limits all consume that budget. A long task can finish minutes later without violating start-delay objective, so completion deadlines need separate policy and evidence.

22.10.12 · Failure scenarios

Crash between run insertion and schedule advancement can duplicate or skip occurrences unless both share an atomic boundary. Lease expiry can overlap old and new execution. Clock shifts, timezone transitions, malformed payloads, and poison jobs need policy. A worker reporting success after its epoch is superseded must be rejected locally, while external effects already accepted before fencing remain a separate reconciliation concern.

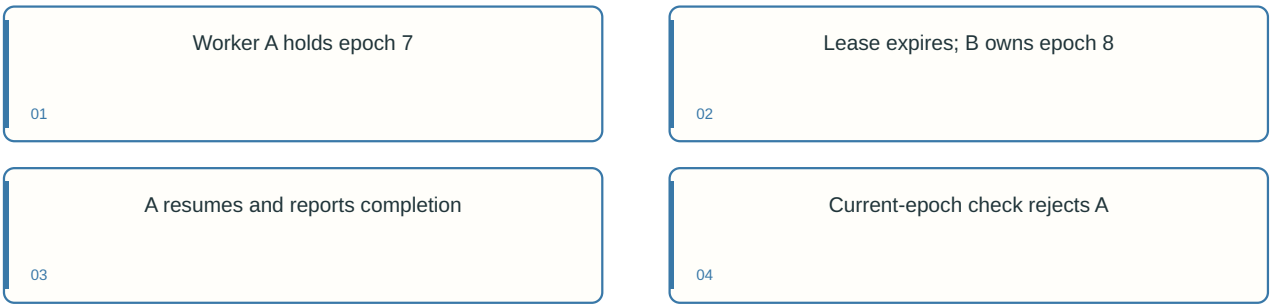


FIG. 22.10.3 **Naive path, failure, improved path, review**

Crash between run insertion and schedule advancement can duplicate or skip occurrences unless both share an atomic boundary. Lease expiry can overlap old and new execution. Clock shifts, timezone transitions, malformed payloads, and poison jobs need policy. A worker reporting success after its epoch is superseded must be rejected locally, while external effects already accepted before fencing remain a separate reconciliation concern.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.13 · Scalability improvements

Partition scans by stable schedule ownership, index due time, batch bounded claims, and scale independent workers according to measured resource use. Avoid one global scan lock for every occurrence if it becomes limiting, but preserve uniqueness and schedule advancement. Isolate long, CPU-heavy, and high-priority workloads. Replay or catch-up after outage needs a separate budget so millions of missed runs do not starve live critical work.

22.10.14 · Consistency requirements

Schedule advancement and occurrence creation need atomicity and uniqueness. Worker delivery can be at least once. Conditional completion under the current epoch prevents stale local results; idempotency at the side-effect boundary or enforced fencing protects business effects. A lease by itself cannot prove exactly-

once execution. Explicitly define misfire and overlap policies instead of letting timing races decide whether a missed occurrence runs.

22.10.15 · Security model

Authorize schedule owners, validate executable types and payload references, isolate worker capabilities, and avoid allowing arbitrary code or unrestricted URLs through a scheduling API. Bind run identity to its tenant and immutable input version. Protect schedule administration and cancellation with audit records. A worker should access only the required resources for that run, not inherit a global administrator credential.

22.10.16 · Deployment approach

Deploy scanners and workers with compatible run schemas and lease semantics. Stop new claims during draining and persist progress before termination. Maintain a tested database recovery procedure and reconcile queues against durable run state. A restored schedule table behind external effects can cause duplicate occurrences unless stable identity and reconciliation are preserved. Test control-plane outage independently from execution-plane failure.

22.10.17 · Observability design

Observe due-to-dispatch and due-to-start delay, oldest eligible run, missed or duplicate occurrence attempts, lease expiry, stale completion rejection, retry counts, execution duration, and per-tenant fairness. Monitor the invariant that one occurrence identity maps to one logical run. Alert on stuck accepted work and capacity saturation. CPU utilization alone cannot reveal that the scanner stopped generating runs.

22.10.18 · Infrastructure cost considerations

Cost includes scan/index work, retained run history, dispatch traffic, worker compute, idle failure capacity, external effects, and operational recovery. At the assumed run rate, raw 2 kB history is 4.8 GB/day before attempts and indexes. Compare scan frequency and batching against delay objectives. A cheap design that silently misses runs is not equivalent to a durable scheduler with measured recovery behavior.

22.10.19 · Alternative architectures and trade-offs

Polling a due index is simple and recoverable; timer wheels or delayed-message mechanisms can improve specific timing loads but need durable reconstruction. Central scanning simplifies ownership at a ceiling; partitioned scanning increases coordination. Long-lived workers reduce startup overhead, while managed

invocation can fit bursty work with different limits. Choose by timing precision, workload cost, and failure evidence rather than the timer data structure alone.

22.10.20 · Architecture evolution from small scale to large scale

Start with atomic occurrence creation and deterministic time-policy tests. Add durable dispatch and bounded workers. Introduce leases, epochs, and effect identity before claiming worker failover. Partition the measured scan or execution bottleneck. Add regional ownership only with safe authority transfer and a defined policy for delayed, duplicated, or missed occurrences during partitions.

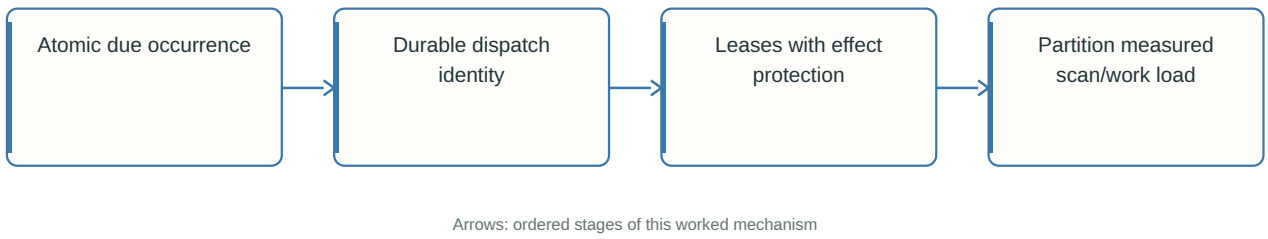


FIG. 22.10.4 **Evolution follows demonstrated requirements**

Start with atomic occurrence creation and deterministic time-policy tests. Add durable dispatch and bounded workers. Introduce leases, epochs, and effect identity before claiming worker failover. Partition the measured scan or execution bottleneck. Add regional ownership only with safe authority transfer and a defined policy for delayed, duplicated, or missed occurrences during partitions.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.21 · Design review identifying weaknesses and improvements

Review the stable run identity, atomic schedule advancement, stale-worker behavior, and recovery from a historical snapshot. Test scanner crash at commit boundaries, worker pause beyond lease, cancellation during execution, timezone transition, and catch-up after a long outage. A scheduler that fires a callback on time once has not demonstrated durable scheduling or duplicate-safe effects.

22.10.22 · A short implementation exercise

Implement a scanner over an injected clock and transactional run table. Run two scanner instances against the same due occurrence and prove uniqueness. Pause a worker after claiming epoch 7, reclaim with epoch

8, then attempt the old completion. Show it is rejected locally and identify what further protection an external side effect would require.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
with transaction():
    schedule = claim_due_schedule(now)
    run = insert_unique_run(schedule.id, schedule.next_due)
    insert_dispatch_outbox(run.id)
    advance_next_due(schedule)
# Worker completion compares the current attempt epoch atomically.
# A stale worker's external effects need resource-side protection.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. Distributed Periodic Scheduling — Google SRE ↗

   Engineering book. Durable scheduling concerns; atlas schedule architecture is an original worked design.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Collaborative Document Editor

Prerequisites: Networking Foundations for System Design, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Create documents, authorize collaborators, apply edits, synchronize connected clients, reconnect after offline work, retain required version history, and support export and deletion. Define the editing model explicitly: plain text, structured rich text, or embedded objects have different conflict semantics. Presence and cursors are secondary ephemeral state, separate from durable document content.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.11.1 · Functional requirements

Create documents, authorize collaborators, apply edits, synchronize connected clients, reconnect after offline work, retain required version history, and support export and deletion. Define the editing model explicitly: plain text, structured rich text, or embedded objects have different conflict semantics. Presence and cursors are secondary ephemeral state, separate from durable document content.

22.11.2 · Nonfunctional requirements

Converge under the chosen editing protocol and recovery assumptions, preserve acknowledged operations, and enforce current access policy. Target responsive local editing with bounded synchronization delay for online clients. Convergence alone does not guarantee desirable intent preservation, Unicode correctness, or arbitrary document invariants. Offline editing also needs a policy for revoked users and conflicting structural changes when they reconnect.

22.11.3 · Explicit assumptions and workload numbers

Assume one hundred thousand active documents, ten thousand simultaneously edited, three active users per edited document, and two operations/s per active user: 60000 operations/s. Assume 200 bytes per retained operation: 12 MB/s and about 1.037 TB/day raw if that rate is sustained. Batching, snapshots, actual activity duration, metadata, and retention change the result.

22.11.4 · Naive initial architecture

Begin with optimistic whole-document replacement using an expected version. Conflicting writers are rejected and asked to reload, which is correct for a simple exclusive-editing contract but poor for real-time collaboration. Moving to operation-based synchronization addresses concurrent edits. Choose and implement a reviewed OT or CRDT model rather than accepting arbitrary text patches in arrival order and calling that collaborative convergence.

22.11.5 · Core API contracts

POST /documents creates a resource; connect or sync exchanges document identity, client replica identity, operation IDs, and known progress; GET /documents/{id}/snapshot returns a versioned base; operations and acknowledgments use documented protocol versions. Membership changes and exports are authorized separately. A client cannot select another user's replica identity to overwrite their operation history.

22.11.6 · Data model and schema

Store document(id,owner,protocol_version,state), membership, stable operation identity with causal or sequence metadata, snapshots with covered positions, and compaction metadata. For a sequence CRDT, character or element identifiers and ordering rules must be stable across replicas. For OT, server history and transform context need retention compatible with supported offline duration. A snapshot is not valid unless clients know which operations it includes.

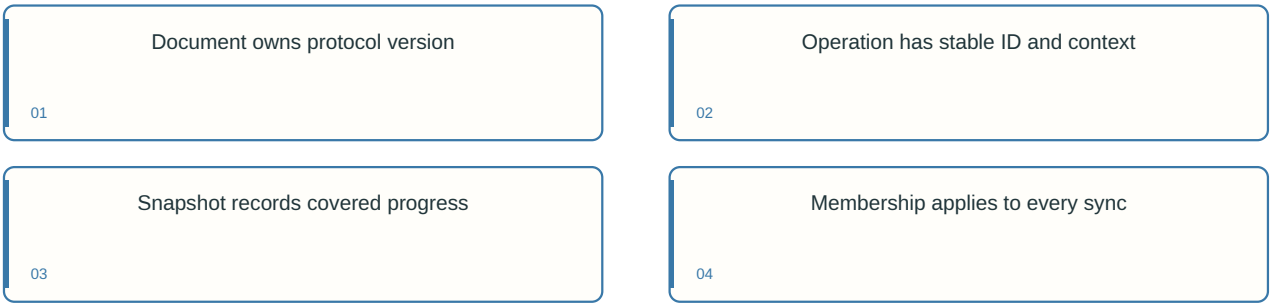


FIG. 22.11.1 **Data ownership and relationships**

Store document(id,owner,protocol_version,state), membership, stable operation identity with causal or sequence metadata, snapshots with covered positions, and compaction metadata. For a sequence CRDT, character or element identifiers and ordering rules must be stable across replicas. For OT, server history and transform context need retention compatible with supported offline duration. A snapshot is not valid unless clients know which operations it includes.

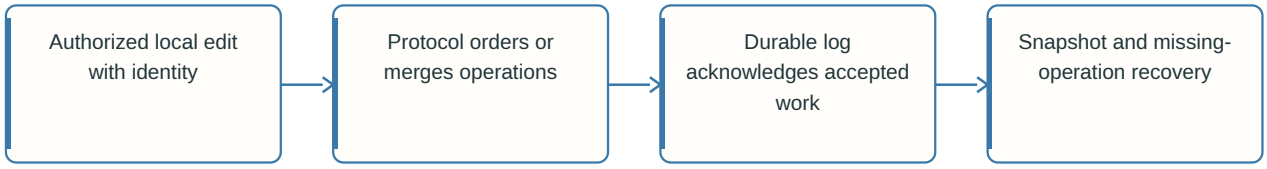
Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.7 · Low-level component organization

DocumentService owns identity and membership, SyncCoordinator interprets protocol messages, EditModel applies OT/CRDT semantics, OperationLog persists accepted edits, SnapshotBuilder creates recoverable bases, and Compactor retires history only under protocol assumptions. CursorPresence uses a separate ephemeral channel. Keep rendering and transport separate from the deterministic editing core so protocol behavior can be tested with explicit interleavings.

22.11.8 · High-level system architecture

Use authenticated collaboration gateways, an appropriate document coordination or merge owner, durable operation storage, snapshot artifacts, and client replicas. OT commonly uses server ordering with transforms; CRDT designs can merge operations under defined identifiers and causal rules. These are alternative families, not interchangeable labels on one vague diagram. Choose one protocol per document version and design migration deliberately.



Arrows: ordered stages of this worked mechanism

FIG. 22.11.2 **The high-level path and responsibility boundaries**

Use authenticated collaboration gateways, an appropriate document coordination or merge owner, durable operation storage, snapshot artifacts, and client replicas. OT commonly uses server ordering with transforms; CRDT designs can merge operations under defined identifiers and causal rules. These are alternative families, not interchangeable labels on one vague diagram. Choose one protocol per document version and design migration deliberately.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.9 · Data and control flows

A client applies a permitted local edit optimistically, assigns stable operation identity, and submits with context. The server checks authorization and protocol, persists accepted work under its contract, and distributes or acknowledges it. Reconnect exchanges missing operations or a compatible snapshot plus continuation. Duplicate operations are ignored by identity. Rejected offline changes must be surfaced without pretending they were durably accepted.

22.11.10 · Capacity estimates

The assumed 60000 operations/s at 200 bytes yields 12 MB/s raw, excluding fan-out and replication. Three participants per active document create additional distribution work. A hot large document can dominate operation history and transform cost. Snapshot cadence trades replay latency against storage and background compute. Measure actual edit batching, offline horizon, tombstone growth, and document-size distribution before sizing servers.

22.11.11 · End-to-end latency estimates

Local optimistic editing can respond within the client frame budget even before remote acknowledgment, but the UI must distinguish pending from durable sync. An illustrative network-sync budget is 30 ms

ingress, 20 ms durable acceptance, and 50 ms peer delivery: 100 ms before queueing. This is not proof of p99. Offline and geographically distant users require different expectations and conflict/recovery behavior.

22.11.12 · Failure scenarios

Concurrent insertion at the same position needs deterministic protocol semantics. A duplicate edit must not duplicate a character. Missing transform history, premature tombstone removal, stale membership, and incompatible snapshots can break recovery even if a short online demo converges. Network partition requires the selected offline policy. Protect against a client submitting malformed dependency graphs or extremely expensive edits that exhaust server resources.



FIG. 22.11.3 **Naive path, failure, improved path, review**

Concurrent insertion at the same position needs deterministic protocol semantics. A duplicate edit must not duplicate a character. Missing transform history, premature tombstone removal, stale membership, and incompatible snapshots can break recovery even if a short online demo converges. Network partition requires the selected offline policy. Protect against a client submitting malformed dependency graphs or extremely expensive edits that exhaust server resources.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.13 · Scalability improvements

Partition independent documents, batch operations, bound gateway buffers, and create snapshots for recovery. A single hot document may need specialized coordination or distribution rather than ordinary hash sharding. Compaction must preserve the supported offline and merge contract; deleting metadata because it is old can make a late operation uninterpretably refer to removed structure. Scale cursor traffic separately from durable edit traffic.

22.11.14 · Consistency requirements

OT transforms concurrent operations against the relevant ordered context; an insertion can shift another insertion's position. CRDTs choose merge semantics that converge under their algorithm's assumptions, often using stable element identifiers and ordering. For two concurrent inserts X and Y after the same element, replicas need the same tie-break rule. Neither acronym supplies all intent or rich-text semantics. Authorization and durable acknowledgment remain separate from convergence.

22.11.15 · Security model

Authorize every sync and operation, not only opening the document page. Recheck membership according to a defined revocation policy for long-lived sessions and offline reconnect. Restrict exports and snapshots, avoid leaking content through presence or logs, and validate protocol inputs. If end-to-end encryption is required, explain how server-side search, validation, history, and access revocation change; transport TLS alone is not that guarantee.

22.11.16 · Deployment approach

Roll out protocol changes with version negotiation and compatible history. Never reinterpret an existing operation identifier under a new algorithm silently. Deploy gateways with reconnect and durable recovery, and validate snapshots before retiring history. Recovery drills must start from persisted snapshots and logs, not a live client's convenient in-memory copy. Plan migration and rollback for stored documents as well as server code.

22.11.17 · Observability design

Measure accepted and pending operations, sync delay, duplicate suppression, reconnect gaps, snapshot age, replay time, compaction size, protocol rejects, and membership denials. Use deterministic multi-client convergence tests over generated interleavings and retain failing histories. Client-visible dirty or unsynced duration is a key indicator beyond gateway latency. Do not log raw confidential document content to diagnose every edit.

22.11.18 · Infrastructure cost considerations

Cost includes operation retention, snapshot storage, fan-out, client connection serving, protocol CPU, compaction, and privacy-aware telemetry. Sustained peak arithmetic is intentionally conservative and must be replaced with measured activity distributions. Longer offline support can require more history or

metadata. Compare costs at equal supported recovery horizons; deleting necessary history makes a cheaper system with a weaker contract.

22.11.19 · Alternative architectures and trade-offs

Whole-document optimistic versioning is simple and fits low-collaboration use. OT can use centralized ordering with transform complexity; CRDTs can support decentralized merging with identifier, metadata, and compaction costs. Neither is universally superior. A structured editor may choose a well-tested library rather than a custom algorithm. Evaluate Unicode grapheme handling, selection behavior, undo, and schema constraints alongside convergence.

22.11.20 · Architecture evolution from small scale to large scale

Begin with versioned documents and explicit conflict errors. Add a tested operation model, durable log, and reconnect. Introduce snapshots and proven compaction rules. Partition independent documents and isolate hot collaboration workloads. Add regional or offline merging only within the selected protocol's actual assumptions, preserving stable IDs and access policy. Preserve deterministic regression histories through every release.

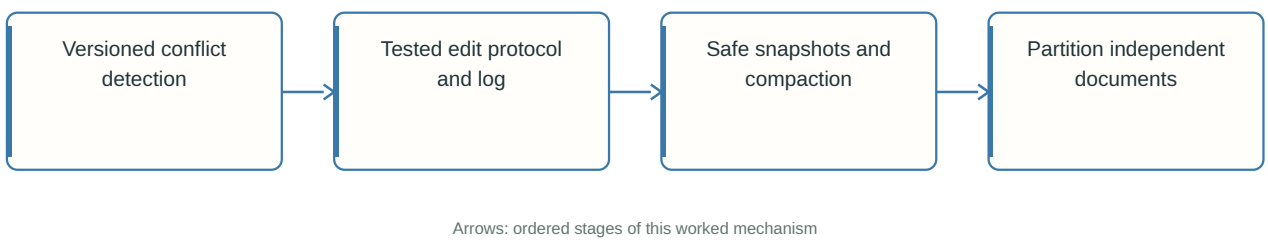


FIG. 22.11.4 **Evolution follows demonstrated requirements**

Begin with versioned documents and explicit conflict errors. Add a tested operation model, durable log, and reconnect. Introduce snapshots and proven compaction rules. Partition independent documents and isolate hot collaboration workloads. Add regional or offline merging only within the selected protocol's actual assumptions, preserving stable IDs and access policy. Preserve deterministic regression histories through every release.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.21 · Design review identifying weaknesses and improvements

Review convergence, supported offline duration, compaction safety, and revocation during a long-lived connection. Test concurrent inserts at one position, duplicate delete, out-of-order delivery, lost acknowledgment, old snapshot plus new operations, and revoked-user reconnect. A test that only compares final strings can miss unauthorized edits or invalid structural states, so verify those properties separately.

22.11.22 · A short implementation exercise

Implement a small deterministic merge of two inserts with stable IDs and an explicit tie-break rule, then deliver them in both orders and repeat each once. Show the same final sequence and no duplicate effect. Document that this miniature is not a complete rich-text CRDT. Extend tests to snapshot context and a revoked client before considering production use.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def receive(operation, document):
    with transaction():
        current = lock_document(document.id)
        require_authorized(operation.actor, current)
        require_protocol_version(operation, current)
        if log.contains(operation.id): return prior_ack(operation.id)
        next_state = edit_model.apply(current.state, operation)
        append_unique_operation(operation)
        commit_state_and_progress(current, next_state)
    return durable_ack(operation.id)
# The edit model must implement a specified, tested algorithm.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

2. RFC 6455: The WebSocket Protocol, 2011 ↗

   Original standard. Bidirectional transport framing; not durable chat delivery.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

Large-Scale Document Processing Platform

Prerequisites: File Systems, Search Infrastructure, and Data Pipelines, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Accept files, validate and classify them, extract structured fields, retain provenance, route uncertain results to human review, publish authorized searchable output, and expose processing state. Support versioned reprocessing, corrections, retention, and deletion. A successful upload means bytes were accepted under a defined contract; it does not mean extraction or review has finished.

Original teaching design. All workload, latency, capacity, and price figures are illustrative assumptions, not company architecture claims or measured benchmarks.

22.12.1 · Functional requirements

Accept files, validate and classify them, extract structured fields, retain provenance, route uncertain results to human review, publish authorized searchable output, and expose processing state. Support versioned reprocessing, corrections, retention, and deletion. A successful upload means bytes were accepted under a defined contract; it does not mean extraction or review has finished.

22.12.2 · Nonfunctional requirements

Preserve accepted inputs and workflow progress under the selected fault model, make results traceable to exact source and model versions, enforce tenant isolation, and bound processing age for supported workloads. Never silently turn uncertain extraction into verified truth. Human review decisions are durable

domain events with actor and rationale. Search and downstream publication have distinct freshness and access obligations.

22.12.3 · Explicit assumptions and workload numbers

Assume one million files/day, two MB average input: two TB/day raw. Average arrival is about 11.6 files/s, with an assumed tenfold peak around 116/s. If extraction uses an illustrative five CPU-seconds per file, daily work is five million CPU-seconds, or about 1389 CPU-hours. This excludes I/O, retries, classification, human wait, and model-specific effects.

22.12.4 · Naive initial architecture

Start with one upload API saving a file and running extraction synchronously. It can demonstrate the model but times out on large files, loses progress after crashes, and makes review awkward. Separate direct resumable ingestion from a durable workflow. Keep immutable source versions and stage-specific outputs so retries and corrections do not overwrite the evidence needed to explain previous results.

22.12.5 · Core API contracts

POST /documents creates an upload intent; complete validates stored bytes; GET /documents/{id}/status exposes stages; GET /documents/{id}/fields returns versioned results and provenance; POST /reviews/{id}/decisions records authorized review; POST /documents/{id}/reprocess selects a supported pipeline version; DELETE starts tracked deletion. Export and search operate only on permitted published output.

22.12.6 · Data model and schema

Store document(tenant,id,source_version,digest,state), upload session, workflow(run,input_version,pipeline_version), stage_attempt(epoch,state), artifact(kind,version,object_ref), field(value,confidence,source_region,extractor_version), review(decision,actor,version), publication generation, and deletion progress. Keys bind tenant and version. Retain minimal auditable provenance without assuming every raw intermediate should live forever.

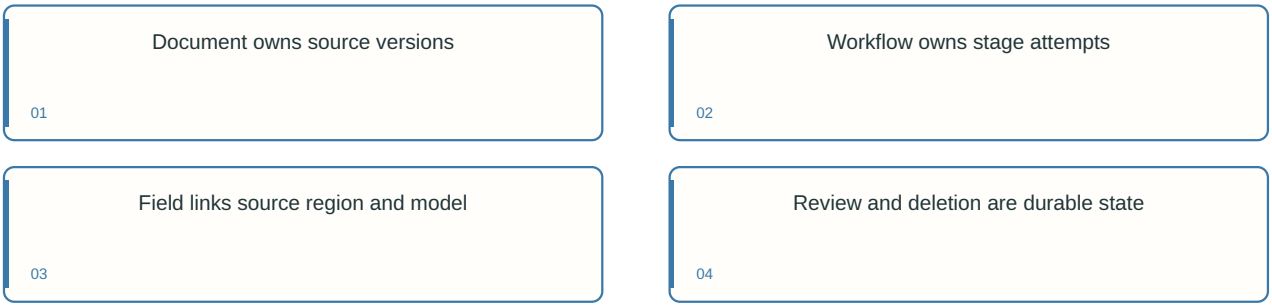


FIG. 22.12.1 **Data ownership and relationships**

Store document(tenant,id,source_version,digest,state), upload session, workflow(run,input_version,pipeline_version), stage_attempt(epoch,state), artifact(kind,version,object_ref), field(value,confidence,source_region,extractor_version), review(decision,actor,version), publication generation, and deletion progress. Keys bind tenant and version. Retain minimal auditable provenance without assuming every raw intermediate should live forever.

Why this view: Identify identities, authoritative relationships, derived records, and the invariant that must remain true. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.7 · Low-level component organization

UploadService and ContentValidator own ingestion, WorkflowCoordinator owns durable stages, Classifier and Extractor are versioned adapters, ProvenanceRecorder links source regions, ReviewPolicy decides whether automation is sufficient, ReviewService owns human decisions, Publisher builds authorized projections, and Reconciler repairs stuck or orphaned work. Workers get narrow capabilities and cannot change tenant identity from an untrusted message.

22.12.8 · High-level system architecture

Use object storage for immutable bytes and artifacts, relational workflow metadata, a durable task/log layer, isolated bounded CPU or model workers, human-review UI/API, search projection, and observability. The workflow is the control path; bulk bytes flow through scoped object access. Separate review waiting from worker occupancy so a document pending for hours does not hold a compute slot or database transaction.

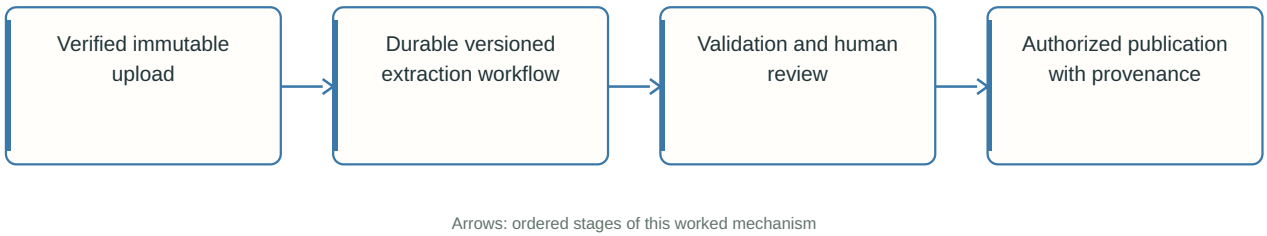


FIG. 22.12.2 **The high-level path and responsibility boundaries**

Use object storage for immutable bytes and artifacts, relational workflow metadata, a durable task/log layer, isolated bounded CPU or model workers, human-review UI/API, search projection, and observability. The workflow is the control path; bulk bytes flow through scoped object access. Separate review waiting from worker occupancy so a document pending for hours does not hold a compute slot or database transaction.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.9 · Data and control flows

Verify upload and source digest, create a stable workflow, classify, extract, validate schema and semantic checks, record evidence, and either publish or request review. Human decisions create a new validated result version and publication intent. Each stage conditionally commits under its current attempt epoch. Reprocessing creates a new run over an identified source version rather than mutating old provenance. Deletion cancels or blocks inappropriate later publication.

22.12.10 · Capacity estimates

Two TB/day raw implies sixty TB for thirty-day retention before copies. At five CPU-seconds/file, average compute demand is about 57.9 continuously busy cores; operating at an assumed 60% target utilization gives about 96.5 cores before other work and failure headroom. Tenfold arrival peaks cannot be handled indefinitely by that average capacity; queue age, scaling delay, and admission determine whether the processing objective holds.

22.12.11 · End-to-end latency estimates

Upload acknowledgment has a separate byte-transfer and metadata budget. Automated processing age includes queue, classification, extraction, validation, and publication; human review is a distinct potentially long state. An illustrative automated path of two seconds queue, one second classification, five seconds

extraction, and one second validation/publication totals nine seconds. These are assumptions to test, not measured model latency or accuracy.

22.12.12 · Failure scenarios

Failures include incomplete uploads, corrupt files, malicious content, worker crash after output write, duplicate stage delivery, incompatible model output, stale completion, review races, missing lineage, and deletion during processing. Stable artifacts and epoch-checked transitions prevent stale overwrite. Quarantine invalid outputs with reasons. A document marked complete without usable authorized output or source evidence is a failed workflow even if no worker raised an exception.

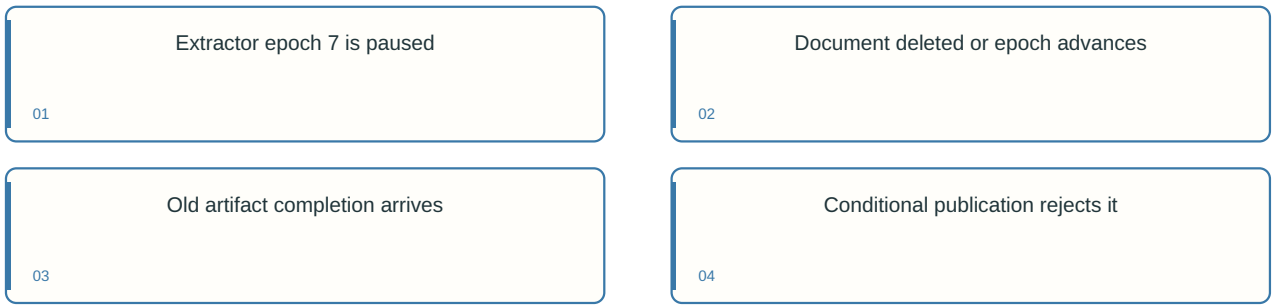


FIG. 22.12.3 **Naive path, failure, improved path, review**

Failures include incomplete uploads, corrupt files, malicious content, worker crash after output write, duplicate stage delivery, incompatible model output, stale completion, review races, missing lineage, and deletion during processing. Stable artifacts and epoch-checked transitions prevent stale overwrite. Quarantine invalid outputs with reasons. A document marked complete without usable authorized output or source evidence is a failed workflow even if no worker raised an exception.

Why this view: Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.13 · Scalability improvements

Scale bytes through direct storage, independent extraction through bounded workers, and search through a derived index. Partition by tenant or workflow while controlling hot tenants and large files. Keep model batching and memory limits explicit; more workers can overload storage or a model endpoint. Assign separate capacity to replay and backfill so a pipeline upgrade does not starve new documents or required deletion work.

22.12.14 · Consistency requirements

Source versions are immutable, workflow transitions and review decisions commit atomically where required, and derived artifacts identify their exact inputs. Stage delivery can repeat, but output identity and conditional commit make local progress safe. Search may lag publication while enforcing access. External model calls can be uncertain or nondeterministic; retain relevant version and request identity, validate output, and avoid claiming reproducibility beyond the actual model contract.

22.12.15 · Security model

Enforce tenant scope on object capabilities, jobs, review, search, exports, and audit. Isolate parsers and model workers, restrict egress and resource usage, scan or validate supported content according to policy, and minimize secrets and personal data in telemetry. Treat extracted text as untrusted data, not instructions to infrastructure. Reviewers need scoped access and an auditable decision trail. Deletion must cover raw and derived sensitive content.

22.12.16 · Deployment approach

Deploy pipeline and schema versions compatibly; canary new models on authorized samples or controlled cohorts with quality review. Preserve old output until replacement validation and publication succeed. Drain workers using durable attempts and tested recovery. Restore drills must recover object references, workflow state, review decisions, and deletion history together. Do not assume restoring the metadata database alone recreates a usable processing platform.

22.12.17 · Observability design

Measure accepted bytes, stage queue age, compute time, retries, stale completion rejects, validation failures, review backlog, provenance completeness, publication lag, tenant denials, and deletion progress. Track automated quality with an appropriate labeled evaluation rather than presenting confidence as accuracy. Link one document's source, workflow, attempt, model, output, and reviewer through stable identities and privacy-safe telemetry.

22.12.18 · Infrastructure cost considerations

Cost includes two TB/day source storage, retained artifacts, model or CPU usage, retries, human review, search, transfer, telemetry, and recovery. At the illustrative extraction rate, five million CPU-seconds/day is the main compute input, but review can dominate real operating expense. Use explicit hypothetical unit

rates and review fraction to compare architectures. Do not invent extraction accuracy or model throughput benchmarks.

22.12.19 · Alternative architectures and trade-offs

Synchronous processing simplifies a tiny prototype but couples upload and extraction failure. Durable staged orchestration improves recovery and review at metadata and operating cost. Batch inference can improve efficiency while adding waiting and memory constraints. A managed workflow or model service transfers selected tasks but does not supply provenance, validation, privacy, or business review policy automatically. Compare equal output and recovery contracts.

22.12.20 · Architecture evolution from small scale to large scale

Begin with immutable upload and explicit processing status. Add durable stages and stable artifacts before scaling. Introduce human review and provenance as first-class output behavior. Scale workers and storage independently, then add authorized search and versioned reprocessing. Introduce regional placement only when data-residency, latency, or disaster requirements justify its state and lifecycle complexity.

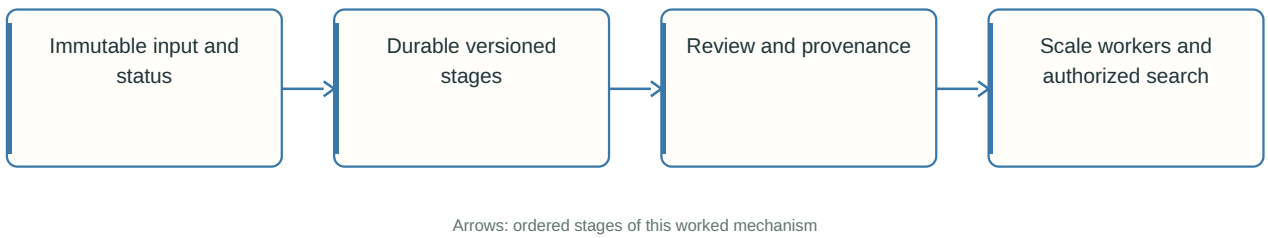


FIG. 22.12.4 **Evolution follows demonstrated requirements**

Begin with immutable upload and explicit processing status. Add durable stages and stable artifacts before scaling. Introduce human review and provenance as first-class output behavior. Scale workers and storage independently, then add authorized search and versioned reprocessing. Introduce regional placement only when data-residency, latency, or disaster requirements justify its state and lifecycle complexity.

Why this view: Follow the stated ordering and identify which step owns the relevant outcome. Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.21 · Design review identifying weaknesses and improvements

Review whether any stage silently changes source versions, whether an old attempt can publish after deletion, and whether human decisions are preserved during reprocessing. Test duplicate tasks, interrupted

extraction, invalid output, reviewer concurrency, stale completion, missing object, index lag, and full restore. A pipeline that extracts a few fields once has not demonstrated the required recoverable, explainable platform.

22.12.22 · A short implementation exercise

Implement a three-stage workflow with immutable input ID, stage result version, and attempt epoch. Fail after writing an artifact but before recording completion; replay safely. Add a human-review state that releases compute capacity. Then delete the document while an old worker is paused and prove its later completion cannot publish the deleted result.

Miniature implementation of the core boundary

LANGUAGE-NEUTRAL PSEUDOCODE · NOT A COMPLETE PRODUCTION SERVICE

```
def complete_stage(run, epoch, artifact):
    require(artifact.input_version == run.input_version)
    validate_output_schema_and_provenance(artifact)
    with transaction():
        current = lock_workflow(run.id)
        require(current.epoch == epoch and not current.deleted)
        insert_artifact_once(run.id, artifact.version, artifact)
        advance_or_request_review(current, artifact)
        insert_next_stage_outbox_if_needed(current)
# Retry reuses identities; old attempts cannot publish new state.
```

Use the final challenge to exercise the invariant and failure window. Runnable local models for conditional claims, deduplication, ledger balance, and workflow fencing are in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

Start with the introductory guidance, then inspect original specifications, research, or implementation details. Access and stated use checked 21 October 2026. Author-created examples and assumed numbers are separate from source guarantees.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

4. Transactional Outbox — Chris Richardson ↗

   Pattern author’s explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

THE COMPLETE EDITION

Contents, from objects to systems

All 22 units and 99 chapters are available. Follow prerequisites or use a focused reading path.

Part 1 · System design foundations

01 · Introduction to System Design — READ NOW

- 1.1 · From a program to a system
- 1.2 · Organising code into modules
- 1.3 · From requirements to design choices
- 1.4 · Drawing the system at different levels

02 · Requirements, Estimation, and Engineering Trade-offs — READ NOW

- 2.1 · Turn user journeys into workloads
- 2.2 · Measure the experience
- 2.3 · Estimate the resources
- 2.4 · Queues, bottlenecks & defensible decisions

03 · Networking Foundations for System Design — READ NOW

- 3.1 · Addresses, sockets & transports
- 3.2 · A secure request, one round trip at a time
- 3.3 · Intermediaries & persistent connections
- 3.4 · Discover, route & survive the network

04 · Runtime, Processes, Threads, and Concurrency READ NOW

4.1 · The runtime owns your resources

4.2 · Tasks, threads & event loops

4.3 · Shared memory & coordination

4.4 · Visibility, cancellation & overload

Part 2 · Low-level design & software architecture

05 · Object-Oriented and Modular Software Design READ NOW

5.1 · Model behavior with objects and values

5.2 · Principles under pressure

5.3 · Side effects, domain models & collaboration

5.4 · Design for change, testing & functions

06 · Design Patterns, Dependency Injection, and Object Lifecycles READ NOW

6.1 · Dependencies, inversion & composition

6.2 · Lifetimes, ownership & cleanup

6.3 · Injection, locators & factories

6.4 · Creational patterns

6.5 · Structural patterns

6.6 · Behavioral patterns

6.7 · Enterprise patterns & boundaries

07 · Application Architecture and Domain-Driven Design READ NOW

7.1 · Layers, monoliths & ports

7.2 · Model a bounded context

7.3 · Keep business policy inside

7.4 · Evolution without a distributed monolith

08 · API Design and Communication Contracts

READ NOW

- 8.1 · Model resources and messages
- 8.2 · Compatibility & idempotency
- 8.3 · Control access, load & long operations
- 8.4 · Evolve a public contract

Part 3 · Data, storage & scalability

09 · Data Models and Storage Technologies

READ NOW

- 9.1 · Storage promises & database families
- 9.2 · Files, objects & analytical layouts
- 9.3 · Model relationships and ownership
- 9.4 · Access patterns, retention & operational cost

10 · Database Internals, Transactions, and Concurrency Control

READ NOW

- 10.1 · Pages, indexes & query plans
- 10.2 · Commit, logging & versions
- 10.3 · Races, anomalies & isolation
- 10.4 · Pools, recovery & ticket correctness

11 · Caching and Content Delivery

READ NOW

- 11.1 · Locality and cache placement
- 11.2 · Copies, freshness & eviction
- 11.3 · Hot keys, stampedes & negative results
- 11.4 · Edges, partitions & the price of a cache

12 · Scalability, Load Balancing, and Data Distribution READ NOW

- 12.1 · Scale compute & route requests
- 12.2 · Replicate and partition data
- 12.3 · Lag, proxies & saturated pools
- 12.4 · Regions, write paths & scale economics

13 · File Systems, Search Infrastructure, and Data Pipelines READ NOW

- 13.1 · Upload bytes without losing identity
- 13.2 · Build and rebuild a search index
- 13.3 · Move data through batch and streams
- 13.4 · Quality, provenance & replay

Part 4 · Distributed systems & reliability

14 · Distributed Systems Fundamentals READ NOW

- 14.1 · Partial failure & the meaning of time
- 14.2 · Consistency as a reader-visible contract
- 14.3 · Partitions, CAP, PACELC & quorums
- 14.4 · Coordination, safety & liveness

15 · Replication, Leader Election, and Consensus READ NOW

- 15.1 · Choose a replication model
- 15.2 · Raft and replicated state machines
- 15.3 · Fences, leases & failure detection
- 15.4 · Conflicts, CRDTs & regional trade-offs

16 · Message Queues, Events, and Distributed Workflows

READ NOW

- 16.1 · Queues, logs & delivery boundaries
- 16.2 · Retries, poison messages & idempotence
- 16.3 · Event sourcing, CQRS & the outbox
- 16.4 · Sagas, durable work & reconciliation

17 · Reliability, Fault Tolerance, and Disaster Recovery

READ NOW

- 17.1 · Failure domains & redundancy
- 17.2 · Retries, circuit breakers & isolation
- 17.3 · Shed load and degrade deliberately
- 17.4 · Recover, test & learn from failure

Part 5 · Security & production engineering

18 · Security Architecture and Privacy

READ NOW

- 18.1 · Threats, identity & authorization
- 18.2 · Secrets, encryption & trust boundaries
- 18.3 · Tenant isolation, audit & deletion
- 18.4 · Supply chains and security reviews

19 · Observability, Monitoring, and Incident Engineering

READ NOW

- 19.1 · Observe behavior with three signals
- 19.2 · Choose indicators, budgets & alerts
- 19.3 · Profile and investigate production
- 19.4 · Learn from incidents and trace async work

20 · Deployment, Cloud Infrastructure, and Platform Architecture READ NOW

20.1 · Environments, VMs & containers

20.2 · Control planes, scheduling & configuration

20.3 · Releases, flags & schema migrations

20.4 · Platforms, serverless & ownership

21 · Performance, Cost Optimization, and System Validation READ NOW

21.1 · Budgets, bottlenecks & honest benchmarks

21.2 · Capacity, architecture & cost per outcome

21.3 · Efficiency and a useful test portfolio

21.4 · Fitness functions and design validation

Part 6 · Complete system design case studies

22 · End-to-End Design, Architecture Reviews, and Evolution READ NOW

22.1 · URL Shortener

22.2 · Rate Limiter

22.3 · Chat Application

22.4 · Notification Platform

22.5 · Social Media Feed

22.6 · Video Streaming Platform

22.7 · Ride-Hailing System

22.8 · Online Ticket Booking

22.9 · Payment System

22.10 · Distributed Job Scheduler

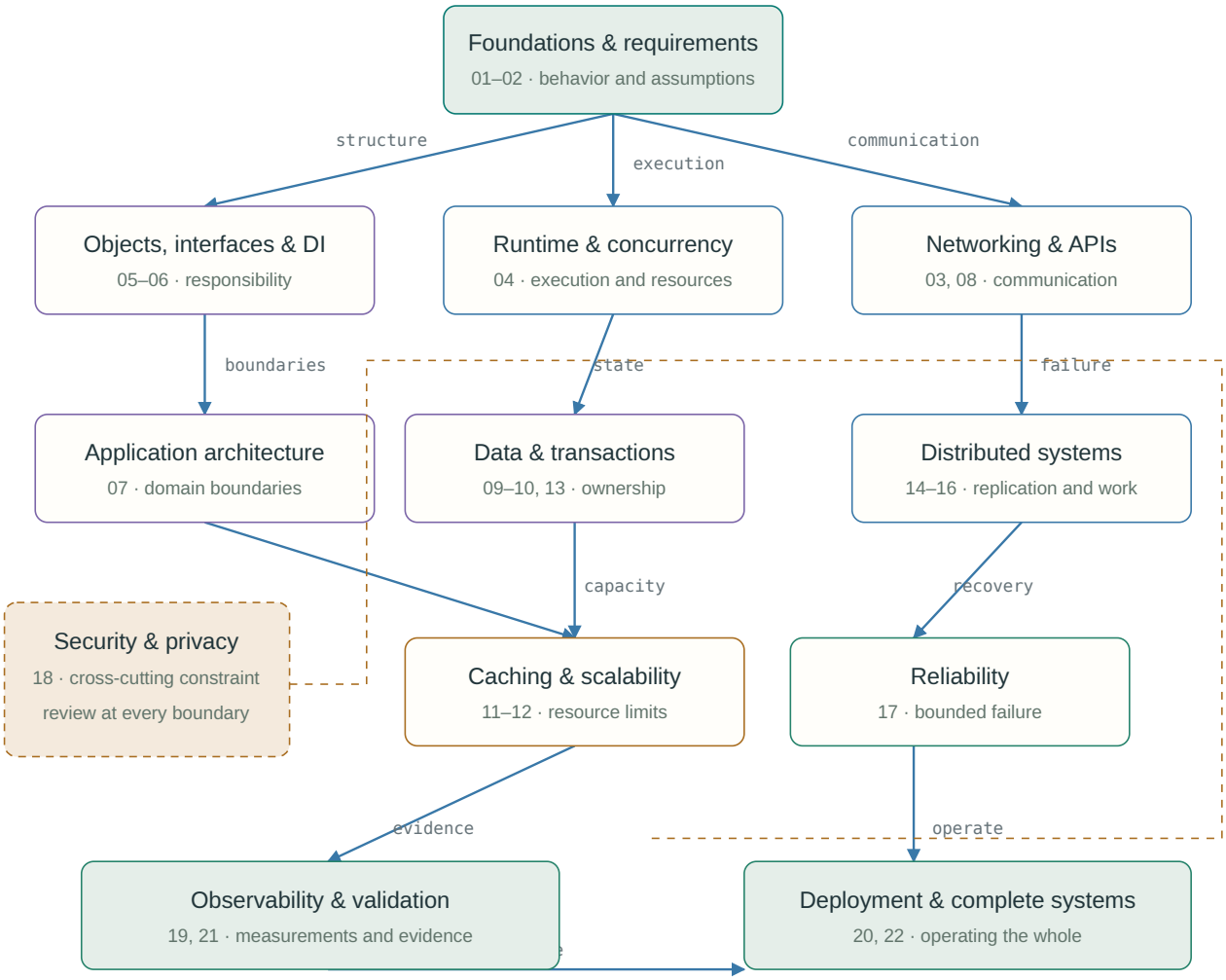
22.11 · Collaborative Document Editor

22.12 · Large-Scale Document Processing Platform

HOW THE IDEAS CONNECT

A map of the territory

The simplified concept spine connects modules, networks, data, distribution, and production engineering.



Simplified concept spine; exact unit and chapter prerequisites appear in Reading paths.

FIG. M.1 **The ideas form a dependency network.**

A simplified prerequisite spine connects low-level structure to storage, networking, distributed behavior, and production engineering. Security constrains the entire design.

Why this view: it shows why a replication or scaling mechanism needs earlier reasoning about state and execution. Author-created illustration.

Solid arrow = conceptual prerequisite · dashed line = cross-cutting constraint. Exact prerequisites are in Reading paths. Author-created illustration.

DEPENDENCY-AWARE ROUTES

Reading paths

Every route below includes the prerequisite closure of its selected units; open any unit for its direct chapter dependencies.

First complete pass

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10

Data and scale

01 → 02 → 03 → 04 → 08 → 09 → 10 → 11 → 12 → 13

Distributed correctness

01 → 02 → 03 → 04 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 17

Production engineering

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 17 → 18 → 19 → 20 → 21

Direct prerequisites

Unit	Before this unit
1 · Introduction to System Design	Basic programming
2 · Requirements, Estimation, and Engineering Trade-offs	1
3 · Networking Foundations for System Design	1
4 · Runtime, Processes, Threads, and Concurrency	1
5 · Object-Oriented and Modular Software Design	1
6 · Design Patterns, Dependency Injection, and Object Lifecycles	5
7 · Application Architecture and Domain-Driven Design	5, 6
8 · API Design and Communication Contracts	1, 3
9 · Data Models and Storage Technologies	1
10 · Database Internals, Transactions, and Concurrency Control	4, 9
11 · Caching and Content Delivery	3, 9
12 · Scalability, Load Balancing, and Data Distribution	2, 3, 9, 10, 11
13 · File Systems, Search Infrastructure, and Data Pipelines	8, 9, 10
14 · Distributed Systems Fundamentals	3, 4, 10
15 · Replication, Leader Election, and Consensus	14
16 · Message Queues, Events, and Distributed Workflows	8, 10, 14, 15
17 · Reliability, Fault Tolerance, and Disaster Recovery	2, 12, 15, 16
18 · Security Architecture and Privacy	3, 7, 8, 9
19 · Observability, Monitoring, and Incident Engineering	2, 3, 4, 10

Unit	Before this unit
20 · Deployment, Cloud Infrastructure, and Platform Architecture	3, 7, 17, 18, 19
21 · Performance, Cost Optimization, and System Validation	2, 4, 10, 12, 17, 19, 20
22 · End-to-End Design, Architecture Reviews, and Evolution	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21

The case-study progression

The URL shortener connects estimation, cache freshness, and scaling. Payments connect object boundaries, API identity, transactions, outbox, provider failures, and ledger reconciliation. Tickets connect local concurrency to expiry and external payment. Documents connect immutable input, lineage, durable stages, human review, and publication.

22.1 · URL Shortener — API Design and Communication Contracts, Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution

22.2 · Rate Limiter — Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery

22.3 · Chat Application — Networking Foundations for System Design, Data Models and Storage Technologies, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

22.4 · Notification Platform — Design Patterns, Dependency Injection, and Object Lifecycles, API Design and Communication Contracts, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

22.5 · Social Media Feed — Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution, Message Queues, Events, and Distributed Workflows

22.6 · Video Streaming Platform — Networking Foundations for System Design, Caching and Content Delivery, File Systems, Search Infrastructure, and Data Pipelines, Deployment, Cloud Infrastructure, and Platform Architecture

22.7 · Ride-Hailing System — Data Models and Storage Technologies, Scalability, Load Balancing, and Data Distribution, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

22.8 · Online Ticket Booking — API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Security Architecture and Privacy

22.9 · Payment System — API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy

22.10 · Distributed Job Scheduler — Runtime, Processes, Threads, and Concurrency, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

22.11 · Collaborative Document Editor — Networking Foundations for System Design, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

22.12 · Large-Scale Document Processing Platform — File Systems, Search Infrastructure, and Data Pipelines, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

ALL 22 UNITS / 242 QUESTIONS

Exercises

Five conceptual questions, three scenarios, one design challenge, one implementation exercise, and one advanced thought experiment per unit.

Unit 1 · Introduction to System Design

CONCEPTUAL / C1

A program adds a task to a dictionary and displays “Saved.” What can it do while running, and what happens when it starts with a new dictionary?

Read the explanation →

CONCEPTUAL / C2

What does the signature create(title) tell a caller? What must the contract explain?

Read the explanation →

CONCEPTUAL / C3

The task service depends on a repository. Does this dependency necessarily make the design difficult to change?

Read the explanation →

CONCEPTUAL / C4

How do a high-level view, a low-level view, and a deployment view help explain the task application?

Read the explanation →

CONCEPTUAL / C5

A service stops accepting requests during recovery but keeps every committed task. Which observation concerns availability, and which concerns durability?

Read the explanation →

SCENARIO / S1

Task 7 is committed, but its success reply is lost. The client waits, then retries. Can it conclude that the first attempt failed?

Read the explanation →

SCENARIO / S2

The web adapter and command-line adapter each enforce an 80-character title limit. Only the web adapter is changed to allow 200 characters. How would you prevent this disagreement?

Read the explanation →

SCENARIO / S3

The team runs two application machines, each with its own unrelated local task database. A user sees a task on one request and cannot find it on the next. What could explain this?

Read the explanation →

DESIGN / D1

A handler adds a task to memory and replies “Saved.” A background writer saves it later. Trace a crash that loses the task, then propose a change that meets the restart requirement.

Read the explanation →

IMPLEMENTATION / I1

Run examples/todo.py. Make a test repository raise a storage error. Then use the real repository to save a task, close it, and read the task through a new connection. What should each check show?

Read the explanation →

THOUGHT EXPERIMENT / T1

The team now requires saved tasks to survive destruction of the only disk. Why is ADR 001 insufficient, and what must the next design address?

Read the explanation →

Unit 2 · Requirements, Estimation, and Engineering Trade-offs

CONCEPTUAL / C1

Why does one feed request not imply only one request of infrastructure work?

Read the explanation →

CONCEPTUAL / C2

How can offered RPS rise while useful throughput stops growing?

Read the explanation →

CONCEPTUAL / C3

Why can a 19.9 ms mean coexist with a one-second request?

Read the explanation →

CONCEPTUAL / C4

Which denominator makes an error budget meaningful?

Read the explanation →

CONCEPTUAL / C5

What assumptions make L = lambda W useful?

Read the explanation →

SCENARIO / S1

A fivefold peak lasts less than instance startup time. What must protect service before autoscaling helps?

Read the explanation →

SCENARIO / S2

Only 80% of work parallelizes. Can four workers provide fourfold speedup?

Read the explanation →

SCENARIO / S3

Why does fifty-way required fan-out increase the chance of a slow request?

Read the explanation →

DESIGN / D1

Design or debug the running system: Build a capacity worksheet from ten million daily users and explicitly assumed request, peak, and per-instance rates.

Read the explanation →

IMPLEMENTATION / I1

Build a capacity worksheet from ten million daily users and explicitly assumed request, peak, and per-instance rates. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would a coordinated retry burst change an estimate based only on independent average user sessions?

Read the explanation →

Unit 3 · Networking Foundations for System Design

CONCEPTUAL / C1

Why does a listening port not prove the application is reachable or authorized?

Read the explanation →

CONCEPTUAL / C2

Which TCP, UDP, and QUIC distinctions affect application design?

Read the explanation →

CONCEPTUAL / C3

Why is a DNS change not an instantaneous global failover?

Read the explanation →

CONCEPTUAL / C4

Which gateway responsibilities should remain distinct from domain authorization?

Read the explanation →

CONCEPTUAL / C5

What cost does streaming exchange for repeated polling?

Read the explanation →

SCENARIO / S1

A client times out after the server commits. What can the client safely conclude?

Read the explanation →

SCENARIO / S2

A cold secure request is slower than a warm one with identical server work. Which waits might explain it?

Read the explanation →

SCENARIO / S3

A database pool is full while CPU is low. What should be measured before increasing the pool?

Read the explanation →

DESIGN / D1

Design or debug the running system: Implement a cold/warm request latency calculator with separate DNS, connection, TLS, application, and response stages.

Read the explanation →

IMPLEMENTATION / I1

Implement a cold/warm request latency calculator with separate DNS, connection, TLS, application, and response stages. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

What changes when a supposedly local service call becomes a cross-region network dependency?

Read the explanation →

Unit 4 · Runtime, Processes, Threads, and Concurrency

CONCEPTUAL / C1

How do process and thread boundaries differ from source-module boundaries?

Read the explanation →

CONCEPTUAL / C2

Why can an async function still block other requests?

Read the explanation →

CONCEPTUAL / C3

Does concurrency require simultaneous execution on several cores?

Read the explanation →

CONCEPTUAL / C4

Why can individually atomic fields fail to protect a multi-field invariant?

Read the explanation →

CONCEPTUAL / C5

Why does wall-clock order not establish memory visibility?

Read the explanation →

SCENARIO / S1

A and B both read counter zero, then each writes one. Why is one increment missing?

Read the explanation →

SCENARIO / S2

A two-second callback runs on one event loop. What happens to other ready tasks?

Read the explanation →

SCENARIO / S3

Arrivals stay at 150 jobs/s while service completes 100/s. Does a larger queue solve the problem?

Read the explanation →

DESIGN / D1

Design or debug the running system: Implement the lost-counter interleaving and a bounded upload work queue.

Read the explanation →

IMPLEMENTATION / I1

Implement the lost-counter interleaving and a bounded upload work queue. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

What should happen to accepted uploads when the request process is cancelled but the business still requires processing?

Read the explanation →

Unit 5 · Object-Oriented and Modular Software Design

CONCEPTUAL / C1

When does composition avoid a subclass explosion?

Read the explanation →

CONCEPTUAL / C2

Which payment semantics must a gateway contract state beyond its method signature?

Read the explanation →

CONCEPTUAL / C3

Why do SOLID heuristics not require an interface for every class?

Read the explanation →

CONCEPTUAL / C4

How do payment identity and a Money value differ?

Read the explanation →

CONCEPTUAL / C5

How can a functional design keep effects and pure decisions separate?

Read the explanation →

SCENARIO / S1

Two callers read and overwrite an account balance. Why should the state owner receive an intent?

Read the explanation →

SCENARIO / S2

A mock proves every private method was called. What important behavior can still be untested?

Read the explanation →

SCENARIO / S3

A refactor changes provider timeout into decline. Why is that not merely a structural improvement?

Read the explanation →

DESIGN / D1

Design or debug the running system: Refactor a hardcoded payment gateway into an injected contract and a recording test implementation.

Read the explanation →

IMPLEMENTATION / I1

Refactor a hardcoded payment gateway into an injected contract and a recording test implementation. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

Which payment abstractions remain useful if the implementation moves from objects to pure functions?

Read the explanation →

Unit 6 · Design Patterns, Dependency Injection, and Object Lifecycles

CONCEPTUAL / C1

Can constructor injection exist without dependency inversion?

Read the explanation →

CONCEPTUAL / C2

Is a container singleton one instance across all hosts?

Read the explanation →

CONCEPTUAL / C3

Why are resource creation and disposal ownership different contracts?

Read the explanation →

CONCEPTUAL / C4

Why can injecting a container preserve the service-locator problem?

Read the explanation →

CONCEPTUAL / C5

How can DI and a factory cooperate without being the same pattern?

Read the explanation →

SCENARIO / S1

A singleton stores a request-scoped unit of work. What fails after the request ends?

Read the explanation →

SCENARIO / S2

Two first callers trigger lazy initialization. What must coordinate them?

Read the explanation →

SCENARIO / S3

A failed request leaks one connection. Why can healthy traffic eventually stop?

Read the explanation →

DESIGN / D1

Design or debug the running system: Build a small resolver that demonstrates transient, scoped, and singleton identities across two request scopes.

Read the explanation →

IMPLEMENTATION / I1

Build a small resolver that demonstrates transient, scoped, and singleton identities across two request scopes. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would the ownership rules change if a job outlives both a request scope and a user session?

Read the explanation →

Unit 7 · Application Architecture and Domain-Driven Design

CONCEPTUAL / C1

What makes a monolith modular even though it has one deployment?

Read the explanation →

CONCEPTUAL / C2

Why do source-dependency arrows differ from runtime call paths?

Read the explanation →

CONCEPTUAL / C3

Why may catalog Product and fulfillment Item be different legitimate models?

Read the explanation →

CONCEPTUAL / C4

How should an aggregate boundary follow a consistency invariant?

Read the explanation →

CONCEPTUAL / C5

What should an architecture fitness check preserve?

Read the explanation →

SCENARIO / S1

An order commits and then the process crashes before event publication. Why is an in-memory event insufficient?

Read the explanation →

SCENARIO / S2

Five services share writable tables and must always release together. What independence is missing?

Read the explanation →

SCENARIO / S3

A small team with a rapidly changing model considers microservices. What evidence would justify extraction?

Read the explanation →

DESIGN / D1

Design or debug the running system: Divide shopping into catalog, orders, and payments modules and implement one boundary test.

Read the explanation →

IMPLEMENTATION / I1

Divide shopping into catalog, orders, and payments modules and implement one boundary test. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

Which evidence would justify extracting payments from a modular monolith while retaining the same business invariants?

Read the explanation →

Unit 8 · API Design and Communication Contracts

CONCEPTUAL / C1

Why should a refund have a separately tracked identity and lifecycle?

Read the explanation →

CONCEPTUAL / C2

Which offset-pagination anomaly does a keyset cursor reduce, and what does it not guarantee?

Read the explanation →

CONCEPTUAL / C3

Which records and atomic boundary make an idempotency key meaningful?

Read the explanation →

CONCEPTUAL / C4

Why is authentication insufficient for GET /payments/42?

Read the explanation →

CONCEPTUAL / C5

Why can event schema evolution break recovery long after a release?

Read the explanation →

SCENARIO / S1

A POST returns 201 with a pending payment resource. Does that mean the provider captured money?

Read the explanation →

SCENARIO / S2

Three attempts occur at each of four layers. What risk does this create?

Read the explanation →

SCENARIO / S3

A webhook receiver commits its effect but loses the acknowledgment. What must happen on redelivery?

Read the explanation →

DESIGN / D1

Design or debug the running system: Implement keyset pagination and a durable idempotency-key state machine over a synthetic payment operation.

Read the explanation →

IMPLEMENTATION / I1

Implement keyset pagination and a durable idempotency-key state machine over a synthetic payment operation. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

Can a public API offer a permanent idempotency promise with finite storage? Define a defensible contract.

Read the explanation →

Unit 9 · Data Models and Storage Technologies

CONCEPTUAL / C1

Which storage promises must be stated before choosing a database family?

Read the explanation →

CONCEPTUAL / C2

Why is a vector embedding not authoritative evidence or permission?

Read the explanation →

CONCEPTUAL / C3

When is duplicating an order address a legitimate historical fact?

Read the explanation →

CONCEPTUAL / C4

Why is data ownership more than choosing who creates a table?

Read the explanation →

CONCEPTUAL / C5

What operating responsibilities are added by a second storage engine?

Read the explanation →

SCENARIO / S1

One huge conversation dominates a wide-column partition. Why do more cold partitions not solve it?

Read the explanation →

SCENARIO / S2

Search still contains a deleted private document. Which serving and lifecycle obligations remain?

Read the explanation →

SCENARIO / S3

An old backup is restored after deletion requests. What must prevent privacy resurrection?

Read the explanation →

DESIGN / D1

Design or debug the running system: Model profiles, transactions, messages, logs, and PDFs with their access patterns and ownership.

Read the explanation →

IMPLEMENTATION / I1

Model profiles, transactions, messages, logs, and PDFs with their access patterns and ownership. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

How does restoration from a historical backup interact with current privacy deletion and derived indexes?

Read the explanation →

Unit 10 · Database Internals, Transactions, and Concurrency Control

CONCEPTUAL / C1

Why do ordered range queries and equality lookup motivate different index choices?

Read the explanation →

CONCEPTUAL / C2

Why does ACID not infer every business invariant?

Read the explanation →

CONCEPTUAL / C3

Why is MVCC not a synonym for serializable behavior?

Read the explanation →

CONCEPTUAL / C4

Why must isolation claims name the engine and statement behavior?

Read the explanation →

CONCEPTUAL / C5

How does serializability differ from real-time linearizability?

Read the explanation →

SCENARIO / S1

Two on-call doctors each see the other and both go off duty. Which invariant did row-level thinking miss?

Read the explanation →

SCENARIO / S2

Two buyers update from expected seat version seven. What does zero affected rows mean?

Read the explanation →

SCENARIO / S3

The backup job succeeded but no restore has run. What evidence is missing?

Read the explanation →

DESIGN / D1

Design or debug the running system: Run the last-seat claim using a conditional database update, then test conflicting writers.

Read the explanation →

IMPLEMENTATION / I1

Run the last-seat claim using a conditional database update, then test conflicting writers. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

Which invariant becomes harder when a local multi-statement transaction is split across independently owned databases?

Read the explanation →

Unit 11 · Caching and Content Delivery

CONCEPTUAL / C1

Why does working-set size matter more than historical object count for cache usefulness?

Read the explanation →

CONCEPTUAL / C2

How can cache-aside repopulate stale data after invalidation?

Read the explanation →

CONCEPTUAL / C3

Why is a TTL not always a bound on source-data age?

Read the explanation →

CONCEPTUAL / C4

How do LRU and FIFO differ for A,B,C,A,D at capacity three?

Read the explanation →

CONCEPTUAL / C5

When can removing a cache be the better design?

Read the explanation →

SCENARIO / S1

A cache restart sends every reader to origin. Which mechanisms limit the resulting storm?

Read the explanation →

SCENARIO / S2

A not-found result is cached just before object creation. What freshness behavior must be defined?

Read the explanation →

SCENARIO / S3

A stale value keeps being served while refresh fails. What boundary prevents unbounded stale operation?

Read the explanation →

DESIGN / D1

Design or debug the running system: Run the LRU fixture and reproduce a stale cache-aside population race.

Read the explanation →

IMPLEMENTATION / I1

Run the LRU fixture and reproduce a stale cache-aside population race. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

When would a product choose lower availability rather than serve a stale cached value?

Read the explanation →

Unit 12 · Scalability, Load Balancing, and Data Distribution

CONCEPTUAL / C1

Why does adding application servers fail to fix some storage bottlenecks?

Read the explanation →

CONCEPTUAL / C2

Which query and invariant costs arrive with sharding?

Read the explanation →

CONCEPTUAL / C3

What does stable hashing reduce, and which skew does it leave?

Read the explanation →

CONCEPTUAL / C4

Why are receive progress and apply progress useful distinctions for replicas?

Read the explanation →

CONCEPTUAL / C5

Why must scale-out costs be compared under equal failure promises?

Read the explanation →

SCENARIO / S1

One partition receives 70% of writes. What should guide the intervention?

Read the explanation →

SCENARIO / S2

Each new application instance creates a large connection pool. How can scaling worsen the outage?

Read the explanation →

SCENARIO / S3

Two replicas share one host. Which failure remains unprotected?

Read the explanation →

DESIGN / D1

Design or debug the running system: Compare rendezvous owners before and after adding node D, and measure skew separately.

Read the explanation →

IMPLEMENTATION / I1

Compare rendezvous owners before and after adding node D, and measure skew separately. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

Can a single globally hot invariant be scaled merely by adding more partitions? Explain the constraint.

Read the explanation →

Unit 13 · File Systems, Search Infrastructure, and Data Pipelines

CONCEPTUAL / C1

Why do acknowledged upload parts not alone prove the final file is correct?

Read the explanation →

CONCEPTUAL / C2

Why does content addressing not authorize access?

Read the explanation →

CONCEPTUAL / C3

What does an inverted index store for blue car and blue bike?

Read the explanation →

CONCEPTUAL / C4

Which snapshot-to-CDC handoff problem can lose changes?

Read the explanation →

CONCEPTUAL / C5

Which identities make an extracted field explainable?

Read the explanation →

SCENARIO / S1

Writes and deletes continue during reindexing. What must be replayed before a serving switch?

Read the explanation →

SCENARIO / S2

An event arrives three minutes after it occurred. Which time and window policies matter?

Read the explanation →

SCENARIO / S3

Only the PDF is deleted; extracted text and index entries remain. What lifecycle contract is incomplete?

Read the explanation →

DESIGN / D1

Design or debug the running system: Implement an input-versioned pipeline with a lineage record and a reindex generation switch.

Read the explanation →

IMPLEMENTATION / I1

Implement an input-versioned pipeline with a lineage record and a reindex generation switch. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would you reproduce a disputed extracted field after both the source and model have new versions?

Read the explanation →

Unit 14 · Distributed Systems Fundamentals

CONCEPTUAL / C1

Why is provider success plus a lost reply a partial failure?

Read the explanation →

CONCEPTUAL / C2

What does Lamport ordering imply, and what does it not imply?

Read the explanation →

CONCEPTUAL / C3

Which real-time relationship must a linearizable register preserve?

Read the explanation →

CONCEPTUAL / C4

Why is CAP not a permanent pick-any-two menu?

Read the explanation →

CONCEPTUAL / C5

Can rejecting every operation satisfy safety while failing useful progress?

Read the explanation →

SCENARIO / S1

A session reads version ten and then routes to a replica at nine. Which guarantee is threatened?

Read the explanation →

SCENARIO / S2

R=2 and W=2 for three replicas intersect. Why is that not a complete linearizability proof?

Read the explanation →

SCENARIO / S3

An old leader resumes after a replacement was elected. What must reject its stale effects?

Read the explanation →

DESIGN / D1

Design or debug the running system: Evaluate logical clocks, vector relations, and fixed-quorum intersection for explicit histories.

Read the explanation →

IMPLEMENTATION / I1

Evaluate logical clocks, vector relations, and fixed-quorum intersection for explicit histories. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

During a partition, which operations could degrade while exclusive allocation must wait or reject?

Read the explanation →

Unit 15 · Replication, Leader Election, and Consensus

CONCEPTUAL / C1

Which acknowledgment meaning must synchronous replication state?

Read the explanation →

CONCEPTUAL / C2

Why must replicated state-machine commands apply deterministically?

Read the explanation →

CONCEPTUAL / C3

Why does Raft commit reasoning need more than two acknowledgments?

Read the explanation →

CONCEPTUAL / C4

Why can disjoint old and new voter configurations be unsafe?

Read the explanation →

CONCEPTUAL / C5

Why does convergence not establish an inventory invariant?

Read the explanation →

SCENARIO / S1

Worker epoch seven resumes after epoch eight wrote. Where must fencing be enforced?

Read the explanation →

SCENARIO / S2

A long pause falsely suggests failure. Which part of the design must remain safe?

Read the explanation →

SCENARIO / S3

Failover succeeds but the old primary later returns. What must a safe failback consider?

Read the explanation →

DESIGN / D1

Design or debug the running system: Create a small fencing receiver and reject an old worker after a newer epoch has been accepted.

Read the explanation →

IMPLEMENTATION / I1

Create a small fencing receiver and reject an old worker after a newer epoch has been accepted. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

What does a region returning after failover have to prove before it can accept authoritative writes again?

Read the explanation →

Unit 16 · Message Queues, Events, and Distributed Workflows

CONCEPTUAL / C1

Why does an ephemeral subscription not supply historical replay?

Read the explanation →

CONCEPTUAL / C2

How does acknowledgment placement trade lost work against duplicate work?

Read the explanation →

CONCEPTUAL / C3

Which boundary can a scoped exactly-once claim actually cover?

Read the explanation →

CONCEPTUAL / C4

Why can a transactional outbox still publish duplicates?

Read the explanation →

CONCEPTUAL / C5

Why is compensation not necessarily a perfect inverse?

Read the explanation →

SCENARIO / S1

An event effect commits but acknowledgment is lost. How does the next delivery avoid a second local effect?

Read the explanation →

SCENARIO / S2

A worker’s lease expires while it is still running. What protects later effects?

Read the explanation →

SCENARIO / S3

A payment remains uncertain locally but charged externally. Why is blind resubmission unsafe?

Read the explanation →

DESIGN / D1

Design or debug the running system: Run the outbox relay through publish-then-crash and process the duplicate with a local inbox transaction.

Read the explanation →

IMPLEMENTATION / I1

Run the outbox relay through publish-then-crash and process the duplicate with a local inbox transaction. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

Can one workflow be exactly once at a broker boundary but still send two emails? Specify the boundaries.

Read the explanation →

Unit 17 · Reliability, Fault Tolerance, and Disaster Recovery

CONCEPTUAL / C1

Why does redundant instance count not establish surviving capacity?

Read the explanation →

CONCEPTUAL / C2

Why must pool acquisition fit the overall deadline?

Read the explanation →

CONCEPTUAL / C3

What does a circuit breaker change, and what does it not repair?

Read the explanation →

CONCEPTUAL / C4

Which shared limits can defeat apparently separate worker bulkheads?

Read the explanation →

CONCEPTUAL / C5

Why are RTO and RPO different objectives?

Read the explanation →

SCENARIO / S1

Nested retries amplify provider load during an outage. Which end-to-end controls are needed?

Read the explanation →

SCENARIO / S2

The ledger is unavailable. Can a cached default balance be a safe fallback?

Read the explanation →

SCENARIO / S3

A repaired old primary returns after failover. Why not just route traffic back?

Read the explanation →

DESIGN / D1

Design or debug the running system: Model nested retries and write a provider-outage response policy.

Read the explanation →

IMPLEMENTATION / I1

Model nested retries and write a provider-outage response policy. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

How can a disaster exercise pass infrastructure checks yet fail the business recovery requirement?

Read the explanation →

Unit 18 · Security Architecture and Privacy

CONCEPTUAL / C1

Why does an internal network not automatically authenticate a caller?

Read the explanation →

CONCEPTUAL / C2

How do OAuth access authorization and OpenID Connect identity differ?

Read the explanation →

CONCEPTUAL / C3

Why is a fast general-purpose hash unsuitable for passwords?

Read the explanation →

CONCEPTUAL / C4

Which paths must preserve tenant isolation beyond the database query?

Read the explanation →

CONCEPTUAL / C5

Why does an artifact signature not prove the artifact is harmless?

Read the explanation →

SCENARIO / S1

An administrator of tenant A guesses a tenant B document ID. What must enforce denial?

Read the explanation →

SCENARIO / S2

A backup survives but its only decryption key was destroyed. Which recovery assumption failed?

Read the explanation →

SCENARIO / S3

Membership is revoked after an export is queued. Which later boundary must follow the chosen revocation policy?

Read the explanation →

DESIGN / D1

Design or debug the running system: Test a tenant-scoped authorization predicate and all cross-tenant paths around it.

Read the explanation →

IMPLEMENTATION / I1

Test a tenant-scoped authorization predicate and all cross-tenant paths around it. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would tenant revocation affect an already issued object capability and a queued export?

Read the explanation →

Unit 19 · Observability, Monitoring, and Incident Engineering

CONCEPTUAL / C1

How do logs, metrics, and traces answer complementary questions?

Read the explanation →

CONCEPTUAL / C2

Why is a trace ID not authorization?

Read the explanation →

CONCEPTUAL / C3

Why can pool saturation explain high latency with low CPU?

Read the explanation →

CONCEPTUAL / C4

What does tenfold error-budget burn mean for a 99.9% target?

Read the explanation →

CONCEPTUAL / C5

Why do unique request IDs make poor general metric labels?

Read the explanation →

SCENARIO / S1

A five-second request spent 4.6 seconds acquiring a connection. Which optimization is the evidence pointing toward?

Read the explanation →

SCENARIO / S2

Serving recovers but uncertain charges remain. Why is the incident not fully resolved?

Read the explanation →

SCENARIO / S3

HTTP 202 returns quickly while jobs wait for hours. Which end-to-end indicators are missing?

Read the explanation →

DESIGN / D1

Design or debug the running system: Build a trace waterfall and an end-to-end asynchronous completion indicator.

Read the explanation →

IMPLEMENTATION / I1

Build a trace waterfall and an end-to-end asynchronous completion indicator. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

Which evidence would make a latency hypothesis convincing rather than merely correlated with a release?

Read the explanation →

Unit 20 · Deployment, Cloud Infrastructure, and Platform Architecture

CONCEPTUAL / C1

Which VM and container isolation and ownership distinctions matter?

Read the explanation →

CONCEPTUAL / C2

How can control-plane failure differ from data-plane failure?

Read the explanation →

CONCEPTUAL / C3

Why can a canary receiving only cheap requests miss a serious regression?

Read the explanation →

CONCEPTUAL / C4

Why can schema changes make code rollback unsafe?

Read the explanation →

CONCEPTUAL / C5

Which responsibilities stay with a team using a managed database?

Read the explanation →

SCENARIO / S1

Scale-down kills workers holding unrecorded jobs. What lifecycle contract was missing?

Read the explanation →

SCENARIO / S2

A feature flag is disabled after new state was written. What compatibility issue remains?

Read the explanation →

SCENARIO / S3

Two application replicas use independent local databases. Why did scheduling not create a correct shared service?

Read the explanation →

DESIGN / D1

Design or debug the running system: Write an expand-and-contract migration and a mixed-version rollout test.

Read the explanation →

IMPLEMENTATION / I1

Write an expand-and-contract migration and a mixed-version rollout test. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

How could a rollback restore old code while making stored data or external effects less correct?

Read the explanation →

Unit 21 · Performance, Cost Optimization, and System Validation

CONCEPTUAL / C1

What must a benchmark disclose before its result is reusable?

Read the explanation →

CONCEPTUAL / C2

How can coordinated omission hide delay?

Read the explanation →

CONCEPTUAL / C3

Why is the short-run throughput ceiling not a safe production target?

Read the explanation →

CONCEPTUAL / C4

Why does cost per attempt sometimes conceal cost per useful outcome?

Read the explanation →

CONCEPTUAL / C5

What should a useful architecture fitness function protect?

Read the explanation →

SCENARIO / S1

An index fixes the actual slow query. Why might sharding have been a worse first intervention?

Read the explanation →

SCENARIO / S2

The process is only killed before starting work. Which dangerous failure windows remain untested?

Read the explanation →

SCENARIO / S3

Source and destination have equal row counts but different ownership. Why did the validation miss a bug?

Read the explanation →

DESIGN / D1

Design or debug the running system: Compare three architectures using explicit fictional costs and equal requirement checks.

Read the explanation →

IMPLEMENTATION / I1

Compare three architectures using explicit fictional costs and equal requirement checks. Include a counterexample and a corrected result.

Read the explanation →

THOUGHT EXPERIMENT / T1

What evidence would justify accepting a more expensive architecture as the simpler complete operating system?

Read the explanation →

Unit 22 · End-to-End Design, Architecture Reviews, and Evolution

CONCEPTUAL / C1

Why do the twelve designs use different authoritative and derived state boundaries?

Read the explanation →

CONCEPTUAL / C2

Why are a chat persistence receipt and a human read receipt different?

Read the explanation →

CONCEPTUAL / C3

Why can a scheduler lease not guarantee a single external execution?

Read the explanation →

CONCEPTUAL / C4

Why does a collaborative editor need more than eventual string equality?

Read the explanation →

CONCEPTUAL / C5

Why is a document-processing complete state inadequate without provenance and publication evidence?

Read the explanation →

SCENARIO / S1

A booking payment succeeds after its seat hold expires. What outcomes are safe?

Read the explanation →

SCENARIO / S2

A payment provider charges, the local process dies, and a client retries. What prevents a second logical charge?

Read the explanation →

SCENARIO / S3

A deleted document has an old extraction worker still running. What must prevent publication?

Read the explanation →

DESIGN / D1

Review the URL shortener at 100000 redirects/s with a cold cache and immediate revocation requirement.

Read the explanation →

IMPLEMENTATION / I1

Choose one case and implement its final challenge with failure injection and an explicit invariant.

Read the explanation →

THOUGHT EXPERIMENT / T1

You may fund only one improvement in a small platform: another region, tested restores, or stronger retry and reconciliation. How do you decide?

Read the explanation →

ALL 22 UNITS / EXPLAINED ANSWERS

Solutions & reasoning

Each explanation connects the outcome to assumptions, mechanism, and failure. A later or linked chapter provides the full surrounding context.

Unit 1 · Introduction to System Design

UNIT 01 / C1

A program adds a task to a dictionary and displays “Saved.” What can it do while running, and what happens when it starts with a new dictionary?

While the program runs, it can list and update the task in that dictionary. After restart, a new empty dictionary contains none of the previous tasks. To keep the task across a restart, the program must save it in persistent storage and wait for the save to finish before reporting success.

← Return to question

UNIT 01 / C2

What does the signature create(title) tell a caller? What must the contract explain?

The signature names the operation and its input; a typed signature can also describe the result type. The contract explains the behaviour: which titles are valid, what changes when the call succeeds, and which errors may occur. In our persistent application, it also says that creation returns successfully only after commit. Callers need these rules to use the operation correctly.

← Return to question

UNIT 01 / C3

The task service depends on a repository. Does this dependency necessarily make the design difficult to change?

No. The service needs a way to save tasks. A repository contract lets it request that work without knowing the SQL or file layout. A column rename can then be handled in the repository. The useful question is how much one module must know about another and whether a change to those details spreads through the application.

← Return to question

UNIT 01 / C4

How do a high-level view, a low-level view, and a deployment view help explain the task application?

The high-level view shows the main parts, such as the application and database. The low-level view explains the service methods, task records, and repository interface inside the application. The deployment view places the running application and database file on machines. These views describe the same system, so their decisions must agree. A separate repository box in a module diagram does not imply a separate server.

← Return to question

UNIT 01 / C5

A service stops accepting requests during recovery but keeps every committed task. Which observation concerns availability, and which concerns durability?

The period when users cannot perform task operations concerns availability. The fact that committed tasks remain concerns durability under that recovery scenario. A system can preserve data while being temporarily unavailable. Conversely, it can keep replying while losing tasks that were stored only in memory.

← Return to question

UNIT 01 / S1

Task 7 is committed, but its success reply is lost. The client waits, then retries. Can it conclude that the first attempt failed?

No. In this sequence, task 7 exists even though the client has not received the reply. A fresh create request may add task 8. To recognise the retry, the client needs to reuse an operation identifier, and the server must keep that identifier with the committed result. The design must also specify how it checks that the repeated request matches and how long it remembers the result. Chapter 8.2 develops these rules.

← Return to question

UNIT 01 / S2

The web adapter and command-line adapter each enforce an 80-character title limit. Only the web adapter is changed to allow 200 characters. How would you prevent this disagreement?

Move the shared title rule into the task service and make both adapters call it. Each adapter still checks its own input format, such as whether a web request contains valid JSON. Check that the database can store the longer title and update any matching storage constraint. Editing both copied rules fixes today’s mismatch but leaves the same risk for the next change.

← Return to question

UNIT 01 / S3

The team runs two application machines, each with its own unrelated local task database. A user sees a task on one request and cannot find it on the next. What could explain this?

The two requests may have reached different machines. A task saved in one local database is absent from the other. The design now needs a way for the applications to use shared or coordinated data. Adding a load balancer distributes requests; it does not copy or merge the task databases.

← Return to question

UNIT 01 / D1

A handler adds a task to memory and replies “Saved.” A background writer saves it later. Trace a crash that loses the task, then propose a change that meets the restart requirement.

The failing sequence is: add to memory, reply “Saved,” crash before the background write commits. After restart, memory is empty and there is no saved task to reload. Change the order so that success follows a durable commit, and report a write error if the save fails. A local database is enough for our stated single-machine example. A background queue would need its own durable-save and acknowledgment rules before it could support the same promise.

← Return to question

UNIT 01 / I1

Run examples/todo.py. Make a test repository raise a storage error. Then use the real repository to save a task, close it, and read the task through a new connection. What should each check show?

When insert raises a storage error, create must pass that error to its caller rather than return a successful Task. With the real SQLite repository, the task should still be present after closing and reopening the same database. Also submit an invalid title and check that no row is added. The first check covers error handling, the second persistence across reopen, and the third validation. An in-memory substitute cannot establish the persistence result.

← Return to question

UNIT 01 / T1

The team now requires saved tasks to survive destruction of the only disk. Why is ADR 001 insufficient, and what must the next design address?

ADR 001 assumes that committed storage remains intact. Destroying its only disk removes the copy it relies on. The new design needs recoverable data on independent storage and a rule for when it may report success. If copying happens only later, a task reported as saved can be lost before the copy is made. Specify which failures the copies must survive, how much loss is acceptable, and how recovery will work. Then test those cases. Simply counting replicas does not answer these questions.

← Return to question

Unit 2 · Requirements, Estimation, and Engineering Trade-offs

UNIT 02 / C1

Why does one feed request not imply only one request of infrastructure work?

Users do not generate a constant stream of identical requests. Break a session into operations and attach payload size, arrival rate, fan-out, and dependency cost to each. In an assumed session, opening a feed causes one feed request, twenty image reads, and one analytics event. Counting only the first request underestimates network work. A journey model helps predict which component saturates; production telemetry should replace its assumptions as soon as evidence exists.

← Return to question

UNIT 02 / C2

How can offered RPS rise while useful throughput stops growing?

Latency measures elapsed time; throughput measures completed work per time interval. Offered requests per second can exceed successful completions per second while a queue grows. QPS usually counts queries, RPS requests, and TPS transactions, but teams must define these denominators. At 200 offered requests/s and 180 completions/s, backlog grows by twenty requests each second unless requests are rejected. Reporting only accepted traffic conceals overload and abandoned work.

← Return to question

UNIT 02 / C3

Why can a 19.9 ms mean coexist with a one-second request?

A mean can hide users who wait much longer. For 100 illustrative requests, 99 at 10 ms and one at 1000 ms, the mean is 19.9 ms; the slow request is still a second. Percentiles describe the distribution under a specified estimation convention. Do not average machine p99 values to obtain fleet p99; merge distributions or appropriate histograms. Include errors and timed-out operations in the experience model rather than quietly dropping them.

← Return to question

UNIT 02 / C4

Which denominator makes an error budget meaningful?

Choose a user-visible SLI, an objective and window, then describe any contractual SLA separately. A 99.9% event-based success objective across one million eligible requests allows 1000 bad events. Excluding failures by definition invalidates the budget. Error budgets inform release and reliability decisions; they do not authorize spending all failures in one outage. A time-based SLO uses time rather than request count and gives a different allowance.

← Return to question

UNIT 02 / C5

What assumptions make L = lambda W useful?

Little's Law relates long-run averages across the same stable boundary: L = lambda W. At 500 successful operations/s and 40 ms average residence time, average in-flight work is twenty. If residence time becomes 400 ms at the same stable throughput, it becomes two hundred. The equation is an accounting relationship, not a p99 model or proof that an overloaded queue will stabilize. Rejections, retries, and work outside the boundary need explicit accounting.

← Return to question

UNIT 02 / S1

A fivefold peak lasts less than instance startup time. What must protect service before autoscaling helps?

Average traffic hides synchronized arrivals. With an assumed peak factor of five, the preceding 5787 requests/s becomes about 28935 at peak. A uniform fivefold multiplier is only a planning hypothesis: image and write peaks may differ, and retries can amplify outages. Evaluate a spike's duration as well as height because an autoscaler that starts new instances after the spike may provide little protection. Admission limits must work before scaling catches up.

← Return to question

UNIT 02 / S2

Only 80% of work parallelizes. Can four workers provide fourfold speedup?

Amdahl's Law bounds ideal fixed-work speedup when only part of the work can be parallelized. If 80% parallelizes perfectly, four workers give 1/(0.2+0.8/4)=2.5 times speedup, while infinitely many approach five. Coordination and changed cache behavior can make real speedup worse. Optimize the actual limiting stage: doubling web workers is ineffective when every request waits for a serialized database operation. Re-measure after each improvement because the bottleneck moves.

← Return to question

UNIT 02 / S3

Why does fifty-way required fan-out increase the chance of a slow request?

A request's tail can be dominated by the slowest dependency. If it waits for all fifty independent calls and each has a 1% chance of exceeding a threshold, the chance that at least one exceeds it is 1−0.99^50, about 39.5%. Independence is an assumption; shared failures can correlate tails. Allocate an end-to-end deadline across network, queues, execution, and response. Hedging may reduce some tails but spends extra capacity and requires safe duplicate handling.

← Return to question

UNIT 02 / D1

Design or debug the running system: Build a capacity worksheet from ten million daily users and explicitly assumed request, peak, and per-instance rates.

Use capacity() and compare 5787.04 average RPS, 28935.19 peak RPS, 81 active instances at the stated hypothetical capacity/utilization, and 82 for one-instance failure headroom. This arithmetic does not prove the database or complete architecture meets the workload. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 02 / I1

Build a capacity worksheet from ten million daily users and explicitly assumed request, peak, and per-instance rates. Include a counterexample and a corrected result.

Use capacity() and compare 5787.04 average RPS, 28935.19 peak RPS, 81 active instances at the stated hypothetical capacity/utilization, and 82 for one-instance failure headroom. This arithmetic does not prove the database or complete architecture meets the workload.

← Return to question

UNIT 02 / T1

How would a coordinated retry burst change an estimate based only on independent average user sessions?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. A decision matrix makes judgments visible but does not turn them into measurements. Apply hard constraints first, then score candidates against defined criteria with explicit weights and uncertainty. If a small weight change reverses the winner, gather more evidence or prefer a reversible choice. Record alternatives and revisit triggers in an ADR. One weighted total cannot establish an invariant, eliminate an unknown dependency, or substitute for a representative load test. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 3 · Networking Foundations for System Design

UNIT 03 / C1

Why does a listening port not prove the application is reachable or authorized?

An IP address identifies a network endpoint within a routing context; a port identifies a transport endpoint at that host. A socket represents a communication endpoint managed by the operating system. Routing forwards packets toward destinations, potentially through several networks and translations. A service listening on port 443 is not necessarily reachable: binding, routing, firewall policy, and return paths all matter. Ports are not authentication credentials, and private address space is not an authorization boundary.

← Return to question

UNIT 03 / C2

Which TCP, UDP, and QUIC distinctions affect application design?

TCP supplies an ordered reliable byte stream; it preserves bytes, not application-message boundaries. UDP sends datagrams without those stream guarantees. QUIC builds secure streams and transport behavior over UDP, including stream-level loss isolation and connection identifiers. Choose by application requirements and deployed infrastructure, not by claiming UDP is always faster. A lost TCP packet can delay later bytes on the connection; QUIC separates streams but still shares congestion and application dependencies.

← Return to question

UNIT 03 / C3

Why is a DNS change not an instantaneous global failover?

A resolver obtains records for a name, often through caches and authoritative servers. TTL limits intended cache lifetime, not an instantaneous global invalidation deadline. Negative answers can also be cached. During a failover, clients with cached addresses may continue contacting an old endpoint, and long-lived connections may bypass DNS entirely. Test resolver behavior and connection lifetimes together; changing a DNS record alone cannot guarantee immediate traffic movement.

← Return to question

UNIT 03 / C4

Which gateway responsibilities should remain distinct from domain authorization?

A load balancer distributes traffic among backends; an API gateway commonly adds API-specific policy such as authentication integration, quotas, and request transformation. Products overlap, so compare responsibilities instead of names. A gateway that centralizes every business rule becomes a coupled deployment bottleneck. Keep domain authorization where resource ownership is known, and let an edge gateway perform appropriate coarse admission and routing. Neither component fixes an overloaded database by itself.

← Return to question

UNIT 03 / C5

What cost does streaming exchange for repeated polling?

Polling repeatedly asks for updates; server-sent events stream server-to-client events over HTTP; WebSockets support bidirectional messaging after protocol setup. With 1000 idle clients polling every five seconds, the baseline is 200 polls/s before useful updates. Streaming reduces redundant polls but holds connections and requires reconnect, heartbeat, buffering, and recovery design. Neither WebSockets nor SSE automatically supplies durable message delivery or application-level replay after a disconnected client returns.

← Return to question

UNIT 03 / S1

A client times out after the server commits. What can the client safely conclude?

A timeout means the caller stopped waiting; it does not reveal whether the remote side executed. Packets may be delayed, dropped, reordered, or blocked while other paths remain healthy. Distinguish connection establishment, idle, and overall request deadlines. Retrying a non-idempotent action can duplicate effects; waiting indefinitely can exhaust resources. Treat partial failure as a normal execution history and reconcile ambiguous outcomes using stable operation identity or status queries.

← Return to question

UNIT 03 / S2

A cold secure request is slower than a warm one with identical server work. Which waits might explain it?

Round trips create a latency floor even before useful computation. In an illustrative cold path, DNS costs 20 ms, TCP 40 ms, TLS 40 ms, and request/response plus execution 55 ms: total 155 ms. This is a simplified sequential example, not a benchmark; resumption, QUIC, cached DNS, and overlap change it. Moving a service closer can reduce propagation delay, while batching or reuse reduces the number of required exchanges.

← Return to question

UNIT 03 / S3

A database pool is full while CPU is low. What should be measured before increasing the pool?

A connection pool reuses a bounded set of connections. Borrowing can itself queue, so include acquisition in the deadline. A leak or oversized pool can exhaust both application workers and database capacity. Keepalive tests a transport path under particular conditions, not business correctness. In an assumed pool of twenty connections with 50 ms mean hold time, the ideal busy-capacity bound is 400 operations/s before overhead and variance; real safe load requires measurement.

← Return to question

UNIT 03 / D1

Design or debug the running system: Implement a cold/warm request latency calculator with separate DNS, connection, TLS, application, and response stages.

Use the stated 20+40+40+55 = 155 ms cold example, then remove only stages actually reused. Do not subtract server execution because the connection is warm. Reject a negative stage time and keep the result labeled an assumed model, not a measured protocol guarantee. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 03 / I1

Implement a cold/warm request latency calculator with separate DNS, connection, TLS, application, and response stages. Include a counterexample and a corrected result.

Use the stated 20+40+40+55 = 155 ms cold example, then remove only stages actually reused. Do not subtract server execution because the connection is warm. Reject a negative stage time and keep the result labeled an assumed model, not a measured protocol guarantee.

← Return to question

UNIT 03 / T1

What changes when a supposedly local service call becomes a cross-region network dependency?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. A connection pool reuses a bounded set of connections. Borrowing can itself queue, so include acquisition in the deadline. A leak or oversized pool can exhaust both application workers and database capacity. Keepalive tests a transport path under particular conditions, not business correctness. In an assumed pool of twenty connections with 50 ms mean hold time, the ideal busy-capacity bound is 400 operations/s before overhead and variance; real safe load requires measurement. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 4 · Runtime, Processes, Threads, and Concurrency

UNIT 04 / C1

How do process and thread boundaries differ from source-module boundaries?

A program is code; a process is an executing instance with an address space and resources; threads are execution contexts within a process. Threads commonly share heap memory while maintaining separate stacks and scheduling state. A process crash can terminate every thread regardless of source-module boundaries. Use processes when isolation or separate resource ownership matters; use threads when shared-address-space execution is appropriate. Neither choice alone makes shared application state race-free.

← Return to question

UNIT 04 / C2

Why can an async function still block other requests?

Synchronous execution ties the caller's progress to completing an operation; asynchronous execution separates initiation from later completion. Blocking describes whether the executing thread waits; the two distinctions are related but not identical. An async function that calls a blocking filesystem operation can still stall an event-loop thread. Specify the entire path, including library and driver behavior. Prefer the simplest model that keeps deadlines and resource use bounded under the expected workload.

← Return to question

UNIT 04 / C3

Does concurrency require simultaneous execution on several cores?

Concurrency means activities can be in progress over overlapping periods; parallelism means execution actually occurs simultaneously. One core can interleave several tasks without running their instructions at the same instant. Concurrent code still has races because interleaving can occur between read and write operations. Parallelism adds hardware execution opportunities but not automatic correctness. Decide independently how much work may be in progress and how many resources may execute it at once.

← Return to question

UNIT 04 / C4

Why can individually atomic fields fail to protect a multi-field invariant?

A mutex grants exclusive access; a semaphore limits concurrent permits; an atomic operation provides defined indivisibility and ordering for specified operations; a condition variable coordinates waiting for a predicate while releasing a lock. Recheck the predicate in a loop because wakeups do not guarantee it holds. Protect related values together if the invariant spans them. Replacing every field with an atomic does not automatically make a multi-field account transfer atomic.

← Return to question

UNIT 04 / C5

Why does wall-clock order not establish memory visibility?

Visibility is not established by intuition about elapsed time. A writer's update and another thread's read need the ordering guarantees defined by the language and synchronization primitives. Happens-before is a formal relation used to reason about such guarantees, not a statement that every earlier wall-clock event is visible. Publish an immutable object through a synchronized channel or appropriate atomic reference. The Go memory model is one concrete specification; other languages must be checked separately.

← Return to question

UNIT 04 / S1

A and B both read counter zero, then each writes one. Why is one increment missing?

A read-modify-write sequence can lose updates when its steps interleave. Starting at zero, A reads zero, B reads zero, A writes one, and B writes one: two increments produced one. Protect the entire invariant, not only the final assignment. A critical section may use a lock or an appropriate atomic operation; separate process or database writers require coordination at their own shared boundary. A local mutex cannot protect another process's memory.

← Return to question

UNIT 04 / S2

A two-second callback runs on one event loop. What happens to other ready tasks?

An event loop runs ready callbacks or task continuations. A future or promise represents a result that may become available later; awaiting it registers a continuation or suspends a task according to the runtime. A callback that computes for two seconds can delay every other task on a single loop. Keep each turn bounded and make cancellation visible. Futures do not guarantee that work is durable, isolated, or successfully executed merely because it was scheduled.

← Return to question

UNIT 04 / S3

Arrivals stay at 150 jobs/s while service completes 100/s. Does a larger queue solve the problem?

Unlimited concurrency converts downstream slowness into memory and connection exhaustion. Backpressure bounds or slows producers according to available capacity. For an assumed worker rate of 100 jobs/s and arrivals of 150 jobs/s, a queue grows by fifty jobs/s until a limit is reached. A larger queue delays rejection but does not fix the rate mismatch. Prefer bounded queues, admission deadlines, compact payloads, and explicit overload responses; monitor queue age as well as length.

← Return to question

UNIT 04 / D1

Design or debug the running system: Implement the lost-counter interleaving and a bounded upload work queue.

Show A-read-zero, B-read-zero, A-write-one, B-write-one before repairing the critical section. Bound queued payload references and active workers separately. For I/O waiting, compare concurrency; for CPU work, account for runtime parallelism. An async keyword alone does not repair the race or add CPU capacity. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 04 / I1

Implement the lost-counter interleaving and a bounded upload work queue. Include a counterexample and a corrected result.

Show A-read-zero, B-read-zero, A-write-one, B-write-one before repairing the critical section. Bound queued payload references and active workers separately. For I/O waiting, compare concurrency; for CPU work, account for runtime parallelism. An async keyword alone does not repair the race or add CPU capacity.

← Return to question

UNIT 04 / T1

What should happen to accepted uploads when the request process is cancelled but the business still requires processing?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Cancellation must propagate to owned child work and release resources. Structured concurrency ties child-task lifetimes to a parent scope, so failures and completion have an explicit owner. Cancelling a caller does not prove an external side effect was undone: a payment may already have committed. Use finally-style cleanup for local resources and reconciliation for uncertain remote effects. Detached tasks need a durable owner when their business outcome must survive the request process. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 5 · Object-Oriented and Modular Software Design

UNIT 05 / C1

When does composition avoid a subclass explosion?

A subclass can violate assumptions inherited by callers: a ReadOnlyAccount subclass that throws from withdraw is not substitutable for a contract promising withdrawal. Composition combines collaborators through explicit contracts instead. A payment service receiving a fraud policy and a gateway can vary them independently without a subclass for every combination. Composition introduces wiring and interface design; inheritance remains useful when the behavioral relationship is genuinely stable, rather than merely when two classes share fields.

← Return to question

UNIT 05 / C2

Which payment semantics must a gateway contract state beyond its method signature?

A gateway interface must specify currency, amount units, timeouts, duplicate identity, and error meaning. charge(100) is ambiguous: one currency unit, one hundred, or one hundred minor units? Return a structured outcome such as approved, declined, or uncertain with a provider reference. A transport exception after submission is not proof of decline. Behavioral contracts let adapters be replaced without silently changing business semantics, but require integration tests against real provider behavior.

← Return to question

UNIT 05 / C3

Why do SOLID heuristics not require an interface for every class?

SOLID is a set of design heuristics: single responsibility, open/closed extension, behavioral substitution, focused interfaces, and dependency inversion. A payment service can depend on a narrow charge contract while its adapter depends on an SDK. These ideas do not require one interface per class or forbid editing existing code. An extension mechanism with twenty configuration switches may be harder to understand than a deliberate edit. Judge each heuristic against the actual change and testing cost.

← Return to question

UNIT 05 / C4

How do payment identity and a Money value differ?

An entity has identity across changes; a value object is defined by its value; a domain model expresses meaningful rules. Payment 42 remains the same entity after status changes. Money(USD,100) is a value that should reject incompatible arithmetic or apply an explicit conversion policy. A domain service can coordinate a rule that belongs to no single entity. Avoid an anemic model that scatters every invariant across controllers, while keeping trivial data transport simple.

← Return to question

UNIT 05 / C5

How can a functional design keep effects and pure decisions separate?

A functional design can represent payment decisions as pure transformations of values, with effects interpreted at the boundary. decide(state, command) may yield events and requested external actions; an interpreter commits events or calls a gateway. This reduces hidden mutation and makes replaying logic easier. It still needs ownership, durable execution, and effect deduplication. Objects and functions are complementary tools; choose the representation that makes invariants and collaboration clearest in the language and team context.

← Return to question

UNIT 05 / S1

Two callers read and overwrite an account balance. Why should the state owner receive an intent?

Tell, Don't Ask encourages sending an intention to the state owner. Rather than reading account.balance, comparing it outside, and writing it back, request reserve(amount) so the account can enforce its invariant. Reading a value for a report remains appropriate. The principle does not authorize hiding surprising side effects behind getters. The owner must still coordinate concurrent changes at the true shared-state boundary, such as an atomic database transaction.

← Return to question

UNIT 05 / S2

A mock proves every private method was called. What important behavior can still be untested?

Testability follows controllable inputs, visible outcomes, and explicit effects. Inject a fake gateway to test declined-payment behavior without network calls, then test the real adapter against a contract environment. A mock expectation that asserts every internal method call can make refactoring expensive while proving little business behavior. Prefer invariant and outcome tests, and retain integration coverage for transactions, serialization, timeouts, and resource cleanup that a fake cannot faithfully establish.

← Return to question

UNIT 05 / S3

A refactor changes provider timeout into decline. Why is that not merely a structural improvement?

Refactoring preserves intended behavior while improving structure. Start by characterizing the tangled payment class: valid charge, decline, provider timeout, and persistence failure. Then extract parsing, fee calculation, gateway calls, and storage incrementally. A long method, duplicated conditional, or feature envy is evidence to investigate, not proof that a design pattern is required. Keep tests focused on observable obligations so moving code does not change payment semantics accidentally.

← Return to question

UNIT 05 / D1

Design or debug the running system: Refactor a hardcoded payment gateway into an injected contract and a recording test implementation.

Test approved, declined, and uncertain outcomes without real charges. Keep integer units and currency explicit. A test fake validates orchestration only; real serialization, timeouts, and persistence need separate integration evidence. Do not map a timeout to decline simply to make a test pass. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 05 / I1

Refactor a hardcoded payment gateway into an injected contract and a recording test implementation. Include a counterexample and a corrected result.

Test approved, declined, and uncertain outcomes without real charges. Keep integer units and currency explicit. A test fake validates orchestration only; real serialization, timeouts, and persistence need separate integration evidence. Do not map a timeout to decline simply to make a test pass.

← Return to question

UNIT 05 / T1

Which payment abstractions remain useful if the implementation moves from objects to pure functions?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. A functional design can represent payment decisions as pure transformations of values, with effects interpreted at the boundary. decide(state, command) may yield events and requested external actions; an interpreter commits events or calls a gateway. This reduces hidden mutation and makes replaying logic easier. It still needs ownership, durable execution, and effect deduplication. Objects and functions are complementary tools; choose the representation that makes invariants and collaboration clearest in the language and team context. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 6 · Design Patterns, Dependency Injection, and Object Lifecycles

UNIT 06 / C1

Can constructor injection exist without dependency inversion?

Dependency inversion is a design principle about which knowledge depends on which abstraction. DI is a construction technique. A service can receive a concrete SDK through its constructor and still violate the intended abstraction boundary; conversely, a manually wired function can depend on a stable protocol without using a container. Review provider-specific errors, schemas, and resource handles crossing the boundary. Syntax alone cannot prove that high-level policy has been isolated from volatile details.

← Return to question

UNIT 06 / C2

Is a container singleton one instance across all hosts?

Transient means a resolution creates a new instance; scoped means instances are reused within a defined scope; singleton means reuse within the container or application boundary specified by the framework. None implies global uniqueness across processes. If two request scopes each resolve a scoped unit of work twice, they see identities A,A and B,B. A singleton shared by many requests must meet its concurrency contract. Framework names and disposal rules differ and must be checked.

← Return to question

UNIT 06 / C3

Why are resource creation and disposal ownership different contracts?

Creation, reference, and ownership are different relationships. A factory may create a connection while the caller owns disposal; a container may create and retain it until scope end. Two components holding a reference must not both assume exclusive closing rights. Write ownership into the contract. Returning a lazily evaluated query from a disposed repository is a common lifetime bug even when the return type matches; materialize bounded data or transfer an explicit streaming owner.

← Return to question

UNIT 06 / C4

Why can injecting a container preserve the service-locator problem?

DI makes collaborators visible in construction or function parameters. A service locator lets code request them from a registry, hiding dependencies behind runtime lookups. Locators can be appropriate at a deliberate integration boundary, but application code that reaches into a global locator is harder to reason about and test. Avoid replacing constructor arguments with one injected container; that often preserves the locator pattern under a different name. Resolve dependencies near the composition boundary.

← Return to question

UNIT 06 / C5

How can DI and a factory cooperate without being the same pattern?

A factory encapsulates creation or selection; DI supplies an already selected collaborator or a factory itself. A notification service receiving sender_factory can choose a channel at runtime without importing concrete SDKs. The factory's returned lifetime must be clear: is each sender owned by the caller, shared, or scoped? Do not dispose a shared singleton as though every factory call created a private object. These patterns answer different questions and frequently work together.

← Return to question

UNIT 06 / S1

A singleton stores a request-scoped unit of work. What fails after the request ends?

A dependency cycle A→B→A often signals mixed responsibilities or missing coordination; breaking it with a service locator can hide rather than solve the model. A captive dependency occurs when a long-lived owner retains a shorter-lived collaborator, such as a singleton keeping a request's unit of work. Refactor the operation to receive a valid scope or narrow factory. The fix must preserve ownership and authorization context rather than merely satisfy the container's graph validator.

← Return to question

UNIT 06 / S2

Two first callers trigger lazy initialization. What must coordinate them?

Lazy initialization defers creation until the resource is needed. This can avoid unused work, but moves latency and failure to the first caller and introduces a concurrent initialization race. Define whether failed initialization is retried, cached, or surfaced permanently. A lock or runtime lazy primitive must protect publication as required. Warming essential dependencies at startup may provide clearer readiness behavior when accepting traffic without them would immediately fail.

← Return to question

UNIT 06 / S3

A failed request leaks one connection. Why can healthy traffic eventually stop?

Cleanup must run on success, failure, and cancellation. Use structured scopes or finally blocks to return pool permits, close streams, and roll back unfinished transactions. Garbage collection is not a timely release guarantee for external resources. Cleanup can itself fail, so preserve the primary failure while recording cleanup problems. Resource lifetimes affect capacity: leaking one database connection per failed request can exhaust a pool even if the healthy request path is fast.

← Return to question

UNIT 06 / D1

Design or debug the running system: Build a small resolver that demonstrates transient, scoped, and singleton identities across two request scopes.

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 06 / I1

Build a small resolver that demonstrates transient, scoped, and singleton identities across two request scopes. Include a counterexample and a corrected result.

Within scope one, a scoped resolution returns A twice; scope two returns B twice. Transients return new identities; a process-owned singleton returns S in both scopes. Add a disposable resource and prove scope end cleans it up. Reject a singleton capturing a request-scoped disposable rather than hiding the bug in a locator.

← Return to question

UNIT 06 / T1

How would the ownership rules change if a job outlives both a request scope and a user session?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Repository mediates domain-oriented access to persistent data; Unit of Work groups changes under one commit decision; Specification represents a reusable predicate; CQRS separates command and query models where their needs justify it. A repository should not pretend remote calls and local transactions share identical failure semantics. A Unit of Work must own its transaction and cleanup. CQRS can be implemented inside one process and one database; event sourcing and independent services are optional additional choices. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 7 · Application Architecture and Domain-Driven Design

UNIT 07 / C1

What makes a monolith modular even though it has one deployment?

A modular monolith is deployed as a unit while its modules maintain explicit boundaries and ownership. Orders, catalog, and payments can have separate interfaces and schema ownership in one application. This keeps local calls and coordinated deployment simple while retaining future extraction options. Shared tables writable by every module defeat much of the benefit. Enforce module dependencies in code review or automated checks, and make cross-module operations pass through agreed contracts.

← Return to question

UNIT 07 / C2

Why do source-dependency arrows differ from runtime call paths?

Clean Architecture places business policy inside dependencies on application-owned abstractions; framework and storage details depend inward. The drawing's circles describe source dependency rules, not the physical path of runtime messages. A use case might call an injected repository implemented by an outer adapter. Avoid leaking ORM entities or HTTP response types into core policy. The cost is extra mapping and contracts; a small CRUD tool may need only a modest version of the same separation.

← Return to question

UNIT 07 / C3

Why may catalog Product and fulfillment Item be different legitimate models?

A bounded context defines where a particular model and vocabulary are valid. The catalog's Product can emphasize description and merchandising; fulfillment's Item can emphasize a physical unit and location. Translating between them is deliberate, not evidence that one universal Product schema is missing. Ubiquitous language means consistent domain language inside a context, not forcing every team to use one model. Map interactions and ownership before deciding whether a context is a separate process.

← Return to question

UNIT 07 / C4

How should an aggregate boundary follow a consistency invariant?

An aggregate is a consistency boundary whose root controls changes to its members. An Order might protect the relationship between line items and total amount within a transaction. Entities carry identity; value objects represent interchangeable values; domain services express rules spanning concepts. A huge aggregate containing the entire shop creates contention and awkward loading. Keep invariants that must change atomically together, and handle cross-aggregate coordination explicitly rather than assuming object references imply distributed transactions.

← Return to question

UNIT 07 / C5

What should an architecture fitness check preserve?

Evolutionary architecture changes in response to evidence while protecting important properties. Add fitness checks for module boundaries, migration compatibility, latency budgets, or tenant isolation so frequent changes do not silently erode them. A check must express a useful property rather than freeze incidental implementation. An ADR records why a boundary exists and what observation should trigger reconsideration. Evolution is not constant rewriting; many successful changes remove complexity rather than introduce another platform.

← Return to question

UNIT 07 / S1

An order commits and then the process crashes before event publication. Why is an in-memory event insufficient?

A domain event records a meaningful fact such as OrderPlaced, not a vague instruction to “do something.” Include stable identity, time semantics, and enough versioned information for intended consumers without exposing every internal field. Recording an event in memory is not reliable publication. If state and publication intent must agree, persist them atomically through an outbox or another justified mechanism. Consumers must still handle duplicates, delayed delivery, and compatible schema evolution.

← Return to question

UNIT 07 / S2

Five services share writable tables and must always release together. What independence is missing?

A distributed monolith has several processes but little independent change or operation. Symptoms include synchronous chains for every action, shared writable schemas, coordinated releases, and failures cascading across services. Moving calls onto a network without clarifying ownership makes debugging harder and adds partial failure. Consolidating unstable boundaries into a modular deployment can be an improvement. Alternatively, redesign contracts, data ownership, and asynchronous coordination before claiming independent services have been achieved.

← Return to question

UNIT 07 / S3

A small team with a rapidly changing model considers microservices. What evidence would justify extraction?

Avoid microservices when the domain is still changing rapidly, the team cannot operate independent services safely, or scale and isolation do not require them. A modular monolith with measured bottlenecks and tested backups can be a stronger starting point. This is not a claim that microservices are inherently wrong: stable capabilities with distinct traffic and ownership can justify them. Evaluate the complete cost of transactions, deployment, telemetry, security, and incident response before extraction.

← Return to question

UNIT 07 / D1

Design or debug the running system: Divide shopping into catalog, orders, and payments modules and implement one boundary test.

Let orders request a payment through a contract; disallow direct imports of the payment storage adapter or writes to its private table. A test should enforce a meaningful dependency rule rather than every incidental class name. Show how a provider adapter can change while the order rule remains stable. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 07 / I1

Divide shopping into catalog, orders, and payments modules and implement one boundary test. Include a counterexample and a corrected result.

Let orders request a payment through a contract; disallow direct imports of the payment storage adapter or writes to its private table. A test should enforce a meaningful dependency rule rather than every incidental class name. Show how a provider adapter can change while the order rule remains stable.

← Return to question

UNIT 07 / T1

Which evidence would justify extracting payments from a modular monolith while retaining the same business invariants?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Avoid microservices when the domain is still changing rapidly, the team cannot operate independent services safely, or scale and isolation do not require them. A modular monolith with measured bottlenecks and tested backups can be a stronger starting point. This is not a claim that microservices are inherently wrong: stable capabilities with distinct traffic and ownership can justify them. Evaluate the complete cost of transactions, deployment, telemetry, security, and incident response before extraction. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 8 · API Design and Communication Contracts

UNIT 08 / C1

Why should a refund have a separately tracked identity and lifecycle?

Resources should reflect stable business identities. POST /payments creates an operation; GET /payments/{id} reports its state; POST /payments/{id}/refunds creates a separately tracked refund. Hiding every action under one generic /execute endpoint loses useful semantics, but forcing every domain action into CRUD can also obscure behavior. A refund resource needs its own amount, authorization, idempotency identity, and lifecycle. Use ownership and state transitions to guide the model.

← Return to question

UNIT 08 / C2

Which offset-pagination anomaly does a keyset cursor reduce, and what does it not guarantee?

Offset pagination skips a number of rows and is easy to express, but inserts before the offset can create duplicates or omissions between pages. Keyset cursors encode the last ordered key, such as (created_at,id), to continue after that position. They avoid some skip cost and shift anomalies but still need a consistency contract for changing rows. Sign or validate opaque cursors and bind them to the applicable filters and tenant context.

← Return to question

UNIT 08 / C3

Which records and atomic boundary make an idempotency key meaningful?

An idempotency key identifies one logical operation across attempts. Store its owner, request fingerprint, execution state, and committed result under a uniqueness and atomicity rule. The same key with a different payload must conflict. If the provider may have charged before a timeout, use its stable operation key or reconcile before resubmitting. Retention and key reuse rules matter. Local deduplication does not provide exactly-once external execution across an uncooperative provider boundary.

← Return to question

UNIT 08 / C4

Why is authentication insufficient for GET /payments/42?

Authentication establishes the caller's identity under a trust model; authorization decides whether that caller may perform this operation on this resource. Derive tenant context from authenticated membership, not a freely supplied tenant_id alone. GET /payments/42 must check resource ownership even if payment identifiers are hard to guess. Coarse gateway checks complement, rather than replace, domain authorization. Validate service-to-service credentials as well as user credentials when requests cross internal boundaries.

← Return to question

UNIT 08 / C5

Why can event schema evolution break recovery long after a release?

Schema evolution applies to stored data and events as well as synchronous APIs. Preserve meanings, defaults, field identity, and unknown-value behavior. A consumer replaying a year of events may encounter every historical version, so deleting a decoder can break recovery even if current traffic looks healthy. Use explicit versions or upcasters with tested historical samples. An additive field is not harmless if it changes a signature calculation, a uniqueness rule, or a required business decision.

← Return to question

UNIT 08 / S1

A POST returns 201 with a pending payment resource. Does that mean the provider captured money?

HTTP status codes describe the protocol outcome, while an operation body may describe asynchronous business state. 201 can identify a created payment resource that is still pending; 202 means accepted for processing, not financially approved. A timeout has no definitive business meaning. Use stable application codes for validation, conflict, authorization, and retry advice. Do not translate every provider error into a generic 500 if clients need to distinguish a correctable request from an uncertain submitted operation.

← Return to question

UNIT 08 / S2

Three attempts occur at each of four layers. What risk does this create?

Retries should fit an overall deadline and target transient, safe-to-repeat work. Budget time for connection acquisition, attempts, and backoff; propagate remaining time downstream. Exponential backoff with jitter reduces synchronized retries but does not create capacity. Avoid retries independently at every layer: three attempts at each of four layers can amplify one intent into 81 attempts. For side effects, retries need stable identity and reconciliation of uncertain outcomes.

← Return to question

UNIT 08 / S3

A webhook receiver commits its effect but loses the acknowledgment. What must happen on redelivery?

A webhook is an asynchronous request to another party. Sign it, include a stable event identifier and timestamp policy, and accept acknowledgments according to a documented delivery contract. A receiver should commit its inbox or effect before acknowledging if persistence is required. A sender can retry a lost acknowledgment, so consumers must deduplicate. A 2xx response proves only what the receiving contract promises, not that every downstream business task has completed.

← Return to question

UNIT 08 / D1

Design or debug the running system: Implement keyset pagination and a durable idempotency-key state machine over a synthetic payment operation.

Use keyset_page() for ordered pairs and test insertion before a page boundary. For idempotency, bind actor, key, fingerprint, state, and result atomically; reject a mismatched payload. A plain dictionary checked before a separate insert does not protect concurrent requests or restart recovery. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 08 / I1

Implement keyset pagination and a durable idempotency-key state machine over a synthetic payment operation. Include a counterexample and a corrected result.

Use keyset_page() for ordered pairs and test insertion before a page boundary. For idempotency, bind actor, key, fingerprint, state, and result atomically; reject a mismatched payload. A plain dictionary checked before a separate insert does not protect concurrent requests or restart recovery.

← Return to question

UNIT 08 / T1

Can a public API offer a permanent idempotency promise with finite storage? Define a defensible contract.

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Documentation should explain complete operation histories: request examples, errors, authorization, pagination, timeouts, idempotency, and asynchronous status. Machine-readable schemas help generate clients, while prose defines semantics those schemas cannot express. A runnable sandbox must distinguish simulated effects from real charges. Track documentation versions with implementation, and test key examples. Discoverability is practical: users should know how to find the right operation, determine its scope, and recover when the happy path fails. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 9 · Data Models and Storage Technologies

UNIT 09 / C1

Which storage promises must be stated before choosing a database family?

Storage choices begin with promises: what is accepted, when it is committed, how it can be read, and which failures it survives. A chat history, an audit ledger, and a thumbnail cache need different retention and correctness contracts. Name the authoritative copy and recovery procedure. A database label alone does not establish transactions, isolation, backups, or acknowledgment durability; these depend on configuration and usage. Choose by access patterns and invariants before comparing product features.

← Return to question

UNIT 09 / C2

Why is a vector embedding not authoritative evidence or permission?

A vector index supports similarity search over numerical embeddings. It typically combines a vector representation, an approximate candidate structure, and metadata filtering. Candidate recall, update cost, memory, distance function, and authorization affect the architecture. An embedding is a derived representation, not an authoritative replacement for a document. Preserve source versions and rebuild paths; approximate retrieval can miss relevant items, and a vector score does not prove a factual answer or permission to expose the source.

← Return to question

UNIT 09 / C3

When is duplicating an order address a legitimate historical fact?

Normalization avoids redundant facts and update anomalies; denormalization deliberately stores derived or repeated data to serve a read path. Store a customer's address once for their current profile, but retain the historical shipping address on a completed order as part of that order's fact. Not all duplication is an error. When a feed stores a cached author name, define whether it is a snapshot or a copy to maintain; ambiguity creates stale-data bugs.

← Return to question

UNIT 09 / C4

Why is data ownership more than choosing who creates a table?

Data ownership names who may define and change a fact. Payments owns financial outcome records; analytics consumes a derived copy. If several services directly update one table, transaction behavior and release coupling become shared even if their code repositories are separate. Publish contracts for reading and changing owned data, and give consumers the least privilege required. Ownership includes retention, incident response, schema evolution, and recovery, not only the right to create a table.

← Return to question

UNIT 09 / C5

What operating responsibilities are added by a second storage engine?

Polyglot persistence gives different workloads suitable engines at the cost of more operating boundaries. A shop might add search only after relational queries fail its relevance or latency needs. It then must track indexing lag, rebuilds, schemas, permissions, and recovery across both stores. Cross-store operations are not one transaction by default. Compare the added capability with total ownership cost, and keep authoritative data and derived representations distinguishable in every diagram and runbook.

← Return to question

UNIT 09 / S1

One huge conversation dominates a wide-column partition. Why do more cold partitions not solve it?

Wide-column systems organize data around partition keys and ordered clustering dimensions in common implementations. A conversation partition with messages ordered by sequence can support bounded history reads. Partition size and hot-key traffic must be planned: one enormous room can concentrate load despite a large cluster. Model queries first, duplicate data deliberately for other access patterns, and document which constraints or joins the application must enforce. The category is not a universal guarantee about every implementation.

← Return to question

UNIT 09 / S2

Search still contains a deleted private document. Which serving and lifecycle obligations remain?

A search index stores representations optimized for retrieval rather than primary transactional ownership. An inverted index maps terms to postings; document scores and filters produce ranked results. Updates commonly propagate asynchronously from an authoritative store, so search may lag or temporarily retain deleted content. Authorization filtering must remain correct despite that lag. Keep enough source identity and version information to rebuild the index and reject stale updates; treating the index as the only copy makes recovery fragile.

← Return to question

UNIT 09 / S3

An old backup is restored after deletion requests. What must prevent privacy resurrection?

Retention is an explicit lifecycle, not a scheduled delete statement added at the end. Decide legal and business purpose, expiration, archival, backup behavior, and derived-copy deletion. Dropping old partitions may be efficient for time-bounded data, while per-user deletion needs identity-aware propagation. A tombstone may prevent reappearance during replay but still retain personal data; minimize it appropriately. Verify that recovery from an old backup reapplies required deletion decisions instead of resurrecting removed records.

← Return to question

UNIT 09 / D1

Design or debug the running system: Model profiles, transactions, messages, logs, and PDFs with their access patterns and ownership.

Justify each choice through point reads, ranges, invariants, retention, and byte size. Do not choose five engines merely to cover a taxonomy. Separate PDF bytes from metadata and preserve recoverable references; explain which derived stores can be rebuilt and which facts must remain authoritative. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 09 / I1

Model profiles, transactions, messages, logs, and PDFs with their access patterns and ownership. Include a counterexample and a corrected result.

Justify each choice through point reads, ranges, invariants, retention, and byte size. Do not choose five engines merely to cover a taxonomy. Separate PDF bytes from metadata and preserve recoverable references; explain which derived stores can be rebuilt and which facts must remain authoritative.

← Return to question

UNIT 09 / T1

How does restoration from a historical backup interact with current privacy deletion and derived indexes?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Polyglot persistence gives different workloads suitable engines at the cost of more operating boundaries. A shop might add search only after relational queries fail its relevance or latency needs. It then must track indexing lag, rebuilds, schemas, permissions, and recovery across both stores. Cross-store operations are not one transaction by default. Compare the added capability with total ownership cost, and keep authoritative data and derived representations distinguishable in every diagram and runbook. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 10 · Database Internals, Transactions, and Concurrency Control

UNIT 10 / C1

Why do ordered range queries and equality lookup motivate different index choices?

B-tree families maintain ordered keys with bounded-height traversal; B+tree designs commonly keep data or record references in leaves linked for range scans. Hash indexing suits equality under its implementation but does not provide ordered range traversal. For keys [10,20,30,40], a range 20–35 benefits from an ordered path and neighboring leaves. Index choice depends on predicates, ordering, maintenance cost, and engine support. “An index is O(log n)” hides page I/O and result-size work.

← Return to question

UNIT 10 / C2

Why does ACID not infer every business invariant?

ACID separates atomicity, consistency, isolation, and durability. Atomicity prevents partial transaction outcomes; consistency means the transaction preserves declared invariants when the application and constraints are correct; isolation governs concurrent observations; durability protects committed state under configuration and failure assumptions. The C is not a promise that all business rules are automatically inferred. A transaction that atomically transfers the wrong amount can satisfy engine mechanics while violating the intended business rule.

← Return to question

UNIT 10 / C3

Why is MVCC not a synonym for serializable behavior?

MVCC maintains versions so readers can observe a defined snapshot while writers create new versions. Visibility depends on transaction rules, not simply the newest timestamp. A long-running reader can retain old versions and delay cleanup. Snapshot reads do not inherently prevent write skew: two transactions may read the same invariant and update different rows. Understand the engine's isolation level and conflict detection rather than treating “uses MVCC” as synonymous with serializable execution.

← Return to question

UNIT 10 / C4

Why must isolation claims name the engine and statement behavior?

Isolation levels specify allowed concurrency behavior, but engines implement them differently. PostgreSQL 18 Read Committed uses statement-level snapshots for ordinary reads; its Repeatable Read provides a transaction snapshot yet can permit serialization anomalies; Serializable adds conflict handling that can abort transactions. Other engines and statement forms differ. Applications must handle retryable failures by restarting the whole affected transaction with bounded attempts. Do not retry only the last statement while retaining assumptions from an invalidated snapshot.

← Return to question

UNIT 10 / C5

How does serializability differ from real-time linearizability?

Serializable means the committed effect is equivalent to some serial transaction order, not necessarily wall-clock order. Linearizability is a distinct real-time single-operation property; strict serializability combines a transactional serial order with real-time constraints. A transaction may need retry after serialization failure. Keep external side effects out of blindly replayed transaction bodies or protect them with stable identity. Serialization makes concurrency reasoning stronger but does not validate incorrect business logic or solve every availability requirement.

← Return to question

UNIT 10 / S1

Two on-call doctors each see the other and both go off duty. Which invariant did row-level thinking miss?

Lost updates overwrite another writer's result; write skew violates a multi-row invariant after transactions update different rows; serialization anomalies are histories without an equivalent serial ordering. If two doctors each see the other on call and both mark themselves off, a snapshot can preserve each read yet violate “at least one on call.” Fix the invariant with appropriate serialization, locking, or a redesigned shared conflict point. Protecting only each individual row is insufficient.

← Return to question

UNIT 10 / S2

Two buyers update from expected seat version seven. What does zero affected rows mean?

Pessimistic locking coordinates access before conflicting work; optimistic concurrency validates that assumptions still hold before committing. A versioned update such as UPDATE seat SET owner=A,version=8 WHERE id=7 AND version=7 succeeds only if its expected version remains current. Zero affected rows means conflict, not success. Retrying must re-read and reconsider business conditions. Locks can wait or deadlock, while optimistic retries can waste work under contention; choose based on conflict rate and transaction cost.

← Return to question

UNIT 10 / S3

The backup job succeeded but no restore has run. What evidence is missing?

Backups provide recoverable historical state; replication keeps live copies and can replicate mistakes. Test restores into a usable environment and verify application invariants, credentials, schema versions, and recovery time. A backup job reporting success does not prove that its archive is complete or decryptable. Specify point-in-time recovery requirements and how logs are retained. Durability is conditional on acknowledged persistence and independent failure protection, not established by merely owning a backup file.

← Return to question

UNIT 10 / D1

Design or debug the running system: Run the last-seat claim using a conditional database update, then test conflicting writers.

Use claim_seat() against the same store and assert one owner wins while the other receives conflict. Explain which local transaction protects the invariant. Add rollback and crash histories; do not claim a local seat claim makes a remote payment part of the same transaction. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 10 / I1

Run the last-seat claim using a conditional database update, then test conflicting writers. Include a counterexample and a corrected result.

Use claim_seat() against the same store and assert one owner wins while the other receives conflict. Explain which local transaction protects the invariant. Add rollback and crash histories; do not claim a local seat claim makes a remote payment part of the same transaction.

← Return to question

UNIT 10 / T1

Which invariant becomes harder when a local multi-statement transaction is split across independently owned databases?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Backups provide recoverable historical state; replication keeps live copies and can replicate mistakes. Test restores into a usable environment and verify application invariants, credentials, schema versions, and recovery time. A backup job reporting success does not prove that its archive is complete or decryptable. Specify point-in-time recovery requirements and how logs are retained. Durability is conditional on acknowledged persistence and independent failure protection, not established by merely owning a backup file. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 11 · Caching and Content Delivery

UNIT 11 / C1

Why does working-set size matter more than historical object count for cache usefulness?

Temporal locality is reuse over time; spatial locality is reuse of nearby data. The working set is the data actively needed over a relevant interval. A cache too small for that set may churn even if total historical data is enormous. Scan traffic can evict useful entries under some policies. Measure bytes, object sizes, skew, and admission behavior, not only key count. Caching every object eagerly can waste memory on cold data while crowding out hot content.

← Return to question

UNIT 11 / C2

How can cache-aside repopulate stale data after invalidation?

Cache-aside loads after a miss; read-through delegates loading to the cache interface; write-through updates a backing store in the write path; write-behind defers backing persistence. Name the actual ordering and failure semantics because implementations vary. In write-behind, acknowledging only a volatile cache can lose accepted changes. Cache-aside still has races: a delayed reader can populate an old value after a newer write invalidated the key. Use versioning or stronger coordination when freshness requires it.

← Return to question

UNIT 11 / C3

Why is a TTL not always a bound on source-data age?

A TTL bounds intended residence after insertion, not necessarily the age of the underlying fact when inserted. A cached value already five minutes old with a one-minute TTL can remain six minutes behind its source. Expiration is distinct from eviction, and clients may retain their own copies. Use jitter to avoid synchronized expirations. Choose TTL from tolerated staleness and refresh behavior; correctness-critical actions should validate authoritative state rather than trust a display cache.

← Return to question

UNIT 11 / C4

How do LRU and FIFO differ for A,B,C,A,D at capacity three?

LRU favors recent access; LFU favors frequent access; FIFO evicts insertion order. Their precise implementations may be approximate. For capacity three and accesses A,B,C,A,D, ideal LRU evicts B; FIFO evicts A; a frequency-based policy also needs tie-breaking. Compare policies with representative traffic and object sizes. A clever eviction algorithm cannot fix a working set vastly beyond memory or a cache that stores low-value objects indiscriminately. Admission and eviction are separate choices.

← Return to question

UNIT 11 / C5

When can removing a cache be the better design?

Caching is harmful when maintenance and correctness cost exceed saved work. A low-reuse result cache may consume memory and serialization time without many hits. A cache with missing authorization scope can leak data; one that masks a slow database during normal operation can hide an origin capacity deficit until outage. Test the system with the cache empty or unavailable. Prefer query/index improvements or simpler data access when they address the bottleneck with fewer state copies.

← Return to question

UNIT 11 / S1

A cache restart sends every reader to origin. Which mechanisms limit the resulting storm?

A stampede occurs when many misses trigger duplicate expensive work. Penetration repeatedly requests absent keys; an avalanche is a broad synchronized loss or expiry of cached entries. Use single-flight loading, bounded refresh workers, negative caching, TTL jitter, and origin admission limits as appropriate. Locks used for refresh also need expiration and failure handling. A cache outage should not let every application instance independently overwhelm the database while attempting to reconstruct the entire working set.

← Return to question

UNIT 11 / S2

A not-found result is cached just before object creation. What freshness behavior must be defined?

Negative caching stores an absence or error under a narrow policy. Caching “product not found” for a short interval can protect against repeated nonexistent lookups. Do not cache a temporary database failure as permanent absence, and do not let one tenant's authorization failure poison another tenant's key. A newly created product may remain hidden until negative-entry expiry or invalidation. Use separate error types, short appropriate lifetimes, and correctly scoped keys.

← Return to question

UNIT 11 / S3

A stale value keeps being served while refresh fails. What boundary prevents unbounded stale operation?

Stale-while-revalidate serves an allowed stale value while one refresh updates it. This trades freshness for latency and origin protection; it is not suitable for every operation. A cached description can be stale, while payment authorization cannot be inferred from an old response. Define fresh, stale-allowed, and unusable age limits. If refresh repeatedly fails, do not extend stale serving forever unless that is an explicit degradation contract visible to users and operators.

← Return to question

UNIT 11 / D1

Design or debug the running system: Run the LRU fixture and reproduce a stale cache-aside population race.

At capacity three, A,B,C,A,D evicts B under ideal LRU. Then pause a reader holding version one, commit version two and invalidate, and resume the stale population. Version checks or another justified protocol must prevent an invalid freshness claim; a short TTL only changes exposure. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 11 / I1

Run the LRU fixture and reproduce a stale cache-aside population race. Include a counterexample and a corrected result.

At capacity three, A,B,C,A,D evicts B under ideal LRU. Then pause a reader holding version one, commit version two and invalidate, and resume the stale population. Version checks or another justified protocol must prevent an invalid freshness claim; a short TTL only changes exposure.

← Return to question

UNIT 11 / T1

When would a product choose lower availability rather than serve a stale cached value?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Caching is harmful when maintenance and correctness cost exceed saved work. A low-reuse result cache may consume memory and serialization time without many hits. A cache with missing authorization scope can leak data; one that masks a slow database during normal operation can hide an origin capacity deficit until outage. Test the system with the cache empty or unavailable. Prefer query/index improvements or simpler data access when they address the bottleneck with fewer state copies. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 12 · Scalability, Load Balancing, and Data Distribution

UNIT 12 / C1

Why does adding application servers fail to fix some storage bottlenecks?

Vertical scaling increases resources of one node; horizontal scaling adds nodes. Neither guarantees proportional throughput. A larger database host may buy time without distributed data management, while extra stateless application instances can distribute independent compute. Shared locks, disks, or downstream services can remain limiting. Compare failure impact, scaling delay, cost, and operational complexity. Choose the smallest change supported by profiling, then rerun the workload because the limiting resource can move.

← Return to question

UNIT 12 / C2

Which query and invariant costs arrive with sharding?

Partitioning divides a dataset; sharding commonly distributes those partitions across separate storage owners. Choose a key that supports important queries and distributes work. Sharding by tenant can isolate some traffic but leave one large tenant hot. Cross-shard joins, uniqueness, transactions, and migrations become more complex. Keep a routing model and relocation procedure. Add partitioning when a measured storage or write bottleneck justifies it, rather than assuming every large user count requires it.

← Return to question

UNIT 12 / C3

What does stable hashing reduce, and which skew does it leave?

Consistent hashing places keys and owners in a mapping that changes only part of the assignment when membership changes; virtual nodes can improve distribution. Rendezvous hashing scores each key-owner pair and chooses the highest eligible score, offering a different implementation of stable assignment. Neither guarantees equal traffic because request popularity is not uniform. Membership agreement, migration, replication, and hot-key handling remain required. Avoid changing hash functions without a plan for old data and client routing.

← Return to question

UNIT 12 / C4

Why are receive progress and apply progress useful distinctions for replicas?

Replication lag is the distance between source and replica state, measured by time, positions, or application version. A small average lag can hide a replica stalled for a particular workload. Expose source progress and replica apply progress, and distinguish transport lag from apply lag. Users may need read-your-writes even when global reads tolerate eventual freshness. Failover decisions must account for lag and acknowledged-write policy; the fastest promotion is not necessarily the safest promotion.

← Return to question

UNIT 12 / C5

Why must scale-out costs be compared under equal failure promises?

Scale-out economics includes duplicate baseline capacity, coordination, networking, telemetry, cache warmup, and operational work. Ten small nodes are not automatically cheaper or faster than one larger node. Compare equal service objectives and failure headroom under the same workload; include the capacity required during a node or zone loss. Managed systems can trade provider charges against staffing and recovery work. Use cost per successful business outcome and investigate bottlenecks before purchasing capacity that cannot help.

← Return to question

UNIT 12 / S1

One partition receives 70% of writes. What should guide the intervention?

Hot partitions arise from uneven key popularity, data size, or write concentration. A heatmap of requests and bytes per partition reveals skew hidden by fleet averages. Splitting a range may help if traffic divides; a single hot entity may need read copies, batching, or a redesigned invariant owner. Randomizing writes can distribute append pressure but makes reads aggregate more partitions. Match the fix to the actual pressure rather than repeatedly adding cold capacity.

← Return to question

UNIT 12 / S2

Each new application instance creates a large connection pool. How can scaling worsen the outage?

When a pool is full, waiting callers accumulate outside the database as well as inside it. More application replicas can multiply total database connections beyond the safe limit. Bound the aggregate across instances, reserve recovery capacity, and include acquisition time in deadlines. A query waiting on a lock may hold its connection while doing no CPU work. Fix hold time, contention, and admission before expanding every pool and increasing the pressure that caused the queue.

← Return to question

UNIT 12 / S3

Two replicas share one host. Which failure remains unprotected?

A single point of failure can be a machine, database, credentials service, router, operator action, or shared dependency. Count independent failure domains rather than only process replicas. Two application containers on one host do not survive that host's loss. Redundancy also needs failover, capacity, detection, and state correctness. Inspect control-plane dependencies during recovery: the data path may be redundant while a missing configuration service prevents every replacement instance from starting.

← Return to question

UNIT 12 / D1

Design or debug the running system: Compare rendezvous owners before and after adding node D, and measure skew separately.

For many deterministic keys, a key either keeps its old owner or moves to the new owner in this model. That property does not guarantee balanced request traffic. Make one key extremely popular and show why adding cold owners cannot split that single-key pressure without a new serving strategy. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 12 / I1

Compare rendezvous owners before and after adding node D, and measure skew separately. Include a counterexample and a corrected result.

For many deterministic keys, a key either keeps its old owner or moves to the new owner in this model. That property does not guarantee balanced request traffic. Make one key extremely popular and show why adding cold owners cannot split that single-key pressure without a new serving strategy.

← Return to question

UNIT 12 / T1

Can a single globally hot invariant be scaled merely by adding more partitions? Explain the constraint.

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Scale-out economics includes duplicate baseline capacity, coordination, networking, telemetry, cache warmup, and operational work. Ten small nodes are not automatically cheaper or faster than one larger node. Compare equal service objectives and failure headroom under the same workload; include the capacity required during a node or zone loss. Managed systems can trade provider charges against staffing and recovery work. Use cost per successful business outcome and investigate bottlenecks before purchasing capacity that cannot help. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 13 · File Systems, Search Infrastructure, and Data Pipelines

UNIT 13 / C1

Why do acknowledged upload parts not alone prove the final file is correct?

Multipart upload splits a file into identifiable parts with explicit completion. Resumability tracks confirmed offsets or parts so a client can retry without restarting all bytes. Duplicate part submission and changed source content need defined behavior. Verify the completed object rather than assuming every acknowledged part implies a correct final file. Multipart identifiers and checksums can be provider-specific; a convenient entity tag is not universally a content hash. Clean up incomplete sessions to prevent silent storage growth.

← Return to question

UNIT 13 / C2

Why does content addressing not authorize access?

A checksum detects accidental corruption under its algorithm's properties; a cryptographic digest can identify content with collision assumptions and appropriate verification. Content addressing uses a digest-derived identity, enabling deduplication or immutable references. It does not authorize access or prove the content is safe. Scope deduplication carefully across tenants to avoid information leakage, and verify downloaded bytes when integrity matters. A hash of a client-provided claim is not a checksum of the bytes actually stored.

← Return to question

UNIT 13 / C3

What does an inverted index store for blue car and blue bike?

An inverted index maps a term to postings describing documents or positions containing it. For documents D1='blue car' and D2='blue bike', blue maps to D1,D2 while car maps only to D1. Analysis choices such as tokenization and normalization affect matching. Search updates need document identity and version to prevent an old extraction from replacing a newer one. Keep source records rebuildable; the index is a derived serving structure with its own freshness and permission obligations.

← Return to question

UNIT 13 / C4

Which snapshot-to-CDC handoff problem can lose changes?

Change Data Capture emits committed changes from a source through a supported mechanism such as a transaction log. It can keep search or analytics current without polling entire tables. A snapshot plus a change stream needs a consistent handoff position so writes are neither missed nor applied incorrectly. Events can repeat, and schema changes affect decoding. Preserve source identity and ordering metadata, deduplicate, and monitor lag. CDC does not automatically make the destination transactionally consistent with the source for every query.

← Return to question

UNIT 13 / C5

Which identities make an extracted field explainable?

Provenance records where an artifact came from; lineage connects transformations between artifacts. An extracted amount should link to file version, page or region, extractor version, parameters, and review decisions. This supports debugging and correction without claiming the extraction is always right. Store stable references, not only a text explanation that cannot be checked. Privacy and retention apply to provenance too; an audit trail containing raw personal data can outlive the source if lifecycle policies are incomplete.

← Return to question

UNIT 13 / S1

Writes and deletes continue during reindexing. What must be replayed before a serving switch?

Reindexing builds a new representation while preserving serving behavior. Take a snapshot or defined source position, process its records, catch up subsequent changes, validate, and switch a serving alias or routing epoch. Do not overwrite the only working index before the new one is verified. Deletions and updates during the build need replay or reconciliation. Rollback must preserve schema compatibility and any new writes; counting indexed documents alone cannot establish that the right versions or permissions were indexed.

← Return to question

UNIT 13 / S2

An event arrives three minutes after it occurred. Which time and window policies matter?

Stream processing maintains results over ongoing events using windows, keys, and state. Event time describes when a source event occurred; processing time describes when it was handled. Watermarks estimate progress under assumptions about late arrivals; they are not proof that no earlier event can ever arrive. Define allowed lateness, correction behavior, checkpoint ownership, and effect boundaries. A windowed count can be updated after a late event, so consumers need to understand whether it is provisional or final.

← Return to question

UNIT 13 / S3

Only the PDF is deleted; extracted text and index entries remain. What lifecycle contract is incomplete?

Lifecycle policies should follow every copy: raw files, extracted text, thumbnails, indexes, logs, and backups. A retention timer on the original PDF alone leaves derived personal data behind. Use deletion workflows with durable progress and reconciliation; maintain narrowly scoped tombstones where required to prevent replay resurrection. Large object collections need efficient listing or ownership indexes for deletion. Test recovery from old snapshots against deletion history so a restore does not silently reverse required privacy actions.

← Return to question

UNIT 13 / D1

Design or debug the running system: Implement an input-versioned pipeline with a lineage record and a reindex generation switch.

Use Workflow to reject stale epochs and wrong input versions. Retain exact source and transform versions. Build a new index generation, account for concurrent changes and deletions, validate, and switch a serving pointer. Matching record counts alone is not sufficient evidence of correct versions or authorization. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 13 / I1

Implement an input-versioned pipeline with a lineage record and a reindex generation switch. Include a counterexample and a corrected result.

Use Workflow to reject stale epochs and wrong input versions. Retain exact source and transform versions. Build a new index generation, account for concurrent changes and deletions, validate, and switch a serving pointer. Matching record counts alone is not sufficient evidence of correct versions or authorization.

← Return to question

UNIT 13 / T1

How would you reproduce a disputed extracted field after both the source and model have new versions?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. A data-intensive design is a network of state transformations, not a collection of product logos. For every edge, define source authority, version, ordering, retry identity, validation, ownership, and recovery. Keep immutable source versions so derived structures can be rebuilt, but do not retain everything indefinitely without purpose. End-to-end reliability comes from coordinating those contracts and observing progress across the pipeline. A scalable queue cannot compensate for an extractor that loses provenance or an index that serves unauthorized copies. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 14 · Distributed Systems Fundamentals

UNIT 14 / C1

Why is provider success plus a lost reply a partial failure?

A partial failure affects some participants or paths while others keep operating. The payment provider can charge successfully while the order service loses the reply. Rolling back the local order cannot reverse that external event. Track an uncertain state with a stable provider reference and reconcile it. Design compensations only for effects that can actually be compensated, and distinguish an operation not attempted from one attempted with unknown outcome. Treating every exception as a clean failure creates duplicate business effects.

← Return to question

UNIT 14 / C2

What does Lamport ordering imply, and what does it not imply?

Lamport clocks advance locally and on receive using max(local,received)+1. If event A happens before B through program or message order, its Lamport value is smaller; the converse need not hold. Vector clocks track per-participant causal progress and can identify concurrent versions under their assumptions, at metadata cost. An example with vectors [1,0] and [0,1] is concurrent. Neither clock by itself resolves a conflict or establishes a global real-time commit order.

← Return to question

UNIT 14 / C3

Which real-time relationship must a linearizable register preserve?

Linearizability makes completed operations appear to take effect atomically at points between invocation and response, respecting real-time precedence. If a write of 2 completes before a read starts, the read cannot return the overwritten 1 in a linearizable register. Concurrent operations can admit more than one valid ordering. Implementing the property requires a suitable protocol, not only majority counts or wall-clock timestamps. A timed-out operation may still have taken effect because it lacks a completed response.

← Return to question

UNIT 14 / C4

Why is CAP not a permanent pick-any-two menu?

CAP's impossibility applies to a defined distributed read/write model under partitions: its formal atomic consistency and availability cannot both be guaranteed for every allowed execution. It is not a menu where a system permanently picks any two attractive qualities. When replicas cannot communicate, an application may reject some operations to protect invariants or serve bounded weaker behavior for others. State the affected operation, partition assumption, and required property. Normal nonpartitioned latency trade-offs require additional reasoning.

← Return to question

UNIT 14 / C5

Can rejecting every operation satisfy safety while failing useful progress?

Safety excludes bad outcomes; liveness requires progress under stated conditions. “No seat is sold twice” is safety. “Every accepted valid booking eventually resolves” is a liveness-oriented obligation with environmental assumptions. A system that rejects all requests may protect the first but fail the second. Network models and failure thresholds can prevent unconditional progress guarantees. Write both properties, state what failures may block completion, and expose stuck work so operators can restore progress without violating safety.

← Return to question

UNIT 14 / S1

A session reads version ten and then routes to a replica at nine. Which guarantee is threatened?

Read-your-writes keeps a session from losing visibility of its own completed updates; monotonic reads prevent that session from moving backward in observed progress. They can be implemented through routing affinity, replication-position tracking, or suitable protocols. They are session guarantees, not necessarily global linearizability. A user switching devices needs a policy for carrying session progress. Sticky routing alone can fail after an instance change, so define how the guarantee survives failover or when a read must wait.

← Return to question

UNIT 14 / S2

R=2 and W=2 for three replicas intersect. Why is that not a complete linearizability proof?

With fixed N replicas, R+W>N ensures a read set intersects a write set; W>N/2 ensures two write quorums intersect. These are set properties, not a complete proof of linearizability. Version ordering, concurrent writes, durable acknowledgments, membership, and read repair matter. Sloppy quorums or changing membership can invalidate naive assumptions. In N=3,R=2,W=2, a read sees at least one participant in a completed write set, but still needs a correct rule for interpreting returned versions.

← Return to question

UNIT 14 / S3

An old leader resumes after a replacement was elected. What must reject its stale effects?

Split brain means participants act as conflicting authorities. A partitioned old leader may keep running after another is elected. A failure detector cannot directly stop it. Use an ownership protocol and enforce epochs or fencing at the resource that accepts effects. Merely expiring a client-side lock variable is insufficient if an old worker can still write to storage. Test paused workers and delayed messages, not only clean process termination.

← Return to question

UNIT 14 / D1

Design or debug the running system: Evaluate logical clocks, vector relations, and fixed-quorum intersection for explicit histories.

lamport_receive(5,9) returns 10; vectors [1,0] and [0,1] are concurrent. For three fixed members, R=2,W=2 intersects. Explain why these calculations do not independently resolve concurrent business updates or prove linearizability. The missing protocol rules are part of the answer. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 14 / I1

Evaluate logical clocks, vector relations, and fixed-quorum intersection for explicit histories. Include a counterexample and a corrected result.

lamport_receive(5,9) returns 10; vectors [1,0] and [0,1] are concurrent. For three fixed members, R=2,W=2 intersects. Explain why these calculations do not independently resolve concurrent business updates or prove linearizability. The missing protocol rules are part of the answer.

← Return to question

UNIT 14 / T1

During a partition, which operations could degrade while exclusive allocation must wait or reject?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Safety excludes bad outcomes; liveness requires progress under stated conditions. “No seat is sold twice” is safety. “Every accepted valid booking eventually resolves” is a liveness-oriented obligation with environmental assumptions. A system that rejects all requests may protect the first but fail the second. Network models and failure thresholds can prevent unconditional progress guarantees. Write both properties, state what failures may block completion, and expose stuck work so operators can restore progress without violating safety. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 15 · Replication, Leader Election, and Consensus

UNIT 15 / C1

Which acknowledgment meaning must synchronous replication state?

Synchronous replication waits for specified acknowledgments before completing a write; asynchronous replication allows acknowledgment earlier. “Synchronous” must name whether an acknowledgment means received, durably stored, or applied. With async copying, loss of the only up-to-date owner can lose recent acknowledged writes. Waiting for independent durable replicas strengthens protection within the fault model but increases latency and may block during failure. Configuration is part of the application contract and must be tested during promotion.

← Return to question

UNIT 15 / C2

Why must replicated state-machine commands apply deterministically?

Consensus makes nonfaulty participants agree on a decision sequence under specified assumptions. A replicated state machine applies the same ordered commands deterministically to maintain equivalent state. Nondeterministic choices such as current time or random IDs must be supplied or reconciled consistently rather than recomputed differently on each replica. Consensus does not automatically make external HTTP effects atomic with state-machine application. Keep those effects explicitly identified and deduplicated outside the replicated decision boundary.

← Return to question

UNIT 15 / C3

Why does Raft commit reasoning need more than two acknowledgments?

Raft separates leader election, log replication, and safety. Terms identify election generations; a candidate needs a majority of votes and appropriate log freshness. Followers persist the protocol's required state. The leader replicates entries and advances commitment using Raft's rules, including the restriction on directly committing by majority counting to entries from its current term. This protects prior history through the protocol, not a generic “two acknowledgments always suffice” rule. Reads need a justified freshness mechanism too.

← Return to question

UNIT 15 / C4

Why can disjoint old and new voter configurations be unsafe?

Membership changes alter which groups can make decisions. Replacing an old majority with a disjoint new majority in one step can allow conflicting histories. Protocols such as Raft use a coordinated transition, including overlapping quorum requirements in joint consensus. Learners may catch up before becoming voting members. A cluster-size setting is not merely an operations toggle: apply the supported membership protocol, preserve persistent identities, and test interruption during reconfiguration.

← Return to question

UNIT 15 / C5

Why does convergence not establish an inventory invariant?

CRDTs arrange updates and merges so replicas converge under stated algebraic and delivery assumptions. A grow-only set can merge by union; an observed-remove set needs metadata to distinguish observed additions from concurrent ones. Convergence does not enforce arbitrary invariants such as “stock never below zero.” Choose a type whose semantics match the domain, preserve required metadata, and bound tombstone growth or compaction safely. An explicit conflict review may be better for business edits without a natural automatic merge.

← Return to question

UNIT 15 / S1

Worker epoch seven resumes after epoch eight wrote. Where must fencing be enforced?

A fencing token is a monotonically ordered ownership epoch checked by the resource receiving an effect. Worker A holding token 7 can pause past its lease while B obtains token 8. When A resumes, the protected store must reject token 7. A token generated but never enforced at the side-effect boundary does not fence anything. The receiving resource must atomically compare and apply, and the design must account for operations already accepted before the newer owner appeared.

← Return to question

UNIT 15 / S2

A long pause falsely suggests failure. Which part of the design must remain safe?

Failure detection observes missing or delayed evidence and produces suspicion under timing assumptions. Short detection intervals can reduce failover delay but increase false suspicions during pauses or network congestion. The safety protocol must remain correct when suspicion is wrong. Separate detector tuning from authority transfer and resource fencing. A successful heartbeat also does not prove that the node can meet every business operation's deadline, so readiness and capacity signals remain useful alongside protocol heartbeats.

← Return to question

UNIT 15 / S3

Failover succeeds but the old primary later returns. What must a safe failback consider?

Regional replication must define write authority, conflict behavior, acknowledgment geography, and disaster recovery. A nearby async acknowledgment has different data-loss exposure from a cross-region durable quorum. During a partition, some operations may wait or reject while stale-tolerant reads continue. Reconnection requires catch-up and reconciliation before arbitrary old owners resume. Test failback as well as failover: restoring an old primary without epoch and history control can reintroduce conflicting state after the visible outage ends.

← Return to question

UNIT 15 / D1

Design or debug the running system: Create a small fencing receiver and reject an old worker after a newer epoch has been accepted.

Use Workflow epoch checks for the local demonstration: epoch seven cannot complete after epoch eight owns the attempt. Explain why external storage must enforce its own fencing if old workers can still call it. A client-side lease flag cannot stop a resumed remote process. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 15 / I1

Create a small fencing receiver and reject an old worker after a newer epoch has been accepted. Include a counterexample and a corrected result.

Use Workflow epoch checks for the local demonstration: epoch seven cannot complete after epoch eight owns the attempt. Explain why external storage must enforce its own fencing if old workers can still call it. A client-side lease flag cannot stop a resumed remote process.

← Return to question

UNIT 15 / T1

What does a region returning after failover have to prove before it can accept authoritative writes again?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Regional replication must define write authority, conflict behavior, acknowledgment geography, and disaster recovery. A nearby async acknowledgment has different data-loss exposure from a cross-region durable quorum. During a partition, some operations may wait or reject while stale-tolerant reads continue. Reconnection requires catch-up and reconciliation before arbitrary old owners resume. Test failback as well as failover: restoring an old primary without epoch and history control can reintroduce conflicting state after the visible outage ends. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 16 · Message Queues, Events, and Distributed Workflows

UNIT 16 / C1

Why does an ephemeral subscription not supply historical replay?

A queue commonly distributes work among consumers; a retained log preserves an ordered append history for replay; pub/sub distributes publications to subscribers under its specific retention and acknowledgment semantics. Products combine these features, so check the actual contract. A consumer that needs replay after an outage cannot rely on an ephemeral subscription alone. Choose retention, partitioning, and consumer progress rules from recovery requirements rather than assuming every broker is a durable event history.

← Return to question

UNIT 16 / C2

How does acknowledgment placement trade lost work against duplicate work?

At-most-once processing can acknowledge before an effect and lose work after a crash. It avoids certain duplicates at the cost of missing effects. It may suit disposable telemetry under a deliberate loss budget, but not an order that has been promised durable processing. The term does not imply “always one”; zero is allowed. Specify whether the guarantee concerns delivery attempts, processing starts, or committed results because different boundaries can produce different counts.

At-least-once processing retries until acknowledged under retention and recovery assumptions, so duplicates are possible. Commit a durable effect before acknowledging and make repetition safe. A crash after commit but before acknowledgment replays the record. A uniqueness constraint on event identity within the same local transaction as the effect can prevent duplicate local changes. If the transaction marks processed before making an unprotected remote call, it can lose that call; the effect boundary matters.

← Return to question

UNIT 16 / C3

Which boundary can a scoped exactly-once claim actually cover?

Exactly-once claims require a precise boundary. A transactional stream processor may atomically couple consumed progress and produced records within its supported system. That does not include an arbitrary email server or payment provider unless another protocol covers the side effect. Idempotency can give a single relevant effect for repeated attempts under a retention and identity contract; it does not mean execution started only once. Name failures and recovery assumptions instead of using the phrase as a universal broker property.

← Return to question

UNIT 16 / C4

Why can a transactional outbox still publish duplicates?

A transactional outbox records state changes and publication intent in the same local transaction. A relay later publishes pending intents and marks progress. If it publishes then crashes before marking, it can publish again, so consumers still deduplicate. The pattern closes a local database-to-publication gap; it does not make arbitrary subscribers part of that transaction. Use stable event IDs, retryable relay progress, retention, and reconciliation for intents that remain unprocessed too long.

← Return to question

UNIT 16 / C5

Why is compensation not necessarily a perfect inverse?

A saga coordinates local transactions and compensating actions across ownership boundaries. Inventory reservation, payment authorization, and shipment creation cannot be treated as one ordinary local transaction. On failure, compensation might release stock or request a refund, but it can fail and may not perfectly reverse the original action. Persist workflow state and compensation progress. Distinguish recoverable intermediate states from final success or failure, and reconcile with external systems after uncertain responses.

← Return to question

UNIT 16 / S1

An event effect commits but acknowledgment is lost. How does the next delivery avoid a second local effect?

An idempotent consumer records event identity and its relevant local effect atomically. INSERT processed(event_id) under uniqueness and the corresponding state update must share one transaction if they protect the same invariant. A duplicate then returns the already established outcome. Retain identity long enough for permitted replay and delayed delivery. This protects local effects only; calling an external provider still requires its operation key or reconciliation. Deduplicating only in volatile memory fails after restart.

← Return to question

UNIT 16 / S2

A worker’s lease expires while it is still running. What protects later effects?

Scheduling combines a durable due-time record with worker coordination. A scheduler claims due work using an atomic transition or lease and creates a stable run identity such as (schedule_id,occurrence). Clock changes, missed periods, overlapping runs, and failover need explicit policy. A lease can expire while the old worker runs, so protect effects with fencing or idempotency at their resource. Exactly one scheduler process is not a substitute for a recoverable occurrence protocol.

← Return to question

UNIT 16 / S3

A payment remains uncertain locally but charged externally. Why is blind resubmission unsafe?

Reconciliation compares intended and observed outcomes after partial failure. A payment marked pending locally may already be charged at the provider; query by stable operation identity and repair the local state rather than blindly charging again. Reconciliation must itself be idempotent, authorized, and observable. Run it periodically and during incident recovery, tracking unresolved discrepancies with age and ownership. It is an essential protocol stage, not an admission that the normal path can ignore persistence or validation.

← Return to question

UNIT 16 / D1

Design or debug the running system: Run the outbox relay through publish-then-crash and process the duplicate with a local inbox transaction.

The example records two deliveries but one counter effect. Deduplication identity and effect commit share a transaction. If an external email or payment is substituted for the counter, the local inbox no longer establishes one external effect; use the provider contract and reconciliation. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 16 / I1

Run the outbox relay through publish-then-crash and process the duplicate with a local inbox transaction. Include a counterexample and a corrected result.

The example records two deliveries but one counter effect. Deduplication identity and effect commit share a transaction. If an external email or payment is substituted for the counter, the local inbox no longer establishes one external effect; use the provider contract and reconciliation.

← Return to question

UNIT 16 / T1

Can one workflow be exactly once at a broker boundary but still send two emails? Specify the boundaries.

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Reconciliation compares intended and observed outcomes after partial failure. A payment marked pending locally may already be charged at the provider; query by stable operation identity and repair the local state rather than blindly charging again. Reconciliation must itself be idempotent, authorized, and observable. Run it periodically and during incident recovery, tracking unresolved discrepancies with age and ownership. It is an essential protocol stage, not an admission that the normal path can ignore persistence or validation. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 17 · Reliability, Fault Tolerance, and Disaster Recovery

UNIT 17 / C1

Why does redundant instance count not establish surviving capacity?

Redundancy adds spare components or copies, but their usefulness depends on independence, capacity, detection, and state correctness. If two instances each run at 70% and one fails, the other cannot absorb all traffic without overload. Plan reduced-capacity behavior or reserve appropriate headroom. Shared configuration mistakes can disable all replicas simultaneously. Redundancy should be paired with tested traffic movement and recovery, rather than counted as a number of boxes in an architecture picture.

← Return to question

UNIT 17 / C2

Why must pool acquisition fit the overall deadline?

Timeouts bound waiting and resource ownership. Use an overall request deadline plus sensible stage limits for connection, pool acquisition, and operations. A timeout should trigger cleanup or cancellation where possible, but does not prove a remote action failed. Choosing every timeout independently can let downstream work outlive the caller's useful window. Propagate remaining time and avoid beginning expensive work that cannot finish within it. Record timeout stage to distinguish queueing from execution failures.

← Return to question

UNIT 17 / C3

What does a circuit breaker change, and what does it not repair?

A circuit breaker tracks dependency failures and temporarily rejects or redirects calls under a policy. Common states are closed, open, and half-open, with bounded probes to test recovery. It prevents futile waiting but does not repair the dependency or decide domain correctness. A fallback that returns a fabricated payment approval is unsafe. Make failure classification, probe concurrency, and recovery thresholds explicit, and avoid every instance probing simultaneously after the same interval.

← Return to question

UNIT 17 / C4

Which shared limits can defeat apparently separate worker bulkheads?

Bulkheads partition resources so one class of work cannot consume everything. Separate pools for payment authorization and optional receipt delivery can keep checkout viable when email stalls. Pool sizing must include shared limits such as database connections and host memory; separate thread pools alone do not isolate every dependency. Over-isolation can waste spare capacity. Choose boundaries by failure harm and observe saturation and rejection in each partition, not only fleet-level averages.

← Return to question

UNIT 17 / C5

Why are RTO and RPO different objectives?

RTO is the recovery time objective; RPO is the acceptable data-loss exposure expressed in time. If an incident begins at 12:00, service restoration at 12:20 gives a twenty-minute interruption; restoring data only through 11:55 creates five minutes of exposure relative to the incident point. These are separate objectives and measurements. Set them per business function, and verify detection, decision, restoration, validation, and traffic return in drills rather than timing only the database copy operation.

← Return to question

UNIT 17 / S1

Nested retries amplify provider load during an outage. Which end-to-end controls are needed?

A retry storm amplifies an outage when many callers repeat work against a struggling dependency. Three attempts at four nested layers can produce 81 downstream attempts per original intent. Random delay reduces synchronization, but an attempt budget and admission control are also needed. Preserve goodput by rejecting excess work and suppressing redundant attempts. Monitor original business intents separately from attempts so apparent traffic growth during an outage is not mistaken for healthy user demand.

← Return to question

UNIT 17 / S2

The ledger is unavailable. Can a cached default balance be a safe fallback?

A fallback substitutes an alternative behavior with an explicit contract. A cached help page can replace a failed content request; a default account balance cannot replace a failed ledger lookup. Fallbacks can create load elsewhere and can mask a prolonged incident. Bound their freshness and duration, signal degraded operation, and check security scope. A second provider is not transparent if its semantics, operation identities, or reconciliation process differ from the first.

← Return to question

UNIT 17 / S3

A repaired old primary returns after failover. Why not just route traffic back?

Failover transfers service or authority to a replacement; failback returns responsibility after repair. Both require state freshness, fencing, routing, and capacity decisions. Promote only under the chosen durability contract and prevent the old owner from accepting conflicting writes. Rejoining an old primary may require rebuilding its state from the new history. DNS, connection pools, caches, and client retries can prolong the transition even after a database promotion succeeds. Test the complete user-visible sequence.

← Return to question

UNIT 17 / D1

Design or debug the running system: Model nested retries and write a provider-outage response policy.

Compute 3^4 = 81 potential attempts for four independently retrying layers. Introduce one end-to-end retry budget, bounded waits, admission, and channel isolation. Do not use a false payment approval as a fallback. Verify recovery behavior and unresolved business state after the provider returns. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 17 / I1

Model nested retries and write a provider-outage response policy. Include a counterexample and a corrected result.

Compute 3^4 = 81 potential attempts for four independently retrying layers. Introduce one end-to-end retry budget, bounded waits, admission, and channel isolation. Do not use a false payment approval as a fallback. Verify recovery behavior and unresolved business state after the provider returns.

← Return to question

UNIT 17 / T1

How can a disaster exercise pass infrastructure checks yet fail the business recovery requirement?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Incident analysis reconstructs the timeline, contributing conditions, detection gaps, and recovery decisions without reducing the explanation to individual blame. “Human error” does not explain why one action could disable every replica without a safe recovery path. Link corrective work to observed mechanisms and assign verification evidence. Prefer changes that reduce recurrence or harm, such as scoped credentials, tested restores, and bounded retries, rather than adding alerts that do not create a useful response action. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 18 · Security Architecture and Privacy

UNIT 18 / C1

Why does an internal network not automatically authenticate a caller?

A trust boundary changes the confidence or authority granted to input. Browser-to-server, server-to-storage, worker-to-model, and operator-to-production are distinct boundaries. Validate identity and relevant data each time a boundary is crossed; do not treat internal network origin as proof of legitimacy. Headers forwarded by a trusted proxy require a specific trust configuration. Uploaded files remain untrusted after storage and must be processed with appropriate resource and execution isolation.

← Return to question

UNIT 18 / C2

How do OAuth access authorization and OpenID Connect identity differ?

OAuth 2.0 concerns delegated authorization; OpenID Connect adds an identity layer. Do not use an access token as an identity assertion without the intended protocol and validation. For appropriate browser/native flows, use current best-practice guidance such as authorization code with PKCE, exact redirect handling, and library-based validation. Issuer, audience, signature, nonce or state requirements, and token type depend on the flow. Never invent a custom shortcut that confuses login identity with permission to a target API.

← Return to question

UNIT 18 / C3

Why is a fast general-purpose hash unsuitable for passwords?

Passwords require purpose-built password hashing with salt and an appropriate work factor, not fast general-purpose hashing or reversible encryption. Use a maintained implementation and current OWASP guidance, choosing parameters that balance offline-guessing resistance with service capacity. Store the encoded algorithm and parameters for migration, and rehash when policy changes. Rate-limit verification and protect recovery flows. Do not invent fixed universal parameters in a textbook; hardware, libraries, and current guidance must be checked during deployment.

← Return to question

UNIT 18 / C4

Which paths must preserve tenant isolation beyond the database query?

Tenant isolation must survive every path: API lookup, cache key, job queue, object URL, search filter, export, telemetry, and administrative action. Carry trusted tenant context and revalidate resource relationships rather than accepting arbitrary tenant IDs in messages. Composite keys or database policies can add enforcement when correctly configured. A global cache keyed only by document ID can bypass an otherwise correct database query. Test two tenants with overlapping local IDs and adversarial cross-access attempts.

← Return to question

UNIT 18 / C5

Why does an artifact signature not prove the artifact is harmless?

Supply-chain security covers dependencies, build tools, artifacts, deployment permissions, and update paths. Pin or otherwise control dependency resolution, verify artifact identity, and keep build and runtime privileges appropriately narrow. A signed artifact proves a signing relationship, not that its code is harmless. Review provenance, vulnerability response, and the ability to revoke a compromised release. Avoid letting an untrusted pull request access deployment secrets merely because the normal branch build is trusted.

← Return to question

UNIT 18 / S1

An administrator of tenant A guesses a tenant B document ID. What must enforce denial?

Authentication establishes identity; authorization decides permitted action on a particular resource in context. An organization administrator may manage their own tenant without any right to another tenant's files. Derive tenancy from authenticated membership and enforce it on every read, update, export, and background operation. Guess-resistant IDs reduce enumeration but do not supply authorization. Deny by default, keep policy decisions explicit, and test both allowed access and cross-tenant denial paths.

← Return to question

UNIT 18 / S2

A backup survives but its only decryption key was destroyed. Which recovery assumption failed?

Key management includes generation, storage, access, rotation, versioning, backup, and destruction. Envelope encryption can separate data-encryption keys from a key-encryption authority, but the design must retain the right key versions to read permitted historical data. Deleting a key before verifying retention and recovery needs can make backups unusable. Conversely, retaining every key indefinitely may defeat deletion goals. Record which data and services depend on each key version and test rotation and restore paths.

← Return to question

UNIT 18 / S3

Membership is revoked after an export is queued. Which later boundary must follow the chosen revocation policy?

A secure-by-design review follows one sensitive operation end to end. For an export, verify actor and tenant, resource filters, object permissions, queue context, signed download lifetime, audit outcome, and deletion. Include failures such as revoked membership after enqueue and a worker retry after cancellation. Record assumptions and ownership of controls. A checklist helps coverage, but its evidence should be concrete tests and reviewed configurations rather than a declaration that the system is secure.

← Return to question

UNIT 18 / D1

Design or debug the running system: Test a tenant-scoped authorization predicate and all cross-tenant paths around it.

permit_demo() denies tenant A reading tenant B even when the action name is allowed. This synthetic predicate is not credential validation. Extend tests to cache keys, queued jobs, exports, and object capabilities; a correct predicate unused by a worker does not enforce anything. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 18 / I1

Test a tenant-scoped authorization predicate and all cross-tenant paths around it. Include a counterexample and a corrected result.

permit_demo() denies tenant A reading tenant B even when the action name is allowed. This synthetic predicate is not credential validation. Extend tests to cache keys, queued jobs, exports, and object capabilities; a correct predicate unused by a worker does not enforce anything.

← Return to question

How would tenant revocation affect an already issued object capability and a queued export?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. A secure-by-design review follows one sensitive operation end to end. For an export, verify actor and tenant, resource filters, object permissions, queue context, signed download lifetime, audit outcome, and deletion. Include failures such as revoked membership after enqueue and a worker retry after cancellation. Record assumptions and ownership of controls. A checklist helps coverage, but its evidence should be concrete tests and reviewed configurations rather than a declaration that the system is secure. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 19 · Observability, Monitoring, and Incident Engineering

UNIT 19 / C1

How do logs, metrics, and traces answer complementary questions?

Logs record discrete events, metrics aggregate measurements, and traces relate operations along a path. They complement one another: a latency histogram reveals the population tail, a trace localizes one slow request, and structured logs explain a particular error. None alone guarantees a complete account. Sampling, dropped telemetry, inconsistent clocks, and privacy limits affect the evidence. Use stable operation and resource identities to connect signals without making unbounded identifiers metric labels.

← Return to question

UNIT 19 / C2

Why is a trace ID not authorization?

Trace IDs identify a trace, span IDs identify individual spans, and propagation carries the relevant context across boundaries. Validate or regenerate untrusted incoming context according to policy and avoid treating trace identity as authorization. For queues, propagate correlation or links so a later worker can be connected to the originating operation without implying it ran during the HTTP request. Standard propagation formats improve interoperability; they do not ensure every library or message transport is instrumented automatically.

← Return to question

UNIT 19 / C3

Why can pool saturation explain high latency with low CPU?

RED focuses on rate, errors, and duration for request-serving components; USE focuses on utilization, saturation, and errors for resources. Apply the lens to the question. A database connection pool can be saturated while host CPU is low, explaining why increasing CPU does not help. Averages hide bursts, so examine time resolution and per-owner skew. These methods organize investigation rather than replace workload-specific indicators such as oldest unprocessed payment age.

← Return to question

UNIT 19 / C4

What does tenfold error-budget burn mean for a 99.9% target?

An SLI measures a chosen service behavior; an SLO sets a target; an error budget quantifies the allowed shortfall over a window. Define both good events and eligible events, including relevant failures. For a 99.9% target, a 1% bad-event rate consumes budget ten times as fast as the allowed 0.1% rate. Segment critical operations appropriately and ensure low-volume denominators are handled thoughtfully. An SLO is not a guarantee about every individual request.

← Return to question

UNIT 19 / C5

Why do unique request IDs make poor general metric labels?

Sampling and retention control telemetry cost but affect which questions can be answered. Head sampling decides early; tail sampling can select traces based on later observed behavior but needs buffering and capacity. High-cardinality labels such as unique request IDs can create unbounded time-series growth. Keep detailed identities in suitable logs or traces and use bounded dimensions for metrics. Protect telemetry ingestion during outages so error bursts do not consume all application or observability capacity.

← Return to question

UNIT 19 / S1

A five-second request spent 4.6 seconds acquiring a connection. Which optimization is the evidence pointing toward?

A distributed trace links spans representing work and waits across components. Parent-child relationships describe instrumentation structure; asynchronous work may need links rather than a misleading synchronous parent lifetime. A five-second root span with 4.6 seconds in pool acquisition suggests a different intervention from 4.6 seconds of database execution. Instrument queue wait and connection wait explicitly. A trace is one sampled execution, so compare distributions and errors before generalizing its cause to every slow request.

← Return to question

UNIT 19 / S2

Serving recovers but uncertain charges remain. Why is the incident not fully resolved?

Incident response coordinates mitigation, communication, evidence, and decision ownership. Assign an incident lead for sufficiently complex events, maintain a timeline, and reduce harmful changes. Restore useful service with a justified reversible intervention while preserving data correctness. A rollback is not safe if it misreads a migrated schema. Track uncertain business outcomes such as pending charges for reconciliation after serving recovers; ending an infrastructure alert does not mean the complete incident outcome is resolved.

← Return to question

UNIT 19 / S3

HTTP 202 returns quickly while jobs wait for hours. Which end-to-end indicators are missing?

Asynchronous observability follows a business operation beyond the original request. Link submission, queue delivery, attempts, stage commits, retries, and terminal outcomes using stable identities. Separate queue wait from processing duration and record duplicate or stale-attempt rejection. A trace ending at HTTP 202 says little about whether an extraction or refund eventually finished. Define end-to-end age and completion indicators, and maintain a way to locate stuck operations across brokers, workers, databases, and external providers.

← Return to question

UNIT 19 / D1

Design or debug the running system: Build a trace waterfall and an end-to-end asynchronous completion indicator.

Use the five-second path with 4.6 seconds in pool acquisition. Separate wait from query execution and compare population evidence. Track submission-to-terminal job age rather than only HTTP acceptance. Avoid raw document content or high-cardinality request IDs as ordinary metric labels. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 19 / I1

Build a trace waterfall and an end-to-end asynchronous completion indicator. Include a counterexample and a corrected result.

Use the five-second path with 4.6 seconds in pool acquisition. Separate wait from query execution and compare population evidence. Track submission-to-terminal job age rather than only HTTP acceptance. Avoid raw document content or high-cardinality request IDs as ordinary metric labels.

← Return to question

UNIT 19 / T1

Which evidence would make a latency hypothesis convincing rather than merely correlated with a release?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Asynchronous observability follows a business operation beyond the original request. Link submission, queue delivery, attempts, stage commits, retries, and terminal outcomes using stable identities. Separate queue wait from processing duration and record duplicate or stale-attempt rejection. A trace ending at HTTP 202 says little about whether an extraction or refund eventually finished. Define end-to-end age and completion indicators, and maintain a way to locate stuck operations across brokers, workers, databases, and external providers. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 20 · Deployment, Cloud Infrastructure, and Platform Architecture

UNIT 20 / C1

Which VM and container isolation and ownership distinctions matter?

A virtual machine presents virtualized hardware with its guest operating system; containers commonly isolate processes while sharing a host kernel. Both need image lifecycle, resource controls, identity, network, and storage design. A container is not a tiny VM with all the same isolation guarantees. Choose based on workload, operating expertise, startup and isolation needs. Multiple containers on one host share failure and resource domains, while VM boundaries still do not remove shared infrastructure risks.

← Return to question

UNIT 20 / C2

How can control-plane failure differ from data-plane failure?

The control plane decides configuration and placement; the data plane serves application traffic. A control-plane outage can stop new scheduling while already running workloads continue under some conditions. Avoid unnecessary synchronous control-plane dependencies in every request. Cached configuration needs an explicit validity and update policy, especially for security-sensitive changes. Draw the two paths separately so a claimed serving redundancy does not conceal a single configuration or credential dependency needed for recovery.

← Return to question

UNIT 20 / C3

Why can a canary receiving only cheap requests miss a serious regression?

Rolling deployment replaces instances gradually; blue-green prepares a parallel environment and switches traffic; canary exposes a controlled subset before wider rollout. All need compatible schemas, traffic drain, capacity, and verification. A canary that receives only cheap requests can miss a critical-path regression. Define cohorts and stop conditions using meaningful indicators. Blue-green duplicates runtime capacity but does not automatically duplicate or safely roll back shared mutable data. Select by failure risk and operating cost.

← Return to question

UNIT 20 / C4

Why can schema changes make code rollback unsafe?

Rollback restores code or configuration only when state and contracts remain compatible. Use expand-and-contract database changes: add compatible structures, deploy code that can operate across the transition, backfill and verify, then remove obsolete fields after old readers are gone. Destructive schema changes can make old code unusable. Long backfills need rate control and resumable progress. A rollback button should identify the affected data and protocol assumptions rather than imply time can be reversed for every side effect.

← Return to question

UNIT 20 / C5

Which responsibilities stay with a team using a managed database?

A managed service transfers selected tasks, not business responsibility. The provider may operate patches or replication while the application still chooses acknowledgment settings, schema, access, retention, backup validation, and regional recovery. Read the actual service contract and failure boundaries. Self-management gives control while requiring expertise and on-call work. Compare what remains your responsibility in either choice, and test client behavior under maintenance, throttling, failover, and credential changes.

← Return to question

UNIT 20 / S1

Scale-down kills workers holding unrecorded jobs. What lifecycle contract was missing?

Autoscaling can change instances, worker concurrency, or provisioned capacity. Select signals related to the workload, preserve safe minimums, and include warmup and draining. Scaling down must stop admission, respect deadlines, and transfer durable work rather than killing unrecorded jobs. Database and provider limits can cap useful growth even if platform capacity is available. Protect total downstream concurrency and spend; an autoscaler responding to retries can otherwise amplify an incident.

← Return to question

UNIT 20 / S2

A feature flag is disabled after new state was written. What compatibility issue remains?

Feature flags separate some release and exposure decisions. They need ownership, safe defaults, scope, observability, and removal policy. A flag does not replace authorization: letting the browser enable a privileged server operation is unsafe. Combinations create a testing surface, and stale flags accumulate complexity. When disabling a feature after it has written new state, ensure old code understands that state or provide a migration path. Runtime configuration is itself part of the release risk.

← Return to question

UNIT 20 / S3

Two application replicas use independent local databases. Why did scheduling not create a correct shared service?

Deployment topology places processes, state, trust boundaries, routes, and failure domains on actual infrastructure. A logical module diagram cannot show whether two replicas share a host, zone, or database. Include ingress, egress, TLS termination, private paths, and operational access. A topology should explain what happens when one placement disappears and how replacement obtains state and configuration. Keep control and data flow meanings clear; every arrow should identify the protocol or dependency it represents.

← Return to question

UNIT 20 / D1

Design or debug the running system: Write an expand-and-contract migration and a mixed-version rollout test.

Add a compatible field, deploy readers that tolerate both versions, backfill under a bounded budget, verify, and remove old structure only after old readers retire. Test rollback before contraction and explicitly identify when rollback ceases to be safe. A canary must receive representative traffic. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 20 / I1

Write an expand-and-contract migration and a mixed-version rollout test. Include a counterexample and a corrected result.

Add a compatible field, deploy readers that tolerate both versions, backfill under a bounded budget, verify, and remove old structure only after old readers retire. Test rollback before contraction and explicitly identify when rollback ceases to be safe. A canary must receive representative traffic.

← Return to question

UNIT 20 / T1

How could a rollback restore old code while making stored data or external effects less correct?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Platform engineering provides reusable paths for teams to build and operate services safely. Useful capabilities include artifact delivery, identity, telemetry, resource policy, and tested deployment patterns. Treat internal developers as users and provide escape hatches with clear ownership rather than an opaque mandatory framework for every workload. Platform abstractions must expose relevant limits and failure behavior. Measure adoption, reliability, and operating burden; another layer is valuable only when it simplifies work without hiding essential decisions. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 21 · Performance, Cost Optimization, and System Validation

UNIT 21 / C1

What must a benchmark disclose before its result is reusable?

Profiling identifies where runtime work occurs; benchmarking compares behavior under a controlled workload. Define hardware or capacity, data distribution, warmup, concurrency, duration, success criteria, and versions. A microbenchmark of one function does not establish application throughput. Include variability and resource cost, and compare like-for-like service promises. If a result changes after a dependency or schema change, preserve enough setup information to reproduce the explanation rather than presenting a single impressive number.

← Return to question

UNIT 21 / C2

How can coordinated omission hide delay?

Benchmark pitfalls include unrealistic data, warmed caches without disclosure, coordinated omission, tiny samples, missing failures, and incompatible versions. Coordinated omission can occur when the generator waits on a slow response before issuing work that should have arrived independently, understating delay. Report distributions and methodology, not fabricated precision. Change one factor or use a justified experimental design. A result is evidence for the tested conditions and does not automatically extrapolate to a new traffic mix or failure state.

← Return to question

UNIT 21 / C3

Why is the short-run throughput ceiling not a safe production target?

Throughput approaches a ceiling when a limiting resource cannot complete work faster. Additional offered traffic then becomes rejection or queue growth. A queue is not extra service capacity. Find the goodput knee by observing successful outcomes, latency, and saturation under increasing demand, then reserve margin for variability and failure. A measured maximum in a short benchmark is not a safe production target. Include maintenance, background work, and the surviving capacity of the promised failure scenario.

← Return to question

UNIT 21 / C4

Why does cost per attempt sometimes conceal cost per useful outcome?

Cost per request can obscure the business outcome if retries or failures inflate the denominator. Prefer cost per successful order, delivered document, or another meaningful unit, while still analyzing technical costs. With an assumed monthly cost of 3000 units and 150000 successful outcomes, unit cost is 0.02. If half the attempts fail, dividing by all attempts understates the cost of useful work. Define period, allocation, and included operating expenses.

← Return to question

UNIT 21 / C5

What should a useful architecture fitness function protect?

Architecture fitness functions continuously test required properties such as allowed imports, tenant-scoped queries, API compatibility, or recovery time. A useful function protects a reasoned architectural characteristic without freezing every incidental class relationship. Some properties need runtime drills or manual review rather than a static rule. Keep tests aligned with current decisions and update them when an ADR legitimately changes the architecture. Passing a collection of checks is evidence for their scope, not a universal architecture certification.

← Return to question

UNIT 21 / S1

An index fixes the actual slow query. Why might sharding have been a worse first intervention?

Tuning improves an implementation within its architecture; an architectural change alters ownership, placement, or major interaction. An index, batch size, or shorter transaction may resolve a bottleneck with less migration risk than sharding. Conversely, an unavoidably centralized invariant can require a different business or partitioning model. Measure before and after, and count correctness, operations, and migration cost alongside speed. The most sophisticated topology is not automatically the most defensible intervention.

← Return to question

UNIT 21 / S2

The process is only killed before starting work. Which dangerous failure windows remain untested?

Fault injection tests behavior when a dependency is slow, unavailable, inconsistent, or interrupted. Place faults at meaningful boundaries: after database commit but before acknowledgment, during partition transfer, or after a worker lease expires. Killing a process only before work starts misses the dangerous windows. Define acceptable outcomes and stop conditions. Combine fault evidence with data reconciliation; an application that recovers request serving while silently losing jobs has not passed the full business contract.

← Return to question

UNIT 21 / S3

Source and destination have equal row counts but different ownership. Why did the validation miss a bug?

Data validation compares source and destination identities, versions, aggregates, and domain invariants. Row counts alone cannot detect swapped ownership, stale updates, or equal-count omissions and duplicates. For a ledger, validate balanced postings and unique external operation identity; for a search index, compare source version and deletion state. Run checks after migration, replay, and recovery. Sampling can reduce cost but weakens the claim, so state whether evidence covers all records or only a sample.

← Return to question

UNIT 21 / D1

Design or debug the running system: Compare three architectures using explicit fictional costs and equal requirement checks.

cost_options() totals 371, 551, and 1402 fictional monthly units. These are cost inputs only, not measured capacities or reliability. Compare each with original latency, failure, correctness, and operating constraints; reject an infeasible cheap option before ranking cost efficiency. Begin with the smallest correct boundary and identify an observable failure of the naive alternative.

← Return to question

UNIT 21 / I1

Compare three architectures using explicit fictional costs and equal requirement checks. Include a counterexample and a corrected result.

cost_options() totals 371, 551, and 1402 fictional monthly units. These are cost inputs only, not measured capacities or reliability. Compare each with original latency, failure, correctness, and operating constraints; reject an infeasible cheap option before ranking cost efficiency.

← Return to question

What evidence would justify accepting a more expensive architecture as the simpler complete operating system?

A defensible answer must state the changed contract, the authoritative owner, and the failure history before choosing a mechanism. Validate the design against its original requirements using a traceable evidence matrix. Every promise should identify a test, measurement, review, or unresolved assumption. Compare functional behavior, latency distributions, capacity, recovery, security, cost, and operational burden under the stated workload. A diagram review and a load test provide different evidence. If a target remains unmeasured, label it as an objective rather than reporting it as achieved. This closes the loop from design hypothesis to defensible engineering decision. A component name or a claim that more replicas solve it is insufficient; show how the proposed history remains valid under the stated failure.

← Return to question

Unit 22 · End-to-End Design, Architecture Reviews, and Evolution

UNIT 22 / C1

Why do the twelve designs use different authoritative and derived state boundaries?

Creation and code ownership require an atomic uniqueness boundary; management reads should meet the chosen read-after-write contract. Redirects may permit bounded stale targets only if revocation and expiry requirements allow it. Analytics can be eventually processed with stable event identities. Do not describe the whole system as simply eventual: the mapping insertion and the click count have different obligations. Accepted post identity and deletion state need strong local transactional handling. Candidate projections and engagement counts can be eventually refreshed. Read-your-writes for an author's own post may require a direct-source overlay. Authorization is not allowed to become weaker merely because candidates are stale. State which parts of the feed are snapshots and which are revalidated at every serving operation.

← Return to question

UNIT 22 / C2

Why are a chat persistence receipt and a human read receipt different?

Send a message to an authorized conversation, retain it durably, deliver online updates, reconnect and fetch missed messages, and expose a meaningful receipt model. Support one-to-one and bounded group conversations. Presence is approximate and ephemeral. Define sent, persisted, delivered-to-device, and read as different states rather than promising that opening a WebSocket makes every message read by a person. Provide monotonic per-conversation sequence under the selected authority, read-after-write for sender confirmation, and replayable catch-up. Cross-conversation ordering is unnecessary unless a product requirement says otherwise. Device delivery is at least retryable and can duplicate frames; clients deduplicate by message identity. Presence and read indicators may be eventually updated but should not move backward under their defined receipt model.

← Return to question

UNIT 22 / C3

Why can a scheduler lease not guarantee a single external execution?

Schedule advancement and occurrence creation need atomicity and uniqueness. Worker delivery can be at least once. Conditional completion under the current epoch prevents stale local results; idempotency at the side-effect boundary or enforced fencing protects business effects. A lease by itself cannot prove exactly-once execution. Explicitly define misfire and overlap policies instead of letting timing races decide whether a missed occurrence runs.

← Return to question

UNIT 22 / C4

Why does a collaborative editor need more than eventual string equality?

OT transforms concurrent operations against the relevant ordered context; an insertion can shift another insertion's position. CRDTs choose merge semantics that converge under their algorithm's assumptions, often using stable element identifiers and ordering. For two concurrent inserts X and Y after the same element, replicas need the same tie-break rule. Neither acronym supplies all intent or rich-text semantics. Authorization and durable acknowledgment remain separate from convergence. Authorize every sync and operation, not only opening the document page. Recheck membership according to a defined revocation policy for long-lived sessions and offline reconnect. Restrict exports and snapshots, avoid leaking content through presence or logs, and validate protocol inputs. If end-to-end encryption is required, explain how server-side search, validation, history, and access revocation change; transport TLS alone is not that guarantee.

← Return to question

UNIT 22 / C5

Why is a document-processing complete state inadequate without provenance and publication evidence?

Failures include incomplete uploads, corrupt files, malicious content, worker crash after output write, duplicate stage delivery, incompatible model output, stale completion, review races, missing lineage, and deletion during processing. Stable artifacts and epoch-checked transitions prevent stale overwrite. Quarantine invalid outputs with reasons. A document marked complete without usable authorized output or source evidence is a failed workflow even if no worker raised an exception. Measure accepted bytes, stage queue age, compute time, retries, stale completion rejects, validation failures, review backlog, provenance completeness, publication lag, tenant denials, and deletion progress. Track automated quality with an appropriate labeled evaluation rather than presenting confidence as accuracy. Link one document's source, workflow, attempt, model, output, and reviewer through stable identities and privacy-safe telemetry.

← Return to question

UNIT 22 / S1

A booking payment succeeds after its seat hold expires. What outcomes are safe?

Claim seats atomically with the reservation, commit, and return its deadline. Payment workflow submits a stable provider operation. Confirmation checks a still-valid authorized transition and commits ticket issuance plus reservation/seat state. If payment succeeds after expiry, follow a defined recovery policy such as reacquisition if safe or refund; do not silently issue a ticket for a now-owned seat. Expiry uses conditional versioned release. Two users reading available must not both win. A process can crash after reservation commit but before reply; idempotency must recover it. Payment can succeed after the hold expires, and stale expiry work can arrive after a renewal or new reservation. Version-checked transitions and reconciliation prevent these from corrupting ownership. Email delivery failure must not imply ticket issuance failed if the ticket already committed.

← Return to question

UNIT 22 / S2

A payment provider charges, the local process dies, and a client retries. What prevents a second logical charge?

Commit intent and operation identity, dispatch a stable provider key, then record authoritative response or uncertainty. Apply settlement or capture evidence through an idempotent transition that creates balanced ledger postings once. Verified callbacks can arrive late or before a synchronous response and must not regress state incorrectly. Reconciliation queries stable provider references and repairs discrepancies without blindly resubmitting a charge. Deduplication, payment-state changes, and required local postings need appropriate atomicity. Provider effects are outside that local boundary and rely on provider identity and reconciliation contracts. Exactly-once local posting can be enforced with uniqueness and transaction semantics; exactly-once external execution is not implied. Refunds have their own lifecycle and may not perfectly reverse timing, fees, or downstream business consequences.

← Return to question

UNIT 22 / S3

A deleted document has an old extraction worker still running. What must prevent publication?

Verify upload and source digest, create a stable workflow, classify, extract, validate schema and semantic checks, record evidence, and either publish or request review. Human decisions create a new validated result version and publication intent. Each stage conditionally commits under its current attempt epoch. Reprocessing creates a new run over an identified source version rather than mutating old provenance. Deletion cancels or blocks inappropriate later publication. Source versions are immutable, workflow transitions and review decisions commit atomically where required, and derived artifacts identify their exact inputs. Stage delivery can repeat, but output identity and conditional commit make local progress safe. Search may lag publication while enforcing access. External model calls can be uncertain or nondeterministic; retain relevant version and request identity, validate output, and avoid claiming reproducibility beyond the actual model contract.

← Return to question

UNIT 22 / D1

Review the URL shortener at 100000 redirects/s with a cold cache and immediate revocation requirement.

At 10000 redirects/s and an assumed 90% cache hit ratio, nominal authoritative reads are 1000/s before coalescing and retries. Peak 100000 redirects/s implies 10000 misses/s if the ratio holds. Peak response traffic at 500 bytes is 50 MB/s before transport overhead. Failure planning must include a cold-cache case, not only this nominal miss rate. A code collision must cause an atomic uniqueness failure and retry, not overwrite the prior target. A cache outage must not flood storage without admission control. A lost creation reply must replay the committed operation result. A stale cache may serve a revoked link unless its policy limits that exposure. Analytics failure should reduce analytics freshness rather than redirect availability. The weakest assumptions are cache freshness during revocation, cold-cache origin capacity, and global uniqueness after partitioning. Review them with a revoke-under-load test, a full cache-loss exercise, and a forced code-collision test spanning the actual ownership boundary. Prefer an explicit bounded-degradation policy over a claim that cache replication automatically solves all three concerns.

← Return to question

UNIT 22 / I1

Choose one case and implement its final challenge with failure injection and an explicit invariant.

Each case ends with a specific miniature implementation challenge. Use stable identities, test the stated failure window, and compare the result with the case invariant. For the included examples, run verify_mechanisms.py and inspect the outbox and ledger fixtures. A success-only demonstration is insufficient because it does not exercise the proposed recovery boundary.

← Return to question

UNIT 22 / T1

You may fund only one improvement in a small platform: another region, tested restores, or stronger retry and reconciliation. How do you decide?

There is no universal winner. Start from the most harmful demonstrated or unacceptable failure and the original requirements. Another region cannot repair a lost idempotency history, and retries cannot restore a destroyed sole copy. Tested restores establish only their tested recovery scope. Compare risk reduction, cost, staffing, and the evidence each choice would produce; record the decision and revisit trigger.

← Return to question

THE REFERENCE DESK

A searchable glossary

Definitions and concept summaries link directly to their complete treatment, including individual patterns.

441 of 441 terms

Abstract Factory	A family factory constructs mutually compatible collaborators, such as a renderer and template engine for one notification format. Read the lesson →
Abstraction	Presenting the operations a caller needs while hiding implementation details; the contract still states the behaviour the caller relies on. Read the lesson →
Abstraction, encapsulation, polymorphism, inheritance	A payment class that validates an order, creates a provider SDK, calculates fees, and writes audit records hides several reasons to change. Read the lesson →
ACID properties	ACID separates atomicity, consistency, isolation, and durability. Read the lesson →
Active-active vs active-passive	Active-active serves from multiple locations or owners; active-passive keeps a standby for takeover. Read the lesson →
Adapter	An adapter translates an existing interface or representation into the contract its client needs. Read the lesson →

Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy	Structural patterns organize collaboration: Adapter translates an interface; Bridge separates two independently varying dimensions; Composite treats leaves and groups uniformly; Decorator adds behavior around a collaborator; Facade offers a focused entry point; Flyweight shares immutable intrinsic state; Proxy controls access to a target. Read the lesson →
ADR	Architecture Decision Record: a short record of a design choice, why it was made, its consequences, and when to reconsider it. Read the lesson →
Aggregates, entities, value objects, domain services	An aggregate is a consistency boundary whose root controls changes to its members. Read the lesson →
Alert design and alert fatigue	Alerts should identify actionable user harm or imminent loss of an objective with a clear owner and response. Read the lesson →
Amdahl's Law and bottlenecks	Amdahl's Law bounds ideal fixed-work speedup when only part of the work can be parallelized. Read the lesson →
Anti-corruption layers	An anti-corruption layer translates a foreign model into the local model so external terminology and quirks do not spread. Read the lesson →
API documentation and discoverability	Documentation should explain complete operation histories: request examples, errors, authorization, pagination, timeouts, idempotency, and asynchronous status. Read the lesson →
API fundamentals and interface contracts	An API is a communication contract, including inputs, outputs, side effects, error semantics, identity, deadlines, and compatibility. Read the lesson →

API gateways and BFF	A gateway provides shared edge concerns; a Backend for Frontend adapts APIs for a particular client experience. Read the lesson →
API security and common vulnerabilities	API risks include broken object authorization, excessive data exposure, unsafe resource consumption, injection, and server-side request abuse. Read the lesson →
API versioning and compatibility	Compatible evolution preserves supported client behavior. Read the lesson →
Application architecture	The organization and interaction of application responsibilities and their dependency rules. Read the lesson →
Architecture fitness functions	Architecture fitness functions continuously test required properties such as allowed imports, tenant-scoped queries, API compatibility, or recovery time. Read the lesson →
At-most-once and at-least-once processing	At-most-once processing can acknowledge before an effect and lose work after a crash. Read the lesson →
Audit trails	Audit trails record security-relevant actions with actor, scope, action, target, outcome, and relevant time or sequence. Read the lesson →
Authentication vs authorization	Authentication establishes identity; authorization decides permitted action on a particular resource in context. Read the lesson →
Authentication vs authorization at API boundaries	Authentication establishes the caller's identity under a trust model; authorization decides whether that caller may perform this operation on this resource. Read the lesson →

Autoscaling	Autoscaling can change instances, worker concurrency, or provisioned capacity. Read the lesson →
Autoscaling metrics	Autoscaling reacts to a measured signal, with delay and limits. Read the lesson →
Availability	Whether a system can provide the required service when needed. A measurement must define which requests and results count. Read the lesson →
Availability vs durability	Availability concerns useful serving behavior; durability concerns retained committed state under stated failures. Read the lesson →
Availability, reliability, durability, consistency	A reachable service can serve stale or lost data. Read the lesson →
Average vs median vs p95/p99 latency	A mean can hide users who wait much longer. Read the lesson →
B-tree/B+tree and hash indexing	B-tree families maintain ordered keys with bounded-height traversal; B+tree designs commonly keep data or record references in leaves linked for range scans. Read the lesson →
Backpressure	A mechanism that communicates or enforces downstream capacity limits on upstream work. Read the lesson →
Backpressure and replay	Backpressure controls admission or producer progress when consumers cannot keep up; replay deliberately revisits retained input. Read the lesson →

Backup, recovery, and durability	Backups provide recoverable historical state; replication keeps live copies and can replicate mistakes. Read the lesson →
Backups and restore testing	A backup must be restorable, authorized, decryptable, consistent enough for the application, and retained separately from the failure it protects against. Read the lesson →
Batch vs streaming pipelines	Batch processing operates on bounded sets; stream processing operates on continuing input with explicit progress and state. Read the lesson →
Benchmark methodology and statistical pitfalls	Benchmark pitfalls include unrealistic data, warmed caches without disclosure, coordinated omission, tiny samples, missing failures, and incompatible versions. Read the lesson →
Binary/object storage	Store large binary content separately from mutable processing metadata when their access and lifecycle needs differ. Read the lesson →
Blameless postmortems	A blameless postmortem records impact, timeline, detection, mitigation, and contributing system conditions without pretending decisions were made with hindsight. Read the lesson →
Blocking vs nonblocking I/O	Nonblocking I/O reports readiness or completion without holding the thread throughout the wait. Read the lesson →
Bottleneck identification	Bottleneck diagnosis follows the critical path and resource saturation. Read the lesson →

Boundary	A line that separates the parts being considered from their surroundings. A diagram should explain whether it marks a system, process, machine, or trust boundary. Read the lesson →
Bounded contexts and ubiquitous language	A bounded context defines where a particular model and vocabulary are valid. Read the lesson →
Bridge	A bridge separates two independent dimensions of variation, such as notification content types and delivery transports. Read the lesson →
Browser, CDN, application, and database caches	Browser, CDN, application, and database caches sit at different ownership and trust boundaries. Read the lesson →
Builder	A builder accumulates a complex construction request and validates it before creating a usable value. Read the lesson →
Bulkheads	Bulkheads partition resources so one class of work cannot consume everything. Read the lesson →
Business logic vs orchestration vs infrastructure	Business logic determines allowed outcomes, orchestration sequences collaborators, and infrastructure performs technical effects. Read the lesson →
Cache consistency and invalidation	Invalidation coordinates derived copies with authoritative changes. Read the lesson →
Cache locality and the working set	Temporal locality is reuse over time; spatial locality is reuse of nearby data. Read the lesson →

Cache partitioning and replication	Partitioning distributes keys; replication creates copies for resilience or reads under a protocol. Read the lesson →
Cache stampedes, penetration, and avalanche	A stampede occurs when many misses trigger duplicate expensive work. Read the lesson →
Cache warming and prefetching	Warming preloads likely useful entries; prefetching predicts near-future requests. Read the lesson →
Cache-aside, read-through, write-through, write-behind	Cache-aside loads after a miss; read-through delegates loading to the cache interface; write-through updates a backing store in the write path; write-behind defers backing persistence. Read the lesson →
Cancellation and structured concurrency	Cancellation must propagate to owned child work and release resources. Read the lesson →
CAP	Under a network partition, the theorem rules out simultaneously guaranteeing its formal consistency and availability properties. It is not a general “pick any two” menu. Read the lesson →
CAP theorem, with its actual assumptions	CAP's impossibility applies to a defined distributed read/write model under partitions: its formal atomic consistency and availability cannot both be guaranteed for every allowed execution. Read the lesson →
Capacity and growth planning	Plan for growth with scenarios rather than a single extrapolation. Read the lesson →

Capacity monitoring	Capacity monitoring observes rates, resource pressure, queue age, growth, and surviving capacity during failure. Read the lesson →
Capacity reservation vs elastic capacity	Reserved capacity can lower some unit rates in exchange for commitment and utilization risk; elastic capacity handles variable demand with provisioning delay and possible premiums. Read the lesson →
Cascading failures	Cascading failure occurs when one fault creates additional pressure and failures elsewhere: a slow provider holds connections, queues grow, callers retry, and formerly healthy dependencies become overloaded. Read the lesson →
CDN and edge networking	A CDN places cacheable representations near users; the origin remains responsible for authoritative content. Read the lesson →
CDN architecture and origin shielding	An edge cache serves representations near clients, while an origin shield can consolidate misses before the origin. Read the lesson →
Chain of Responsibility	A chain passes a request through ordered handlers that may handle, transform, reject, or forward it. Read the lesson →
Change Data Capture (CDC)	Change Data Capture emits committed changes from a source through a supported mechanism such as a transaction log. Read the lesson →
Chaos engineering	Chaos engineering tests hypotheses about behavior under controlled faults. Read the lesson →

Checksums and content addressing	A checksum detects accidental corruption under its algorithm's properties; a cryptographic digest can identify content with collision assumptions and appropriate verification. Read the lesson →
Choosing storage based on access patterns	Choose storage by the operations and invariants it must support. Read the lesson →
Choreography vs orchestration	Choreography lets participants react to events without one central workflow coordinator; orchestration stores and directs the workflow explicitly. Read the lesson →
CI/CD	CI checks changes; delivery prepares releasable artifacts; deployment moves an artifact into an environment. Read the lesson →
Circuit breakers	A circuit breaker tracks dependency failures and temporarily rejects or redirects calls under a policy. Read the lesson →
Clean Architecture	Clean Architecture places business policy inside dependencies on application-owned abstractions; framework and storage details depend inward. Read the lesson →
Cloud vs on-premises trade-offs	Cloud and on-premises choices distribute ownership differently. Read the lesson →
Cohesion	How closely the work inside a module belongs together. Read the lesson →
Command	A command represents an intention with explicit inputs and identity, enabling scheduling or audit. Read the lesson →

Command-query separation	Command-query separation distinguishes operations that change state from queries that report it. Read the lesson →
Component	A part of a system with a defined responsibility. Several components may run inside one process. Read the lesson →
Composite	A composite exposes a compatible interface for leaves and groups. Read the lesson →
Composition root	The startup code that creates the objects an application needs and passes them to the parts that use them. Read the lesson →
Composition roots and DI containers	The composition root builds the application graph in one comprehensible place. Read the lesson →
Composition vs inheritance	A subclass can violate assumptions inherited by callers: a ReadOnlyAccount subclass that throws from withdraw is not substitutable for a contract promising withdrawal. Read the lesson →
Compute, storage, networking, and telemetry costs	Total cost includes compute, storage, replicas, indexes, backups, networking, requests, telemetry, and operational work. Read the lesson →
Concurrency	Multiple activities in progress with potentially interleaved execution; parallel execution is not required. Read the lesson →
Concurrency vs parallelism	Concurrency means activities can be in progress over overlapping periods; parallelism means execution actually occurs simultaneously. Read the lesson →

Configuration and secret delivery	Configuration and secrets need versioned, observable delivery with validation. Read the lesson →
Conflict resolution and CRDT introduction	CRDTs arrange updates and merges so replicas converge under stated algebraic and delivery assumptions. Read the lesson →
Connection establishment and connection reuse	An existing connection avoids some setup work. Read the lesson →
Connection pool saturation	When a pool is full, waiting callers accumulate outside the database as well as inside it. Read the lesson →
Connection pooling and keepalive	A connection pool reuses a bounded set of connections. Read the lesson →
Consensus and replicated state machines	Consensus makes nonfaulty participants agree on a decision sequence under specified assumptions. Read the lesson →
Consistency model	A contract specifying which observations of shared or replicated state are allowed. Read the lesson →
Consistency models	A consistency model defines legal observation histories. Read the lesson →
Consistent hashing and rendezvous hashing	Consistent hashing places keys and owners in a mapping that changes only part of the assignment when membership changes; virtual nodes can improve distribution. Read the lesson →

Constructor, property/setter, and method injection	Constructor injection makes mandatory dependencies available throughout an object's valid lifetime. Read the lesson →
Consumer acknowledgements	A consumer acknowledgment permits the broker to advance or remove delivery state according to the protocol. Read the lesson →
Consumer-driven contracts	Consumer-driven contracts record assumptions that real consumers make about a provider. Read the lesson →
Container (C4)	An application or data store in a C4 diagram. It need not be a Docker or other operating-system container. Read the lesson →
Container fundamentals	A container packages an executable environment while runtime controls constrain CPU, memory, filesystem, and network access. Read the lesson →
Contract	The rules for how an operation behaves, including valid inputs, results, state changes, and errors. Read the lesson →
Control plane vs data plane	The control plane decides configuration and placement; the data plane serves application traffic. Read the lesson →
Controllers, services, repositories, adapters	Controllers translate transport, services coordinate application behavior, repositories mediate data, and adapters implement boundary protocols. Read the lesson →

Cost per request or transaction	Cost per request can obscure the business outcome if retries or failures inflate the denominator. Read the lesson →
Cost, security, operational complexity, and maintainability as constraints	Capacity is not the only constraint. Read the lesson →
Cost-performance optimization	Cost-performance optimization seeks a feasible point under service, security, and maintainability constraints. Read the lesson →
Coupling	The dependencies between parts of a system: what one part must know about or rely on in another. Read the lesson →
CPU, memory, disk, network, and database bottlenecks	CPU, memory, disk, network, and databases can each limit useful work. Read the lesson →
CPU-bound vs I/O-bound workloads	CPU-bound work spends its limiting time computing; I/O-bound work spends much of it waiting on external operations. Read the lesson →
CQRS	Separate command and query models when they need different behavior or shape. Read the lesson →
Cursor vs offset pagination	Offset pagination skips a number of rows and is easy to express, but inserts before the offset can create duplicates or omissions between pages. Read the lesson →

Dashboards	A dashboard should answer a decision question: is the service meeting its objective, what changed, and where is capacity constrained? Put user outcomes before infrastructure vanity metrics. Read the lesson →
Data consistency validation	Data validation compares source and destination identities, versions, aggregates, and domain invariants. Read the lesson →
Data lakes and warehouses	A data lake stores flexible datasets, often in object storage; a warehouse provides curated analytical structures and serving capabilities. Read the lesson →
Data minimization	Data minimization retains only information required for a defined purpose. Read the lesson →
Data ownership	Data ownership names who may define and change a fact. Read the lesson →
Data provenance and lineage	Provenance records where an artifact came from; lineage connects transformations between artifacts. Read the lesson →
Data quality, schema validation, and replay	Data quality checks validate shape, ranges, identity, relationships, and completeness against explicit contracts. Read the lesson →
Data retention and lifecycle	Retention is an explicit lifecycle, not a scheduled delete statement added at the end. Read the lesson →
Database connection pooling	A database pool caps active connections and queues acquisition. Read the lesson →

Database proxying	A database proxy can pool, route, or multiplex connections according to implementation. Read the lesson →
Deadlocks, starvation, livelocks	Deadlock is a cycle of dependencies that prevents progress; starvation repeatedly denies a participant needed service; livelock keeps participants active without useful progress. Read the lesson →
Decision matrices and trade-off analysis	A decision matrix makes judgments visible but does not turn them into measurements. Read the lesson →
Decorator	A decorator preserves a collaborator interface while adding behavior around calls, such as metrics or formatting. Read the lesson →
Defense in depth	Defense in depth places complementary controls around a risk: tenant-scoped queries, database permissions, signed object access, worker isolation, and audit evidence can all reduce cross-tenant harm. Read the lesson →
Delivery guarantees	Delivery terminology must identify where messages and effects are counted. Read the lesson →
Dependency cycles and captive dependencies	A dependency cycle A→B→A often signals mixed responsibilities or missing coordination; breaking it with a service locator can hide rather than solve the model. Read the lesson →
Dependency injection	Supplying collaborators to an object or function rather than having it locate or construct them internally. Read the lesson →
Dependency Injection (DI)	Dependency Injection supplies a collaborator rather than constructing or locating it internally. Read the lesson →

Dependency inversion principle vs DI	Dependency inversion is a design principle about which knowledge depends on which abstraction. Read the lesson →
Dependency rules and module boundaries	Dependency rules constrain which modules can know about others. Read the lesson →
Dependency, dependency direction, and inversion	A notification service that imports a concrete email SDK knows both business behavior and infrastructure details. Read the lesson →
Deployment topologies and networking	Deployment topology places processes, state, trust boundaries, routes, and failure domains on actual infrastructure. Read the lesson →
Designing data-intensive applications	A data-intensive design is a network of state transformations, not a collection of product logos. Read the lesson →
Development vs staging vs production	Development, staging, and production serve different purposes but need deliberate compatibility. Read the lesson →
DI vs factory pattern	A factory encapsulates creation or selection; DI supplies an already selected collaborator or a factory itself. Read the lesson →
DI vs service locator	DI makes collaborators visible in construction or function parameters. Read the lesson →
Dirty reads, nonrepeatable reads, phantom reads	A dirty read observes uncommitted changes; a nonrepeatable read sees a previously read row change; a phantom changes the set matching a predicate. Read the lesson →

Disposal, cleanup, and resource lifecycles	Cleanup must run on success, failure, and cancellation. Read the lesson →
Distributed coordination	Coordination establishes shared decisions such as ownership, configuration, or ordered metadata. Read the lesson →
Distributed locks and leases	A lock grants a form of exclusive permission; a lease grants it for a time under clock and expiry assumptions. Read the lesson →
Distributed monolith antipattern	A distributed monolith has several processes but little independent change or operation. Read the lesson →
Distributed tracing	A distributed trace links spans representing work and waits across components. Read the lesson →
DNS resolution and DNS caching	A resolver obtains records for a name, often through caches and authoritative servers. Read the lesson →
Document databases	A document database groups nested data under a document identity and commonly permits flexible structures. Read the lesson →
Domain events	A domain event records a meaningful fact such as OrderPlaced, not a vague instruction to “do something.” Include stable identity, time semantics, and enough versioned information for intended consumers without exposing every internal field. Read the lesson →
Domain models, entities, value objects	An entity has identity across changes; a value object is defined by its value; a domain model expresses meaningful rules. Read the lesson →

Domain-Driven Design (DDD)	Domain-Driven Design builds a shared model of a complex business with domain experts. Read the lesson →
DRY, KISS, YAGNI	DRY avoids duplicated knowledge, not every repeated line. Read the lesson →
Durability	Whether committed data survives the specified failures, such as a process restart or loss of a disk. Read the lesson →
Durable job execution	Durable jobs have a persistent identity, input reference, state, attempt history, and ownership protocol. Read the lesson →
Encapsulation	Controlling access to internal data so callers use the operations that protect its rules. Read the lesson →
Encryption in transit and at rest	Encryption in transit protects communication under a protocol; encryption at rest protects stored bytes under key and access assumptions. Read the lesson →
ER diagrams and relational modeling	An ER model identifies entities, relationships, cardinalities, and constraints. Read the lesson →
Error budget	The allowed unreliability implied by a chosen SLO over its measurement window. Read the lesson →
ETL vs ELT	ETL transforms before loading into the target; ELT loads and transforms in the target environment. Read the lesson →

Evaluating a design against its original requirements	Validate the design against its original requirements using a traceable evidence matrix. Read the lesson →
Event loops, tasks, futures, promises	An event loop runs ready callbacks or task continuations. Read the lesson →
Event sourcing	Event sourcing stores domain events as the authoritative history from which state is derived. Read the lesson →
Event-driven architecture	Event-driven architecture connects components through events with independent processing. Read the lesson →
Eventual consistency	A convergence property under the model's assumptions; it alone gives neither an immediate freshness bound nor a complete conflict policy. Read the lesson →
Eviction: LRU, LFU, FIFO, and alternatives	LRU favors recent access; LFU favors frequent access; FIFO evicts insertion order. Read the lesson →
Evolutionary architecture	Evolutionary architecture changes in response to evidence while protecting important properties. Read the lesson →
Exactly-once processing	A scoped guarantee about effects or records within a defined system boundary and assumptions. It does not automatically include arbitrary external side effects. Read the lesson →
Exactly-once processing within defined boundaries	Exactly-once claims require a precise boundary. Read the lesson →

Facade	A facade offers a focused entry point to a subsystem, such as submit_notification() coordinating validation, templating, and durable intent. Read the lesson →
Factory Method	A creator varies the product it constructs through an overridable or supplied creation operation. Read the lesson →
Factory Method, Abstract Factory, Builder, Prototype, Singleton	Creational patterns organize construction. Read the lesson →
Failover and failback	Failover transfers service or authority to a replacement; failback returns responsibility after repair. Read the lesson →
Failure detection	Failure detection observes missing or delayed evidence and produces suspicion under timing assumptions. Read the lesson →
Failure domain	Elements exposed to a common cause of failure, such as a process, host, disk, or region. Read the lesson →
Failure domains and fault isolation	A failure domain groups elements exposed to a common cause: a process, host, zone, dependency, or administrative action. Read the lesson →
Fallbacks	A fallback substitutes an alternative behavior with an explicit contract. Read the lesson →

Fault models	A fault model states what kinds of failure a protocol tolerates: crash-stop, crash-recovery, message loss, partitions, or arbitrary Byzantine behavior. Read the lesson →
Fault-injection tests	Fault injection tests behavior when a dependency is slow, unavailable, inconsistent, or interrupted. Read the lesson →
Feature flags	Feature flags separate some release and exposure decisions. Read the lesson →
Flyweight	A flyweight shares immutable intrinsic state while keeping per-use context external. Read the lesson →
Functional programming alternatives to object-centric design	A functional design can represent payment decisions as pure transformations of values, with effects interpreted at the boundary. Read the lesson →
Gathering and prioritizing requirements	A request to build a feed hides several products: reading posts, publishing, moderating, and following. Read the lesson →
Golden signals	Golden signals commonly include latency, traffic, errors, and saturation. Read the lesson →
Graceful degradation	Graceful degradation preserves a useful subset of behavior under failure. Read the lesson →
Graph databases	Graph storage makes relationships and traversals first-class, useful when variable-depth relationships are central to queries. Read the lesson →

Health checking	Health checks should answer a useful local question and be bounded. Read the lesson →
Health checks and failover	Health checks approximate readiness, not omniscient correctness. Read the lesson →
Hexagonal/Ports and Adapters	Hexagonal architecture names inbound and outbound ports with adapters around the application. Read the lesson →
High cohesion and low coupling	Cohesion improves when related payment rules live together; coupling falls when business code does not depend on provider-specific field names. Read the lesson →
HLD	High-Level Design: a view of a system's main parts, where data is kept, and how the parts work together. Read the lesson →
Horizontal scaling economics	Horizontal scaling adds coordination, baseline replicas, discovery, and often network traffic. Read the lesson →
Hot keys	A hot key concentrates traffic on one shard or owner. Read the lesson →
Hot partitions and skew	Hot partitions arise from uneven key popularity, data size, or write concentration. Read the lesson →
How databases store and retrieve records	A database translates logical records into physical structures and coordinates access while preserving its promises. Read the lesson →

How the internet works from the application perspective	Entering a URL begins a chain involving name resolution, a route, transport, a secure application exchange, server execution, and rendering. Read the lesson →
HTTP/1.1, HTTP/2, HTTP/3	HTTP/1.1 often needs multiple connections to avoid response blocking among requests. Read the lesson →
Idempotency	Repeated application of an operation has the same relevant effect as one application, under its defined identity and scope. Read the lesson →
Idempotency and idempotency keys	An idempotency key identifies one logical operation across attempts. Read the lesson →
Idempotent consumers	An idempotent consumer records event identity and its relevant local effect atomically. Read the lesson →
Immutability and side-effect management	Immutable values make state transitions easier to inspect: compute a new PaymentSummary from an old value rather than mutating a shared object in several helpers. Read the lesson →
Immutable infrastructure	Immutable infrastructure replaces instances from defined artifacts rather than relying on undocumented manual changes. Read the lesson →
In-process vs distributed caches	An in-process cache is fast and simple but creates independent copies per application instance. Read the lesson →
Incident analysis and prevention	Incident analysis reconstructs the timeline, contributing conditions, detection gaps, and recovery decisions without reducing the explanation to individual blame. Read the lesson →

Incident response	Incident response coordinates mitigation, communication, evidence, and decision ownership. Read the lesson →
Index selection and query optimization	An index speeds selected access paths while consuming space and write work. Read the lesson →
Indexing pipelines	An indexing pipeline transforms authoritative content into searchable representations. Read the lesson →
Infrastructure as Code	Infrastructure as Code describes desired resources and policy in reviewable form. Read the lesson →
Infrastructure ownership and managed services	A managed service transfers selected tasks, not business responsibility. Read the lesson →
Interface	The operations a component offers to its callers. Its contract explains how those operations behave. Read the lesson →
Interfaces and contracts	A gateway interface must specify currency, amount units, timeouts, duplicate identity, and error meaning. Read the lesson →
Invariant	A rule that must hold whenever the system is in a valid state, such as unique task identifiers. Read the lesson →
Inversion of Control (IoC)	Inversion of Control means some control is delegated to a framework or external coordinator: an event loop invokes your callback, or a host constructs your request handler. Read the lesson →

IP addresses, ports, sockets, routing	An IP address identifies a network endpoint within a routing context; a port identifies a transport endpoint at that host. Read the lesson →
Isolation levels and implementation differences	Isolation levels specify allowed concurrency behavior, but engines implement them differently. Read the lesson →
Iterator	An iterator exposes traversal without requiring clients to know the collection representation. Read the lesson →
Key management and rotation	Key management includes generation, storage, access, rotation, versioning, backup, and destruction. Read the lesson →
Key-value databases	A key-value store maps keys to values under a defined read/write contract. Read the lesson →
Kubernetes fundamentals	Kubernetes reconciles declared objects with cluster state through its control plane and node agents. Read the lesson →
L4 vs L7 load balancing	Layer-4 routing selects using transport information and may pass encrypted traffic through. Read the lesson →
Lamport clocks and vector clocks	Lamport clocks advance locally and on receive using max(local,received)+1. Read the lesson →
Large-file upload architecture	A large-file architecture can issue a scoped upload authorization so the client streams directly to storage, while the application owns metadata and authorization. Read the lesson →

Latency	Elapsed time for an operation under a specified measurement boundary. Read the lesson →
Latency caused by network round trips	Round trips create a latency floor even before useful computation. Read the lesson →
Latency, throughput, QPS, RPS, TPS	Latency measures elapsed time; throughput measures completed work per time interval. Read the lesson →
Law of Demeter	The Law of Demeter discourages reaching deeply through another object's internal graph. Read the lesson →
Layered/N-tier architecture	Layered architecture separates presentation, application behavior, domain policy, and infrastructure at a chosen granularity. Read the lesson →
Lazy initialization	Lazy initialization defers creation until the resource is needed. Read the lesson →
Leader/follower replication	Leader/follower replication sends an ordered stream from one leader to followers. Read the lesson →
Leaderless replication	Leaderless designs let clients or coordinators contact several replicas and reconcile returned versions. Read the lesson →
Least privilege	Least privilege limits identities to the resources and operations required for their work. Read the lesson →
Lifecycle policies and retention	Lifecycle policies should follow every copy: raw files, extracted text, thumbnails, indexes, logs, and backups. Read the lesson →

Linearizability	A correctness condition in which operations appear atomic and respect real-time precedence. Read the lesson →
Little's Law and queueing intuition	Little's Law relates long-run averages across the same stable boundary: L = lambda W. Read the lesson →
LLD	Low-Level Design: the internal detail of a component, including its functions, classes, interfaces, and algorithms. Read the lesson →
Load balancers vs API gateways	A load balancer distributes traffic among backends; an API gateway commonly adds API-specific policy such as authentication integration, quotas, and request transformation. Read the lesson →
Load distribution algorithms	Round-robin spreads selections, least-connections approximates active load, weighted policies reflect capacity, and hashing can preserve affinity. Read the lesson →
Load shedding	Load shedding rejects lower-value or excessive work before resources are exhausted. Read the lesson →
Load, stress, spike, and soak testing	Load tests exercise expected demand; stress tests explore limits; spike tests expose response to abrupt change; soak tests reveal long-duration issues such as leaks or compaction. Read the lesson →
Locking and optimistic concurrency control	Pessimistic locking coordinates access before conflicting work; optimistic concurrency validates that assumptions still hold before committing. Read the lesson →
Log ordering and durability	A replicated log needs both an ordering rule and a durability rule. Read the lesson →

Logs, metrics, and traces	Logs record discrete events, metrics aggregate measurements, and traces relate operations along a path. Read the lesson →
Lost updates, write skew, serialization anomalies	Lost updates overwrite another writer's result; write skew violates a multi-row invariant after transactions update different rows; serialization anomalies are histories without an equivalent serial ordering. Read the lesson →
LSM trees and write amplification	An LSM-based engine buffers writes, flushes sorted runs, and compacts them into a searchable organization. Read the lesson →
Mediator	A mediator coordinates collaborators that would otherwise communicate through many direct relationships. Read the lesson →
Membership changes	Membership changes alter which groups can make decisions. Read the lesson →
Memento	A memento captures state for a supported restoration boundary without exposing arbitrary internals. Read the lesson →
Memory visibility and happens-before relationships	Visibility is not established by intuition about elapsed time. Read the lesson →
Memory, stack, heap, and object lifetimes	A stack records call-local execution state; a heap stores dynamically managed objects under the runtime's allocation model. Read the lesson →

Metadata vs binary data	Binary bytes and metadata have different update patterns. Read the lesson →
Modular monoliths	A modular monolith is deployed as a unit while its modules maintain explicit boundaries and ownership. Read the lesson →
Modularity	Organising a system into parts with clear responsibilities and interfaces. Read the lesson →
Monitoring vs observability	Monitoring tracks known signals and conditions; observability supports explaining system behavior from outputs and instrumentation. Read the lesson →
Monolith vs microservices	Microservices add independently deployable processes and data ownership boundaries, along with network failures, version skew, and distributed operations. Read the lesson →
Multi-environment configuration	Environment configuration should change endpoints, credentials, capacity, and declared policy without silently changing business semantics. Read the lesson →
Multi-leader replication	Multi-leader replication allows writes at several authorities, often reducing local write latency or supporting disconnected regions. Read the lesson →
Multi-region replication trade-offs	Regional replication must define write authority, conflict behavior, acknowledgment geography, and disaster recovery. Read the lesson →
Multi-region strategies	Multi-region designs trade latency, data placement, conflict, cost, and failure behavior. Read the lesson →

Multi-tenant isolation	Tenant isolation must survive every path: API lookup, cache key, job queue, object URL, search filter, export, telemetry, and administrative action. Read the lesson →
Multi-zone and multi-region deployments	Multi-zone and multi-region deployments protect against different correlated failures. Read the lesson →
Multipart and resumable uploads	Multipart upload splits a file into identifiable parts with explicit completion. Read the lesson →
Mutexes, semaphores, atomics, condition variables	A mutex grants exclusive access; a semaphore limits concurrent permits; an atomic operation provides defined indivisibility and ordering for specified operations; a condition variable coordinates waiting for a predicate while releasing a lock. Read the lesson →
MVCC	Multiversion concurrency control: retaining versions to help coordinate concurrent database access. Exact visibility and isolation depend on implementation. Read the lesson →
NAT, firewalls, ingress, egress	NAT rewrites addressing, firewalls apply traffic policy, ingress enters a boundary, and egress leaves it. Read the lesson →
Negative caching	Negative caching stores an absence or error under a narrow policy. Read the lesson →
Network partitions, timeouts, packet loss	A timeout means the caller stopped waiting; it does not reveal whether the remote side executed. Read the lesson →
Network segmentation	Network segmentation limits reachable paths and reduces some lateral movement. Read the lesson →

Network uncertainty	A slow network and a failed node can look the same to a participant for a while. Read the lesson →
Nonfunctional requirement	A requirement about the quality or conditions of a system's behaviour, such as response time or persistence. Read the lesson →
Normalization vs denormalization	Normalization avoids redundant facts and update anomalies; denormalization deliberately stores derived or repeated data to serve a read path. Read the lesson →
OAuth 2.0 and OpenID Connect	OAuth 2.0 concerns delegated authorization; OpenID Connect adds an identity layer. Read the lesson →
Object collaboration	An object collaboration describes messages and ownership, not just fields. Read the lesson →
Object creation vs object ownership	Creation, reference, and ownership are different relationships. Read the lesson →
Object storage vs block vs file storage	Object storage exposes named blobs and metadata; block storage exposes addressable blocks; file storage exposes a filesystem namespace and file operations. Read the lesson →
Observability across asynchronous workflows	Asynchronous observability follows a business operation beyond the original request. Read the lesson →
Observer	An observer subscribes to notifications from a subject. Read the lesson →

OLTP vs OLAP	OLTP emphasizes many short transactions over current operational state; OLAP emphasizes analytical scans and aggregation over larger histories. Read the lesson →
On-call and runbooks	On-call needs actionable alerts, access, runbooks, escalation, and sustainable ownership. Read the lesson →
Onion Architecture	Onion Architecture also emphasizes dependencies toward domain policy, often organizing domain, application services, and infrastructure as concentric responsibilities. Read the lesson →
Ordering guarantees	Ordering can mean publication order, per-key log order, consumer start order, or effect-commit order. Read the lesson →
PACELC as a useful extension	PACELC highlights two questions: during a partition, availability versus consistency; otherwise, latency versus consistency under the considered design. Read the lesson →
Pages, records, indexes, and buffer pools	A page is a fixed-size storage unit in many engines; records occupy pages; indexes point toward searchable values or records; a buffer pool caches useful pages. Read the lesson →
Pagination, filtering, sorting	Pagination bounds work and response size. Read the lesson →
Partial failures	A partial failure affects some participants or paths while others keep operating. Read the lesson →
Partitioning and sharding	Partitioning divides a dataset; sharding commonly distributes those partitions across separate storage owners. Read the lesson →

Partitions, consumer groups, offsets	Partitions provide ordered lanes in systems such as Kafka; consumer groups commonly distribute partitions among members, and offsets represent progress within a lane. Read the lesson →
Password hashing	Passwords require purpose-built password hashing with salt and an appropriate work factor, not fast general-purpose hashing or reversible encryption. Read the lesson →
Paxos: conceptual foundations	Paxos uses numbered proposals and intersecting majorities to ensure that once a value is chosen, later successful proposals preserve it under the protocol. Read the lesson →
Peak vs average load	Average traffic hides synchronized arrivals. Read the lesson →
Performance budgets	A performance budget allocates acceptable latency, resource, and cost across an operation under a named workload. Read the lesson →
Performance tuning vs architecture changes	Tuning improves an implementation within its architecture; an architectural change alters ownership, placement, or major interaction. Read the lesson →
Physical vs logical clocks	Physical clocks estimate real time; logical clocks represent ordering constraints. Read the lesson →
Platform engineering basics	Platform engineering provides reusable paths for teams to build and operate services safely. Read the lesson →
Poison messages	A poison message repeatedly fails because of malformed data, incompatible schema, or an invariant violation. Read the lesson →

Polyglot persistence and its operational costs	Polyglot persistence gives different workloads suitable engines at the cost of more operating boundaries. Read the lesson →
Primary/replica architectures	A primary/replica design gives one participant authority for a class of writes and copies its state elsewhere. Read the lesson →
Privacy, retention, and deletion	Privacy lifecycle includes collection, access, correction, retention, and deletion across raw and derived copies. Read the lesson →
Producers, consumers, brokers	Producers create messages, brokers accept or route them, and consumers interpret them. Read the lesson →
Profiling	Profiling samples or instruments CPU, allocation, blocking, and other runtime behavior. Read the lesson →
Profiling and benchmarking	Profiling identifies where runtime work occurs; benchmarking compares behavior under a controlled workload. Read the lesson →
Program vs process vs thread	A program is code; a process is an executing instance with an address space and resources; threads are execution contexts within a process. Read the lesson →
Prototype	A prototype creates an object by copying an existing configured instance. Read the lesson →
Proxy	A proxy implements a target interface while controlling access, placement, or initialization. Read the lesson →

Public vs internal APIs	An internal API still crosses a trust, version, or failure boundary. Read the lesson →
Query and application performance diagnosis	Diagnose a slow application by splitting waiting from execution. Read the lesson →
Query planning and EXPLAIN	A query planner selects a physical plan from logical operations using statistics and cost estimates. Read the lesson →
Queue saturation and throughput ceilings	Throughput approaches a ceiling when a limiting resource cannot complete work faster. Read the lesson →
Queueing and overload	A queue absorbs temporary mismatch, not sustained overload. Read the lesson →
Queues vs logs vs pub/sub	A queue commonly distributes work among consumers; a retained log preserves an ordered append history for replay; pub/sub distributes publications to subscribers under its specific retention and acknowledgment semantics. Read the lesson →
Quorum	A set of participants whose responses meet a protocol's decision rule. Intersection alone does not establish all read or write correctness properties. Read the lesson →
Quorum-based systems	A quorum is a permitted decision set under a protocol. Read the lesson →
Quorums and read/write trade-offs	With fixed N replicas, R+W>N ensures a read set intersects a write set; W>N/2 ensures two write quorums intersect. Read the lesson →

Race conditions and critical sections	A read-modify-write sequence can lose updates when its steps interleave. Read the lesson →
Raft: terms, elections, log replication, commit	Raft separates leader election, log replication, and safety. Read the lesson →
Range partitioning vs hash partitioning	Range partitioning preserves ordered locality and can make time-based retention efficient, but a newest range may receive most writes. Read the lesson →
Rate limiting	Rate limiting protects a boundary from excessive traffic under a defined identity and burst policy. Read the lesson →
Rate limits and quotas	A rate limit controls request pace; a quota limits accumulated use over a period or resource dimension. Read the lesson →
Read replicas and read/write splitting	Read replicas can distribute suitable reads while the primary retains writes. Read the lesson →
Read-heavy vs write-heavy systems	A read/write ratio is meaningful only at a named boundary. Read the lesson →
Read-your-writes and monotonic reads	Read-your-writes keeps a session from losing visibility of its own completed updates; monotonic reads prevent that session from moving backward in observed progress. Read the lesson →
Rebalancing	Rebalancing moves data or ownership while serving traffic. Read the lesson →

Reconciliation after partial failure	Reconciliation compares intended and observed outcomes after partial failure. Read the lesson →
RED and USE methods	RED focuses on rate, errors, and duration for request-serving components; USE focuses on utilization, saturation, and errors for resources. Read the lesson →
Redis-like data structures	Redis-like structures include strings, hashes, lists, sets, sorted sets, and streams, each with different operations and cost models. Read the lesson →
Redundancy	Redundancy adds spare components or copies, but their usefulness depends on independence, capacity, detection, and state correctness. Read the lesson →
Refactoring and recognizing code smells	Refactoring preserves intended behavior while improving structure. Read the lesson →
Referential integrity	Referential integrity ensures a stored relationship refers to an allowed target under the chosen constraint. Read the lesson →
Regression testing	Regression testing preserves important behavior as implementations change. Read the lesson →
Relational databases and SQL	A relational model represents facts as relations with keys and constraints; SQL expresses queries and changes over that model. Read the lesson →
Replication	Replication maintains copies under an acknowledgment and reconciliation protocol. Read the lesson →

Replication lag	Replication lag is the distance between source and replica state, measured by time, positions, or application version. Read the lesson →
Repository	A repository mediates domain-oriented persistence rather than exposing storage syntax everywhere. Read the lesson →
Repository, Unit of Work, Specification, CQRS, and dependency boundaries	Repository mediates domain-oriented access to persistent data; Unit of Work groups changes under one commit decision; Specification represents a reusable predicate; CQRS separates command and query models where their needs justify it. Read the lesson →
Request scopes, session scopes, and custom scopes	A request scope lasts for one request, while a session scope can span many requests and hold different privacy or memory risks. Read the lesson →
Request/response schemas	Validate structural shape and business meaning separately. Read the lesson →
Resource exhaustion and backpressure	Unlimited concurrency converts downstream slowness into memory and connection exhaustion. Read the lesson →
Resource modeling	Resources should reflect stable business identities. Read the lesson →
REST, GraphQL, gRPC, RPC	REST organizes resource-oriented interactions using HTTP semantics; GraphQL exposes a typed query graph; gRPC commonly offers schema-defined RPC with streaming; RPC generally models operations. Read the lesson →

Retries and dead-letter queues	Retry transient failures with a bounded policy and delay; dead-letter persistent failures with context and a path for repair. Read the lesson →
Retries and timeouts	Retries should fit an overall deadline and target transient, safe-to-repeat work. Read the lesson →
Retries, exponential backoff, jitter	Retries can recover transient faults but spend capacity. Read the lesson →
Retry storms	A retry storm amplifies an outage when many callers repeat work against a struggling dependency. Read the lesson →
Reverse proxies vs forward proxies	A forward proxy acts on behalf of clients; a reverse proxy fronts servers. Read the lesson →
Role-based and attribute-based access control	RBAC grants permissions through roles; ABAC evaluates attributes such as tenant, resource classification, ownership, and context. Read the lesson →
Rollbacks and database migrations	Rollback restores code or configuration only when state and contracts remain compatible. Read the lesson →
Rolling, blue-green, and canary deployments	Rolling deployment replaces instances gradually; blue-green prepares a parallel environment and switches traffic; canary exposes a controlled subset before wider rollout. Read the lesson →
Root-cause investigation	Root-cause investigation tests explanations against evidence and distinguishes triggers from contributing conditions. Read the lesson →

Row-oriented vs column-oriented storage	Row layouts keep fields of a record together; column layouts keep a field's values together across many records. Read the lesson →
RPO	Recovery Point Objective: an objective for the amount of data loss measured in time. Read the lesson →
RTO	Recovery Time Objective: an objective for restoring service after disruption. Read the lesson →
RTO and RPO	RTO is the recovery time objective; RPO is the acceptable data-loss exposure expressed in time. Read the lesson →
Safety vs liveness	Safety excludes bad outcomes; liveness requires progress under stated conditions. Read the lesson →
Saga	A coordinated sequence of local transactions with compensating actions; compensation need not be a perfect inverse. Read the lesson →
Sagas and compensating transactions	A saga coordinates local transactions and compensating actions across ownership boundaries. Read the lesson →
Sampling and telemetry cost	Sampling and retention control telemetry cost but affect which questions can be answered. Read the lesson →
Scale-out economics	Scale-out economics includes duplicate baseline capacity, coordination, networking, telemetry, cache warmup, and operational work. Read the lesson →

Scaling writes vs scaling reads	Read scaling often replicates or caches immutable or sufficiently stale-tolerant data. Read the lesson →
Scheduling and context switching	A scheduler decides which runnable execution context receives CPU time. Read the lesson →
Scheduling and service discovery	A scheduler assigns work to eligible resources according to constraints; service discovery exposes reachable service endpoints. Read the lesson →
Scheduling, leases, and worker coordination	Scheduling combines a durable due-time record with worker coordination. Read the lesson →
Schema evolution	Schema evolution applies to stored data and events as well as synchronous APIs. Read the lesson →
Search engines and inverted indexes	An inverted index maps a term to postings describing documents or positions containing it. Read the lesson →
Search indexes	A search index stores representations optimized for retrieval rather than primary transactional ownership. Read the lesson →
Search ingestion and reindexing	Reindexing builds a new representation while preserving serving behavior. Read the lesson →
Secrets management	Secrets need an owner, distribution mechanism, access policy, rotation, and revocation procedure. Read the lesson →

Secure-by-design reviews	A secure-by-design review follows one sensitive operation end to end. Read the lesson →
Security vs usability trade-offs	Security and usability interact through real user behavior. Read the lesson →
Sequential and causal consistency	Sequential consistency preserves each participant's program order in some shared ordering without necessarily honoring real-time order across participants. Read the lesson →
Serializability	A transactional correctness condition requiring equivalence to some serial execution; real-time order is a separate issue. Read the lesson →
Serializability vs linearizability	Serializability concerns equivalence of transactions to a serial execution; linearizability concerns real-time atomic behavior of operations. Read the lesson →
Serializable execution and serializability	Serializable means the committed effect is equivalent to some serial transaction order, not necessarily wall-clock order. Read the lesson →
Serverless and function-oriented architectures	Serverless functions can provide managed invocation and elastic execution, but do not remove systems design. Read the lesson →
Serverless compute	Serverless compute delegates some provisioning and execution management to a platform while preserving limits and failure semantics. Read the lesson →
Service discovery	Service discovery maps a logical service to reachable instances. Read the lesson →

Sessions, cookies, tokens	Sessions commonly store server-side state referenced by a credential; tokens can carry claims verified under a protocol. Read the lesson →
Shared memory vs message passing	Shared memory allows direct access but needs synchronization; message passing transfers information between owners and requires protocol decisions. Read the lesson →
Single points of failure	A single point of failure can be a machine, database, credentials service, router, operator action, or shared dependency. Read the lesson →
Singleton	A singleton restricts instance access within a specified program or container boundary. Read the lesson →
SLI / SLO / SLA	Indicator: measured behavior. Objective: target for that behavior. Agreement: an explicit service agreement, potentially with consequences. Read the lesson →
SLI, SLO, SLA, and error budgets	Choose a user-visible SLI, an objective and window, then describe any contractual SLA separately. Read the lesson →
SLIs, SLOs, error budgets	An SLI measures a chosen service behavior; an SLO sets a target; an error budget quantifies the allowed shortfall over a window. Read the lesson →
SLO-based reliability decisions	Reliability effort should follow user-visible objectives and error-budget evidence. Read the lesson →
SOLID principles, including their limitations	SOLID is a set of design heuristics: single responsibility, open/closed extension, behavioral substitution, focused interfaces, and dependency inversion. Read the lesson →

Specification	A specification names a reusable predicate or selection rule, such as eligible_for_refund(payment,now). Read the lesson →
Split-brain prevention and fencing tokens	A fencing token is a monotonically ordered ownership epoch checked by the resource receiving an effect. Read the lesson →
Split-brain situations	Split brain means participants act as conflicting authorities. Read the lesson →
Stale-while-revalidate	Stale-while-revalidate serves an allowed stale value while one refresh updates it. Read the lesson →
State	State objects or transition tables vary behavior by an entity lifecycle. Read the lesson →
Stateless vs stateful services	A stateless service retains no required per-client conversational state locally between requests; it still operates on state elsewhere. Read the lesson →
Status codes and error models	HTTP status codes describe the protocol outcome, while an operation body may describe asynchronous business state. Read the lesson →
Sticky sessions	Sticky sessions keep a client associated with an instance and can support local state or reduce cache misses. Read the lesson →
Storage, memory, bandwidth, and CPU estimates	Assume ten million daily users, fifty API requests each, one 2 kB post per user daily, and ten 200 kB images viewed each. Read the lesson →

Strategy	A strategy supplies an interchangeable algorithm behind a stable behavioral contract, such as channel selection or fee calculation. Read the lesson →
Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor	Behavioral patterns organize decisions and coordination. Read the lesson →
Stream processing fundamentals	Stream processing maintains results over ongoing events using windows, keys, and state. Read the lesson →
Streaming APIs and long-running operations	A long-running operation can return an operation resource and let clients poll or subscribe for progress. Read the lesson →
Strong consistency and eventual consistency	Strong consistency is often used informally for linearizable behavior, but must be defined in context. Read the lesson →
Strongly consistent metadata services	Strongly consistent metadata services can coordinate configuration, ownership epochs, and small control-plane decisions. Read the lesson →
Structured logging	Structured logging uses named fields with stable meanings rather than relying only on free-form messages. Read the lesson →
Supply-chain security	Supply-chain security covers dependencies, build tools, artifacts, deployment permissions, and update paths. Read the lesson →

Sustainability and resource efficiency	Resource efficiency includes avoiding idle work, unnecessary transfer, redundant computation, and over-retention. Read the lesson →
Synchronous vs asynchronous communication	Synchronous communication makes the caller wait for the result; asynchronous communication records or sends work whose completion is observed later. Read the lesson →
Synchronous vs asynchronous execution	Synchronous execution ties the caller's progress to completing an operation; asynchronous execution separates initiation from later completion. Read the lesson →
Synchronous vs asynchronous replication	Synchronous replication waits for specified acknowledgments before completing a write; asynchronous replication allows acknowledgment earlier. Read the lesson →
Tail latency analysis	Tail analysis examines distributions and critical dependencies rather than only means. Read the lesson →
Tail latency and latency budgets	A request's tail can be dominated by the slowest dependency. Read the lesson →
TCP vs UDP vs QUIC	TCP supplies an ordered reliable byte stream; it preserves bytes, not application-message boundaries. Read the lesson →
Technical vs business boundaries	Technical boundaries group by mechanism, such as controllers or databases; business boundaries group coherent capabilities, such as orders or fulfillment. Read the lesson →
Tell, Don't Ask	Tell, Don't Ask encourages sending an intention to the state owner. Read the lesson →

Template Method	A template method fixes an algorithm skeleton and permits selected steps to vary, commonly through inheritance. Read the lesson →
Testability and design for change	Testability follows controllable inputs, visible outcomes, and explicit effects. Read the lesson →
The fallacies of distributed computing	The fallacies of distributed computing warn against assuming reliable networks, zero latency, infinite bandwidth, static topology, free transport, homogeneous infrastructure, automatic security, or one administrator. Read the lesson →
Thread pools and worker pools	A worker pool caps simultaneously executing jobs, while a bounded queue caps waiting jobs. Read the lesson →
Thread safety and immutability	Thread safety is a behavioral property under a stated concurrent-use contract. Read the lesson →
Threat modeling	Threat modeling starts from assets, actors, trust boundaries, and abuse goals. Read the lesson →
Throughput	Completed work per unit time under a specified workload and success definition. Read the lesson →
Time, clocks, and event ordering	Wall clocks support timestamps and scheduling but can drift, jump, or be adjusted. Read the lesson →
Time-series databases	Time-series data combines timestamps, measurements, and dimensions. Read the lesson →

Timeouts	Timeouts bound waiting and resource ownership. Read the lesson →
TLS and HTTPS	TLS authenticates peers according to the configured trust model and protects traffic integrity and confidentiality. Read the lesson →
Trace IDs, span IDs, context propagation	Trace IDs identify a trace, span IDs identify individual spans, and propagation carries the relevant context across boundaries. Read the lesson →
Trade-off	A choice that accepts one cost or limitation to gain another benefit, judged against the requirements. Read the lesson →
Transaction commit and rollback	Commit establishes a transaction's accepted outcome according to the engine's protocol; rollback abandons its uncommitted work. Read the lesson →
Transactional outbox	Recording a state change and publication intent in one local transaction, then relaying the intent; duplicate delivery still requires handling. Read the lesson →
Transactions across multiple operations	A business operation spanning several statements needs one deliberate atomic boundary if its invariant requires all-or-nothing changes. Read the lesson →
Transient, scoped, singleton, and framework-specific lifetimes	Transient means a resolution creates a new instance; scoped means instances are reused within a defined scope; singleton means reuse within the container or application boundary specified by the framework. Read the lesson →

Trust boundaries	A trust boundary changes the confidence or authority granted to input. Read the lesson →
Trust boundary	A point across which identity, privilege, or confidence in input changes. Read the lesson →
TTL and cache expiration	A TTL bounds intended residence after insertion, not necessarily the age of the underlying fact when inserted. Read the lesson →
Twelve-Factor App principles	The Twelve-Factor App is methodology for service-oriented application operation: explicit dependencies and configuration, backing services, stateless processes where appropriate, logs as event streams, and related deployment practices. Read the lesson →
Unit of Work	A Unit of Work groups local changes under one commit and tracks their ownership. Read the lesson →
Unit, integration, contract, end-to-end, and performance tests	Unit tests isolate local behavior; integration tests verify real collaborators; contract tests verify boundary expectations; end-to-end tests exercise complete journeys. Read the lesson →
User journeys and workload characteristics	Users do not generate a constant stream of identical requests. Read the lesson →
Vector indexes: introductory architecture only	A vector index supports similarity search over numerical embeddings. Read the lesson →
Vertical vs horizontal scaling	Vertical scaling increases resources of one node; horizontal scaling adds nodes. Read the lesson →

Virtual machines vs containers	A virtual machine presents virtualized hardware with its guest operating system; containers commonly isolate processes while sharing a host kernel. Read the lesson →
Visitor	A visitor adds operations over a known structure by dispatching on element type. Read the lesson →
Webhooks and delivery acknowledgments	A webhook is an asynchronous request to another party. Read the lesson →
WebSockets, Server-Sent Events, and polling	Polling repeatedly asks for updates; server-sent events stream server-to-client events over HTTP; WebSockets support bidirectional messaging after protocol setup. Read the lesson →
What data storage must guarantee	Storage choices begin with promises: what is accepted, when it is committed, how it can be read, and which failures it survives. Read the lesson →
What makes a distributed system difficult	Distribution removes the ability to observe one coherent machine state. Read the lesson →
When caching is harmful	Caching is harmful when maintenance and correctness cost exceed saved work. Read the lesson →
When not to use microservices	Avoid microservices when the domain is still changing rapidly, the team cannot operate independent services safely, or scale and isolation do not require them. Read the lesson →
Why caching works	Caching works when requests reuse data or nearby data often enough to avoid more expensive work. Read the lesson →

Wide-column stores

Wide-column systems organize data around partition keys and ordered clustering dimensions in common implementations.

Read the lesson →

Workflow engines

A workflow engine persists execution progress and resumes work after failures under its programming model.

Read the lesson →

Write-ahead logging

Write-ahead logging records recovery information before the corresponding dirty data pages are considered safely persisted.

Read the lesson →

Zero-trust principles

Zero-trust principles reject implicit authorization based solely on network location.

Read the lesson →

No terms match that search.

THE VISUAL INDEX

Every diagram has a job

575 original figures: chapter mechanisms, full-design views, and the required unit plates. The separate master map is Figure M.1.

FIG. 1.1.1

The application and storage work together

Include the parts needed to explain the behaviour you are studying.

FIG. 1.1.2

Creating a task changes the stored state

Read the state before and after the operation to see exactly what changed.

FIG. 1.1.3

Save before reporting success

Compare the two versions by restarting the process.

FIG. 1.1.4

Two levels of detail in one design

Check that both views agree about saving a task before reporting success.

FIG. 1.2.1

Shared storage details make changes spread

Follow the arrows to find which parts know about the shared data.

FIG. 1.2.2

Separate responsibilities inside one process

Each kind of work has a clear place in the code.

FIG. 1.2.3

The service uses a repository contract

The dashed arrows show code dependencies. These objects can run in the same process.

FIG. 1.2.4

Each layer works at a different level of detail

Follow one request through the layers to connect the user's action to the work below it.

FIG. 1.3.1

Turn “safe” into a checkable requirement

A specific requirement tells us which test to run.

FIG. 1.3.2

Choose, record, and review a design

Keep the reasons for a decision alongside the decision itself.

FIG. 1.3.3

Check requirements before scoring preferences

An easy-to-run choice is useful only if it meets the requirements.

FIG. 1.3.4

Availability and durability answer different questions

Check both whether requests succeed and whether saved data survives the stated failure.

FIG. 1.4.1

C4: zoom from the system into its code

Select the level that helps your reader answer a particular question.

FIG. 1.4.2

The application and database share one machine

Show physical placement when asking what can fail together.

FIG. 1.4.3

The task is saved, but the reply is lost

Trace the events in order to see why the client can be unsure even when a task exists.

FIG. 1.4.4

Each design change answers a new need

State the requirement that makes each change necessary.

FIG. 2.1.1

Gathering and prioritizing requirements · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.2

User journeys and workload characteristics · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.3

Read-heavy vs write-heavy systems · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.1

Latency, throughput, QPS, RPS, TPS · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.2

A small mean can hide one slow request

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.2.3

Availability, reliability, durability, consistency · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.4

SLI, SLO, SLA, and error budgets · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.1

Capacity and growth planning · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.2

Different workload dimensions have different magnitudes

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.3.3

Peak vs average load · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.1

Little's Law and queueing intuition · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.2

Amdahl's Law and bottlenecks · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.3

Tail latency and latency budgets · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.4

Cost, security, operational complexity, and maintainability as constraints · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.5

Decision matrices and trade-off analysis · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.1

How the internet works from the application perspective · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.2

IP addresses, ports, sockets, routing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.3

TCP vs UDP vs QUIC · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.1

DNS resolution and DNS caching · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.2

TLS and HTTPS · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.3

HTTP/1.1, HTTP/2, HTTP/3 · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.4

Connection establishment and connection reuse · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.5

A cold request spends time before server work

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 3.3.1

Reverse proxies vs forward proxies · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.2

Load balancers vs API gateways · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.3

L4 vs L7 load balancing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.4

WebSockets, Server-Sent Events, and polling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.1

NAT, firewalls, ingress, egress · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.2

Service discovery · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.3

CDN and edge networking · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.4

Network partitions, timeouts, packet loss · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.5

Connection pooling and keepalive · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.1

Threads share a heap but have execution-local stacks

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.1.2

Memory, stack, heap, and object lifetimes · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.3

Scheduling and context switching · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.4

CPU-bound vs I/O-bound workloads · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.5

Synchronous vs asynchronous execution · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.1

Blocking vs nonblocking I/O · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.2

Concurrency vs parallelism · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.3

Event loops, tasks, futures, promises · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.4

Thread pools and worker pools · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.1

Shared memory vs message passing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.2

Race conditions and critical sections · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.3

Mutexes, semaphores, atomics, condition variables · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.4

A circular wait prevents both participants progressing

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.4.1

Memory visibility and happens-before relationships · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.2

Thread safety and immutability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.3

Resource exhaustion and backpressure · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.4

Cancellation and structured concurrency · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.1

Abstraction, encapsulation, polymorphism, inheritance · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.2

Composition vs inheritance · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.3

Interfaces and contracts · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.4

High cohesion and low coupling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.5

SOLID principles, including their limitations · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.1

DRY, KISS, YAGNI · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.2

Law of Demeter · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.3

Tell, Don't Ask · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.4

Command-query separation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.1

Immutability and side-effect management · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.2

Domain models, entities, value objects · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.3

Object collaboration · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.4

Testability and design for change · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.1

Refactoring and recognizing code smells · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.2

Functional programming alternatives to object-centric design · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.3

The running system · ordered responsibility

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.4.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.1

Dependency, dependency direction, and inversion · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.2

Inversion of Control (IoC) · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.3

Dependency Injection (DI) · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.4

Constructor, property/setter, and method injection · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.5

Dependency inversion principle vs DI · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.6

Composition roots and DI containers · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.1

Scope identities make lifetime semantics visible

Make the internal structure or state relation visible before claiming its behavior.

FIG. 6.2.2

Request scopes, session scopes, and custom scopes · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.3

Object creation vs object ownership · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.4

Disposal, cleanup, and resource lifecycles · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.5

Lazy initialization · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.6

Dependency cycles and captive dependencies · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.1

DI vs service locator · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.2

DI vs factory pattern · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.3

The running system · ordered responsibility

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.3.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.4.1

Factory Method, Abstract Factory, Builder, Prototype, Singleton · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.4.2

Factory Method · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.3

Abstract Factory · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.4

Builder · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.5

Prototype · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.6

Singleton · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.1

Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.5.2

Adapter · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.3

Bridge · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.4

Composite · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.5

Decorator · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.6

Facade · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.7

Flyweight · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.8

Proxy · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.1

Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.6.2

Strategy · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.3

Observer · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.4

Command · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.5

State · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.6

Chain of Responsibility · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.7

Template Method · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.8

Iterator · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.9

Mediator · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.10

Memento · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.11

Visitor · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.1

Repository, Unit of Work, Specification, CQRS, and dependency boundaries · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.7.2

Repository · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.3

Unit of Work · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.4

Specification · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.5

CQRS · collaborator roles

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.1.1

Layered/N-tier architecture · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.2

Modular monoliths · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.3

Monolith vs microservices · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.4

Clean Architecture · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.5

Hexagonal/Ports and Adapters · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.6

Onion Architecture · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.1

Domain-Driven Design (DDD) · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.2

Context boundaries protect different business meanings

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.2.3

Aggregates, entities, value objects, domain services · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.4

Domain events · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.1

Dependency rules and module boundaries · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.2

Business logic vs orchestration vs infrastructure · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.3

Controllers, services, repositories, adapters · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.4

Anti-corruption layers · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.5

Event-driven architecture · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.1

Serverless and function-oriented architectures · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.2

Evolutionary architecture · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.3

Technical vs business boundaries · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.4

Distributed monolith antipattern · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.5

When not to use microservices · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.1

API fundamentals and interface contracts · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.2

REST, GraphQL, gRPC, RPC · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.3

Resource modeling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.4

Request/response schemas · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.5

Status codes and error models · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.6

Pagination, filtering, sorting · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.1

Cursor vs offset pagination · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.2

API versioning and compatibility · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.3

A payment retry follows durable operation identity

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.2.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.1

Authentication vs authorization at API boundaries · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.2

Rate limits and quotas · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.3

Retries and timeouts · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.4

Streaming APIs and long-running operations · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.5

Webhooks and delivery acknowledgments · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.1

Schema evolution · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.2

Consumer-driven contracts · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.3

API gateways and BFF · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.4

Public vs internal APIs · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.5

API documentation and discoverability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.1

What data storage must guarantee · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.2

Relational databases and SQL · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.3

Document databases · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.4

Key-value databases · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.5

Wide-column stores · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.6

Graph databases · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.7

Time-series databases · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.8

Search indexes · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.9

Vector indexes: introductory architecture only · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.1

Object storage vs block vs file storage · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.2

OLTP vs OLAP · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.3

Row-oriented vs column-oriented storage · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.4

Normalization vs denormalization · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.1

An order schema preserves relationships and history

Make the internal structure or state relation visible before claiming its behavior.

FIG. 9.3.2

Referential integrity · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.3

Data ownership · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.4

Choosing storage based on access patterns · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.1

Data retention and lifecycle · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.2

Polyglot persistence and its operational costs · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.3

The running system · ordered responsibility

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 9.4.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.1

How databases store and retrieve records · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.2

Pages, records, indexes, and buffer pools · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.3

B+tree lookup narrows an ordered key range

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.1.4

LSM trees and write amplification · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.5

Query planning and EXPLAIN · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.6

Index selection and query optimization · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.1

ACID properties · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.2

Transaction commit and rollback · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.3

Write-ahead logging · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.4

Two snapshots can legitimately see different versions

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.3.1

Locking and optimistic concurrency control · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.2

Dirty reads, nonrepeatable reads, phantom reads · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.3

Lost updates, write skew, serialization anomalies · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.4

Isolation levels and implementation differences · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.5

Serializable execution and serializability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.1

Database connection pooling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.2

Transactions across multiple operations · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.3

Backup, recovery, and durability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.1

Why caching works · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.2

Cache locality and the working set · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.3

In-process vs distributed caches · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.4

Browser, CDN, application, and database caches · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.1

Cache-aside, read-through, write-through, write-behind · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.2

TTL and cache expiration · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.3

A stale cache is a separate state copy

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.2.4

LRU and FIFO disagree after the same accesses

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.3.1

Hot keys · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.2

Cache stampedes, penetration, and avalanche · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.3

Negative caching · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.4

Stale-while-revalidate · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.5

Cache warming and prefetching · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.1

Redis-like data structures · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.2

CDN architecture and origin shielding · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.3

Cache partitioning and replication · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.4

When caching is harmful · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.1

Vertical vs horizontal scaling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.2

Stateless vs stateful services · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.3

Load distribution algorithms · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.4

Health checks and failover · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.5

Sticky sessions · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.6

Autoscaling metrics · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.1

Replication · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.2

Partitioning and sharding · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.3

Range partitioning vs hash partitioning · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.4

A new owner takes only part of the hash space

Make the internal structure or state relation visible before claiming its behavior.

FIG. 12.2.5

Rebalancing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.6

Averages hide a hot partition

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 12.3.1

Read replicas and read/write splitting · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.2

Replication lag · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.3

Database proxying · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.4

Connection pool saturation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.5

Bottleneck identification · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.1

Queueing and overload · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.2

Single points of failure · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.3

Multi-region strategies · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.4

Scaling writes vs scaling reads · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.5

Scale-out economics · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.1

Binary/object storage · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.2

The upload pipeline separates bytes from control

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.1.3

Multipart and resumable uploads · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.4

Checksums and content addressing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.5

Metadata vs binary data · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.1

An inverted index is a term-to-document map

Make the internal structure or state relation visible before claiming its behavior.

FIG. 13.2.2

Indexing pipelines · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.3

Search ingestion and reindexing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.1

Batch vs streaming pipelines · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.2

ETL vs ELT · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.3

Change Data Capture (CDC) · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.4

Data lakes and warehouses · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.5

Stream processing fundamentals · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.1

Data provenance and lineage · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.2

Data quality, schema validation, and replay · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.3

Lifecycle policies and retention · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.4

Designing data-intensive applications · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.1

What makes a distributed system difficult · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.2

Partial failures · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.3

Network uncertainty · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.4

Fault models · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.5

Time, clocks, and event ordering · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.6

Physical vs logical clocks · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.7

Vector comparison exposes concurrent events

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.2.1

Consistency models · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.2

Strong consistency and eventual consistency · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.3

Linearizability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.4

Serializability vs linearizability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.5

Sequential and causal consistency · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.6

Read-your-writes and monotonic reads · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.1

CAP theorem, with its actual assumptions · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.2

PACELC as a useful extension · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.3

Intersecting quorums still need a protocol

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.3.4

A partition can leave two live but disconnected sides

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.4.1

Distributed coordination · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.2

The fallacies of distributed computing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.3

Safety vs liveness · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.1

Primary/replica architectures · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.2

Leader/follower replication · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.3

Multi-leader replication · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.4

Leaderless replication · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.5

Synchronous vs asynchronous replication · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.6

Quorum-based systems · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.1

Consensus and replicated state machines · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.2

Raft election changes authority under protocol rules

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.2.3

Paxos: conceptual foundations · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.4

Presence in a log is not the same as commitment

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.2.5

Membership changes · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.1

Split-brain prevention and fencing tokens · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.2

Failure detection · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.3

Strongly consistent metadata services · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.4

Distributed locks and leases · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.1

Conflict resolution and CRDT introduction · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.2

Multi-region replication trade-offs · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.3

The running system · ordered responsibility

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 15.4.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.1

Synchronous vs asynchronous communication · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.2

Queues vs logs vs pub/sub · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.3

Producers, consumers, brokers · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.4

Partitions, consumer groups, offsets · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.5

Delivery guarantees · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.6

At-most-once and at-least-once processing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.7

Exactly-once processing within defined boundaries · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.8

Ordering guarantees · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.9

Consumer acknowledgements · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.1

Retries and dead-letter queues · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.2

Poison messages · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.3

Idempotent consumers · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.4

Event sourcing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.3.1

CQRS · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.3.2

The outbox closes a local publication gap

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.3.3

A failed shipment starts tracked compensation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.3.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.1

Choreography vs orchestration · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.2

Workflow engines · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.3

Durable job execution · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.4

Scheduling, leases, and worker coordination · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.5

Backpressure and replay · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.6

Reconciliation after partial failure · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.1

Failure domains and fault isolation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.2

Availability vs durability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.3

Redundancy · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.4

Active-active vs active-passive · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.1

Timeouts · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.2

Retries, exponential backoff, jitter · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.3

Retry storms · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.4

A circuit breaker bounds futile calls

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.2.5

Bulkheads · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.6

Rate limiting · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.1

Load shedding · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.2

Graceful degradation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.3

Fallbacks · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.4

Health checking · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.1

Recovery time and recoverable data are different

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.4.2

Backups and restore testing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.3

Failover and failback · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.4

Multi-zone and multi-region deployments · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.5

Chaos engineering · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.6

SLO-based reliability decisions · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.7

Cascading failures · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.8

Incident analysis and prevention · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.1

Threat modeling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.2

Trust boundaries · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.3

Authentication vs authorization · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.4

Sessions, cookies, tokens · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.5

OAuth 2.0 and OpenID Connect · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.6

Role-based and attribute-based access control · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.7

Least privilege · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.1

Defense in depth · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.2

Secrets management · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.3

Encryption in transit and at rest · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.4

Password hashing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.5

Key management and rotation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.6

Network segmentation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.7

Zero-trust principles · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.1

API security and common vulnerabilities · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.2

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.3.3

Data minimization · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.4

Audit trails · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.5

Privacy, retention, and deletion · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.1

Supply-chain security · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.2

Security vs usability trade-offs · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.3

Secure-by-design reviews · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.1

Monitoring vs observability · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.2

Logs, metrics, and traces · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.3

Structured logging · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.4

This slow request is dominated by waiting

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 19.1.5

Trace IDs, span IDs, context propagation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.6

Golden signals · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.1

RED and USE methods · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.2

SLIs, SLOs, error budgets · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.3

Alert design and alert fatigue · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.4

Sampling and telemetry cost · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.5

Dashboards · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.1

Profiling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.2

Query and application performance diagnosis · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.3

Incident response · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.4

On-call and runbooks · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.5

Root-cause investigation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.1

Blameless postmortems · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.2

Capacity monitoring · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.3

Observability across asynchronous workflows · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.4

The running system · invariant and recovery review

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.1

Development vs staging vs production · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.2

VMs and containers isolate at different layers

Make the internal structure or state relation visible before claiming its behavior.

FIG. 20.1.3

Container fundamentals · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.4

Kubernetes fundamentals · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.1

Control plane vs data plane · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.2

Scheduling and service discovery · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.3

Autoscaling · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.4

Configuration and secret delivery · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.5

Immutable infrastructure · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.1

Infrastructure as Code · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.2

CI/CD · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.3

Canary exposure expands only after verification

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 20.3.4

Feature flags · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.5

Rollbacks and database migrations · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.6

Multi-environment configuration · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.1

Twelve-Factor App principles · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.2

Serverless compute · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.3

Cloud vs on-premises trade-offs · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.4

Infrastructure ownership and managed services · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.5

Deployment topologies and networking · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.6

Platform engineering basics · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.1

Performance budgets · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.2

CPU, memory, disk, network, and database bottlenecks · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.3

Profiling and benchmarking · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.4

Load, stress, spike, and soak testing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.5

Benchmark methodology and statistical pitfalls · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.6

Tail latency analysis · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.1

Queue saturation and throughput ceilings · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.2

Performance tuning vs architecture changes · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.3

Architecture costs are not capacity guarantees

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.2.4

Compute, storage, networking, and telemetry costs · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.5

Cost per request or transaction · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.6

Capacity reservation vs elastic capacity · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.1

Cost-performance optimization · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.2

Sustainability and resource efficiency · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.3

Unit, integration, contract, end-to-end, and performance tests · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.4

Fault-injection tests · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.1

Data consistency validation · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.2

Architecture fitness functions · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.3

Regression testing · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.4

Evaluating a design against its original requirements · worked snapshots

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.1.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.1.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.1.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.1.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.2.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.2.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.2.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.2.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.3.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.3.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.3.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.3.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.4.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.4.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.4.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.4.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.5.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.5.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.5.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.5.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.6.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.6.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.6.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.6.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.7.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.7.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.7.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.7.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.8.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.8.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.8.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.8.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.9.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.9.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.9.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.9.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.10.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.10.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.10.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.10.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.11.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.11.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.11.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.11.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.12.1

Data ownership and relationships

Identify identities, authoritative relationships, derived records, and the invariant that must remain true.

FIG. 22.12.2

The high-level path and responsibility boundaries

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.12.3

Naive path, failure, improved path, review

Read this failure history alongside the naive design and the protocol; the surrounding text names the ambiguity and correction.

FIG. 22.12.4

Evolution follows demonstrated requirements

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 2.0.1

Request-volume timeline

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.0.2

throughput-latency curve

Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark.

FIG. 2.0.3

latency waterfall

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 2.0.4

capacity planning worksheet

Compare the stated dimensions under one explicit contract.

FIG. 3.0.1

Complete HTTP request timeline

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 3.0.2

TCP/TLS interaction

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 3.0.3

load-balancing topology

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 3.0.4

WebSocket vs polling communication

Compare the stated dimensions under one explicit contract.

FIG. 4.0.1

Threads share a heap but have execution-local stacks

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.0.2

scheduling timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 4.0.3

race-condition interleaving

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 4.0.4

A circular wait prevents both participants progressing

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.0.5

event-loop flow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.0.1

Before/after class diagrams

Compare the stated dimensions under one explicit contract.

FIG. 5.0.2

dependency graph

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.0.3

inheritance vs composition

Compare the stated dimensions under one explicit contract.

FIG. 5.0.4

cohesion/coupling illustrations

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.0.1

Object creation timelines

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.0.2

Scope identities make lifetime semantics visible

Make the internal structure or state relation visible before claiming its behavior.

FIG. 6.0.3

UML class relationships

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.0.4

dependency inversion before/after

Compare the stated dimensions under one explicit contract.

FIG. 6.0.5

pattern comparisons

Compare the stated dimensions under one explicit contract.

FIG. 7.0.1

Layered vs Hexagonal

Compare the stated dimensions under one explicit contract.

FIG. 7.0.2

modular monolith vs microservices

Compare the stated dimensions under one explicit contract.

FIG. 7.0.3

dependency arrows

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.0.4

bounded context map

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.0.1

REST request flow

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 8.0.2

API gateway topology

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.0.3

synchronous vs asynchronous API

Compare the stated dimensions under one explicit contract.

FIG. 8.0.4

payment state-machine diagram

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 9.0.1

Database taxonomy

Compare the stated dimensions under one explicit contract.

FIG. 9.0.2

physical storage representations

Compare the stated dimensions under one explicit contract.

FIG. 9.0.3

normalized vs denormalized schema

Compare the stated dimensions under one explicit contract.

FIG. 9.0.4

An order schema preserves relationships and history

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.1

B+tree lookup narrows an ordered key range

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.2

Two snapshots can legitimately see different versions

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.3

concurrent transaction interleavings

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 10.0.4

lock wait graph

Compare the stated dimensions under one explicit contract.

FIG. 10.0.5

WAL and recovery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.1

Cache hit/miss sequence

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 11.0.2

cache hierarchy

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.3

LRU and FIFO disagree after the same accesses

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.0.4

stampede timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.5

geographic CDN delivery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 12.0.1

Architecture evolution across four load levels

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 12.0.2

A new owner takes only part of the hash space

Make the internal structure or state relation visible before claiming its behavior.

FIG. 12.0.3

Averages hide a hot partition

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 12.0.4

load-balancer failover

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.1

Upload pipeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.2

An inverted index is a term-to-document map

Make the internal structure or state relation visible before claiming its behavior.

FIG. 13.0.3

ingestion DAG

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.4

batch vs stream timeline

Compare the stated dimensions under one explicit contract.

FIG. 13.0.5

lineage graph

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 14.0.1

A partition can leave two live but disconnected sides

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.0.2

causality timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 14.0.3

consistency model comparisons

Compare the stated dimensions under one explicit contract.

FIG. 14.0.4

Intersecting quorums still need a protocol

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.1

Raft election changes authority under protocol rules

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.2

election timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 15.0.3

Presence in a log is not the same as commitment

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.4

quorum intersection

Compare the stated dimensions under one explicit contract.

FIG. 15.0.5

network partition recovery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.1

Queue lifecycle

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 16.0.2

consumer partition assignment

Compare the stated dimensions under one explicit contract.

FIG. 16.0.3

Saga happy/failure paths

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.4

The outbox closes a local publication gap

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.5

event replay

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 17.0.1

Retry amplification timeline

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 17.0.2

A circuit breaker bounds futile calls

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.0.3

bulkhead isolation

Compare the stated dimensions under one explicit contract.

FIG. 17.0.4

failure-domain map

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 17.0.5

Recovery time and recoverable data are different

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.0.1

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.0.2

authentication sequence

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 18.0.3

authorization decision flow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 18.0.4

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 19.0.1

This slow request is dominated by waiting

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 19.0.2

request ID propagation

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 19.0.3

metrics-to-root-cause workflow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 19.0.4

error-budget burn chart

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 20.0.1

VMs and containers isolate at different layers

Make the internal structure or state relation visible before claiming its behavior.

FIG. 20.0.2

Kubernetes cluster

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 20.0.3

CI/CD pipeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 20.0.4

blue-green deployment

Compare the stated dimensions under one explicit contract.

FIG. 20.0.5

Canary exposure expands only after verification

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.0.1

Performance bottleneck tree

Each branch is a hypothesis requiring profiling, traces, and workload evidence.

FIG. 21.0.2

throughput-latency curve

Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark.

FIG. 21.0.3

Architecture costs are not capacity guarantees

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.0.4

test pyramid

Test scope and feedback cost differ; choose by the failure evidence needed.

FIG. 21.0.5

design decision matrix

Compare the stated dimensions under one explicit contract.

THE FORMULA REFERENCE

Symbols with assumptions

Every indexed model is introduced in the lessons. Inputs and guarantee boundaries accompany the notation.

State transition

```
T(S, c) → (S′, result)
```

Identify valid state changes and preserved invariants. This abstracts away implementation and failures.

Derivation and worked example →

Weighted preference

```
score(a) = Σ wᵢ sᵢ(a) / Σ wᵢ
```

Use nonnegative weights with positive total. Compare only feasible designs. Scores are illustrative judgments, not benchmark results.

Derivation and worked example →

Request rate

```
λ = daily users × requests per user per day / 86,400
```

An average workload estimate. Peak factors must be assumed or measured separately.

Derivation and worked example →

Little's Law

```
L = λ W
```

Long-run stable averages with consistent boundaries and units. It is not a universal p99 prediction.

Derivation and worked example →

Amdahl's Law

```
speedup(N) = 1 / [(1 − p) + p/N]
```

Idealized fixed workload and parallel fraction p; excludes extra coordination overhead unless modeled.

Derivation and worked example →

Storage growth

```
bytes/day = records/day × bytes/record × retained copies
```

Account separately for indexes, compression, retention, metadata, and headroom.

Derivation and worked example →

Error allowance

```
allowed bad events = (1 − target fraction) × eligible events
```

Valid for an event-based SLO; time-based objectives need their own denominator.

Derivation and worked example →

Token refill

```
tokens′ = min(capacity, tokens + refill_rate × elapsed)
```

Admission consumes tokens atomically. Distributed clocks and state ownership need additional design.

Derivation and worked example →

Quorum intersection

```
R + W > N
```

An intersection condition for reads and writes on a fixed replica set. Not by itself a proof of linearizability.

Derivation and worked example →

Cost per outcome

```
cost / successful business outcomes
```

Define accounting period and include capacity, storage, networking, telemetry, and operational work where relevant.

Derivation and worked example →

Ideal M/M/1 mean residence

```
W = S / (1 − rho)
```

Stable single-server Markovian model, rho < 1. A mean model, not a p99 prediction.

Derivation and worked example →

Fan-out slow probability

```
P(any slow) = 1 − (1 − p)^N
```

Requires independent equal-probability call outcomes in this model. Shared failures can correlate tails.

Derivation and worked example →

Retry amplification

```
maximum attempts = a^L
```

a allowed attempts at each of L nested independently retrying layers; actual execution can terminate earlier.

Derivation and worked example →

Busy-capacity worksheet

```
instances = ceil(peak RPS / (assumed capacity × utilization target))
```

Capacity is an explicit hypothetical or tested input; other bottlenecks and surviving failure capacity remain separate.

Derivation and worked example →

Error-budget burn

```
burn = observed bad fraction / allowed bad fraction
```

Use the same eligible-event denominator and a positive allowance.

Derivation and worked example →

Cache service-time average

```
E[T] = h × T_hit + (1 − h) × T_miss
```

A simple service-time mixture, excluding queue effects, dependence, and tail inference.

Derivation and worked example →

Backlog growth

```
growth/s = arrival/s − completion/s
```

Sustained positive growth requires rejection, reduction, or more useful capacity; a bigger queue only changes waiting.

Derivation and worked example →

Raw daily storage

```
bytes/day = records/s × bytes/record × 86400
```

Add indexes, history, metadata, replication, retention, compression and migration separately.

Derivation and worked example →

THE PATTERN INDEX

Start with the problem, then the pattern

Individual LLD patterns include their intent, collaborators, miniature implementation, costs, and alternatives.

6.1 · Dependencies, inversion & composition

Dependency Injection | Composition Root | Inversion of Control

6.4 · Creational patterns

Factory Method | Abstract Factory | Builder | Prototype | Singleton

6.5 · Structural patterns

Adapter | Bridge | Composite | Decorator | Facade | Flyweight | Proxy

6.6 · Behavioral patterns

Strategy | Observer | Command | State | Chain of Responsibility | Template Method | Iterator | Mediator | Memento | Visitor

6.7 · Enterprise patterns & boundaries

Repository | Unit of Work | Specification | CQRS

Application Architecture and Domain-Driven Design

Modular Monolith | Layered Architecture | Hexagonal Architecture | Clean Architecture | Anti-corruption Layer

Caching and Content Delivery

Cache-aside | Read-through | Write-through | Write-behind | Stale-while-revalidate

Message Queues, Events, and Distributed Workflows

Transactional Outbox | Saga | Event Sourcing | Orchestration | Choreography

Reliability, Fault Tolerance, and Disaster Recovery

Circuit Breaker | Bulkhead | Retry with Jitter | Load Shedding

Deployment, Cloud Infrastructure, and Platform Architecture

Rolling Deployment | Blue-green Deployment | Canary Release

THE READING ROOM

Source bibliography

Original research and specifications are distinguished from university teaching, implementation documentation, and engineering guidance.

Access and the stated use were reviewed on 21 October 2026. Versioned documentation is pinned where practical. A checked link does not prove a production system correct; worked workloads, costs, and architectures are original illustrative models. Recheck security guidance and implementation versions when applying them.

61 of 61 sources

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Atomic Commit in SQLite ↗

Commit and operating-system assumptions

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 9110: HTTP Semantics, June 2022 ↗

HTTP semantics; application commit policy is separate

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · ACM DOI

On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

Module decomposition criteria; metadata and abstract verified

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

Dependency injection and wiring

Access reviewed · 21 October 2026

ORIGINAL PRACTICE ARTICLE

Documenting Architecture Decisions — Michael Nygard, 2011 ↗

ADR structure and decision history

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

AWS Well-Architected: the six pillars ↗

Review lenses, not correctness guarantees

Access reviewed · 21 October 2026

ORIGINAL MODEL DOCUMENTATION

C4: diagrams ↗

Formal diagram levels and supporting views

Access reviewed · 21 October 2026

ORIGINAL MODEL DOCUMENTATION

C4: container abstraction ↗

Meaning of container

Access reviewed · 21 October 2026

ENGINEERING BOOK

Implementing SLOs — The SRE Workbook ↗

Service indicators, objectives, and error budgets; not a source of the atlas workload numbers

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · PUBLISHER RECORD

A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

Long-run queue accounting; publisher record inspected, full proof access may depend on access rights

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

The Tail at Scale — Dean and Barroso, 2013 ↗

Tail-latency considerations; atlas probabilities and latency examples are author-calculated

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION

RFC 9000: QUIC Transport, 2021 ↗

QUIC streams and transport behavior; application walkthrough is author-created

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION

RFC 8446: TLS 1.3, 2018 ↗

Secure transport handshake and protocol scope

Access reviewed · 21 October 2026

UNIVERSITY TEXTBOOK

Operating Systems: Three Easy Pieces ↗

Runtime, concurrency, and persistence

Access reviewed · 21 October 2026

LANGUAGE SPECIFICATION · ACCESSED 21 OCT 2026

The Go Memory Model ↗

One concrete memory-model contract; other languages differ

Access reviewed · 21 October 2026

ENGINEERING TEACHING REFERENCE

The Catalog of Design Patterns — Refactoring Guru ↗

Pattern taxonomy; miniature examples and judgments are original to this atlas

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Architecture styles — Microsoft Architecture Center ↗

Style comparisons; provider-specific recommendations are not mandatory in the atlas

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING EXPLANATION

Bounded Context — Martin Fowler, 2014 ↗

Model boundaries and language

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

Microservices — James Lewis and Martin Fowler, 2014 ↗

Architecture characteristics and costs; no exact company architecture is asserted here

Access reviewed · 21 October 2026

PROJECT DOCUMENTATION

OWASP API Security Top 10, 2023 edition ↗

Threat taxonomy for Unit 18

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL SECURITY GUIDANCE

RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

OAuth threat and flow guidance; implementation should use maintained protocol libraries

Access reviewed · 21 October 2026

UNIVERSITY COURSES

Carnegie Mellon Database Group: courses ↗

Database internals reading

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Index Types ↗

Supported access structures and their purposes

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Transaction Isolation ↗

Specific PostgreSQL isolation behavior; no all-database equivalence claim

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Write-Ahead Logging ↗

Logging and recovery ordering; durability remains configuration-dependent

Access reviewed · 21 October 2026

OFFICIAL IMPLEMENTATION DOCUMENTATION · ACCESSED 21 OCT 2026

RocksDB Overview ↗

An LSM-based engine; exact tuning is implementation-specific

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 9111: HTTP Caching, 2022 ↗

HTTP cache semantics; application cache races are separate examples

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Redis: Key eviction ↗

Implementation policies can approximate ideal LRU/LFU

Access reviewed · 21 October 2026

OFFICIAL ENGINEERING EXPLANATION

Content delivery networks — Cloudflare Learning Center ↗

CDN role; atlas delivery and traffic numbers are illustrative

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Application best practices — Microsoft Architecture Center ↗

Scaling and application boundary review

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · AUTHOR ORGANIZATION

Dynamo: Amazon’s highly available key-value store, 2007 ↗

Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments

Access reviewed · 21 October 2026

OFFICIAL SPECIFICATION DOCUMENTATION · 1.54.0 AT ACCESS

OpenLineage: core model and documentation ↗

Dataset, job, and run lineage concepts

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Elasticsearch: Reindex documents ↗

Implementation reindex controls; full generation-switch protocol in the atlas is an original design

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

Apache Kafka 4.1: Design ↗

Partitioning and scoped transactional/processing behavior; external effects are separate

Access reviewed · 21 October 2026

PRIMARY PRACTITIONER RESEARCH REFERENCE

Jepsen: Consistency models ↗

Formal-history distinctions and references to underlying literature

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

Ordering reference for Unit 14

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · UNIVERSITY-HOSTED PAPER

Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected

Access reviewed · 21 October 2026

UNIVERSITY COURSE

MIT 6.5840: Distributed Systems, Spring 2026 ↗

Advanced distributed-systems reading

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

Raft reference for Unit 15

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

Paxos Made Simple — Leslie Lamport, 2001 ↗

Conceptual consensus safety; atlas does not present toy code as production Paxos

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · 2017 REVISION

A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

One formal CRDT model; the miniature tie-break exercise is not this complete implementation

Access reviewed · 21 October 2026

OFFICIAL ENGINEERING GUIDANCE

Saga distributed transactions pattern — Microsoft ↗

Local transactions and compensation; compensation is not always a perfect inverse

Access reviewed · 21 October 2026

PATTERN AUTHOR'S EXPLANATION

Transactional Outbox — Chris Richardson ↗

Publication intent and duplicate delivery

Access reviewed · 21 October 2026

ENGINEERING BOOK

Handling Overload — Google SRE ↗

Admission, capacity, and overload response

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Disaster recovery planning guide — Google Cloud ↗

Recovery planning, distinct from a measured achievement of an objective

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · 2024 REVISION AT ACCESS

Reliability Pillar — AWS Well-Architected ↗

Reliability review lenses and operations; provider-neutral teaching uses the concepts

Access reviewed · 21 October 2026

OFFICIAL PROJECT GUIDANCE · ACCESSED 21 OCT 2026

OWASP: Authorization ↗

Least privilege, deny by default, and per-resource enforcement

Access reviewed · 21 October 2026

OFFICIAL PROJECT GUIDANCE · ACCESSED 21 OCT 2026

OWASP: Password Storage ↗

Password-specific hashing and current deployment parameter guidance

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION · DECEMBER 2023

OpenID Connect Core 1.0, errata set 2 ↗

Identity layer and validation requirements; distinct from API authorization

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

OpenTelemetry: Signals ↗

Complementary observability signals

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

OpenTelemetry: Context propagation ↗

Trace context relationships; trace identity does not authorize access

Access reviewed · 21 October 2026

ENGINEERING BOOK

Alerting on SLOs — The SRE Workbook ↗

Budget burn and actionable alerts

Access reviewed · 21 October 2026

ORIGINAL METHODOLOGY

The Twelve-Factor App ↗

Application deployment conventions

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Kubernetes components ↗

Platform reference for Unit 20

Access reviewed · 21 October 2026

ENGINEERING BOOK

Canarying Releases — The SRE Workbook ↗

Representative rollout evaluation and stop conditions

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

The Practical Test Pyramid — Ham Vocke ↗

Test scope and feedback trade-offs, not a mandatory test-count ratio

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL DESCRIPTION · INFORMATIONAL RFC

RFC 2697: A Single Rate Three Color Marker, 1999 ↗

Token-based traffic metering example; atlas admission bucket is a simpler model

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 6455: The WebSocket Protocol, 2011 ↗

Bidirectional transport framing; not durable chat delivery

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL DESCRIPTION · INFORMATIONAL RFC

RFC 8216: HTTP Live Streaming, 2017 ↗

One adaptive streaming protocol; do not confuse Informational RFC with Standards Track

Access reviewed · 21 October 2026

OFFICIAL IMPLEMENTATION DOCUMENTATION · ACCESSED 21 OCT 2026

PostGIS: ST_DWithin ↗

Coordinate-system-aware proximity predicate, not exclusive dispatch assignment

Access reviewed · 21 October 2026

ENGINEERING BOOK

Distributed Periodic Scheduling — Google SRE ↗

Durable scheduling concerns; atlas schedule architecture is an original worked design

Access reviewed · 21 October 2026

No sources match that search.

PRODUCTION AND EVIDENCE

Completion manifest

All units are produced. Coverage, technical review, executable tests, and browser verification are separate kinds of evidence.

COMPLETE FIRST EDITION

22 units · 99 chapters · 390 exact syllabus items · 12 full case studies · 242 questions with separate explained solutions.

Unit	Chapters produced	Concept / case coverage	Questions / answers
1 · Introduction to System Design	4 / 4	13	11 / 11
2 · Requirements, Estimation, and Engineering Trade-offs	4 / 4	15	11 / 11
3 · Networking Foundations for System Design	4 / 4	17	11 / 11
4 · Runtime, Processes, Threads, and Concurrency	4 / 4	17	11 / 11
5 · Object-Oriented and Modular Software Design	4 / 4	15	11 / 11
6 · Design Patterns, Dependency Injection, and Object Lifecycles	7 / 7	18	11 / 11
7 · Application Architecture and Domain-Driven Design	4 / 4	20	11 / 11
8 · API Design and Communication Contracts	4 / 4	19	11 / 11

Unit	Chapters produced	Concept / case coverage	Questions / answers
9 · Data Models and Storage Technologies	4 / 4	19	11 / 11
10 · Database Internals, Transactions, and Concurrency Control	4 / 4	18	11 / 11
11 · Caching and Content Delivery	4 / 4	17	11 / 11
12 · Scalability, Load Balancing, and Data Distribution	4 / 4	22	11 / 11
13 · File Systems, Search Infrastructure, and Data Pipelines	4 / 4	17	11 / 11
14 · Distributed Systems Fundamentals	4 / 4	20	11 / 11
15 · Replication, Leader Election, and Consensus	4 / 4	17	11 / 11
16 · Message Queues, Events, and Distributed Workflows	4 / 4	22	11 / 11
17 · Reliability, Fault Tolerance, and Disaster Recovery	4 / 4	22	11 / 11
18 · Security Architecture and Privacy	4 / 4	22	11 / 11
19 · Observability, Monitoring, and Incident Engineering	4 / 4	19	11 / 11
20 · Deployment, Cloud Infrastructure, and Platform Architecture	4 / 4	21	11 / 11
21 · Performance, Cost Optimization, and System Validation	4 / 4	20	11 / 11

Unit	Chapters produced	Concept / case coverage	Questions / answers
22 · End-to-End Design, Architecture Reviews, and Evolution	12 / 12	12 × 22 sections	11 / 11

What the evidence means

Content checks establish presence, exact syllabus allocation, figures, references, and assessment. Executable examples test bounded arithmetic and local invariants. Browser checks test reading, interactions, layouts, offline behavior, and print assembly. The book does not report illustrative objectives as measured production achievements or toy protocols as reviewed production consensus, security, or financial integrations.

The original request is retained in BRIEF.txt. Additional optional case subjects are suggestions beyond the twelve mandatory worked designs, not omitted required chapters.